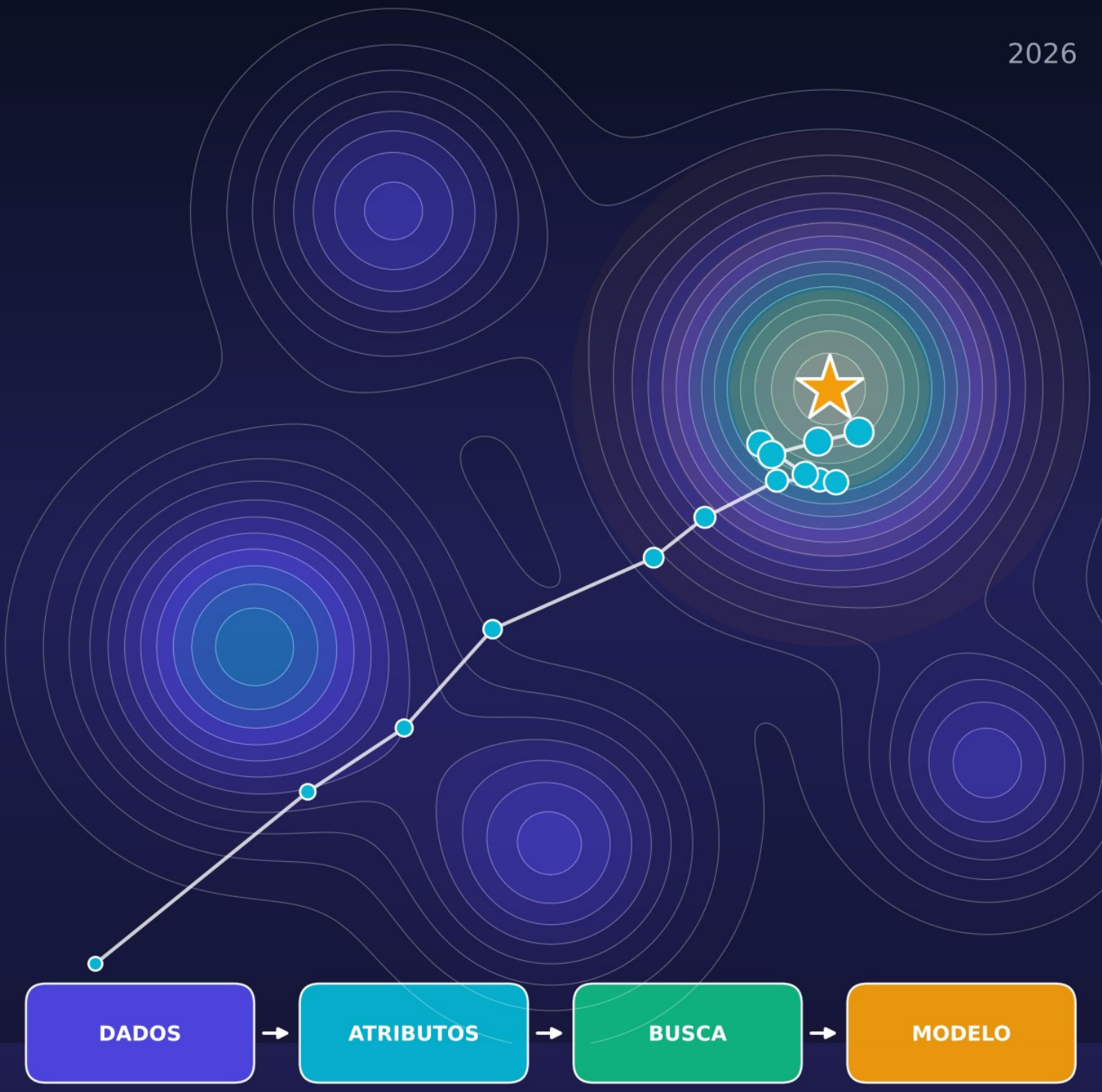
2026



AutoML

Teoria e Prática

Araken de Medeiros Santos

AUTOML

Teoria e Prática

Araken de Medeiros Santos

2026

Sumário

Apresentação **13**
Para quem é este livro 13
O que é AutoML, em uma frase 13
Como o livro está organizado 13
Ícones e caixas 14
Ambiente de programação 14
Um aviso importante sobre o que foi testado 14
Como estudar com este livro 15

Parte I — Fundamentos **16**

Capítulo 1 — Bem-vindo ao AutoML **17**
1.1 Uma história para começar 17
1.2 O que é AutoML? 18
1.3 O caminho do dado até o modelo 19
1.4 Por que o AutoML existe? 19
1.5 O problema CASH 20
1.6 Uma breve história do AutoML 21
1.7 Quanto é "automático"? 22
1.8 O que o AutoML não faz 23
1.9 Preparando o ambiente 23
1.10 Dados deste capítulo: o conjunto Wine 24
1.11 Do jeito manual: testando três modelos 24
1.12 Do jeito automático: um mini-AutoML 25
1.13 Os três ingredientes de todo AutoML 26
1.14 Como este livro está organizado 27
Resumo do capítulo 27
Exercícios de múltipla escolha 28
Exercícios discursivos 32
Exercícios práticos em Python (scikit-learn) 33
Exercício 1 — Por que precisamos de uma linha de base 33
Exercício 2 — Ampliando o espaço de busca 33
Exercício 3 — O efeito da semente 33
Exercício 4 — Contando o tamanho do espaço de busca 33
Projeto integrador do capítulo 34
Orçamento de busca e honestidade na avaliação 34

Capítulo 2 — Aprendizado de máquina em um capítulo **35**
2.1 Por que um capítulo de revisão? 35
2.2 Os dados: exemplos, atributos e alvo 35
2.3 Quatro algoritmos clássicos 36
2.4 Parâmetros e hiperparâmetros 37
2.5 Aprender é errar menos 38
2.6 Treino, validação e teste 39
2.7 Validação cruzada 40
2.8 Métricas: como medir o acerto 41
2.8.1 Métricas de classificação 41
2.8.2 Métricas de regressão 43
2.9 Linha de base: o ponto de comparação 43
2.10 O perigo escondido: o "efeito do vencedor" 44
2.11 Boas práticas para levar daqui 45

Resumo do capítulo ... 45
Exercícios de múltipla escolha ... 46
Exercícios discursivos ... 50
Exercícios práticos em Python (scikit-learn) ... 51
Exercício 1 — Curvas de aprendizado ... 51
Exercício 2 — Métricas em dados desbalanceados ... 51
Exercício 3 — Uma divisão ou várias? ... 51
Exercício 4 — Reproduzindo o efeito do vencedor ... 51
Projeto integrador do capítulo ... 52
Protocolo completo de avaliação em Digits ... 52
Capítulo 3 — Pipelines e pré-processamento ... 53
3.1 A roupa suja vem antes do modelo ... 53
3.2 Os dados deste capítulo ... 54
3.3 O que acontece se ignorarmos a sujeira? ... 55
3.4 Valores ausentes: imputação ... 55
3.5 Escalas diferentes: padronização ... 55
3.6 Categorias em texto: codificação ... 56
3.7 Pipelines: uma linha de montagem em código ... 57
3.8 O perigo do vazamento de dados ... 58
Um experimento revelador ... 59
3.9 O pipeline como espaço de busca ... 60
3.10 Outras transformações úteis ... 61
3.11 Salvando e reutilizando o pipeline ... 61
3.12 Reprodutibilidade ... 62
Resumo do capítulo ... 62
Exercícios de múltipla escolha ... 63
Exercícios discursivos ... 67
Exercícios práticos em Python (scikit-learn) ... 68
Exercício 1 — Qual imputação escolher? ... 68
Exercício 2 — Ordinal ou one-hot? ... 68
Exercício 3 — Vazamento: a nota honesta ... 68
Exercício 4 — Buscando dentro do pipeline ... 68
Projeto integrador do capítulo ... 69
Pipeline robusto para dados sujos ... 69
Parte II — Otimização de Hiperparâmetros ... 70
Capítulo 4 — Busca em grade e aleatória ... 71
4.1 Os botões do algoritmo ... 71
4.2 O espaço de busca ... 72
A escala logarítmica merece atenção ... 72
4.3 Busca em grade (grid search) ... 73
4.4 O custo da grade ... 73
4.5 Busca aleatória (random search) ... 74
4.6 A paisagem de hiperparâmetros ... 75
Um experimento justo ... 76
4.7 Quanto orçamento é suficiente? ... 77
4.8 O perigo escondido: o otimismo da busca ... 77
4.9 Quais hiperparâmetros importam? ... 79
4.10 Boas práticas ... 80
Resumo do capítulo ... 80
Exercícios de múltipla escolha ... 81
Exercícios discursivos ... 85
Exercícios práticos em Python (scikit-learn) ... 86

Exercício 1 — Sua primeira busca em grade....86
Exercício 2 — Grade ou aleatória? Com a floresta, empatou....86
Exercício 3 — Escala logarítmica faz diferença....86
Exercício 4 — Medindo o otimismo da busca....86
Projeto integrador do capítulo....87
Relatório de otimização de hiperparâmetros em Digits....87
Capítulo 5 — Otimização bayesiana....88
5.1 A caixa-preta cara.... 88
5.2 A ideia central: um modelo do modelo....89
5.3 Explorar ou aproveitar?.... 89
5.4 Processos gaussianos: previsão com incerteza.... 90
5.5 Funções de aquisição.... 91
5.6 Construindo uma otimização bayesiana do zero.... 91
5.7 TPE: outro modelo substituto.... 93
5.8 Outros modelos substitutos: o SMAC.... 94
5.9 Optuna: otimização bayesiana na prática.... 94
5.10 Resolvendo o CASH com o Optuna.... 95
5.11 Quanto ganhamos de verdade?.... 96
5.12 Limitações.... 97
5.13 Boas práticas.... 97
Resumo do capítulo.... 98
Exercícios de múltipla escolha.... 99
Exercícios discursivos.... 103
Exercícios práticos em Python (scikit-learn)....104
Exercício 1 — Sua própria otimização bayesiana....104
Exercício 2 — Optuna em uma floresta aleatória....104
Exercício 3 — Reproduzindo e corrigindo a falha do CASH....104
Exercício 4 — Quanto explorar? O coeficiente do UCB....104
Projeto integrador do capítulo....106
AutoML minimalista com Optuna em regressão (Diabetes)....106
Capítulo 6 — Métodos multifidelidade: Successive Halving, Hyperband e BOHB....107
6.1 A peneira de talentos.... 107
6.2 O que é fidelidade?.... 108
6.3 Successive Halving: dividir ao meio, repetidamente.... 108
Implementando do zero....109
6.4 Quando o corte precoce funciona (e quando não)....111
6.5 No scikit-learn: busca com halving.... 112
6.6 Hyperband: não apostar tudo em um corte....113
Uma comparação justa....114
6.7 BOHB: multifidelidade com inteligência....115
6.8 Podas no Optuna.... 116
6.9 Armadilhas e boas práticas.... 117
Resumo do capítulo.... 118
Exercícios de múltipla escolha.... 119
Exercícios discursivos.... 123
Exercícios práticos em Python (scikit-learn)....124
Exercício 1 — Fazendo a conta do Successive Halving....124
Exercício 2 — Halving aleatório na floresta....124
Exercício 3 — Comparando podadores....124
Exercício 4 — O fator η importa....124
Projeto integrador do capítulo....126
Halving em um modelo de boosting: custo e qualidade....126

Capítulo 7 — Métodos evolutivos e programação genética..... 127
7.1 A natureza como otimizadora..... 127
7.2 O vocabulário dos algoritmos genéticos..... 128
7.3 Um algoritmo genético do zero..... 129
7.4 Evolução contra busca aleatória..... 132
7.5 Programação genética e pipelines em árvore..... 133
7.6 Dois objetivos ao mesmo tempo..... 134
7.7 O TPOT na prática..... 135
7.8 Vantagens e limites dos métodos evolutivos..... 136
7.9 Boas práticas..... 137
Resumo do capítulo..... 138
Exercícios de múltipla escolha..... 139
Exercícios discursivos..... 143
Exercícios práticos em Python (scikit-learn)..... 144
Exercício 1 — Seleção por torneio: quem é escolhido?..... 144
Exercício 2 — Algoritmo genético para os hiperparâmetros de um SVM..... 144
Exercício 3 — Fronteira de Pareto: acurácia × esparsidade..... 144
Exercício 4 — Mutação e elitismo em um GA simples..... 144
Projeto integrador do capítulo..... 146
Um mini-TPOT com fronteira de Pareto em Breast Cancer..... 146

Parte III — Automação do Pipeline..... 147

Capítulo 8 — Engenharia e seleção automática de atributos..... 148
8.1 A matéria-prima do modelo..... 148
8.2 Transformações que ajudam..... 149
Mais atributos nem sempre ajudam..... 150
8.3 Seleção de atributos..... 151
8.4 Atributos a partir de várias tabelas: DFS..... 152
8.5 Aprender a criar atributos..... 154
8.6 Criação automática "na marra" e seus perigos..... 154
8.7 Riscos e cuidados..... 157
8.8 Boas práticas..... 157
Resumo do capítulo..... 157
Exercícios de múltipla escolha..... 158
Exercícios discursivos..... 162
Exercícios práticos em Python (scikit-learn)..... 163
Exercício 1 — Quando o produto de dois atributos é a resposta..... 163
Exercício 2 — Quantos atributos manter?..... 163
Exercício 3 — RFECV: deixar a validação cruzada escolher..... 163
Exercício 4 — Mais agregados, mais ganho?..... 163
Projeto integrador do capítulo..... 164
Engenharia de atributos honesta em Diabetes..... 164

Capítulo 9 — Meta-aprendizado, warm start e ensembles..... 165
9.1 Aprender com a experiência..... 165
9.2 O quadro de Rice: escolher o algoritmo por características do problema..... 166
9.3 Meta-atributos..... 166
9.4 Construindo um meta-aprendizado do zero..... 168
9.5 Warm start: partida a quente..... 170
9.6 Ensembles: a sabedoria das multidões..... 172
9.7 Seleção de ensembles (Caruana)..... 174
9.8 O Auto-sklearn: tudo junto..... 176
9.9 Boas práticas..... 177
Resumo do capítulo..... 178

Exercícios de múltipla escolha ... 179
Exercícios discursivos ... 183
Exercícios práticos em Python (scikit-learn) ... 184
Exercício 1 — O retrato de um conjunto de dados ... 184
Exercício 2 — Votação dura ou suave? ... 184
Exercício 3 — Diversidade importa ... 184
Exercício 4 — Warm start com o Optuna ... 184
Projeto integrador do capítulo ... 185
Um mini Auto-sklearn: warm start, busca TPE e ensemble ... 185
Capítulo 10 — Ferramentas de AutoML na prática ... 186
10.1 Uma feira de ferramentas ... 186
10.1.1 O que há de comum entre elas ... 187
10.2 O FLAML na prática ... 187
10.2.1 Lendo a "caixa de ferramentas" do resultado ... 188
10.2.2 Os botões do fit ... 189
10.3 Mais tempo significa melhor modelo? ... 190
10.4 As outras ferramentas: o código de uso ... 191
10.4.1 Auto-sklearn ... 192
10.4.2 TPOT ... 192
10.4.3 H2O AutoML ... 192
10.4.4 AutoGluon ... 193
10.5 Montando uma comparação justa ... 193
10.6 Comparando métodos em várias bases: postos e o teste de Friedman ... 195
10.7 O que as comparações da literatura nos ensinam ... 197
10.8 Escolhendo uma ferramenta ... 197
10.9 Boas práticas ... 198
Resumo do capítulo ... 198
Exercícios de múltipla escolha ... 199
Exercícios discursivos ... 203
Exercícios práticos em Python (scikit-learn) ... 204
Exercício 1 — Primeiro contato com o FLAML ... 204
Exercício 2 — Tempo ou tentativas? ... 204
Exercício 3 — O AutoML bate a linha de base? ... 204
Exercício 4 — Postos e Friedman à mão ... 204
Projeto integrador do capítulo ... 205
Duelo de AutoML em miniatura ... 205
Parte IV — Redes Neurais e Novos Domínios ... 206
Capítulo 11 — Busca de arquitetura neural (NAS) ... 207
11.1 Quem desenha a rede? ... 207
11.2 Os três pilares do NAS ... 208
11.3 O espaço de busca ... 209
11.4 O mapa do tesouro: avaliando todo o espaço ... 209
11.5 Estratégias de busca ... 211
11.6 Mini-NAS: aleatória ou evolução? ... 212
11.7 Estimar sem treinar tudo: fidelidade, pesos compartilhados e preditores ... 214
11.8 Redes boas e pequenas: busca com mais de um objetivo ... 216
11.9 O custo do NAS e a "IA verde" ... 218
11.10 Ferramentas: Auto-Keras ... 218
11.11 Boas práticas ... 218
Resumo do capítulo ... 219
Exercícios de múltipla escolha ... 220
Exercícios discursivos ... 224

Exercícios práticos em Python (scikit-learn) ... 225
Exercício 1 — Contando arquiteturas ... 225
Exercício 2 — Maior é sempre melhor? ... 225
Exercício 3 — Aleatória ou hill climbing? ... 225
Exercício 4 — Quanto se pode confiar numa avaliação barata? ... 225
Projeto integrador do capítulo ... 227
Uma rede enxuta por busca de arquitetura ... 227
Capítulo 12 — AutoML para séries temporais, texto e imagens ... 228
12.1 Além das tabelas ... 228
12.2 Séries temporais: o tempo não volta ... 229
12.2.1 O que muda ... 229
12.2.2 Prever com modelos de tabela: atrasos (lags) ... 229
12.2.3 Um mini-AutoML para previsão ... 231
12.2.4 O que a literatura diz ... 233
12.3 Texto: transformar palavras em números ... 233
12.3.1 O pipeline de texto ... 233
12.3.2 Um experimento com texto simulado ... 234
12.3.3 Modelos pré-treinados e a era dos Transformers ... 236
12.4 Imagens: formas, deslocamentos e dados escassos ... 236
12.4.1 O que muda com imagens ... 236
12.4.2 Um mini-AutoAugment ... 236
12.4.3 Transferência e modelos pré-treinados ... 238
12.5 Quadro-resumo ... 238
12.6 Boas práticas ... 239
Resumo do capítulo ... 239
Exercícios de múltipla escolha ... 240
Exercícios discursivos ... 244
Exercícios práticos em Python (scikit-learn) ... 245
Exercício 1 — Validação aleatória ou temporal? ... 245
Exercício 2 — Três linhas de base ... 245
Exercício 3 — Quando n-gramas importam? ... 245
Exercício 4 — Qual distorção ajuda mais? ... 245
Projeto integrador do capítulo ... 246
Previsão de manchas solares com mini-AutoML ... 246
Parte V — Responsabilidade e Prática ... 247
Capítulo 13 — Explicabilidade, ética e limitações do AutoML ... 248
13.1 Três perguntas antes de confiar ... 248
13.2 Explicabilidade ... 249
13.2.1 Interpretável ou explicado depois? ... 249
13.2.2 Explicação global e local ... 250
13.3 Justiça e vieses ... 253
13.3.1 De onde vem o viés? ... 253
13.3.2 Como medir ... 253
13.3.3 "Basta remover o atributo sensível?" ... 253
13.3.4 Mitigação: ajustar o limiar por grupo ... 254
13.4 O custo da busca ... 256
13.5 Os limites do AutoML ... 256
13.6 Lista de verificação ética para um projeto de AutoML ... 258
13.7 Boas práticas ... 258
Resumo do capítulo ... 259
Exercícios de múltipla escolha ... 260
Exercícios discursivos ... 264

Exercícios práticos em Python (scikit-learn) ... 265
Exercício 1 — Quem importa para o modelo? ... 265
Exercício 2 — Medindo a disparidade por grupo ... 265
Exercício 3 — Detectando proxies ... 265
Exercício 4 — Igualdade de oportunidade com os rótulos disponíveis ... 265
Projeto integrador do capítulo ... 266
Auditoria ética de um modelo de AutoML ... 266
Capítulo 14 — MLOps, implantação e projeto integrador ... 267
14.1 Do notebook ao mundo real ... 267
14.2 O ciclo de vida de um modelo ... 268
14.3 Salvar e reproduzir ... 268
14.4 Servir o modelo: lote ou tempo real ... 269
14.5 Monitoramento: o mundo muda ... 271
14.6 Retreinar: campeão e desafiante ... 274
14.7 Um pipeline completo, de ponta a ponta ... 275
14.8 O projeto integrador final ... 276
14.9 Boas práticas ... 276
Resumo do capítulo ... 276
Exercícios de múltipla escolha ... 277
Exercícios discursivos ... 281
Exercícios práticos em Python (scikit-learn) ... 282
Exercício 1 — Ficha do modelo e reprodutibilidade ... 282
Exercício 2 — Quanto custa cada previsão? ... 282
Exercício 3 — Detectando deriva com KS e PSI ... 282
Exercício 4 — Quando trocar o campeão? ... 282
Projeto integrador final ... 283
Projeto integrador: AutoML de ponta a ponta, com ética e produção ... 283
Apêndice A — Gabarito dos exercícios ... 284
Capítulo 1 — Bem-vindo ao AutoML ... 285
Múltipla escolha ... 285
Discursivas (respostas-modelo) ... 286
Capítulo 2 — Aprendizado de máquina em um capítulo ... 288
Múltipla escolha ... 288
Discursivas (respostas-modelo) ... 288
Capítulo 3 — Pipelines e pré-processamento ... 290
Múltipla escolha ... 290
Discursivas (respostas-modelo) ... 290
Capítulo 4 — Busca em grade e aleatória ... 292
Múltipla escolha ... 292
Discursivas (respostas-modelo) ... 292
Capítulo 5 — Otimização bayesiana ... 294
Múltipla escolha ... 294
Discursivas (respostas-modelo) ... 294
Capítulo 6 — Métodos multifidelidade ... 296
Múltipla escolha ... 296
Discursivas (respostas-modelo) ... 296
Capítulo 7 — Métodos evolutivos e programação genética ... 298
Múltipla escolha ... 298
Discursivas (respostas-modelo) ... 298

Capítulo 8 — Engenharia e seleção automática de atributos 300
Múltipla escolha 300
Discursivas (respostas-modelo) 300
Capítulo 9 — Meta-aprendizado, warm start e ensembles 302
Múltipla escolha 302
Discursivas (respostas-modelo) 302
Capítulo 10 — Ferramentas de AutoML na prática 304
Múltipla escolha 304
Discursivas (respostas-modelo) 304
Capítulo 11 — Busca de arquitetura neural (NAS) 306
Múltipla escolha 306
Discursivas (respostas-modelo) 306
Capítulo 12 — AutoML para séries temporais, texto e imagens 308
Múltipla escolha 308
Discursivas (respostas-modelo) 308
Capítulo 13 — Explicabilidade, ética e limitações do AutoML 310
Múltipla escolha 310
Discursivas (respostas-modelo) 310
Capítulo 14 — MLOps, implantação e projeto integrador 312
Múltipla escolha 312
Discursivas (respostas-modelo) 312
Apêndice B — Glossário 314
Apêndice C — Guia rápido 318
C.1 Qual método de busca usar? 318
C.2 Receitas de código 318
Pipeline sem vazamento 318
Busca aleatória 318
Validação aninhada (estimar o ganho da busca) 319
Successive Halving no scikit-learn 319
Otimização bayesiana (TPE) com Optuna 319
FLAML 319
Ensemble por votação suave 319
Séries temporais: validação sem olhar o futuro 319
Explicação e auditoria 320
Salvar e recarregar o pipeline 320
C.3 Fórmulas e métricas em palavras 320
C.4 Lista de verificação de um projeto 320
C.5 As dez armadilhas mais comuns 321
C.6 Mapa dos capítulos 321
Apêndice D — Soluções dos exercícios práticos 322
Capítulo 1 — Bem-vindo ao AutoML 323
Exercício 1 — Por que precisamos de uma linha de base 323
Exercício 2 — Ampliando o espaço de busca 323
Exercício 3 — O efeito da semente 324
Exercício 4 — Contando o tamanho do espaço de busca 325
Capítulo 2 — Aprendizado de máquina em um capítulo 327
Exercício 1 — Curvas de aprendizado 327
Exercício 2 — Métricas em dados desbalanceados 327
Exercício 3 — Uma divisão ou várias? 328

Exercício 4 — Reproduzindo o efeito do vencedor 329

Capítulo 3 — Pipelines e pré-processamento 331

Exercício 1 — Qual imputação escolher? 331
Exercício 2 — Ordinal ou one-hot? 331
Exercício 3 — Vazamento: a nota honesta 332
Exercício 4 — Buscando dentro do pipeline 333

Capítulo 4 — Busca em grade e aleatória 334

Exercício 1 — Sua primeira busca em grade 334
Exercício 2 — Grade ou aleatória? Com a floresta, empatou 334
Exercício 3 — Escala logarítmica faz diferença 335
Exercício 4 — Medindo o otimismo da busca 336

Capítulo 5 — Otimização bayesiana 337

Exercício 1 — Sua própria otimização bayesiana 337
Exercício 2 — Optuna em uma floresta aleatória 338
Exercício 3 — Reproduzindo e corrigindo a falha do CASH 339
Exercício 4 — Quanto explorar? O coeficiente do UCB 340

Capítulo 6 — Métodos multifidelidade 341

Exercício 1 — Fazendo a conta do Successive Halving 341
Exercício 2 — Halving aleatório na floresta 341
Exercício 3 — Comparando podadores 342
Exercício 4 — O fator η importa 343

Capítulo 7 — Métodos evolutivos e programação genética 345

Exercício 1 — Seleção por torneio: quem é escolhido? 345
Exercício 2 — Algoritmo genético para os hiperparâmetros de um SVM 345
Exercício 3 — Fronteira de Pareto: acurácia × esparsidade 346
Exercício 4 — Mutação e elitismo em um GA simples 347

Capítulo 8 — Engenharia e seleção automática de atributos 349

Exercício 1 — Quando o produto de dois atributos é a resposta 349
Exercício 2 — Quantos atributos manter? 349
Exercício 3 — RFECV: deixar a validação cruzada escolher 350
Exercício 4 — Mais agregados, mais ganho? 351

Capítulo 9 — Meta-aprendizado, warm start e ensembles 353

Exercício 1 — O retrato de um conjunto de dados 353
Exercício 2 — Votação dura ou suave? 353
Exercício 3 — Diversidade importa 354
Exercício 4 — Warm start com o Optuna 355

Capítulo 10 — Ferramentas de AutoML na prática 357

Exercício 1 — Primeiro contato com o FLAML 357
Exercício 2 — Tempo ou tentativas? 357
Exercício 3 — O AutoML bate a linha de base? 358
Exercício 4 — Postos e Friedman à mão 359

Capítulo 11 — Busca de arquitetura neural (NAS) 361

Exercício 1 — Contando arquiteturas 361
Exercício 2 — Maior é sempre melhor? 361
Exercício 3 — Aleatória ou hill climbing? 362
Exercício 4 — Quanto se pode confiar numa avaliação barata? 363

Capítulo 12 — AutoML para séries temporais, texto e imagens 365

Exercício 1 — Validação aleatória ou temporal? 365
Exercício 2 — Três linhas de base 365
Exercício 3 — Quando n-gramas importam? 366

Exercício 4 — Qual distorção ajuda mais? 367
Capítulo 13 — Explicabilidade, ética e limitações do AutoML 368
Exercício 1 — Quem importa para o modelo? 368
Exercício 2 — Medindo a disparidade por grupo 368
Exercício 3 — Detectando proxies 369
Exercício 4 — Igualdade de oportunidade com os rótulos disponíveis 370
Capítulo 14 — MLOps, implantação e projeto integrador 372
Exercício 1 — Ficha do modelo e reprodutibilidade 372
Exercício 2 — Quanto custa cada previsão? 372
Exercício 3 — Detectando deriva com KS e PSI 373
Exercício 4 — Quando trocar o campeão? 374
Apêndice E — Soluções de referência dos projetos integradores 375
Capítulo 1 — Bem-vindo ao AutoML 376
Orçamento de busca e honestidade na avaliação 376
Capítulo 2 — Aprendizado de máquina em um capítulo 378
Protocolo completo de avaliação em Digits 378
Capítulo 3 — Pipelines e pré-processamento 380
Pipeline robusto para dados sujos 380
Capítulo 4 — Busca em grade e aleatória 382
Relatório de otimização de hiperparâmetros em Digits 382
Capítulo 5 — Otimização bayesiana 384
AutoML minimalista com Optuna em regressão (Diabetes) 384
Capítulo 6 — Métodos multifidelidade 386
Halving em um modelo de boosting: custo e qualidade 386
Capítulo 7 — Métodos evolutivos e programação genética 388
Um mini-TPOT com fronteira de Pareto em Breast Cancer 388
Capítulo 8 — Engenharia e seleção automática de atributos 391
Engenharia de atributos honesta em Diabetes 391
Capítulo 9 — Meta-aprendizado, warm start e ensembles 393
Um mini Auto-sklearn: warm start, busca TPE e ensemble 393
Capítulo 10 — Ferramentas de AutoML na prática 395
Duelo de AutoML em miniatura 395
Capítulo 11 — Busca de arquitetura neural (NAS) 397
Uma rede enxuta por busca de arquitetura 397
Capítulo 12 — AutoML para séries temporais, texto e imagens 399
Previsão de manchas solares com mini-AutoML 399
Capítulo 13 — Explicabilidade, ética e limitações do AutoML 401
Auditoria ética de um modelo de AutoML 401
Capítulo 14 — MLOps, implantação e projeto integrador 403
Projeto integrador: AutoML de ponta a ponta, com ética e produção 403
Referências 407

Apresentação

Para quem é este livro

Este livro foi escrito para um **público misto**: estudantes de graduação e pós-graduação, profissionais de dados e pessoas curiosas que querem entender como as máquinas conseguem **escolher e ajustar, sozinhas, modelos de aprendizado de máquina**.

Você **não precisa** de conhecimento prévio. O Capítulo 2 faz uma revisão do essencial de aprendizado de máquina, e cada termo técnico é explicado no momento em que aparece. Quando surgir uma fórmula, ela virá com uma tradução "em palavras" e, quase sempre, com um exemplo numérico pequeno.

Ao mesmo tempo, o livro avança. Os capítulos partem do nível básico, passam pelo intermediário e chegam a temas avançados: otimização bayesiana, métodos de multifidelidade, programação genética, meta-aprendizado, busca de arquiteturas neurais, explicabilidade, ética e MLOps.

O que é AutoML, em uma frase

AutoML (aprendizado de máquina automatizado) é o conjunto de técnicas que **automatiza as escolhas** feitas na construção de um modelo: preparo dos dados, atributos, algoritmo, hiperparâmetros e combinação de modelos. O livro mostra **como** essas escolhas são automatizadas, e não apenas como apertar o botão de uma ferramenta.

Como o livro está organizado

O livro tem **cinco partes** e **catorze capítulos**.

- **Parte I — Fundamentos** (Caps. 1 a 3): o que é AutoML e o problema CASH; o essencial de aprendizado de máquina; pipelines e pré-processamento sem vazamento de dados.
- **Parte II — Otimização de Hiperparâmetros** (Caps. 4 a 7): busca em grade e aleatória; otimização bayesiana; Successive Halving, Hyperband e BOHB; métodos evolutivos e TPOT.
- **Parte III — Automação do Pipeline** (Caps. 8 a 10): engenharia e seleção automática de atributos; meta-aprendizado, *warm start* e ensembles; ferramentas de AutoML na prática.
- **Parte IV — Redes Neurais e Novos Domínios** (Caps. 11 e 12): busca de arquiteturas neurais (NAS); AutoML para séries temporais, texto e imagens.
- **Parte V — Responsabilidade e Prática** (Caps. 13 e 14): explicabilidade, ética, vieses e limitações; MLOps e um projeto integrador final.

Cada capítulo segue o mesmo roteiro:

1. **Objetivos de aprendizagem**, no início.
2. **Conteúdo** com analogias, figuras, exemplos e códigos em Python.
3. **Resumo** em cartões de memorização.
4. **30 questões de múltipla escolha** e **20 questões discursivas**, com gabarito no Apêndice A.
5. **Exercícios práticos em Python** (scikit-learn), com soluções e saídas no Apêndice D.
6. **Projeto integrador do capítulo**, com solução de referência e critérios de avaliação no Apêndice E.

O Capítulo 14 termina com o **projeto integrador final**, que reúne o que foi aprendido em um estudo completo.

Ícones e caixas

Algumas caixas coloridas aparecem ao longo do texto.

- **No seu dia a dia:** liga o conceito a uma situação cotidiana.
- **Exemplo:** um caso numérico ou prático.
- **Dica:** um atalho ou boa prática.
- **Atenção:** um cuidado para evitar erros comuns.
- **Saiba mais:** uma indicação para ir além.
- **Memorize:** a ideia que vale a pena guardar.

Ambiente de programação

Os códigos usam **Python** e, sobretudo, a biblioteca **scikit-learn**, acompanhada de NumPy, pandas, SciPy e Matplotlib. Para os capítulos de AutoML, usamos também **Optuna** e **FLAML**; **statsmodels** (séries temporais), **LightGBM** e **XGBoost** (modelos de árvores) e, na explicabilidade, **SHAP**. Todos os exemplos executados foram rodados com estas versões:

Biblioteca	Versão
Python	3.13
scikit-learn	1.9.1
NumPy / pandas / SciPy	2.5.3 / 3.0.5 / 1.18.1
Matplotlib	3.11.2
Optuna / FLAML	5.0.0 / 2.7.0
SHAP / XGBoost / LightGBM	0.54.1 / 3.4.1 / 4.7.0
statsmodels	0.15.0

Diferenças pequenas nos números são normais com outras versões. Você pode executar tudo no seu computador ou no **Google Colab**, gratuito e no navegador.

Um aviso importante sobre o que foi testado

Este livro procura ser transparente sobre o que foi executado durante a sua elaboração:

- **Códigos executados.** Todos os códigos do texto e das soluções que **mostram uma saída** foram executados de fato, e as saídas impressas são as obtidas na execução. Os experimentos usam sementes fixas para que você consiga reproduzi-los.
- **Códigos não executados.** Algumas ferramentas de AutoML (TPOT, Auto-sklearn, AutoGluon, H2O AutoML, Auto-Keras e o serviço FastAPI do Capítulo 14) **não puderam ser instaladas** no ambiente em que o livro foi preparado. Os trechos de código que as usam aparecem **sem saída** e são acompanhados de um aviso. Como as interfaces dessas bibliotecas mudam entre versões, consulte a documentação atual antes de usá-los.
- **Dados.** Os exercícios executados usam conjuntos de dados que acompanham o scikit-learn: Iris, Wine, Breast Cancer Wisconsin e Digits, originários do repositório **UCI Machine Learning**, e Diabetes, originário de um estudo estatístico clássico. Há também a série de CO_2 de Mauna Loa e a série de manchas solares, que acompanham a biblioteca statsmodels, e dados **simulados** (crédito, textos de avaliação, séries e

ruídos), sempre indicados como tais no texto. Os dados simulados ilustram fenômenos; **não** medem o desempenho no mundo real. As bases do UCI e do Kaggle que aparecem na seção "Para ir além" **não foram usadas nem testadas** nos códigos do livro: confira descrição, licença e colunas antes de usá-las.

- **Referências.** Os artigos, livros e eventos citados foram conferidos em buscas na internet durante a elaboração. Quando uma informação (como o intervalo de páginas) não pôde ser confirmada, ela foi **omitida** da referência, e não preenchida de memória. Mesmo assim, confira o DOI e as páginas antes de citar o livro em um trabalho acadêmico.
- **Resultados honestos.** Em vários experimentos, o resultado foi **menos favorável ao método sofisticado** do que se poderia esperar, ou foi um empate com a busca aleatória. Isso foi mantido e comentado de propósito: saber quando uma técnica **não** ajuda é parte da formação.

Como estudar com este livro

1. **Leia o texto com o código ao lado.** Execute cada bloco e altere os valores para ver o que muda.
2. **Tente os exercícios antes de olhar as soluções.** O aprendizado vem do esforço de resolver.
3. **Faça os projetos.** Eles são o melhor treino para situações reais.
4. **Use o glossário** (Apêndice B) e o **guia rápido** (Apêndice C) como consulta.
5. **Não confie cegamente em nenhum resultado**, nem nos do livro: repita com outras sementes, compare com uma linha de base e desconfie de notas boas demais.

Boa leitura, e bons experimentos!

Araken de Medeiros Santos

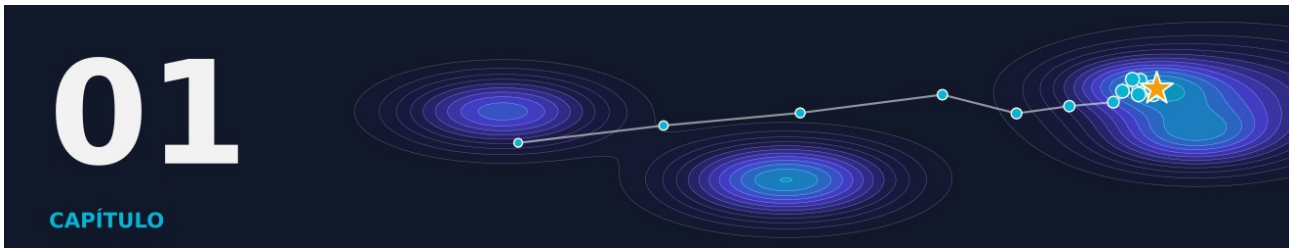
PARTE I

Parte I — Fundamentos

O terreno comum: o que é AutoML e por que ele existe, o essencial de aprendizado de máquina para quem começa do zero e como montar pipelines confiáveis, sem vazamento de dados.

Nesta parte:

- Cap. 1 — Bem-vindo ao AutoML
- Cap. 2 — Aprendizado de máquina em um capítulo
- Cap. 3 — Pipelines e pré-processamento



Capítulo 1 — Bem-vindo ao AutoML

OBJETIVOS DE APRENDIZAGEM

Ao final deste capítulo você será capaz de:

- Explicar, com suas palavras, o que é AutoML e por que ele existe.
- Descrever as etapas de um projeto de aprendizado de máquina e identificar quais podem ser automatizadas.
- Formular o problema CASH (escolha de algoritmo e de hiperparâmetros) e explicar por que ele é difícil.
- Reconhecer os marcos históricos e as principais famílias de métodos de AutoML.
- Apontar o que o AutoML faz bem, o que ele não faz e qual é o papel da pessoa no processo.
- Preparar o ambiente Python e construir seu primeiro "mini-AutoML" em poucas linhas.

1.1 Uma história para começar

Imagine que você abriu um restaurante. Hoje é o grande dia. Você quer criar **a melhor receita de lasanha da cidade**.

Mas há um problema. Existem **milhares de combinações possíveis**. Qual massa usar? Que queijo? Quanto de molho? Quantos minutos no forno? Em que temperatura?

Você começa a testar. Faz uma lasanha, prova, anota. Faz outra, ajusta o sal, prova de novo. Depois de uma semana, você testou cinco receitas. Está cansado. E ainda não sabe se a melhor receita do mundo estava entre as combinações que você nem tentou.

Agora imagine um **ajudante incansável**. Ele testa centenas de receitas, anota cada resultado, percebe quais combinações dão certo e passa a investigar justamente as mais promissoras. No fim do dia, entrega a você as três melhores. Você prova e decide.

Esse ajudante é o **AutoML**.

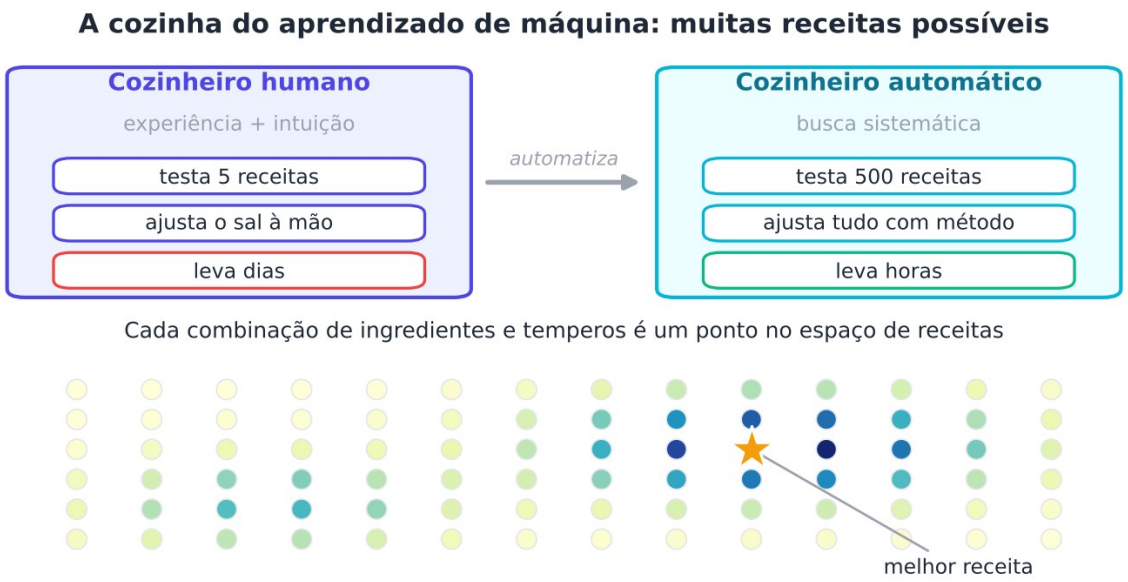


Figura 1.1 — O cozinheiro humano testa poucas receitas. O cozinheiro automático explora o espaço de receitas de forma sistemática e guiada.

> **No seu dia a dia**
>
> Sempre que você compara três planos de celular, testa vários caminhos até o trabalho ou ajusta a temperatura do chuveiro até ficar boa, você faz uma **busca** por uma boa configuração. O AutoML é essa mesma ideia, aplicada a modelos de aprendizado de máquina e com método.

Neste capítulo, você vai aprender o que é AutoML, de onde ele vem e como ele pensa. Vamos com calma. Cada termo técnico será explicado assim que aparecer.

1.2 O que é AutoML?

Aprendizado de máquina (*machine learning*) é a área da Inteligência Artificial que cria programas capazes de aprender padrões a partir de **dados**, sem que cada regra seja escrita à mão (Hastie; Tibshirani; Friedman, 2009). Um programa que aprendeu um padrão é chamado de **modelo**.

AutoML (*Automated Machine Learning*, ou aprendizado de máquina automatizado) é o conjunto de técnicas que **automatiza as decisões necessárias para construir um bom modelo**. Em vez de uma pessoa testar tudo à mão, um programa faz a busca (Hutter; Kotthoff; Vanschoren, 2019).

Veja o que "decisões" quer dizer. Para criar um modelo, é preciso escolher:

- como **preparar** os dados (completar valores que faltam, colocar tudo na mesma escala);
- quais **atributos** usar (os atributos são as "colunas" dos dados, como idade ou renda);
- qual **algoritmo** aplicar (uma árvore de decisão? uma regressão? uma rede neural?);
- quais **hiperparâmetros** usar (os "botões de ajuste" do algoritmo, que você define *antes* do treino, como a profundidade de uma árvore);
- como **combinar** vários modelos;
- como **avaliar** se o resultado é bom.

Cada escolha influencia as demais. Ao conjunto de todas as combinações possíveis damos o nome de **espaço de busca**. O AutoML é, no fundo, **a arte de explorar esse espaço com inteligência**.

> **Memorize**
>
> **AutoML = automatizar as escolhas** que antes eram feitas por tentativa, erro e experiência. A pessoa continua responsável pelo problema, pelos dados e pela decisão final.

1.3 O caminho do dado até o modelo

Para entender o que o AutoML automatiza, precisamos enxergar o caminho completo. Pense em uma **linha de montagem**.

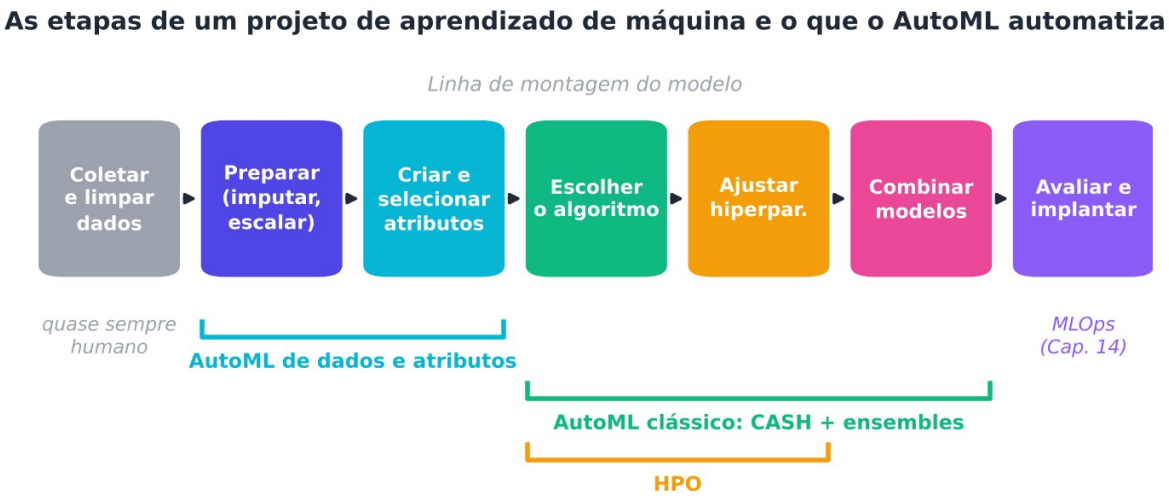


Figura 1.2 — A linha de montagem do aprendizado de máquina. O AutoML cobre, sobretudo, do preparo dos dados até a combinação de modelos.

Cada etapa tem um papel:

1. **Coletar e limpar dados.** Reunir as informações e corrigir erros. É a etapa mais trabalhosa e, hoje, ainda muito dependente de pessoas.
2. **Preparar.** Completar valores ausentes e padronizar escalas. Veremos no Capítulo 3.
3. **Criar e selecionar atributos.** Criar colunas novas e descartar as inúteis (Capítulo 8).
4. **Escolher o algoritmo.** Entre dezenas de opções (Capítulos 2 e 9).
5. **Ajustar hiperparâmetros.** Girar os botões do algoritmo. É a especialidade do AutoML (Capítulos 4 a 7).
6. **Combinar modelos.** Juntar vários modelos para errar menos (Capítulo 9).
7. **Avaliar e implantar.** Medir o desempenho e colocar o modelo para trabalhar (Capítulos 13 e 14).

Chamamos esse encadeamento de **pipeline** (do inglês, "tubulação" ou "linha de montagem"). Um pipeline de aprendizado de máquina é uma **sequência de etapas** que transforma dados brutos em previsões. Você vai usá-lo o tempo todo.

1.4 Por que o AutoML existe?

Há quatro razões principais.

1. Faltam especialistas. Montar um bom modelo exige experiência. Muitas empresas, escolas, hospitais e prefeituras têm dados valiosos, mas poucas pessoas com esse treino. O AutoML reduz a barreira de entrada.

2. O espaço de busca é gigante. Combine apenas 15 algoritmos, 14 métodos de seleção de atributos e dezenas de hiperparâmetros. Já são **mais de cem botões** para ajustar. O sistema Auto-sklearn, por exemplo, trabalha com um espaço dessa ordem (Feurer *et al.*, 2015). Nenhuma pessoa testa tudo isso à mão.

3. O tempo humano é caro. Uma pessoa especialista pode gastar dias ajustando um modelo. Um programa pode fazer boa parte desse trabalho enquanto ela cuida de tarefas que exigem julgamento.

4. Reprodutibilidade. Ajustes feitos "no olhômetro" são difíceis de repetir e de explicar. Uma busca automática **registra cada tentativa**, e isso torna o processo auditável.

> **Atenção**
>
> Automatizar **não** significa abandonar o raciocínio. Um modelo ótimo para o problema errado continua sendo inútil. O AutoML otimiza o que você mandar otimizar. Se a métrica estiver errada, ele otimizará a coisa errada, e com muita eficiência.

1.5 O problema CASH

Os pesquisadores deram um nome ao desafio central do AutoML clássico: **CASH**, sigla em inglês para *Combined Algorithm Selection and Hyperparameter optimization*. Em português: **seleção combinada de algoritmo e otimização de hiperparâmetros** (Thornton *et al.*, 2013).

Por que "combinada"? Porque as duas escolhas dependem uma da outra. Observe a figura.

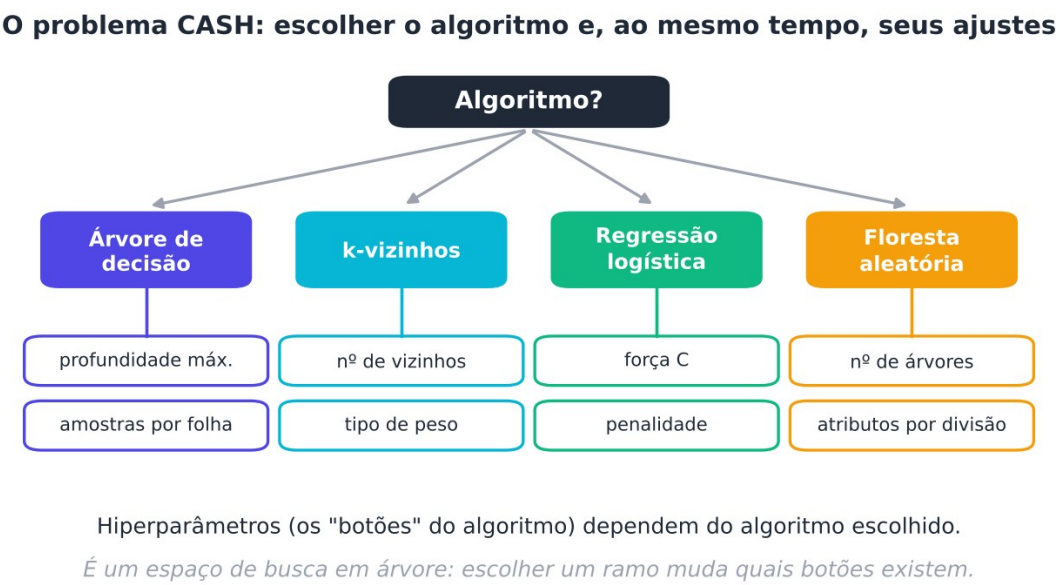


Figura 1.3 — O espaço CASH tem forma de árvore. Cada algoritmo traz seus próprios hiperparâmetros, e escolher um ramo muda quais botões existem.

Uma árvore de decisão tem como botão a **profundidade máxima**. Os k-vizinhos têm o **número de vizinhos**. Esses botões nem existem no outro algoritmo. Por isso não dá para ajustar tudo separadamente: a melhor escolha de algoritmo depende de quão bem cada um foi ajustado.

Esse problema tem raízes antigas. Em 1976, John Rice formulou o **problema da seleção de algoritmos**: dado um problema, qual algoritmo, entre vários, resolve melhor? (Rice, 1976). E duas décadas depois, David Wolpert demonstrou os chamados teoremas do **"almoço grátis não existe"** (*No Free Lunch*): **nenhum algoritmo é o**

melhor para todos os problemas (Wolpert, 1996). Em outras palavras, o melhor algoritmo depende dos dados. Por isso precisamos *testar* e não apenas *supor*.

Podemos escrever o CASH em uma linha. Não se assuste: logo abaixo vem a tradução em palavras.

$$(A^*, \lambda^*) = \operatorname{argmin}_{A,\lambda} \frac{1}{k} \sum_{i=1..k} \text{Erro}(A_\lambda, D_{\text{treino}}^i, D_{\text{validação}}^i)$$

Em palavras: procure o par (algoritmo *A*, ajustes λ) que **erre menos, em média, nos dados de validação** (a parte dos dados que o modelo não usou para aprender). A média é feita em *k* divisões diferentes dos dados, para não depender de um sorteio de sorte.

> **Exemplo numérico**
>
> Você testa 3 receitas em 5 degustações e anota a nota média: A = 6,1; B = 7,4; C = 6,8. Qual escolher? A **B**, a de maior média (ou menor "erro"). Fazer isso com algoritmos e hiperparâmetros é exatamente o CASH, só que no lugar de "nota" usamos uma métrica de erro.

1.6 Uma breve história do AutoML

O AutoML é um campo jovem. Os principais marcos aparecem na linha do tempo.

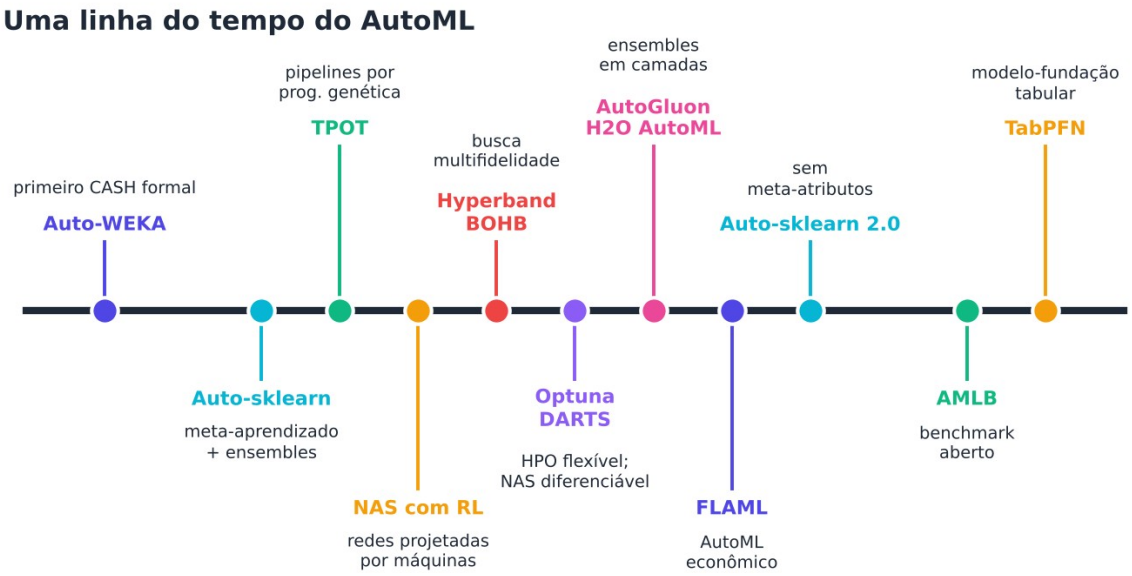


Figura 1.4 — Marcos do AutoML. Cada marco traz uma ideia que aparecerá ao longo do livro.

Veja os destaques:

- **Auto-WEKA (2013)** foi o primeiro sistema a tratar o CASH de forma completa, usando otimização bayesiana sobre a biblioteca WEKA (Thornton *et al.*, 2013).
- **Auto-sklearn (2015)** trouxe o AutoML para o ecossistema Python do scikit-learn. Introduziu o **meta-aprendizado** (aprender com problemas anteriores) e a **combinação automática de modelos** (Feurer *et al.*, 2015). Sua versão 2.0 buscou um funcionamento "sem configuração manual" (*hands-free*), apoiado em meta-aprendizado (Feurer *et al.*, 2022).
- **TPOT (2016)** usou **programação genética** (inspirada na evolução) para montar pipelines inteiros (Olson; Moore, 2016).
- **NAS (2017)** mostrou que máquinas podem projetar **a arquitetura de redes neurais** (Zoph; Le, 2017).

- **Hyperband e BOHB (2018)** aceleraram a busca, abandonando cedo as tentativas ruins (Li *et al.*, 2018; Falkner; Klein; Hutter, 2018).
- **Optuna (2019)** popularizou a otimização flexível de hiperparâmetros (Akiba *et al.*, 2019).
- **AutoGluon e H2O AutoML (2020)** apostaram em combinar muitos modelos em camadas (Erickson *et al.*, 2020; LeDell; Poirier, 2020).
- **FLAML (2021)** priorizou o baixo custo computacional (Wang *et al.*, 2021).
- **AMLB (2024)** ofereceu um *benchmark* aberto para comparar ferramentas de forma justa (Gijsbers *et al.*, 2024).
- **TabPFN (2025)** apresentou um modelo pré-treinado para dados em tabelas pequenas, que dispensa busca longa (Hollmann *et al.*, 2025).

> **Saiba mais**
>
> Para uma visão panorâmica e confiável do campo, consulte o livro organizado por Hutter, Kotthoff e Vanschoren (2019), de acesso aberto, e o levantamento de He, Zhao e Chu (2021). Para uma comparação experimental de ferramentas, veja Zöller e Huber (2021).

1.7 Quanto é "automático"?

"Automático" é uma questão de grau. Uma analogia ajuda: os **carros autônomos** têm níveis de autonomia, do carro que só avisa se você sair da faixa até o que dirige sozinho em qualquer lugar. Com o AutoML ocorre algo parecido.

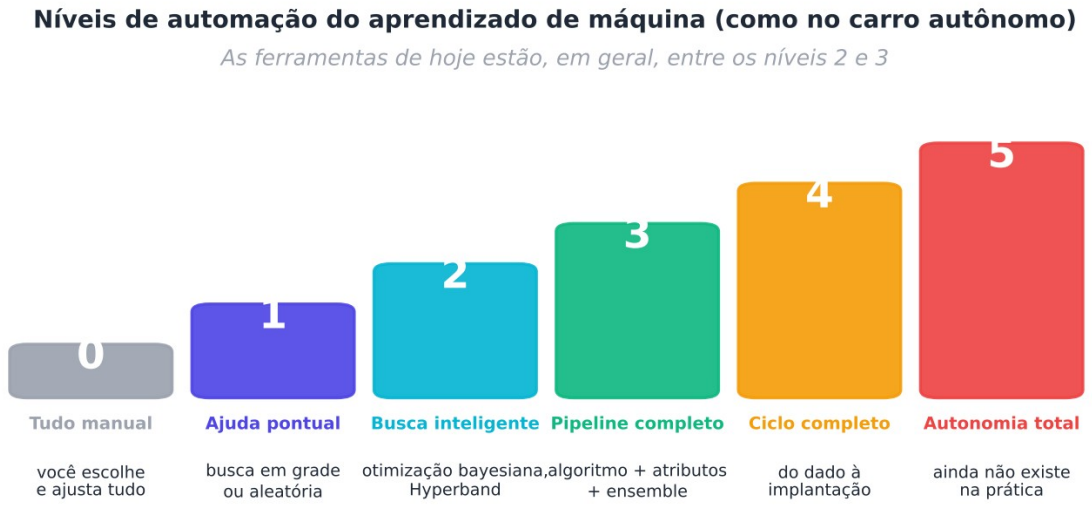


Figura 1.5 — Níveis de automação do aprendizado de máquina. É uma proposta didática deste livro, inspirada nos níveis de automação veicular, e não uma classificação oficial.

- **Nível 0:** você escolhe e ajusta tudo manualmente.
- **Nível 1:** você automatiza apenas uma tarefa pontual, como uma busca em grade.
- **Nível 2:** uma busca inteligente guia os ajustes (Capítulos 5 e 6).
- **Nível 3:** o sistema monta o pipeline inteiro, com atributos e combinação de modelos (Capítulos 8 a 10).
- **Nível 4:** a automação cobre o ciclo completo, do dado à implantação e ao monitoramento (Capítulo 14).
- **Nível 5:** autonomia total. **Isso ainda não existe na prática**.

1.8 O que o AutoML não faz

É importante desfazer alguns mitos.

Mito 1: "O AutoML elimina a necessidade de especialistas." Não elimina. Ele **muda o trabalho** do especialista: menos tempo ajustando botões, mais tempo formulando o problema, checando os dados e interpretando resultados.

Mito 2: "O AutoML sempre vence um modelo feito à mão." Nem sempre. Em dados pequenos ou com conhecimento especializado forte, um bom modelo manual pode competir. Veremos como comparar de forma justa no Capítulo 10.

Mito 3: "O AutoML corrige dados ruins." Não corrige. Se os dados tiverem erros, vieses ou vazamento de informação, o resultado será ruim, só que mais bem ajustado (Kapoor; Narayanan, 2023).

Mito 4: "Mais busca é sempre melhor." Não é. Buscar demais pode **ajustar o modelo ao acaso** da validação. É o *overfitting* da busca, que estudaremos nos Capítulos 4 e 13 (Cawley; Talbot, 2010).

Mito 5: "O AutoML é de graça." Tem custo de tempo, energia e dinheiro. Isso também será discutido (Strubell; Ganesh; McCallum, 2019; Schwartz *et al.*, 2020).

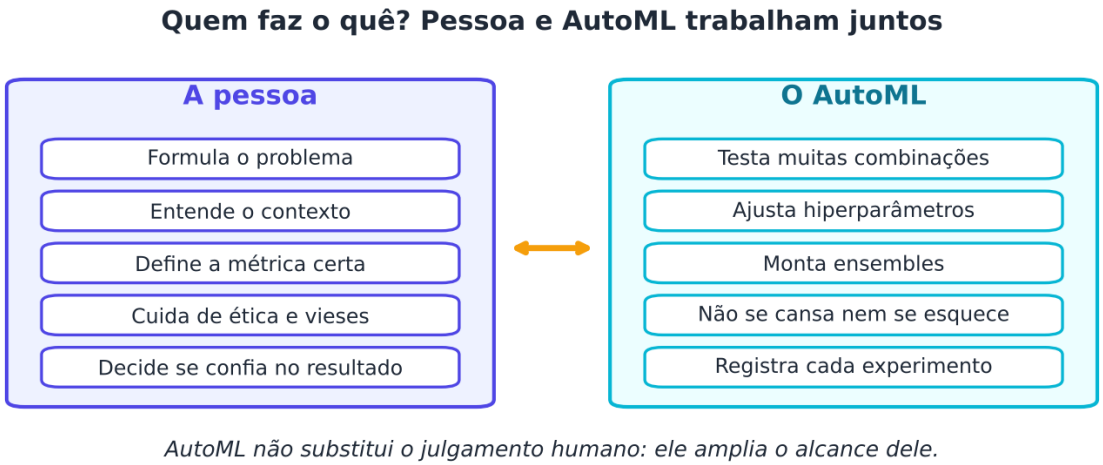


Figura 1.6 — Divisão de trabalho entre a pessoa e o AutoML. As decisões de contexto, ética e confiança continuam humanas.

> MEMORIZE **Regra de ouro**
>
> O AutoML **procura**. A pessoa **define o que procurar** e **decide o que fazer** com o que foi encontrado.

1.9 Preparando o ambiente

Ao longo do livro, você vai praticar em **Python**, a linguagem mais usada em ciência de dados, com a biblioteca **scikit-learn** (Pedregosa *et al.*, 2011). Você pode usar o seu computador ou o **Google Colab** (um ambiente gratuito no navegador).

Instale as bibliotecas principais no terminal (ou em uma célula do Colab, com `!` na frente):

```bash
pip install numpy pandas matplotlib scikit-learn scipy optuna flaml shap
```

Verifique se tudo funciona:

PYTHON
```python
import sklearn, numpy, pandas, scipy
print("scikit-learn:", sklearn.__version__)
print("numpy       :", numpy.__version__)
print("pandas      :", pandas.__version__)
print("scipy       :", scipy.__version__)
```

SAÍDA
```
scikit-learn: 1.9.1
numpy       : 2.5.3
pandas      : 3.0.5
scipy       : 1.18.1
```

Os exemplos do livro foram executados com estas versões. Diferenças pequenas nos números são normais se você usar versões diferentes.

1.10 Dados deste capítulo: o conjunto Wine

Usaremos o conjunto **Wine**, do repositório UCI (Dua; Graff, 2019). Ele descreve **178 vinhos** de três cultivares da mesma região da Itália, por **13 medidas químicas** (álcool, acidez, cor etc.). A tarefa: **descobrir a categoria do vinho a partir das medidas**. Como a categoria é conhecida, trata-se de **classificação** (prever uma categoria).

PYTHON
```python
from sklearn.datasets import load_wine
import numpy as np

dados = load_wine()
X, y = dados.data, dados.target
print("Formato de X:", X.shape, "(linhas = vinhos, colunas = medidas)")
print("Medidas:", list(dados.feature_names)[:4], "...")
print("Vinhos por categoria:", np.bincount(y))
```

SAÍDA
```
Formato de X: (178, 13) (linhas = vinhos, colunas = medidas)
Medidas: ['alcohol', 'malic_acid', 'ash', 'alcalinity_of_ash'] ...
Vinhos por categoria: [59 71 48]
```

1.11 Do jeito manual: testando três modelos

Primeiro, façamos como o cozinheiro humano. Escolhemos três algoritmos, usamos os botões padrão e avaliamos cada um.

Para avaliar, usaremos a **validação cruzada** (*cross-validation*). A ideia: dividir os dados em partes (aqui, 5), treinar com 4 e testar na restante, repetindo 5 vezes. Assim todos os vinhos servem de teste uma vez. A média das notas é mais confiável que uma única divisão (Kohavi, 1995). No Capítulo 2 vamos aprofundar isso.

PYTHON
```python
from sklearn.datasets import load_wine
from sklearn.model_selection import cross_val_score
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.tree import DecisionTreeClassifier
```

```python
from sklearn.neighbors import KNeighborsClassifier
from sklearn.linear_model import LogisticRegression

X, y = load_wine(return_X_y=True)

candidatos = {
    "Árvore de decisão": DecisionTreeClassifier(random_state=0),
    "k-vizinhos":        make_pipeline(StandardScaler(), KNeighborsClassifier()),
    "Regressão logística": make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000)),
}

for nome, modelo in candidatos.items():
    notas = cross_val_score(modelo, X, y, cv=5, scoring="accuracy")
    print(f"{nome:20s} acurácia média = {notas.mean():.3f}  (±{notas.std():.3f})")
```

SAÍDA

```
Árvore de decisão    acurácia média = 0.888  (±0.040)
k-vizinhos           acurácia média = 0.949  (±0.038)
Regressão logística  acurácia média = 0.983  (±0.014)
```

A **acurácia** é a fração de acertos. O símbolo ± mostra o quanto a nota varia entre as 5 divisões. Note que `StandardScaler` coloca as medidas na mesma escala, pois algoritmos como k-vizinhos sofrem se uma coluna tem números enormes e outra tem números pequenos.

1.12 Do jeito automático: um mini-AutoML

Agora o ajudante incansável. Vamos escrever, em poucas linhas, **um AutoML de verdade**, ainda que simples. Ele faz o seguinte, 40 vezes:

1. **Sorteia** um algoritmo e seus hiperparâmetros (um ponto do espaço CASH);
2. **Avalia** esse candidato por validação cruzada;
3. **Guarda** o melhor encontrado.

Essa estratégia se chama **busca aleatória** (*random search*). Ela é simples, mas surpreendentemente forte, como veremos no Capítulo 4 (Bergstra; Bengio, 2012).

PYTHON

```python
import numpy as np
from sklearn.datasets import load_wine
from sklearn.model_selection import cross_val_score
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier

X, y = load_wine(return_X_y=True)
rng = np.random.default_rng(0)

def sortear_candidato():
    """Sorteia um ponto do espaço CASH: algoritmo + hiperparâmetros."""
    alg = rng.choice(["arvore", "knn", "logistica", "floresta"])
    if alg == "arvore":
```

```
        hp = {"max_depth": int(rng.integers(1, 15)), "min_samples_leaf": int(rng.integers(1, 20))}
        modelo = DecisionTreeClassifier(random_state=0, **hp)
    elif alg == "knn":
        hp = {"n_neighbors": int(rng.integers(1, 30)), "weights": str(rng.choice(["uniform",
"distance"]))}
        modelo = make_pipeline(StandardScaler(), KNeighborsClassifier(**hp))
    elif alg == "logistica":
        hp = {"C": float(10 ** rng.uniform(-3, 3))}
        modelo = make_pipeline(StandardScaler(), LogisticRegression(max_iter=2000, **hp))
    else:
        hp = {"n_estimators": int(rng.integers(10, 200)), "max_depth": int(rng.integers(2, 15))}
        modelo = RandomForestClassifier(random_state=0, **hp)
    return alg, hp, modelo

melhor = (-1, None, None)
for tentativa in range(40):
    alg, hp, modelo = sortear_candidato()
    nota = float(cross_val_score(modelo, X, y, cv=5).mean())
    if nota > melhor[0] + 1e-9:
        melhor = (nota, str(alg), hp)
        print(f"tentativa {tentativa:2d}: novo melhor = {nota:.3f}  {alg}")

nota, alg, hp = melhor
print(f"\nMelhor encontrado: {alg}, acurácia = {nota:.3f}")
print("Hiperparâmetros:", {k: round(v, 3) if isinstance(v, float) else v for k, v in hp.items()})
```

SAÍDA

```
tentativa  0: novo melhor = 0.972  floresta
tentativa  4: novo melhor = 0.978  logistica
tentativa  5: novo melhor = 0.983  logistica
tentativa 23: novo melhor = 0.983  logistica

Melhor encontrado: logistica, acurácia = 0.983
Hiperparâmetros: {'C': 0.073}
```

Parabéns! Você acabou de construir a essência de um sistema de AutoML: **um espaço de busca, um avaliador e uma estratégia de busca**. Os capítulos seguintes melhoram cada uma dessas peças.

> **Dica**
>
> Mude a semente (`default_rng(0)` para `default_rng(1)`) e rode de novo. O melhor encontrado muda? Quanto? Essa variação mostra por que sempre devemos relatar mais de uma execução.

1.13 Os três ingredientes de todo AutoML

Qualquer sistema de AutoML, simples ou sofisticado, tem os mesmos três ingredientes.

Ingrediente	Pergunta que responde	Exemplo do mini-AutoML	Onde estudar
Espaço de busca	O que posso escolher?	4 algoritmos e seus botões	Caps. 3, 4 e 11
Estratégia de busca	Como escolho o próximo candidato?	Sorteio aleatório	Caps. 4 a 7
Estimativa de desempenho	Como sei se um candidato é bom?	Validação cruzada de 5 partes	Caps. 2, 4 e 6

Esse esquema de três peças também organiza a literatura de busca de arquiteturas neurais (Elsken; Metzen; Hutter, 2019) e aparecerá de novo no Capítulo 11.

1.14 Como este livro está organizado

O livro tem **cinco partes** e **catorze capítulos**.

- **Parte I — Fundamentos** (Caps. 1 a 3): o que é AutoML, o essencial de aprendizado de máquina e pipelines confiáveis.
- **Parte II — Otimização de hiperparâmetros** (Caps. 4 a 7): busca em grade e aleatória, otimização bayesiana, Hyperband e métodos evolutivos.
- **Parte III — Automação do pipeline** (Caps. 8 a 10): atributos, meta-aprendizado, ensembles e ferramentas.
- **Parte IV — AutoML avançado** (Caps. 11 e 12): busca de arquiteturas neurais e AutoML para séries temporais, texto e imagens.
- **Parte V — AutoML responsável e em produção** (Caps. 13 e 14): explicabilidade, ética, MLOps e um projeto final.

Cada capítulo termina com **30 questões de múltipla escolha**, **20 questões discursivas**, **exercícios práticos em Python** e um **projeto integrador**. O gabarito está no Apêndice A.

Resumo do capítulo

MEMORIZE **Cartões de memorização**

- **AutoML** automatiza as escolhas na construção de modelos: preparo, atributos, algoritmo, hiperparâmetros, combinação.
- **Hiperparâmetros** são os botões de ajuste definidos *antes* do treino.
- **CASH** = escolher algoritmo **e** hiperparâmetros ao mesmo tempo (Thornton *et al.*, 2013).
- **Não existe almoço grátis:** nenhum algoritmo vence em todos os problemas (Wolpert, 1996).
- **Três ingredientes:** espaço de busca, estratégia de busca, estimativa de desempenho.
- **Limites:** AutoML não corrige dados ruins, não escolhe a métrica certa e não dispensa julgamento humano.
- **Regra de ouro:** o AutoML procura; a pessoa define o que procurar e decide.

Exercícios de múltipla escolha

Escolha **uma** alternativa em cada questão. O gabarito está no Apêndice A.

Questão 1. AutoML (aprendizado de máquina automatizado) é, essencialmente:

a) o conjunto de técnicas que automatiza as escolhas necessárias para construir um bom modelo.
b) uma linguagem de programação própria para ciência de dados.
c) um tipo específico de rede neural profunda.
d) um programa que dispensa totalmente a participação de pessoas.

Questão 2. Na analogia da cozinha, o "ajudante incansável" representa:

a) o conjunto de dados de teste.
b) o AutoML explorando muitas combinações de forma sistemática.
c) o cozinheiro que prova o prato no final.
d) o espaço de busca.

Questão 3. Chamamos de hiperparâmetro:

a) qualquer coluna da tabela de dados.
b) um valor que o modelo aprende sozinho durante o treino.
c) um "botão de ajuste" do algoritmo, definido antes do treino.
d) o rótulo que se deseja prever.

Questão 4. O que significa a sigla CASH?

a) Coleta automática de séries históricas.
b) Cálculo adaptativo de erros sistemáticos.
c) Seleção combinada de algoritmo e otimização de hiperparâmetros.
d) Classificação automática de sistemas heterogêneos.

Questão 5. Por que o problema CASH não pode ser resolvido ajustando algoritmo e hiperparâmetros em separado?

a) Porque os hiperparâmetros são iguais em todos os algoritmos.
b) Porque a validação cruzada impede o ajuste.
c) Porque a melhor escolha de algoritmo depende de quão bem cada um foi ajustado, e os botões variam conforme o algoritmo.
d) Porque os algoritmos não aceitam ajustes.

Questão 6. O sistema que tratou o CASH de forma completa pela primeira vez, usando otimização bayesiana sobre a biblioteca WEKA, foi o:

a) AutoGluon.
b) Auto-WEKA.
c) TPOT.
d) FLAML.

Questão 7. O que significa dizer que "não existe almoço grátis" (No Free Lunch) em aprendizado de máquina?

a) Todo algoritmo exige pagamento de licença.
b) Algoritmos simples são sempre melhores.
c) Nenhum algoritmo é o melhor para todos os problemas.
d) Algoritmos complexos sempre vencem os simples.

Questão 8. Qual é a consequência prática do teorema do "almoço grátis não existe" para o AutoML?

a) Só devemos usar redes neurais.
b) É preciso testar vários algoritmos nos dados em questão, e não apenas supor qual é o melhor.
c) Basta escolher sempre o algoritmo mais recente.
d) O AutoML se torna desnecessário.

Questão 9. Em 1976, John Rice formulou o problema:

a) da maldição da dimensionalidade.
b) do viés de confirmação.
c) da seleção de algoritmos.
d) da validação cruzada.

Questão 10. O espaço de busca do AutoML é:

a) o conjunto de todas as combinações de escolhas (algoritmos, hiperparâmetros etc.) que o sistema pode testar.
b) o espaço físico dos servidores.
c) a memória do computador usada no treino.
d) a tabela de dados de treino.

Questão 11. Quais são os três ingredientes de qualquer sistema de AutoML apresentados no capítulo?

a) Dados, rótulos e métricas.
b) Treino, teste e produção.
c) Hardware, software e rede.
d) Espaço de busca, estratégia de busca e estimativa de desempenho.

Questão 12. No mini-AutoML do capítulo, qual era a estratégia de busca?

a) Sorteio aleatório de candidatos (busca aleatória).
b) Otimização bayesiana.
c) Busca em grade.
d) Programação genética.

Questão 13. No mini-AutoML, a estimativa de desempenho de cada candidato foi feita por:

a) avaliação por especialistas humanos.
b) validação cruzada de 5 partes.
c) uma única divisão treino/teste fixa.
d) sorteio do resultado.

Questão 14. Pipeline, em aprendizado de máquina, é:

a) um cano físico entre servidores.
b) o conjunto de dados de teste.
c) um tipo de rede neural.
d) uma sequência de etapas que transforma dados brutos em previsões.

Questão 15. Qual das etapas abaixo ainda depende fortemente de pessoas, segundo o capítulo?

a) A coleta e a limpeza dos dados.
b) A busca em grade.
c) A combinação de modelos.
d) O ajuste de hiperparâmetros.

Questão 16. Em qual nível de automação (escala didática do capítulo) as ferramentas atuais costumam estar?

a) Nível 0.
b) Nível 5.
c) Nível 4 sempre.
d) Entre os níveis 2 e 3.

Questão 17. O AutoML "corrige" automaticamente dados com erros e vieses?

a) Sim, quando usa validação cruzada.
b) Sim, mas só em dados tabulares.
c) Sim, sempre.
d) Não: dados ruins produzem resultados ruins, apenas mais bem ajustados.

Questão 18. Por que "mais busca" nem sempre é melhor?

a) Porque computadores se cansam.
b) Porque a busca nunca melhora o resultado.
c) Porque a busca aleatória é proibida.
d) Porque buscar demais pode ajustar o modelo ao acaso dos dados de validação.

Questão 19. Quem formulou, em 2010, o alerta de que o excesso de ajuste na seleção de modelos gera viés na avaliação de desempenho?

a) Wolpert.
b) Rice.
c) Cawley e Talbot.
d) Olson e Moore.

Questão 20. O TPOT, citado no capítulo, usa qual técnica para montar pipelines?

a) Aprendizado por reforço profundo.
b) Busca em grade exaustiva.
c) Otimização bayesiana com processos gaussianos.
d) Programação genética, inspirada na evolução.

Questão 21. O Auto-sklearn introduziu, no AutoML em Python, duas ideias importantes. Quais?

a) Busca em grade e validação simples.
b) Redes adversariais e difusão.
c) Quantização e poda de redes.
d) Meta-aprendizado e combinação automática de modelos.

Questão 22. Hyperband e BOHB se destacaram por:

a) acelerar a busca abandonando cedo as tentativas ruins.
b) eliminar a necessidade de dados de validação.
c) usar apenas busca em grade.
d) projetar arquiteturas de redes neurais.

Questão 23. Qual projeto do capítulo ofereceu um benchmark aberto para comparar ferramentas de AutoML?

a) FLAML.
b) DARTS.
c) Optuna.
d) AMLB.

Questão 24. Qual é a principal função da validação cruzada no mini-AutoML?

a) Aumentar o número de colunas.
b) Obter uma estimativa de desempenho mais confiável do que uma única divisão.
c) Embaralhar os rótulos.
d) Treinar o modelo mais rápido.

Questão 25. Qual alternativa descreve melhor o papel da pessoa em um projeto com AutoML?

a) Nenhum, pois o sistema decide tudo.
b) Apenas escolher cores dos gráficos.
c) Formular o problema, escolher a métrica, cuidar da ética e decidir se confia no resultado.
d) Apenas apertar um botão.

Questão 26. Se você usar a métrica errada, o AutoML:

a) usará automaticamente a acurácia.
b) otimizará a métrica errada com grande eficiência.
c) deixará de funcionar.
d) detectará o erro e trocará a métrica.

Questão 27. No conjunto Wine usado no capítulo, qual é a tarefa?

a) Classificar a categoria do vinho a partir de 13 medidas químicas.
b) Prever o preço do vinho.
c) Agrupar vinhos sem usar rótulos.
d) Detectar vinhos falsificados.

Questão 28. Para que serve o StandardScaler usado antes do k-vizinhos?

a) Para aumentar o número de vizinhos.
b) Para colocar as colunas na mesma escala.
c) Para remover os rótulos.
d) Para escolher o algoritmo.

Questão 29. Qual afirmação sobre o AutoML é um mito, segundo o capítulo?

a) "O AutoML muda o trabalho do especialista."
b) "O AutoML tem custo de tempo e energia."
c) "O AutoML sempre vence um modelo feito à mão."
d) "O AutoML registra cada tentativa da busca."

Questão 30. Qual das opções NÃO é uma razão para a existência do AutoML?

a) O espaço de busca é gigantesco.
b) Faltam especialistas em aprendizado de máquina.
c) Algoritmos de aprendizado de máquina nunca precisam de ajustes.
d) A busca automática registra tentativas, favorecendo a reprodutibilidade.

Exercícios discursivos

Responda com suas palavras, em poucas linhas. Compare depois com a resposta-modelo do Apêndice A.

1. Explique com suas palavras, usando uma analogia do cotidiano diferente da cozinha, o que é AutoML.
2. Defina hiperparâmetro e dê dois exemplos de algoritmos diferentes com seus hiperparâmetros.
3. O que é o problema CASH e por que ele é considerado difícil?
4. Explique o significado do teorema "No Free Lunch" e sua relação com a necessidade do AutoML.
5. Descreva as sete etapas da linha de montagem do aprendizado de máquina apresentadas no capítulo e indique quais o AutoML mais automatiza.
6. Por que o AutoML não elimina a necessidade de especialistas? Cite pelo menos três tarefas que continuam humanas.
7. Quais são os três ingredientes de qualquer sistema de AutoML? Identifique-os no mini-AutoML do capítulo.
8. Explique por que dados ruins produzem modelos ruins mesmo com AutoML. Dê um exemplo.
9. O que é overfitting da busca e como ele pode ocorrer em um AutoML que testa milhares de configurações?
10. Compare a abordagem manual e a automática para escolher um algoritmo, destacando vantagens e limitações de cada uma.
11. Resuma a contribuição do Auto-WEKA e do Auto-sklearn para o campo do AutoML.
12. Explique a ideia central do TPOT e por que a evolução é uma metáfora útil para a busca de pipelines.
13. Por que métodos como Hyperband economizam tempo em relação a treinar todos os candidatos até o fim?
14. Cite três custos (além de dinheiro) associados ao uso intensivo de AutoML e comente um deles.
15. O que é validação cruzada e por que ela foi usada no mini-AutoML em vez de uma única divisão treino/teste?
16. Interprete a fórmula do CASH em palavras, explicando o papel da média sobre k divisões.
17. Por que fixar a semente aleatória e relatar mais de uma execução é uma boa prática em AutoML?
18. Discuta o mito "o AutoML sempre vence modelos feitos à mão". Em que situações isso pode não valer?
19. Proponha um problema do seu cotidiano ou trabalho em que o AutoML poderia ajudar. Indique o que a pessoa ainda precisaria decidir.
20. Por que o capítulo afirma que o nível 5 de automação "ainda não existe na prática"? Que tarefas ainda exigem pessoas?

Exercícios práticos em Python (scikit-learn)

Resolva os exercícios no seu computador (ou no Google Colab). Eles usam conjuntos de dados da UCI que já vêm no scikit-learn ou dados simulados. Tente resolver **antes** de consultar a solução, no Apêndice D. As saídas apresentadas lá foram obtidas com as versões de bibliotecas listadas na Apresentação.

Exercício 1 — Por que precisamos de uma linha de base

NÍVEL: BÁSICO

Carregue o conjunto **Breast Cancer Wisconsin (Diagnostic)** (`load_breast_cancer`), do repositório UCI, que já vem no scikit-learn. Avalie, por validação cruzada de 5 partes, dois modelos: um `DummyClassifier(strategy="most_frequent")`, que sempre responde a classe mais comum, e uma `LogisticRegression` com `StandardScaler`.

Pergunta: qual é a acurácia do modelo "bobo"? Por que ela é útil como ponto de comparação?

Exercício 2 — Ampliando o espaço de busca

NÍVEL: INTERMEDIÁRIO

Reescreva o mini-AutoML do capítulo para o conjunto **Wine**, acrescentando mais dois algoritmos ao espaço de busca: **SVM com kernel RBF** (`SVC`, hiperparâmetros `C` e `gamma`, ambos em escala logarítmica) e **Gradient Boosting** (`GradientBoostingClassifier`, hiperparâmetros `n_estimators` e `learning_rate`). Faça 60 tentativas e mostre os **três melhores candidatos**.

Pergunta: o vencedor mudou em relação ao capítulo?

Exercício 3 — O efeito da semente

NÍVEL: INTERMEDIÁRIO

Rode o mini-AutoML do Exercício 2 com **5 sementes diferentes** (0 a 4), usando 30 tentativas em cada uma. Para cada semente, registre a **melhor nota** encontrada. Calcule a média e o desvio padrão das 5 melhores notas.

Pergunta: por que relatar apenas uma execução pode enganar?

Exercício 4 — Contando o tamanho do espaço de busca

NÍVEL: BÁSICO

Um espaço de busca tem três hiperparâmetros: `max_depth` com 10 valores possíveis, `min_samples_leaf` com 8 valores e `criterion` com 2 valores. (a) Quantas combinações existem? (b) Se cada avaliação por validação cruzada levar 3 segundos, quanto tempo levaria testar todas? (c) Quanto tempo se acrescentarmos um quarto hiperparâmetro com 10 valores?

Pergunta: o que isso sugere sobre testar tudo à mão?

> SAIBA MAIS **Para ir além: bases públicas da UCI e do Kaggle**
>
> - **UCI:** Wine Quality (vinhos portugueses, com notas de qualidade) e Adult (renda de adultos, com atributos categóricos).
> - **Kaggle:** Titanic (sobreviventes, com dados faltantes e categóricos).
>
> Estes conjuntos não foram usados nem testados nos códigos do livro. Confira, nos sites, a descrição, a licença e as colunas antes de usá-los.

Projeto integrador do capítulo

Orçamento de busca e honestidade na avaliação

Fonte dos dados. *Breast Cancer Wisconsin (Diagnostic)*, do repositório **UCI Machine Learning**, incluído no scikit-learn.

Contexto. Uma equipe quer saber **quanto vale a pena buscar**: dobrar o número de tentativas do mini-AutoML melhora, de fato, o resultado em dados novos?

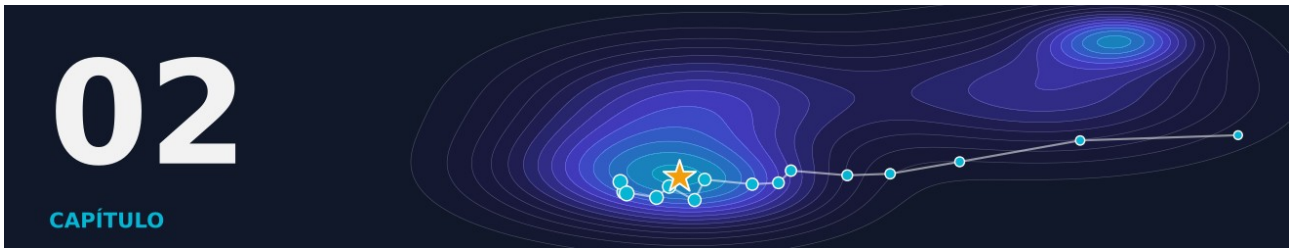
Atenção ética. Os dados são de uso didático. O modelo que você construir **não é diagnóstico médico**.

Tarefas.

1. Separe 25% dos dados como **teste final** (`train_test_split`, com `stratify=y` e `random_state=0`). Não toque neles até o fim.
2. Use o mini-AutoML do capítulo (algoritmos à sua escolha) para buscar, **apenas no conjunto de treino**, com orçamentos de **5, 10, 20, 40 e 80 tentativas**. Repita cada orçamento com 5 sementes.
3. Para cada orçamento, registre a **melhor nota de validação cruzada** (média das 5 sementes).
4. Treine novamente o melhor candidato de cada busca no treino completo e meça a acurácia no **teste final**. Registre a média das 5 sementes.
5. Desenhe um gráfico com duas curvas (validação e teste) em função do orçamento.
6. Escreva um parágrafo (5 a 8 linhas): o que o gráfico mostra? A nota de validação é uma estimativa honesta do desempenho final? Por quê?

Entregáveis. Um notebook (ou script) comentado, o gráfico e o parágrafo de conclusões.

A solução de referência está no Apêndice E.



Capítulo 2 — Aprendizado de máquina em um capítulo

OBJETIVOS DE APRENDIZAGEM

Ao final deste capítulo você será capaz de:

- Identificar exemplos, atributos e alvo em uma tabela de dados e distinguir classificação de regressão.
- Descrever, com analogias, o funcionamento de árvores, k-vizinhos, regressão logística e florestas.
- Diferenciar parâmetros de hiperparâmetros.
- Reconhecer subajuste e sobreajuste e explicar a curva de erro de treino e de validação.
- Usar conjuntos de treino, validação e teste, e a validação cruzada, de forma correta.
- Escolher métricas adequadas e entender por que a acurácia pode enganar.
- Explicar por que otimizar demais sobre a validação cria notas otimistas, um risco central do AutoML.

2.1 Por que um capítulo de revisão?

O AutoML automatiza decisões sobre modelos. Para entender o que ele decide, você precisa conhecer o **vocabulário básico** do aprendizado de máquina. Este capítulo é um "curso intensivo". Se você já tem experiência, aproveite para revisar. Se está começando, vá com calma: tudo será explicado com exemplos do cotidiano.

Imagine que você está aprendendo a **dirigir**. Antes de usar o piloto automático, você precisa saber o que são volante, freio e acelerador. Aqui, o volante e os pedais são: dados, algoritmos, hiperparâmetros, validação e métricas.

2.2 Os dados: exemplos, atributos e alvo

A maior parte dos problemas de AutoML usa **dados em tabela** (dados tabulares). Veja um exemplo: uma loja quer prever se um cliente vai comprar.

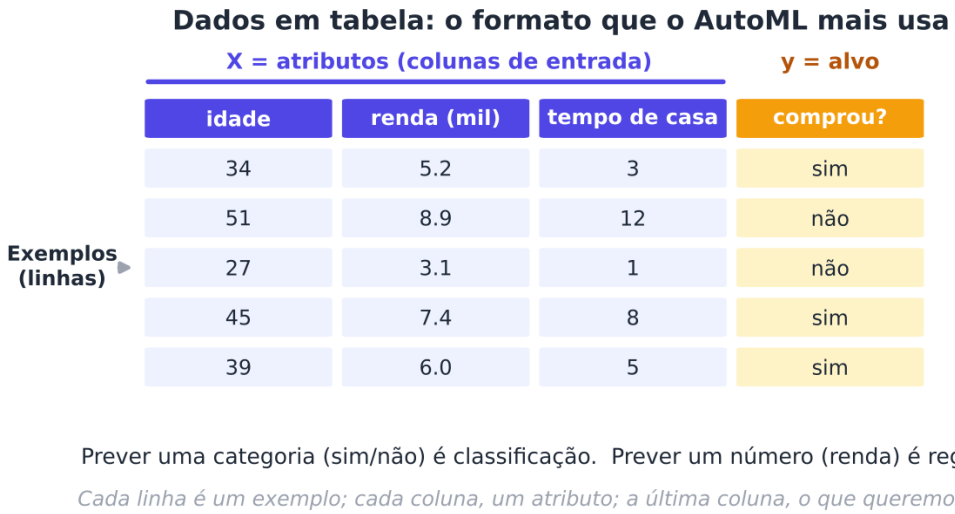


Figura 2.1 — Uma tabela de dados. Cada linha é um exemplo, cada coluna de entrada é um atributo e a última coluna é o alvo.

Vamos fixar o vocabulário:

- **Exemplo** (ou instância, amostra): uma linha da tabela. Aqui, um cliente.
- **Atributo** (ou *feature*, característica): uma coluna de entrada. Aqui, idade, renda e tempo de casa.
- **X**: o conjunto de todos os atributos, ou seja, a tabela de entrada.
- **Alvo** (ou rótulo, *label*, **y**): o que queremos prever. Aqui, "comprou?".

Há dois grandes tipos de tarefa:

- **Classificação:** prever uma **categoria** (sim ou não, spam ou não spam, tipo de vinho).
- **Regressão:** prever um **número** (preço de um imóvel, temperatura de amanhã).

Quando temos o alvo conhecido para os exemplos de treino, dizemos que o aprendizado é **supervisionado**: a máquina aprende com respostas corretas, como um aluno que estuda com o gabarito.

> **No seu dia a dia**
>
> O corretor de e-mails que separa spam é um classificador. O aplicativo que estima o preço da sua corrida antes de você pedir o carro é um regressor.

2.3 Quatro algoritmos clássicos

Um **algoritmo** de aprendizado é uma receita que transforma dados em um **modelo**. Conheça quatro receitas que aparecerão ao longo do livro (Hastie; Tibshirani; Friedman, 2009).

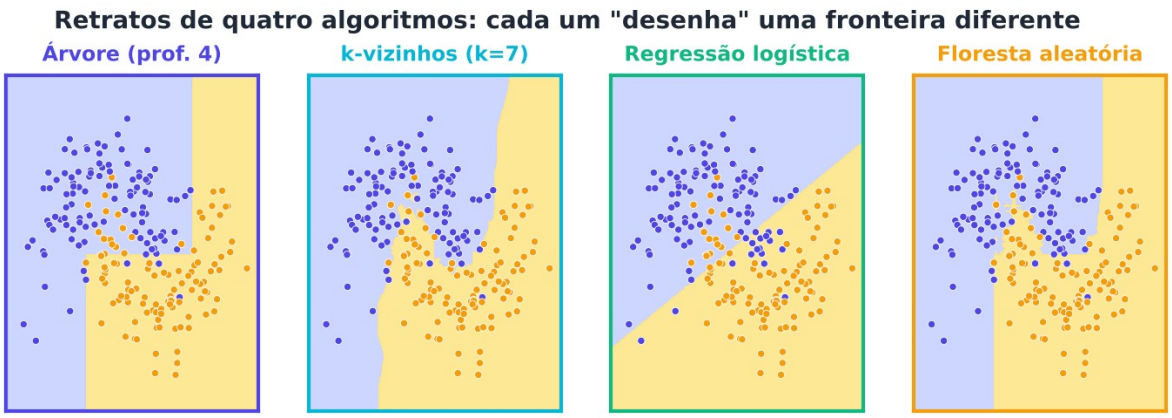


Figura 2.2 — Os quatro algoritmos aprendem a separar as mesmas duas "luas". Cada um desenha uma fronteira com estilo próprio.

Árvore de decisão. Faz uma série de perguntas de sim ou não, como no jogo "20 perguntas". "A renda é maior que 5? Se sim, o tempo de casa é maior que 3?" O resultado é uma fronteira em "degraus". É fácil de explicar, mas pode decorar demais os dados.

k-vizinhos (k-NN). Para classificar um novo exemplo, olha os *k* exemplos mais parecidos e adota a categoria mais frequente entre eles. É como pedir a opinião dos *k* vizinhos de rua. Depende muito da **escala** dos atributos.

Regressão logística. Apesar do nome, é um classificador. Traça uma fronteira reta (um "plano") entre as classes e converte a distância até ela em uma probabilidade. É simples, rápida e muito robusta.

Floresta aleatória (*random forest*). Treina **centenas de árvores**, cada uma com uma amostra diferente dos dados e dos atributos, e deixa todas votarem (Breiman, 2001). É como consultar um comitê de especialistas, e não um único especialista. Costuma ser muito boa sem muito ajuste.

Existe ainda a família do **gradient boosting**, que constrói árvores em sequência, cada uma corrigindo os erros da anterior (Friedman, 2001). Implementações como XGBoost e LightGBM tornaram-se campeãs em dados tabulares (Chen; Guestrin, 2016; Ke *et al.*, 2017). Você as verá muito dentro de ferramentas de AutoML.

> **Memorize**
>
> Não existe algoritmo vencedor em tudo (lembra do almoço grátis?). Árvores são fáceis de explicar. k-NN depende da escala. Regressão logística é simples e robusta. Florestas e *boosting* costumam ir muito bem em tabelas.

2.4 Parâmetros e hiperparâmetros

Esta é a distinção **mais importante** do capítulo para o AutoML.

- **Parâmetros** são valores que o modelo **aprende sozinho** durante o treino. Em uma árvore, são as perguntas escolhidas e os limites de corte.
- **Hiperparâmetros** são valores que **você define antes do treino**. Em uma árvore, são a profundidade máxima ou o número mínimo de exemplos por folha.

Analogia: num forno, os *parâmetros* são o resultado do bolo (que depende da massa), e os *hiperparâmetros* são a temperatura e o tempo que você programa antes de assar.

```python
from sklearn.datasets import load_breast_cancer
from sklearn.tree import DecisionTreeClassifier
```

```
dados = load_breast_cancer()
X, y = dados.data, dados.target

# Hiperparâmetro: escolhido por nós ANTES do treino
arvore = DecisionTreeClassifier(max_depth=3, random_state=0)
arvore.fit(X, y)

# Parâmetros: APRENDIDOS pelo modelo durante o treino
raiz = arvore.tree_.feature[0]
print("Hiperparâmetro max_depth =", arvore.get_params()["max_depth"])
print("Nº de nós aprendidos     =", arvore.tree_.node_count)
print("Pergunta da raiz         =", dados.feature_names[raiz], "<=", round(arvore.tree_.threshold[0],
2))
```

SAÍDA

```
Hiperparâmetro max_depth = 3
Nº de nós aprendidos     = 15
Pergunta da raiz         = worst radius <= 16.8
```

O trabalho do AutoML é, em grande parte, **escolher bons hiperparâmetros** (e bons algoritmos). Já os parâmetros, o próprio algoritmo aprende.

2.5 Aprender é errar menos

Como o modelo "aprende"? Ele ajusta seus parâmetros para **reduzir o erro** nos exemplos de treino. Para medir o erro, usamos uma **função de perda** (*loss*): um número que diz o quanto o modelo errou. Quanto menor, melhor.

Mas há uma armadilha. Um modelo pode errar **zero** no treino simplesmente por **decorar** os exemplos, como um aluno que decora as respostas do simulado sem entender a matéria. Na prova de verdade, ele vai mal.



Figura 2.3 — Três modelos ajustados aos mesmos pontos. O da esquerda é simples demais; o da direita é complexo demais e persegue o ruído.

Dois nomes que você precisa conhecer:

- **Subajuste** (*underfitting*): o modelo é simples demais e erra até no treino. É como tentar resumir um romance em uma palavra.
- **Sobreajuste** (*overfitting*): o modelo é complexo demais, decora o treino (incluindo o ruído) e erra em dados novos.

O que queremos é a **generalização**: ir bem em dados que o modelo **nunca viu**. A complexidade do modelo, controlada por hiperparâmetros, é o botão que equilibra tudo isso.

PYTHON

```python
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import validation_curve
from sklearn.tree import DecisionTreeClassifier

X, y = load_breast_cancer(return_X_y=True)
profundidades = [1, 2, 3, 5, 8, 12]
treino, valid = validation_curve(DecisionTreeClassifier(random_state=0), X, y,
                                 param_name="max_depth", param_range=profundidades, cv=5)

print("profundidade | acurácia no treino | acurácia na validação")
for p, t, v in zip(profundidades, treino.mean(axis=1), valid.mean(axis=1)):
    print(f"{p:12d} | {t:18.3f} | {v:21.3f}")
```

SAÍDA

```
profundidade | acurácia no treino | acurácia na validação
           1 |              0.924 |                 0.900
           2 |              0.952 |                 0.928
           3 |              0.974 |                 0.917
           5 |              0.991 |                 0.916
           8 |              1.000 |                 0.916
          12 |              1.000 |                 0.917
```

Observe: a acurácia no treino **sobe sempre**, chegando perto de 100%. A da validação sobe no início e depois **para de melhorar** (e pode até piorar). A distância entre as duas é a medida do sobreajuste.

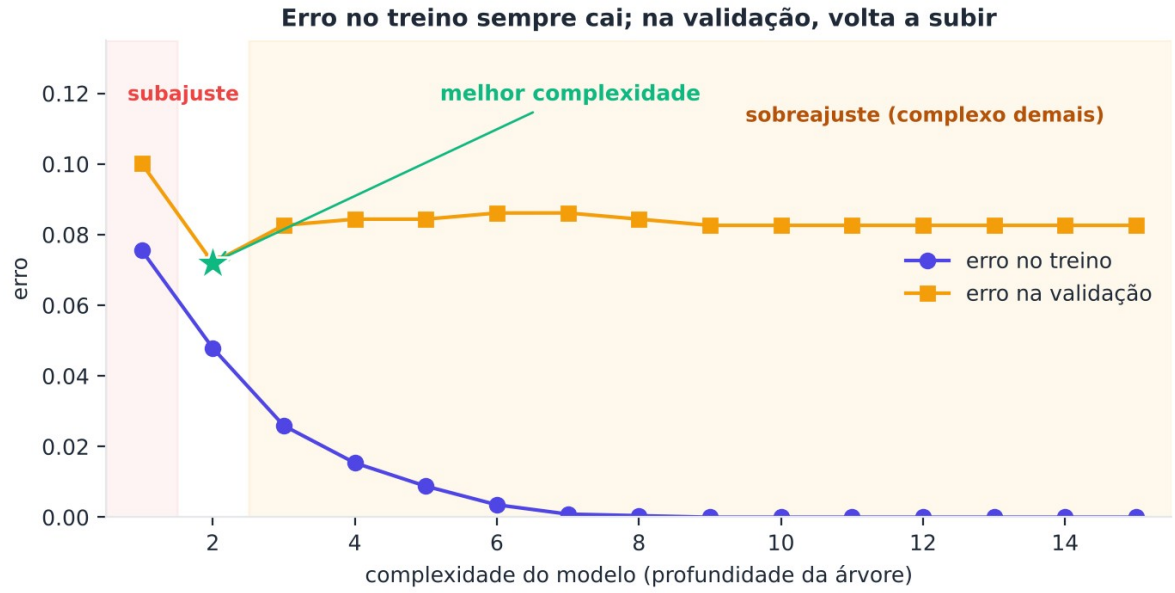


Figura 2.4 — O erro de treino só cai. O erro de validação indica a complexidade adequada. Essa figura resume o que todo AutoML tenta encontrar.

2.6 Treino, validação e teste

Para medir a generalização, **separamos os dados em três conjuntos** com papéis diferentes. Pense em um estudante.

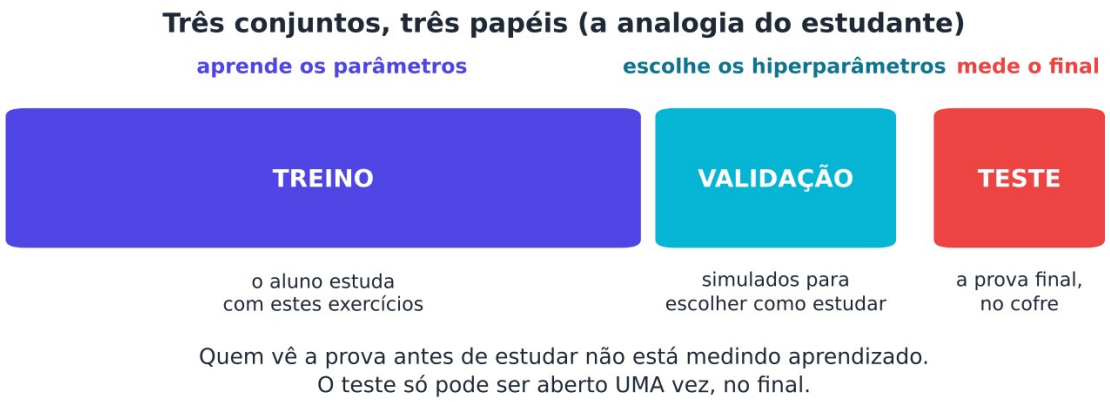


Figura 2.5 — Treino para aprender, validação para escolher, teste para medir no final. O teste fica "no cofre" até a última hora.

- **Treino:** os exercícios com que o modelo aprende seus parâmetros.
- **Validação:** os "simulados" usados para **escolher hiperparâmetros** e comparar modelos. É aqui que o AutoML trabalha.
- **Teste:** a "prova final". Só é aberta **uma vez**, no fim, para medir o desempenho real.

```python
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split

X, y = load_breast_cancer(return_X_y=True)

# 1) Separa o TESTE (cofre): 20% dos dados
X_resto, X_teste, y_resto, y_teste = train_test_split(X, y, test_size=0.20, stratify=y, random_state=0)
# 2) Do que sobrou, separa a VALIDAÇÃO: 25% do resto (= 20% do total)
X_treino, X_valid, y_treino, y_valid = train_test_split(X_resto, y_resto, test_size=0.25,
stratify=y_resto, random_state=0)

print("treino:", len(X_treino), "| validação:", len(X_valid), "| teste:", len(X_teste))
```

```
treino: 341 | validação: 114 | teste: 114
```

O argumento `stratify=y` mantém a **mesma proporção de classes** em cada parte. O `random_state` fixa o sorteio, para que o resultado seja repetível.

> **Atenção**
>
> **Nunca** use o teste para escolher nada: nem hiperparâmetros, nem algoritmo, nem atributos. No instante em que o teste influencia uma decisão, ele deixa de ser teste e passa a ser validação, e você perde a medida honesta do desempenho.

2.7 Validação cruzada

Dividir uma única vez tem um defeito: o resultado depende da **sorte do sorteio**. Se a validação tiver, por acaso, exemplos "fáceis", a nota vai parecer ótima.

A solução é a **validação cruzada** (*cross-validation*), representada abaixo (Kohavi, 1995).



Figura 2.6 — Na validação cruzada de 5 partes, o modelo é treinado e avaliado 5 vezes. A nota final é a média.

Em *k* partes (*k-fold*), repetimos *k* vezes: treinamos com $k - 1$ partes e validamos na restante. A média das *k* notas é mais estável que uma nota única, e o desvio padrão mostra o quanto ela varia.

```python
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import cross_val_score, StratifiedKFold
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

X, y = load_breast_cancer(return_X_y=True)
modelo = make_pipeline(StandardScaler(), LogisticRegression(max_iter=2000))

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=0)
notas = cross_val_score(modelo, X, y, cv=cv, scoring="accuracy")
print("Notas das 5 rodadas:", notas.round(3))
print(f"Média = {notas.mean():.3f} | desvio padrão = {notas.std():.3f}")
```

```
Notas das 5 rodadas: [0.956 0.974 0.982 1.    0.982]
Média = 0.979 | desvio padrão = 0.014
```

`StratifiedKFold` mantém a proporção de classes em cada parte. Em geral, usamos $k = 5$ ou 10. Quanto maior o *k*, mais treino por rodada, porém mais tempo gasto.

2.8 Métricas: como medir o acerto

Uma **métrica** é a régua que mede o desempenho. **A escolha da métrica define o que o AutoML vai otimizar**, portanto é uma decisão de enorme responsabilidade.

2.8.1 Métricas de classificação

A **matriz de confusão** conta os acertos e erros de cada tipo.

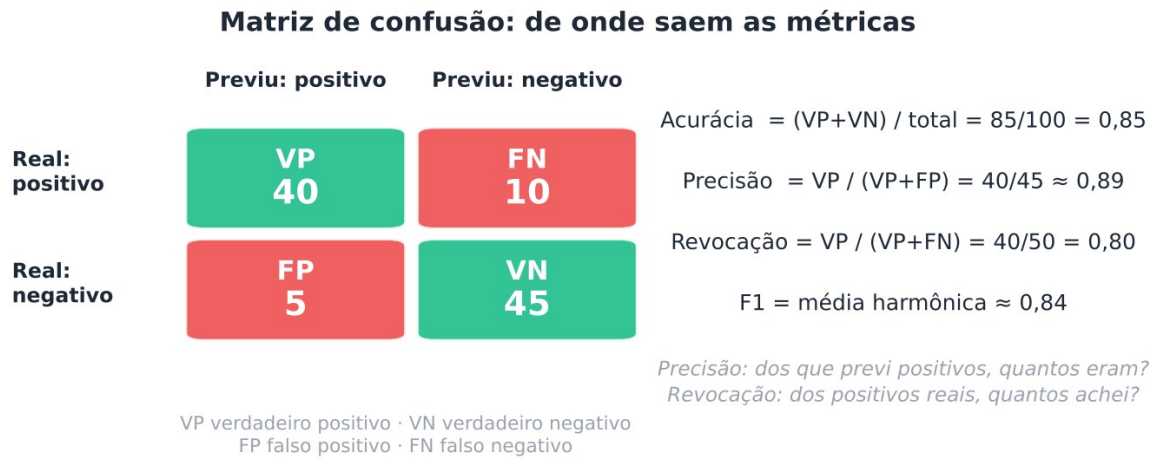


Figura 2.7 — A matriz de confusão e as métricas que dela derivam.

- **Acurácia:** fração de acertos. Simples, mas pode enganar.
- **Precisão:** entre os que o modelo chamou de positivos, quantos eram de fato?
- **Revocação** (*recall*): entre os positivos reais, quantos o modelo encontrou?
- **F1:** uma média que equilibra precisão e revocação.
- **AUC-ROC:** mede quão bem o modelo **ordena** positivos acima de negativos, para qualquer limiar de decisão.

A armadilha da acurácia. Imagine uma doença rara, que afeta 5% das pessoas. Um modelo que responde sempre "saudável" acerta **95%**. Parece ótimo, mas não encontra nenhum doente! Veja na prática.

PYTHON

```python
from sklearn.datasets import make_classification
from sklearn.dummy import DummyClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, recall_score, f1_score, roc_auc_score

# Dados simulados: apenas 5% de casos positivos (doença rara)
X, y = make_classification(n_samples=4000, n_features=10, n_informative=4, weights=[0.95, 0.05],
class_sep=1.5, flip_y=0.0, random_state=0)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, stratify=y, random_state=0)

bobo = DummyClassifier(strategy="most_frequent").fit(Xtr, ytr)
logis = LogisticRegression(max_iter=1000, class_weight="balanced").fit(Xtr, ytr)

for nome, m in [("Sempre 'saudável'", bobo), ("Regressão logística", logis)]:
    p = m.predict(Xte)
    prob = m.predict_proba(Xte)[:, 1]
    print(f"{nome:20s} acurácia={accuracy_score(yte, p):.3f}  revocação={recall_score(yte, p):.3f}
F1={f1_score(yte, p):.3f}  AUC={roc_auc_score(yte, prob):.3f}")
```

SAÍDA

```
Sempre 'saudável'    acurácia=0.950  revocação=0.000  F1=0.000  AUC=0.500
Regressão logística  acurácia=0.763  revocação=0.650  F1=0.215  AUC=0.803
```

A acurácia do modelo "bobo" é altíssima (95%), mas a revocação é zero: ele não encontra nenhum caso raro. A regressão logística com `class_weight="balanced"` tem acurácia **menor**, porém encontra cerca de dois terços dos casos raros e tem AUC bem acima de 0,5. Para o problema real, ela é muito mais útil. Em problemas desbalanceados, prefira F1, revocação, precisão ou AUC, conforme o custo de cada tipo de erro.

> **Exemplo: qual erro custa mais?**
>
> Em um exame de triagem, deixar passar um doente (falso negativo) é mais grave do que chamar um saudável para um exame extra (falso positivo). Então valorizamos a **revocação**. Em um filtro de spam, mandar um e-mail importante para a lixeira (falso positivo) é pior. Então valorizamos a **precisão**.

2.8.2 Métricas de regressão

Em regressão, comparamos o número previsto com o real.

- **MAE** (erro absoluto médio): a média do tamanho dos erros, na mesma unidade do alvo.
- **RMSE** (raiz do erro quadrático médio): parecida com o MAE, mas pune mais os erros grandes.
- **R^2**: a fração da variação dos dados que o modelo explica. 1 é perfeito; 0 equivale a prever sempre a média.

PYTHON

```python
import numpy as np
from sklearn.datasets import load_diabetes
from sklearn.dummy import DummyRegressor
from sklearn.linear_model import Ridge
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

X, y = load_diabetes(return_X_y=True)   # progressão de diabetes em 442 pacientes
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.25, random_state=0)

for nome, m in [("Sempre a média", DummyRegressor()), ("Regressão Ridge", Ridge(alpha=1.0))]:
    p = m.fit(Xtr, ytr).predict(Xte)
    rmse = np.sqrt(mean_squared_error(yte, p))
    print(f"{nome:16s} MAE={mean_absolute_error(yte, p):6.1f}  RMSE={rmse:6.1f}  R²={r2_score(yte,
p):6.3f}")
```

SAÍDA

```
Sempre a média   MAE=  58.3  RMSE=  70.5  R²=-0.000
Regressão Ridge  MAE=  44.9  RMSE=  56.5  R²= 0.357
```

2.9 Linha de base: o ponto de comparação

Antes de qualquer modelo sofisticado, calcule uma **linha de base** (*baseline*): o desempenho de uma regra trivial, como "prever sempre a classe mais comum" (ou "prever sempre a média"). Se o seu modelo não supera essa linha com folga, ele não aprendeu nada útil.

No scikit-learn, use `DummyClassifier` e `DummyRegressor`, como acima. **Todo projeto de AutoML deve registrar a linha de base**, pois ela dá sentido à nota final.

2.10 O perigo escondido: o "efeito do vencedor"

Chegamos ao ponto em que este capítulo conversa diretamente com o AutoML. Um sistema de AutoML testa **centenas ou milhares** de modelos e fica com o que teve a **melhor nota de validação**. Isso cria um viés sutil.

Analogia: 200 pessoas jogam uma moeda 10 vezes, só por diversão. Alguém vai acertar "cara" 9 vezes. Aquela pessoa é uma ótima adivinhadora? Não: **entre muitas tentativas, alguma sempre parece boa por pura sorte**.

Com modelos acontece o mesmo. Vamos provar com um experimento. Criamos dados **completamente aleatórios**: o alvo é sorteado e **não tem nenhuma relação** com os atributos. Qualquer modelo honesto deve acertar em torno de 50%.

PYTHON

```python
import numpy as np
from sklearn.model_selection import cross_val_score
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier

rng = np.random.default_rng(0)
# Dados SEM nenhum sinal: o alvo é puro sorteio
X = rng.normal(size=(120, 15)); y = rng.integers(0, 2, 120)
X_novo = rng.normal(size=(5000, 15)); y_novo = rng.integers(0, 2, 5000)

melhor_nota, melhor_modelo = 0, None
for _ in range(300):                        # 300 candidatos aleatórios
    if rng.random() < 0.5:
        m = KNeighborsClassifier(n_neighbors=int(rng.integers(1, 40)))
    else:
        m = DecisionTreeClassifier(max_depth=int(rng.integers(1, 10)), random_state=0)
    nota = cross_val_score(m, X, y, cv=5).mean()
    if nota > melhor_nota:
        melhor_nota, melhor_modelo = nota, m

melhor_modelo.fit(X, y)
print(f"Melhor nota de validação entre 300 modelos: {melhor_nota:.3f}")
print(f"Acurácia do mesmo modelo em dados NOVOS     : {melhor_modelo.score(X_novo, y_novo):.3f}")
```

SAÍDA

```
Melhor nota de validação entre 300 modelos: 0.667
Acurácia do mesmo modelo em dados NOVOS     : 0.503
```

A melhor nota de validação fica **bem acima de 50%**, mesmo sem nenhuma informação nos dados! Mas, em dados novos, o desempenho cai de volta para perto do acaso. A nota de validação do vencedor foi **inflada pela seleção**.

Esse fenômeno é conhecido na literatura como *over-fitting* na seleção de modelos, e seu efeito pode ter o mesmo tamanho das diferenças entre algoritmos (Cawley; Talbot, 2010). Também foi documentado que a validação cruzada usada para escolher parâmetros produz estimativas otimistas do erro (Varma; Simon, 2006).

> **Memorize**
>
> **Quanto mais modelos você testa, mais otimista fica a nota do melhor.** A cura é guardar um **teste final intocado** (ou usar validação aninhada, no Capítulo 4) e relatar o desempenho nele.

2.11 Boas práticas para levar daqui

1. **Fixe sementes aleatórias** (`random_state`) para repetir os resultados.
2. **Estratifique** as divisões em problemas de classificação.
3. **Separe o teste primeiro** e não o toque até o fim.
4. **Calcule uma linha de base** antes de modelos sofisticados.
5. **Escolha a métrica conforme o custo dos erros**, e não por hábito.
6. **Relate média e desvio padrão** de validação cruzada, e não uma nota única.

Resumo do capítulo

MEMORIZE **Cartões de memorização**

- **Exemplo** = linha; **atributo** = coluna de entrada; **alvo** = o que prever.
- **Classificação** prevê categorias; **regressão** prevê números.
- **Parâmetros** o modelo aprende; **hiperparâmetros** você define antes.
- **Subajuste:** simples demais. **Sobreajuste:** decorou o treino.
- **Treino** aprende, **validação** escolhe, **teste** mede no final (uma vez).
- **Validação cruzada** dá uma nota mais estável que uma divisão única.
- **Acurácia engana** em dados desbalanceados: prefira F1, revocação, precisão ou AUC.
- **Linha de base** dá sentido à nota.
- **Efeito do vencedor:** entre muitos modelos, o melhor parece bom por sorte.

Exercícios de múltipla escolha

Escolha **uma** alternativa em cada questão. O gabarito está no Apêndice A.

Questão 1. Em uma tabela de dados, cada linha representa normalmente:

a) um exemplo (instância).
b) um atributo.
c) um algoritmo.
d) um hiperparâmetro.

Questão 2. Em aprendizado supervisionado, o "alvo" (y) é:

a) a tabela de atributos.
b) o número de exemplos.
c) o valor que se deseja prever.
d) o conjunto de hiperparâmetros.

Questão 3. Prever o preço de um imóvel é um problema de:

a) agrupamento.
b) detecção de duplicatas.
c) regressão.
d) classificação.

Questão 4. Decidir se um e-mail é spam ou não spam é um problema de:

a) classificação.
b) otimização contínua.
c) redução de dimensionalidade.
d) regressão.

Questão 5. Qual algoritmo funciona fazendo uma série de perguntas de sim ou não, como no jogo "20 perguntas"?

a) Regressão logística.
b) k-means.
c) Árvore de decisão.
d) k-vizinhos.

Questão 6. O algoritmo k-vizinhos classifica um novo exemplo:

a) adotando a categoria mais frequente entre os k exemplos mais parecidos.
b) treinando centenas de árvores.
c) traçando uma reta entre as classes.
d) sorteando uma classe.

Questão 7. Por que o k-vizinhos costuma exigir padronização das escalas dos atributos?

a) Porque ele não tem hiperparâmetros.
b) Porque ele usa apenas categorias.
c) Porque ele usa distâncias, e atributos com números grandes dominariam o cálculo.
d) Porque ele não aceita números.

Questão 8. Uma floresta aleatória é formada por:

a) uma única árvore muito profunda.
b) centenas de árvores treinadas com amostras diferentes que votam juntas.
c) vários k-vizinhos.
d) uma regressão logística em cascata.

Questão 9. No gradient boosting, as árvores são construídas:

a) em sequência, cada uma corrigindo os erros das anteriores.
b) todas ao mesmo tempo e de forma independente.
c) apenas com atributos categóricos.
d) sem usar o erro.

Questão 10. Qual é a diferença entre parâmetro e hiperparâmetro?

a) Não há diferença.
b) Hiperparâmetros só existem em redes neurais.
c) Parâmetros são aprendidos durante o treino; hiperparâmetros são definidos antes dele.
d) Parâmetros são definidos pelo usuário; hiperparâmetros são aprendidos.

Questão 11. Qual dos itens abaixo é um hiperparâmetro de uma árvore de decisão?

a) O atributo escolhido na raiz.
b) O limite de corte escolhido em um nó.
c) A profundidade máxima.
d) A classe prevista em uma folha.

Questão 12. Subajuste (underfitting) ocorre quando o modelo:

a) decora o ruído do treino.
b) usa validação cruzada.
c) tem erro zero no treino.
d) é simples demais e erra até no treino.

Questão 13. Sobreajuste (overfitting) ocorre quando o modelo:

a) é complexo demais e se ajusta ao ruído do treino, errando mais em dados novos.
b) tem erro alto no treino.
c) usa poucos atributos.
d) é simples demais.

Questão 14. Na curva de complexidade, o erro de treino, ao aumentar a complexidade do modelo:

a) sobe sempre.
b) tende a cair sempre.
c) oscila sem tendência.
d) é sempre igual ao erro de validação.

Questão 15. O que significa generalização?

a) Uso de muitos atributos.
b) Quantidade de algoritmos testados.
c) Desempenho nos dados de treino.
d) Desempenho em dados que o modelo nunca viu.

Questão 16. Qual o papel do conjunto de validação?

a) Escolher hiperparâmetros e comparar modelos.
b) Ser aberto uma única vez no final para medir o desempenho real.
c) Treinar os parâmetros do modelo.
d) Substituir o conjunto de teste.

Questão 17. O conjunto de teste deve ser usado:

a) várias vezes, a cada ajuste de hiperparâmetro.
b) para escolher o algoritmo.
c) apenas uma vez, no final, para medir o desempenho real.
d) no lugar do treino.

Questão 18. O argumento stratify=y em train_test_split serve para:

a) remover atributos.
b) aumentar o conjunto de treino.
c) manter a proporção de classes em cada parte.
d) embaralhar os rótulos.

Questão 19. Qual é a vantagem da validação cruzada sobre uma única divisão treino/validação?

a) Dispensa o conjunto de teste.
b) Dá uma estimativa menos dependente da sorte do sorteio.
c) É sempre mais rápida.
d) Elimina o sobreajuste.

Questão 20. Em uma validação cruzada de 5 partes, cada exemplo é usado como validação:

a) cinco vezes.
b) exatamente uma vez.
c) duas vezes.
d) nenhuma vez.

Questão 21. Um classificador responde sempre "saudável" em uma doença que afeta 5% da população. Qual é sua acurácia aproximada?

a) 0%.
b) 5%.
c) 50%.
d) 95%.

Questão 22. No exemplo anterior, qual métrica revelaria o problema?

a) Revocação (recall) da classe doente.
b) Acurácia.
c) Número de exemplos.
d) Proporção de saudáveis.

Questão 23. A precisão (precision) responde à pergunta:

a) Qual a fração de acertos totais?
b) Entre os que o modelo chamou de positivos, quantos realmente eram?
c) Qual o tamanho do conjunto de teste?
d) Entre os positivos reais, quantos o modelo encontrou?

Questão 24. A revocação (recall) responde à pergunta:

a) Qual é o erro quadrático médio?
b) Quantos exemplos há no total?
c) Entre os que o modelo chamou de positivos, quantos eram?
d) Entre os positivos reais, quantos o modelo encontrou?

Questão 25. Em uma triagem médica, em que deixar passar um doente é mais grave que pedir um exame extra, qual métrica deve ser priorizada?

a) Revocação.
b) Precisão.
c) Número de atributos.
d) Acurácia apenas.

Questão 26. O que é uma linha de base (baseline)?

a) O conjunto de teste.
b) Um tipo de validação cruzada.
c) O melhor modelo possível.
d) Uma regra trivial de comparação, como prever sempre a classe mais comum.

Questão 27. O MAE em regressão mede:

a) a fração de acertos.
b) a média do tamanho dos erros, na unidade do alvo.
c) a proporção de variância explicada.
d) o número de atributos úteis.

Questão 28. Um R^2 próximo de zero indica que o modelo:

a) é perfeito.
b) tem erro zero.
c) explica quase nada além de prever sempre a média.
d) está sobreajustado ao teste.

Questão 29. No experimento com dados sem nenhum sinal, o melhor entre 300 modelos teve nota de validação bem acima de 50%. Isso ilustra:

a) que os dados continham informação.
b) que k-vizinhos é superior a árvores.
c) que a validação cruzada é inútil.
d) que a seleção entre muitos candidatos infla a nota do vencedor por sorte.

Questão 30. Qual função de scikit-learn mantém a proporção de classes ao dividir os dados em partes na validação cruzada?

a) KFold sem embaralhar.
b) PolynomialFeatures.
c) StratifiedKFold.
d) DummyClassifier.

Exercícios discursivos

Responda com suas palavras, em poucas linhas. Compare depois com a resposta-modelo do Apêndice A.

1. Diferencie exemplo, atributo e alvo usando uma tabela de clientes de uma loja.
2. Explique a diferença entre classificação e regressão, dando um exemplo de cada.
3. Explique, com uma analogia, o funcionamento do k-vizinhos e cite uma limitação.
4. Por que uma floresta aleatória costuma ser mais estável que uma única árvore?
5. Defina parâmetro e hiperparâmetro e explique por que a distinção importa para o AutoML.
6. Explique subajuste e sobreajuste e descreva como o erro de treino e o de validação se comportam em cada caso.
7. Por que o erro no treino não serve para escolher o melhor modelo?
8. Descreva o papel dos conjuntos de treino, validação e teste usando a analogia do estudante.
9. O que acontece se você usar o conjunto de teste para escolher hiperparâmetros? Por quê?
10. Explique como funciona a validação cruzada de k partes e por que ela é preferível a uma única divisão.
11. Mostre, com um exemplo numérico, por que a acurácia pode ser enganosa em dados desbalanceados.
12. Defina precisão, revocação e F1, e dê um exemplo de situação em que cada uma seria priorizada.
13. Calcule precisão, revocação e F1 para VP=30, FP=10, FN=20.
14. Qual é a utilidade de uma linha de base e como você a calcularia em um problema de classificação?
15. Diferencie MAE e RMSE e explique quando o RMSE é mais adequado.
16. O que o R^2 mede e o que significa um R^2 negativo?
17. Explique o "efeito do vencedor" e relacione-o com um sistema de AutoML que testa milhares de modelos.
18. Como evitar que a nota de validação do vencedor seja confundida com o desempenho real?
19. Por que o capítulo diz que "a escolha da métrica define o que o AutoML vai otimizar"? Dê um exemplo de escolha ruim.
20. Dê um exemplo do seu cotidiano de sobreajuste (decorar sem entender) e relacione-o ao conceito em aprendizado de máquina.

Exercícios práticos em Python (scikit-learn)

Resolva os exercícios no seu computador (ou no Google Colab). Eles usam conjuntos de dados da UCI que já vêm no scikit-learn ou dados simulados. Tente resolver **antes** de consultar a solução, no Apêndice D. As saídas apresentadas lá foram obtidas com as versões de bibliotecas listadas na Apresentação.

Exercício 1 — Curvas de aprendizado

NÍVEL: INTERMEDIÁRIO

Com o conjunto **Breast Cancer Wisconsin** (UCI, via `load_breast_cancer`), use `learning_curve` para comparar uma `DecisionTreeClassifier()` (sem limite de profundidade) e uma `LogisticRegression` com `StandardScaler`. Use `cv=5` e os tamanhos de treino 10%, 30%, 60% e 100%. Mostre a acurácia média de treino e de validação para cada tamanho.

Pergunta: qual dos modelos tem maior distância entre treino e validação? O que isso indica?

Exercício 2 — Métricas em dados desbalanceados

NÍVEL: INTERMEDIÁRIO

Transforme o conjunto Breast Cancer em um problema desbalanceado: mantenha todos os casos benignos (classe 1) e apenas **15 casos malignos** (classe 0, a "rara"). Divida em treino e teste (30%, estratificado). Compare: `DummyClassifier(strategy="most_frequent")`, uma `LogisticRegression` comum e uma `LogisticRegression(class_weight="balanced")`. Calcule acurácia, precisão e revocação **da classe maligna** e a AUC.

Pergunta: qual métrica enganaria você? Qual modelo você escolheria em uma triagem?

Exercício 3 — Uma divisão ou várias?

NÍVEL: BÁSICO

No conjunto **Wine** (UCI), avalie um `KNeighborsClassifier` (com `StandardScaler`) de duas formas: (a) repita **30 vezes** uma divisão treino/teste (70/30) com sementes diferentes e registre o menor e o maior resultado; (b) faça uma validação cruzada de 5 partes. Compare.

Pergunta: o que o intervalo de (a) diz sobre confiar em uma única divisão?

Exercício 4 — Reproduzindo o efeito do vencedor

NÍVEL: AVANÇADO

Use os atributos reais do conjunto **Breast Cancer**, mas **embaralhe o alvo** (`rng.permutation(y)`) para destruir qualquer relação entre X e y. Faça uma busca aleatória com 100 candidatos (árvores de profundidade 1 a 12 e k-vizinhos com k de 1 a 40), avaliando por validação cruzada. Depois avalie o melhor candidato em um **conjunto de teste separado**, também embaralhado.

Pergunta: qual a diferença entre a melhor nota de validação e a nota no teste? O que isso ensina sobre AutoML?

> SAIBA MAIS **Para ir além: bases públicas da UCI e do Kaggle**
>
> - **UCI:** Optical Recognition of Handwritten Digits (versão completa de 32×32) e Wine Quality (vinhos portugueses).
> - **Kaggle:** Digit Recognizer (versão MNIST, com 28×28 pixels).
>
> Estes conjuntos não foram usados nem testados nos códigos do livro. Confira, nos sites, a descrição, a licença e as colunas antes de usá-los.

Projeto integrador do capítulo

Protocolo completo de avaliação em Digits

Fonte dos dados. *Optical Recognition of Handwritten Digits* (Digits), do repositório **UCI Machine Learning**, incluído no scikit-learn (`load_digits`): 1.797 imagens 8×8 de dígitos manuscritos, com 64 atributos (intensidades de pixels) e 10 classes.

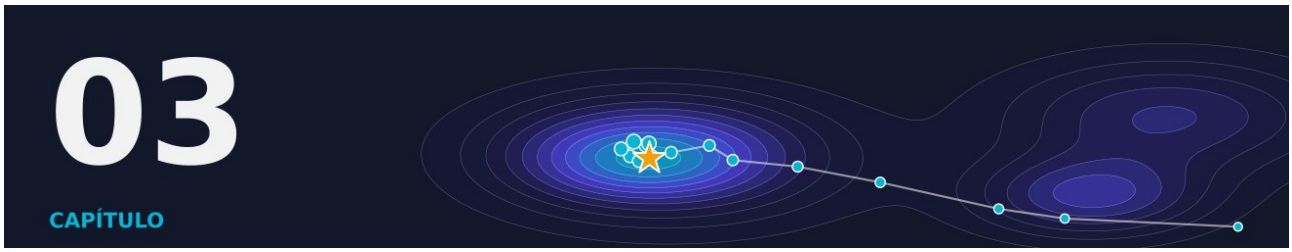
Contexto. Uma equipe quer reconhecer dígitos escritos à mão. Antes de qualquer AutoML, você deve montar o **protocolo de avaliação** que será reutilizado em todo o livro.

Tarefas.

1. Carregue os dados e mostre o formato e a proporção de cada classe.
2. Separe 25% para **teste final**, com estratificação e semente fixa. Não toque nele até o fim.
3. Calcule a **linha de base** (classe mais comum) no treino, por validação cruzada.
4. Compare, por validação cruzada de 5 partes **no treino**, quatro algoritmos: árvore, k-vizinhos (com padronização), regressão logística (com padronização) e floresta aleatória. Reporte média e desvio padrão da acurácia.
5. Escolha o melhor, treine no treino completo e avalie **uma única vez** no teste final. Mostre a acurácia, o F1 médio entre as classes (`f1_score(average="macro")`) e a **matriz de confusão** como imagem.
6. Escreva um parágrafo (6 a 8 linhas): qual algoritmo venceu e por quê pode ter vencido? A nota do teste é coerente com a de validação? Quais dígitos mais se confundem?

Entregáveis. Um notebook (ou script) comentado, a matriz de confusão e o parágrafo de conclusões.

A solução de referência está no Apêndice E.



Capítulo 3 — Pipelines e pré-processamento

OBJETIVOS DE APRENDIZAGEM

Ao final deste capítulo você será capaz de:

- Reconhecer os problemas mais comuns de dados tabulares: valores ausentes, escalas diferentes, categorias em texto e atributos inúteis.
- Aplicar imputação, padronização e codificação com o scikit-learn.
- Montar pipelines com `Pipeline` e `ColumnTransformer`.
- Explicar o que é vazamento de dados e evitá-lo ao colocar as etapas dentro do pipeline.
- Enxergar o pipeline como um espaço de busca e ajustá-lo com `GridSearchCV`.
- Salvar, recarregar e reutilizar um pipeline treinado.

3.1 A roupa suja vem antes do modelo

Todo cozinheiro sabe: antes de cozinhar, é preciso **lavar, descascar e cortar** os ingredientes. Com dados é igual. Os dados reais chegam sujos: células vazias, números em escalas muito diferentes, categorias escritas como texto.

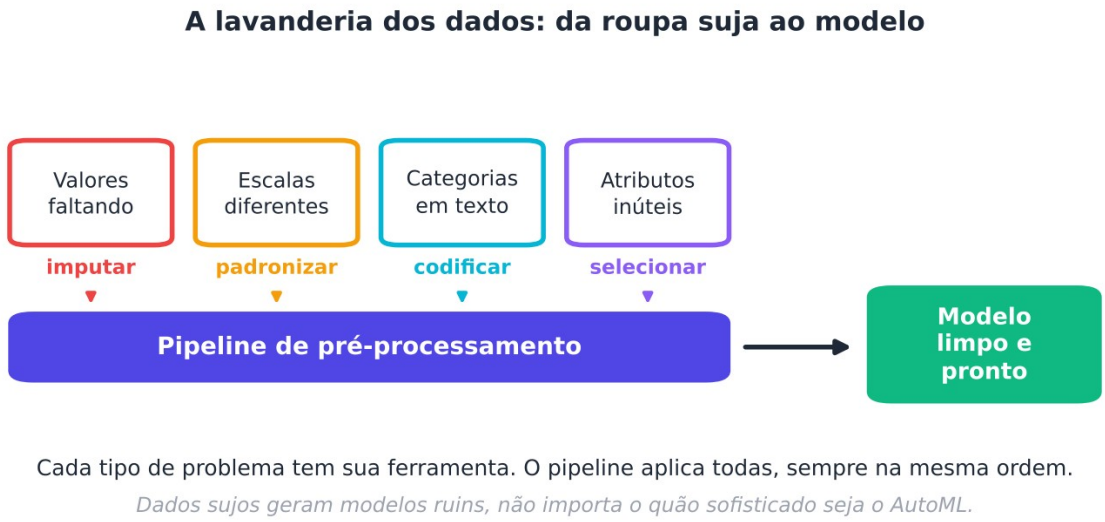


Figura 3.1 — Cada problema dos dados tem sua ferramenta. O pipeline de pré-processamento aplica todas, sempre na mesma ordem.

Esse trabalho é chamado de **pré-processamento**. É a base de qualquer projeto de aprendizado de máquina e é parte do que o AutoML precisa decidir. O Auto-sklearn, por exemplo, inclui no seu espaço de busca métodos de preparo dos dados e de pré-processamento de atributos (Feurer *et al.*, 2015). Levantamentos de AutoML também revisam o pipeline passo a passo (Zöller; Huber, 2021).

> **Atenção**
>
> O AutoML pode escolher **como** tratar os dados, mas não adivinha **o que significam**. Uma coluna "999" pode ser idade ou um código para "não informado". Essa interpretação continua sendo humana.

3.2 Os dados deste capítulo

Para praticar, vamos criar um conjunto de dados **propositalmente sujo**. Partiremos do conjunto **Wine** (UCI), com 13 medidas químicas de 178 vinhos (Dua; Graff, 2019). Acrescentaremos, por simulação, dois problemas comuns:

- uma coluna **categórica** de texto (a "safra" do vinho);
- **valores ausentes** (8% de células vazias em quatro colunas).

É importante dizer: esses problemas foram inseridos por nós, para fins didáticos. Em um projeto real, os dados já chegam assim.

PYTHON

```python
import numpy as np
import pandas as pd
from sklearn.datasets import load_wine

rng = np.random.default_rng(0)
df = load_wine(as_frame=True).frame.copy()

# Coluna categórica simulada ("safra"), levemente ligada à classe do vinho
safras = np.array(["2018", "2019", "2020"])
df["safra"] = np.where(rng.random(len(df)) < 0.8, safras[df["target"].values % 3], rng.choice(safras,
len(df)))

# 8% de valores ausentes, ao acaso, em quatro colunas
for coluna in ["alcohol", "malic_acid", "flavanoids", "proline"]:
    df.loc[rng.random(len(df)) < 0.08, coluna] = np.nan

y = df.pop("target")
X = df
print("Formato de X:", X.shape)
print("\nValores ausentes por coluna:")
print(X.isna().sum()[X.isna().sum() > 0].to_string())
print("\nColunas numéricas:", X.select_dtypes("number").shape[1], "| colunas de texto:",
X.select_dtypes(exclude="number").shape[1])
```

SAÍDA

```
Formato de X: (178, 14)

Valores ausentes por coluna:
alcohol       10
malic_acid     9
flavanoids    11
proline       14

Colunas numéricas: 13 | colunas de texto: 1
```

3.3 O que acontece se ignorarmos a sujeira?

Vamos tentar treinar um modelo diretamente nesses dados.

```python
from sklearn.linear_model import LogisticRegression

try:
    LogisticRegression().fit(X, y)
except ValueError as erro:
    print("O scikit-learn recusou os dados:")
    print(str(erro).splitlines()[0][:110])
```

```
O scikit-learn recusou os dados:
Input X contains NaN.
```

O modelo recusa dados com valores vazios e com texto. Precisamos tratar cada problema.

3.4 Valores ausentes: imputação

Imputar é preencher os valores que faltam com um palpite razoável. As opções mais comuns:

- **Média** ou **mediana** da coluna. A mediana é mais robusta quando há valores extremos.
- **Valor mais frequente**, para colunas categóricas.
- **Vizinhos mais próximos** (`KNNImputer`): usa os exemplos mais parecidos para preencher.
- **Indicador de ausência**: cria uma coluna extra dizendo "este valor estava faltando". Às vezes **o fato de faltar** já é informação (por exemplo, quem não informa renda).

Analogia: você esqueceu o sal na lasanha. Pode usar "uma pitada média" (a média), "a quantidade da receita mais comum" (a mediana) ou copiar a quantidade de uma receita parecida (vizinhos).

> **Dica**
>
> Escolher *como* imputar é um hiperparâmetro do pipeline. Você pode deixar o AutoML testar média, mediana e vizinhos, e escolher o melhor pela validação.

3.5 Escalas diferentes: padronização

Observe os números do Wine: o álcool varia de 11 a 15, enquanto a prolina vai de quase 300 a mais de 1600. Para um algoritmo baseado em distância, como o k-vizinhos, a prolina vai **dominar tudo**.

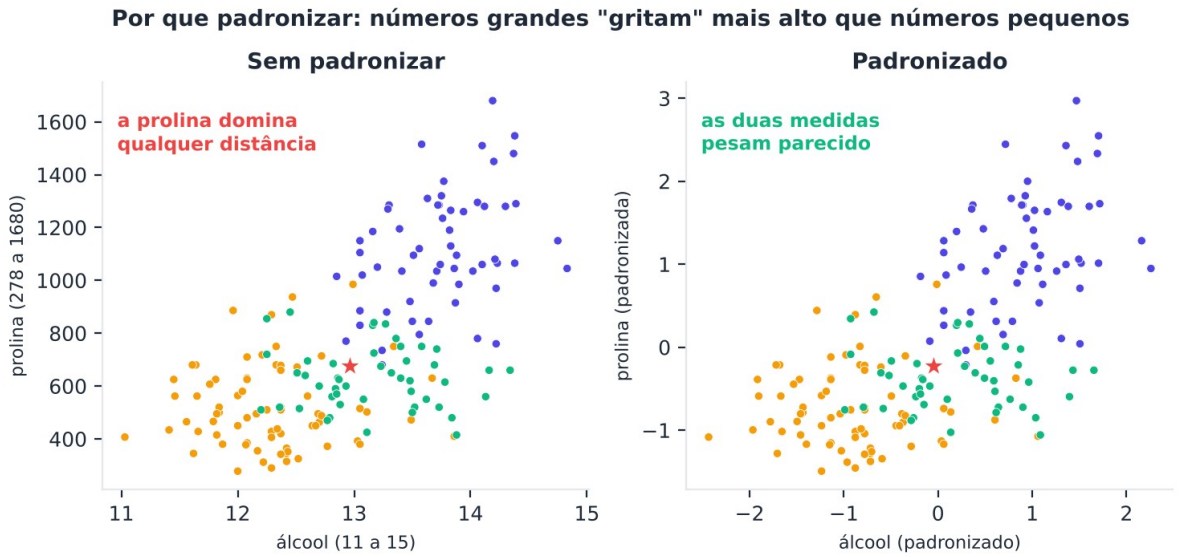


Figura 3.2 — Sem padronização, a prolina "grita" mais alto que o álcool em qualquer cálculo de distância. Padronizado, as duas medidas pesam parecido.

As técnicas mais usadas:

- **Padronização** (`StandardScaler`): subtrai a média e divide pelo desvio padrão. O resultado tem média 0 e desvio 1.
- **Min-máx** (`MinMaxScaler`): leva os valores para o intervalo de 0 a 1.
- **Robusta** (`RobustScaler`): usa mediana e quartis, e sofre menos com valores extremos.

Nem todo algoritmo precisa disso. **Árvores e florestas** não se importam com a escala, pois só comparam "maior ou menor". Mas **k-vizinhos, SVM, regressão logística e redes neurais** dependem dela.

PYTHON

```python
from sklearn.datasets import load_wine
from sklearn.model_selection import cross_val_score
from sklearn.neighbors import KNeighborsClassifier
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

X, y = load_wine(return_X_y=True)   # versão completa (sem ausentes) só para este teste
sem = cross_val_score(KNeighborsClassifier(), X, y, cv=5).mean()
com = cross_val_score(make_pipeline(StandardScaler(), KNeighborsClassifier()), X, y, cv=5).mean()
print(f"k-vizinhos sem padronizar: {sem:.3f}")
print(f"k-vizinhos padronizado   : {com:.3f}")
```

SAÍDA

```
k-vizinhos sem padronizar: 0.691
k-vizinhos padronizado   : 0.949
```

Uma simples padronização muda muito o resultado. Um bom AutoML considera a escala como parte da busca.

3.6 Categorias em texto: codificação

Modelos entendem números, e não palavras. É preciso **codificar** as categorias.

Uma ideia ingênua é trocar "Recife", "Natal" e "Salvador" por 1, 2 e 3. Mas isso ensina ao modelo uma **ordem que não existe** (como se Salvador fosse "maior" que Natal). A solução usual é a **codificação one-hot**.



Figura 3.3 — Na codificação one-hot, cada categoria vira uma coluna de 0 ou 1.

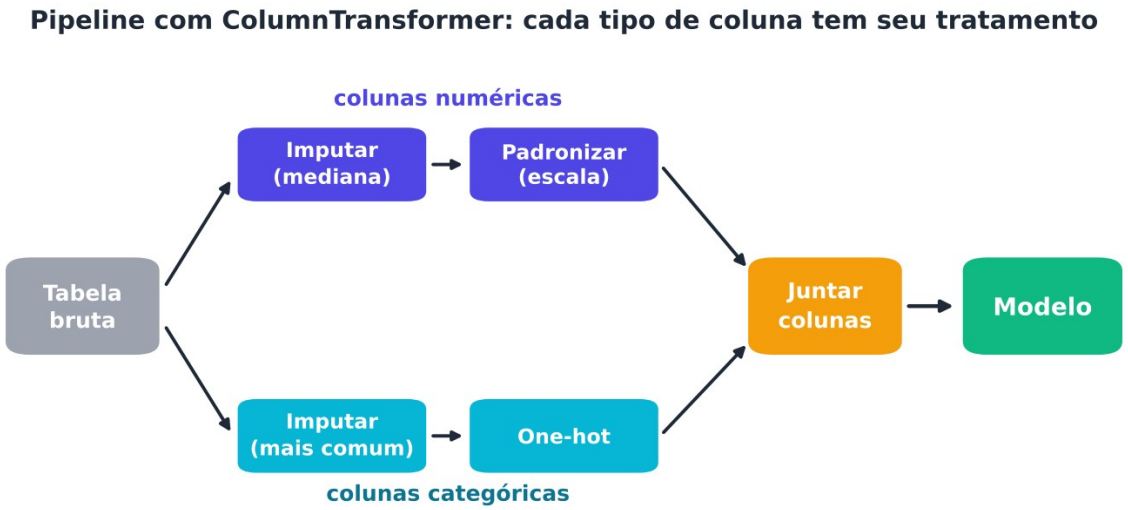
Use a **codificação ordinal** (1, 2, 3) apenas quando existe ordem natural, como "baixo, médio, alto". Para categorias com **milhares de valores** (como CEP), o one-hot gera colunas demais, e são necessárias técnicas especiais.

> **Atenção**
>
> Na produção, podem aparecer **categorias novas**, que o modelo nunca viu. Com `OneHotEncoder(handle_unknown="ignore")`, elas são codificadas como "todos zeros", em vez de derrubar o sistema.

3.7 Pipelines: uma linha de montagem em código

Fazer cada passo à mão é trabalhoso e propenso a erros. O scikit-learn oferece duas peças para organizar tudo (Pedregosa *et al.*, 2011):

- `Pipeline`: encadeia etapas, em que a saída de uma é a entrada da próxima. O último passo é o modelo.
- `ColumnTransformer`: aplica tratamentos **diferentes** a grupos diferentes de colunas.



Tudo isso é um único objeto Pipeline: um só .fit() e um só .predict().

Figura 3.4 — Colunas numéricas e categóricas recebem tratamentos próprios. No fim, tudo se junta e alimenta o modelo.

PYTHON

```python
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_val_score

colunas_num = X.select_dtypes(include="number").columns.tolist()
colunas_cat = ["safra"]

preparo = ColumnTransformer([
    ("num", Pipeline([("imputar", SimpleImputer(strategy="median")),
                      ("escala", StandardScaler())]), colunas_num),
    ("cat", Pipeline([("imputar", SimpleImputer(strategy="most_frequent")),
                      ("onehot", OneHotEncoder(handle_unknown="ignore"))]), colunas_cat),
])

pipeline = Pipeline([("preparo", preparo), ("modelo", LogisticRegression(max_iter=2000))])

notas = cross_val_score(pipeline, X, y, cv=5)
print("Notas:", notas.round(3), "| média =", round(notas.mean(), 3))
```

SAÍDA

```
Notas: [0.972 0.944 1.    0.971 0.971] | média = 0.972
```

Pronto: um único objeto que **recebe a tabela suja e devolve previsões**. Ao chamar `.fit(X, y)`, cada etapa aprende o que precisa (a mediana, a média, as categorias) usando apenas os dados de treino. Ao chamar `.predict(X_novo)`, aplica as mesmas transformações.

3.8 O perigo do vazamento de dados

Chegamos a um dos conceitos **mais importantes** do livro: o **vazamento de dados** (*data leakage*).

Vazamento ocorre quando **informação do conjunto de validação (ou teste) "escapa" para o treino**. O modelo parece melhor do que é, e você só percebe quando ele falha na vida real.

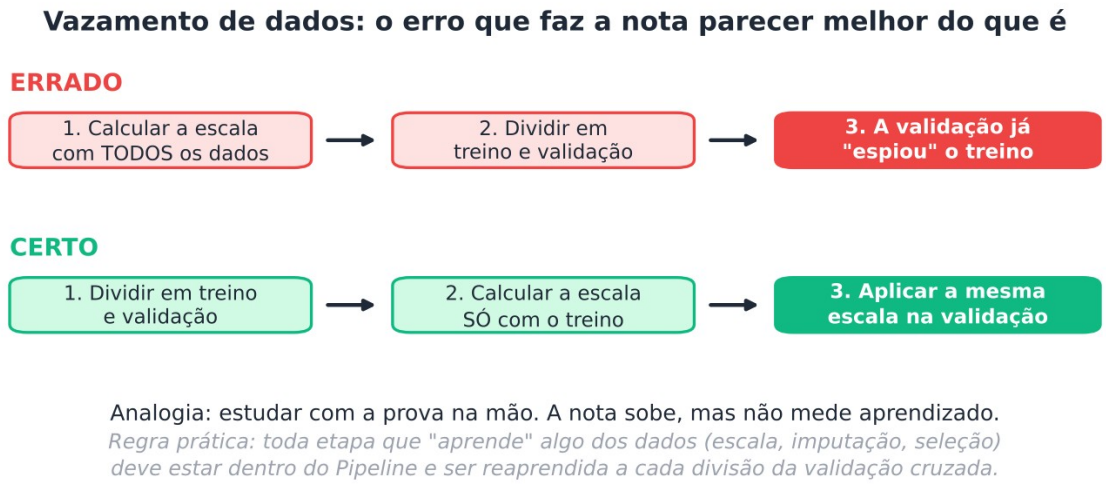


Figura 3.5 — Calcular a escala com todos os dados antes de dividir deixa a validação "espiar" o treino. O certo é aprender tudo apenas com o treino.

Analogia: estudar com a prova na mão. A nota sobe, mas não mede aprendizado.

Um estudo de grande escala encontrou vazamento em dezenas de campos que usam aprendizado de máquina, afetando centenas de artigos e levando a conclusões exageradas (Kapoor; Narayanan, 2023). Não é um erro só de iniciantes.

Um experimento revelador

Um exemplo clássico, descrito por Hastie, Tibshirani e Friedman (2009), mostra o estrago. Criamos dados **totalmente aleatórios**: 100 exemplos, 2000 atributos sem nenhuma relação com o alvo. Qualquer método honesto deve acertar cerca de 50%.

Fazemos de duas formas. Na **errada**, escolhemos os 20 "melhores" atributos olhando *todos* os dados e só depois validamos. Na **certa**, a escolha dos atributos fica *dentro* do pipeline, refeita a cada divisão.

```python
import numpy as np
from sklearn.feature_selection import SelectKBest, f_classif
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_val_score
from sklearn.pipeline import make_pipeline

rng = np.random.default_rng(0)
X = rng.normal(size=(100, 2000))     # atributos sem nenhuma relação com y
y = rng.integers(0, 2, 100)          # alvo sorteado

# ERRADO: seleciona atributos com TODOS os dados, depois valida
X_sel = SelectKBest(f_classif, k=20).fit_transform(X, y)
errado = cross_val_score(LogisticRegression(), X_sel, y, cv=5).mean()

# CERTO: a seleção está DENTRO do pipeline (refeita em cada divisão)
certo_pipe = make_pipeline(SelectKBest(f_classif, k=20), LogisticRegression())
certo = cross_val_score(certo_pipe, X, y, cv=5).mean()
```

```
print(f"Seleção fora do pipeline (errado): acurácia = {errado:.3f}")
print(f"Seleção dentro do pipeline (certo): acurácia = {certo:.3f}")
```

SAÍDA

```
Seleção fora do pipeline (errado): acurácia = 0.780
Seleção dentro do pipeline (certo): acurácia = 0.590
```

No jeito errado, a acurácia fica **muito acima de 50%**, e isso é impossível em dados sem sinal! A seleção olhou os rótulos da validação. No jeito certo, o resultado volta para perto do acaso (a pequena diferença em relação a 50% é ruído, natural em uma amostra de apenas 100 exemplos).

MEMORIZE **Regra de ouro contra o vazamento**

Toda etapa que aprende algo dos dados (imputação, escala, seleção de atributos, redução de dimensões) deve estar **dentro do `Pipeline`**. Assim, ela é reaprendida só com o treino em cada divisão.

3.9 O pipeline como espaço de busca

Agora a ligação com o AutoML. Cada etapa do pipeline tem **escolhas**: como imputar, qual escala usar, se haverá seleção de atributos, qual algoritmo. Juntas, formam o espaço de busca.

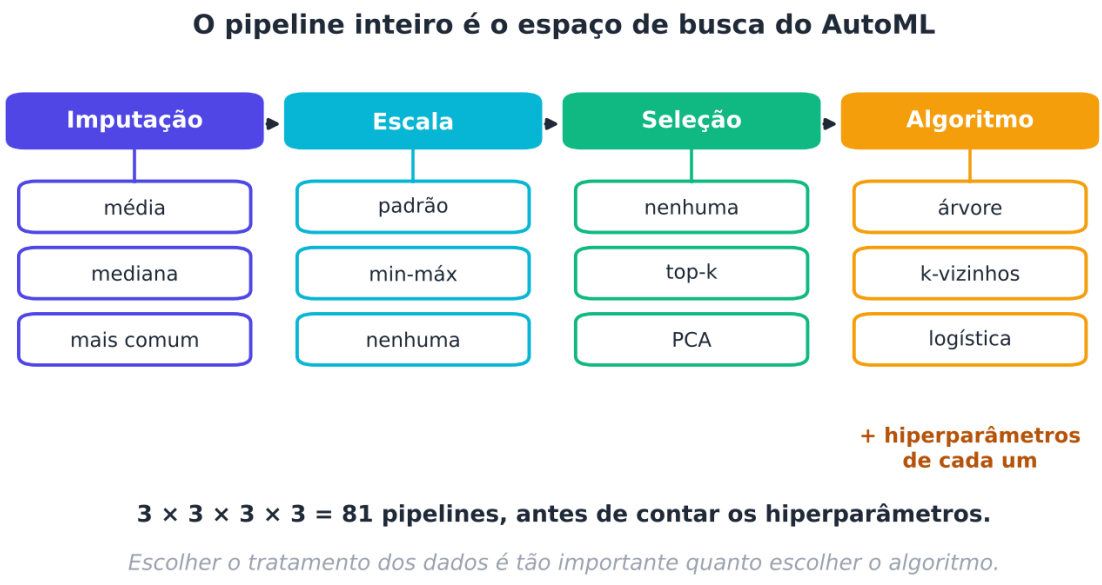


Figura 3.6 — Mesmo com poucas opções por etapa, o número de pipelines cresce rápido, antes de contar os hiperparâmetros.

O scikit-learn permite buscar **dentro do pipeline** usando nomes com dois sublinhados (`__`), que indicam "etapa, parâmetro". Etapas inteiras também podem ser trocadas, como o tipo de escala.

PYTHON

```
from sklearn.model_selection import GridSearchCV
from sklearn.preprocessing import MinMaxScaler, RobustScaler

busca_pipe = Pipeline([
    ("preparo", ColumnTransformer([
        ("num", Pipeline([("imputar", SimpleImputer()), ("escala", StandardScaler())]), colunas_num),
        ("cat", Pipeline([("imputar", SimpleImputer(strategy="most_frequent")),
```

```
                          ("onehot", OneHotEncoder(handle_unknown="ignore"))]), colunas_cat),
    ])),
    ("modelo", LogisticRegression(max_iter=3000)),
])

grade = {
    "preparo__num__imputar__strategy": ["mean", "median"],
    "preparo__num__escala": [StandardScaler(), MinMaxScaler(), RobustScaler()],
    "modelo__C": [0.1, 1, 10],
}
busca = GridSearchCV(busca_pipe, grade, cv=5, n_jobs=-1).fit(X, y)

print("Combinações testadas:", len(busca.cv_results_["params"]))
print("Melhor nota de validação:", round(busca.best_score_, 3))
for nome, valor in busca.best_params_.items():
    print(f"  {nome} = {valor}")
```

SAÍDA

```
Combinações testadas: 18
Melhor nota de validação: 0.977
  modelo__C = 0.1
  preparo__num__escala = StandardScaler()
  preparo__num__imputar__strategy = mean
```

Isto já é um **mini-AutoML** sobre pipelines: um espaço de busca, uma estratégia (grade) e uma estimativa (validação cruzada). Nos próximos capítulos, a estratégia ficará muito mais inteligente.

3.10 Outras transformações úteis

Além de imputar, padronizar e codificar, há outras transformações que um pipeline pode incluir.

- **Transformações de cauda** (`PowerTransformer`, `QuantileTransformer`): deixam distribuições muito assimétricas (como renda) mais parecidas com um sino.
- **Remoção de colunas quase constantes** (`VarianceThreshold`): descarta atributos que quase não mudam.
- **Seleção de atributos** (`SelectKBest`): fica com os atributos mais ligados ao alvo (veja o Capítulo 8).
- **Redução de dimensionalidade** (`PCA`): resume muitas colunas em poucas, preservando a maior parte da variação.
- **Desbalanceamento de classes:** use `class_weight="balanced"` em muitos algoritmos, ou ajuste a métrica (Capítulo 2).

Todas elas entram no pipeline como mais uma etapa, e todas podem ser escolhidas por uma busca automática.

3.11 Salvando e reutilizando o pipeline

Depois de treinar, você precisará **guardar** o pipeline para usá-lo mais tarde, em outro programa ou servidor. Como o pré-processamento faz parte do objeto, nada se perde: quem receber o arquivo aplica exatamente as mesmas transformações.

PYTHON

```
import joblib
```

```
melhor_pipeline = busca.best_estimator_           # já treinado com todos os dados
joblib.dump(melhor_pipeline, "/tmp/pipeline_vinho.joblib")

recarregado = joblib.load("/tmp/pipeline_vinho.joblib")
posicoes = [0, 30, 70, 100, 140, 170]             # vinhos de classes diferentes, com ausentes e texto
novos = X.iloc[posicoes]
print("Classes reais            :", y.iloc[posicoes].tolist())
print("Previsões do original    :", melhor_pipeline.predict(novos).tolist())
print("Previsões do recarregado:", recarregado.predict(novos).tolist())
```

SAÍDA

```
Classes reais            : [0, 0, 1, 1, 2, 2]
Previsões do original    : [0, 0, 1, 1, 2, 2]
Previsões do recarregado: [0, 0, 1, 1, 2, 2]
```

Esse hábito será essencial no Capítulo 14, quando falarmos de colocar modelos em produção.

> **Atenção**
>
> Arquivos do `joblib` (e do `pickle`) podem executar código ao serem abertos. **Nunca carregue** um arquivo de origem desconhecida.

3.12 Reprodutibilidade

Dois cuidados simples tornam seus experimentos **repetíveis**:

1. **Fixe as sementes aleatórias** (`random_state`) em divisões, em modelos e em buscas.
2. **Registre as versões** das bibliotecas (como fizemos no Capítulo 1), pois pequenas mudanças de versão podem alterar resultados.

Uma busca automática registra cada tentativa. Aproveite: guarde esse histórico junto com o pipeline final.

Resumo do capítulo

MEMORIZE **Cartões de memorização**

- **Pré-processamento:** imputar (ausentes), padronizar (escalas), codificar (categorias).
- **Escala:** importa para k-vizinhos, SVM, regressão e redes; não importa para árvores.
- **One-hot** evita inventar uma ordem entre categorias.
- **Pipeline** encadeia etapas; **ColumnTransformer** trata colunas de tipos diferentes.
- **Vazamento:** informação da validação "espiando" o treino. A nota fica falsamente alta.
- **Regra de ouro:** tudo que aprende dos dados fica dentro do pipeline.
- **O pipeline inteiro é o espaço de busca** do AutoML. Use `etapa__parametro` para buscar dentro dele.
- **Salve o pipeline completo**, não só o modelo.

Exercícios de múltipla escolha

Escolha **uma** alternativa em cada questão. O gabarito está no Apêndice A.

Questão 1. O que significa imputar valores ausentes?

a) Converter texto em números.
b) Dividir os dados em treino e teste.
c) Remover todas as linhas com dados faltando.
d) Preencher os valores que faltam com um palpite razoável.

Questão 2. Qual estratégia de imputação é mais robusta a valores extremos em uma coluna numérica?

a) Média.
b) Soma.
c) Mediana.
d) Máximo.

Questão 3. Para uma coluna categórica com valores faltando, qual imputação costuma ser adequada?

a) Média.
b) Desvio padrão.
c) Valor mais frequente.
d) Mediana.

Questão 4. Qual a utilidade de um indicador de ausência?

a) Padronizar a coluna.
b) Apagar a coluna com dados faltantes.
c) Aumentar o número de classes.
d) Registrar que o valor estava faltando, pois isso pode ser informativo.

Questão 5. Por que a escala dos atributos é importante para o k-vizinhos?

a) Porque ele não usa distâncias.
b) Porque ele ignora atributos numéricos.
c) Porque ele só aceita valores entre 0 e 1.
d) Porque ele usa distâncias, e atributos com valores grandes dominam o cálculo.

Questão 6. Qual algoritmo é, em geral, insensível à escala dos atributos?

a) SVM com kernel RBF.
b) Regressão logística regularizada.
c) k-vizinhos.
d) Árvore de decisão.

Questão 7. O que faz o StandardScaler?

a) Remove valores extremos.
b) Converte categorias em números.
c) Leva os valores para o intervalo de 0 a 1.
d) Subtrai a média e divide pelo desvio padrão.

Questão 8. O que faz o MinMaxScaler?

a) Subtrai a mediana.
b) Leva os valores para o intervalo de 0 a 1.
c) Remove colunas constantes.
d) Cria colunas de 0 e 1 para categorias.

Questão 9. Por que não é recomendável codificar "Recife, Natal, Salvador" como 1, 2 e 3 para a maioria dos modelos?

a) Porque os números ficariam grandes demais.
b) Porque isso reduz o número de linhas.
c) Porque o scikit-learn não aceita números inteiros.
d) Porque cria uma ordem que não existe entre as categorias.

Questão 10. Na codificação one-hot, cada categoria é transformada em:

a) uma coluna de 0 ou 1.
b) um número inteiro único.
c) uma linha nova.
d) um número entre 0 e 1.

Questão 11. Quando a codificação ordinal (1, 2, 3) é apropriada?

a) Apenas em regressão.
b) Quando existe uma ordem natural, como baixo, médio e alto.
c) Nunca.
d) Sempre que houver texto.

Questão 12. O que faz handle_unknown="ignore" no OneHotEncoder?

a) Troca categorias por números aleatórios.
b) Remove as categorias raras.
c) Codifica categorias nunca vistas no treino como "todos zeros", sem gerar erro.
d) Ignora todas as colunas categóricas.

Questão 13. Qual é a função do ColumnTransformer?

a) Escolher o melhor algoritmo.
b) Treinar vários modelos ao mesmo tempo.
c) Aplicar tratamentos diferentes a grupos diferentes de colunas.
d) Dividir os dados em treino e teste.

Questão 14. Um Pipeline do scikit-learn:

a) encadeia etapas, sendo a saída de uma a entrada da seguinte.
b) elimina a necessidade de validação.
c) só serve para modelos de redes neurais.
d) executa as etapas em paralelo, sem ordem.

Questão 15. Em um Pipeline, ao chamar fit(), cada transformador aprende suas estatísticas usando:

a) nenhum dado.
b) apenas os dados que recebe para treino.
c) todos os dados, inclusive o teste.
d) dados sorteados da internet.

Questão 16. Vazamento de dados (data leakage) é:

a) a divulgação pública dos dados.
b) o uso de dados categóricos.
c) perda de arquivos por falha de disco.
d) o escape de informação da validação ou do teste para o processo de treino.

Questão 17. Qual das situações abaixo é um exemplo de vazamento?

a) Fixar uma semente aleatória.
b) Salvar o pipeline com joblib.
c) Escolher os atributos com base em todos os dados e só depois fazer a validação cruzada.
d) Usar StratifiedKFold.

Questão 18. No experimento com 2000 atributos aleatórios, por que a acurácia subiu com a seleção feita fora do pipeline?

a) Porque a seleção usou os rótulos de todos os exemplos, inclusive os de validação.
b) Porque a regressão logística é muito poderosa.
c) Porque os atributos continham sinal real.
d) Porque a amostra era grande.

Questão 19. Segundo a regra de ouro do capítulo, onde devem ficar as etapas que aprendem algo dos dados?

a) Antes da divisão em treino e teste.
b) Apenas após a avaliação final.
c) Dentro do Pipeline.
d) Em uma planilha separada.

Questão 20. Em um GridSearchCV sobre um pipeline, o nome "modelo__C" indica:

a) a classe C.
b) uma coluna chamada C.
c) o parâmetro C da etapa chamada "modelo".
d) o número de conjuntos.

Questão 21. Em um espaço com 3 opções de imputação, 3 de escala, 3 de seleção e 3 de algoritmo, quantos pipelines existem (sem contar hiperparâmetros)?

a) 81.
b) 9.
c) 12.
d) 27.

Questão 22. Qual transformação é indicada para reduzir muitas colunas em poucas, preservando a maior parte da variação?

a) PCA.
b) OneHotEncoder.
c) SimpleImputer.
d) LabelEncoder.

Questão 23. Qual cuidado é essencial ao carregar um arquivo de pipeline salvo com joblib?

a) Nunca carregar arquivos de origem desconhecida, pois podem executar código.
b) Abri-lo em um editor de texto.
c) Compactá-lo antes.
d) Renomear o arquivo.

Questão 24. Por que é melhor salvar o pipeline completo e não apenas o modelo?

a) Porque o pré-processamento vai junto e é aplicado exatamente da mesma forma.
b) Porque evita usar validação cruzada.
c) Porque o arquivo fica menor.
d) Porque o modelo sozinho não funciona.

Questão 25. Qual o papel do random_state em divisões e modelos?

a) Fixar o sorteio para que os resultados sejam repetíveis.
b) Escolher o melhor algoritmo.
c) Remover valores ausentes.
d) Aumentar a acurácia.

Questão 26. Um PowerTransformer ou QuantileTransformer é útil, por exemplo, para:

a) preencher valores ausentes.
b) deixar distribuições muito assimétricas mais parecidas com um sino.
c) dividir os dados.
d) codificar texto.

Questão 27. O que o VarianceThreshold faz?

a) Cria interações entre atributos.
b) Descarta atributos que quase não variam.
c) Padroniza as colunas.
d) Calcula o desempenho do modelo.

Questão 28. Segundo o capítulo, o que o AutoML NÃO faz sozinho em relação aos dados?

a) Testar diferentes tipos de imputação.
b) Descobrir o significado real de uma coluna ambígua, como um código "999".
c) Combinar etapas em um pipeline.
d) Testar diferentes escalas.

Questão 29. Qual objeto do scikit-learn é adequado para preencher valores ausentes com a mediana de cada coluna?

a) OneHotEncoder.
b) SimpleImputer(strategy="median").
c) MinMaxScaler.
d) PCA.

Questão 30. Por que é arriscado calcular a média e o desvio padrão da padronização usando todos os dados antes de dividir em treino e validação?

a) Porque os dados deixam de ser numéricos.
b) Porque o desvio padrão será sempre zero.
c) Porque a validação influencia a escala usada no treino, o que caracteriza vazamento.
d) Porque a padronização só funciona com dados categóricos.

Exercícios discursivos

Responda com suas palavras, em poucas linhas. Compare depois com a resposta-modelo do Apêndice A.

1. Explique por que os dados reais precisam de pré-processamento antes do treino. Cite três problemas comuns.
2. Descreva três formas de imputar valores ausentes e quando cada uma é preferível.
3. Explique, com um exemplo numérico, por que atributos em escalas diferentes atrapalham o k-vizinhos.
4. Diferencie padronização, escala min-máx e escala robusta.
5. Por que árvores e florestas não precisam de padronização, enquanto SVM e regressão logística precisam?
6. Explique a codificação one-hot e o problema que ela evita.
7. Quando uma codificação ordinal é adequada? Dê um exemplo.
8. Quais problemas pode causar o one-hot em colunas com milhares de categorias? Que cuidado tomar na produção com categorias novas?
9. Explique o que é um Pipeline e por que ele ajuda a organizar e a proteger o experimento.
10. O que faz o ColumnTransformer? Dê um exemplo de uso com colunas numéricas e categóricas.
11. Defina vazamento de dados e explique, com uma analogia, por que ele é perigoso.
12. Descreva o experimento com 2000 atributos aleatórios e o que ele ensina.
13. Dê dois exemplos de vazamento, além da seleção de atributos feita fora do pipeline.
14. Explique a regra "tudo que aprende dos dados fica dentro do pipeline" e cite quatro etapas a que ela se aplica.
15. Explique em que sentido o pipeline é o espaço de busca do AutoML e como o scikit-learn permite buscar dentro dele.
16. Calcule o número de pipelines em um espaço com 2 imputações, 3 escalas, 2 seleções e 4 algoritmos, e comente o que isso sugere sobre a busca manual.
17. Por que salvar o pipeline completo é preferível a salvar apenas o modelo? Qual risco de segurança deve ser lembrado?
18. Que cuidados garantem a reprodutibilidade dos experimentos de pré-processamento e modelagem?
19. Dê um exemplo de coluna cuja interpretação o AutoML não consegue deduzir sozinho e explique o risco.
20. Compare tratar os dados "à mão" e deixar o AutoML escolher o pré-processamento, apontando uma vantagem e um cuidado.

Exercícios práticos em Python (scikit-learn)

Resolva os exercícios no seu computador (ou no Google Colab). Eles usam conjuntos de dados da UCI que já vêm no scikit-learn ou dados simulados. Tente resolver **antes** de consultar a solução, no Apêndice D. As saídas apresentadas lá foram obtidas com as versões de bibliotecas listadas na Apresentação.

Exercício 1 — Qual imputação escolher?

NÍVEL: INTERMEDIÁRIO

Com o conjunto **Breast Cancer Wisconsin** (UCI, via `load_breast_cancer`), apague **15% dos valores ao acaso** em todas as colunas. Compare, por validação cruzada de 5 partes e **dentro de um pipeline** com `StandardScaler` e `LogisticRegression`, quatro tratamentos: média, mediana, `KNNImputer(n_neighbors=5)` e mediana com indicador de ausência (`add_indicator=True`).

Pergunta: as diferenças são grandes? Elas são maiores que o desvio padrão da validação cruzada?

Exercício 2 — Ordinal ou one-hot?

NÍVEL: BÁSICO

Simule uma coluna categórica "cidade" com 5 categorias (A a E), cuja **taxa de sucesso** é diferente em cada uma: A=0,10; B=0,80; C=0,20; D=0,90; E=0,15. Gere 2.000 exemplos. Compare, por validação cruzada, uma `LogisticRegression` que recebe a cidade codificada com `OrdinalEncoder` (0 a 4) e outra com `OneHotEncoder`.

Pergunta: por que a codificação ordinal vai mal aqui?

Exercício 3 — Vazamento: a nota honesta

NÍVEL: INTERMEDIÁRIO

Use o conjunto **Wine** e acrescente **1.000 colunas de ruído** (`rng.normal`). Simule um problema de **poucos dados**: separe apenas **60 exemplos para treino** (`train_size=60`, estratificado) e deixe o restante como **teste final**, sem usá-lo em nada antes do fim. No treino, estime o desempenho de duas formas: (a) selecione os 15 melhores atributos (`SelectKBest`) com todo o treino e só depois valide; (b) deixe a seleção dentro do pipeline. Para cada forma, meça também o resultado no teste final. Repita com **5 sementes** e calcule as médias.

Pergunta: qual estimativa de validação ficou mais perto do resultado no teste final? Qual delas foi otimista?

Exercício 4 — Buscando dentro do pipeline

NÍVEL: BÁSICO

Com o conjunto **Wine**, monte um pipeline com um transformador de escala e um `KNeighborsClassifier`. Use `GridSearchCV` para escolher, ao mesmo tempo, o tipo de escala (`StandardScaler`, `MinMaxScaler` ou nenhuma, com `"passthrough"`) e o número de vizinhos (1, 3, 5, 9, 15). Mostre os 3 melhores resultados.

Pergunta: a escala importou?

> SAIBA MAIS **Para ir além: bases públicas da UCI e do Kaggle**
>
> - **UCI:** Adult (renda de adultos, com muitos atributos categóricos e ausentes) e Heart Disease (cardiologia, com ausentes).
> - **Kaggle:** Titanic (com idade ausente e atributos categóricos) e House Prices (muitos atributos categóricos e ausentes).
>
> Estes conjuntos não foram usados nem testados nos códigos do livro. Confira, nos sites, a descrição, a licença e as colunas antes de usá-los.

Projeto integrador do capítulo

Pipeline robusto para dados sujos

Fonte dos dados. *Breast Cancer Wisconsin (Diagnostic)*, do repositório **UCI Machine Learning**, incluído no scikit-learn. Para o projeto, você vai **sujar** os dados de propósito, simulando o cotidiano.

Contexto. Um laboratório quer um modelo robusto a dados incompletos. Você deve construir um pipeline que receba uma tabela "suja" e entregue previsões confiáveis.

Atenção ética. Os dados são de uso didático. O modelo **não é diagnóstico médico**.

Tarefas.

1. Carregue os dados em um `DataFrame`. Crie duas colunas categóricas de texto: "tamanho" (a coluna `mean radius` dividida em três faixas: pequeno, médio, grande) e "textura" (a coluna `mean texture` dividida em três faixas). Remova as colunas numéricas originais usadas para criá-las. Apague 10% dos valores ao acaso nas demais colunas numéricas.
2. Separe 25% para **teste final** (estratificado, semente fixa).
3. Monte um `ColumnTransformer` com tratamentos separados para colunas numéricas (imputação e padronização) e categóricas (imputação e one-hot).
4. Compare, por validação cruzada no treino, três modelos no mesmo pré-processamento: regressão logística, k-vizinhos e floresta aleatória. Escolha o melhor.
5. Faça uma busca em grade **dentro do pipeline** do melhor modelo, incluindo a estratégia de imputação (média ou mediana).
6. Avalie **uma única vez** no teste final. Salve o pipeline com `joblib`, recarregue-o e preveja três linhas novas, **uma delas com uma categoria nunca vista** (por exemplo, "gigante").
7. Escreva um parágrafo (6 a 8 linhas): o que o pipeline resolveu? Que cuidado evitou vazamento? O que ainda exigiria atenção humana?

Entregáveis. Um notebook (ou script) comentado, o pipeline salvo e o parágrafo de conclusões.

A solução de referência está no Apêndice E.

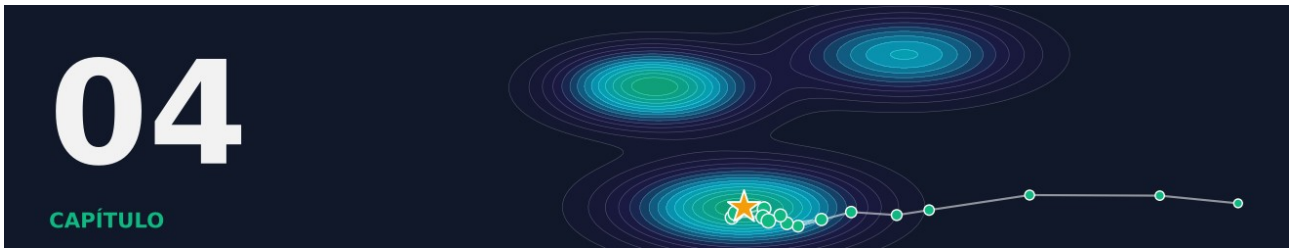
PARTE II

Parte II — Otimização de Hiperparâmetros

O coração do AutoML: como buscar bons ajustes de forma inteligente. Da busca em grade e aleatória à otimização bayesiana, ao Hyperband e aos métodos evolutivos.

Nesta parte:

- Cap. 4 — Busca em grade e aleatória
- Cap. 5 — Otimização bayesiana
- Cap. 6 — Métodos multifidelidade: Successive Halving, Hyperband e BOHB
- Cap. 7 — Métodos evolutivos e programação genética



Capítulo 4 — Busca em grade e aleatória

OBJETIVOS DE APRENDIZAGEM

Ao final deste capítulo você será capaz de:

- Definir hiperparâmetros e descrever os principais tipos de espaço de busca (contínuo, inteiro, categórico, em escala logarítmica e condicional).
- Executar uma busca em grade e uma busca aleatória com o scikit-learn.
- Explicar por que a busca em grade sofre com muitos hiperparâmetros e por que a aleatória costuma ser mais eficiente.
- Interpretar a "paisagem" de hiperparâmetros e os retornos decrescentes de buscar mais.
- Aplicar a validação aninhada para estimar o desempenho de todo o processo de busca.
- Identificar quais hiperparâmetros mais importam em um modelo.

4.1 Os botões do algoritmo

Este é o primeiro capítulo da Parte II, que trata do **coração do AutoML**: a otimização de hiperparâmetros (em inglês, *hyperparameter optimization*, ou **HPO**).

Lembre-se: **hiperparâmetros** são os botões do algoritmo, definidos *antes* do treino. Cada algoritmo tem os seus.

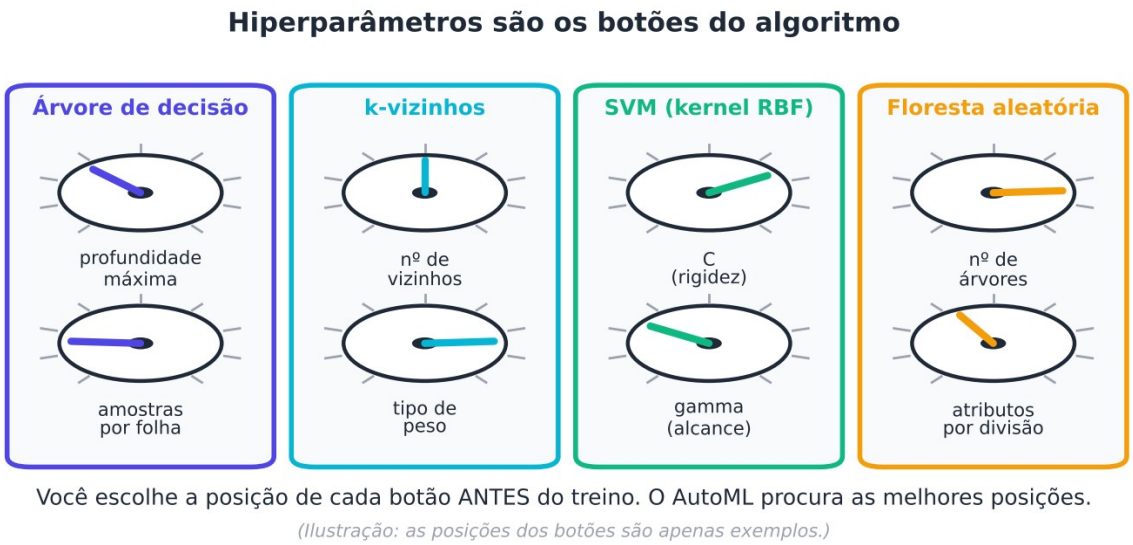


Figura 4.1 — Cada algoritmo tem botões próprios. A posição de cada botão influencia muito o resultado.

Analogia: um rádio antigo. As músicas (os dados) estão no ar. Se você deixar o botão de sintonia na posição errada, só ouvirá chiado. O trabalho do HPO é achar a posição certa de **todos os botões ao mesmo tempo**.

O impacto é grande. Em um mesmo conjunto de dados, o mesmo algoritmo pode ter 90% ou 99% de acerto, dependendo apenas dos hiperparâmetros. Por isso, a literatura trata o HPO como peça central do AutoML (Bischl *et al.*, 2023).

4.2 O espaço de busca

Para buscar, precisamos dizer **quais valores são permitidos** para cada botão. Essa descrição é o **espaço de busca**. Há vários tipos de hiperparâmetro.

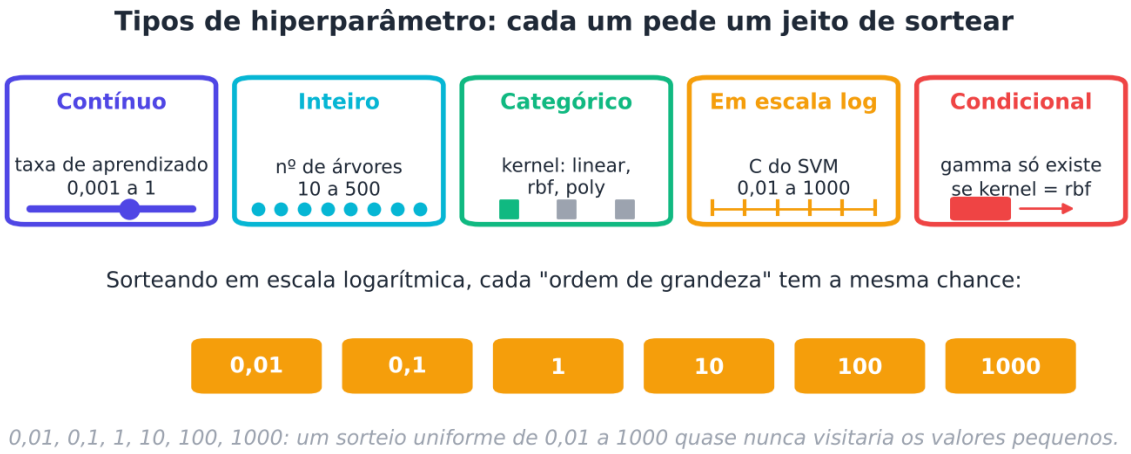


Figura 4.2 — Tipos de hiperparâmetro. Cada tipo pede um jeito diferente de sortear valores.

- **Contínuo:** qualquer número em um intervalo (taxa de aprendizado entre 0,001 e 1).
- **Inteiro:** números inteiros (número de árvores, de 10 a 500).
- **Categórico:** uma escolha entre opções (tipo de kernel: linear, rbf ou poly).
- **Em escala logarítmica:** valores que variam em ordens de grandeza (como o *C* do SVM).
- **Condicional:** existe apenas se outra escolha foi feita (o *gamma* só importa se o kernel é rbf).

A escala logarítmica merece atenção

Imagine sortear o *C* de um SVM entre 0,01 e 1000. Se o sorteio for **uniforme**, quase todos os valores caem acima de 1 (pois 99,9% do intervalo está entre 1 e 1000). Os valores pequenos, que podem ser os melhores, quase nunca aparecem.

Na **escala logarítmica**, cada ordem de grandeza (0,01 a 0,1; 0,1 a 1; 1 a 10...) tem a **mesma chance**. É a escolha certa para hiperparâmetros como *C*, *gamma* e taxas de aprendizado.

```python
import numpy as np
from scipy.stats import loguniform, uniform

rng = np.random.default_rng(0)
uni = uniform(0.01, 1000 - 0.01).rvs(10000, random_state=0)
log = loguniform(0.01, 1000).rvs(10000, random_state=0)

print("Fração dos valores abaixo de 1:")
print(f"  sorteio uniforme    : {np.mean(uni < 1):.3f}")
```

```
print(f"  sorteio logarítmico : {np.mean(log < 1):.3f}")
```

SAÍDA

```
Fração dos valores abaixo de 1:
  sorteio uniforme    : 0.001
  sorteio logarítmico : 0.403
```

4.3 Busca em grade (grid search)

A ideia mais direta é a **busca em grade**: escolher alguns valores para cada botão e **testar todas as combinações**, como numa tabela de multiplicar.

Usaremos o conjunto **Digits** (UCI): 1.797 imagens 8×8 de dígitos manuscritos, com 64 atributos e 10 classes (Dua; Graff, 2019). O modelo será um **SVM** (máquina de vetores de suporte) com kernel RBF, que tem dois botões importantes:

- **C:** o quanto o modelo é "rígido" para acertar o treino (valores altos = menos tolerância a erros).
- **gamma:** o "alcance" de cada exemplo (valores altos = influência bem local).

PYTHON

```python
import numpy as np
from sklearn.datasets import load_digits
from sklearn.svm import SVC
from sklearn.model_selection import GridSearchCV, StratifiedKFold

X, y = load_digits(return_X_y=True)
X = X / 16.0                                  # pixels vão de 0 a 16: colocamos entre 0 e 1
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=0)

grade = {"C": [0.1, 1, 10, 100], "gamma": [0.0001, 0.001, 0.01, 0.1]}
busca = GridSearchCV(SVC(), grade, cv=cv).fit(X, y)

print("Combinações testadas:", len(busca.cv_results_["params"]))
print("Treinos realizados  :", len(busca.cv_results_["params"]) * 5, "(16 combinações x 5 partes)")
print("Melhor combinação   :", busca.best_params_)
print("Melhor acurácia     :", round(busca.best_score_, 3))
```

SAÍDA

```
Combinações testadas: 16
Treinos realizados  : 80 (16 combinações x 5 partes)
Melhor combinação   : {'C': 10, 'gamma': 0.1}
Melhor acurácia     : 0.988
```

O `GridSearchCV` faz tudo: testa cada combinação, avalia por validação cruzada e guarda a melhor. Note quantos treinos foram feitos: combinações × partes da validação cruzada.

4.4 O custo da grade

A grade parece inocente, mas tem um defeito grave: o número de combinações **cresce de forma multiplicativa**. Com *d* hiperparâmetros e *k* valores cada, são *k* elevado a *d* combinações.

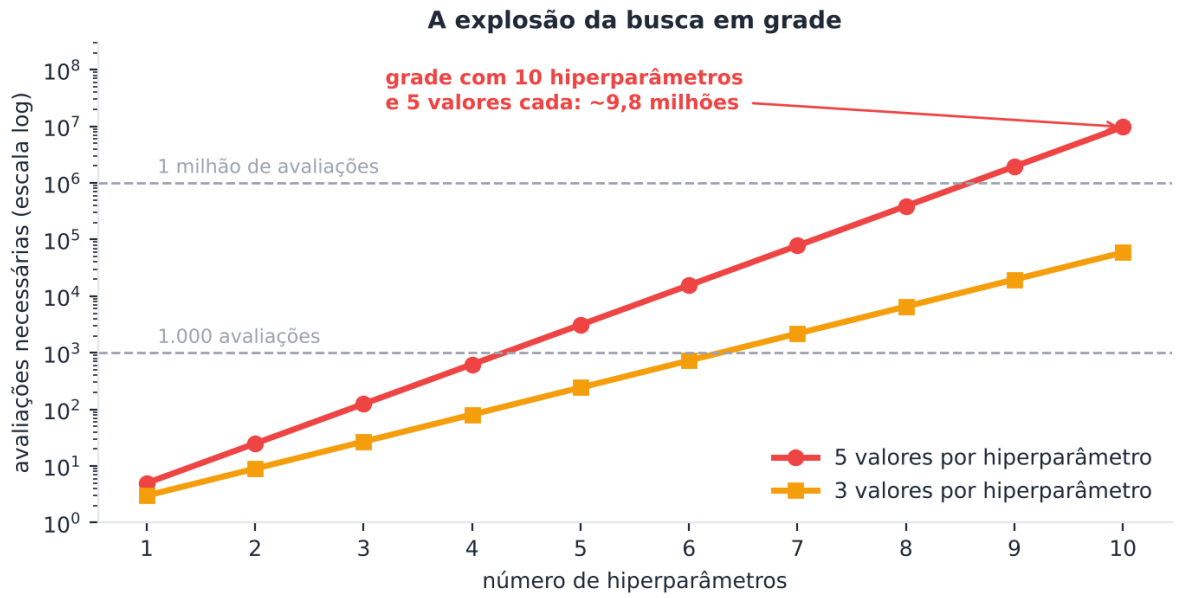


Figura 4.3 — O número de avaliações cresce exponencialmente com o número de hiperparâmetros.

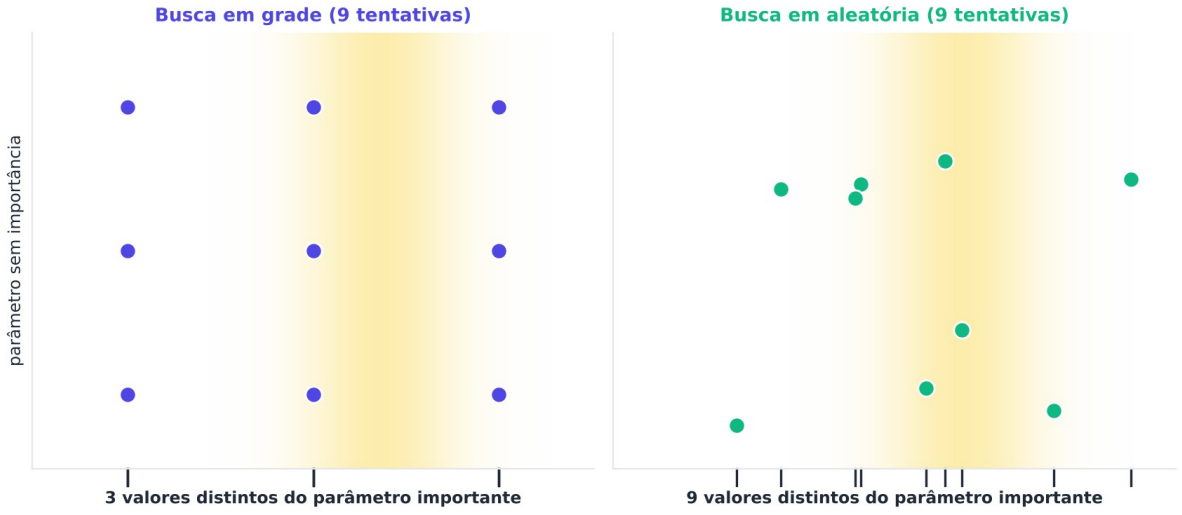
Com 5 valores em cada um de 10 hiperparâmetros, a conta passa de **9 milhões** de avaliações. Mesmo com 3 valores por hiperparâmetro, são dezenas de milhares. Isso é a **maldição da dimensionalidade** aplicada à busca.

Há ainda um segundo problema, mais sutil. Na grade, os mesmos poucos valores de cada botão se repetem muitas vezes. Se um botão for **irrelevante**, você gastou 3 vezes mais para testar os mesmos 3 valores dos botões importantes.

4.5 Busca aleatória (random search)

A alternativa é surpreendentemente simples: **sortear** combinações ao acaso, dentro do espaço de busca, e testar quantas o orçamento permitir. É a **busca aleatória**.

Em um estudo clássico, Bergstra e Bengio (2012) mostraram que a busca aleatória encontra modelos tão bons ou melhores que a grade, em uma fração do tempo de computação. A razão é intuitiva, e eles a demonstraram: na maioria dos conjuntos de dados, **só poucos hiperparâmetros realmente importam**, e os importantes variam de problema para problema.



Horizontal = parâmetro que importa (faixa amarela é a região boa). Vertical = parâmetro quase irrelevante.

Figura 4.4 — Com 9 tentativas, a grade testa apenas 3 valores do parâmetro importante. A busca aleatória testa 9 valores distintos.

Veja a figura: na grade, o botão importante (eixo horizontal) é testado em só 3 posições. Na aleatória, cada tentativa traz um valor novo. Com o mesmo orçamento, a aleatória "explora" muito mais do que importa.

Analogia: você procura o ponto ideal de sintonia de um rádio. Testar "10, 50 e 90" repetidas vezes é pior que girar o botão em posições sempre diferentes.

No scikit-learn, usamos `RandomizedSearchCV`, com **distribuições** em vez de listas fixas.

PYTHON

```python
from scipy.stats import loguniform
from sklearn.model_selection import RandomizedSearchCV

espaco = {"C": loguniform(1e-2, 1e3), "gamma": loguniform(1e-5, 1)}
aleatoria = RandomizedSearchCV(SVC(), espaco, n_iter=16, cv=cv, random_state=0).fit(X, y)

print("Combinações testadas:", len(aleatoria.cv_results_["params"]), "(mesmo orçamento da grade)")
print("Melhor acurácia     :", round(aleatoria.best_score_, 3))
print("Melhor C e gamma    : C = %.2f, gamma = %.4f" % (aleatoria.best_params_["C"],
aleatoria.best_params_["gamma"]))
```

SAÍDA

```
Combinações testadas: 16 (mesmo orçamento da grade)
Melhor acurácia     : 0.988
Melhor C e gamma    : C = 6.92, gamma = 0.4246
```

4.6 A paisagem de hiperparâmetros

Para enxergar o que está acontecendo, avaliamos o SVM em uma **malha fina** de valores de *C* e *gamma* e colorimos o resultado. O mapa abaixo é a **paisagem de hiperparâmetros**. Cada ponto é uma combinação; a cor é a acurácia.

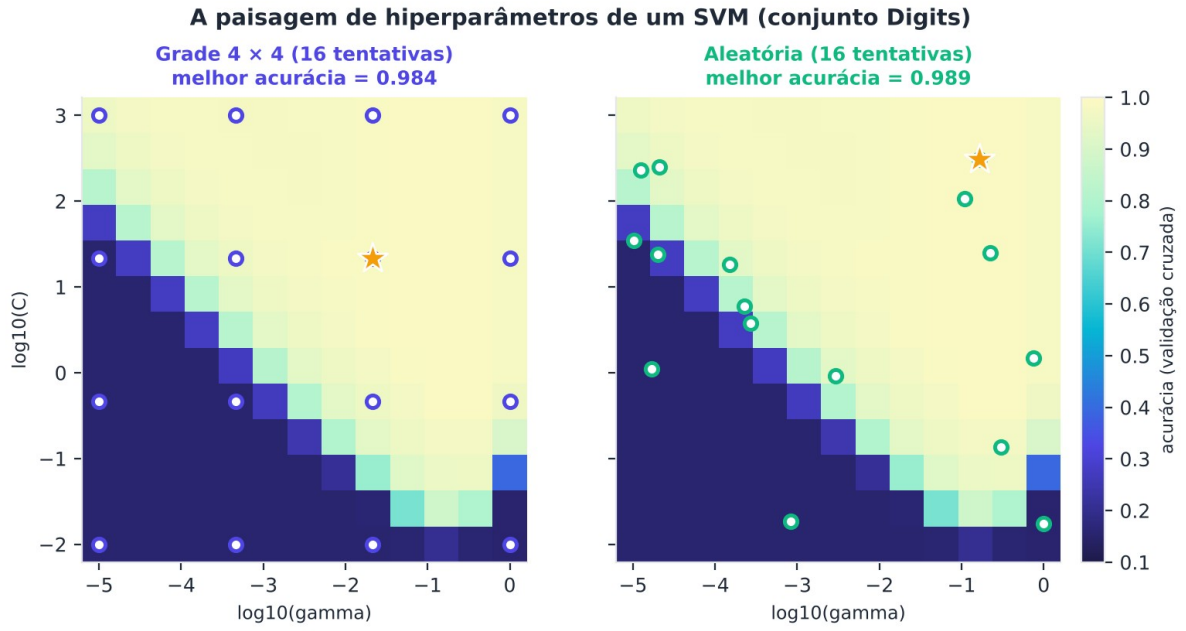


Figura 4.5 — A paisagem de hiperparâmetros de um SVM no conjunto Digits (calculada de verdade). À esquerda, uma grade de 16 pontos; à direita, 16 pontos aleatórios. A estrela marca o melhor de cada busca.

Repare em dois fatos:

1. Há uma região ampla com ótimos resultados (amarelo) e uma região muito ruim (azul-escuro), onde o *gamma* é grande demais ou o *C* é pequeno demais.
2. A grade, por ter valores "redondos" e repetidos, pode passar ao lado do melhor ponto. A busca aleatória espalha as tentativas por posições distintas.

Um experimento justo

Um único exemplo pode ter sido sorte. Por isso, vamos comparar de forma mais cuidadosa, com um terceiro botão **quase irrelevante** (`tol`, a tolerância de parada). A grade usa 3 valores de cada botão (27 combinações). A busca aleatória faz 27 tentativas e é repetida com 8 sementes diferentes.

PYTHON

```python
import numpy as np
from sklearn.datasets import load_digits
from sklearn.svm import SVC
from sklearn.model_selection import cross_val_score, StratifiedKFold

X, y = load_digits(return_X_y=True); X = X / 16.0
cv = StratifiedKFold(5, shuffle=True, random_state=0)
nota = lambda C, g, tol: cross_val_score(SVC(C=C, gamma=g, tol=tol), X, y, cv=cv).mean()

# Grade: 3 valores de cada botão = 27 combinações
melhor_grade = max(nota(C, g, t) for C in [0.1, 10, 1000] for g in [1e-4, 1e-3, 1e-2] for t in [1e-4,
1e-3, 1e-2])

# Aleatória: 27 sorteios, repetida com 8 sementes
resultados = []
for semente in range(8):
    rng = np.random.default_rng(semente)
    resultados.append(max(nota(10 ** rng.uniform(-2, 3), 10 ** rng.uniform(-5, 0), 10 ** rng.uniform(-
4, -2)) for _ in range(27)))

print(f"Grade (27 tentativas)         : melhor = {melhor_grade:.3f}")
print(f"Aleatória (27 tentativas) x 8 : melhor = {np.mean(resultados):.3f} em média | pior das 8 =
{min(resultados):.3f}")
```

SAÍDA

```
Grade (27 tentativas)         : melhor = 0.983
Aleatória (27 tentativas) x 8 : melhor = 0.989 em média | pior das 8 = 0.988
```

Com o mesmo orçamento de 27 avaliações, **todas as 8 repetições** da busca aleatória superaram a grade. É o padrão previsto por Bergstra e Bengio (2012): como o botão `tol` quase não importa, a grade desperdiça 2/3 do orçamento repetindo os mesmos pares de *C* e *gamma*.

> **Atenção**
>
> Isso **não** quer dizer que a busca aleatória sempre vence. Se houver poucos hiperparâmetros e você souber exatamente em que faixa procurar, uma grade fina pode ser perfeita. A grade também é ótima para **explorar e entender**, pois produz tabelas organizadas. Mas, em espaços maiores, ela perde.

4.7 Quanto orçamento é suficiente?

Quantas tentativas fazer? A resposta é: **depende, mas os ganhos diminuem rápido**. Veja a figura, construída sorteando configurações da paisagem acima.

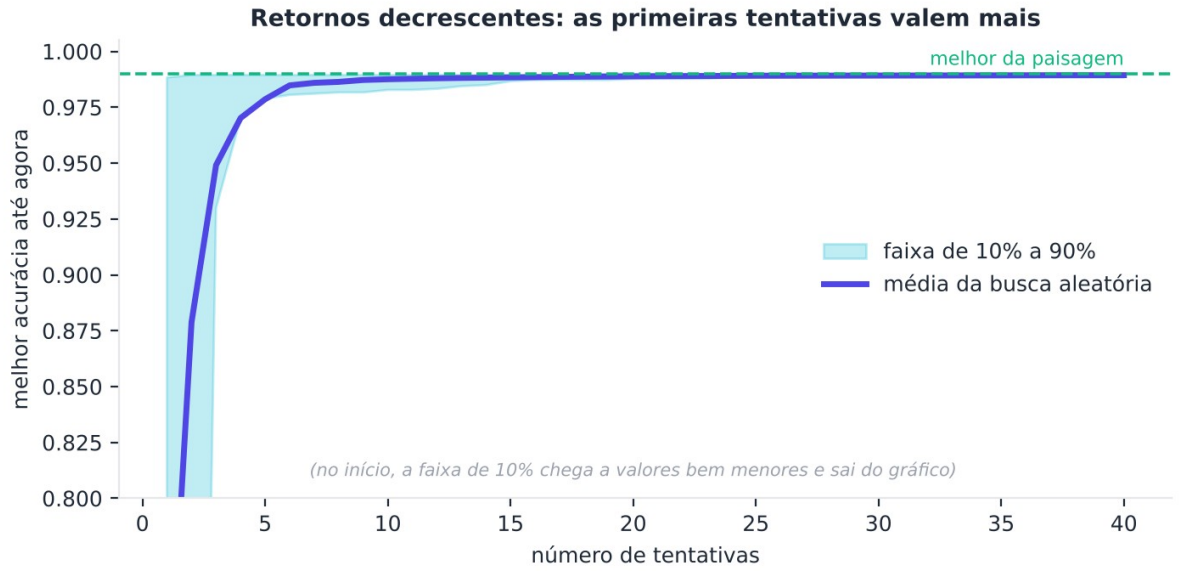


Figura 4.6 — Retornos decrescentes: a melhor nota até agora sobe muito nas primeiras tentativas e depois quase não muda.

Esse formato é típico: as **primeiras tentativas** trazem a maior parte do ganho. Depois, cada nova tentativa traz muito pouco. Isso tem uma consequência prática importante: **definir um orçamento** (de tempo ou de número de tentativas) e **parar** quando as melhorias se tornam ínfimas.

Duas dicas para economizar:

- **Paralelismo:** as tentativas da busca aleatória são independentes, então podem rodar ao mesmo tempo. Em `GridSearchCV` e `RandomizedSearchCV`, o argumento `n_jobs=-1` usa todos os núcleos do computador.
- **Começar com defaults:** meça primeiro o desempenho com os valores padrão da biblioteca. Muitas vezes, a distância até o melhor ajuste é menor do que se imagina, o que ajuda a decidir se vale buscar (Probst; Boulesteix; Bischl, 2019).

4.8 O perigo escondido: o otimismo da busca

Lembra do "efeito do vencedor" (Capítulo 2)? A busca testa muitas configurações e fica com a **melhor nota de validação**. Essa nota é, em média, **otimista**: o vencedor foi escolhido justamente por ter tido sorte na validação.

Como medir com honestidade o desempenho **de todo o processo de busca**? Com a **validação aninhada** (*nested cross-validation*), proposta como remédio para esse viés (Varma; Simon, 2006; Cawley; Talbot, 2010).



Figura 4.7 — Na validação aninhada, o laço externo mede e o laço interno escolhe. O teste do laço externo nunca participa da escolha.

Funciona assim:

1. O **laço externo** divide os dados em partes de treino e teste.
2. Para cada divisão externa, o **laço interno** roda a busca de hiperparâmetros **apenas dentro do treino externo**.
3. O melhor modelo da busca é avaliado na parte de teste externa, que ele nunca viu.
4. A nota final é a média dos testes externos.

No scikit-learn, a validação aninhada é elegante: passamos o próprio objeto de busca para `cross_val_score`.

Vamos medir o otimismo em um cenário com **poucos dados** (80 exemplos do conjunto Breast Cancer Wisconsin, UCI), onde o problema é mais forte. Repetimos com 6 sementes.

```python
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.svm import SVC
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import GridSearchCV, cross_val_score, StratifiedKFold, train_test_split

X0, y0 = load_breast_cancer(return_X_y=True)
grade = {"svc__C": np.logspace(-2, 3, 8), "svc__gamma": np.logspace(-5, 0, 8)}   # 64 combinações

comum, aninhada = [], []
for s in range(6):
    X, _, y, _ = train_test_split(X0, y0, train_size=80, stratify=y0, random_state=s)
    pipe = make_pipeline(StandardScaler(), SVC())
    interno = StratifiedKFold(5, shuffle=True, random_state=s)
    externo = StratifiedKFold(5, shuffle=True, random_state=100 + s)
    comum.append(GridSearchCV(pipe, grade, cv=interno, n_jobs=-1).fit(X, y).best_score_)
    aninhada.append(cross_val_score(GridSearchCV(pipe, grade, cv=interno, n_jobs=-1), X, y,
cv=externo).mean())

print(f"Nota da busca (best_score_)  : {np.mean(comum):.3f}")
print(f"Nota aninhada (honesta)      : {np.mean(aninhada):.3f}")
```

```
print(f"Otimismo médio da busca      : {np.mean(comum) - np.mean(aninhada):+.3f}")
```

SAÍDA

```
Nota da busca (best_score_)  : 0.979
Nota aninhada (honesta)      : 0.950
Otimismo médio da busca      : +0.029
```

A nota "comum" da busca é **mais alta** que a aninhada, ou seja, otimista. Quanto **menos dados** e **mais combinações** testadas, maior tende a ser essa diferença.

> **Memorize**
>
> `best_score_` mede quão bem o **vencedor** se saiu na validação que o elegeu. A **validação aninhada** (ou um teste final intocado) mede quão bem **o processo inteiro** generaliza. Relate a segunda.

A validação aninhada tem um preço: é muito mais cara (multiplica o trabalho pelo número de partes do laço externo). Por isso, na prática, muitos projetos usam um **teste final separado**, como no Capítulo 2. A ideia é a mesma: a avaliação final não pode participar da busca.

4.9 Quais hiperparâmetros importam?

Nem todo botão pesa igual. Descobrir quais importam economiza orçamento: você pode **fixar** os irrelevantes e **concentrar** a busca nos importantes. Probst, Boulesteix e Bischl (2019) estudaram isso formalmente em seis algoritmos e dezenas de conjuntos, propondo medidas de "ajustabilidade" (*tunability*) de cada hiperparâmetro.

Uma forma simples de investigar é aproveitar os resultados de uma busca aleatória. Calculamos, para cada hiperparâmetro, o quanto ele se **relaciona** com a nota (correlação de postos de Spearman).

PYTHON

```python
import numpy as np
import pandas as pd
from scipy.stats import spearmanr, randint, uniform
from sklearn.datasets import load_digits
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import RandomizedSearchCV

X, y = load_digits(return_X_y=True)
espaco = {"n_estimators": randint(10, 200), "max_depth": randint(2, 20),
          "max_features": uniform(0.05, 0.9), "min_samples_leaf": randint(1, 15)}
busca = RandomizedSearchCV(RandomForestClassifier(random_state=0, n_jobs=-1), espaco, n_iter=40, cv=3,
random_state=0).fit(X, y)

res = pd.DataFrame(busca.cv_results_)
print("Correlação de Spearman entre cada hiperparâmetro e a acurácia:")
for h in espaco:
    rho = spearmanr(res["param_" + h].astype(float), res["mean_test_score"])[0]
    print(f"  {h:17s} rho = {rho:+.2f}")
print(f"Acurácia: mínima = {res['mean_test_score'].min():.3f} | máxima =
{res['mean_test_score'].max():.3f}")
```

SAÍDA

```
Correlação de Spearman entre cada hiperparâmetro e a acurácia:
  n_estimators      rho = +0.10
```

```
  max_depth         rho = +0.69
  max_features      rho = -0.43
  min_samples_leaf  rho = -0.05
Acurácia: mínima = 0.595 | máxima = 0.931
```

Hiperparâmetros com correlação perto de zero parecem pouco importantes **nesse problema**; os de correlação forte merecem atenção. No resultado acima, a profundidade máxima foi a mais associada à nota (ρ ≈ +0,69), seguida de `max_features` (ρ ≈ −0,43: valores menores foram melhores). Número de árvores e tamanho mínimo da folha pesaram pouco. Só com esses botões, a acurácia variou de 0,595 a 0,931. Cuidado: é uma análise simples, e os efeitos de vários hiperparâmetros podem se combinar. Métodos mais rigorosos existem, e o resultado depende do conjunto de dados.

4.10 Boas práticas

1. **Comece pelo default** e por uma **linha de base**. Saiba de onde está partindo.
2. **Use escala logarítmica** para hiperparâmetros como *C*, *gamma* e taxas de aprendizado.
3. **Prefira a busca aleatória à grade** em espaços com mais de 2 ou 3 hiperparâmetros (Bergstra; Bengio, 2012).
4. **Defina um orçamento** e pare quando os ganhos forem mínimos.
5. **Coloque o pré-processamento dentro do pipeline** (Capítulo 3) e busque sobre o pipeline inteiro.
6. **Meça com honestidade:** teste final intocado ou validação aninhada.
7. **Registre tudo:** o `cv_results_` guarda cada tentativa. Salve-o.
8. **Repita com sementes diferentes** e relate a variação.

A busca aleatória é também a **linha de base obrigatória** de qualquer método de HPO mais sofisticado. Nos próximos capítulos, você vai conhecer métodos que fazem melhor, usando o que aprenderam nas tentativas anteriores para decidir a próxima.

Resumo do capítulo

MEMORIZE **Cartões de memorização**

- **HPO** = achar boas posições para todos os botões ao mesmo tempo.
- **Tipos de espaço:** contínuo, inteiro, categórico, logarítmico e condicional.
- **Escala logarítmica** para *C*, *gamma* e taxas de aprendizado.
- **Grade:** testa todas as combinações. Custo exponencial; repete valores.
- **Aleatória:** sorteia combinações. Explora mais valores do que importa (Bergstra; Bengio, 2012).
- **Retornos decrescentes:** as primeiras tentativas trazem o maior ganho.
- **Otimismo da busca:** a nota do vencedor é inflada pela seleção.
- **Validação aninhada** (ou teste final) mede o processo inteiro com honestidade.
- **Nem todo botão importa:** descubra quais e concentre o orçamento.

Exercícios de múltipla escolha

Escolha **uma** alternativa em cada questão. O gabarito está no Apêndice A.

Questão 1. HPO é a sigla de:

a) Hyperparameter Optimization (otimização de hiperparâmetros).
b) Hybrid Prediction Operator.
c) Hidden Parameter Observation.
d) High Performance Output.

Questão 2. Qual das opções é um hiperparâmetro do SVM com kernel RBF?

a) A classe prevista para um exemplo.
b) Os vetores de suporte aprendidos.
c) Os coeficientes do hiperplano.
d) O valor de C.

Questão 3. Em um espaço de busca, um hiperparâmetro condicional é aquele que:

a) não pode ser otimizado.
b) só existe ou só faz sentido se outra escolha foi feita.
c) assume apenas valores negativos.
d) é sempre inteiro.

Questão 4. Por que sortear o valor de C de um SVM em escala logarítmica?

a) Porque cada ordem de grandeza passa a ter a mesma chance de ser testada.
b) Porque isso elimina o sobreajuste.
c) Porque assim os valores ficam sempre acima de 1.
d) Porque o scikit-learn exige.

Questão 5. No experimento do capítulo, qual fração dos valores sorteados de forma uniforme entre 0,01 e 1000 ficou abaixo de 1?

a) Cerca de 50%.
b) Cerca de 40%.
c) Cerca de 90%.
d) Cerca de 0,1% (praticamente nenhum).

Questão 6. A busca em grade (grid search):

a) usa os resultados anteriores para decidir a próxima tentativa.
b) testa todas as combinações de valores previamente escolhidos.
c) elimina a necessidade de validação.
d) sorteia combinações ao acaso.

Questão 7. Se há 4 hiperparâmetros com 5 valores cada, quantas combinações a grade testa?

a) 1.024.
b) 120.
c) 20.
d) 625.

Questão 8. Qual é o principal problema da busca em grade com muitos hiperparâmetros?

a) Ela sempre sobreajusta.
b) Ela não consegue testar valores inteiros.
c) Ela só funciona com árvores.
d) O número de combinações cresce exponencialmente.

Questão 9. Segundo Bergstra e Bengio (2012), por que a busca aleatória costuma superar a grade?

a) Porque o sorteio é sempre mais preciso.
b) Porque a grade não aceita escala logarítmica.
c) Porque a aleatória usa menos memória.
d) Porque poucos hiperparâmetros realmente importam, e a aleatória testa mais valores distintos deles.

Questão 10. Na busca em grade, se um dos botões for irrelevante, o que acontece?

a) O orçamento é desperdiçado, pois os mesmos valores dos botões importantes se repetem.
b) O botão é ignorado automaticamente.
c) A grade falha.
d) A busca fica mais rápida.

Questão 11. Qual classe do scikit-learn realiza busca aleatória com validação cruzada?

a) GridSearchCV.
b) RandomizedSearchCV.
c) RandomForestClassifier.
d) StratifiedKFold.

Questão 12. No RandomizedSearchCV, o argumento n_iter define:

a) o número de configurações sorteadas.
b) o número de núcleos usados.
c) o número de partes da validação cruzada.
d) o número de árvores da floresta.

Questão 13. A distribuição loguniform(1e-2, 1e3) do scipy serve para:

a) sortear categorias.
b) sortear valores em escala logarítmica entre 0,01 e 1000.
c) calcular logaritmos.
d) sortear inteiros.

Questão 14. A "paisagem de hiperparâmetros" é:

a) um mapa que relaciona combinações de hiperparâmetros ao desempenho.
b) a tabela de dados de treino.
c) a lista de bibliotecas instaladas.
d) um tipo de gráfico de dispersão dos rótulos.

Questão 15. No mapa de Digits com SVM, a região de resultados muito ruins corresponde a:

a) C muito grande e gamma muito pequeno.
b) gamma grande demais ou C pequeno demais.
c) C e gamma ambos intermediários.
d) qualquer valor de C.

Questão 16. No experimento com 27 avaliações, o botão tol era quase irrelevante. O que o resultado mostrou?

a) A grade venceu todas as repetições.
b) A aleatória superou a grade em todas as 8 repetições.
c) A aleatória ficou pior que a grade.
d) Grade e aleatória empataram exatamente.

Questão 17. O que significam "retornos decrescentes" na busca de hiperparâmetros?

a) O modelo piora na validação.
b) As primeiras tentativas trazem a maior parte do ganho; depois, cada tentativa adiciona pouco.
c) A nota cai a cada tentativa.
d) O tempo de busca diminui.

Questão 18. Qual argumento permite paralelizar a busca usando todos os núcleos do computador no scikit-learn?

a) n_jobs=-1.
b) random_state=-1.
c) cv=-1.
d) scoring=-1.

Questão 19. Por que a nota best_score_ de uma busca tende a ser otimista?

a) Porque o vencedor foi escolhido justamente por ter a melhor nota na validação, o que inclui sorte.
b) Porque o teste final contamina a busca.
c) Porque a busca usa dados do futuro.
d) Porque a validação cruzada sempre erra para cima.

Questão 20. Na validação aninhada, o laço interno serve para:

a) embaralhar os rótulos.
b) padronizar os dados.
c) escolher os hiperparâmetros dentro do treino de cada rodada externa.
d) medir o desempenho final.

Questão 21. Na validação aninhada, o que o laço externo faz?

a) Padroniza as colunas.
b) Ajusta os hiperparâmetros.
c) Estima o desempenho do processo inteiro, usando partes de teste que não participaram da busca.
d) Treina vários algoritmos.

Questão 22. Como se faz a validação aninhada de forma simples no scikit-learn?

a) Usando DummyClassifier.
b) Usando duas vezes o train_test_split sem buscar.
c) Aumentando n_jobs.
d) Passando um objeto GridSearchCV ao cross_val_score.

Questão 23. No experimento com 80 exemplos, a nota aninhada em relação à nota comum da busca foi:

a) impossível de calcular.
b) menor, indicando otimismo da busca comum.
c) maior.
d) igual.

Questão 24. Quando o otimismo da busca tende a ser maior?

a) Com muitos dados e poucas combinações.
b) Com muitos dados e muitas combinações.
c) Com poucos dados e muitas combinações.
d) Nunca varia.

Questão 25. Qual é a principal desvantagem da validação aninhada?

a) Ela produz notas otimistas.
b) Ela é muito mais cara computacionalmente.
c) Ela não funciona com SVM.
d) Ela exige rótulos aleatórios.

Questão 26. Uma alternativa prática à validação aninhada é:

a) reduzir o número de dados.
b) padronizar o alvo.
c) separar um teste final intocado, usado só no fim.
d) usar o teste final apenas ao longo da busca.

Questão 27. Em Probst, Boulesteix e Bischl (2019), o conceito de "ajustabilidade" (tunability) refere-se a:

a) quanto o desempenho pode melhorar ao ajustar um hiperparâmetro em relação a um valor padrão.
b) o número de classes do problema.
c) o tamanho do conjunto de teste.
d) o tempo de treino de um modelo.

Questão 28. Qual é uma boa prática ao iniciar um projeto de HPO?

a) Ignorar a escala logarítmica.
b) Começar pelo default e por uma linha de base.
c) Começar direto por uma grade enorme.
d) Usar apenas o teste final para escolher.

Questão 29. Qual é o papel da busca aleatória em relação a métodos mais sofisticados de HPO?

a) Ela é sempre pior e dispensável.
b) Ela não serve como comparação.
c) Ela só se aplica a redes neurais.
d) Ela é uma linha de base obrigatória.

Questão 30. Em RandomizedSearchCV, qual é a vantagem de passar distribuições (como loguniform) em vez de listas fixas?

a) Elimina a necessidade de validação cruzada.
b) Permite sortear valores novos a cada tentativa, em vez de repetir uma lista curta.
c) Reduz o número de dados.
d) Garante que o melhor valor seja encontrado.

Exercícios discursivos

Responda com suas palavras, em poucas linhas. Compare depois com a resposta-modelo do Apêndice A.

1. O que são hiperparâmetros e por que a posição de todos os botões deve ser buscada em conjunto?
2. Descreva os cinco tipos de hiperparâmetro apresentados e dê um exemplo de cada.
3. Explique por que sortear C uniformemente entre 0,01 e 1000 é um erro e como a escala logarítmica o corrige.
4. Explique como funciona a busca em grade e calcule o número de treinos para 3 hiperparâmetros com 4 valores cada, usando validação cruzada de 5 partes.
5. Por que o custo da busca em grade é chamado de explosão combinatória? Dê um exemplo numérico.
6. Explique a ideia de Bergstra e Bengio (2012) sobre a eficiência da busca aleatória.
7. Por que a grade pode ser uma boa escolha em alguns casos? Quando a aleatória é preferível?
8. Descreva o experimento que comparou grade e aleatória com 27 avaliações. O que ele mostrou e por quê?
9. O que é a paisagem de hiperparâmetros e que informações ela fornece sobre o problema?
10. Explique os retornos decrescentes na busca de hiperparâmetros e como isso orienta a definição de orçamento.
11. Por que a nota best_score_ de uma busca é otimista? Relacione com o efeito do vencedor.
12. Explique, passo a passo, como funciona a validação aninhada.
13. Compare a validação aninhada com um teste final separado. Quando usar cada uma?
14. No experimento com 80 exemplos, a nota comum foi 0,979 e a aninhada 0,950. Interprete a diferença.
15. De que fatores depende o tamanho do otimismo da busca?
16. Como identificar quais hiperparâmetros mais importam usando os resultados de uma busca aleatória? Que cuidado tomar?
17. Por que fixar hiperparâmetros pouco importantes pode ser vantajoso para o orçamento?
18. Cite quatro boas práticas de HPO apresentadas no capítulo e justifique uma delas.
19. Por que o pré-processamento deve estar dentro do pipeline durante a busca de hiperparâmetros?
20. Por que a busca aleatória é considerada a linha de base obrigatória para métodos de HPO mais sofisticados?

Exercícios práticos em Python (scikit-learn)

Resolva os exercícios no seu computador (ou no Google Colab). Eles usam conjuntos de dados da UCI que já vêm no scikit-learn ou dados simulados. Tente resolver **antes** de consultar a solução, no Apêndice D. As saídas apresentadas lá foram obtidas com as versões de bibliotecas listadas na Apresentação.

Exercício 1 — Sua primeira busca em grade

NÍVEL: BÁSICO

No conjunto **Wine** (UCI), monte um pipeline com `StandardScaler` e `KNeighborsClassifier`. Use `GridSearchCV` (validação cruzada de 5 partes, estratificada) para testar `n_neighbors` de 1 a 25 (apenas ímpares) e `weights` em `["uniform", "distance"]`. Mostre o número de combinações e de treinos, a melhor combinação e a acurácia dela. Depois, desenhe a nota de validação em função de `n_neighbors` para cada tipo de peso.

Pergunta: a escolha de `weights` fez diferença? Existe uma região ampla de bons valores de `n_neighbors`?

Exercício 2 — Grade ou aleatória? Com a floresta, empatou

NÍVEL: INTERMEDIÁRIO

Com o conjunto **Breast Cancer Wisconsin** (UCI), compare uma **grade** de 8 combinações (`n_estimators` em [20, 60], `max_depth` em [3, None], `max_features` em ["sqrt", 0.5]) com uma **busca aleatória** de 8 tentativas (`n_estimators` inteiro de 10 a 80, `max_depth` inteiro de 2 a 11, `max_features` real de 0,05 a 0,95). Use a mesma validação cruzada e repita a aleatória com 4 sementes.

Pergunta: a aleatória venceu? O que isso diz sobre generalizar a regra "aleatória é melhor que grade"?

Exercício 3 — Escala logarítmica faz diferença

NÍVEL: INTERMEDIÁRIO

Use o conjunto **Wine** e uma `LogisticRegression` (com `StandardScaler`). Faça buscas aleatórias com 10 tentativas para o hiperparâmetro `C`, sorteando em dois espaços: **uniforme** entre 0,001 e 1000 e **logarítmico** (`loguniform`) entre 0,001 e 1000. Repita cada uma com 10 sementes e compare as médias e os piores resultados.

Pergunta: qual espaço funciona melhor e por quê?

Exercício 4 — Medindo o otimismo da busca

NÍVEL: AVANÇADO

Use o conjunto **Wine**, mas com apenas **60 exemplos** (`train_test_split` com `train_size=60`, estratificado, semente variando de 0 a 5). Para cada semente, faça uma busca aleatória de 40 tentativas (SVM com `C` e `gamma` em escala logarítmica, dentro de um pipeline com `StandardScaler`). Registre (a) o `best_score_` da busca e (b) a nota da **validação aninhada** (passe o objeto de busca ao `cross_val_score`). Calcule o otimismo (a − b) em cada semente.

Pergunta: o otimismo foi positivo em todas as sementes? O que isso mostra sobre variabilidade?

> SAIBA MAIS **Para ir além: bases públicas da UCI e do Kaggle**
>
> - **UCI:** Optical Recognition of Handwritten Digits (32×32) e Letter Recognition (letras maiúsculas).
> - **Kaggle:** Digit Recognizer (MNIST), onde você pode repetir o projeto em escala maior.
>
> Estes conjuntos não foram usados nem testados nos códigos do livro. Confira, nos sites, a descrição, a licença e as colunas antes de usá-los.

Projeto integrador do capítulo

Relatório de otimização de hiperparâmetros em Digits

Fonte dos dados. *Optical Recognition of Handwritten Digits* (Digits), do repositório **UCI Machine Learning**, incluído no scikit-learn (`load_digits`).

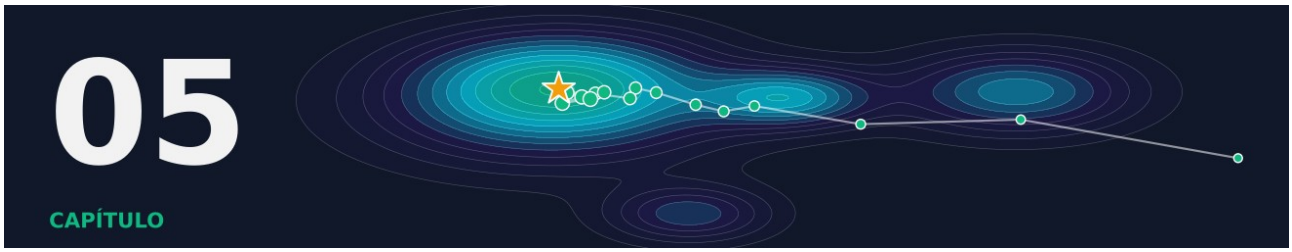
Contexto. Uma equipe quer escolher, entre três algoritmos, o que melhor reconhece dígitos manuscritos, com **orçamento fixo de 12 tentativas por algoritmo**. O relatório deve mostrar como a busca evoluiu e quais hiperparâmetros mais importaram.

Tarefas.

1. Separe 25% dos dados como **teste final** (estratificado, semente fixa). Não toque nele até o fim.
2. Para cada um de três algoritmos (SVM com kernel RBF, floresta aleatória e `HistGradientBoostingClassifier`), defina um **espaço de busca** apropriado (use escala logarítmica quando fizer sentido).
3. Rode `RandomizedSearchCV` com 12 tentativas e validação cruzada de 3 partes, **apenas no treino**.
4. Desenhe um gráfico com a **melhor nota até agora** em função do número de tentativas, uma curva por algoritmo.
5. Para o algoritmo vencedor na validação, calcule a correlação de Spearman entre cada hiperparâmetro e a nota.
6. Avalie o vencedor **uma única vez** no teste final.
7. Escreva um parágrafo (6 a 8 linhas): qual algoritmo venceu? A nota de validação do vencedor é otimista? O orçamento foi suficiente (olhando as curvas)? O que você faria com mais tempo?

Entregáveis. Um notebook (ou script) comentado, o gráfico de convergência e o parágrafo de conclusões.

A solução de referência está no Apêndice E.



Capítulo 5 — Otimização bayesiana

OBJETIVOS DE APRENDIZAGEM

Ao final deste capítulo você será capaz de:

- Explicar por que avaliar hiperparâmetros é caro e como um "modelo substituto" ajuda a economizar avaliações.
- Descrever o equilíbrio entre explorar e aproveitar e o papel da função de aquisição.
- Entender, de forma intuitiva, processos gaussianos, TPE e florestas aleatórias como modelos substitutos.
- Implementar uma otimização bayesiana mínima em Python.
- Usar o Optuna para otimizar hiperparâmetros e até resolver o problema CASH.
- Reconhecer quando a otimização bayesiana ajuda e quando ela não supera a busca aleatória.

5.1 A caixa-preta cara

No Capítulo 4, a busca aleatória sorteava combinações **sem aprender nada com as tentativas anteriores**. É como um garimpeiro que escolhe onde cavar jogando dados, mesmo depois de ter achado ouro em certo lugar. Isso parece desperdício.

Imagine um garimpo em que **cada furo custa caro**: horas de máquina e muito dinheiro. Você não pode furar o terreno inteiro. Depois de alguns furos, você começa a raciocinar: "achei ouro aqui; talvez haja mais por perto. Mas também não conheço aquela região. Vale a pena olhar?"

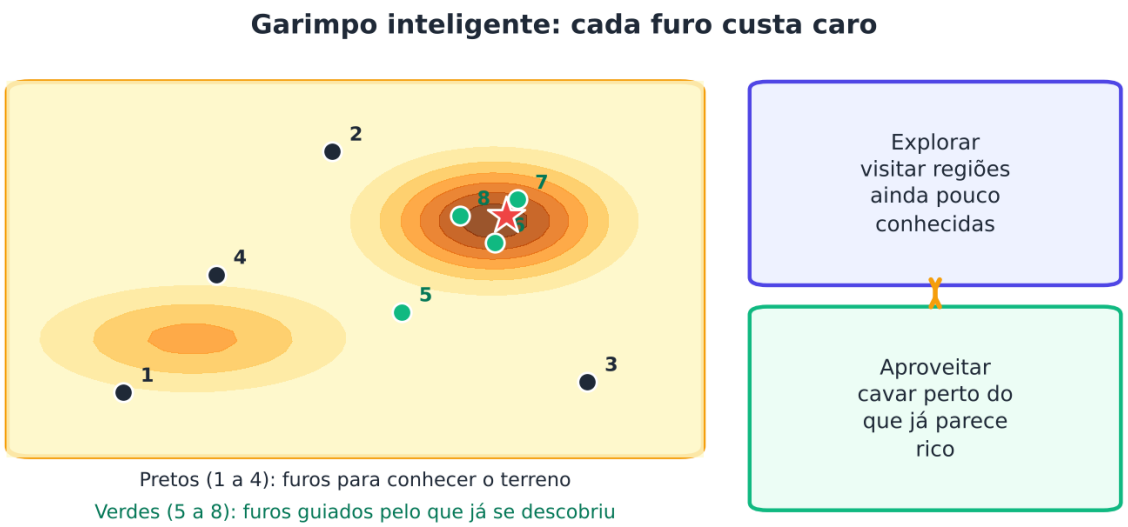


Figura 5.1 — O garimpeiro inteligente decide onde furar com base no que já descobriu. Esse é o espírito da otimização bayesiana.

Em aprendizado de máquina, o "furo" é **treinar um modelo e validá-lo** com certa combinação de hiperparâmetros. Pode levar minutos, horas ou dias. Chamamos a função "hiperparâmetros → nota" de **caixa-preta cara**:

- **caixa-preta**, porque não sabemos a fórmula; só vemos a nota para cada entrada;
- **cara**, porque cada avaliação consome muito tempo.

A **otimização bayesiana** (*Bayesian optimization*, ou BO) é a família de métodos criada para essa situação: **gastar poucas avaliações caras, escolhendo cada uma com inteligência** (Shahriari *et al.*, 2016; Snoek; Larochelle; Adams, 2012).

5.2 A ideia central: um modelo do modelo

A ideia da BO é elegante. Em vez de testar às cegas, construímos um **modelo barato que imita a caixa-preta cara**. Esse modelo é chamado de **modelo substituto** (*surrogate model*).

O modelo substituto aprende com o **histórico** de tentativas: cada par (hiperparâmetros, nota) é um ponto de treino. Depois, ele prevê **como seria a nota** em qualquer outra combinação, sem precisar treinar o modelo de verdade. E, melhor ainda, ele informa **o quanto está incerto** sobre cada previsão.

Com esse "mapa aproximado", escolhemos o próximo ponto a avaliar. O ciclo é este:

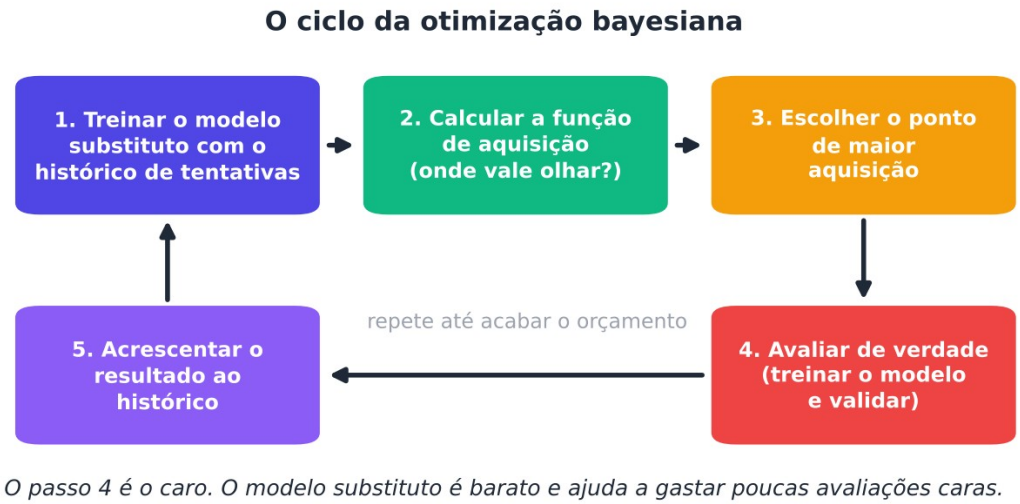


Figura 5.2 — O ciclo da otimização bayesiana. Só o passo 4 é caro; os demais são baratos.

1. Treinar o modelo substituto com o histórico.
2. Calcular a **função de aquisição** (que diz onde vale olhar).
3. Escolher o ponto com maior valor de aquisição.
4. **Avaliar de verdade** esse ponto.
5. Acrescentar o resultado ao histórico e repetir.

5.3 Explorar ou aproveitar?

Toda escolha de "onde olhar" enfrenta um dilema clássico:

- **Aproveitar** (*exploitation*): olhar perto do melhor ponto já encontrado. É seguro, mas pode nos prender em um "morro" pequeno.
- **Explorar** (*exploration*): olhar regiões pouco conhecidas. Arrisca perder uma avaliação, mas pode revelar um "morro" muito maior.

Analogia: você escolhe um restaurante. Pode voltar ao seu favorito de sempre (aproveitar) ou experimentar um novo (explorar). Só aproveitar, e você nunca descobre algo melhor. Só explorar, e você come mal com frequência.

A **função de aquisição** (*acquisition function*) resolve esse dilema com uma conta. Ela dá uma "nota de interesse" para cada ponto, combinando a **previsão** do modelo substituto (aproveitar) com a **incerteza** dele (explorar). O próximo ponto avaliado é o de maior interesse.

5.4 Processos gaussianos: previsão com incerteza

O modelo substituto mais clássico é o **processo gaussiano** (GP, de *Gaussian process*) (Rasmussen; Williams, 2006). Você não precisa dominar a matemática. Guarde esta intuição:

> *Um processo gaussiano é um modelo que, além de **prever um valor**, diz **o quanto confia nessa previsão**. Perto dos pontos já medidos, a incerteza é pequena. Longe deles, a incerteza é grande.*

Isso é exatamente o que precisamos. O GP assume que **pontos próximos têm notas parecidas** (suavidade). A "régua" que define o que é "próximo" é o **kernel**.

A figura abaixo é um **cálculo real**. A função verdadeira (tracejada) tem um morro largo e uma "agulha" estreita, que é o máximo global. O algoritmo não conhece a função. Ele vê apenas os pontos pretos.

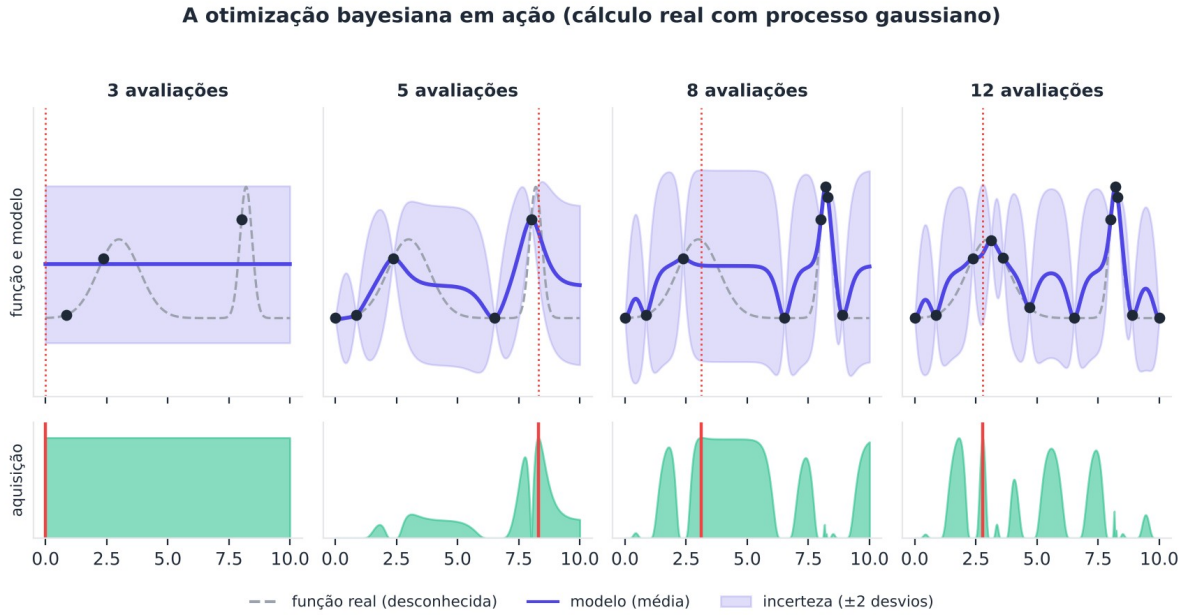


Figura 5.3 — Quatro momentos de uma otimização bayesiana real. A linha azul é a média do modelo; a faixa é a incerteza. A curva verde é a aquisição, e a linha vermelha marca o próximo ponto escolhido.

Observe:

- Com **3 pontos**, o modelo quase nada sabe: a faixa de incerteza é enorme.
- Aos poucos, a incerteza **diminui perto dos pontos medidos** e **continua alta nos espaços vazios**.
- A aquisição (verde) é alta onde a média é alta **ou** a incerteza é grande. Por isso o algoritmo ora refina um morro, ora vai olhar um canto desconhecido.

5.5 Funções de aquisição

Existem várias formas de combinar média e incerteza em uma nota de interesse. As três mais conhecidas:

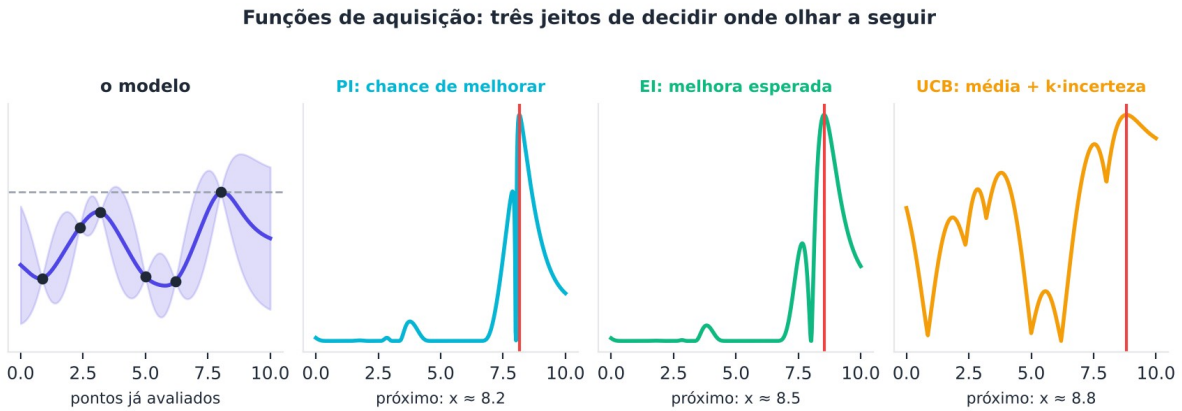


Figura 5.4 — Três funções de aquisição sobre o mesmo modelo. Cada uma sugere um próximo ponto ligeiramente diferente.

- **PI** (probabilidade de melhoria): qual a chance de este ponto superar o melhor atual? Só se importa com *se* melhora, e não com *quanto*.
- **EI** (melhoria esperada): em média, quanto este ponto deve melhorar o melhor atual? Pondera chance e tamanho do ganho. É a mais usada (Jones; Schonlau; Welch, 1998).
- **UCB** (limite superior de confiança): média mais *k* vezes a incerteza. O valor *k* controla o apetite por explorar.

A fórmula da EI parece complicada, mas a ideia é simples. Com média μ, incerteza σ e melhor valor atual f*:

$$\mathrm{EI}(x) = (\mu(x) - f^*)\cdot \Phi(z) + \sigma(x)\cdot \varphi(z), \quad z = (\mu(x) - f^*) / \sigma(x)$$

Em palavras: o primeiro termo premia pontos com **média alta**; o segundo premia pontos com **incerteza alta**. Φ e φ são funções-padrão da curva normal (o "sino"). Você não precisa calculá-las à mão: a biblioteca faz isso.

5.6 Construindo uma otimização bayesiana do zero

Nada melhor do que ver funcionando. Vamos usar a função com "agulha" da figura. Ela é barata de calcular, mas **fingiremos** que cada avaliação é cara, e por isso só permitimos **15 avaliações**. Compararemos a BO (com GP e EI) com a busca aleatória, repetindo 40 vezes para não depender de sorte.

```python
import numpy as np
from scipy.stats import norm
from sklearn.gaussian_process import GaussianProcessRegressor
from sklearn.gaussian_process.kernels import Matern, WhiteKernel, ConstantKernel as Ck

def f(x):                                   # função "cara": um morro largo e uma agulha estreita
    return 0.6 * np.exp(-((x - 3) / 1.2) ** 2) + 1.0 * np.exp(-((x - 8.2) / 0.35) ** 2)

candidatos = np.linspace(0, 10, 501)[:, None]

def otimizacao_bayesiana(semente, n_total=15, n_inicial=3):
    rng = np.random.default_rng(semente)
    X = list(rng.uniform(0, 10, n_inicial)); Y = [f(x) for x in X]       # pontos iniciais aleatórios
```

```
    for _ in range(n_total - n_inicial):
        # 1) modelo substituto: processo gaussiano treinado no histórico
        kernel = Ck(1.0) * Matern(length_scale=1.0, nu=2.5) + WhiteKernel(1e-6, (1e-10, 1e-3))
        gp = GaussianProcessRegressor(kernel, normalize_y=True, n_restarts_optimizer=2,
random_state=semente)
        gp.fit(np.array(X)[:, None], Y)
        # 2) função de aquisição: melhoria esperada (EI) em todos os candidatos
        media, desvio = gp.predict(candidatos, return_std=True)
        desvio = np.maximum(desvio, 1e-9)
        z = (media - max(Y) - 0.01) / desvio
        ei = (media - max(Y) - 0.01) * norm.cdf(z) + desvio * norm.pdf(z)
        # 3) escolher o ponto de maior aquisição e 4) avaliar de verdade
        x_novo = float(candidatos[np.argmax(ei), 0])
        X.append(x_novo); Y.append(f(x_novo))                          # 5) atualizar o histórico
    return np.maximum.accumulate(Y)

def busca_aleatoria(semente, n_total=15):
    rng = np.random.default_rng(1000 + semente)
    return np.maximum.accumulate(f(rng.uniform(0, 10, n_total)))

bo = np.array([otimizacao_bayesiana(s) for s in range(40)])
al = np.array([busca_aleatoria(s) for s in range(40)])
print("avaliações | melhor valor médio (bayesiana x aleatória) | % que achou a agulha (> 0,95)")
for k in (5, 10, 15):
    print(f"{k:10d} | {bo[:, k-1].mean():.3f} x {al[:, k-1].mean():.3f}"
          f"                      | {100*(bo[:, k-1] > 0.95).mean():3.0f}% x {100*(al[:, k-1] >
0.95).mean():3.0f}%")
```

SAÍDA

```
avaliações | melhor valor médio (bayesiana x aleatória) | % que achou a agulha (> 0,95)
         5 | 0.544 x 0.616                      |   5% x  25%
        10 | 0.719 x 0.700                      |  30% x  30%
        15 | 0.813 x 0.740                      |  48% x  32%
```

Veja o que ocorre: no começo (5 avaliações), a busca aleatória está **à frente**, porque a BO precisa de pontos iniciais para formar um modelo útil e, às vezes, gasta avaliações explorando. Com 10, empatam. Com 15, a BO **passa à frente**: acha a agulha com mais frequência.

A moral é importante: a BO **não é mágica**. Ela precisa de um "aquecimento" e rende mais quando o orçamento permite que o modelo substituto aprenda alguma coisa.

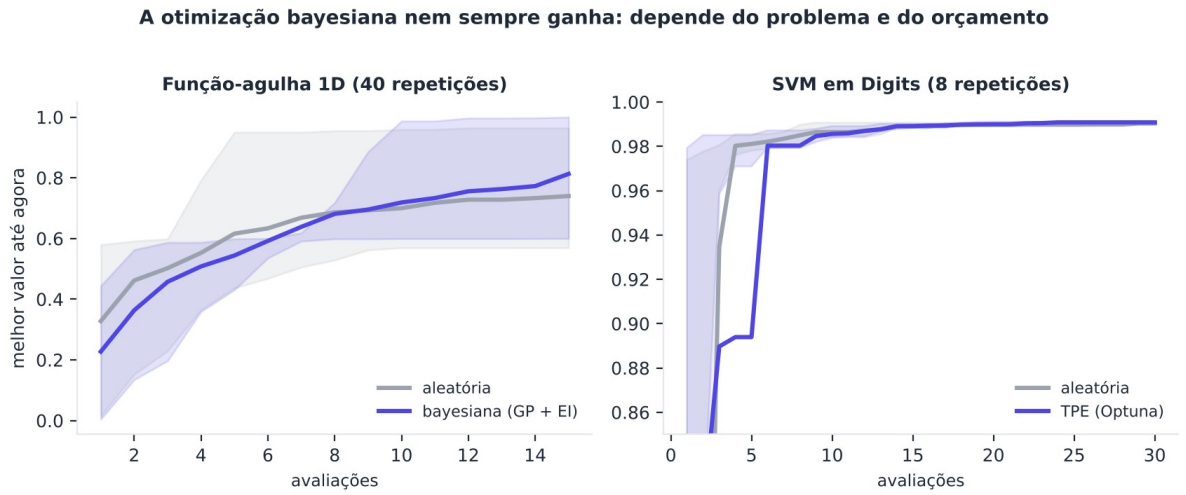


Figura 5.5 — À esquerda, a função-agulha (BO passa à frente com mais avaliações). À direita, um problema fácil de ajuste de SVM, em que BO e busca aleatória empatam.

5.7 TPE: outro modelo substituto

O processo gaussiano é elegante, mas sofre com **muitos hiperparâmetros**, com **categorias** e com hiperparâmetros **condicionais**. Uma alternativa muito usada é o **TPE** (*Tree-structured Parzen Estimator*), proposto por Bergstra e colaboradores (2011), junto a uma abordagem baseada em melhoria esperada.

A ideia do TPE é "virar a pergunta de cabeça para baixo". Em vez de modelar "dada a configuração, qual a nota?", ele modela "dada uma nota boa (ou ruim), **onde** as configurações costumam estar?". O passo a passo:

1. Separe o histórico em duas partes: as **tentativas boas** (por exemplo, o melhor quarto) e as **ruins** (o resto).
2. Estime duas distribuições: **l(x)**, onde as boas caem, e **g(x)**, onde as ruins caem.
3. Escolha a configuração em que a razão **l(x) / g(x)** é maior: onde as boas são comuns e as ruins são raras.

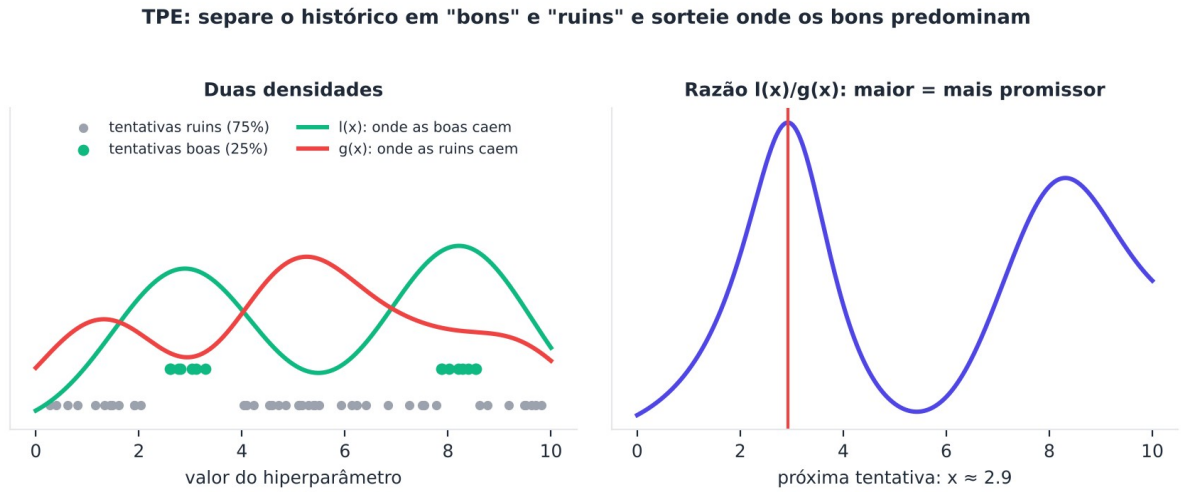


Figura 5.6 — O TPE compara a densidade das boas tentativas (verde) com a das ruins (vermelho). O próximo ponto fica onde a razão é maior.

O TPE lida bem com espaços **em árvore**, isto é, condicionais: "se o algoritmo é SVM, sorteie C e gamma; se é floresta, sorteie profundidade". Isso o torna natural para o problema **CASH** do Capítulo 1.

5.8 Outros modelos substitutos: o SMAC

Outra estratégia popular usa **florestas aleatórias** como modelo substituto. É o caso do **SMAC** (Hutter; Hoos; Leyton-Brown, 2011), que lida bem com categorias, valores condicionais e muitos hiperparâmetros. Uma versão moderna em Python é o **SMAC3** (Lindauer *et al.*, 2022).

Esses métodos sustentaram os primeiros sistemas de AutoML. O Auto-WEKA usou otimização bayesiana para resolver o CASH (Thornton *et al.*, 2013), e o Auto-sklearn seguiu o mesmo caminho, acrescentando meta-aprendizado e ensembles (Feurer *et al.*, 2015).

Modelo substituto	Ideia	Pontos fortes	Cuidado
Processo gaussiano	Previsão suave com incerteza	Poucos dados; poucos hiperparâmetros contínuos	Sofre com muitas dimensões e categorias
TPE	Densidades de tentativas boas e ruins	Espaços condicionais; categóricos	Menos preciso na incerteza
Floresta aleatória (SMAC)	Árvores estimam média e variância	Categorias e condicionais; muitas dimensões	Previsões menos suaves

5.9 Optuna: otimização bayesiana na prática

O **Optuna** (Akiba *et al.*, 2019) é uma das bibliotecas mais usadas para HPO. Suas características-chave são um espaço de busca descrito **dentro do próprio código** (*define-by-run*), estratégias eficientes de busca e poda, e uma arquitetura simples de usar.

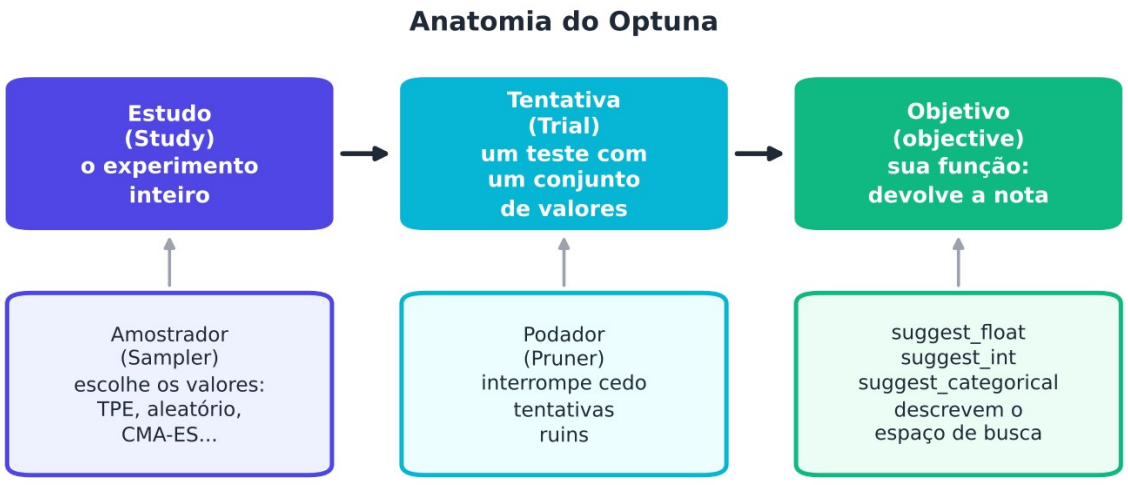


Figura 5.7 — As peças do Optuna: o estudo reúne tentativas; cada tentativa chama a sua função-objetivo; o amostrador escolhe valores; o podador interrompe tentativas ruins.

O uso tem três passos: **(1)** escrever uma função `objetivo` que recebe uma `trial`, sugere valores e devolve a nota; **(2)** criar um estudo; **(3)** chamar `optimize`.

```python
import optuna
from sklearn.datasets import load_digits
from sklearn.svm import SVC
from sklearn.model_selection import cross_val_score, StratifiedKFold
```

```
optuna.logging.set_verbosity(optuna.logging.WARNING)      # silencia mensagens de progresso

X, y = load_digits(return_X_y=True)
X = X / 16.0
cv = StratifiedKFold(3, shuffle=True, random_state=0)

def objetivo(trial):
    # O espaço de busca é descrito aqui dentro, com código comum
    C = trial.suggest_float("C", 1e-3, 1e4, log=True)
    gamma = trial.suggest_float("gamma", 1e-8, 1e1, log=True)
    return cross_val_score(SVC(C=C, gamma=gamma), X, y, cv=cv).mean()

estudo = optuna.create_study(direction="maximize", sampler=optuna.samplers.TPESampler(seed=0))
estudo.optimize(objetivo, n_trials=30)

print("Melhor acurácia:", round(estudo.best_value, 4))
print("Melhores valores: C = %.2f, gamma = %.5f" % (estudo.best_params["C"],
estudo.best_params["gamma"]))
print("Tentativa em que o melhor apareceu:", estudo.best_trial.number)
```

SAÍDA

```
Melhor acurácia: 0.9911
Melhores valores: C = 16.77, gamma = 0.15476
Tentativa em que o melhor apareceu: 25
```

O argumento `log=True` pede amostragem em escala logarítmica, como estudamos no Capítulo 4. O amostrador padrão do Optuna é o TPE.

5.10 Resolvendo o CASH com o Optuna

Como o espaço é definido por código comum, o Optuna resolve o **CASH** de forma natural: basta **escolher o algoritmo primeiro** e, **dentro de um `if`**, sugerir os hiperparâmetros daquele algoritmo. Isso é um espaço condicional, sem nenhum truque.

PYTHON

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

def objetivo_cash(trial):
    algoritmo = trial.suggest_categorical("algoritmo", ["svm", "floresta", "logistica"])
    if algoritmo == "svm":
        modelo = SVC(C=trial.suggest_float("svm_C", 1e-3, 1e4, log=True),
                     gamma=trial.suggest_float("svm_gamma", 1e-8, 1e1, log=True))
    elif algoritmo == "floresta":
        modelo = RandomForestClassifier(n_estimators=trial.suggest_int("rf_arvores", 20, 150),
                                        max_depth=trial.suggest_int("rf_profundidade", 3, 20),
                                        random_state=0, n_jobs=-1)
    else:
        modelo = make_pipeline(StandardScaler(), LogisticRegression(
                     C=trial.suggest_float("lr_C", 1e-4, 1e3, log=True), max_iter=2000))
    return cross_val_score(modelo, X, y, cv=cv).mean()

cash = optuna.create_study(direction="maximize", sampler=optuna.samplers.TPESampler(seed=0))
```

```
cash.optimize(objetivo_cash, n_trials=30)

print("Algoritmo vencedor:", cash.best_params["algoritmo"], "| acurácia =", round(cash.best_value, 4))
resumo = {}
for t in cash.trials:
    r = resumo.setdefault(t.params["algoritmo"], {"tentativas": 0, "melhor": 0.0})
    r["tentativas"] += 1
    r["melhor"] = max(r["melhor"], t.value)
for nome, r in resumo.items():
    print(f"{nome:10s} tentativas = {r['tentativas']:2d} | melhor nota = {r['melhor']:.3f}")
```

SAÍDA

```
Algoritmo vencedor: floresta | acurácia = 0.9766
floresta   tentativas = 20 | melhor nota = 0.977
logistica  tentativas =  5 | melhor nota = 0.971
svm        tentativas =  5 | melhor nota = 0.299
```

Observe o que **a busca fez**. Um único estudo escolheu o algoritmo e seus hiperparâmetros, e o TPE **concentrou as tentativas** em um deles. Mas veja o detalhe: o algoritmo vencedor aqui **não é o melhor possível** para este problema. No estudo da seção anterior, só o SVM, bem ajustado, chegou a 0,991.

O que aconteceu? Com apenas 30 tentativas divididas entre três algoritmos, as 5 tentativas de SVM caíram em regiões ruins do espaço (a melhor nota delas foi só 0,299, pois o espaço de *gamma* é enorme e a região boa é estreita). O TPE interpretou "SVM = ruim" e passou a investir em algoritmos que deram boas notas cedo. É o dilema **explorar × aproveitar** em ação, e dessa vez o "aproveitar" saiu caro.

Esse é um aviso importante sobre o AutoML em geral: **buscas com orçamento curto podem desistir cedo de um bom candidato**. Há remédios que veremos adiante: mais orçamento, espaços mais bem desenhados (por exemplo, limitar o *gamma* a uma faixa razoável), começar de configurações conhecidas (*warm start*, Capítulo 9) e avaliar candidatos com orçamento crescente (Capítulo 6).

> **Dica**
>
> O Optuna tem recursos que valem explorar: `study.trials_dataframe()` devolve todas as tentativas em uma tabela; `optuna.importance.get_param_importances(study)` estima quais hiperparâmetros mais influenciam a nota; e `study.enqueue_trial()` permite começar por configurações que você já conhece (o *warm start*, que veremos no Capítulo 9).

5.11 Quanto ganhamos de verdade?

É hora de ser honesto sobre o desempenho. Vamos repetir a comparação do início, agora com um problema real de aprendizado de máquina: o SVM em Digits, com um espaço de busca **amplo** (C de 0,001 a 10.000, gamma de 10^{-8} a 10). Rodamos busca aleatória e TPE, com 30 avaliações e 8 sementes cada.

PYTHON

```
import numpy as np

def nota(C, gamma):
    return cross_val_score(SVC(C=C, gamma=gamma), X, y, cv=cv).mean()

N = 30
aleatoria, tpe = [], []
for semente in range(8):
```

```
    rng = np.random.default_rng(semente)
    aleatoria.append(np.maximum.accumulate([nota(10 ** rng.uniform(-3, 4), 10 ** rng.uniform(-8, 1))
for _ in range(N)]))
    est = optuna.create_study(direction="maximize", sampler=optuna.samplers.TPESampler(seed=semente,
n_startup_trials=8))
    est.optimize(lambda t: nota(t.suggest_float("C", 1e-3, 1e4, log=True), t.suggest_float("g", 1e-8,
1e1, log=True)), n_trials=N)
    tpe.append(np.maximum.accumulate([t.value for t in est.trials]))

aleatoria, tpe = np.array(aleatoria), np.array(tpe)
print("avaliações | aleatória | TPE")
for k in (5, 10, 20, 30):
    print(f"{k:10d} |   {aleatoria[:, k-1].mean():.3f}   | {tpe[:, k-1].mean():.3f}")
```

```
SAÍDA
avaliações | aleatória | TPE
         5 |   0.981   | 0.894
        10 |   0.986   | 0.986
        20 |   0.990   | 0.990
        30 |   0.990   | 0.991
```

Neste problema, **a busca aleatória e o TPE terminam empatados**, e a aleatória até está melhor nas primeiras avaliações. Por quê? Porque o SVM em Digits tem **só dois botões** e uma região boa **bem larga**: sortear ao acaso já acerta com facilidade, e o TPE gasta as primeiras tentativas (8, no nosso caso) também sorteando ao acaso.

Isso é coerente com a literatura: a BO tende a ajudar mais quando **a avaliação é cara**, o **orçamento é pequeno**, o espaço é **maior** e a **região boa é estreita**. Em problemas fáceis e baratos, o ganho é pequeno. Por isso, a regra do Capítulo 4 continua valendo: **a busca aleatória é a linha de base** que qualquer método mais elaborado precisa superar com o mesmo orçamento.

> **Memorize**
>
> A otimização bayesiana **não é garantia de melhora**: ela é uma aposta de que **aprender com o histórico** compensa seu custo extra. A aposta é boa quando as avaliações são caras e a região boa é difícil de achar.

5.12 Limitações

- **Muitas dimensões.** Com dezenas de hiperparâmetros, GPs ficam imprecisos e lentos. TPE e florestas lidam melhor.
- **Ruído.** A nota de validação varia por sorte; o modelo substituto precisa tolerar isso.
- **Custo do próprio modelo substituto.** GPs custam cada vez mais conforme o histórico cresce.
- **Paralelismo.** A BO é naturalmente **sequencial** (cada escolha depende das anteriores). Há adaptações para rodar várias avaliações ao mesmo tempo (Snoek; Larochelle; Adams, 2012), mas a busca aleatória paraleliza sem esforço.
- **Overfitting da busca.** Como no Capítulo 4, quanto mais se busca, mais otimista fica a nota do vencedor. Use teste final intocado.

5.13 Boas práticas

1. **Compare sempre com a busca aleatória**, no mesmo orçamento e com várias sementes.

2. **Use escala logarítmica** onde couber (`log=True`).
3. **Comece com pontos iniciais variados** (o Optuna faz isso por padrão, com tentativas aleatórias iniciais).
4. **Fixe sementes** para reprodutibilidade.
5. **Guarde o histórico** (`trials_dataframe`) e analise-o.
6. **Meça com honestidade** em um teste final separado.

Resumo do capítulo

MEMORIZE **Cartões de memorização**

- **BO** serve para caixas-pretas **caras**: poucas avaliações, escolhidas com inteligência.
- **Modelo substituto:** um modelo barato do modelo caro, com previsão **e incerteza**.
- **Função de aquisição** (PI, EI, UCB): nota de interesse que equilibra **explorar** e **aproveitar**.
- **Processo gaussiano:** ótimo com poucos pontos e poucas dimensões contínuas.
- **TPE:** compara l(x) (boas) com g(x) (ruins); bom para espaços condicionais.
- **SMAC:** usa florestas aleatórias como substituto.
- **Optuna:** espaço *define-by-run*; resolve CASH com um simples `if`.
- **Nem sempre vence a aleatória:** em problemas fáceis e baratos, empata.

Exercícios de múltipla escolha

Escolha **uma** alternativa em cada questão. O gabarito está no Apêndice A.

Questão 1. O que é uma "caixa-preta cara" no contexto da otimização de hiperparâmetros?

a) Um modelo de rede neural muito grande.
b) Um conjunto de dados confidencial.
c) Um computador sem acesso à internet.
d) Uma função hiperparâmetros → nota, cuja fórmula é desconhecida e cuja avaliação consome muito tempo.

Questão 2. Qual é o objetivo da otimização bayesiana?

a) Gastar poucas avaliações caras, escolhendo cada uma com inteligência.
b) Testar todas as combinações possíveis.
c) Substituir o conjunto de teste.
d) Eliminar a necessidade de validação.

Questão 3. O que é o modelo substituto (surrogate)?

a) Um modelo treinado com dados do teste.
b) Um modelo barato que imita a função cara, a partir do histórico de tentativas.
c) Um algoritmo de pré-processamento.
d) O modelo final que será implantado.

Questão 4. Além de prever a nota, o modelo substituto de uma BO deve informar:

a) o número de atributos.
b) o tempo de treino.
c) o tamanho do conjunto de teste.
d) a incerteza da previsão.

Questão 5. Em qual passo do ciclo da BO está o custo principal?

a) Treinar o modelo substituto.
b) Escolher o ponto de maior aquisição.
c) Calcular a função de aquisição.
d) Avaliar de verdade o ponto escolhido (treinar e validar o modelo).

Questão 6. O dilema explorar × aproveitar consiste em:

a) escolher entre treino e teste.
b) decidir entre olhar perto do melhor ponto conhecido ou investigar regiões pouco conhecidas.
c) escolher entre padronizar ou não os dados.
d) escolher entre dois algoritmos de classificação.

Questão 7. A função de aquisição serve para:

a) medir o desempenho final do modelo.
b) padronizar os atributos.
c) dividir os dados em partes.
d) dar uma nota de interesse a cada ponto, combinando previsão e incerteza.

Questão 8. Qual das funções abaixo é uma função de aquisição?

a) PCA.
b) StandardScaler.
c) Matriz de confusão.
d) EI (melhoria esperada).

Questão 9. A melhoria esperada (EI) combina:

a) o número de tentativas e o tempo de treino.
b) a média prevista (aproveitar) e a incerteza (explorar).
c) apenas a média prevista pelo modelo substituto.
d) apenas a incerteza.

Questão 10. Na função UCB, o que controla o apetite por explorar?

a) O tamanho do conjunto de teste.
b) A semente aleatória.
c) O coeficiente k que multiplica a incerteza.
d) O número de partes da validação cruzada.

Questão 11. Qual a principal propriedade que um processo gaussiano oferece ao ser usado como modelo substituto?

a) Previsões sem erro.
b) Uma previsão acompanhada de incerteza, pequena perto dos pontos medidos e grande longe deles.
c) Treino instantâneo com milhões de pontos.
d) Dispensa de função de aquisição.

Questão 12. O kernel de um processo gaussiano define:

a) o conjunto de teste.
b) a métrica de avaliação.
c) quais pontos são considerados "próximos" e, portanto, com notas parecidas.
d) o número de tentativas.

Questão 13. Na figura da BO com 3 avaliações, a faixa de incerteza era enorme porque:

a) a busca terminou.
b) a função de aquisição estava errada.
c) o modelo substituto tinha poucos dados e quase nada sabia da função.
d) a função era constante.

Questão 14. No experimento da função-agulha, com 5 avaliações, quem estava à frente?

a) A BO, claramente.
b) Ambas empatadas.
c) Nenhuma encontrou nada.
d) A busca aleatória, pois a BO ainda precisava de pontos para formar um modelo útil.

Questão 15. No mesmo experimento, com 15 avaliações, o que ocorreu?

a) A BO achou a agulha com mais frequência.
b) Nenhuma achou.
c) Ambas acharam sempre.
d) A aleatória achou a agulha com mais frequência.

Questão 16. O TPE (Tree-structured Parzen Estimator) funciona:

a) sorteando valores uniformemente.
b) treinando redes neurais profundas.
c) estimando as densidades l(x) das tentativas boas e g(x) das ruins e escolhendo onde l/g é maior.
d) ajustando um processo gaussiano ao histórico.

Questão 17. Por que o TPE é natural para o problema CASH?

a) Porque é paralelo por natureza.
b) Porque lida bem com espaços em árvore (condicionais).
c) Porque não precisa de histórico.
d) Porque só aceita um hiperparâmetro.

Questão 18. O SMAC usa como modelo substituto:

a) uma floresta aleatória.
b) um processo gaussiano.
c) uma regressão linear.
d) um k-vizinhos.

Questão 19. Qual dos sistemas abaixo usou otimização bayesiana para resolver o CASH?

a) K-means.
b) Auto-WEKA.
c) PCA.
d) DBSCAN.

Questão 20. No Optuna, o que significa a abordagem define-by-run?

a) O Optuna define as métricas automaticamente.
b) O Optuna só roda em nuvem.
c) O espaço de busca é descrito dentro da própria função-objetivo, com código comum.
d) O espaço de busca é definido em um arquivo separado de configuração.

Questão 21. Em Optuna, qual método sugere um hiperparâmetro contínuo em escala logarítmica?

a) optuna.logscale().
b) study.log().
c) trial.suggest_float(nome, a, b, log=True).
d) trial.suggest_cat(nome).

Questão 22. No Optuna, o que é uma Trial?

a) O experimento inteiro.
b) O modelo final.
c) O conjunto de dados.
d) Um teste com um conjunto de valores de hiperparâmetros, que a sua função avalia.

Questão 23. Em uma função-objetivo do Optuna, o que a função deve devolver?

a) O modelo treinado.
b) A nota (valor a otimizar) da tentativa.
c) Uma lista de hiperparâmetros.
d) Os dados de treino.

Questão 24. Como o Optuna resolve o CASH de forma natural?

a) Ignorando os hiperparâmetros.
b) Usando apenas busca em grade.
c) Treinando todos os algoritmos com os mesmos hiperparâmetros.
d) Sugerindo primeiro o algoritmo (categórico) e, dentro de if, os hiperparâmetros daquele algoritmo.

Questão 25. No experimento CASH do capítulo, por que o TPE perdeu o SVM, apesar de ele poder chegar a 0,99?

a) Porque o Optuna não suporta SVM.
b) Porque o SVM não funciona em Digits.
c) Porque as primeiras tentativas de SVM caíram em regiões ruins e o TPE passou a evitá-lo com orçamento curto.
d) Porque o TPE sorteia sempre a floresta.

Questão 26. Qual conclusão geral o episódio do CASH ilustra?

a) Que o TPE nunca funciona.
b) Que a floresta é sempre melhor.
c) Que buscas com orçamento curto podem desistir cedo de um bom candidato.
d) Que o SVM é inferior.

Questão 27. No SVM em Digits com espaço amplo, qual foi o resultado da comparação entre TPE e aleatória com 30 avaliações?

a) A aleatória venceu por grande margem.
b) Nenhuma passou de 0,5.
c) Empataram (cerca de 0,990 e 0,991).
d) TPE venceu por grande margem.

Questão 28. Em que situação a BO costuma ajudar mais?

a) Avaliação barata, orçamento enorme e região boa larga.
b) Dados muito pequenos, sem hiperparâmetros.
c) Avaliação cara, orçamento pequeno, espaço grande e região boa estreita.
d) Sempre, independentemente do problema.

Questão 29. Uma limitação típica dos processos gaussianos é:

a) Exigirem rótulos aleatórios.
b) Só funcionarem com categorias.
c) Perderem precisão e velocidade com muitas dimensões e muitos pontos.
d) Não fornecerem incerteza.

Questão 30. Qual é a boa prática ao avaliar um novo método de HPO?

a) Compará-lo apenas em uma execução.
b) Usar um orçamento diferente para cada método.
c) Compará-lo com a busca aleatória, com o mesmo orçamento e várias sementes.
d) Usar o teste final para escolher o método.

Exercícios discursivos

Responda com suas palavras, em poucas linhas. Compare depois com a resposta-modelo do Apêndice A.

1. Explique, com uma analogia, por que a busca aleatória "desperdiça" informação e como a otimização bayesiana tenta corrigir isso.
2. O que é uma caixa-preta cara e por que ela justifica a otimização bayesiana?
3. Descreva os cinco passos do ciclo da otimização bayesiana.
4. Explique o dilema explorar × aproveitar com um exemplo do cotidiano e diga como a função de aquisição o resolve.
5. O que é um processo gaussiano e por que ele é útil como modelo substituto?
6. Compare PI, EI e UCB.
7. Explique a fórmula da EI em palavras.
8. Descreva o experimento da função-agulha e o que ele mostra sobre o desempenho da BO ao longo do orçamento.
9. Explique a ideia do TPE e por que ele lida bem com espaços condicionais.
10. Compare processo gaussiano, TPE e floresta aleatória (SMAC) como modelos substitutos, indicando um ponto forte de cada um.
11. O que são Study, Trial, Sampler e Pruner no Optuna?
12. Escreva, em palavras, os passos para resolver o CASH com o Optuna.
13. No experimento CASH do capítulo, a floresta venceu, mas o SVM poderia chegar a 0,99. Explique o que aconteceu e como evitar.
14. Por que, no SVM em Digits, TPE e aleatória empataram? O que isso ensina?
15. Cite três limitações da otimização bayesiana.
16. Por que a BO é naturalmente sequencial e que consequência isso traz em relação à busca aleatória?
17. Qual é o papel do argumento log=True no suggest_float e por que ele importa?
18. Por que o capítulo recomenda comparar um método de HPO com a busca aleatória em várias sementes e com o mesmo orçamento?
19. Descreva como o aquecimento (pontos iniciais) afeta a otimização bayesiana e por que ele importa para orçamentos curtos.
20. Explique por que a nota de validação do melhor ponto de uma BO continua sujeita ao otimismo discutido no Capítulo 4.

Exercícios práticos em Python (scikit-learn)

Resolva os exercícios no seu computador (ou no Google Colab). Eles usam conjuntos de dados da UCI que já vêm no scikit-learn ou dados simulados. Tente resolver **antes** de consultar a solução, no Apêndice D. As saídas apresentadas lá foram obtidas com as versões de bibliotecas listadas na Apresentação.

Exercício 1 — Sua própria otimização bayesiana

NÍVEL: INTERMEDIÁRIO

Reutilize o código do capítulo para maximizar uma nova função "cara" no intervalo de 0 a 6: `g(x) = sin(3x) · exp(−0,25x) + 0,5 · sin(9x)`. Faça 12 avaliações (3 iniciais aleatórias e 9 guiadas por EI). Repita com 20 sementes e compare com a busca aleatória de 12 sorteios, mostrando a média do melhor valor encontrado e a fração de execuções que ficou a menos de 0,05 do máximo verdadeiro (calculado em uma malha fina).

Pergunta: a BO superou a aleatória neste problema? Em que ponto do orçamento?

Exercício 2 — Optuna em uma floresta aleatória

NÍVEL: INTERMEDIÁRIO

Use o conjunto **Breast Cancer Wisconsin** (UCI). Com o Optuna (TPE, semente 0), otimize 25 tentativas de uma `RandomForestClassifier`: `n_estimators` (inteiro de 20 a 150), `max_depth` (inteiro de 2 a 15), `min_samples_leaf` (inteiro de 1 a 10) e `max_features` (real de 0,1 a 1,0). Use validação cruzada estratificada de 5 partes. Mostre a melhor nota e os melhores valores, desenhe a curva "melhor nota até agora" e calcule a importância dos hiperparâmetros com `optuna.importance.get_param_importances`.

Pergunta: qual hiperparâmetro foi o mais importante? A importância faz sentido?

Exercício 3 — Reproduzindo e corrigindo a falha do CASH

NÍVEL: AVANÇADO

Use o conjunto **Digits**. Rode o CASH do capítulo (SVM, floresta e regressão logística, com *gamma* do SVM entre 10^{-8} e 10, 30 tentativas, TPE com semente 0). Mostre a melhor nota por algoritmo. Em seguida, **corrija o espaço de busca**: limite o *gamma* a uma faixa mais razoável (10^{-5} a 1, com `X/16` já normalizado) e repita. Compare as duas execuções.

Pergunta: a correção resolveu o problema? O que ela revela sobre o papel humano no desenho do espaço de busca?

Exercício 4 — Quanto explorar? O coeficiente do UCB

NÍVEL: AVANÇADO

Implemente a BO com função de aquisição **UCB** (média + κ · desvio) na função-agulha do capítulo e compare três valores de κ: 0,5, 2 e 6. Faça 15 avaliações (3 iniciais), repetindo com 30 sementes. Para cada κ, mostre a média do melhor valor encontrado e a fração de execuções que achou a agulha (valor maior que 0,95).

Pergunta: qual κ funcionou melhor? Como isso se relaciona ao dilema explorar × aproveitar?

> SAIBA MAIS **Para ir além: bases públicas da UCI e do Kaggle**
>
> - **UCI:** Concrete Compressive Strength (resistência de concreto, regressão) e Wine Quality (qualidade de vinhos, regressão ou classificação).
> - **Kaggle:** House Prices (preços de imóveis, muitas variáveis) e Medical Cost Personal Datasets (custos de seguro de saúde).
>
> Estes conjuntos não foram usados nem testados nos códigos do livro. Confira, nos sites, a descrição, a licença e as colunas antes de usá-los.

Projeto integrador do capítulo

AutoML minimalista com Optuna em regressão (Diabetes)

Fonte dos dados. *Diabetes*, incluído no scikit-learn (`load_diabetes`): 442 pacientes, 10 atributos e, como alvo, uma medida de progressão da doença após um ano. A base é de uso didático e tem origem em estudo estatístico clássico, distribuída com o scikit-learn.

Contexto. Uma equipe quer prever a progressão de uma doença e comparar um AutoML minimalista (Optuna, resolvendo o CASH para regressão) com a busca aleatória.

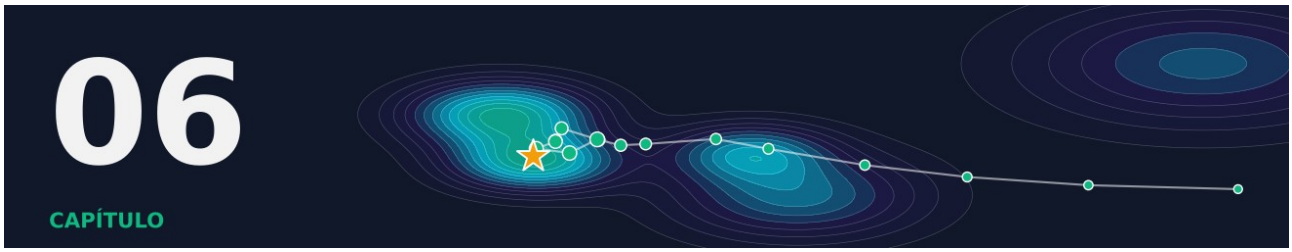
Atenção ética. Os dados são de uso didático. O modelo **não é ferramenta clínica**.

Tarefas.

1. Separe 25% dos dados como **teste final** (semente fixa). Não toque nele até o fim.
2. Calcule uma **linha de base** (`DummyRegressor`, que prevê a média) e a nota de uma `Ridge` com os valores padrão, ambas por validação cruzada de 5 partes no treino (métrica R^2).
3. Escreva uma função `objetivo` do Optuna que escolha entre `Ridge`, `RandomForestRegressor`, `GradientBoostingRegressor` e `SVR` (com padronização), cada um com seus hiperparâmetros (use escala logarítmica quando fizer sentido).
4. Rode **40 tentativas** com TPE. Em seguida, rode a **busca aleatória** (`RandomSampler`) com o mesmo orçamento. Repita ambos com 3 sementes.
5. Desenhe a curva de "melhor R^2 até agora" (média das sementes) para TPE e aleatória.
6. Treine o melhor modelo encontrado (de uma das buscas) no treino completo e avalie **uma vez** no teste final, com R^2 e RMSE.
7. Escreva um parágrafo (6 a 8 linhas): o AutoML superou a linha de base e a `Ridge` padrão? TPE superou a aleatória? O desempenho final é compatível com a validação? Que limitações existem?

Entregáveis. Um notebook (ou script) comentado, o gráfico e o parágrafo de conclusões.

A solução de referência está no Apêndice E.



Capítulo 6 — Métodos multifidelidade: Successive Halving, Hyperband e BOHB

OBJETIVOS DE APRENDIZAGEM

Ao final deste capítulo você será capaz de:

- Explicar por que treinar todos os candidatos até o fim é um desperdício.
- Definir fidelidade e dar exemplos (épocas, quantidade de dados, atributos).
- Descrever o funcionamento do Successive Halving e calcular seu orçamento.
- Descrever o Hyperband e o dilema entre testar muitas configurações e treinar cada uma por mais tempo.
- Explicar a ideia do BOHB e o uso de podas (*pruning*) no Optuna.
- Reconhecer os riscos do corte precoce, quando o ranking muda com o treino.

6.1 A peneira de talentos

Pense em uma **peneira** de um clube de futebol. Chegam 270 jovens. O clube não pode treinar todos por um ano inteiro. O que faz?

1. Dá **um treino curto** para todos e dispensa os que se mostram fracos.
2. Os que ficam recebem **mais treino**, e outra rodada de cortes acontece.
3. Ao fim, poucos jogadores, os mais promissores, recebem **o treino completo**.



Figura 6.1 — A peneira de talentos. Candidatos fracos saem cedo e gastam pouco; os fortes recebem cada vez mais treino.

Com hiperparâmetros é a mesma lógica. Nos capítulos anteriores, cada configuração era treinada **até o fim** antes de recebermos a nota. Isso é um grande desperdício: muitas configurações ruins já mostram sua fraqueza **logo no início** do treino.

Os métodos deste capítulo são chamados de **multifidelidade** (*multi-fidelity*). Eles avaliam muitos candidatos de forma **barata e aproximada** e reservam a avaliação **cara e precisa** para os mais promissores.

6.2 O que é fidelidade?

Fidelidade é o quanto uma avaliação "se parece" com a avaliação completa. Uma avaliação de **baixa fidelidade** é barata, mas aproximada. Uma de **alta fidelidade** é cara, mas precisa.

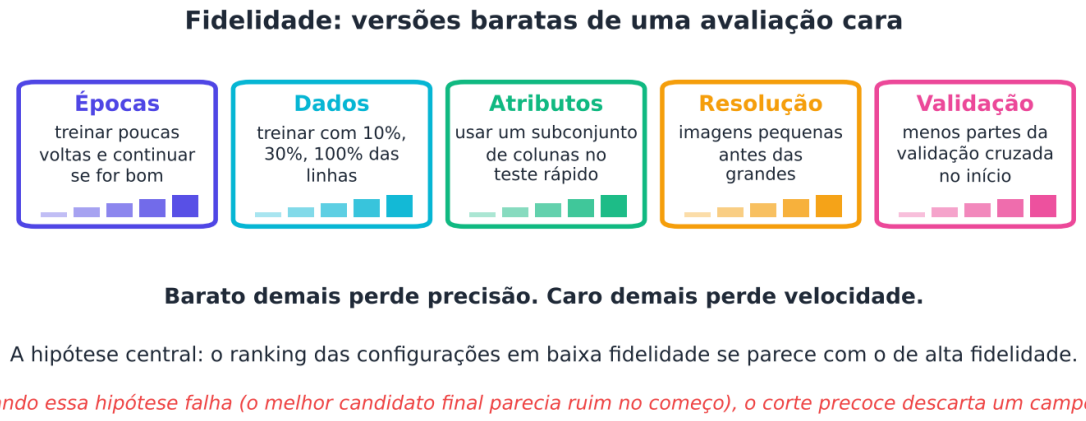


Figura 6.2 — Formas de criar versões baratas de uma avaliação cara: menos épocas, menos dados, menos atributos, imagens menores, menos partes na validação.

Os exemplos mais comuns:

- **Épocas:** treinar o modelo por poucas voltas (épocas) nos dados e continuar apenas se promissor.
- **Quantidade de dados:** treinar com 10%, depois 30%, depois 100% das linhas.
- **Atributos:** testar primeiro com um subconjunto de colunas.
- **Resolução:** imagens pequenas antes das grandes.
- **Partes da validação cruzada:** começar com poucas partes.

A grande **aposta** de todos esses métodos é: **o ranking em baixa fidelidade se parece com o ranking em alta fidelidade**. Isto é, quem vai bem no começo tende a ir bem no fim. Se a aposta for falsa, o corte precoce descarta justamente os campeões. Voltaremos a esse risco na seção 6.9.

6.3 Successive Halving: dividir ao meio, repetidamente

O método mais simples é o ***Successive Halving*** (algo como "metades sucessivas"), que se tornou conhecido pelo trabalho de Jamieson e Talwalkar (2016). Em português, podemos chamá-lo de **corte sucessivo**. O algoritmo é:

1. Comece com n candidatos e um orçamento pequeno r (por exemplo, 1 época).
2. Treine **todos** com esse orçamento e avalie.
3. **Mantenha apenas a fração $1/\eta$ melhor** (com $\eta = 3$, a terça parte melhor).
4. **Multiplique o orçamento por η** para os que ficaram.

5. Repita até restar **um** candidato.

Veja um exemplo com n = 27, r = 1 e η = 3:

Rodada	Candidatos	Épocas cada	Épocas gastas na rodada
1	27	1	27
2	9	3	27
3	3	9	27
4	1	27	27

Dá para notar uma beleza: **cada rodada custa a mesma coisa** (27 épocas). O total é 108 épocas. Para comparar, treinar os 27 candidatos até 27 épocas cada custaria **729 épocas**, ou seja, mais de seis vezes mais.

Implementando do zero

Vamos implementar o Successive Halving com um modelo real: o `SGDClassifier` do scikit-learn, que aprende por **gradiente descendente estocástico** (ajusta os pesos aos poucos, a cada volta pelos dados). O método `partial_fit` permite **treinar uma época por vez e retomar depois**, o que é perfeito para a multifidelidade: o recurso é o número de **épocas**.

Usaremos o conjunto **Digits** (UCI), com os pixels padronizados, e uma única separação treino/validação para simplificar. O espaço de busca tem três hiperparâmetros: `alpha` (regularização), `eta0` (passo de aprendizado, que mede o quanto os pesos mudam a cada ajuste) e `loss` (função de perda).

```python
import math
import numpy as np
from sklearn.datasets import load_digits
from sklearn.linear_model import SGDClassifier
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

def carregar_dados(semente=0):
    X, y = load_digits(return_X_y=True)
    Xtr, Xva, ytr, yva = train_test_split(X, y, test_size=0.3, stratify=y, random_state=semente)
    escala = StandardScaler().fit(Xtr)
    return escala.transform(Xtr), escala.transform(Xva), ytr, yva

class Candidato:
    """Uma configuração que pode ser treinada aos poucos (pausar e retomar)."""
    def __init__(self, rng, dados):
        self.hp = {"alpha": 10 ** rng.uniform(-8, 1), "eta0": 10 ** rng.uniform(-6, 1),
                   "loss": str(rng.choice(["hinge", "log_loss", "modified_huber"]))}
        self.modelo = SGDClassifier(loss=self.hp["loss"], alpha=self.hp["alpha"],
learning_rate="constant",
                                    eta0=self.hp["eta0"], random_state=int(rng.integers(1e6)))
        self.dados, self.epocas, self.nota, self.curva = dados, 0, 0.0, []

    def treinar_ate(self, epocas):
        Xtr, Xva, ytr, yva = self.dados
        while self.epocas < epocas:
            self.modelo.partial_fit(Xtr, ytr, classes=np.arange(10))      # uma época
            self.epocas += 1
            self.nota = self.modelo.score(Xva, yva)                        # acurácia na validação
```

```
            self.curva.append(self.nota)
        return self.nota
```

Agora o algoritmo em si. Ele recebe uma lista de candidatos e aplica o corte sucessivo.

PYTHON

```python
def sucessivo(candidatos, r_min, eta=3, r_max=None, verbose=False):
    """Successive Halving: treina todos com r_min épocas, mantém o melhor 1/eta e multiplica o
orçamento por eta."""
    r, gasto = r_min, 0
    while True:
        for c in candidatos:
            antes = c.epocas
            c.treinar_ate(int(r))
            gasto += c.epocas - antes                                    # só contamos as épocas
NOVAS
        if verbose:
            print(f"  {len(candidatos):2d} candidatos com {int(r):2d} épocas | melhor acurácia =
{max(c.nota for c in candidatos):.3f}")
        if len(candidatos) <= 1 or (r_max and r >= r_max):
            break
        candidatos = sorted(candidatos, key=lambda c: -c.nota)[:max(1, len(candidatos) // eta)]
        r *= eta
    return max(candidatos, key=lambda c: c.nota), gasto
```

PYTHON

```python
dados = carregar_dados()
rng = np.random.default_rng(0)
candidatos = [Candidato(rng, dados) for _ in range(81)]

print("Successive Halving com 81 candidatos (η = 3):")
vencedor, gasto = sucessivo(candidatos, r_min=1, eta=3, r_max=81, verbose=True)
print(f"\nÉpocas de treino gastas: {gasto}")
print(f"Treinar os 81 candidatos por 81 épocas custaria: {81 * 81}")
print(f"Vencedor: {vencedor.hp['loss']}, alpha = {vencedor.hp['alpha']:.2e}, eta0 =
{vencedor.hp['eta0']:.2e}")
```

SAÍDA

```
Successive Halving com 81 candidatos (η = 3):
  81 candidatos com  1 épocas | melhor acurácia = 0.948
  27 candidatos com  3 épocas | melhor acurácia = 0.961
   9 candidatos com  9 épocas | melhor acurácia = 0.969
   3 candidatos com 27 épocas | melhor acurácia = 0.969
   1 candidatos com 81 épocas | melhor acurácia = 0.956

Épocas de treino gastas: 297
Treinar os 81 candidatos por 81 épocas custaria: 6561
Vencedor: modified_huber, alpha = 1.39e-04, eta0 = 3.57e-03
```

Veja a eficiência: o vencedor emerge com 297 épocas, e não com as 6.561 que treinar todos os 81 candidatos exigiria. Cada rodada mostra menos candidatos e mais épocas, como na peneira.

Repare também em um detalhe: a melhor acurácia da última rodada (0,956) é **menor** que a de rodadas anteriores (0,969). Como o SGD com passo constante oscila de uma época para outra, a nota de um candidato

não cresce de forma regular com o treino. Esse ruído é uma limitação real de qualquer método que decide com base em notas parciais.

6.4 Quando o corte precoce funciona (e quando não)

Para entender a aposta do corte precoce, observe **curvas de aprendizado reais**, isto é, como a acurácia evolui com as épocas, para 40 configurações do nosso problema.

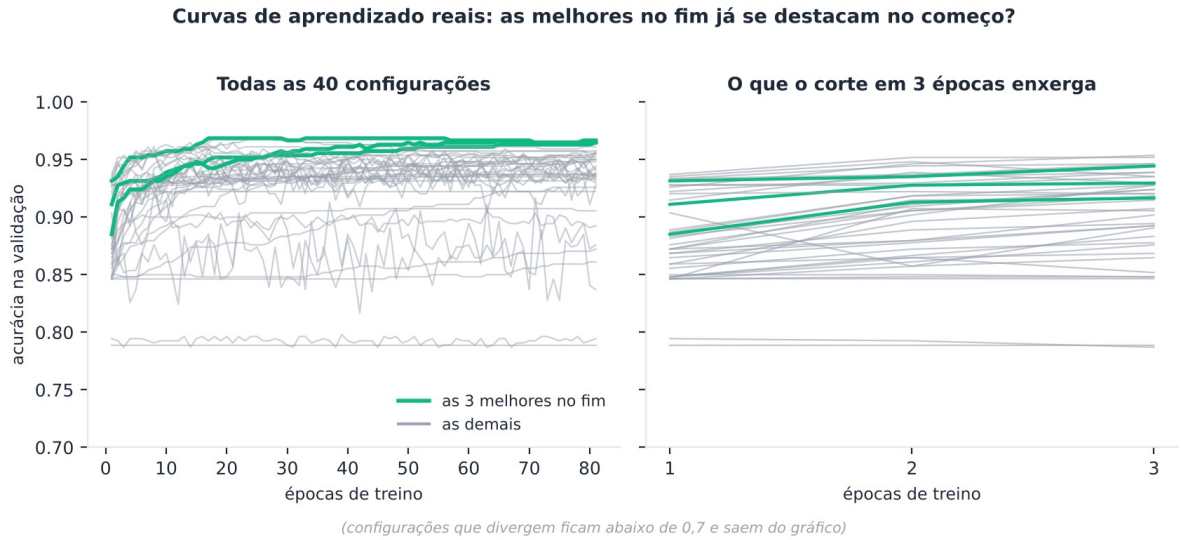


Figura 6.3 — À esquerda, as curvas completas de 40 configurações (as 3 melhores no fim, em verde). À direita, o que um corte em 3 épocas enxerga.

Dois fatos merecem atenção:

1. Algumas configurações são **claramente ruins desde as primeiras épocas** (curvas que ficam baixas ou que oscilam muito). Cortá-las cedo é seguro e economiza muito.
2. Entre as boas, as curvas **se cruzam**: uma configuração que começa devagar pode ultrapassar outra no fim. Um corte muito agressivo pode eliminar uma campeã "de largada lenta".

Podemos medir isso. O bloco abaixo treina as mesmas 40 configurações da figura e compara o **posto final** (1 = melhor) das três campeãs com o posto que elas tinham **após 3 épocas**.

```python
rng = np.random.default_rng(4)
dados = carregar_dados(0)
configs = [Candidato(rng, dados) for _ in range(40)]
for c in configs:
    c.treinar_ate(81)

final = np.array([c.curva[-1] for c in configs])
apos3 = np.array([c.curva[2] for c in configs])
posto_final = (-final).argsort().argsort() + 1          # 1 = melhor
posto_apos3 = (-apos3).argsort().argsort() + 1

print("As 3 melhores no final (epoch 81) e o posto delas após 3 épocas:")
for i in np.argsort(-final)[:3]:
    print(f"  configuração {i:2d}: posto final = {posto_final[i]} | posto após 3 épocas =
{posto_apos3[i]:2d} de 40")
corte = len(configs) // 3
print(f"Se cortássemos para as {corte} melhores após 3 épocas, "
```

```
      f"{sum(posto_apos3[i] <= corte for i in np.argsort(-final)[:3])} das 3 campeãs sobreviveriam.")
```

SAÍDA

```
As 3 melhores no final (epoch 81) e o posto delas após 3 épocas:
  configuração 30: posto final = 1 | posto após 3 épocas =  5 de 40
  configuração 14: posto final = 2 | posto após 3 épocas = 10 de 40
  configuração 35: posto final = 3 | posto após 3 épocas = 20 de 40
Se cortássemos para as 13 melhores após 3 épocas, 2 das 3 campeãs sobreviveriam.
```

Dois terços das campeãs sobreviveriam ao corte; a terceira, que estava no posto 20 de 40 após 3 épocas, seria eliminada. É o risco do corte precoce, medido, e não apenas imaginado.

É por isso que a escolha do **orçamento inicial** r e do fator **η** importa. Orçamentos iniciais muito pequenos cortam campeãs lentas; muito grandes perdem a economia.

6.5 No scikit-learn: busca com halving

O scikit-learn tem a ideia pronta em `HalvingGridSearchCV` e `HalvingRandomSearchCV`. Aqui, o "recurso" é, por padrão, o **número de exemplos de treino**: as primeiras rodadas usam poucos dados, e as seguintes, mais. Essas classes são marcadas como **experimentais**: é preciso habilitá-las explicitamente.

PYTHON

```python
import numpy as np
from sklearn.experimental import enable_halving_search_cv    # habilita as classes experimentais
from sklearn.model_selection import GridSearchCV, HalvingGridSearchCV, StratifiedKFold
from sklearn.datasets import load_digits
from sklearn.svm import SVC

X, y = load_digits(return_X_y=True); X = X / 16.0
cv = StratifiedKFold(5, shuffle=True, random_state=0)
grade = {"C": np.logspace(-1, 3, 6), "gamma": np.logspace(-5, -1, 6)}      # 36 combinações

completa = GridSearchCV(SVC(), grade, cv=cv).fit(X, y)
halving = HalvingGridSearchCV(SVC(), grade, cv=cv, factor=3, random_state=0).fit(X, y)

n_treinos = len(completa.cv_results_["params"]) * 5
custo_completa = n_treinos * int(len(X) * 0.8)
custo_halving = sum(c * r * 5 * 0.8 for c, r in zip(halving.n_candidates_, halving.n_resources_))

print(f"Grade completa : {n_treinos} treinos | melhor acurácia = {completa.best_score_:.3f}")
print(f"Halving        : candidatos por rodada {halving.n_candidates_} | exemplos por rodada
{halving.n_resources_}")
print(f"                 melhor acurácia (na última rodada) = {halving.best_score_:.3f}")
print(f"Custo aproximado (exemplos processados): grade = {custo_completa:,.0f} | halving =
{custo_halving:,.0f}")
print(f"Economia: {100 * (1 - custo_halving / custo_completa):.0f}%")
```

SAÍDA

```
Grade completa : 180 treinos | melhor acurácia = 0.989
Halving        : candidatos por rodada [36, 12, 4] | exemplos por rodada [100, 300, 900]
                 melhor acurácia (na última rodada) = 0.987
Custo aproximado (exemplos processados): grade = 258,660 | halving = 43,200
Economia: 83%
```

A busca *halving* chega a configurações de qualidade parecida usando uma fração do custo, porque descarta cedo os candidatos fracos, quando só 'viu' poucos exemplos.

> **Atenção**
>
> A nota final de uma busca *halving* vem da **última rodada**, que usa mais dados, mas só para poucos candidatos. Ela não é diretamente comparável à nota de uma grade completa. Compare modelos finais em um teste separado.

6.6 Hyperband: não apostar tudo em um corte

O Successive Halving tem um dilema escondido. Para um orçamento total fixo, devemos testar **muitas configurações com pouco treino** (apostando que cortes precoces são confiáveis) ou **poucas configurações com muito treino** (apostando que o começo das curvas engana)? Não sabemos a resposta de antemão.

O **Hyperband** (Li *et al.*, 2018) resolve com uma ideia simples: **não escolha, faça os dois**. Ele roda **vários torneios** de Successive Halving, cada um com uma aposta diferente, do mais agressivo ao mais conservador, e fica com o melhor resultado entre todos. Os autores formularam o problema como uma busca em que um recurso (épocas, dados, atributos) é alocado a configurações sorteadas, e mostraram ganhos de velocidade em problemas com redes profundas e métodos de kernel.

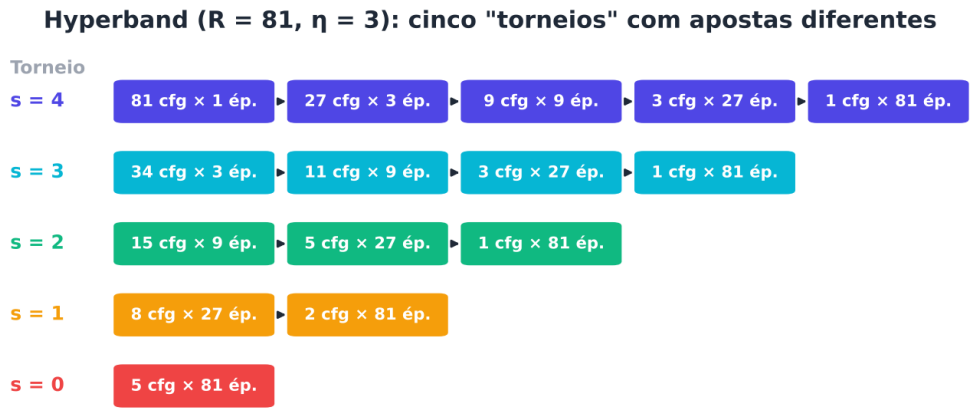


Figura 6.4 — Os torneios (brackets) do Hyperband com R = 81 e η = 3. O primeiro testa 81 configurações com 1 época; o último testa 5 configurações com 81 épocas.

Dois parâmetros definem o Hyperband:

- **R:** o máximo de recurso (épocas) que uma configuração pode receber.
- **η:** o fator de corte (mantém 1/η a cada rodada).

Cada torneio *s* começa com um número *n* de configurações e um recurso inicial *r*. Quanto maior o *s*, mais candidatos e menos treino inicial.

```python
def hyperband(rng, dados, R=81, eta=3):
    s_max = int(math.log(R) / math.log(eta) + 1e-9)
    B = (s_max + 1) * R
    melhor, gasto, n_config = None, 0, 0
    for s in range(s_max, -1, -1):
        n = int(math.ceil(B / R * eta ** s / (s + 1)))        # nº de configurações do torneio
```

```
        r = R * eta ** (-s)                                    # recurso inicial
        candidatos = [Candidato(rng, dados) for _ in range(n)]
        n_config += n
        vencedor, g = sucessivo(candidatos, r, eta, R)
        gasto += g
        if melhor is None or vencedor.nota > melhor.nota:
            melhor = vencedor
    return melhor, gasto, n_config

def busca_aleatoria(rng, dados, orcamento, R=81):
    """Linha de base: treina cada configuração até R épocas, até acabar o orçamento."""
    melhor, gasto = 0.0, 0
    while gasto + R <= orcamento:
        c = Candidato(rng, dados)
        c.treinar_ate(R)
        gasto += R
        melhor = max(melhor, c.nota)
    return melhor, gasto
```

Uma comparação justa

Compararemos Successive Halving, Hyperband e busca aleatória com o **mesmo orçamento** (mesmo número de épocas), repetindo com 12 sementes. Isso leva alguns minutos de processamento.

PYTHON

```
resultados = []
for s in range(12):
    d = carregar_dados(0)
    sh, gasto_sh = sucessivo([Candidato(np.random.default_rng(2000 + s * 1000 + i), d) for i in
range(81)], 1, 3, 81)
    aleat_sh, _ = busca_aleatoria(np.random.default_rng(3000 + s), d, gasto_sh)
    hb, gasto_hb, n_cfg = hyperband(np.random.default_rng(s), d)
    aleat_hb, _ = busca_aleatoria(np.random.default_rng(1000 + s), d, gasto_hb)
    resultados.append((sh.nota, aleat_sh, hb.nota, aleat_hb, gasto_sh, gasto_hb, n_cfg))

r = np.array(resultados)
print(f"Orçamento de {int(r[0,4])} épocas:")
print(f"  Successive Halving (81 configurações): {r[:,0].mean():.3f} (±{r[:,0].std():.3f})")
print(f"  Aleatória (só {int(r[0,4] // 81)} configurações)         : {r[:,1].mean():.3f} (±
{r[:,1].std():.3f})")
print(f"Orçamento de {int(r[0,5])} épocas:")
print(f"  Hyperband ({int(r[0,6])} configurações)      : {r[:,2].mean():.3f} (±{r[:,2].std():.3f})")
print(f"  Aleatória (só {int(r[0,5] // 81)} configurações)      : {r[:,3].mean():.3f} (±
{r[:,3].std():.3f})")
```

SAÍDA

```
Orçamento de 297 épocas:
  Successive Halving (81 configurações): 0.963 (±0.005)
  Aleatória (só 3 configurações)         : 0.955 (±0.012)
Orçamento de 1581 épocas:
  Hyperband (143 configurações)      : 0.969 (±0.002)
  Aleatória (só 19 configurações)      : 0.965 (±0.007)
```

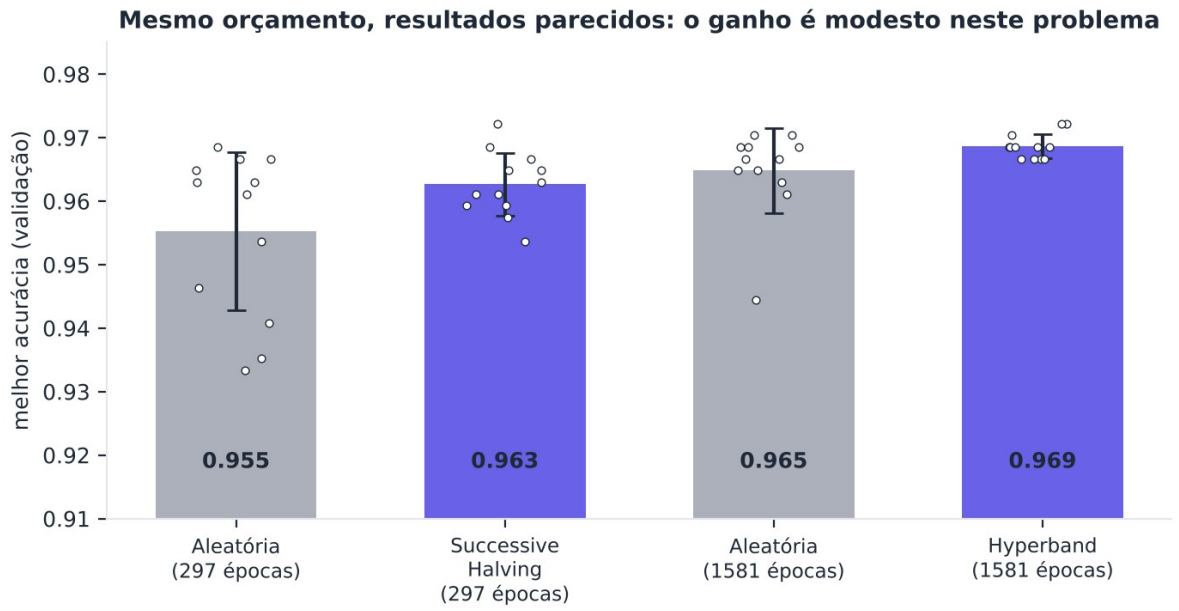


Figura 6.5 — Acurácia média de validação (12 sementes) para busca aleatória e métodos multifidelidade, com o mesmo orçamento de épocas. As barras de erro mostram o desvio padrão.

O que os números mostram:

- **Successive Halving contra aleatória.** Com o mesmo orçamento de 297 épocas, o corte sucessivo avalia 81 configurações, enquanto a busca aleatória só consegue treinar 3 até o fim. O resultado: 0,963 contra 0,955, com variação bem menor entre as sementes (a aleatória oscila mais, porque três sorteios dependem muito da sorte).
- **Hyperband contra aleatória.** Com 1.581 épocas, o Hyperband explora 143 configurações; a aleatória treina 19. A diferença é menor, 0,969 contra 0,965, mas o Hyperband é muito mais **estável** (desvio padrão de cerca de 0,002 contra 0,007).

Ou seja, o ganho em **acurácia média** é **modesto** neste problema, porque o espaço tem muitas configurações boas. O benefício mais claro aparece em dois pontos: o **orçamento pequeno** (o Halving) e a **estabilidade** (menos dependência de sorte). Em problemas em que cada treino é muito caro (redes neurais profundas, por exemplo), economizar épocas pesa mais ainda.

6.7 BOHB: multifidelidade com inteligência

O Hyperband sorteia os candidatos **ao acaso**. Uma melhoria natural é gerar candidatos **de forma inteligente**, aproveitando o que se aprendeu. É exatamente a ideia do **BOHB** (*Bayesian Optimization and Hyperband*), de Falkner, Klein e Hutter (2018).

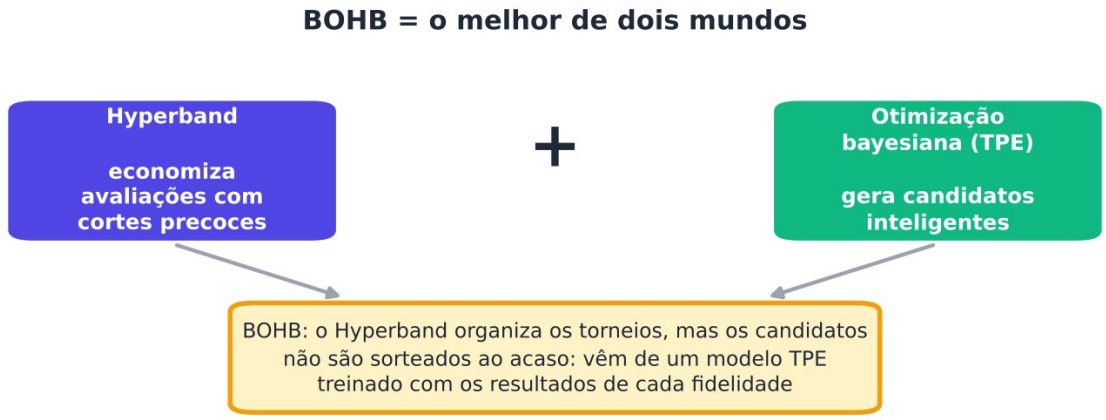


Figura 6.6 — O BOHB combina a organização em torneios do Hyperband com um modelo bayesiano (TPE) que sugere os candidatos.

O BOHB:

- usa o **Hyperband** para decidir quanto orçamento cada candidato recebe e quando cortar;
- usa um modelo do tipo **TPE** (Capítulo 5) para sugerir novos candidatos, treinado com os resultados observados em cada nível de fidelidade.

Os autores reportam que o BOHB combina **bom desempenho logo no início** (o Hyperband) com **convergência rápida para boas configurações** (a otimização bayesiana), superando ambos em uma série de problemas, de máquinas de vetores de suporte a redes neurais profundas. No Capítulo 10, você verá que ferramentas de AutoML embutem ideias desse tipo.

6.8 Podas no Optuna

O Optuna implementa a multifidelidade por meio de **podadores** (*pruners*). A ideia é permitir que uma tentativa **seja interrompida cedo**, se estiver indo mal em comparação com as outras (Akiba *et al.*, 2019).

Dentro da função-objetivo, a cada época você faz duas coisas:

1. `trial.report(nota, epoca)`: informa a nota parcial;
2. `if trial.should_prune(): raise optuna.TrialPruned()`: pergunta ao podador se a tentativa deve parar.

Dois podadores populares são o `MedianPruner` (interrompe tentativas cuja nota parcial está abaixo da **mediana** das outras na mesma época) e o `HyperbandPruner` (aplica a lógica do Hyperband).

```python
import numpy as np
import optuna

optuna.logging.set_verbosity(optuna.logging.WARNING)
from sklearn.datasets import load_digits
from sklearn.linear_model import SGDClassifier
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

X, y = load_digits(return_X_y=True)
Xtr, Xva, ytr, yva = train_test_split(X, y, test_size=0.3, stratify=y, random_state=0)
esc = StandardScaler().fit(Xtr); Xtr, Xva = esc.transform(Xtr), esc.transform(Xva)
```

```
EPOCAS = 60
def objetivo(trial):
    modelo = SGDClassifier(loss=trial.suggest_categorical("loss", ["hinge", "log_loss",
"modified_huber"]),
                           alpha=trial.suggest_float("alpha", 1e-8, 1e1, log=True),
                           eta0=trial.suggest_float("eta0", 1e-6, 1e1, log=True),
                           learning_rate="constant", random_state=0)
    for epoca in range(EPOCAS):
        modelo.partial_fit(Xtr, ytr, classes=np.arange(10))
        nota = modelo.score(Xva, yva)
        trial.report(nota, epoca)                    # nota parcial
        if trial.should_prune():                     # o podador manda parar?
            raise optuna.TrialPruned()
    return nota

for nome, podador in [("sem poda", optuna.pruners.NopPruner()), ("MedianPruner",
optuna.pruners.MedianPruner(n_startup_trials=5, n_warmup_steps=3))]:
    estudo = optuna.create_study(direction="maximize", sampler=optuna.samplers.TPESampler(seed=0),
pruner=podador)
    estudo.optimize(objetivo, n_trials=40)
    podadas = sum(t.state == optuna.trial.TrialState.PRUNED for t in estudo.trials)
    epocas = sum(len(t.intermediate_values) for t in estudo.trials)
    print(f"{nome:13s}: tentativas podadas = {podadas:2d}/40 | épocas treinadas = {epocas:4d} | melhor
acurácia = {estudo.best_value:.3f}")
```

SAÍDA

```
sem poda     : tentativas podadas =  0/40 | épocas treinadas = 2400 | melhor acurácia = 0.972
MedianPruner : tentativas podadas = 20/40 | épocas treinadas = 1297 | melhor acurácia = 0.969
```

O `MedianPruner` interrompe parte das tentativas, **economizando épocas** e chegando a uma nota parecida. Em tarefas caras (como redes neurais profundas), essa economia vira horas ou dias.

6.9 Armadilhas e boas práticas

1. **O ranking pode mudar.** Candidatos de "largada lenta" podem ser eliminados cedo. Use um orçamento inicial razoável e, se possível, um fator η menor (como 2) para cortes mais suaves.
2. **Fidelidade baixa demais não discrimina.** Se com 1 época todos parecem iguais, o corte é um sorteio.
3. **A fidelidade precisa ser monotônica o bastante.** Mais recurso deve, em geral, dar uma estimativa melhor. Verifique isso nas curvas de aprendizado.
4. **Não compare notas de fidelidades diferentes.** Compare apenas candidatos avaliados com o mesmo orçamento.
5. **Compare sempre com a busca aleatória** no mesmo orçamento, como fizemos. O ganho depende do problema.
6. **Guarde o estado do modelo** (checkpoints) para retomar o treino sem recomeçar, como o `partial_fit` permite.
7. **Avalie o vencedor em um teste final** treinado com orçamento completo.

Resumo do capítulo

MEMORIZE **Cartões de memorização**

- **Multifidelidade:** avaliar muitos candidatos barato e os promissores caro.
- **Fidelidade:** épocas, dados, atributos, resolução ou partes da validação.
- **Aposta central:** o ranking em baixa fidelidade se parece com o de alta.
- **Successive Halving:** treina todos pouco, mantém $1/\eta$ melhor, multiplica o orçamento por η.
- **Hyperband:** vários torneios com apostas diferentes; não depende de escolher o corte certo.
- **BOHB:** Hyperband + TPE (candidatos inteligentes).
- **Podadores (Optuna):** `report` e `should_prune` interrompem tentativas ruins.
- **Risco:** curvas que se cruzam fazem o corte precoce eliminar campeãs lentas.

Exercícios de múltipla escolha

Escolha **uma** alternativa em cada questão. O gabarito está no Apêndice A.

Questão 1. Qual é a ideia central dos métodos multifidelidade?

a) Usar apenas um candidato, escolhido ao acaso.
b) Eliminar a validação cruzada.
c) Treinar todos os candidatos até o fim, sem exceção.
d) Avaliar muitos candidatos de forma barata e aproximada e reservar a avaliação cara para os mais promissores.

Questão 2. Na analogia da peneira de futebol, o treino curto inicial corresponde a:

a) uma avaliação de baixa fidelidade.
b) o teste final.
c) o ajuste de hiperparâmetros do clube.
d) a validação cruzada.

Questão 3. Qual das opções NÃO é uma forma de fidelidade?

a) Número de atributos usados no teste rápido.
b) O nome do arquivo de dados.
c) Fração dos dados usada no treino.
d) Número de épocas de treino.

Questão 4. A aposta central da multifidelidade é que:

a) o ranking em baixa fidelidade se parece com o ranking em alta fidelidade.
b) o treino curto é sempre melhor que o longo.
c) a busca aleatória é inútil.
d) todos os candidatos terão o mesmo desempenho.

Questão 5. O que acontece quando a aposta central da multifidelidade falha?

a) Todos os candidatos são mantidos.
b) O método passa a treinar mais rápido.
c) A validação cruzada se torna desnecessária.
d) O corte precoce pode descartar candidatos que seriam campeões.

Questão 6. No Successive Halving com fator $\eta = 3$, após cada rodada:

a) reduz-se o orçamento por candidato a um terço.
b) mantêm-se todos os candidatos e triplica-se o orçamento.
c) mantém-se 1/3 melhor dos candidatos e triplica-se o orçamento por candidato.
d) descartam-se os melhores candidatos.

Questão 7. Com $n = 27$, $r = 1$ e $\eta = 3$, quantas épocas são gastas na rodada 2 do Successive Halving?

a) 3.
b) 27.
c) 81.
d) 9.

Questão 8. Com $n = 27$, $r = 1$ e $\eta = 3$, qual é o custo total do Successive Halving?

a) 27 épocas.
b) 729 épocas.
c) 81 épocas.
d) 108 épocas.

Questão 9. Treinar os mesmos 27 candidatos por 27 épocas cada custaria:

a) 27 épocas.
b) 729 épocas.
c) 81 épocas.
d) 108 épocas.

Questão 10. Qual método do scikit-learn aplica a ideia do halving a uma grade de hiperparâmetros?

a) DummyClassifier.
b) StratifiedKFold.
c) GridSearchCV padrão.
d) HalvingGridSearchCV.

Questão 11. O que é preciso fazer para usar HalvingGridSearchCV no scikit-learn?

a) Usar apenas com redes neurais.
b) Importar sklearn.experimental.enable_halving_search_cv antes.
c) Nada, pois é padrão.
d) Instalar um pacote extra pago.

Questão 12. Em HalvingGridSearchCV, o recurso padrão que cresce a cada rodada é:

a) o número de atributos.
b) o valor do hiperparâmetro C.
c) o número de classes.
d) o número de exemplos de treino.

Questão 13. No exemplo do capítulo, qual foi a economia aproximada de custo da busca halving em relação à grade completa?

a) Cerca de 83%.
b) Cerca de 99,9%.
c) Nenhuma.
d) Cerca de 10%.

Questão 14. O que é o Hyperband?

a) Um algoritmo que roda vários torneios de Successive Halving com apostas diferentes.
b) Um tipo de rede neural.
c) Um método de pré-processamento.
d) Um modelo de regressão.

Questão 15. Que dilema o Hyperband resolve?

a) Escolher entre treino e teste.
b) Escolher a métrica.
c) Decidir entre testar muitas configurações com pouco treino ou poucas com muito treino.
d) Escolher entre classificação e regressão.

Questão 16. No Hyperband, qual é o papel do parâmetro R?

a) O número de sementes.
b) O máximo de recurso (por exemplo, épocas) que uma configuração pode receber.
c) O número de classes.
d) O fator de regularização.

Questão 17. Com R = 81 e η = 3, quantos torneios (brackets) tem o Hyperband?

a) 5.
b) 81.
c) 3.
d) 4.

Questão 18. No torneio s = 0 do Hyperband, o que ocorre?

a) Usa-se apenas o teste final.
b) Muitas configurações recebem 1 época.
c) Poucas configurações recebem o treino completo, sem cortes.
d) Nenhuma configuração é treinada.

Questão 19. No experimento do capítulo, como o Successive Halving se saiu em relação à busca aleatória com o mesmo orçamento?

a) Não pôde ser comparado.
b) Muito pior.
c) Idêntico, sem diferença alguma.
d) Ligeiramente melhor e mais estável.

Questão 20. No experimento do capítulo, o principal benefício do Hyperband sobre a aleatória foi:

a) dispensar o teste final.
b) menos tempo de processamento que qualquer método.
c) maior estabilidade (menor desvio entre sementes), com ganho modesto de acurácia.
d) acurácia muito maior.

Questão 21. O que é o BOHB?

a) Um método de pré-processamento de texto.
b) Uma métrica de classificação.
c) Um novo tipo de kernel.
d) Uma combinação do Hyperband com otimização bayesiana (TPE).

Questão 22. Qual a vantagem do BOHB sobre o Hyperband puro?

a) Dispensa o orçamento.
b) Elimina a validação.
c) Os candidatos são escolhidos de forma inteligente, e não ao acaso.
d) Evita qualquer corte.

Questão 23. No Optuna, o método trial.report(nota, epoca) serve para:

a) informar a nota parcial da tentativa na época atual.
b) encerrar o estudo.
c) salvar o modelo em disco.
d) trocar o amostrador.

Questão 24. No Optuna, o que faz trial.should_prune()?

a) Reinicia o estudo.
b) Calcula a importância dos hiperparâmetros.
c) Remove todos os dados.
d) Pergunta ao podador se a tentativa deve ser interrompida.

Questão 25. O MedianPruner interrompe uma tentativa quando:

a) o número de épocas passa de 10.
b) a semente é par.
c) sua nota parcial está abaixo da mediana das notas das outras tentativas na mesma época.
d) a nota é igual a 1.

Questão 26. No experimento com o MedianPruner, o que se observou?

a) Cerca de metade das tentativas foi podada, economizando épocas, com nota final parecida.
b) A nota final caiu para 0,3.
c) O tempo triplicou.
d) Nenhuma tentativa foi podada.

Questão 27. Por que fidelidade baixa demais pode ser um problema?

a) Porque a validação cruzada para de funcionar.
b) Porque a avaliação passa a ser quase um sorteio, sem discriminar bons e maus candidatos.
c) Porque o treino fica muito caro.
d) Porque os dados somem.

Questão 28. Por que não se deve comparar notas obtidas em fidelidades diferentes?

a) Porque isso é proibido pelo scikit-learn.
b) Porque as notas são sempre iguais.
c) Porque só a acurácia pode ser comparada.
d) Porque elas medem coisas diferentes: uma avaliação com mais recurso é mais precisa.

Questão 29. Qual cuidado é recomendado para reduzir o risco de eliminar campeões de largada lenta?

a) Usar um orçamento inicial razoável e, se possível, um fator η menor.
b) Evitar usar curvas de aprendizado.
c) Usar um orçamento inicial de zero épocas.
d) Cortar 99% dos candidatos na primeira rodada.

Questão 30. Em um torneio do Hyperband, a quantidade de recurso por candidato:

a) aumenta a cada rodada, enquanto o número de candidatos diminui.
b) é sorteada aleatoriamente a cada época.
c) permanece constante em todas as rodadas.
d) diminui a cada rodada, enquanto o número de candidatos aumenta.

Exercícios discursivos

Responda com suas palavras, em poucas linhas. Compare depois com a resposta-modelo do Apêndice A.

1. Explique, com uma analogia do cotidiano, por que treinar todos os candidatos até o fim é um desperdício.
2. Defina fidelidade e dê quatro exemplos de como variá-la.
3. Qual é a hipótese em que se baseiam os métodos multifidelidade e o que ocorre quando ela falha?
4. Descreva o algoritmo Successive Halving passo a passo.
5. Para n = 81, r = 1 e η = 3, descreva as rodadas e calcule o custo total em épocas, comparando com treinar todos até 81 épocas.
6. Por que a nota de um candidato pode não crescer de forma regular com as épocas? Dê um exemplo do capítulo.
7. Interprete o resultado dos postos das três campeãs após 3 épocas (5, 10 e 20 de 40). O que isso mostra sobre o corte precoce?
8. Explique como funciona HalvingGridSearchCV e por que a nota final dele não é diretamente comparável à de uma grade completa.
9. Qual é o dilema que o Hyperband resolve e como ele o faz?
10. Descreva os parâmetros R e η do Hyperband e o que os torneios s = 4 e s = 0 representam.
11. Resuma os resultados do experimento do capítulo comparando Successive Halving, Hyperband e busca aleatória.
12. Por que o ganho em acurácia média foi modesto no experimento? Em que situações a multifidelidade tende a ajudar mais?
13. Explique a ideia do BOHB e por que ele combina o melhor dos dois métodos.
14. Explique como usar poda no Optuna: o que a função-objetivo deve fazer em cada época?
15. Interprete os resultados do MedianPruner no capítulo (20 de 40 tentativas podadas).
16. Cite três armadilhas dos métodos multifidelidade e como mitigá-las.
17. Por que é importante guardar o estado do modelo (checkpoint) em métodos multifidelidade?
18. Como o fator η afeta o Successive Halving? Compare η = 2 e η = 5.
19. Por que o capítulo recomenda avaliar o vencedor em um teste final treinado com orçamento completo?
20. Por que o Hyperband é mais robusto que um único Successive Halving quando não sabemos o quão confiável é o corte precoce?

Exercícios práticos em Python (scikit-learn)

Resolva os exercícios no seu computador (ou no Google Colab). Eles usam conjuntos de dados da UCI que já vêm no scikit-learn ou dados simulados. Tente resolver **antes** de consultar a solução, no Apêndice D. As saídas apresentadas lá foram obtidas com as versões de bibliotecas listadas na Apresentação.

Exercício 1 — Fazendo a conta do Successive Halving

NÍVEL: BÁSICO

Escreva uma função que, dados `n` (candidatos iniciais), `r` (recurso inicial) e `eta`, devolva a tabela de rodadas (candidatos, recurso por candidato, custo da rodada) e o custo total. Calcule para (n=64, r=1, η=4) e para (n=81, r=1, η=3). Compare cada custo total com o de treinar todos os candidatos até o recurso máximo atingido.

Pergunta: qual a economia em cada caso?

Exercício 2 — Halving aleatório na floresta

NÍVEL: INTERMEDIÁRIO

Com o conjunto **Digits** (UCI), compare `RandomizedSearchCV` e `HalvingRandomSearchCV` (com `resource="n_estimators"`, `max_resources=150`, `min_resources=10`, `factor=3`) para uma `RandomForestClassifier`. Use os mesmos hiperparâmetros: `max_depth` (inteiro de 3 a 25), `max_features` (real de 0,05 a 0,6) e `min_samples_leaf` (inteiro de 1 a 8). Use 30 candidatos em ambos e `cv=3`. Mostre candidatos por rodada, recursos por rodada, a melhor nota de cada um e o custo aproximado em "árvores treinadas".

Pergunta: quanto o halving economizou? A nota final ficou parecida?

Exercício 3 — Comparando podadores

NÍVEL: INTERMEDIÁRIO

No problema do `SGDClassifier` em Digits do capítulo (60 épocas, 40 tentativas, TPE com semente 0), compare três podadores do Optuna: `NopPruner` (sem poda), `MedianPruner(n_startup_trials=5, n_warmup_steps=3)` e `HyperbandPruner(min_resource=3, max_resource=60, reduction_factor=3)`. Para cada um, mostre quantas tentativas foram podadas, quantas épocas foram treinadas e a melhor acurácia.

Pergunta: qual podador economizou mais? A qualidade do resultado foi mantida?

Exercício 4 — O fator η importa

NÍVEL: AVANÇADO

Reutilize `Candidato` e `sucessivo` do capítulo. Para 81 candidatos por semente e 8 sementes, rode o Successive Halving com η = 2, 3 e 5 (com recurso inicial de 1 época e máximo de 81). Para cada η, mostre o gasto médio em épocas e a acurácia média da melhor configuração encontrada.

Pergunta: o que aconteceu com o custo e com a qualidade quando η cresce?

> SAIBA MAIS **Para ir além: bases públicas da UCI e do Kaggle**
>
> - **UCI:** Letter Recognition (letras, 20.000 exemplos, treinos mais caros) e Covertype (cobertura florestal, mais de meio milhão de exemplos, ideal para halving).
> - **Kaggle:** Santander Customer Transaction Prediction e Porto Seguro Safe Driver (conjuntos grandes em que a multifidelidade se destaca).
>
> Estes conjuntos não foram usados nem testados nos códigos do livro. Confira, nos sites, a descrição, a licença e as colunas antes de usá-los.

Projeto integrador do capítulo

Halving em um modelo de boosting: custo e qualidade

Fonte dos dados. *Optical Recognition of Handwritten Digits* (Digits), do repositório **UCI Machine Learning**, incluído no scikit-learn (`load_digits`).

Contexto. Uma equipe quer ajustar um `HistGradientBoostingClassifier`, cujo treino é mais caro. Ela precisa decidir se a busca com *halving* economiza custo sem perder qualidade em relação à busca aleatória comum.

Tarefas.

1. Separe 25% dos dados como **teste final** (estratificado, semente fixa).
2. Defina um espaço de busca para `learning_rate` (log, de 0,02 a 0,5), `max_leaf_nodes` (inteiro de 4 a 40), `l2_regularization` (log, de 1e-4 a 10) e `min_samples_leaf` (inteiro de 5 a 40).
3. Rode `RandomizedSearchCV` com 24 candidatos, `max_iter=100` fixo e `cv=3`, apenas no treino.
4. Rode `HalvingRandomSearchCV` com 24 candidatos e `resource="max_iter"` (`min_resources=10`, `max_resources=100`, `factor=3`), com o mesmo espaço e `cv=3`.
5. Calcule o **custo aproximado** de cada busca (candidatos × iterações × partes) e a economia.
6. Avalie os dois vencedores **uma única vez** no teste final.
7. Escreva um parágrafo (6 a 8 linhas): o halving economizou? A qualidade final foi parecida? O que o resultado diz sobre quando a multifidelidade compensa?

Entregáveis. Um notebook (ou script) comentado, a tabela de custos e o parágrafo de conclusões.

A solução de referência está no Apêndice E.



Capítulo 7 — Métodos evolutivos e programação genética

OBJETIVOS DE APRENDIZAGEM

Ao final deste capítulo você será capaz de:

- Explicar a analogia entre evolução biológica e busca de soluções (população, aptidão, seleção, cruzamento e mutação).
- Implementar um algoritmo genético simples para evoluir pipelines de aprendizado de máquina.
- Descrever a programação genética e a representação de pipelines como árvores.
- Explicar o que é otimização multiobjetivo, a fronteira de Pareto e a ideia do NSGA-II.
- Descrever como o TPOT funciona e usá-lo conceitualmente.
- Comparar métodos evolutivos, bayesianos e de bandidos quanto a vantagens e limites.

7.1 A natureza como otimizadora

Durante milhares de anos, agricultores e criadores melhoraram plantas e animais sem saber nada de genética. O método era simples. A cada safra, eles **escolhiam as melhores plantas**, usavam suas sementes para a safra seguinte e, de vez em quando, uma variação inesperada aparecia e, se fosse boa, era preservada. Geração após geração, o milho ficou mais produtivo, e os cães, mais adequados a cada tarefa.

Esse processo tem três ingredientes:

1. **Variação:** os indivíduos são diferentes entre si.
2. **Seleção:** os melhores têm mais chance de deixar descendentes.
3. **Hereditariedade:** os filhos se parecem com os pais, e misturam características deles.

É a lógica da **evolução**. E ela funciona como **método de busca**: quando um problema tem muitas soluções possíveis e não sabemos a fórmula do melhor ponto, podemos **simular a evolução** em um computador. Os métodos baseados nessa ideia são chamados de **computação evolutiva**.

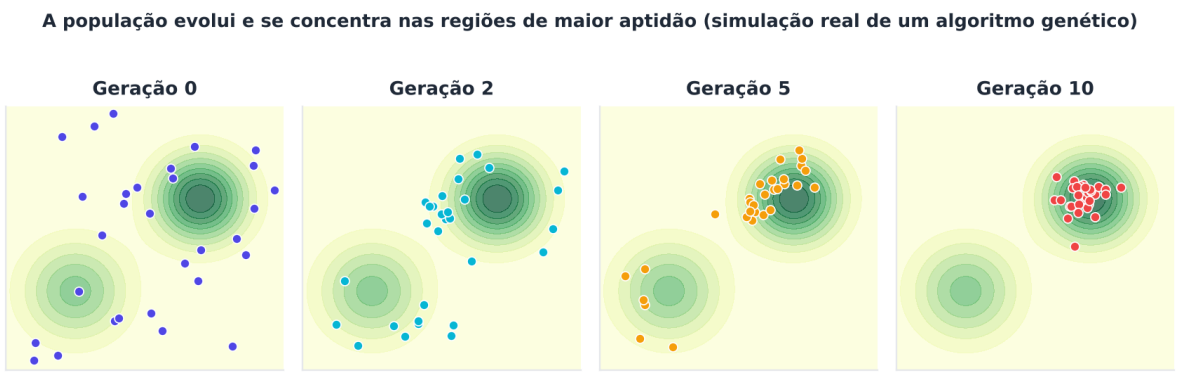


Figura 7.1 — Uma simulação real: uma população de soluções (pontos) evolui e se concentra nas regiões de maior aptidão (verde mais escuro).

Na figura acima, cada ponto é uma solução candidata em uma "paisagem" com dois morros. Aos poucos, a população deixa de se espalhar e passa a se concentrar nas regiões mais altas.

7.2 O vocabulário dos algoritmos genéticos

Os **algoritmos genéticos** (GA, de *genetic algorithms*) usam termos emprestados da biologia. Vamos traduzi-los para o nosso problema, que é montar um **pipeline de aprendizado de máquina**.

Termo	Significado em biologia	Significado para nós
Indivíduo	Um organismo	Um pipeline completo (escala, redução, modelo e hiperparâmetros)
Genoma	O DNA	A descrição do pipeline: lista de escolhas
Gene	Um trecho do DNA	Uma escolha (tipo de escala, valor de C...)
População	O conjunto dos organismos	Um grupo de pipelines (por exemplo, 14)
Aptidão (*fitness*)	Quão bem o organismo sobrevive	A nota de validação cruzada do pipeline
Geração	Um ciclo de reprodução	Uma rodada de seleção, cruzamento e mutação
Seleção	Os mais aptos reproduzem mais	Os pipelines de maior nota têm mais chance de ser "pais"
Cruzamento	Mistura de genes de dois pais	Combinar escolhas de dois pipelines
Mutação	Alteração aleatória de um gene	Mudar uma escolha ao acaso
Elitismo	—	Copiar os melhores direto para a próxima geração

O ciclo de um algoritmo genético é este:

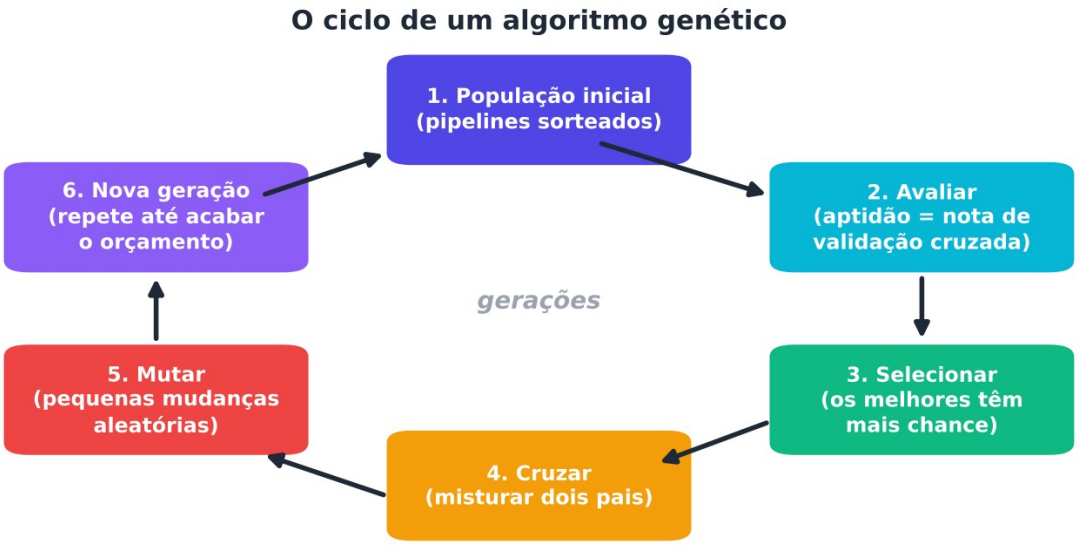


Figura 7.2 — O ciclo do algoritmo genético. A cada geração, a população é avaliada, os melhores são selecionados, e filhos são criados por cruzamento e mutação.

Os dois operadores de variação merecem um olhar mais atento.

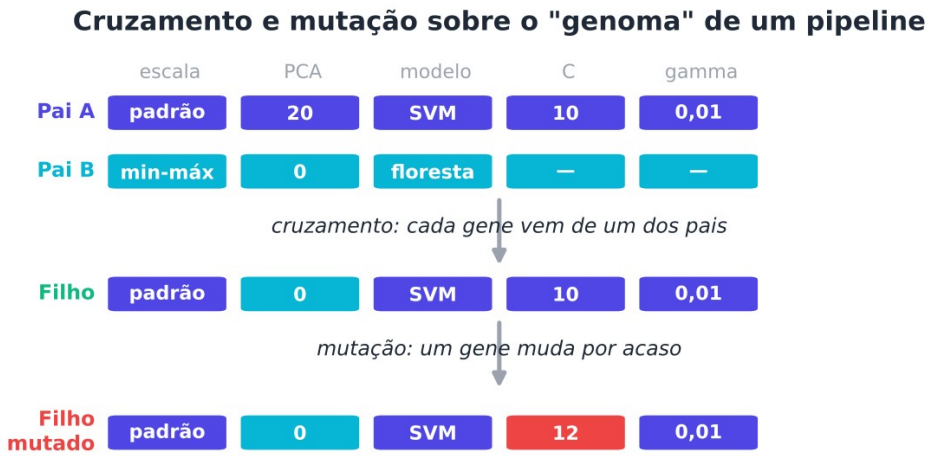


Figura 7.3 — O cruzamento combina genes de dois pais; a mutação altera um gene ao acaso.

- O **cruzamento** combina boas ideias: se o Pai A tem uma boa escala e o Pai B um bom modelo, o filho pode herdar as duas coisas.
- A **mutação** introduz novidade: sem ela, a população perderia diversidade e ficaria presa em um tipo de solução.

> **Memorize**
>
> **Seleção** concentra a busca nas boas soluções (aproveitar). **Mutação** mantém a diversidade (explorar). O mesmo dilema do Capítulo 5, resolvido com a ideia da evolução.

7.3 Um algoritmo genético do zero

Vamos construir um GA que **evolui pipelines** de classificação. É uma versão minúscula do que o TPOT faz, mas com todas as peças.

O problema. Para tornar a busca mais interessante, usaremos um conjunto **difícil de propósito**: 300 imagens do conjunto **Digits** (UCI), às quais acrescentamos **200 colunas de ruído** (números aleatórios sem informação). Assim, a escolha de escala e de redução de dimensionalidade realmente importa.

O genoma. Cada indivíduo é um dicionário com: o tipo de **escala** (nenhuma, padrão ou min-máx), o número de componentes do **PCA** (0 = sem PCA), o **modelo** (k-vizinhos, SVM, regressão logística ou floresta) e os hiperparâmetros do modelo escolhido.

```python
import copy
import numpy as np
from sklearn.datasets import load_digits
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler, MinMaxScaler
from sklearn.decomposition import PCA
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import cross_val_score, StratifiedKFold
```

```
# Dados: 300 imagens de Digits + 200 colunas de ruído
X0, y0 = load_digits(return_X_y=True)
X0 = X0 / 16.0
rng_dados = np.random.default_rng(0)
idx = rng_dados.choice(len(y0), 300, replace=False)
X = np.hstack([X0[idx], rng_dados.normal(size=(300, 200))])
y = y0[idx]
print("Formato dos dados:", X.shape, "(64 colunas úteis + 200 de ruído)")

def sortear_modelo(rng, modelo):
    """Sorteia os hiperparâmetros do modelo escolhido."""
    if modelo == "knn":       return {"k": int(rng.integers(1, 30))}
    if modelo == "svm":       return {"C": float(10 ** rng.uniform(-2, 3)), "gamma": float(10 **
rng.uniform(-5, 0))}
    if modelo == "logistica": return {"C": float(10 ** rng.uniform(-3, 3))}
    return {"arvores": int(rng.integers(20, 150)), "prof": int(rng.integers(3, 20))}

def sortear_individuo(rng):
    g = {"escala": str(rng.choice(["nenhuma", "padrao", "minmax"])),
         "pca": int(rng.choice([0, 0, 10, 20, 30, 40])),
         "modelo": str(rng.choice(["knn", "svm", "logistica", "floresta"]))}
    g.update(sortear_modelo(rng, g["modelo"]))
    return g
```

Agora transformamos o genoma em um pipeline do scikit-learn e definimos a **aptidão**: a nota de validação cruzada. Guardamos as notas já calculadas em um dicionário (*cache*), para não avaliar duas vezes o mesmo pipeline.

PYTHON

```
def construir(g):
    """Traduz o genoma em um pipeline do scikit-learn."""
    passos = []
    if g["escala"] == "padrao": passos.append(StandardScaler())
    if g["escala"] == "minmax": passos.append(MinMaxScaler())
    if g["pca"] > 0:            passos.append(PCA(g["pca"], random_state=0))
    m = g["modelo"]
    if m == "knn":        passos.append(KNeighborsClassifier(g["k"]))
    elif m == "svm":      passos.append(SVC(C=g["C"], gamma=g["gamma"]))
    elif m == "logistica": passos.append(LogisticRegression(C=g["C"], max_iter=300))
    else:                 passos.append(RandomForestClassifier(g["arvores"], max_depth=g["prof"],
random_state=0, n_jobs=-1))
    return make_pipeline(*passos)

def aptidao(g, X, y, cache):
    chave = str(sorted(g.items()))
    if chave not in cache:
        cache[chave] = float(cross_val_score(construir(g), X, y, cv=StratifiedKFold(3, shuffle=True,
random_state=0)).mean())
    return cache[chave]
```

Os **operadores genéticos** são o cruzamento (mistura de dois pais) e a mutação (mudanças aleatórias).

PYTHON

```
def cruzar(a, b, rng):
    filho = {}
```

```
    for k in ("escala", "pca"):                                  # cada gene vem de um dos pais
        filho[k] = a[k] if rng.random() < 0.5 else b[k]
    pai = a if rng.random() < 0.5 else b                         # o modelo (com seus hiperparâmetros)
vem de um pai
    filho["modelo"] = pai["modelo"]
    filho.update({k: v for k, v in pai.items() if k not in ("escala", "pca", "modelo")})
    return filho

def mutar(g, rng, p=0.3):
    g = copy.deepcopy(g)
    if rng.random() < p: g["escala"] = str(rng.choice(["nenhuma", "padrao", "minmax"]))
    if rng.random() < p: g["pca"] = int(rng.choice([0, 0, 10, 20, 30, 40]))
    if rng.random() < p * 0.7:                                   # às vezes troca o modelo inteiro
        g["modelo"] = str(rng.choice(["knn", "svm", "logistica", "floresta"]))
        for k in [k for k in g if k not in ("escala", "pca", "modelo")]: del g[k]
        g.update(sortear_modelo(rng, g["modelo"]))
    else:                                                        # ou ajusta levemente os
hiperparâmetros
        for k, v in list(g.items()):
            if k in ("escala", "pca", "modelo") or rng.random() >= 0.6: continue
            if k in ("C", "gamma"): g[k] = float(np.clip(v * 10 ** rng.normal(0, 0.4), 1e-5 if k ==
"gamma" else 1e-3, 1 if k == "gamma" else 1e3))
            elif k == "k":          g[k] = int(np.clip(v + rng.integers(-3, 4), 1, 40))
            elif k == "arvores":    g[k] = int(np.clip(v + rng.integers(-20, 21), 10, 200))
            elif k == "prof":       g[k] = int(np.clip(v + rng.integers(-3, 4), 2, 25))
    return g
```

Por fim, o laço evolutivo. A **seleção por torneio** sorteia 3 indivíduos e escolhe o melhor deles: os bons têm mais chance, mas os fracos ainda podem ser pais. O **elitismo** copia os 2 melhores direto para a geração seguinte.

PYTHON

```
def evoluir(X, y, semente, pop=14, geracoes=5, elite=2):
    rng = np.random.default_rng(semente)
    cache, populacao, historico, todos = {}, [sortear_individuo(rng) for _ in range(pop)], [], []
    for ger in range(geracoes + 1):
        notas = np.array([aptidao(g, X, y, cache) for g in populacao])
        todos += list(zip(populacao, notas))
        historico.append((len(cache), float(notas.max()), float(notas.mean())))
        if ger == geracoes:
            break
        ordem = np.argsort(-notas)
        nova = [copy.deepcopy(populacao[i]) for i in ordem[:elite]]                # elitismo
        torneio = lambda: populacao[max(rng.choice(pop, 3, replace=False), key=lambda i: notas[i])]
        while len(nova) < pop:
            nova.append(mutar(cruzar(torneio(), torneio(), rng), rng))
        populacao = nova
    return historico, todos

historico, todos = evoluir(X, y, semente=0)
```

PYTHON

```
print("geração | pipelines avaliados | melhor nota | nota média da população")
for ger, (n, melhor, media) in enumerate(historico):
    print(f"{ger:7d} | {n:19d} | {melhor:11.3f} | {media:.3f}")
```

SAÍDA

```
geração | pipelines avaliados | melhor nota | nota média da população
      0 |                  14 |       0.900 | 0.624
      1 |                  24 |       0.930 | 0.832
      2 |                  36 |       0.930 | 0.740
      3 |                  47 |       0.930 | 0.697
      4 |                  57 |       0.937 | 0.750
      5 |                  68 |       0.937 | 0.727
```

Observe duas coisas. A **melhor nota** sobe aos poucos (de 0,900 para 0,937) e, depois, estaciona. A **nota média da população** dá um salto logo na primeira geração (de 0,62 para 0,83), pois os piores pipelines são substituídos, e depois **oscila** entre 0,70 e 0,75. Esse é o efeito da seleção: a população inteira melhora logo de início, e não apenas o campeão. A oscilação posterior vem das mutações, como veremos abaixo.

> **Atenção**
>
> A nota média pode oscilar entre gerações, pois a mutação cria filhos ruins. Isso é normal e até desejável: a mutação é a "pesquisa" do algoritmo. Por isso o elitismo preserva os melhores.

7.4 Evolução contra busca aleatória

Resta a pergunta de sempre: **vale a pena?** Vamos comparar o algoritmo genético com a busca aleatória, dando a **ambos o mesmo número de pipelines avaliados**, em 5 sementes diferentes. Isso leva alguns minutos de processamento.

PYTHON

```python
def busca_aleatoria(X, y, semente, n_avaliacoes):
    rng = np.random.default_rng(10_000 + semente)
    cache, melhor = {}, 0.0
    for _ in range(n_avaliacoes):
        melhor = max(melhor, aptidao(sortear_individuo(rng), X, y, cache))
    return melhor

ga, al = [], []
for s in range(5):
    hist, _ = evoluir(X, y, semente=s)
    n_aval = hist[-1][0]
    ga.append(hist[-1][1]); al.append(busca_aleatoria(X, y, s, n_aval))
    print(f"semente {s}: {n_aval} pipelines avaliados | genético = {ga[-1]:.3f} | aleatória = {al[-
1]:.3f}")

print(f"\nMédia: genético = {np.mean(ga):.3f} | aleatória = {np.mean(al):.3f}")
print(f"O genético venceu em {sum(g > a for g, a in zip(ga, al))} de 5 sementes (empates: {sum(g == a
for g, a in zip(ga, al))}).")
```

SAÍDA

```
semente 0: 68 pipelines avaliados | genético = 0.937 | aleatória = 0.907
semente 1: 62 pipelines avaliados | genético = 0.900 | aleatória = 0.903
semente 2: 67 pipelines avaliados | genético = 0.930 | aleatória = 0.927
semente 3: 67 pipelines avaliados | genético = 0.930 | aleatória = 0.903
semente 4: 62 pipelines avaliados | genético = 0.920 | aleatória = 0.903

Média: genético = 0.923 | aleatória = 0.909
O genético venceu em 4 de 5 sementes (empates: 0).
```

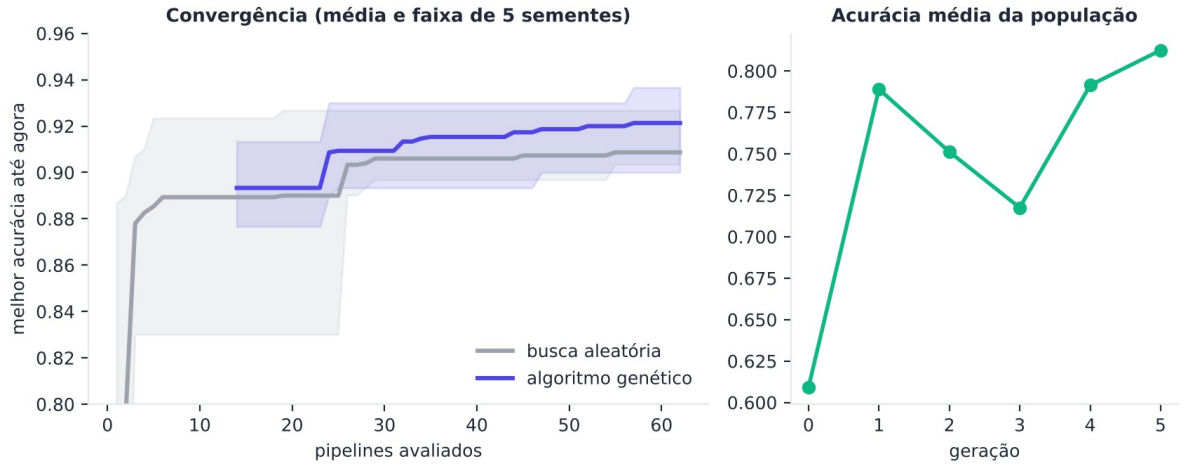


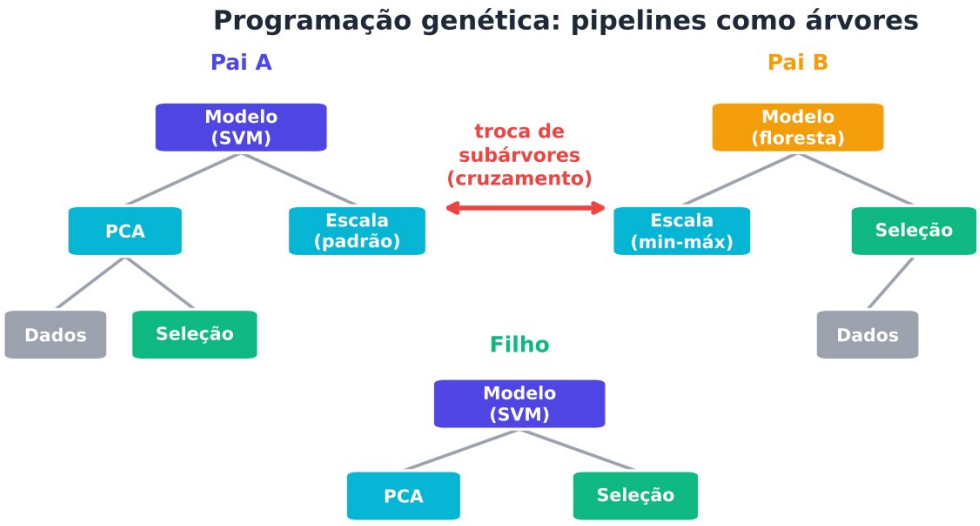
Figura 7.4 — À esquerda, a convergência do algoritmo genético e da busca aleatória (média e faixa de 5 sementes). À direita, a acurácia média da população ao longo das gerações.

Neste problema difícil, o algoritmo genético termina **à frente em média**, e vence em 4 das 5 sementes. Mas note a diferença modesta e a variação entre sementes: com apenas 5 repetições, não dá para dizer que ele é superior em geral. O que há de mais robusto é a **tendência**: o GA reaproveita boas combinações (cruzamento) e testa variações próximas delas (mutação), enquanto a aleatória nunca "aprende" com o que viu.

7.5 Programação genética e pipelines em árvore

O nosso genoma era uma **lista fixa** de escolhas. Mas um pipeline real pode ter formas muito variadas: duas etapas de pré-processamento, ou cinco; seleção de atributos antes ou depois; vários modelos empilhados. Uma lista fixa não representa isso bem.

A **programação genética** (GP) resolve isso representando cada solução como uma **árvore**. Cada **nó** é uma operação, e os filhos de um nó são as entradas dela.



Cada nó é uma etapa do pipeline. Trocar subárvores troca pedaços inteiros do tratamento dos dados.

Figura 7.5 — Pipelines como árvores. O cruzamento troca subárvores inteiras; a mutação substitui nós ou ramos.

Nesse formato:

- O **cruzamento** escolhe um ramo de cada pai e **troca as subárvores**. Pedaços inteiros do tratamento dos dados mudam de dono.
- A **mutação** pode trocar o tipo de um nó, mudar um hiperparâmetro, inserir ou remover um nó.
- Pipelines de **tamanhos e formas diferentes** convivem na mesma população.

Foi exatamente essa a ideia do **TPOT** (*Tree-based Pipeline Optimization Tool*). Olson e colaboradores (2016) implementaram, em Python, uma ferramenta que usa programação genética para evoluir pipelines do scikit-learn, incluindo pré-processadores, seletores de atributos e modelos (Olson *et al.*, 2016; Olson; Moore, 2016). No trabalho apresentado na oficina de AutoML do ICML, a ferramenta foi avaliada em 150 tarefas de classificação supervisionada (Olson; Moore, 2016).

7.6 Dois objetivos ao mesmo tempo

Há um problema clássico em programação genética: a evolução tende a produzir pipelines **cada vez maiores e mais complicados**, porque acrescentar etapas costuma aumentar levemente a nota. Mas um pipeline complicado é difícil de explicar, lento e arriscado.

A solução natural é pedir **dois objetivos** ao mesmo tempo: **maximizar a acurácia** e **minimizar a complexidade**. É a **otimização multiobjetivo**.

Com dois objetivos, geralmente não existe um "melhor" absoluto. Um pipeline pode ser mais preciso, mas mais complexo; outro, mais simples, mas um pouco menos preciso. Dizemos que um pipeline **domina** outro quando é **melhor ou igual em ambos os objetivos e estritamente melhor em pelo menos um**. O conjunto dos pipelines que **ninguém domina** é a **fronteira de Pareto**: são as melhores concessões possíveis entre os dois objetivos.

Analogia: você quer um carro rápido e econômico. Nenhum carro é o melhor nos dois. Mas há uma "fronteira" de carros em que, para ganhar em um aspecto, você precisa perder no outro. Os carros que estão fora da fronteira são sempre piores que algum carro dentro dela.

O algoritmo **NSGA-II** (Deb *et al.*, 2002) é um algoritmo genético multiobjetivo muito usado. Ele introduz uma forma rápida de ordenar os indivíduos em camadas de dominância (a "ordenação rápida por não dominância") e valoriza a diversidade ao longo da fronteira. Uma variante do TPOT usa a seleção do NSGA-II para tratar o número de operadores do pipeline como segundo objetivo, produzindo pipelines compactos sem sacrificar a acurácia (Olson *et al.*, 2016).

Vamos calcular uma fronteira de Pareto de verdade. Reaproveitamos todos os pipelines avaliados pelo nosso GA (semente 0) e tomamos como segundo objetivo o **número de atributos usados pelo modelo**: o PCA reduz de 264 colunas para 10 a 40 componentes, e uma floresta ou SVM sem PCA usa todas as 264.

```python
import numpy as np

# Cada indivíduo avaliado: (genoma, nota). Objetivo 1: nota (maximizar). Objetivo 2: nº de atributos (minimizar).
avaliados = {}
for g, nota in todos:
    avaliados[str(sorted(g.items()))] = (g, float(nota))
pontos = [(g["pca"] if g["pca"] > 0 else X.shape[1], nota, g) for g, nota in avaliados.values()]

def domina(a, b):
    """a domina b: não é pior em nenhum objetivo e é melhor em pelo menos um."""
    return a[0] <= b[0] and a[1] >= b[1] and (a[0] < b[0] or a[1] > b[1])
```

```
fronteira = [p for p in pontos if not any(domina(q, p) for q in pontos)]
fronteira.sort(key=lambda p: p[0])
print(f"Pipelines distintos avaliados: {len(pontos)} | na fronteira de Pareto: {len(fronteira)}")
print("\natributos | acurácia | pipeline")
for n_atr, nota, g in fronteira:
    ajustes = {k: round(v, 3) if isinstance(v, float) else v for k, v in g.items() if k not in
("escala", "pca", "modelo")}
    descricao = f"escala={g['escala']}, PCA={g['pca']}, {g['modelo']} {ajustes}"
    print(f"{n_atr:9d} | {nota:8.3f} | {descricao}")
```

SAÍDA

```
Pipelines distintos avaliados: 68 | na fronteira de Pareto: 4

atributos | acurácia | pipeline
       10 |    0.903 | escala=minmax, PCA=10, svm {'C': 725.389, 'gamma': 0.031}
       20 |    0.930 | escala=minmax, PCA=20, svm {'C': 1000.0, 'gamma': 0.027}
       30 |    0.937 | escala=minmax, PCA=30, svm {'C': 327.15, 'gamma': 0.042}
       30 |    0.937 | escala=minmax, PCA=30, svm {'C': 327.15, 'gamma': 0.046}
```

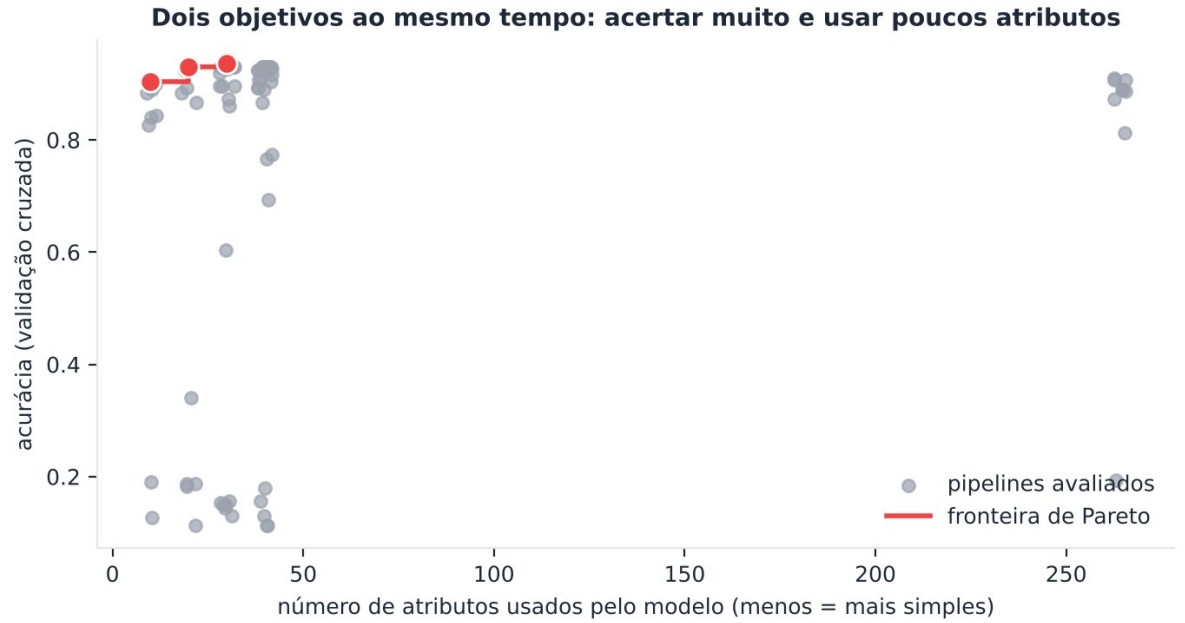


Figura 7.6 — A fronteira de Pareto (em vermelho) mostra as melhores concessões entre acurácia e simplicidade. Pontos cinza estão dominados.

A fronteira dá **opções** a quem decide: o pipeline mais preciso, o mais enxuto e algumas concessões no meio. Uma pessoa pode escolher o ponto que melhor equilibra acerto e simplicidade para o seu contexto. Essa é a vantagem dos métodos multiobjetivo: em vez de esconder a decisão em uma média, eles **mostram o dilema**.

7.7 O TPOT na prática

O TPOT usa exatamente a estrutura que estudamos: população de pipelines em árvore, avaliação por validação cruzada, seleção multiobjetivo, cruzamento e mutação. Ao terminar, ele **exporta o melhor pipeline como código Python do scikit-learn**, que você pode ler, editar e reutilizar. Isso é um grande ponto a favor da transparência.

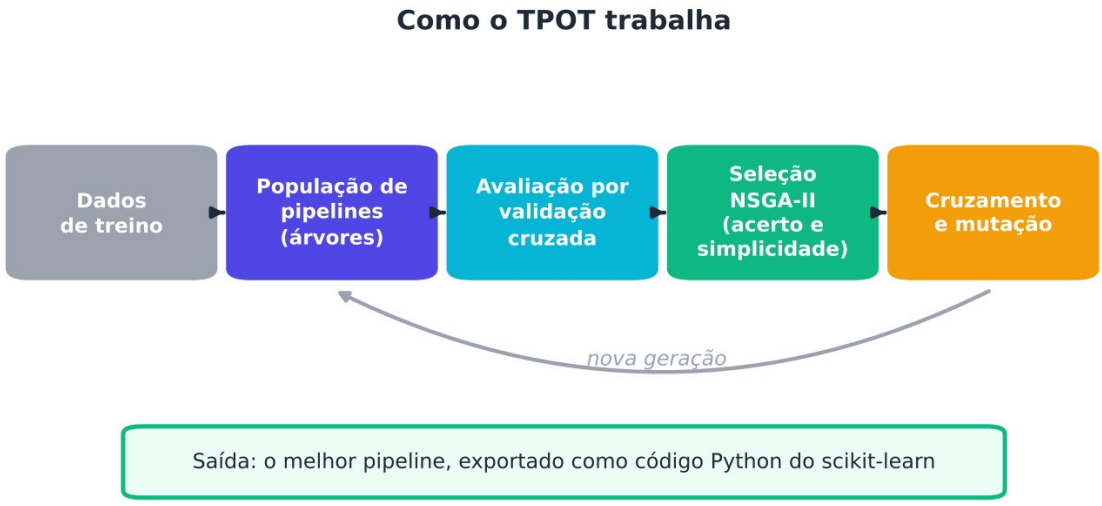


Figura 7.7 — O fluxo do TPOT: de uma população de pipelines evoluída por validação cruzada e seleção multiobjetivo até o código Python do melhor pipeline.

> **Atenção**
>
> O código abaixo **não foi executado** durante a elaboração deste livro: não foi possível instalar o TPOT no ambiente em que o livro foi preparado. Ele mostra a estrutura de uso no estilo da versão clássica da biblioteca. As interfaces de bibliotecas de AutoML mudam entre versões; consulte a documentação atual do TPOT antes de usar e adapte os nomes dos argumentos.

```python
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from tpot import TPOTClassifier

X, y = load_digits(return_X_y=True)
X_treino, X_teste, y_treino, y_teste = train_test_split(X, y, test_size=0.25, stratify=y,
random_state=0)

tpot = TPOTClassifier(generations=5,        # nº de gerações
                      population_size=20,   # nº de pipelines por geração
                      cv=5,                 # validação cruzada
                      random_state=0,
                      n_jobs=-1,
                      verbosity=2)
tpot.fit(X_treino, y_treino)                # evolui os pipelines (pode levar de minutos a horas)
print("Acurácia no teste:", tpot.score(X_teste, y_teste))
tpot.export("melhor_pipeline.py")           # grava o código Python do melhor pipeline
```

Mesmo sem executá-lo, você já sabe o que acontece dentro dele, porque construímos uma versão minúscula do mesmo mecanismo na Seção 7.3.

7.8 Vantagens e limites dos métodos evolutivos

Vantagens:

- **Flexibilidade.** Lidam com espaços de busca estranhos: estruturas variáveis, categorias, condicionais, objetivos múltiplos.

- **Paralelismo natural.** Os indivíduos de uma geração podem ser avaliados ao mesmo tempo.
- **Diversidade.** Uma população mantém várias soluções vivas, o que reduz o risco de ficar preso em uma só.
- **Transparência.** O resultado pode ser um pipeline legível, exportado como código.

Limites:

- **Custo.** Cada geração avalia dezenas de pipelines. Em dados grandes, o custo cresce depressa.
- **Aleatoriedade.** Duas execuções podem dar resultados diferentes. Fixe sementes e repita.
- **Muitos parâmetros do próprio método.** Tamanho da população, número de gerações, taxas de mutação e de cruzamento. Escolhê-los é, em si, um problema de hiperparâmetros.
- **Tendência à complexidade.** Sem controle (como o segundo objetivo), os pipelines crescem.
- **Overfitting da busca.** Como em todo AutoML, testar muitos pipelines infla a nota do melhor (Capítulos 2 e 4). Use teste final intocado.

Família	Ideia central	Pontos fortes	Cuidados
Aleatória / grade	Sorteio ou varredura	Simples; paralela; linha de base	Não aprende com o histórico
Bayesiana (GP, TPE, SMAC)	Modelo substituto + aquisição	Eficiente com poucas avaliações caras	Sequencial; sofre com dimensões altas (GP)
Multifidelidade (Halving, Hyperband)	Cortar cedo candidatos fracos	Economiza treino	O ranking pode mudar com o treino
Evolutiva (GA, GP, TPOT)	População, seleção, cruzamento, mutação	Estruturas variáveis; multiobjetivo; paralela	Custo; aleatoriedade; complexidade

> **Memorize**
>
> Nenhuma família vence sempre. A escolha depende do **espaço de busca** (simples ou estruturado), do **custo de cada avaliação** e do **orçamento**. E a **busca aleatória** continua sendo a linha de base de comparação.

7.9 Boas práticas

1. **Defina o orçamento:** população, número de gerações e tempo máximo.
2. **Fixe sementes** e repita a evolução, relatando média e variação.
3. **Compare com a busca aleatória** no mesmo número de avaliações.
4. **Use elitismo** para não perder os melhores.
5. **Controle a complexidade** (segundo objetivo ou limite de tamanho).
6. **Use o cache** de avaliações, para não repetir pipelines idênticos.
7. **Avalie o resultado final** em um teste intocado e **leia o pipeline exportado**: entenda o que ele faz antes de usá-lo.

Resumo do capítulo

MEMORIZE **Cartões de memorização**

- **Computação evolutiva:** busca por simulação da evolução (variação + seleção + hereditariedade).
- **Algoritmo genético:** população de indivíduos; aptidão = nota de validação; seleção, cruzamento, mutação, elitismo.
- **Cruzamento** combina boas ideias; **mutação** mantém diversidade.
- **Programação genética:** soluções como árvores; cruzamento troca subárvores.
- **TPOT:** evolui pipelines do scikit-learn por programação genética e exporta código Python.
- **Multiobjetivo:** acurácia × complexidade; **fronteira de Pareto** = melhores concessões.
- **NSGA-II:** GA multiobjetivo com ordenação por não dominância.
- **Nenhuma família vence sempre:** compare com a busca aleatória.

Exercícios de múltipla escolha

Escolha **uma** alternativa em cada questão. O gabarito está no Apêndice A.

Questão 1. Quais são os três ingredientes da evolução que inspiram os algoritmos evolutivos?

a) Treino, validação e teste.
b) Média, mediana e moda.
c) Variação, seleção e hereditariedade.
d) Entrada, saída e erro.

Questão 2. Em um algoritmo genético aplicado a pipelines, o que representa um "indivíduo"?

a) Uma classe do problema.
b) Um atributo isolado.
c) Um exemplo da tabela de dados.
d) Um pipeline completo, descrito pelo seu genoma.

Questão 3. O que é a aptidão (fitness) de um indivíduo no nosso problema?

a) O número de genes.
b) O tempo de execução do computador.
c) A nota de validação cruzada do pipeline.
d) O tamanho do conjunto de dados.

Questão 4. O que é o genoma de um pipeline?

a) O conjunto de dados de treino.
b) O arquivo em que o modelo é salvo.
c) A lista de escolhas que o descreve (escala, PCA, modelo, hiperparâmetros).
d) A tabela de resultados.

Questão 5. O papel da seleção em um algoritmo genético é:

a) dar mais chance de reprodução aos indivíduos mais aptos.
b) eliminar a necessidade de avaliar os indivíduos.
c) mudar genes ao acaso.
d) padronizar os atributos.

Questão 6. O cruzamento (crossover) consiste em:

a) combinar genes de dois pais para formar um filho.
b) apagar o indivíduo de menor aptidão.
c) mudar um gene ao acaso.
d) treinar o modelo com mais dados.

Questão 7. Qual é a principal função da mutação?

a) Garantir que o melhor indivíduo seja preservado.
b) Manter a diversidade da população e introduzir novidade.
c) Reduzir o número de gerações.
d) Calcular a aptidão.

Questão 8. O elitismo, em um algoritmo genético, é:

a) copiar os melhores indivíduos diretamente para a próxima geração.
b) eliminar os melhores indivíduos.
c) usar apenas um pai.
d) aumentar a taxa de mutação.

Questão 9. Na seleção por torneio com 3 competidores, o que se faz?

a) Escolhem-se os 3 melhores da população inteira.
b) Escolhem-se os 3 piores.
c) Sorteiam-se 3 indivíduos e escolhe-se o melhor deles como pai.
d) Cruzam-se os 3 entre si.

Questão 10. O que o dicionário cache faz na função aptidao?

a) Remove genes ruins.
b) Aumenta a taxa de mutação.
c) Evita avaliar duas vezes o mesmo pipeline.
d) Embaralha a população.

Questão 11. Por que o capítulo usou 300 imagens de Digits com 200 colunas de ruído?

a) Porque o scikit-learn exige colunas de ruído.
b) Para reduzir o número de classes.
c) Para tornar a busca mais difícil e fazer as escolhas de escala e PCA importarem.
d) Para evitar o uso de validação cruzada.

Questão 12. O que se observa na nota média da população ao longo das gerações?

a) É sempre constante.
b) Tende a subir, embora possa oscilar, pois a mutação cria filhos ruins.
c) Cai sempre.
d) É sempre igual à melhor nota.

Questão 13. No experimento do capítulo, em quantas das 5 sementes o algoritmo genético superou a busca aleatória?

a) 4 de 5.
b) 2 de 5.
c) 5 de 5.
d) 0 de 5.

Questão 14. Por que a vantagem do algoritmo genético no experimento deve ser interpretada com cautela?

a) Porque o GA não foi avaliado.
b) Porque o experimento não usou validação cruzada.
c) Porque a busca aleatória usou mais avaliações.
d) Porque foram só 5 repetições e a diferença média foi pequena.

Questão 15. Por que uma lista fixa de escolhas representa mal alguns pipelines reais?

a) Porque os modelos não têm hiperparâmetros.
b) Porque pipelines reais podem ter números e formas variados de etapas.
c) Porque listas não funcionam em Python.
d) Porque pipelines só têm uma etapa.

Questão 16. Na programação genética, cada solução é representada como:

a) um vetor de 0 e 1 fixo.
b) uma árvore, em que cada nó é uma operação.
c) um número real.
d) uma tabela de dados.

Questão 17. No cruzamento da programação genética, o que acontece?

a) As árvores são ordenadas por tamanho.
b) Subárvores dos dois pais são trocadas.
c) O pai de maior nota é copiado sem alterações.
d) Os dados são divididos em treino e teste.

Questão 18. O que é o TPOT?

a) Um tipo de rede neural.
b) Um algoritmo de busca em grade.
c) Uma ferramenta de AutoML que usa programação genética para evoluir pipelines do scikit-learn.
d) Uma métrica de classificação.

Questão 19. Qual é o produto final do TPOT?

a) Um relatório em PDF sem código.
b) O código Python do scikit-learn do melhor pipeline encontrado, que pode ser lido e editado.
c) Apenas a acurácia.
d) O conjunto de teste.

Questão 20. Qual problema comum em programação genética motiva um segundo objetivo de simplicidade?

a) A ausência de validação cruzada.
b) O excesso de dados.
c) A falta de aleatoriedade.
d) A tendência de produzir pipelines cada vez maiores e mais complicados.

Questão 21. Em otimização multiobjetivo, quando um pipeline "domina" outro?

a) Quando é melhor ou igual em todos os objetivos e estritamente melhor em pelo menos um.
b) Quando tem a maior acurácia, independentemente da complexidade.
c) Quando foi avaliado mais vezes.
d) Quando é o mais simples, independentemente da acurácia.

Questão 22. O que é a fronteira de Pareto?

a) A lista de todos os pipelines avaliados.
b) O pipeline de maior acurácia, apenas.
c) O conjunto de soluções que nenhuma outra domina: as melhores concessões entre os objetivos.
d) O pior conjunto de soluções.

Questão 23. Qual algoritmo multiobjetivo clássico é citado no capítulo?

a) DBSCAN.
b) PCA.
c) K-means.
d) NSGA-II.

Questão 24. No capítulo, qual foi o segundo objetivo usado no cálculo da fronteira de Pareto?

a) A semente aleatória.
b) O número de classes.
c) O número de atributos usados pelo modelo.
d) O tempo do dia.

Questão 25. No exemplo de código do TPOT, o que faz tpot.export("melhor_pipeline.py")?

a) Calcula a fronteira de Pareto.
b) Grava o código do melhor pipeline em um arquivo Python.
c) Remove o pipeline.
d) Treina o modelo novamente.

Questão 26. Por que o capítulo avisa que o código do TPOT não foi executado na elaboração do livro?

a) Porque o TPOT só funciona em Windows.
b) Porque o TPOT é pago.
c) Porque não foi possível instalar o TPOT no ambiente de preparo, e a interface da biblioteca muda entre versões.
d) Porque o código tem erros.

Questão 27. Qual das opções é uma desvantagem típica dos métodos evolutivos?

a) Incapacidade de lidar com categorias.
b) Custo computacional elevado e dependência da aleatoriedade.
c) Obrigação de usar sempre um único objetivo.
d) Impossibilidade de paralelização.

Questão 28. Qual alternativa descreve melhor o paralelismo nos algoritmos genéticos?

a) Só um indivíduo pode ser avaliado por vez.
b) Não existe paralelismo possível.
c) O paralelismo exige sempre uma GPU.
d) Os indivíduos de uma geração podem ser avaliados ao mesmo tempo.

Questão 29. Qual das famílias abaixo usa um modelo substituto com função de aquisição?

a) Algoritmos genéticos.
b) Busca aleatória.
c) Busca em grade.
d) Otimização bayesiana.

Questão 30. O que acontece com a diversidade de uma população se não houver mutação?

a) Ela deixa de importar.
b) Ela tende a diminuir, e a busca pode ficar presa em um tipo de solução.
c) Ela aumenta indefinidamente.
d) Ela permanece idêntica sempre.

Exercícios discursivos

Responda com suas palavras, em poucas linhas. Compare depois com a resposta-modelo do Apêndice A.

1. Explique a analogia entre o trabalho de um criador de plantas e um algoritmo genético.
2. Defina, para o problema de evoluir pipelines, os termos indivíduo, genoma, aptidão e geração.
3. Descreva o ciclo de um algoritmo genético.
4. Compare cruzamento e mutação quanto ao papel de cada um na busca.
5. O que é elitismo e por que ele é útil?
6. Explique a seleção por torneio e por que ela mantém alguma diversidade.
7. Descreva o experimento do capítulo que comparou o algoritmo genético e a busca aleatória e interprete o resultado.
8. Por que a nota média da população pode oscilar entre gerações, mesmo com o algoritmo funcionando bem?
9. Explique por que a programação genética representa soluções como árvores e dê uma vantagem disso.
10. Descreva, em linhas gerais, como o TPOT funciona.
11. Por que a programação genética tende a produzir pipelines cada vez maiores e como mitigar isso?
12. Explique o conceito de dominância de Pareto e o de fronteira de Pareto.
13. Dê um exemplo do cotidiano (diferente do capítulo) de fronteira de Pareto.
14. Como o capítulo calculou a fronteira de Pareto de pipelines e o que ela oferece a quem decide?
15. Cite duas vantagens e duas limitações dos métodos evolutivos.
16. Por que o capítulo diz que o tamanho da população e a taxa de mutação são "hiperparâmetros do método"? Que consequência isso traz?
17. Compare as famílias aleatória, bayesiana, multifidelidade e evolutiva quanto à ideia central e a um cuidado de cada uma.
18. Por que é importante ler o pipeline exportado pelo TPOT antes de usá-lo?
19. Como o overfitting da busca se manifesta em métodos evolutivos e como evitá-lo?
20. Liste quatro boas práticas ao usar métodos evolutivos para AutoML.

Exercícios práticos em Python (scikit-learn)

Resolva os exercícios no seu computador (ou no Google Colab). Eles usam conjuntos de dados da UCI que já vêm no scikit-learn ou dados simulados. Tente resolver **antes** de consultar a solução, no Apêndice D. As saídas apresentadas lá foram obtidas com as versões de bibliotecas listadas na Apresentação.

Exercício 1 — Seleção por torneio: quem é escolhido?

NÍVEL: BÁSICO

Considere uma população de 20 indivíduos com aptidões de 1 a 20 (posto 1 = pior, posto 20 = melhor). Simule 20.000 torneios para cada tamanho de torneio k = 2, 3 e 5 (sorteie k indivíduos sem repetição e escolha o de maior aptidão). Mostre a frequência com que o melhor indivíduo e o pior indivíduo são escolhidos, e a aptidão média dos escolhidos.

Pergunta: como o tamanho do torneio controla a "pressão de seleção"?

Exercício 2 — Algoritmo genético para os hiperparâmetros de um SVM

NÍVEL: INTERMEDIÁRIO

No conjunto **Digits** (UCI), implemente um GA com codificação real para otimizar `log10(C)` (de −2 a 3) e `log10(gamma)` (de −5 a 0) de um SVM. Use população de 8, 5 gerações (48 avaliações no total, contando a população inicial), seleção por torneio de 3, cruzamento por média ponderada, mutação gaussiana (desvio de 0,3) e elitismo de 1. A aptidão é a acurácia de validação cruzada de 3 partes. Compare, com 5 sementes, com a busca aleatória de 48 sorteios.

Pergunta: o GA superou a aleatória? Por que o resultado pode ser um empate?

Exercício 3 — Fronteira de Pareto: acurácia × esparsidade

NÍVEL: INTERMEDIÁRIO

No conjunto **Breast Cancer Wisconsin** (UCI), ajuste uma `LogisticRegression` com penalidade L1 (`penalty="l1"`, `solver="liblinear"`) para 15 valores de `C` entre 10^{-3} e 10^{2} (escala logarítmica), com `StandardScaler`. Para cada valor, calcule a acurácia de validação cruzada de 5 partes e o **número de coeficientes diferentes de zero** (ajuste no conjunto completo). Calcule a fronteira de Pareto (maximizar acurácia, minimizar coeficientes) e mostre-a.

Pergunta: qual a menor quantidade de atributos que ainda mantém a acurácia próxima do máximo?

Exercício 4 — Mutação e elitismo em um GA simples

NÍVEL: AVANÇADO

Maximize a função-agulha do Capítulo 5, `f(x) = 0,6·exp(−((x−3)/1,2)²) + 1,0·exp(−((x−8,2)/0,35)²)`, em [0, 10], com um GA de população 10 e 8 gerações (80 avaliações). Compare quatro configurações: (mutação σ = 0 e sem elitismo), (σ = 0 e com elitismo), (σ = 0,5 e sem elitismo) e (σ = 0,5 e com elitismo de 1). Use 40 sementes e relate a média do melhor valor encontrado e a fração das execuções que achou a agulha (valor maior que 0,95).

Pergunta: qual é o papel de cada componente?

SAIBA MAIS **Para ir além: bases públicas da UCI e do Kaggle**

- **UCI:** Wine Quality e Heart Disease (conjuntos pequenos, bons para treinar a fronteira de Pareto).
- **Kaggle:** Heart Failure Prediction (poucos atributos clínicos) e Credit Card Fraud Detection (muito desbalanceado, para estudar métricas).

Estes conjuntos não foram usados nem testados nos códigos do livro. Confira, nos sites, a descrição, a licença e as colunas antes de usá-los.

Projeto integrador do capítulo

Um mini-TPOT com fronteira de Pareto em Breast Cancer

Fonte dos dados. *Breast Cancer Wisconsin (Diagnostic)*, do repositório **UCI Machine Learning**, incluído no scikit-learn. Para tornar o problema mais interessante, você vai usar 250 exemplos e acrescentar 100 colunas de ruído.

Contexto. Uma equipe quer um pipeline que acerte muito e que seja o mais **enxuto** possível. Você usará um algoritmo genético de pipelines (como o do capítulo) e analisará a fronteira de Pareto.

Atenção ética. Os dados são de uso didático. O modelo **não é diagnóstico médico**.

Tarefas.

1. Monte o conjunto: 250 exemplos de `load_breast_cancer` mais 100 colunas de ruído normal. Separe 30% para **teste final** (estratificado, semente fixa).
2. Implemente um genoma com: escala (nenhuma ou padrão), `SelectKBest` com `k` em {5, 10, 20, 40, todos} e modelo (regressão logística, k-vizinhos ou floresta) com hiperparâmetros.
3. Rode um GA (população 12, 5 gerações, elitismo 2, torneio de 3) **apenas no treino**, com validação cruzada de 3 partes, e uma busca aleatória com o mesmo número de avaliações. Use 3 sementes.
4. Para a semente 0, calcule a **fronteira de Pareto** entre acurácia de validação e número de atributos usados.
5. Escolha o pipeline mais acurado e o pipeline "enxuto" (o menor da fronteira que perde no máximo 1 ponto percentual de acurácia para o mais acurado). Avalie ambos **uma única vez** no teste final.
6. Escreva um parágrafo (6 a 8 linhas): o GA superou a aleatória? Qual pipeline você escolheria e por quê? Quais limitações existem?

Entregáveis. Um notebook (ou script) comentado, a tabela da fronteira e o parágrafo de conclusões.

A solução de referência está no Apêndice E.

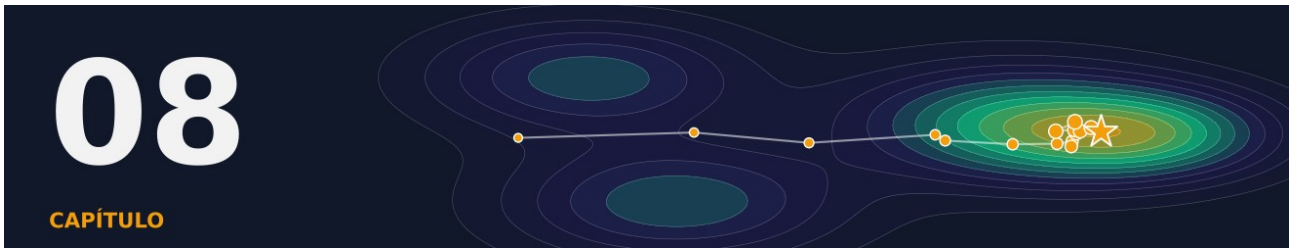
PARTE III

Parte III — Automação do Pipeline

Automatizar o que vem antes e depois do modelo: atributos, meta-aprendizado, ensembles e as principais ferramentas de AutoML em ação.

Nesta parte:

- Cap. 8 — Engenharia e seleção automática de atributos
- Cap. 9 — Meta-aprendizado, warm start e ensembles
- Cap. 10 — Ferramentas de AutoML na prática



Capítulo 8 — Engenharia e seleção automática de atributos

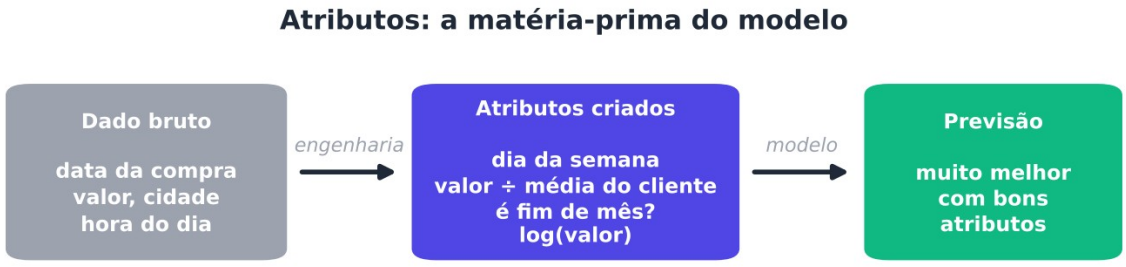
OBJETIVOS DE APRENDIZAGEM

Ao final deste capítulo você será capaz de:

- Explicar o que é engenharia de atributos e por que ela costuma pesar mais que a escolha do algoritmo.
- Aplicar transformações úteis (quadrados, logaritmos, interações, razões) e avaliar se ajudam.
- Distinguir seleção por filtros, por wrappers e por métodos embutidos, e aplicá-las com o scikit-learn.
- Descrever a ideia da síntese profunda de atributos (DFS) para dados em várias tabelas.
- Montar um gerador automático de atributos dentro de um pipeline e avaliar com honestidade se ele ajuda.
- Reconhecer os riscos da criação automática de atributos: explosão de candidatos, vazamento e sobreajuste da busca.

8.1 A matéria-prima do modelo

Um **atributo** (*feature*) é uma coluna que descreve cada exemplo. Mas as colunas que **chegam** nos dados nem sempre são as que **melhor ajudam** o modelo. Uma data de compra, sozinha, diz pouco; já o "dia da semana", o "é fim de mês?" ou o "valor dividido pela média do cliente" podem ser muito informativos.



"*Um algoritmo médio com bons atributos costuma vencer um algoritmo ótimo com atributos ruins.*"

Engenharia de atributos = transformar o que se mede em informação que o modelo consegue usar.

Analogia: o chef que corta, tempera e combina os ingredientes antes de levar ao fogo.

Figura 8.1 — A engenharia de atributos transforma o que se mede em informação que o modelo consegue usar.

A **engenharia de atributos** (*feature engineering*) é o trabalho de **criar, transformar e escolher** atributos. Muitos praticantes afirmam que ela pesa mais que a escolha do algoritmo. **Analogia:** o chef que corta, tempera e

combina os ingredientes antes de levá-los ao fogo. Mesmo o melhor forno não salva ingredientes mal preparados.

Esse trabalho tem duas faces:

- **Criar e transformar:** produzir atributos novos a partir dos existentes (Seções 8.2 e 8.4 a 8.6).
- **Selecionar:** descartar atributos inúteis, redundantes ou ruidosos (Seção 8.3).

Os sistemas de AutoML tratam as duas faces. O Auto-sklearn, por exemplo, inclui no seu espaço de busca diversos métodos de pré-processamento de atributos (Feurer *et al.*, 2015), e o TPOT combina seletores e transformadores de atributos nos pipelines evoluídos (Olson *et al.*, 2016). Neste capítulo, vamos entender o que eles fazem por dentro.

8.2 Transformações que ajudam

Algumas relações são **invisíveis** para certos modelos nos atributos originais, mas **evidentes** depois de uma transformação simples. Veja um exemplo clássico: duas classes organizadas em **círculos concêntricos**.

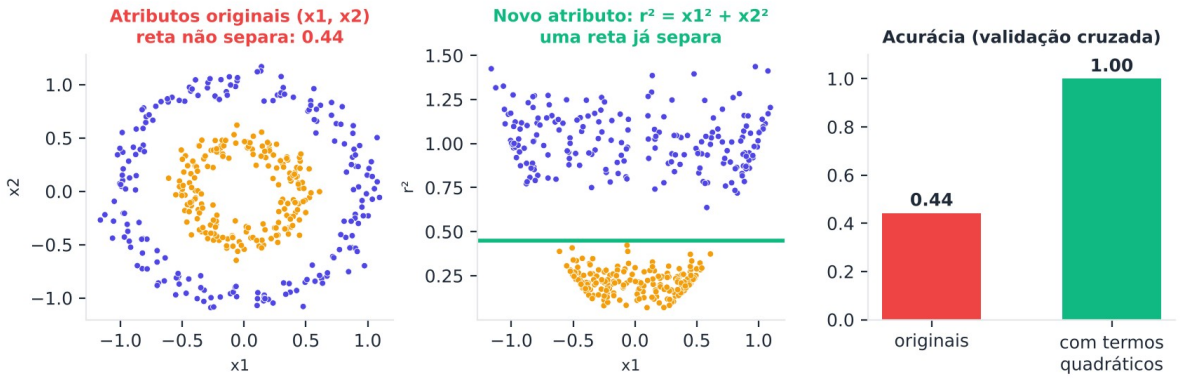


Figura 8.2 — Nos atributos originais (x1, x2), nenhuma reta separa os dois círculos. Com o novo atributo $r^2 = x1^2 + x2^2$, uma reta basta.

Um modelo linear só sabe traçar retas. Com os atributos originais, ele não consegue separar os círculos. Acrescentando os **termos quadráticos**, o problema vira trivial.

```python
import numpy as np
import warnings; warnings.filterwarnings("ignore")
from sklearn.datasets import make_circles
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import PolynomialFeatures
from sklearn.model_selection import cross_val_score

X, y = make_circles(400, noise=0.08, factor=0.45, random_state=0)
original = make_pipeline(LogisticRegression())
com_quadrados = make_pipeline(PolynomialFeatures(2, include_bias=False),
LogisticRegression(max_iter=2000))

print("Atributos originais        : acurácia =", round(cross_val_score(original, X, y, cv=5).mean(),
3))
print("Com termos quadráticos     : acurácia =", round(cross_val_score(com_quadrados, X, y,
cv=5).mean(), 3))
print("Atributos criados por PolynomialFeatures(2):", PolynomialFeatures(2,
include_bias=False).fit(X).get_feature_names_out(["x1", "x2"]))
```

```
SAÍDA
Atributos originais        : acurácia = 0.443
Com termos quadráticos     : acurácia = 1.0
Atributos criados por PolynomialFeatures(2): ['x1' 'x2' 'x1^2' 'x1 x2' 'x2^2']
```

As transformações mais usadas são:

Transformação	O que faz	Quando ajuda
Quadrado, cubo (`PolynomialFeatures`)	Cria x^2, x^3 e produtos entre atributos	Relações curvas; interações
Logaritmo, raiz	Comprime valores muito grandes	Variáveis muito assimétricas (renda, preços)
Razão	Divide um atributo por outro	"Gasto por visita", "preço por metro quadrado"
Faixas (*binning*)	Agrupa números em categorias	Efeitos que mudam em degraus (faixas de idade)
Data e hora	Extrai dia da semana, mês, "é feriado?"	Padrões sazonais
Agregações	Médias, máximos e contagens por grupo	Dados em várias tabelas (Seção 8.4)

Mais atributos nem sempre ajudam

Atenção: **transformar não garante melhora**. Em dados reais, acrescentar termos quadráticos pode piorar o modelo, pois aumenta o número de atributos e o risco de sobreajuste. Testemos no conjunto **Diabetes**, que descreve 442 pacientes com 10 atributos e uma medida de progressão da doença (um problema de regressão).

```python
from sklearn.datasets import load_diabetes
from sklearn.linear_model import Ridge
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import KFold

Xd, yd = load_diabetes(return_X_y=True)
kf = KFold(5, shuffle=True, random_state=0)
base = make_pipeline(StandardScaler(), Ridge(alpha=1.0))
poly = make_pipeline(PolynomialFeatures(2, include_bias=False), StandardScaler(), Ridge(alpha=10.0))

print("Atributos originais (10)        : R² =", round(cross_val_score(base, Xd, yd, cv=kf,
scoring="r2").mean(), 3))
print("Com termos quadráticos (65)     : R² =", round(cross_val_score(poly, Xd, yd, cv=kf,
scoring="r2").mean(), 3))
```

```
SAÍDA
Atributos originais (10)        : R² = 0.49
Com termos quadráticos (65)     : R² = 0.459
```

Neste caso, **mais atributos pioraram o R^2**. Lição: a engenharia de atributos precisa ser **avaliada**, e não presumida.

8.3 Seleção de atributos

Ter atributos demais também atrapalha: eles trazem **ruído**, deixam o treino mais lento e dificultam a interpretação. A **seleção de atributos** escolhe um subconjunto útil (Guyon; Elisseeff, 2003). Seus objetivos clássicos são: melhorar a previsão, tornar os modelos mais rápidos e baratos e facilitar a compreensão do problema.



Analogia: filtros = triagem por currículo; wrappers = entrevistar de verdade cada candidato; embutidos = avaliar durante o estágio.

Figura 8.3 — As três famílias de seleção de atributos: filtros, wrappers e métodos embutidos.

- **Filtros:** avaliam cada atributo **sozinho**, antes do modelo, com uma medida estatística (correlação, teste F, informação mútua). São rápidos e simples, mas **ignoram interações** entre atributos. (`SelectKBest`)
- **Wrappers:** testam subconjuntos **treinando o modelo de verdade**. O **RFE** (*recursive feature elimination*) treina o modelo, descarta os atributos menos importantes e repete. São mais precisos, porém mais caros.
- **Embutidos:** a seleção acontece **durante o treino**. Modelos com **penalidade L1** zeram os coeficientes de atributos pouco úteis; **árvores e florestas** informam a importância de cada atributo.

Vamos compará-las em um conjunto **difícil de propósito**: o **Wine** (13 medidas químicas reais, UCI) com **100 colunas de ruído** acrescentadas. Sabemos que só as 13 primeiras são úteis, e por isso podemos ver quantas cada método acerta. Toda seleção fica **dentro do pipeline**, para evitar vazamento.

```python
import pandas as pd
from sklearn.datasets import load_wine
from sklearn.feature_selection import SelectKBest, f_classif, mutual_info_classif, RFE, SelectFromModel
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import StratifiedKFold

Xw, yw = load_wine(return_X_y=True)
rng = np.random.default_rng(0)
Xr = np.hstack([Xw, rng.normal(size=(len(yw), 100))])          # 13 reais + 100 de ruído
cv = StratifiedKFold(5, shuffle=True, random_state=0)
logis = lambda: LogisticRegression(max_iter=3000)

metodos = {
    "sem seleção (113 atributos)":    make_pipeline(StandardScaler(), logis()),
    "filtro: teste F (k=10)":         make_pipeline(StandardScaler(), SelectKBest(f_classif, k=10),
logis()),
    "filtro: informação mútua (k=10)": make_pipeline(StandardScaler(), SelectKBest(lambda a, b:
mutual_info_classif(a, b, random_state=0), k=10), logis()),
    "wrapper: RFE (10)":              make_pipeline(StandardScaler(), RFE(logis(),
```

```
n_features_to_select=10, step=0.2), logis()),
    "embutido: penalidade L1":        make_pipeline(StandardScaler(),
SelectFromModel(LogisticRegression(penalty="l1", solver="saga", C=0.2, max_iter=5000)), logis()),
    "embutido: floresta (mediana)":   make_pipeline(SelectFromModel(RandomForestClassifier(100,
random_state=0), threshold="median"), StandardScaler(), logis()),
}

print(f"{'método':34s} | acurácia | atributos | reais entre eles")
for nome, pipe in metodos.items():
    acc = cross_val_score(pipe, Xr, yw, cv=cv).mean()
    pipe.fit(Xr, yw)
    sel = pipe[0].get_support() if nome.startswith("embutido: floresta") else (pipe[1].get_support() if
len(pipe) > 2 else np.ones(Xr.shape[1], bool))
    print(f"{nome:34s} | {acc:8.3f} | {sel.sum():9d} | {sel[:13].sum():2d} de 13")
```

SAÍDA

```
método                             | acurácia | atributos | reais entre eles
sem seleção (113 atributos)        |    0.949 |       113 | 13 de 13
filtro: teste F (k=10)             |    0.955 |        10 | 10 de 13
filtro: informação mútua (k=10)    |    0.960 |        10 | 10 de 13
wrapper: RFE (10)                  |    0.983 |        10 | 10 de 13
embutido: penalidade L1            |    0.983 |         9 |  9 de 13
embutido: floresta (mediana)       |    0.955 |        57 | 13 de 13
```

Observe o que aparece:

- Sem seleção, os 100 atributos de ruído **prejudicam** o modelo.
- Filtros baratos já ajudam, e **RFE** e **L1** ajudam mais, ao escolher quase só atributos reais.
- A floresta com limiar na mediana é "generosa": mantém metade dos atributos, o que inclui bastante ruído. O **limiar** de corte é um hiperparâmetro como outro qualquer.

> **Dica**
>
> O número de atributos a manter (*k*) também pode ser buscado: coloque `SelectKBest` no pipeline e inclua `selectkbest__k` na grade, como vimos no Capítulo 3.

8.4 Atributos a partir de várias tabelas: DFS

Muitos dados reais não vêm em uma única tabela. Um banco tem uma tabela de **clientes** e outra de **compras**, ligadas por um identificador. Cada cliente tem **muitas** compras. Como usar as compras para prever, por exemplo, quem vai cancelar o serviço?

A resposta humana é: **resumir** as compras de cada cliente com estatísticas. Quanto ele gasta em média? Qual o maior gasto? Quão variável é? Quantas compras fez? A **síntese profunda de atributos** (*Deep Feature Synthesis*, DFS), proposta por Kanter e Veeramachaneni (2015), **automatiza** esse raciocínio.

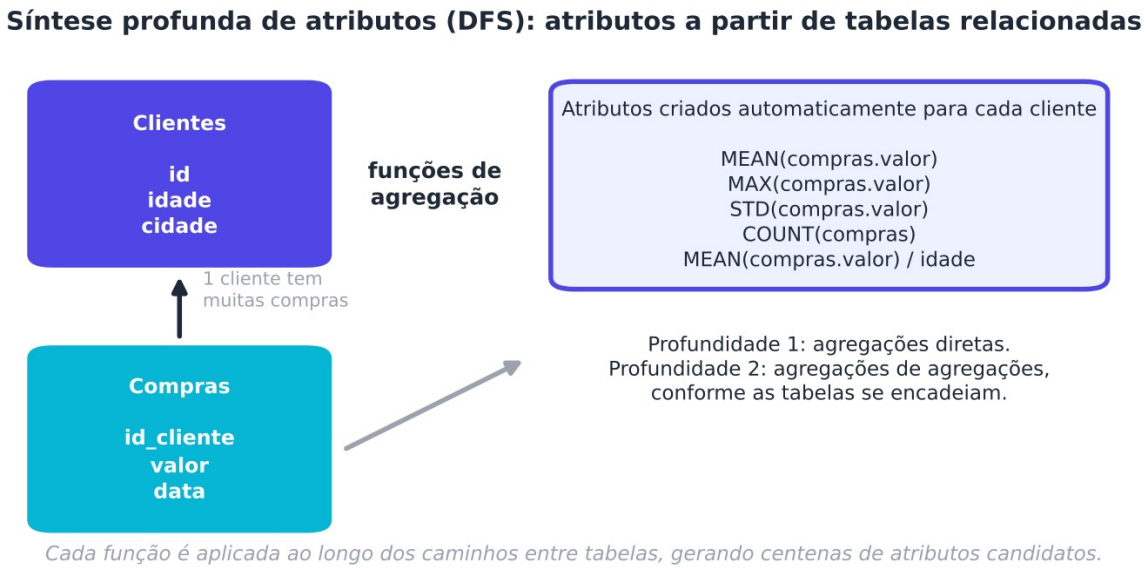


Figura 8.4 — A DFS aplica funções de agregação ao longo dos caminhos entre tabelas relacionadas, criando atributos candidatos para cada cliente.

A ideia é seguir os **relacionamentos** entre tabelas até um campo base e aplicar sucessivamente funções matemáticas (média, máximo, desvio padrão, contagem...) ao longo do caminho. Com **profundidade 1**, agregamos diretamente. Com **profundidade 2**, agregamos agregações. A biblioteca Featuretools implementa essa ideia.

Vamos reproduzir à mão a essência da DFS com `pandas`. Para podermos conferir se o método funciona, **simulamos** os dados: criamos clientes e compras e fazemos o cancelamento depender de um padrão que sabemos existir (a variabilidade relativa dos gastos e o número de compras). **Esses dados são artificiais**, e o resultado mostra o potencial da técnica quando existe informação escondida nas tabelas relacionadas, e não o desempenho em dados reais.

```python
import pandas as pd
from sklearn.ensemble import GradientBoostingClassifier

rng = np.random.default_rng(0)
n = 800
clientes = pd.DataFrame({"id": range(n), "idade": rng.integers(18, 75, n), "cidade": rng.choice(["A",
"B", "C"], n)})
n_compras = rng.poisson(8, n) + 1
media = rng.uniform(30, 120, n)
variab = rng.uniform(0.1, 1.2, n)
linhas = []
for i in range(n):
    valores = media[i] * np.exp(rng.normal(0, variab[i], n_compras[i]))
    linhas += [(i, v) for v in valores]
compras = pd.DataFrame(linhas, columns=["id_cliente", "valor"])

# "DFS" manual: agregações das compras de cada cliente
agg = compras.groupby("id_cliente")["valor"].agg(media_valor="mean", max_valor="max", std_valor="std",
n_compras="count").reset_index()
agg = agg.rename(columns={"id_cliente": "id"}).fillna(0)
dados = clientes.merge(agg, on="id")
dados["cv_valor"] = dados["std_valor"] / dados["media_valor"]          # atributo derivado (razão)
```

```
# O cancelamento depende da variabilidade relativa e do nº de compras (padrão que simulamos)
risco = 2.0 * dados["cv_valor"] - 0.02 * dados["n_compras"]
y = (rng.random(n) < 1 / (1 + np.exp(-3 * (risco - risco.median())))).astype(int)

so_clientes = pd.get_dummies(dados[["idade", "cidade"]]).astype(float)
com_agregados = pd.concat([so_clientes, dados[["media_valor", "max_valor", "std_valor", "n_compras",
"cv_valor"]]], axis=1)

cv = StratifiedKFold(5, shuffle=True, random_state=0)
for nome, Xa in [("apenas a tabela de clientes", so_clientes), ("com atributos agregados das compras",
com_agregados)]:
    auc = cross_val_score(GradientBoostingClassifier(random_state=0), Xa, y, cv=cv,
scoring="roc_auc").mean()
    print(f"{nome:38s} AUC = {auc:.3f}")
```

SAÍDA

```
apenas a tabela de clientes            AUC = 0.483
com atributos agregados das compras    AUC = 0.857
```

Sem usar as compras, o modelo fica no nível do acaso (AUC ≈ 0,5). Com os atributos agregados, o AUC sobe muito. A informação estava nas tabelas relacionadas, e **agregar** é o que a trouxe à tona.

> **Atenção**
>
> Em dados reais, atributos agregados também geram **vazamento temporal** com facilidade. Se a "média de compras" incluir compras **posteriores** à data da previsão, o modelo "viu o futuro". Ao agregar, sempre respeite a linha do tempo.

8.5 Aprender a criar atributos

A DFS gera candidatos por **regras fixas**. Uma linha de pesquisa complementar usa **aprendizado por reforço** para decidir **quais transformações aplicar**. Khurana, Samulowitz e Turaga (2018) propõem representar o espaço de transformações como um **grafo** e aprender, a partir de execuções anteriores, uma estratégia de busca nesse grafo. Os atributos resultantes continuam **interpretáveis**, pois são construídos com funções matemáticas padrão.

8.6 Criação automática "na marra" e seus perigos

E se, em vez de pensar, **gerarmos todas as transformações possíveis** e deixarmos a seleção escolher? É tentador. O problema é a **explosão de candidatos**.

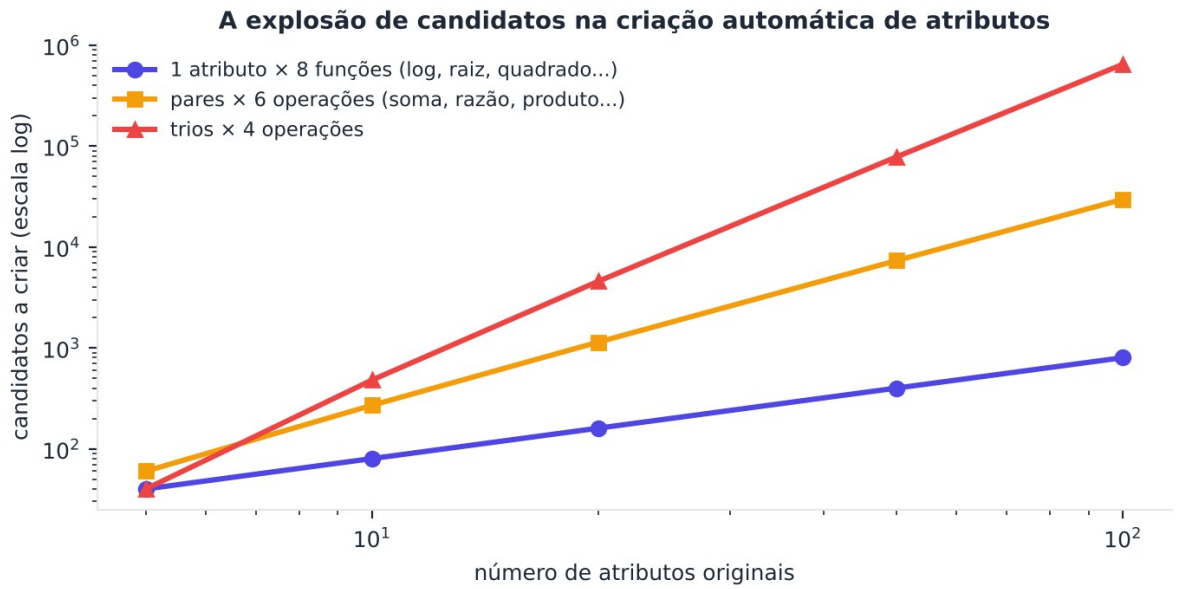


Figura 8.5 — O número de atributos candidatos cresce muito depressa com o número de atributos originais e com a ordem das combinações.

Com 100 atributos originais, só os pares com 6 operações já geram quase 30 mil candidatos. Com tantos candidatos, a seleção encontra, **por pura sorte**, atributos que parecem informativos nos dados de treino e não são: é o sobreajuste da busca de novo.

O ciclo típico é este:

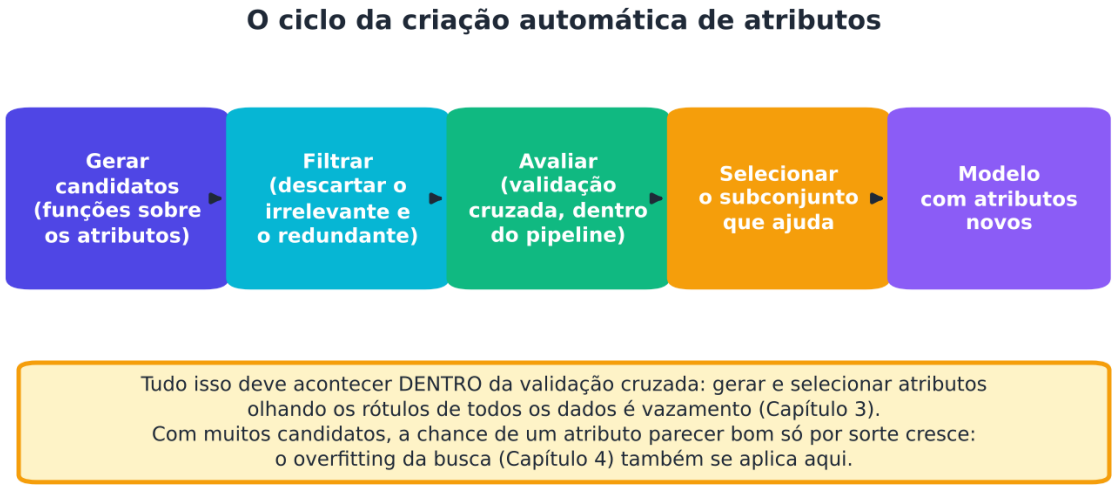


Figura 8.6 — O ciclo da criação automática de atributos. Gerar e selecionar devem ocorrer dentro da validação cruzada.

Vamos construir um **gerador automático** de atributos como uma etapa do pipeline. Ele recebe as primeiras *m* colunas e acrescenta: seus quadrados, seus logaritmos e, para cada par, o produto e a razão. Depois, o `SelectKBest` escolhe 15 atributos. Tudo vai dentro do pipeline (sem vazamento) e é avaliado por validação cruzada.

```python
from sklearn.base import BaseEstimator, TransformerMixin
from sklearn.datasets import load_breast_cancer

class GeradorDeAtributos(BaseEstimator, TransformerMixin):
    """Acrescenta quadrados, logaritmos, produtos e razões das primeiras `m` colunas."""
```

```
    def __init__(self, m=8):
        self.m = m
    def fit(self, X, y=None):
        self.colunas_ = np.arange(min(self.m, np.asarray(X).shape[1]))
        return self
    def transform(self, X):
        X = np.asarray(X); c = X[:, self.colunas_]
        novos = [X, c ** 2, np.log1p(np.abs(c))]
        for i in range(c.shape[1]):
            for j in range(i + 1, c.shape[1]):
                novos.append((c[:, i] * c[:, j])[:, None])
                novos.append((c[:, i] / (np.abs(c[:, j]) + 1e-3))[:, None])
        return np.hstack(novos)

print("Atributos gerados a partir de 8 colunas:", GeradorDeAtributos(8).fit(Xw).transform(Xw).shape[1]
- Xw.shape[1], "(mais as originais)")

cv = StratifiedKFold(5, shuffle=True, random_state=0)
print(f"\n{'conjunto':8s} | {'modelo':22s} | {'original':8s} | com atributos gerados")
for nome, carregar in [("Wine", load_wine), ("Cancer", load_breast_cancer)]:
    Xc, yc = carregar(return_X_y=True)
    pares = [("regressão logística", make_pipeline(StandardScaler(),
LogisticRegression(max_iter=3000)),
              make_pipeline(GeradorDeAtributos(8), StandardScaler(), SelectKBest(f_classif, k=15),
LogisticRegression(max_iter=3000))),
             ("gradient boosting", GradientBoostingClassifier(random_state=0),
              make_pipeline(GeradorDeAtributos(8), SelectKBest(f_classif, k=15),
GradientBoostingClassifier(random_state=0)))]
    for modelo, base, com_fe in pares:
        a, b = cross_val_score(base, Xc, yc, cv=cv).mean(), cross_val_score(com_fe, Xc, yc,
cv=cv).mean()
        print(f"{nome:8s} | {modelo:22s} | {a:8.3f} | {b:.3f}")
```

SAÍDA

```
Atributos gerados a partir de 8 colunas: 72 (mais as originais)

conjunto | modelo                 | original | com atributos gerados
Wine     | regressão logística    |    0.983 | 0.950
Wine     | gradient boosting      |    0.949 | 0.921
Cancer   | regressão logística    |    0.979 | 0.963
Cancer   | gradient boosting      |    0.965 | 0.946
```

O resultado é **desfavorável** à criação automática: em todos os casos, os atributos gerados **pioraram** a acurácia. Por quê?

- Os atributos originais do Wine e do Câncer **já são bons**: há pouco a ganhar com combinações.
- O gerador cria **dezenas de candidatos redundantes**. O `SelectKBest` univariado escolhe os 15 mais "associados ao alvo" **individualmente**, sem considerar que muitos são quase cópias uns dos outros, e **descarta atributos originais úteis** para acomodar os novos.
- Com poucos exemplos (178 e 569), a chance de um candidato parecer bom por sorte é grande.

Isso não quer dizer que gerar atributos seja inútil: na Seção 8.4 ele foi decisivo. A lição é que **criar atributos precisa de um motivo** (conhecimento do problema, estrutura nos dados) e de **avaliação honesta**. A "chuva de candidatos" sem critério raramente compensa.

8.7 Riscos e cuidados

1. **Vazamento.** Gerar ou selecionar atributos olhando todos os rótulos antes da validação contamina a avaliação (Capítulo 3). Mantenha tudo dentro do pipeline.
2. **Vazamento temporal.** Atributos agregados não podem usar informação posterior ao momento da previsão.
3. **Sobreajuste da busca.** Quanto mais candidatos, maior a chance de acharmos "sinais" falsos.
4. **Redundância.** Atributos muito correlacionados entre si desperdiçam capacidade e confundem a interpretação.
5. **Interpretabilidade.** Atributos como "razão entre o logaritmo da coluna 3 e o quadrado da coluna 7" são difíceis de explicar. Prefira transformações com sentido no problema.
6. **Custo.** Mais colunas significam mais tempo e memória, no treino e na produção. Todo atributo criado precisa ser **calculado também em produção** (Capítulo 14).
7. **Estabilidade.** Um atributo que funciona hoje pode deixar de funcionar se os dados mudarem.

8.8 Boas práticas

1. **Comece pelo conhecimento do problema.** Converse com quem entende do assunto: que informações ajudariam?
2. **Meça o ganho** de cada família de atributos novos contra uma linha de base, com validação cruzada e teste final.
3. **Coloque a engenharia dentro do pipeline** (`FunctionTransformer`, transformadores próprios).
4. **Selecione com critério:** teste filtros, RFE e L1; trate o número de atributos como hiperparâmetro.
5. **Limite a explosão:** gere transformações a partir de poucos atributos promissores, e não de todos.
6. **Documente** cada atributo criado: sua fórmula, o motivo e a data de referência.
7. **Cheque o vazamento temporal** em atributos agregados.

Resumo do capítulo

MEMORIZE **Cartões de memorização**

- **Engenharia de atributos:** criar, transformar e selecionar colunas; muitas vezes pesa mais que o algoritmo.
- **Transformações:** quadrados, logaritmos, razões, faixas, data e hora, agregações. **Avalie** sempre: mais atributos nem sempre ajudam.
- **Seleção:** filtros (rápidos, univariados), wrappers (RFE; precisos e caros) e embutidos (L1, árvores).
- **DFS:** agrega informações de tabelas relacionadas (média, máximo, contagem...); respeite a linha do tempo.
- **Criação em massa:** explosão de candidatos, redundância e sorte; em Wine e Câncer, **piorou** o resultado.
- **Tudo dentro do pipeline** para evitar vazamento.

Exercícios de múltipla escolha

Escolha **uma** alternativa em cada questão. O gabarito está no Apêndice A.

Questão 1. O que é engenharia de atributos?

a) O trabalho de criar, transformar e selecionar atributos para ajudar o modelo.
b) A escolha do algoritmo de aprendizado.
c) A divisão dos dados em treino e teste.
d) O ajuste dos hiperparâmetros.

Questão 2. Na analogia do chef, a engenharia de atributos corresponde a:

a) cortar, temperar e combinar os ingredientes antes de ir ao fogo.
b) provar o prato pronto.
c) escolher o forno.
d) lavar a louça.

Questão 3. Por que o modelo linear não separa os dois círculos concêntricos usando apenas x1 e x2?

a) Porque a regressão logística não aceita números reais.
b) Porque há classes demais.
c) Porque os dados têm valores ausentes.
d) Porque ele só traça retas, e nenhuma reta separa as classes nesses atributos.

Questão 4. No exemplo dos círculos, o que torna o problema fácil para um modelo linear?

a) Acrescentar o atributo $r^2 = x1^2 + x2^2$ (termos quadráticos).
b) Remover x2.
c) Usar mais exemplos.
d) Padronizar x1 e x2.

Questão 5. Qual transformação é mais útil para variáveis muito assimétricas, como renda?

a) Codificação one-hot.
b) Cubo.
c) Remover a variável.
d) Logaritmo.

Questão 6. Qual transformação cria atributos como "gasto por visita"?

a) Faixas (binning).
b) Razão entre dois atributos.
c) Quadrado.
d) Padronização.

Questão 7. No experimento com o conjunto Diabetes, acrescentar termos quadráticos:

a) impossibilitou o treino.
b) piorou o R^2 em relação aos atributos originais.
c) melhorou muito o R^2.
d) não mudou nada.

Questão 8. Qual a lição do experimento com o conjunto Diabetes?

a) O conjunto Diabetes não pode ser modelado.
b) Termos quadráticos sempre ajudam.
c) A engenharia de atributos precisa ser avaliada, e não presumida.
d) A validação cruzada é inútil.

Questão 9. Qual não é um objetivo clássico da seleção de atributos?

a) Melhorar a previsão.
b) Aumentar o número de colunas.
c) Facilitar a compreensão do problema.
d) Tornar os modelos mais rápidos e baratos.

Questão 10. Os métodos de filtro avaliam:

a) atributos durante o treino do modelo.
b) cada atributo isoladamente, antes do modelo, por medidas estatísticas.
c) apenas atributos categóricos.
d) subconjuntos de atributos treinando o modelo de verdade.

Questão 11. Qual é uma limitação típica dos filtros univariados?

a) Ignoram as interações entre atributos.
b) Não funcionam com números.
c) São lentos demais.
d) Só funcionam em regressão.

Questão 12. O RFE (recursive feature elimination) é um exemplo de:

a) transformação de cauda.
b) método embutido.
c) filtro.
d) wrapper.

Questão 13. Qual característica descreve melhor os wrappers?

a) Precisos, porém mais caros computacionalmente.
b) Não usam validação.
c) Independentes do modelo.
d) Muito rápidos e sempre superficiais.

Questão 14. Um método embutido de seleção é:

a) a penalidade L1, que zera coeficientes de atributos pouco úteis.
b) o teste F.
c) o train_test_split.
d) o StandardScaler.

Questão 15. No experimento do Wine com 100 colunas de ruído, o que ocorreu sem seleção?

a) Todos os métodos empataram.
b) O modelo ficou prejudicado pelo ruído.
c) O modelo atingiu 100% de acurácia.
d) O modelo não pôde ser treinado.

Questão 16. Qual conjunto de métodos obteve as maiores acurácias no experimento de seleção com ruído?

a) Apenas a floresta com limiar na mediana.
b) RFE e penalidade L1.
c) Nenhum, todos pioraram.
d) Apenas o teste F.

Questão 17. Por que a floresta com limiar na mediana manteve muito ruído?

a) Porque florestas não calculam importâncias.
b) Porque o pipeline estava fora da validação.
c) Porque o ruído era informativo.
d) Porque o limiar na mediana mantém metade dos atributos, incluindo ruído.

Questão 18. Por que colocar a seleção de atributos dentro do pipeline?

a) Para dispensar a validação cruzada.
b) Para deixar o código mais curto apenas.
c) Para evitar vazamento: ela é reaprendida só com o treino de cada divisão.
d) Para aumentar o número de atributos.

Questão 19. A síntese profunda de atributos (DFS) serve principalmente para:

a) dividir os dados.
b) padronizar atributos numéricos.
c) escolher hiperparâmetros.
d) criar atributos agregando informações de tabelas relacionadas.

Questão 20. Na DFS, o que significa "profundidade 2"?

a) Agregações de agregações, conforme as tabelas se encadeiam.
b) Duas validações cruzadas.
c) Duas colunas de saída.
d) O dobro de exemplos.

Questão 21. No exemplo com clientes e compras, qual é um atributo criado por DFS?

a) O nome do arquivo.
b) A semente aleatória.
c) O número de linhas da tabela de clientes.
d) A média dos valores das compras de cada cliente.

Questão 22. No experimento simulado da DFS, qual foi o AUC usando apenas a tabela de clientes?

a) Cerca de 0,5, no nível do acaso.
b) Cerca de 0,75.
c) Cerca de 1,0.
d) Cerca de 0,95.

Questão 23. Por que o capítulo diz que o resultado da DFS simulada "mostra o potencial, e não o desempenho em dados reais"?

a) Porque não houve validação cruzada.
b) Porque a DFS só funciona em simulações.
c) Porque os dados foram simulados com um padrão que sabíamos existir.
d) Porque o AUC não é uma métrica válida.

Questão 24. Qual é o risco de vazamento temporal ao agregar atributos?

a) Usar variáveis categóricas.
b) Usar a validação cruzada.
c) Usar um número ímpar de linhas.
d) Usar compras posteriores à data da previsão, "vendo o futuro".

Questão 25. Qual a abordagem de Khurana, Samulowitz e Turaga (2018) para criar atributos?

a) Padronização dos dados.
b) Aprendizado por reforço sobre um grafo de transformações.
c) Busca em grade de hiperparâmetros.
d) Seleção por torneio em um algoritmo genético.

Questão 26. Com 100 atributos originais, quantos candidatos de pares × 6 operações são possíveis, aproximadamente?

a) Cerca de 30 mil.
b) 100.
c) 6.
d) 600.

Questão 27. Por que gerar todas as transformações possíveis e selecionar depois pode falhar?

a) Porque a seleção não funciona em nenhum caso.
b) Porque isso elimina o vazamento.
c) Porque os modelos não aceitam muitos atributos.
d) Há explosão de candidatos, redundância e chance de sinais falsos por sorte.

Questão 28. No experimento do gerador automático em Wine e Câncer, o que ocorreu?

a) Os atributos gerados melhoraram sempre muito a acurácia.
b) Todos os modelos atingiram 100%.
c) O gerador não pôde ser treinado.
d) Os atributos gerados pioraram a acurácia nos casos testados.

Questão 29. Qual das opções é uma boa prática ao criar atributos?

a) Começar pelo conhecimento do problema e medir o ganho contra uma linha de base.
b) Ignorar o vazamento temporal.
c) Gerar o máximo possível sem avaliar.
d) Usar o teste final para escolher atributos.

Questão 30. Qual método da biblioteca scikit-learn gera automaticamente termos quadráticos e produtos entre atributos?

a) StandardScaler.
b) OneHotEncoder.
c) SimpleImputer.
d) PolynomialFeatures.

Exercícios discursivos

Responda com suas palavras, em poucas linhas. Compare depois com a resposta-modelo do Apêndice A.

1. Explique, com uma analogia, por que a engenharia de atributos costuma pesar mais que a escolha do algoritmo.
2. Explique o exemplo dos círculos concêntricos e o que ele ensina sobre transformações.
3. Cite quatro transformações de atributos e uma situação em que cada uma ajuda.
4. Por que a criação de termos quadráticos piorou o resultado no conjunto Diabetes e o que isso ensina?
5. Compare filtros, wrappers e métodos embutidos de seleção de atributos.
6. Explique como o RFE funciona.
7. Por que a seleção de atributos deve ficar dentro do pipeline? Relacione com o experimento do Capítulo 3.
8. Interprete os resultados do experimento de seleção no Wine com 100 colunas de ruído.
9. Explique a ideia da DFS e dê um exemplo de atributos que ela cria para uma tabela de clientes.
10. O que mostrou o experimento simulado da DFS e que cuidado se deve ter ao interpretá-lo?
11. Explique o vazamento temporal em atributos agregados e como evitá-lo.
12. Descreva a explosão de candidatos na criação automática de atributos e uma consequência estatística dela.
13. Interprete o resultado do gerador automático em Wine e Câncer e explique três razões para a piora.
14. Em que situação criar atributos automaticamente é promissor e em qual é questionável?
15. Cite quatro riscos da criação automática de atributos e uma medida contra um deles.
16. Por que atributos criados precisam também ser calculados em produção e que implicação isso traz?
17. Como o número de atributos a manter na seleção pode ser tratado como hiperparâmetro?
18. Descreva a abordagem de aprendizado por reforço para criar atributos (Khurana e colegas, 2018).
19. Resuma quatro boas práticas de engenharia de atributos apresentadas no capítulo.
20. Por que o número de atributos a manter e o limiar de corte de um seletor devem ser tratados como hiperparâmetros?

Exercícios práticos em Python (scikit-learn)

Resolva os exercícios no seu computador (ou no Google Colab). Eles usam conjuntos de dados da UCI que já vêm no scikit-learn ou dados simulados. Tente resolver **antes** de consultar a solução, no Apêndice D. As saídas apresentadas lá foram obtidas com as versões de bibliotecas listadas na Apresentação.

Exercício 1 — Quando o produto de dois atributos é a resposta

NÍVEL: BÁSICO

Simule 1.000 exemplos com dois atributos `x1` e `x2` (normais) e o alvo `y = x1 · x2 + ruído` (ruído normal com desvio 0,3). Compare, por validação cruzada de 5 partes (métrica R^2), uma `LinearRegression` com (a) apenas `x1` e `x2` e (b) `x1`, `x2` e o atributo criado `x1·x2`.

Pergunta: por que o modelo linear falha em (a) e acerta em (b)? Esse resultado "prova" que a engenharia de atributos sempre ajuda?

Exercício 2 — Quantos atributos manter?

NÍVEL: INTERMEDIÁRIO

Com o conjunto **Breast Cancer Wisconsin** (UCI), acrescente 200 colunas de ruído normal. Em um pipeline com `StandardScaler`, `SelectKBest(f_classif)` e `LogisticRegression`, varie `k` em [2, 5, 10, 20, 30, 50, 100, 230] e calcule a acurácia de validação cruzada de 5 partes. Mostre também o resultado sem seleção.

Pergunta: qual faixa de `k` funciona melhor? O que acontece com `k` muito pequeno e com `k` muito grande?

Exercício 3 — RFECV: deixar a validação cruzada escolher

NÍVEL: INTERMEDIÁRIO

No conjunto **Wine** (UCI) com 100 colunas de ruído acrescentadas, use `RFECV` (eliminação recursiva com validação cruzada) com uma `LogisticRegression` e `StandardScaler`, `step=5`, `cv=5`. Mostre quantos atributos o `RFECV` escolheu, quantos desses são dos 13 reais e a acurácia de validação cruzada **externa** do pipeline completo (`RFECV` + modelo), para evitar vazamento.

Pergunta: o número escolhido faz sentido? Por que é importante avaliar o pipeline completo com uma validação externa?

Exercício 4 — Mais agregados, mais ganho?

NÍVEL: INTERMEDIÁRIO

Reconstrua os dados simulados de clientes e compras da Seção 8.4 (800 clientes, mesma semente). Compare o AUC (gradient boosting, validação cruzada estratificada de 5 partes) usando: (a) os agregados do capítulo (média, máximo, desvio, contagem e razão desvio/média) e (b) esses agregados mais três novos: **mínimo**, **mediana** e **número de compras acima de 100**.

Pergunta: os atributos extras ajudaram? Por quê?

> SAIBA MAIS **Para ir além: bases públicas da UCI e do Kaggle**
>
> - **UCI:** Wine Quality (regressão), Concrete Compressive Strength (relações não lineares entre ingredientes) e Online Retail (transações para praticar agregações).
> - **Kaggle:** House Prices (muitos atributos para criar e selecionar) e Instacart Market Basket Analysis (várias tabelas relacionadas, ideal para DFS).
>
> Estes conjuntos não foram usados nem testados nos códigos do livro. Confira, nos sites, a descrição, a licença e as colunas antes de usá-los.

Projeto integrador do capítulo

Engenharia de atributos honesta em Diabetes

Fonte dos dados. *Diabetes*, incluído no scikit-learn (`load_diabetes`): 442 pacientes, 10 atributos e uma medida de progressão da doença após um ano (regressão). A base é de uso didático.

Contexto. Uma equipe quer saber se vale a pena sofisticar os atributos para prever a progressão da doença: termos quadráticos, seleção de atributos ou um modelo mais flexível.

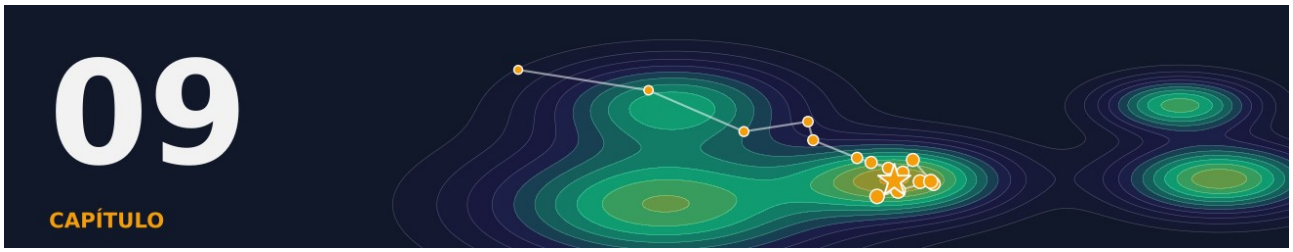
Atenção ética. Os dados são de uso didático. O modelo **não é ferramenta clínica**.

Tarefas.

1. Separe 25% dos dados como **teste final** (semente fixa).
2. Monte cinco pipelines (todos com a engenharia **dentro** do pipeline): (A) `StandardScaler` + `Ridge`; (B) termos quadráticos + `StandardScaler` + `Ridge`; (C) termos quadráticos + `SelectKBest(f_regression, k=15)` + `Ridge`; (D) termos quadráticos + `StandardScaler` + `Lasso` (seleção embutida); (E) `GradientBoostingRegressor`.
3. Avalie cada um por **validação cruzada repetida** no treino (`RepeatedKFold`, 5 partes × 3 repetições, métrica R^2) e registre média e desvio padrão.
4. Escolha o melhor pelo critério da validação. Avalie **apenas esse** (e, para comparação, o pipeline A) no teste final, com R^2 e RMSE.
5. Escreva um parágrafo (6 a 8 linhas): a engenharia de atributos ajudou? A diferença entre os pipelines é maior que a variação da validação cruzada? O que você recomendaria à equipe?

Entregáveis. Um notebook (ou script) comentado, a tabela de resultados e o parágrafo de conclusões.

A solução de referência está no Apêndice E.



Capítulo 9 — Meta-aprendizado, warm start e ensembles

OBJETIVOS DE APRENDIZAGEM

Ao final deste capítulo você será capaz de:

- Explicar o que é meta-aprendizado e como ele usa a experiência de problemas anteriores.
- Calcular meta-atributos de um conjunto de dados e construir um meta-aprendizado simples.
- Aplicar o warm start (partida a quente) para acelerar a busca de hiperparâmetros.
- Explicar por que combinar modelos (ensembles) costuma ser melhor do que escolher só um.
- Usar votação e stacking no scikit-learn e implementar a seleção de ensembles de Caruana.
- Descrever a arquitetura do Auto-sklearn e reconhecer os limites de cada técnica.

9.1 Aprender com a experiência

Imagine dois médicos diante de um paciente com um quadro novo. O **médico iniciante** pede todos os exames possíveis. O **médico experiente** reconhece o caso: "parece com outros que já vi" e vai direto aos exames mais prováveis de ajudar.

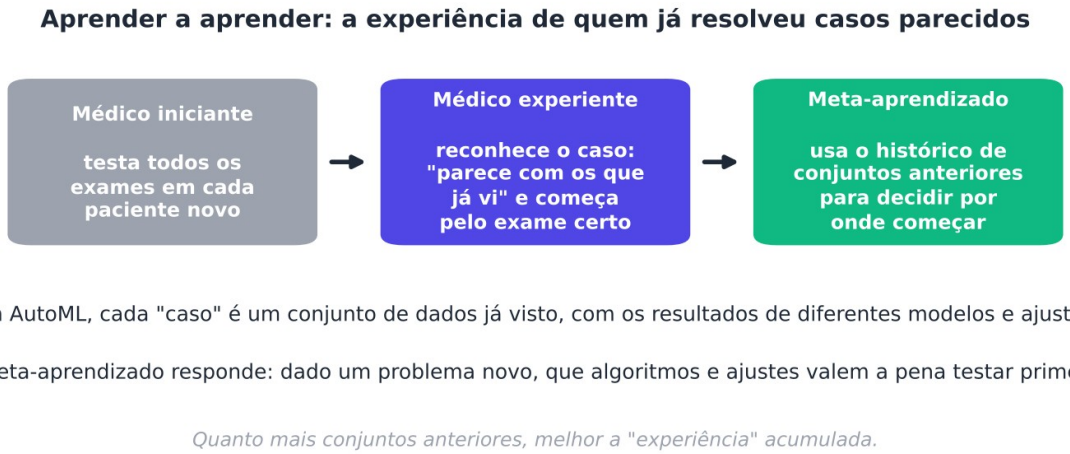


Figura 9.1 — O meta-aprendizado dá ao AutoML a experiência de problemas anteriores.

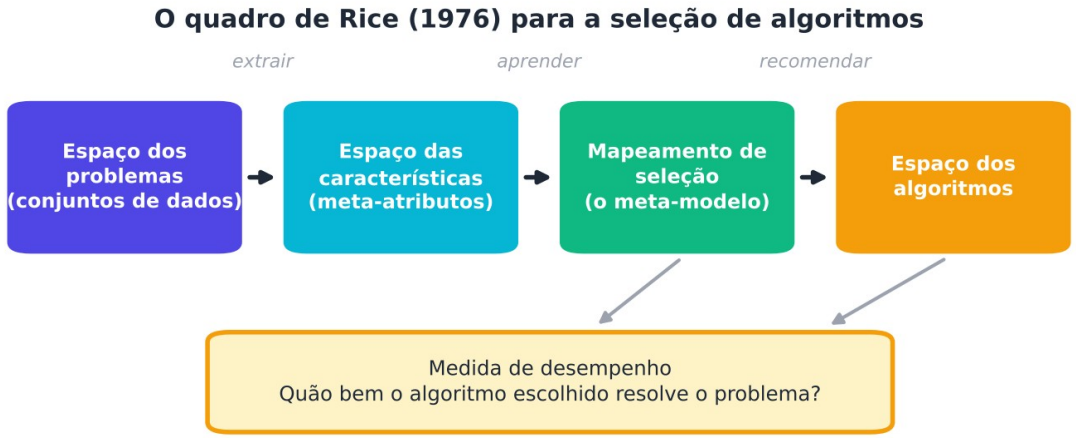
Nos capítulos anteriores, cada busca de hiperparâmetros **começava do zero**, como o médico iniciante: nada se sabia sobre o novo conjunto de dados. Mas já resolvemos muitos problemas parecidos! Será que essa experiência pode ajudar?

É a ideia do **meta-aprendizado** (*meta-learning*, ou "aprender a aprender"): usar os resultados de **conjuntos de dados anteriores** para decidir mais depressa e melhor o que fazer em um **novo**. Esse é um dos temas centrais do AutoML moderno, com livros dedicados (Brazdil *et al.*, 2022) e capítulos no livro de referência da área (Hutter; Kotthoff; Vanschoren, 2019).

Neste capítulo, vamos ver duas formas de aproveitar essa experiência e uma terceira ideia, independente, que tornou o AutoML muito forte: **combinar modelos**.

9.2 O quadro de Rice: escolher o algoritmo por características do problema

Já mencionamos o **problema da seleção de algoritmos**, formulado por Rice (1976): dado um problema, qual algoritmo resolve melhor? Rice o descreveu com **quatro espaços**.



Rice formulou o problema em 1976; o meta-aprendizado moderno é a sua versão guiada por dados (Brazdil et al., 2022).

Figura 9.2 — O quadro de Rice: extrair características do problema, aprender uma regra de seleção e recomendar um algoritmo, avaliando o desempenho.

1. **Espaço dos problemas:** os conjuntos de dados.
2. **Espaço das características:** descrições resumidas de cada problema, os **meta-atributos**.
3. **Mapeamento de seleção:** a regra (um modelo, o **meta-modelo**) que, dadas as características, escolhe o algoritmo.
4. **Medida de desempenho:** o quanto o algoritmo escolhido acerta naquele problema.

Em outras palavras: se pudermos **descrever** cada conjunto de dados por alguns números, podemos **treinar um modelo** que aprenda "que tipo de problema se dá bem com que tipo de algoritmo".

9.3 Meta-atributos

Os **meta-atributos** (*meta-features*) são descrições de um conjunto de dados inteiro. Eles são calculados rápido, sem treinar modelos caros.



Figura 9.3 — Quatro famílias de meta-atributos.

- **Simples e estatísticos:** número de exemplos, de atributos e de classes; assimetria média dos atributos.
- **Teoria da informação:** entropia das classes (quão balanceadas são).
- **Complexidade do problema:** medidas de sobreposição entre classes e de suavidade da fronteira (Lorena *et al.*, 2019).
- **Landmarkers:** a nota de **modelos simples e rápidos** (uma árvore rasa, por exemplo) no problema. É uma "sonda" do tipo de estrutura que os dados têm.

A literatura mostra que a descrição, a organização e o cálculo dos meta-atributos variam entre trabalhos, o que dificulta a reprodução (Rivolli *et al.*, 2022). Vamos calcular oito meta-atributos de quatro conjuntos de dados reais.

```python
import numpy as np
import warnings; warnings.filterwarnings("ignore")
from scipy.stats import skew, entropy
from sklearn.datasets import make_classification, load_iris, load_wine, load_breast_cancer, load_digits
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier, HistGradientBoostingClassifier
from sklearn.model_selection import cross_val_score, StratifiedKFold, train_test_split
from sklearn.metrics import accuracy_score, log_loss

NOMES = ["log(n)", "log(d)", "n/d", "classes", "entropia", "assimetria", "correlação", "landmarker"]

def meta_atributos(X, y):
    n, d = X.shape; c = len(np.unique(y)); p = np.bincount(y) / n
    assim = np.abs(skew(X, axis=0))                                  # assimetria de cada atributo
    cor = np.abs(np.corrcoef(X.T)); np.fill_diagonal(cor, 0)         # correlação entre atributos
    landmarker = cross_val_score(DecisionTreeClassifier(max_depth=1, random_state=0), X, y,
cv=3).mean()
    return [np.log(n), np.log(d), n / d, c, entropy(p) / np.log(c), np.nanmean(assim), np.nanmean(cor),
landmarker]
```

```
print(f"{'conjunto':8s} | " + " | ".join(f"{n:>11s}" for n in NOMES))
for nome, carregar in [("Iris", load_iris), ("Wine", load_wine), ("Cancer", load_breast_cancer),
("Digits", load_digits)]:
    X, y = carregar(return_X_y=True)
    print(f"{nome:8s} | " + " | ".join(f"{v:11.2f}" for v in meta_atributos(X, y)))
```

SAÍDA

```
conjunto |      log(n) |      log(d) |         n/d |     classes |    entropia |  assimetria |
correla…
Iris     |        5.01 |        1.39 |       37.50 |        3.00 |        1.00 |        0.25 |
0…
Wine     |        5.18 |        2.56 |       13.69 |        3.00 |        0.99 |        0.43 |
0…
Cancer   |        6.34 |        3.40 |       18.97 |        2.00 |        0.95 |        1.74 |
0…
Digits   |        7.49 |        4.16 |       28.08 |       10.00 |        1.00 |        4.89 |
0…
```

Repare como os conjuntos têm "retratos" diferentes: o Digits tem muitos exemplos e atributos e 10 classes; o Iris é minúsculo, com 3 classes balanceadas. Um meta-modelo pode aprender com essas diferenças.

9.4 Construindo um meta-aprendizado do zero

Vamos montar um **meta-conjunto**: uma coleção de problemas, cada um descrito por seus meta-atributos e rotulado com o **melhor algoritmo** entre quatro candidatos (k-vizinhos, SVM, regressão logística e floresta). Depois, treinamos um **meta-modelo** que prevê o melhor algoritmo a partir dos meta-atributos.

Como não temos centenas de conjuntos reais à mão, vamos **simular 150 conjuntos de dados** com `make_classification`, variando tamanho, número de atributos, de classes, separabilidade e ruído. Mais adiante, testaremos o meta-modelo em **conjuntos reais** que ele nunca viu.

Para avaliar com honestidade, usamos a validação **"deixe um conjunto de fora"**: para cada conjunto, o meta-modelo é treinado com os outros 149 e testado no conjunto excluído. Como linha de base, comparamos com a regra boba "recomendar sempre o algoritmo que mais vence".

PYTHON

```
def gerar_conjunto(rng, i):
    n = int(rng.integers(150, 500)); d = int(rng.integers(8, 40)); inf = int(rng.integers(4, min(d - 2,
12) + 1)); c = int(rng.integers(2, 5))
    return make_classification(n_samples=n, n_features=d, n_informative=inf,
n_redundant=int(rng.integers(0, (d - inf) // 2 + 1)),
                               n_classes=c, n_clusters_per_class=int(rng.integers(1, 3)),
class_sep=float(rng.uniform(0.4, 2.0)),
                               flip_y=float(rng.uniform(0, 0.1)), random_state=i)

ALGORITMOS = {"knn": lambda: make_pipeline(StandardScaler(), KNeighborsClassifier()),
              "svm": lambda: make_pipeline(StandardScaler(), SVC()),
              "logistica": lambda: make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000)),
              "floresta": lambda: RandomForestClassifier(60, random_state=0, n_jobs=-1)}

def avaliar_todos(X, y):
    cv = StratifiedKFold(3, shuffle=True, random_state=0)
    return {nome: cross_val_score(f(), X, y, cv=cv).mean() for nome, f in ALGORITMOS.items()}
```

PYTHON

```python
import collections

rng = np.random.default_rng(0)
M, R = [], []
for i in range(150):
    X, y = gerar_conjunto(rng, i)
    M.append(meta_atributos(X, y)); R.append(avaliar_todos(X, y))
M = np.array(M); melhor = np.array([max(r, key=r.get) for r in R])
print("Quantas vezes cada algoritmo foi o melhor nos 150 conjuntos:",
dict(collections.Counter(melhor)))

acertos, perdas = [], []
for i in range(len(M)):
    treino = np.arange(len(M)) != i
    previsto = RandomForestClassifier(200, random_state=0).fit(M[treino],
melhor[treino]).predict(M[i:i+1])[0]
    acertos.append(previsto == melhor[i]); perdas.append(max(R[i].values()) - R[i][previsto])

frequente = collections.Counter(melhor).most_common(1)[0][0]
perdas_freq = [max(r.values()) - r[frequente] for r in R]
print(f"\nAcerto do meta-modelo ao indicar o melhor algoritmo : {np.mean(acertos):.3f}")
print(f"Acerto de 'sempre {frequente}' (linha de base)           : {collections.Counter(melhor)
[frequente] / len(M):.3f}")
print(f"Perda média de acurácia frente ao melhor: meta-modelo = {np.mean(perdas):.4f} | sempre
{frequente} = {np.mean(perdas_freq):.4f}")

meta_modelo = RandomForestClassifier(300, random_state=0).fit(M, melhor)
print("\nTeste em conjuntos REAIS (nunca vistos pelo meta-modelo):")
for nome, carregar in [("Iris", load_iris), ("Wine", load_wine), ("Cancer", load_breast_cancer),
("Digits", load_digits)]:
    X, y = carregar(return_X_y=True); notas = avaliar_todos(X, y)
    rec = meta_modelo.predict(np.array(meta_atributos(X, y))[None])[0]
    real = max(notas, key=notas.get)
    print(f"  {nome:7s} recomendado = {rec:9s} ({notas[rec]:.3f}) | melhor de verdade = {real:9s}
({notas[real]:.3f})")
```

SAÍDA

```
Quantas vezes cada algoritmo foi o melhor nos 150 conjuntos: {np.str_('svm'): 89, np.str_('knn'): 9,
np…

Acerto do meta-modelo ao indicar o melhor algoritmo : 0.593
Acerto de 'sempre svm' (linha de base)           : 0.593
Perda média de acurácia frente ao melhor: meta-modelo = 0.0100 | sempre svm = 0.0110

Teste em conjuntos REAIS (nunca vistos pelo meta-modelo):
  Iris    recomendado = svm       (0.947) | melhor de verdade = knn       (0.960)
  Wine    recomendado = svm       (0.983) | melhor de verdade = svm       (0.983)
  Cancer  recomendado = svm       (0.972) | melhor de verdade = logistica (0.977)
  Digits  recomendado = svm       (0.982) | melhor de verdade = svm       (0.982)
```

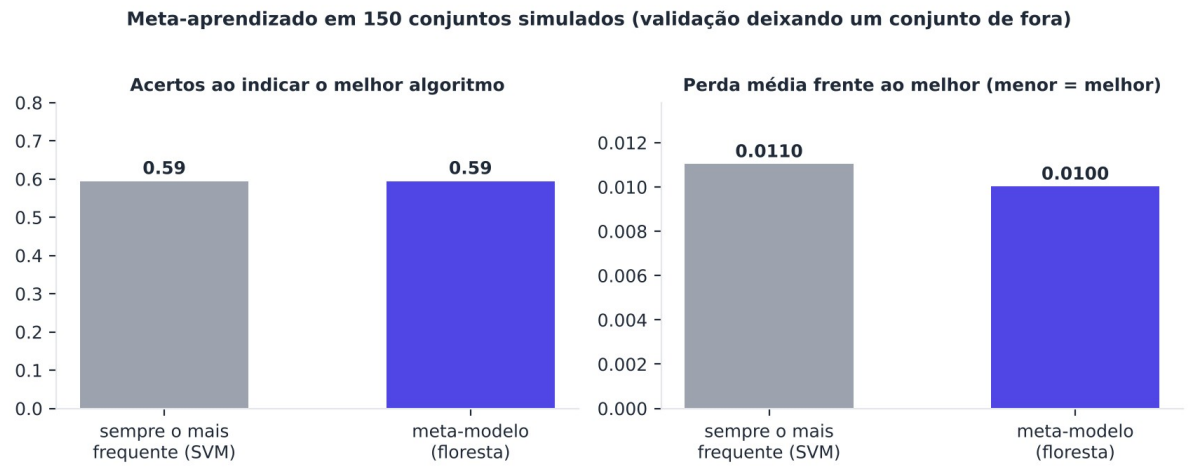


Figura 9.4 — O meta-modelo contra a regra "sempre o mais frequente" em 150 conjuntos simulados.

O resultado é **humilde**. O meta-modelo acerta o melhor algoritmo na mesma proporção que a regra boba (cerca de 59%), e a perda média de acurácia é só ligeiramente menor. Nos conjuntos reais, ele recomenda o SVM, que vai bem em todos, ainda que não seja o primeiro colocado em Iris e Cancer, com perdas pequenas.

Por que o ganho foi pequeno? Porque (1) nossos 150 conjuntos são **poucos e simulados**; (2) usamos apenas **oito meta-atributos** simples; (3) com só quatro algoritmos "padrão", as diferenças entre eles costumam ser pequenas, e o SVM é muito competitivo na maioria dos casos. Sistemas reais usam **centenas de conjuntos** (por exemplo, do repositório OpenML; Vanschoren *et al.*, 2014) e meta-atributos mais ricos.

> **Memorize**
>
> Recomendar o **algoritmo** é a forma mais difícil de meta-aprendizado. Há uma forma mais robusta e mais usada na prática: recomendar **por onde começar a busca** (configurações iniciais). É o *warm start*.

9.5 Warm start: partida a quente

Imagine que você precisa ajustar o SVM em um conjunto novo. A busca aleatória ou bayesiana começa com valores sorteados, muitos dos quais são ruins. Mas a experiência anterior diz que certas combinações de *C* e *gamma* costumam funcionar bem em **muitos** problemas. Por que não **começar por elas**?

Essa é a ideia do ***warm start*** (partida a quente). Feurer, Springenberg e Hutter (2015) propuseram inicializar a otimização bayesiana com configurações que se saíram bem em conjuntos de dados parecidos, como fazem os especialistas, e obtiveram ganhos modestos na busca de hiperparâmetros de um algoritmo e maiores no problema CASH.

Vamos reproduzir a ideia com um **portfólio**: um conjunto pequeno de configurações **complementares**, escolhidas para funcionar bem em conjunto. O procedimento:

1. Avaliar 25 configurações do SVM (5 valores de *C* × 5 de *gamma*) em 40 conjuntos simulados de **origem**.
2. Montar um portfólio de 3 configurações pelo critério guloso: cada nova configuração é a que mais melhora, na média, o **melhor resultado do portfólio** nos conjuntos de origem.
3. No conjunto **alvo** (real), testar essas 3 configurações primeiro e comparar com 3 configurações sorteadas.

Um detalhe técnico importante: o melhor *gamma* depende do número de atributos *d*. Para que as configurações sejam transferíveis, usamos um *gamma* **relativo**: `gamma = gamma_rel / d`.

PYTHON

```
C_VALORES = [0.1, 1, 10, 100, 1000]
G_RELATIVOS = [1e-3, 1e-2, 1e-1, 1, 10]
CONFIGS = [(c, g) for c in C_VALORES for g in G_RELATIVOS]

def desempenho_svm(X, y):
    cv = StratifiedKFold(3, shuffle=True, random_state=0)
    return np.array([cross_val_score(make_pipeline(StandardScaler(), SVC(C=c, gamma=g / X.shape[1])),
X, y, cv=cv).mean() for c, g in CONFIGS])

rng = np.random.default_rng(5)
origem = []
for i in range(40):
    n = int(rng.integers(150, 500)); d = int(rng.integers(8, 40)); inf = int(rng.integers(4, min(d - 2,
12) + 1)); c = int(rng.integers(2, 5))
    X, y = make_classification(n_samples=n, n_features=d, n_informative=inf,
n_redundant=int(rng.integers(0, (d - inf) // 2 + 1)),
                               n_classes=c, n_clusters_per_class=int(rng.integers(1, 3)),
class_sep=float(rng.uniform(0.4, 2.0)),
                               flip_y=float(rng.uniform(0, 0.1)), random_state=1000 + i)
    origem.append(desempenho_svm(X, y))
origem = np.array(origem)

portfolio = []
for _ in range(3):                                          # escolha gulosa
    candidatos = [j for j in range(len(CONFIGS)) if j not in portfolio]
    portfolio.append(max(candidatos, key=lambda j: np.mean(np.max(origem[:, portfolio + [j]],
axis=1))))
print("Portfólio (C, gamma relativo):", [CONFIGS[j] for j in portfolio])

print("\nMelhor acurácia após k tentativas, no conjunto alvo (real):")
print(f"{'alvo':8s} | portfólio: 1, 2, 3 tentativas | aleatório: 1, 3, 6 tentativas | melhor possível")
for nome, carregar in [("Iris", load_iris), ("Wine", load_wine), ("Cancer", load_breast_cancer),
("Digits", load_digits)]:
    X, y = carregar(return_X_y=True); p = desempenho_svm(X, y); r = np.random.default_rng(0)
    port = [p[portfolio[0]], p[portfolio[:2]].max(), p[portfolio].max()]
    aleat = [np.mean([p[r.choice(len(CONFIGS), k, replace=False)].max() for _ in range(500)]) for k in
(1, 3, 6)]
    print(f"{nome:8s} |        {port[0]:.3f}, {port[1]:.3f}, {port[2]:.3f}      |       {aleat[0]:.3f},
{aleat[1]:.3f}, {aleat[2]:.3f}      |      {p.max():.3f}")
```

SAÍDA

```
Portfólio (C, gamma relativo): [(1, 1), (10, 1), (10, 0.1)]

Melhor acurácia após k tentativas, no conjunto alvo (real):
alvo     | portfólio: 1, 2, 3 tentativas | aleatório: 1, 3, 6 tentativas | melhor possível
Iris     |        0.947, 0.953, 0.967      |       0.848, 0.956, 0.963      |      0.967
Wine     |        0.983, 0.989, 0.989      |       0.788, 0.965, 0.988      |      0.994
Cancer   |        0.972, 0.972, 0.977      |       0.897, 0.965, 0.973      |      0.977
Digits   |        0.983, 0.985, 0.985      |       0.801, 0.969, 0.982      |      0.985
```

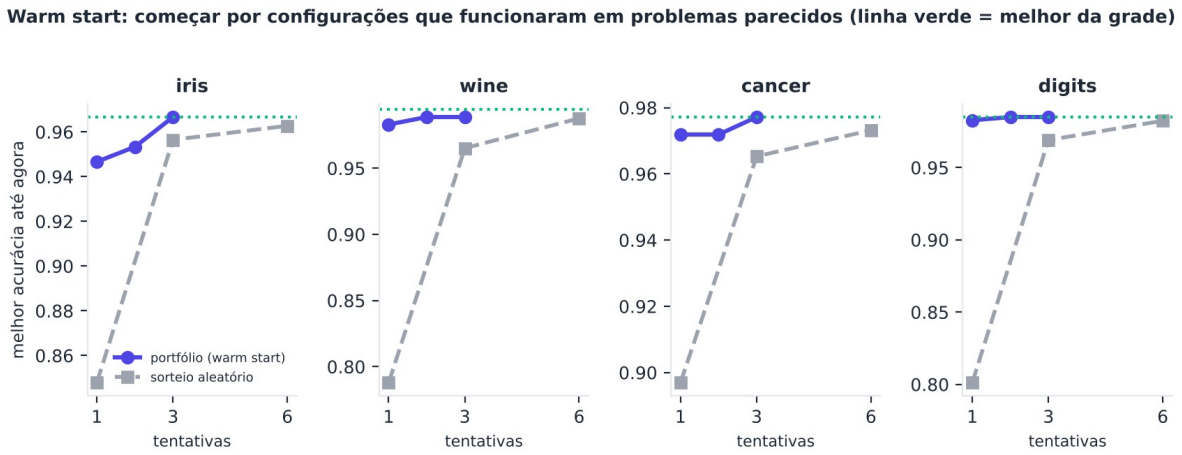


Figura 9.5 — Com o warm start, as primeiras tentativas já chegam perto do melhor resultado; o sorteio aleatório precisa de mais tentativas.

Aqui o ganho é **claro**: com apenas **3 configurações do portfólio**, chega-se perto do melhor valor possível em todos os conjuntos reais. O sorteio aleatório de 3 configurações fica visivelmente atrás, e mesmo com 6 tentativas ele só alcança, no melhor caso, o que o portfólio já tinha com 3. O benefício é maior no **começo** da busca, quando cada tentativa vale mais.

É essa ideia, e não a recomendação do algoritmo, que está no coração do Auto-sklearn. A segunda geração do sistema levou a ideia adiante, com um portfólio construído **sem meta-atributos**, e uma estratégia de alocação de orçamento inspirada nos métodos de bandidos do Capítulo 6 (Feurer *et al.*, 2022).

> **Dica**
>
> No Optuna, você pode dar um *warm start* com `study.enqueue_trial({...})`, que coloca configurações conhecidas na fila das primeiras tentativas, antes do amostrador começar a trabalhar.

9.6 Ensembles: a sabedoria das multidões

Mudamos agora de assunto, para uma ideia independente e muito poderosa: em vez de escolher **um** modelo, **combinar vários**.

Analogia: o concurso do "boi gordo" de uma feira. Cada visitante chuta o peso do animal. Individualmente, os chutes erram bastante. Mas a **média** dos chutes costuma ficar muito próxima do peso verdadeiro, porque os erros de uns compensam os erros de outros.

Com modelos acontece o mesmo, **desde que eles errem de maneiras diferentes**, isto é, tenham **diversidade**.

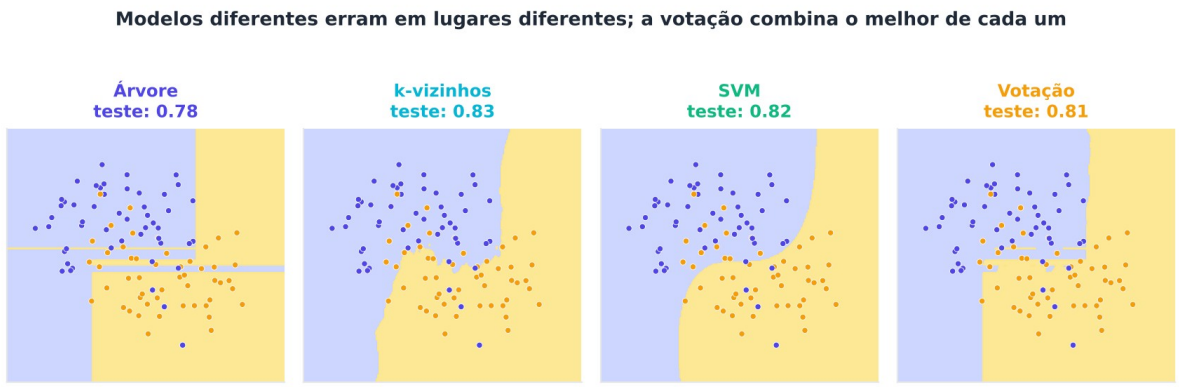


Figura 9.6 — Três modelos desenham fronteiras diferentes e erram em lugares diferentes. A votação combina os três e fica à frente de cada um.

Os principais tipos de combinação são:

- **Bagging e florestas:** treinam muitos modelos do mesmo tipo com amostras diferentes e fazem a média ou votação (Breiman, 2001).
- **Boosting:** treina modelos em sequência, cada um corrigindo os erros dos anteriores (Friedman, 2001).
- **Votação:** combina modelos **de tipos diferentes** por média das probabilidades ou por maioria.
- **Stacking:** um **meta-modelo** aprende a melhor forma de combinar os outros (Wolpert, 1992).

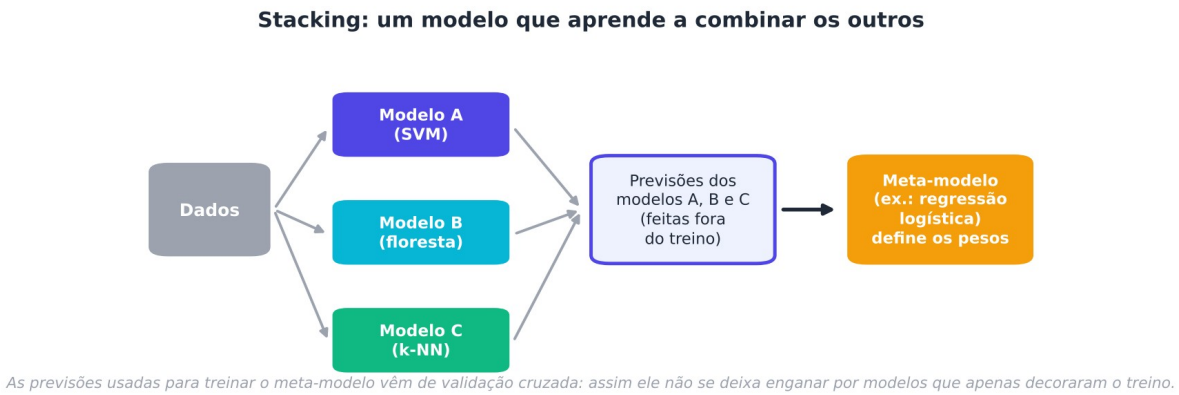


Figura 9.7 — No stacking, um meta-modelo aprende a combinar as previsões dos modelos de base.

No *stacking*, as previsões usadas para treinar o meta-modelo devem ser feitas **fora do treino** (por validação cruzada). Caso contrário, o meta-modelo confiaria demais em modelos que apenas decoraram os dados.

Para ver o efeito de verdade, precisamos de um problema em que os modelos **discordem**. O conjunto Digits é fácil demais: o melhor modelo sozinho já acerta 98,7%. Vamos usar um problema **simulado e difícil**: 2.000 exemplos, 3 classes, 20 atributos e 8% de ruído nos rótulos.

```python
from sklearn.ensemble import VotingClassifier, StackingClassifier

X, y = make_classification(n_samples=2000, n_features=20, n_informative=8, n_redundant=4, n_classes=3,
                           n_clusters_per_class=3, class_sep=0.9, flip_y=0.08, random_state=1)
Xa, Xte, ya, yte = train_test_split(X, y, test_size=0.25, stratify=y, random_state=0)      # Xte: teste final

base = [("svm", SVC(C=10, gamma=0.02, probability=True, random_state=0)),
        ("knn", KNeighborsClassifier(5, weights="distance")),
```

```
        ("lr", LogisticRegression(C=1, max_iter=2000)),
        ("rf", RandomForestClassifier(100, random_state=0, n_jobs=-1))]
cv = StratifiedKFold(5, shuffle=True, random_state=0)

print("Validação cruzada (5 partes) no conjunto de treino:")
for nome, modelo in base:
    print(f"  {nome:20s} acurácia = {cross_val_score(modelo, Xa, ya, cv=cv).mean():.3f}")
print(f"  {'votação (soft)':20s} acurácia = {cross_val_score(VotingClassifier(base, voting='soft'), Xa,
ya, cv=cv).mean():.3f}")
print(f"  {'stacking':20s} acurácia = {cross_val_score(StackingClassifier(base,
final_estimator=LogisticRegression(max_iter=2000), cv=5), Xa, ya, cv=cv).mean():.3f}")
```

SAÍDA

```
Validação cruzada (5 partes) no conjunto de treino:
  svm                  acurácia = 0.750
  knn                  acurácia = 0.740
  lr                   acurácia = 0.603
  rf                   acurácia = 0.723
  votação (soft)       acurácia = 0.765
  stacking             acurácia = 0.791
```

A votação e o *stacking* ficam **acima do melhor modelo individual**, e o *stacking* vai um pouco mais longe. Note que a regressão logística, sozinha, é fraca (esse problema tem fronteiras não lineares), mas o *stacking* a aproveita onde ela ajuda.

9.7 Seleção de ensembles (Caruana)

E se tivermos **dezenas de modelos** já treinados, por exemplo, todos os testados durante a busca de hiperparâmetros? Em vez de jogar fora os "perdedores", podemos usá-los. Foi o que propuseram Caruana e colaboradores (2004): construir um ensemble a partir de uma **biblioteca de modelos**, por **seleção gulosa para frente**.

O algoritmo é simples:

1. Comece com um ensemble vazio.
2. Em cada passo, **acrescente o modelo da biblioteca que mais melhora** o ensemble nos dados de **validação** (a mesma pode ser escolhida **mais de uma vez**: isso funciona como um peso maior).
3. Repita até um número máximo de passos. As contagens finais viram os pesos.

Vamos implementá-lo. A biblioteca terá 40 modelos de cinco tipos, com hiperparâmetros sorteados (como numa busca aleatória). Dividimos os dados de treino em treino e **validação** (a validação serve para escolher o ensemble), e deixamos o **teste final** intocado.

PYTHON

```
Xtr, Xva, ytr, yva = train_test_split(Xa, ya, test_size=0.33, stratify=ya, random_state=0)
rng = np.random.default_rng(0)
biblioteca = []
for i in range(40):
    tipo = str(rng.choice(["svm", "knn", "lr", "rf", "hgb"]))
    if tipo == "svm":   m = SVC(C=10 ** rng.uniform(-1, 3), gamma=10 ** rng.uniform(-4, -0.5),
probability=True, random_state=0)
    elif tipo == "knn": m = KNeighborsClassifier(int(rng.integers(1, 25)), weights="distance")
    elif tipo == "lr":  m = LogisticRegression(C=10 ** rng.uniform(-3, 2), max_iter=2000)
```

```
    elif tipo == "rf":  m = RandomForestClassifier(int(rng.integers(20, 120)),
max_depth=int(rng.integers(3, 20)), random_state=0, n_jobs=-1)
    else:               m = HistGradientBoostingClassifier(learning_rate=10 ** rng.uniform(-1.7, -0.3),
max_iter=60, max_leaf_nodes=int(rng.integers(4, 30)), random_state=0)
    m.fit(Xtr, ytr)
    biblioteca.append((tipo, m.predict_proba(Xva), m.predict_proba(Xte)))

nota_val = [accuracy_score(yva, pv.argmax(1)) for _, pv, _ in biblioteca]
nota_teste = [accuracy_score(yte, pt.argmax(1)) for _, _, pt in biblioteca]
melhor = int(np.argmax(nota_val))

# Seleção gulosa para frente, com reposição, minimizando a perda logarítmica na validação
escolhidos, atual = [], None
for passo in range(25):
    melhor_j, melhor_perda = None, np.inf
    for j, (_, pv, _) in enumerate(biblioteca):
        candidato = pv if not escolhidos else (atual * len(escolhidos) + pv) / (len(escolhidos) + 1)
        perda = log_loss(yva, candidato, labels=range(3))
        if perda < melhor_perda: melhor_j, melhor_perda = j, perda
    atual = biblioteca[melhor_j][1] if not escolhidos else (atual * len(escolhidos) +
biblioteca[melhor_j][1]) / (len(escolhidos) + 1)
    escolhidos.append(melhor_j)

pesos = np.bincount(escolhidos, minlength=len(biblioteca))
ensemble_teste = sum(pesos[j] * biblioteca[j][2] for j in range(len(biblioteca))) / pesos.sum()
media_todos = sum(pt for _, _, pt in biblioteca) / len(biblioteca)

print(f"Melhor modelo escolhido pela validação ({biblioteca[melhor][0]}): validação =
{nota_val[melhor]:.3f} | TESTE = {nota_teste[melhor]:.3f}")
print(f"Média simples dos 40 modelos                        : TESTE = {accuracy_score(yte,
media_todos.argmax(1)):.3f}")
print(f"Ensemble de Caruana ({(pesos > 0).sum()} modelos diferentes)        : TESTE =
{accuracy_score(yte, ensemble_teste.argmax(1)):.3f}")
print(f"(Melhor modelo individual no teste, com informação do futuro: {max(nota_teste):.3f})")
print("Modelos mais usados:", [(biblioteca[j][0], int(pesos[j])) for j in np.argsort(-pesos)[:5] if
pesos[j] > 0])
```

SAÍDA

```
Melhor modelo escolhido pela validação (rf): validação = 0.778 | TESTE = 0.684
Média simples dos 40 modelos                        : TESTE = 0.730
Ensemble de Caruana (11 modelos diferentes)        : TESTE = 0.738
(Melhor modelo individual no teste, com informação do futuro: 0.738)
Modelos mais usados: [('hgb', 6), ('knn', 4), ('hgb', 3), ('knn', 3), ('hgb', 2)]
```

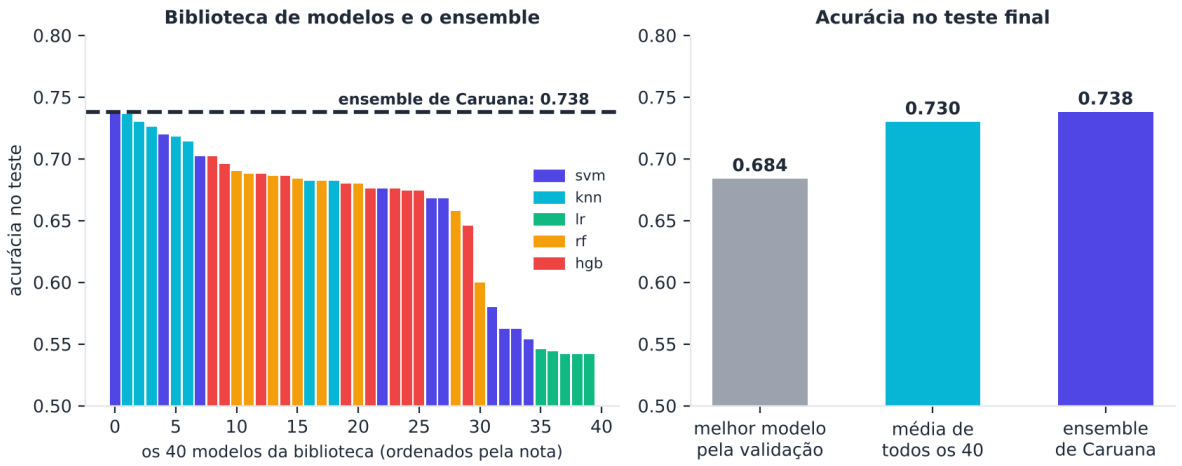


Figura 9.8 — À esquerda, os 40 modelos da biblioteca (cores = tipo) e a linha do ensemble. À direita, a comparação no teste final.

O resultado é instrutivo. O ensemble (0,738) supera o **melhor modelo escolhido pela validação** (0,684) e fica um pouco acima da **simples média de todos os modelos** (0,730). Ele apenas **empata** com o melhor modelo individual no teste (0,738), mas esse "campeão" só seria conhecido olhando o futuro: na prática, ninguém consegue escolhê-lo. Ou seja, o ensemble não vence o oráculo; ele nos protege de escolher mal. Grande parte do ganho vem da própria diversidade (a média simples já chega perto), e a seleção gulosa acrescenta um pouco, ao dar mais peso a modelos bons e diferentes entre si.

> **Atenção**
>
> A seleção de ensemble usa os dados de validação para escolher os pesos. Como vimos no Capítulo 4, isso também pode causar sobreajuste à validação. Por isso mantemos o **teste final intocado** e relatamos a nota nele.

9.8 O Auto-sklearn: tudo junto

Agora podemos entender o **Auto-sklearn** (Feurer *et al.*, 2015), um dos sistemas mais influentes da área. Ele combina três ideias:

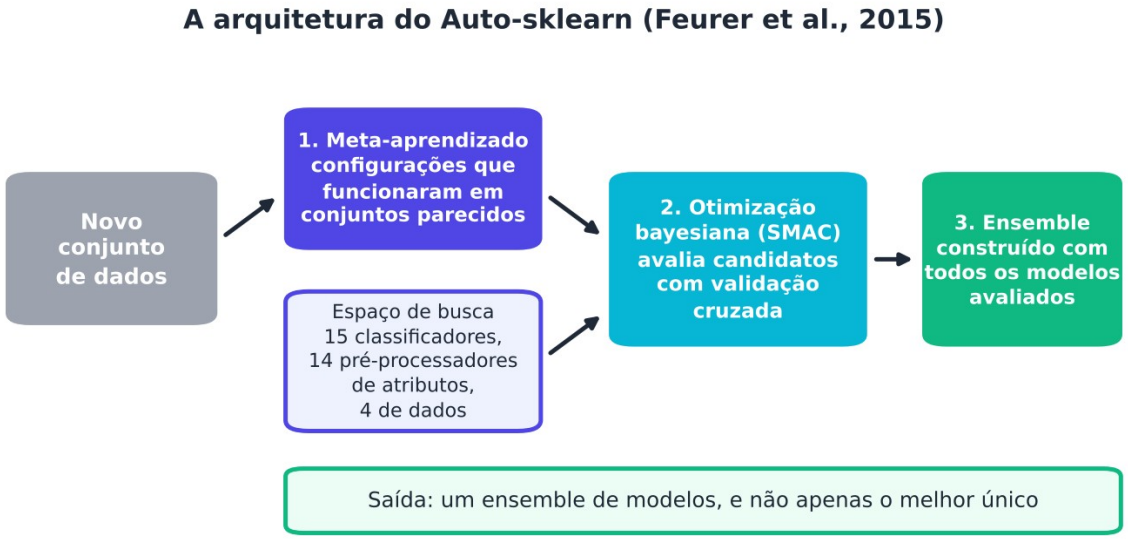


Figura 9.9 — A arquitetura do Auto-sklearn: meta-aprendizado, otimização bayesiana e construção de ensembles.

1. **Meta-aprendizado para o *warm start*:** usa o desempenho em conjuntos de dados anteriores para começar a busca por configurações promissoras.

2. **Otimização bayesiana** sobre um grande espaço de busca (15 classificadores, 14 métodos de pré-processamento de atributos e 4 de pré-processamento de dados, com 110 hiperparâmetros) (Feurer *et al.*, 2015).
3. **Construção automática de ensembles** a partir dos modelos avaliados durante a otimização, no espírito da seleção de Caruana.

Os autores relatam que o sistema superou de forma substancial o estado da arte da época em mais de 100 conjuntos de dados (Feurer *et al.*, 2015). Na versão 2.0, o foco foi um sistema "sem configuração manual", com meta-aprendizado **sem meta-atributos** e uma estratégia de bandidos para alocar o orçamento; em 39 conjuntos de um *benchmark* aberto, os autores reportam redução de erro relativo de até 4,5 vezes em relação a outras ferramentas e à versão anterior, e desempenho em 10 minutos equivalente ao que a versão 1.0 obtinha em 1 hora (Feurer *et al.*, 2022).

> **Atenção**
>
> O Auto-sklearn não pôde ser instalado no ambiente em que este livro foi preparado, por isso **não há código executado** dele aqui. No Capítulo 10, você verá uma visão de conjunto das ferramentas e o código de uso, com os devidos avisos.

Observe o que todos os ingredientes têm em comum com o que aprendemos: busca bayesiana (Capítulo 5), orçamento por bandidos (Capítulo 6), pipelines (Capítulo 3), validação honesta (Capítulos 2 e 4). O AutoML é uma **orquestra** dessas ideias.

9.9 Boas práticas

1. **Use o warm start** quando houver configurações conhecidas que funcionam em problemas parecidos (inclusive as suas, de projetos anteriores).
2. **Seja cético com a recomendação de algoritmos:** o ganho sobre "SVM ou floresta" costuma ser pequeno sem um meta-conjunto grande.
3. **Procure diversidade** ao montar ensembles: modelos de tipos diferentes e hiperparâmetros variados.
4. **Teste a média simples** antes de métodos sofisticados de combinação: ela costuma capturar boa parte do ganho.
5. **Escolha os pesos do ensemble na validação e meça no teste final.**
6. **Considere o custo:** um ensemble de 30 modelos custa 30 vezes mais em memória e tempo de previsão. Em produção, isso pesa (Capítulo 14).
7. **Perda de interpretabilidade:** um ensemble é mais difícil de explicar do que um modelo simples (Capítulo 13).

Resumo do capítulo

MEMORIZE **Cartões de memorização**

- **Meta-aprendizado:** usar a experiência de problemas anteriores (conjuntos, modelos, resultados).
- **Quadro de Rice:** problemas → meta-atributos → meta-modelo → algoritmo → desempenho.
- **Meta-atributos:** simples, teoria da informação, complexidade e landmarkers.
- **Recomendar algoritmo** é difícil: no nosso experimento, ficou no nível da regra "sempre o mais frequente".
- **Warm start** (portfólio de configurações): ganho claro nas primeiras tentativas.
- **Ensembles:** combinar modelos diversos; votação, stacking e seleção gulosa de Caruana.
- **Média simples** captura boa parte do ganho; a seleção acrescenta um pouco.
- **Auto-sklearn:** meta-aprendizado + otimização bayesiana + ensemble.

Exercícios de múltipla escolha

Escolha **uma** alternativa em cada questão. O gabarito está no Apêndice A.

Questão 1. O que é meta-aprendizado?

a) Dividir os dados em treino e validação.
b) Usar a experiência de conjuntos de dados anteriores para decidir melhor em um problema novo.
c) Treinar um modelo com dados do teste.
d) Aumentar o número de camadas de uma rede neural.

Questão 2. Na analogia do capítulo, o meta-aprendizado corresponde ao:

a) laboratório de exames.
b) médico experiente, que reconhece o caso e começa pelo exame certo.
c) paciente.
d) médico iniciante, que pede todos os exames.

Questão 3. O problema da seleção de algoritmos foi formulado por:

a) Feurer, em 2015.
b) Rice, em 1976.
c) Wolpert, em 1992.
d) Caruana, em 2004.

Questão 4. Qual dos itens abaixo NÃO é um dos quatro espaços do quadro de Rice?

a) Espaço dos usuários.
b) Espaço dos problemas.
c) Espaço dos algoritmos.
d) Espaço das características.

Questão 5. O que são meta-atributos?

a) Colunas de um conjunto de dados.
b) Rótulos de classes.
c) Descrições resumidas de um conjunto de dados inteiro.
d) Hiperparâmetros de um modelo.

Questão 6. O que é um landmarker?

a) Uma função de aquisição.
b) A nota de um modelo simples e rápido no problema, usada como sonda.
c) Um monumento histórico da área.
d) Um hiperparâmetro do Optuna.

Questão 7. A entropia das classes, como meta-atributo, indica:

a) quão balanceadas estão as classes.
b) quantos atributos existem.
c) o tempo de treino.
d) a correlação entre atributos.

Questão 8. Por que a literatura aponta dificuldades de reprodução com meta-atributos?

a) Porque dependem de GPU.
b) Porque sua descrição, organização e cálculo variam entre trabalhos.
c) Porque meta-atributos não podem ser calculados.
d) Porque só funcionam em dados de texto.

Questão 9. Como foi construído o meta-conjunto do capítulo?

a) Com 150 conjuntos simulados, descritos por meta-atributos e rotulados com o melhor entre quatro algoritmos.
b) Com imagens de dígitos.
c) Com 150 conjuntos reais do Kaggle.
d) Com 4 conjuntos reais.

Questão 10. O que significa a validação "deixe um conjunto de fora" no meta-aprendizado?

a) O meta-modelo é treinado com os outros conjuntos e testado no excluído.
b) Uma coluna de cada conjunto é removida.
c) O meta-modelo só usa um conjunto.
d) O conjunto de teste é removido do estudo.

Questão 11. Qual era a linha de base do experimento de meta-aprendizado?

a) Sortear o algoritmo.
b) Recomendar sempre o algoritmo que mais vence.
c) Não recomendar nada.
d) Recomendar sempre a floresta aleatória.

Questão 12. Qual foi o resultado do meta-modelo frente à linha de base no experimento?

a) Foi muito superior, com 95% de acerto.
b) Acertou na mesma proporção (cerca de 59%) e teve perda média só ligeiramente menor.
c) Não pôde ser avaliado.
d) Foi muito inferior, com 20% de acerto.

Questão 13. Qual das razões abaixo explica o pequeno ganho do meta-modelo?

a) Poucos conjuntos simulados, poucos meta-atributos e algoritmos competitivos entre si.
b) O uso de validação cruzada.
c) Erro de digitação no código.
d) Falta de memória do computador.

Questão 14. Qual forma de meta-aprendizado é mais robusta e mais usada na prática, segundo o capítulo?

a) Recomendar por onde começar a busca (warm start).
b) Dividir os dados.
c) Recomendar o algoritmo com poucos dados.
d) Escolher os atributos.

Questão 15. O que é warm start na otimização de hiperparâmetros?

a) Usar o teste como validação.
b) Aquecer a CPU antes do treino.
c) Iniciar a busca com configurações que se saíram bem em problemas parecidos.
d) Sortear a primeira configuração.

Questão 16. No experimento de portfólio, como foi escolhido o portfólio de 3 configurações?

a) Por votação humana.
b) Por escolha gulosa: cada nova configuração melhora ao máximo o melhor resultado médio nos conjuntos de origem.
c) Pelo teste final.
d) Por sorteio.

Questão 17. Por que se usou um gamma relativo (gamma = gamma_rel / d) no portfólio?

a) Para evitar o uso de validação.
b) Para acelerar o treino.
c) Porque o scikit-learn exige.
d) Para tornar as configurações transferíveis entre conjuntos com números diferentes de atributos.

Questão 18. No experimento do portfólio, qual foi o resultado com 3 configurações do portfólio nos conjuntos reais?

a) Chegaram perto do melhor valor possível em todos.
b) Empataram com a pior configuração.
c) Não puderam ser avaliadas.
d) Foram sempre piores que o sorteio.

Questão 19. Em que momento da busca o warm start é mais vantajoso?

a) No começo, quando cada tentativa vale mais.
b) Sempre no final.
c) Apenas depois de 1000 tentativas.
d) Nunca.

Questão 20. Qual método do Optuna permite dar um warm start com configurações conhecidas?

a) trial.skip().
b) study.pause().
c) optuna.freeze().
d) study.enqueue_trial().

Questão 21. Para que ensembles funcionem bem, os modelos precisam:

a) usar sempre a mesma fronteira.
b) errar de maneiras diferentes (diversidade).
c) ter a mesma semente.
d) ser todos idênticos.

Questão 22. Na analogia do "boi gordo", o que corresponde à votação de modelos?

a) A média dos chutes de vários visitantes.
b) O preço do boi.
c) O chute do melhor visitante.
d) O peso verdadeiro do boi.

Questão 23. O que é stacking?

a) Empilhar dados de treino.
b) Escolher o modelo de maior nota.
c) Rodar modelos em paralelo sem combinar.
d) Um meta-modelo aprende a combinar as previsões dos modelos de base.

Questão 24. Por que as previsões usadas para treinar o meta-modelo do stacking devem ser feitas fora do treino?

a) Para acelerar a validação cruzada.
b) Para que o meta-modelo não confie demais em modelos que apenas decoraram os dados.
c) Para reduzir o tamanho dos dados.
d) Para evitar o uso de probabilidades.

Questão 25. No experimento com votação e stacking, qual foi o resultado em relação ao melhor modelo individual?

a) Ficaram abaixo de todos os modelos.
b) Votação e stacking ficaram acima do melhor modelo individual.
c) Ficaram idênticos à regressão logística.
d) Não puderam ser comparados.

Questão 26. Por que o conjunto Digits não foi usado para demonstrar o ganho dos ensembles?

a) Porque não tem rótulos.
b) Porque tem poucas linhas.
c) Porque é fácil demais: um único modelo já passa de 98%.
d) Porque é proibido.

Questão 27. O que faz a seleção de ensembles de Caruana?

a) Remove o pior modelo a cada passo.
b) Escolhe aleatoriamente os modelos.
c) Acrescenta, passo a passo, o modelo da biblioteca que mais melhora o ensemble na validação.
d) Usa apenas o melhor modelo.

Questão 28. Por que o mesmo modelo pode ser escolhido mais de uma vez na seleção de Caruana?

a) Por erro de implementação.
b) Isso funciona como dar a ele um peso maior no ensemble.
c) Para reduzir o número de modelos.
d) Para evitar a validação.

Questão 29. No experimento de Caruana, a média simples dos 40 modelos:

a) foi igual a 50%.
b) não foi calculada.
c) já chegou perto do ensemble de Caruana, mostrando que grande parte do ganho vem da diversidade.
d) foi a pior opção, muito abaixo dos demais.

Questão 30. Qual das ideias abaixo NÃO faz parte do Auto-sklearn (versão original)?

a) Construção de ensembles com os modelos avaliados.
b) Otimização bayesiana.
c) Busca de arquiteturas neurais por gradiente.
d) Meta-aprendizado para warm start.

Exercícios discursivos

Responda com suas palavras, em poucas linhas. Compare depois com a resposta-modelo do Apêndice A.

1. Explique a ideia do meta-aprendizado usando uma analogia do cotidiano.
2. Descreva os quatro espaços do quadro de Rice e a medida de desempenho.
3. Cite as quatro famílias de meta-atributos e dê um exemplo de cada.
4. Explique o que é um landmarker e por que ele é um meta-atributo útil.
5. Descreva como foi construído e avaliado o meta-modelo do capítulo.
6. Interprete o resultado do meta-modelo: ele funcionou? Cite três razões para o ganho ser pequeno.
7. Por que o capítulo diz que recomendar o algoritmo é a forma mais difícil de meta-aprendizado?
8. Explique o warm start e por que ele é mais robusto do que recomendar o algoritmo.
9. Descreva como se construiu o portfólio de configurações e como ele foi avaliado.
10. Interprete os resultados do portfólio. Em que situação o benefício é maior?
11. Por que se usa um gamma relativo no portfólio de SVM?
12. Explique por que combinar modelos costuma funcionar melhor do que escolher só um.
13. Compare bagging/florestas, boosting, votação e stacking.
14. Descreva o algoritmo de seleção de ensembles de Caruana.
15. Interprete os resultados do experimento de Caruana, incluindo o papel da média simples.
16. Por que o ensemble também está sujeito ao sobreajuste da validação e como evitar?
17. Descreva a arquitetura do Auto-sklearn e a contribuição de cada componente.
18. O que mudou na versão 2.0 do Auto-sklearn, segundo o capítulo?
19. Cite três cuidados práticos com ensembles (custo, interpretabilidade e avaliação) e comente um.
20. Como o capítulo mostra que o AutoML é uma "orquestra" de ideias vistas antes? Cite quatro delas.

Exercícios práticos em Python (scikit-learn)

Resolva os exercícios no seu computador (ou no Google Colab). Eles usam conjuntos de dados da UCI que já vêm no scikit-learn ou dados simulados. Tente resolver **antes** de consultar a solução, no Apêndice D. As saídas apresentadas lá foram obtidas com as versões de bibliotecas listadas na Apresentação.

Exercício 1 — O retrato de um conjunto de dados

NÍVEL: BÁSICO

Calcule cinco meta-atributos (número de exemplos, número de atributos, razão exemplos/atributos, número de classes e entropia normalizada das classes) para três versões do conjunto **Wine** (UCI): (a) original; (b) com 100 colunas de ruído acrescentadas; (c) uma amostra de apenas 60 exemplos.

Pergunta: quais meta-atributos mudaram e quais permaneceram iguais? O que isso sugere sobre o que cada meta-atributo captura?

Exercício 2 — Votação dura ou suave?

NÍVEL: BÁSICO

No problema simulado difícil do capítulo (2.000 exemplos, 3 classes, 20 atributos, 8% de ruído, `random_state=1`), compare, por validação cruzada de 5 partes, a votação **dura** (`voting="hard"`, cada modelo vota em uma classe) e a **suave** (`voting="soft"`, média das probabilidades) com os quatro modelos de base (SVM, k-vizinhos, regressão logística e floresta).

Pergunta: qual funcionou melhor? Por que a votação suave costuma ser preferível?

Exercício 3 — Diversidade importa

NÍVEL: INTERMEDIÁRIO

No mesmo problema, divida os dados em treino e teste (25%, estratificado). Compare a acurácia no teste de: (a) um único `RandomForestClassifier`; (b) um ensemble por votação suave de **três florestas com sementes diferentes** (pouca diversidade); (c) um ensemble por votação suave de **quatro modelos de tipos diferentes** (SVM, k-vizinhos, regressão logística e floresta). Calcule também a **taxa de discordância** média entre os pares de modelos de cada ensemble (fração de exemplos de teste em que as previsões diferem).

Pergunta: o ensemble menos diverso ajudou? Qual a relação entre diversidade e ganho?

Exercício 4 — Warm start com o Optuna

NÍVEL: AVANÇADO

No conjunto **Wine** (UCI), ajuste um `SVC` (com `StandardScaler`) com o Optuna (TPE) por 10 tentativas, em duas versões: (a) partida a frio (sem fila) e (b) com warm start, enfileirando com `study.enqueue_trial` as três configurações do portfólio do capítulo: (C = 1, γ_rel = 1), (C = 10, γ_rel = 1) e (C = 10, γ_rel = 0,1), com `gamma = γ_rel / d`. Repita com 5 sementes e mostre a melhor nota média após 1, 3 e 10 tentativas.

Pergunta: o warm start ajudou? Em que parte da busca?

> SAIBA MAIS **Para ir além: bases públicas da UCI e do Kaggle**
>
> - **UCI:** Wine Quality (classificação de qualidade), Adult (renda) e Spambase (spam; bom para ensembles).
> - **Kaggle:** Titanic e Spaceship Titanic (poucos dados, onde ensembles costumam ajudar).
>
> Estes conjuntos não foram usados nem testados nos códigos do livro. Confira, nos sites, a descrição, a licença e as colunas antes de usá-los.

Projeto integrador do capítulo

Um mini Auto-sklearn: warm start, busca TPE e ensemble

Fonte dos dados. *Breast Cancer Wisconsin (Diagnostic)* (UCI, via scikit-learn), com duas alterações **simuladas por você** para tornar o problema mais difícil: 100 colunas de ruído e 10% de rótulos trocados ao acaso (ruído de rótulo).

Contexto. Uma equipe quer montar, com as peças do capítulo, um AutoML minimalista: começa por configurações conhecidas (warm start), busca com o TPE do Optuna e combina os modelos avaliados em um ensemble.

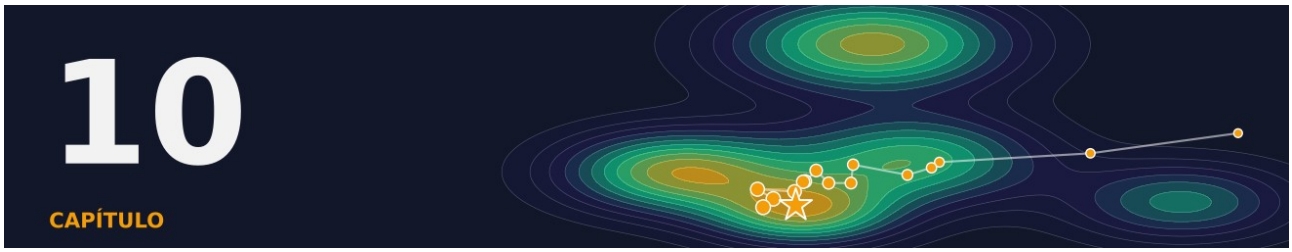
Atenção ética. Os dados são de uso didático e foram deliberadamente alterados. O modelo **não é diagnóstico médico**.

Tarefas.

1. Monte o conjunto e separe 25% para **teste final** (estratificado). Do restante, separe um conjunto de **validação** (um terço), usado **apenas** para escolher os pesos do ensemble.
2. Defina um espaço CASH condicional no Optuna com `LogisticRegression`, `SVC` e `RandomForestClassifier` (use escala logarítmica quando fizer sentido).
3. Enfileire como *warm start* duas configurações conhecidas (um SVM com C = 1 e gamma = `1/d`; uma floresta com 100 árvores e profundidade 8).
4. Rode 30 tentativas. Em cada uma, avalie a nota por **validação cruzada de 3 partes no treino**; depois, treine o modelo no treino completo e **guarde as probabilidades** na validação e no teste.
5. Calcule: (a) o melhor modelo único pela validação cruzada; (b) a média simples de todos os modelos; (c) o ensemble de Caruana (seleção gulosa, 20 passos, na perda logarítmica da validação).
6. Compare as três opções no **teste final**, uma única vez.
7. Escreva um parágrafo (6 a 8 linhas): o ensemble ajudou? A média simples bastou? Que riscos permanecem (tamanho do teste, ruído)?

Entregáveis. Um notebook (ou script) comentado, a tabela comparativa e o parágrafo de conclusões.

A solução de referência está no Apêndice E.



Capítulo 10 — Ferramentas de AutoML na prática

OBJETIVOS DE APRENDIZAGEM

Ao final deste capítulo você será capaz de:

- Reconhecer as principais ferramentas de AutoML e as ideias que cada uma emprega.
- Usar o FLAML para resolver um problema de classificação com orçamento de tempo.
- Ler e interpretar o resultado de uma busca automática (modelo escolhido, configuração e histórico).
- Montar um pequeno experimento comparativo entre ferramentas, com várias divisões dos dados.
- Aplicar o teste de Friedman e a leitura de postos para comparar métodos em vários conjuntos.
- Escolher uma ferramenta de AutoML de acordo com o problema, o tempo e os recursos disponíveis.

10.1 Uma feira de ferramentas

Até aqui, construímos as peças do AutoML **com as nossas mãos**: busca em grade, busca aleatória, Optuna, Hyperband, algoritmo genético, ensembles. Chegou a hora de visitar a "feira" onde essas peças já vêm montadas em **ferramentas prontas**.

Pense numa cozinha. Você já sabe cortar, refogar e temperar (os capítulos anteriores). Agora vai conhecer as **cozinhas profissionais**, que servem o prato inteiro. Cada uma tem seu estilo, sua especialidade e seu preço (em tempo e computação).

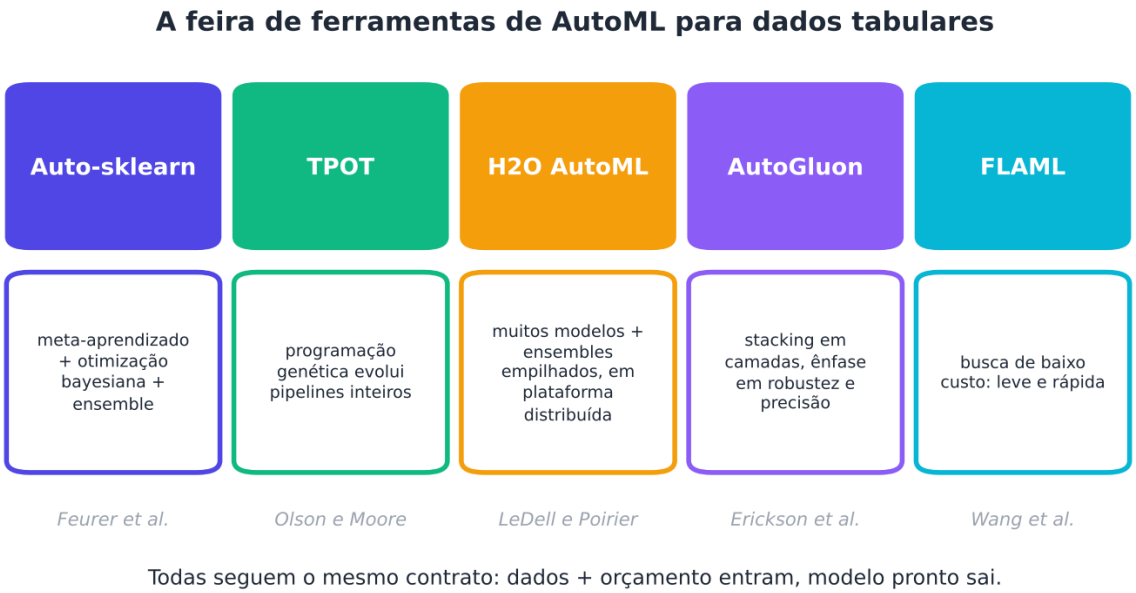


Figura 10.1 — Cinco ferramentas de AutoML para dados tabulares, cada uma com sua "receita" principal.

Eis as ferramentas que vamos conhecer, todas descritas em publicações científicas:

- **Auto-sklearn** (Feurer *et al.*, 2015; 2022): meta-aprendizado, otimização bayesiana e ensemble sobre o scikit-learn (Capítulo 9).
- **TPOT** (Olson; Moore, 2016; Olson *et al.*, 2016): programação genética que evolui *pipelines* inteiros (Capítulo 7).
- **H2O AutoML** (LeDell; Poirier, 2020): treina uma série de modelos em uma plataforma distribuída e combina todos em *ensembles* empilhados.
- **AutoGluon-Tabular** (Erickson *et al.*, 2020): treina muitos modelos e os combina em camadas de *stacking*, com ênfase em robustez e precisão.
- **FLAML** (Wang *et al.*, 2021): biblioteca **leve e rápida**, que busca o melhor modelo gastando o mínimo de tempo e de custo computacional.

Há ainda ferramentas voltadas para redes neurais, como o Auto-Keras (Jin; Song; Hu, 2019), que veremos nos Capítulos 11 e 12.

> **DICA** **Um aviso sobre a escolha**
>
> Nenhuma ferramenta é a melhor em tudo. O que muda de uma para outra é o **equilíbrio** entre precisão, velocidade, facilidade e recursos exigidos. Por isso, o foco deste capítulo é aprender a **usar bem** e a **comparar com cuidado**, mais do que eleger um campeão.

10.1.1 O que há de comum entre elas

Apesar dos estilos diferentes, quase todas seguem o mesmo roteiro:

1. Você entrega os **dados** (treino) e um **orçamento** (tempo máximo).
2. A ferramenta decide **sozinha** como tratar os dados, quais algoritmos testar e com quais hiperparâmetros.
3. Ela avalia os candidatos por **validação** (cruzada ou separada).
4. Ela devolve o **melhor modelo** (ou um ensemble) pronto para prever.



Figura 10.2 — O roteiro comum das ferramentas de AutoML: dados e orçamento entram, um modelo pronto sai.

10.2 O FLAML na prática

Vamos começar pelo **FLAML** (*Fast and Lightweight AutoML*), porque ele é leve, instala sem dificuldade e roda bem em um computador comum. A ideia central do artigo original é buscar com **custo baixo primeiro**: testar

configurações *baratas* (modelos pequenos, poucas árvores) antes das *caras*, e só aumentar o esforço quando vale a pena (Wang *et al.*, 2021).

> NO SEU DIA A DIA **Analogia: o garimpeiro prudente**
>
> Um garimpeiro experiente não cava um poço profundo em cada ponto do rio. Ele começa com **peneiradas rápidas** em vários lugares e só cava fundo onde viu brilho. O FLAML faz algo parecido: começa com modelos pequenos e baratos e vai investindo mais onde há sinal de ganho.

Vamos usar o conjunto **Breast Cancer Wisconsin** (UCI), disponível no scikit-learn, com 569 exemplos e 30 atributos. Reservamos 25% para o teste final, que usaremos **uma única vez**.

PYTHON

```python
import warnings; warnings.filterwarnings("ignore")
import numpy as np
from flaml import AutoML
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

X, y = load_breast_cancer(return_X_y=True)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.25, stratify=y, random_state=0)

automl = AutoML()
automl.fit(Xtr, ytr, task="classification", time_budget=10, metric="accuracy",
           eval_method="cv", n_splits=3, seed=0, verbose=0)

print("Modelo escolhido        :", automl.best_estimator)
print("Erro de validação (cv)  : %.4f" % automl.best_loss)
print("Acurácia no TESTE final : %.4f" % accuracy_score(yte, automl.predict(Xte)))
print("Melhorias registradas   :", len(automl.config_history))
```

SAÍDA

```
Modelo escolhido        : extra_tree
Erro de validação (cv)  : 0.0399
Acurácia no TESTE final : 0.9371
Melhorias registradas   : 9
```

O resultado muda um pouco de uma execução para outra, porque **o orçamento é em tempo** e o computador não executa sempre a mesma quantidade de trabalho em 10 segundos. Isso é importante: com orçamento de tempo, a **reprodutibilidade exata** exige fixar também o número de tentativas, algo que veremos a seguir.

10.2.1 Lendo a "caixa de ferramentas" do resultado

Depois do `fit`, o objeto guarda muita informação útil. Vale a pena olhar:

- `best_estimator`: o nome do algoritmo vencedor (por exemplo, `lgbm`, `rf`, `extra_tree`, `lrl1`).
- `best_config`: os hiperparâmetros do vencedor.
- `best_loss`: o erro de validação do vencedor.
- `config_history`: o histórico das configurações que **melhoraram** o melhor resultado até então (tentativa, algoritmo, configuração e tempo).
- `model.estimator`: o modelo do scikit-learn ou da biblioteca original, pronto para uso.

PYTHON

```python
import warnings; warnings.filterwarnings("ignore")
from flaml import AutoML
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split

X, y = load_breast_cancer(return_X_y=True)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.25, stratify=y, random_state=0)

a = AutoML()
a.fit(Xtr, ytr, task="classification", time_budget=8, metric="accuracy", eval_method="cv",
      n_splits=3, seed=0, verbose=0, estimator_list=["lgbm", "rf", "extra_tree", "lrl1"])

print("Vencedor:", a.best_estimator)
print("Configuração:")
for k, v in a.best_config.items(): print(f"  {k:<14s} {v if isinstance(v, str) else round(float(v),
3)}")
print("Modelo pronto:", type(a.model.estimator).__name__)
print()
print("Melhor erro por algoritmo (validação cruzada):")
for nome, erro in sorted(a.best_loss_per_estimator.items(), key=lambda t: t[1]):
    print(f"  {nome:<11s} erro = {erro:.4f}")
```

SAÍDA

```
Vencedor: extra_tree
Configuração:
  n_estimators   13.0
  max_features   0.1
  max_leaves     10.0
  criterion      entropy
Modelo pronto: ExtraTreesClassifier

Melhor erro por algoritmo (validação cruzada):
  extra_tree  erro = 0.0399
  lgbm        erro = 0.0469
  rf          erro = 0.0493
  lrl1        erro = 0.0986
```

Repare que o FLAML **testou os algoritmos permitidos** (`estimator_list`) e guardou o melhor resultado de cada um. Isso é o problema **CASH** do Capítulo 5 funcionando por baixo dos panos: escolher o algoritmo e seus hiperparâmetros ao mesmo tempo.

10.2.2 Os botões do fit

Os parâmetros mais importantes de uma chamada de AutoML são parecidos entre as ferramentas. Em FLAML, os principais são:

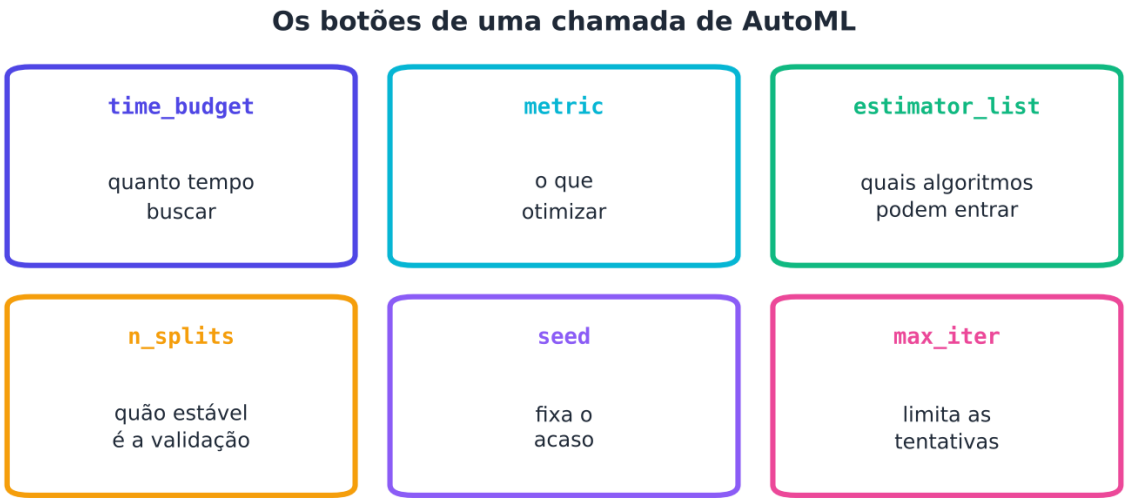


Figura 10.3 — Os principais "botões" de uma chamada de AutoML e o que cada um controla.

Parâmetro	Para que serve	Cuidado
`time_budget`	Tempo máximo (em segundos)	Mais tempo, mais busca, mas com retornos decrescentes
`metric`	Métrica a otimizar	Escolha a métrica que reflete o **seu** problema (Cap. 2)
`estimator_list`	Algoritmos permitidos	Restringir pode acelerar, mas pode excluir o melhor
`eval_method`, `n_splits`	Validação (cruzada ou holdout)	Validação cruzada é mais estável, porém mais cara
`seed`	Semente aleatória	Fixa a parte aleatória, mas não o tempo de relógio
`max_iter`	Número máximo de tentativas	Ajuda a obter resultados mais reprodutíveis

> **Atenção: orçamento de tempo e reprodutibilidade**
>
> Se você precisa que dois colegas obtenham **exatamente o mesmo** resultado, não basta fixar a semente: o tempo de relógio depende da máquina. Nesse caso, limite o número de tentativas (`max_iter`) em vez do tempo, e registre as **versões das bibliotecas** (Capítulo 14).

10.3 Mais tempo significa melhor modelo?

Uma pergunta que todo usuário faz: "se eu der **mais tempo**, o resultado melhora?". A resposta honesta é: **melhora no começo e depois estabiliza**. É a curva clássica de retornos decrescentes.

Vamos medir no conjunto **Digits** (1.797 imagens 8×8 de dígitos, com 10 classes), repetindo a execução com duas sementes para cada orçamento. Para manter o experimento rápido, os orçamentos são curtos.

```python
import warnings; warnings.filterwarnings("ignore")
import json, time
import numpy as np
from flaml import AutoML
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
```

```
X, y = load_digits(return_X_y=True)
res = {}
for b in (2, 5, 10, 20):
    acc = []
    for s in (0, 1):
        Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.25, stratify=y, random_state=s)
        a = AutoML()
        a.fit(Xtr, ytr, task="classification", time_budget=b, metric="accuracy", eval_method="cv",
              n_splits=3, seed=s, verbose=0)
        acc.append((a.predict(Xte) == yte).mean())
    res[b] = acc
    print(f"orçamento {b:>2d} s : acurácia no teste = {np.mean(acc):.3f} (divisões: {acc[0]:.3f},
{acc[1]:.3f})")
json.dump({str(k): v for k, v in res.items()},
open("/home/user/claude/automl/build/f10_orcamento.json", "w"))
```

SAÍDA

```
orçamento  2 s : acurácia no teste = 0.956 (divisões: 0.940, 0.971)
orçamento  5 s : acurácia no teste = 0.937 (divisões: 0.929, 0.944)
orçamento 10 s : acurácia no teste = 0.970 (divisões: 0.969, 0.971)
orçamento 20 s : acurácia no teste = 0.970 (divisões: 0.976, 0.964)
```

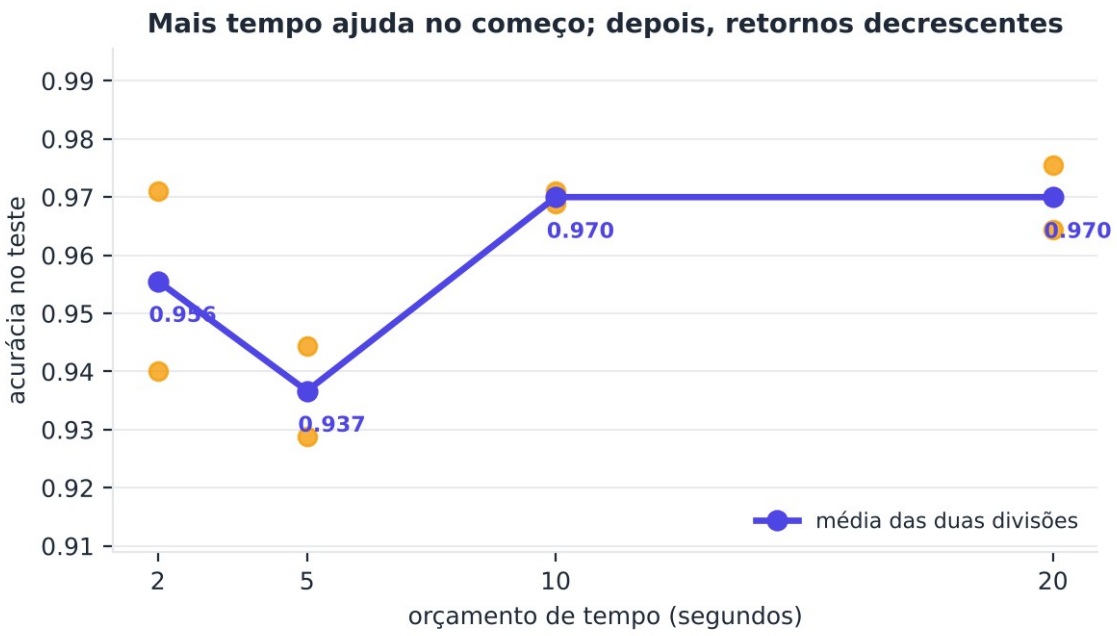


Figura 10.4 — Efeito do orçamento de tempo sobre a acurácia no teste (Digits, duas divisões dos dados).

Os números reais desta execução pedem cautela: a média sobe de 0,956 (2 s) para 0,970 (10 s e 20 s), mas **cai a 0,937 em 5 s**. Esse "buraco" não é uma lei da natureza: com poucas tentativas, a busca pode estacionar num algoritmo ruim por azar (veja mais adiante que, no Digits, o FLAML chegou a escolher modelos lineares). Observe a forma da curva e o tamanho das variações entre as duas divisões. Com apenas duas divisões, **não devemos extrair conclusões finas**: o gráfico mostra a tendência, não uma lei. Se a diferença entre 10 e 20 segundos cabe dentro da variação entre as divisões, o tempo extra não comprou nada que possamos medir.

10.4 As outras ferramentas: o código de uso

Os códigos abaixo mostram como cada ferramenta é usada. Mas há um aviso importante.

> **Atenção: código não executado**
>
> Nenhum dos trechos desta seção foi **executado** para a elaboração deste livro: essas bibliotecas não puderam ser instaladas no ambiente de preparação (algumas exigem compiladores, outras têm dependências pesadas ou restrições de sistema). O código segue a documentação oficial, mas **as versões mudam**. Se algo falhar, consulte a documentação da versão que você instalou. Os **resultados** de cada ferramenta, portanto, **não são reportados aqui**.

10.4.1 Auto-sklearn

O Auto-sklearn funciona em Linux (e depende de compilação de pacotes). A interface imita a do scikit-learn:

```python
import autosklearn.classification
from sklearn.metrics import accuracy_score

automl = autosklearn.classification.AutoSklearnClassifier(
    time_left_for_this_task=300,      # orçamento total (segundos)
    per_run_time_limit=30,            # limite por modelo
    seed=0,
)
automl.fit(Xtr, ytr)
print(automl.leaderboard())           # tabela dos modelos do ensemble
print(accuracy_score(yte, automl.predict(Xte)))
```

10.4.2 TPOT

O TPOT evolui *pipelines* e pode **exportar o código Python** do melhor deles, o que é ótimo para estudar e auditar:

```python
from tpot import TPOTClassifier

tpot = TPOTClassifier(generations=5, population_size=20, cv=3,
                      random_state=0, verbosity=2)
tpot.fit(Xtr, ytr)
print(tpot.score(Xte, yte))
tpot.export("melhor_pipeline.py")     # gera o código do pipeline encontrado
```

10.4.3 H2O AutoML

O H2O roda sobre uma máquina virtual Java e usa quadros de dados próprios (`H2OFrame`):

```python
import h2o
from h2o.automl import H2OAutoML

h2o.init()
treino = h2o.H2OFrame(df_treino)       # df_treino: DataFrame com a coluna-alvo "y"
treino["y"] = treino["y"].asfactor()
aml = H2OAutoML(max_runtime_secs=300, seed=1)
aml.train(y="y", training_frame=treino)
print(aml.leaderboard)                 # ranking dos modelos
```

10.4.4 AutoGluon

O AutoGluon-Tabular é famoso pelo uso mínimo de código e por combinar muitos modelos em camadas:

```python
from autogluon.tabular import TabularPredictor

predictor = TabularPredictor(label="y").fit(df_treino, time_limit=300)
print(predictor.leaderboard(df_teste))
predicoes = predictor.predict(df_teste)
```

> **Dica: o mesmo contrato**
>
> Veja como as quatro interfaces são parecidas: você informa **dados** e **orçamento**, a ferramenta devolve uma **tabela de modelos** (*leaderboard*) e um preditor. Aprender uma ferramenta ajuda a aprender as demais.

10.5 Montando uma comparação justa

É tentador rodar duas ferramentas uma única vez e declarar um vencedor. Não faça isso. Uma comparação justa precisa de **cuidados** que já conhecemos:

1. **Os mesmos dados e as mesmas divisões** para todos os métodos.
2. **O mesmo orçamento** (tempo ou número de tentativas).
3. **Um teste final intocado**, usado só uma vez por método.
4. **Várias divisões** (ou várias sementes) para ver a variação.
5. **Várias bases de dados**, porque um método pode ganhar numa base e perder em outra.
6. **Linhas de base simples**: se o AutoML não bate o "modelo padrão", ele não se justifica.



Figura 10.5 — Seis cuidados para uma comparação justa.

Esses cuidados aparecem também em *benchmarks* de pesquisa, como o AMLB, que compara ferramentas de AutoML em dezenas de tarefas, com divisões e orçamentos controlados (Gijsbers *et al.*, 2024), e no estudo de Zöller e Huber (2021), que compara estruturas e os sistemas mais usados.

Vamos fazer uma **versão em miniatura** desse experimento. Compararemos quatro métodos em **quatro bases**, com **três divisões** cada:

Método	O que faz
Floresta padrão	`RandomForestClassifier` com 200 árvores, sem ajuste
Boosting padrão	`HistGradientBoostingClassifier` sem ajuste
Busca aleatória	10 sorteios de hiperparâmetros do boosting, por validação cruzada
FLAML	AutoML com orçamento de 10 segundos

As bases são **Wine** (UCI, 178 exemplos), **Breast Cancer** (UCI, 569), **Digits** (UCI, 1.797) e uma base **simulada ruidosa** (1.500 exemplos, 30 atributos, 10% de rótulos trocados), que imita os dados "sujos" do mundo real.

```python
import warnings; warnings.filterwarnings("ignore")
import json
import numpy as np
from flaml import AutoML
from scipy.stats import loguniform, randint
from sklearn.datasets import load_wine, load_breast_cancer, load_digits, make_classification
from sklearn.model_selection import train_test_split, RandomizedSearchCV
from sklearn.ensemble import RandomForestClassifier, HistGradientBoostingClassifier

bases = {
    "Wine": load_wine(return_X_y=True),
    "Cancer": load_breast_cancer(return_X_y=True),
    "Digits": load_digits(return_X_y=True),
    "Ruidosa": make_classification(1500, 30, n_informative=8, n_redundant=4, flip_y=0.10,
random_state=0),
}
metodos = ["Floresta padrão", "Boosting padrão", "Busca aleatória", "FLAML (10 s)"]
espaco = {"learning_rate": loguniform(0.01, 0.5), "max_leaf_nodes": randint(4, 40),
          "min_samples_leaf": randint(5, 60), "l2_regularization": loguniform(1e-4, 10)}

res = {b: {m: [] for m in metodos} for b in bases}
escolhido = {b: [] for b in bases}
for b, (X, y) in bases.items():
    for s in (0, 1, 2):
        Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.25, stratify=y, random_state=s)
        m1 = RandomForestClassifier(200, random_state=s, n_jobs=-1).fit(Xtr, ytr)
        m2 = HistGradientBoostingClassifier(random_state=s).fit(Xtr, ytr)
        m3 = RandomizedSearchCV(HistGradientBoostingClassifier(random_state=s, max_iter=60), espaco,
                                n_iter=10, cv=3, random_state=s, n_jobs=-1).fit(Xtr, ytr)
        a = AutoML()
        a.fit(Xtr, ytr, task="classification", time_budget=10, metric="accuracy", eval_method="cv",
              n_splits=3, seed=s, verbose=0)
        for nome, mdl in zip(metodos, (m1, m2, m3, a)):
            res[b][nome].append(float((mdl.predict(Xte) == yte).mean()))
        escolhido[b].append(a.best_estimator)
    print(f"{b:<8s}", "  ".join(f"{np.mean(res[b][m]):.3f}" for m in metodos), "| FLAML escolheu:", ",
".join(escolhido[b]))
print("\nColunas:", " | ".join(metodos))
json.dump({"res": res, "metodos": metodos, "escolhido": escolhido},
open("/home/user/claude/automl/build/f10_cache.json", "w"))
```

SAÍDA

```
Wine     0.993  1.000  0.993  1.000 | FLAML escolheu: xgboost, xgboost, lgbm
Cancer   0.951  0.949  0.958  0.953 | FLAML escolheu: xgb_limitdepth, lgbm, xgboost
Digits   0.978  0.971  0.964  0.965 | FLAML escolheu: sgd, xgboost, lrl1
Ruidosa  0.880  0.872  0.862  0.879 | FLAML escolheu: xgboost, lgbm, lgbm

Colunas: Floresta padrão | Boosting padrão | Busca aleatória | FLAML (10 s)
```

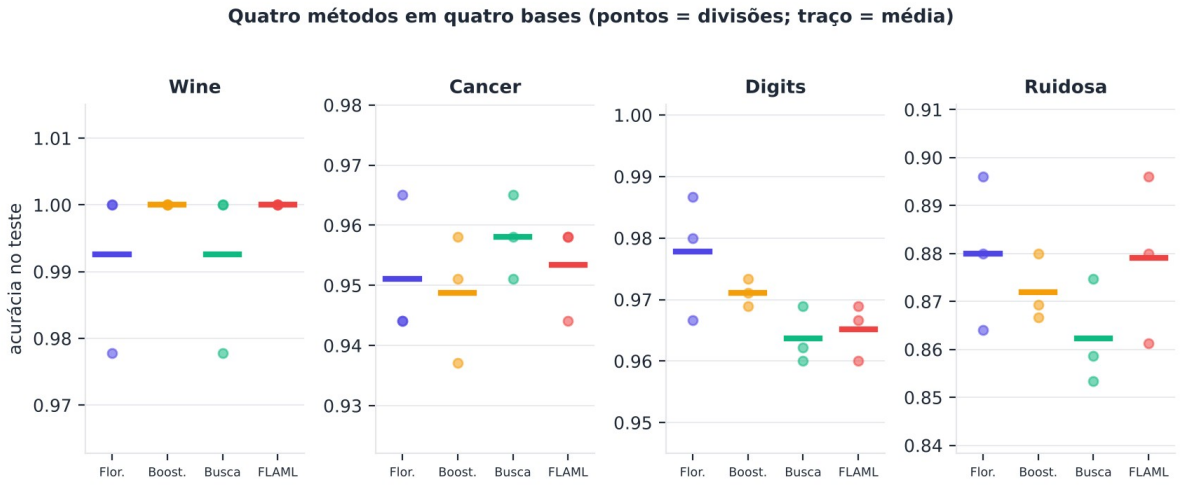


Figura 10.6 — Acurácia média no teste (três divisões) de quatro métodos em quatro bases. Cada ponto é uma divisão; a barra é a média.

Os números desta execução contam uma história **modesta**, e é bom que seja assim: em nenhuma base o FLAML foi claramente superior às linhas de base. Na **Wine**, ele empata no topo com o boosting padrão (1,000 contra 0,993 da floresta); no **Cancer**, a busca aleatória lidera por pouco (0,958); no **Digits**, a floresta padrão é a melhor (0,978) e o FLAML, com 10 segundos, escolheu em duas divisões modelos como `sgd` e `lrl1`, bem menos adequados a imagens do que as árvores; na base **ruidosa**, a floresta (0,880) e o FLAML (0,879) praticamente empatam. A **busca aleatória** com apenas 10 sorteios e 60 iterações de boosting ficou, na média, atrás do boosting padrão em três das quatro bases: ajustar sem orçamento suficiente pode até **piorar** um bom padrão.

Antes de interpretar, lembre-se do tamanho do **teste**: na base Wine, 25% de 178 exemplos são apenas 45 exemplos; **um único exemplo** vale mais de 2 pontos percentuais. Diferenças pequenas ficam dentro do ruído. O gráfico mostra cada divisão como um ponto justamente para que você veja essa dispersão.

10.6 Comparando métodos em várias bases: postos e o teste de Friedman

Como resumir a comparação em **várias bases** sem ser enganado por uma base em que as notas são todas altas? Demšar (2006) recomenda, para comparar vários métodos em várias bases, trabalhar com **postos** (*ranks*): em cada base, ordenamos os métodos do melhor (posto 1) ao pior, e depois calculamos o **posto médio** de cada método.

NO SEU DIA A DIA **Analogia: o campeonato de pontos corridos**

Em um campeonato, o título não vai para quem fez o gol mais bonito, mas para quem teve a **melhor posição média** ao longo das rodadas. Cada base de dados é uma "rodada", e cada método é um "time".

Para saber se as diferenças de posto médio são grandes o bastante para não serem acaso, usa-se o **teste de Friedman**: ele testa a hipótese de que **todos os métodos são equivalentes**. Se a hipótese é rejeitada, comparam-se os pares com um teste posterior (como o de Nemenyi) (Demšar, 2006).

PYTHON

```python
import json
import numpy as np
from scipy.stats import friedmanchisquare, rankdata

d = json.load(open("/home/user/claude/automl/build/f10_cache.json"))
metodos, res = d["metodos"], d["res"]
bases = list(res)
M = np.array([[np.mean(res[b][m]) for m in metodos] for b in bases])     # bases x métodos

print("Acurácia média por base e método:")
print(" " * 9 + "  ".join(f"{m[:11]:>11s}" for m in metodos))
for b, lin in zip(bases, M):
    print(f"{b:<9s}" + "  ".join(f"{v:11.3f}" for v in lin))

postos = np.array([rankdata(-lin) for lin in M])     # posto 1 = maior acurácia; empates recebem a média
print("\nPosto médio (1 = melhor):")
for m, p in zip(metodos, postos.mean(0)): print(f"  {m:<16s} {p:.2f}")

est, p = friedmanchisquare(*[M[:, j] for j in range(M.shape[1])])
print(f"\nTeste de Friedman: estatística = {est:.2f}, valor-p = {p:.3f}  (k = {M.shape[1]} métodos, N = {M.shape[0]} bases)")
d["postos"] = postos.mean(0).tolist(); d["friedman_p"] = float(p)
json.dump(d, open("/home/user/claude/automl/build/f10_cache.json", "w"))
```

SAÍDA

```
Acurácia média por base e método:
          Floresta pa  Boosting pa  Busca aleat  FLAML (10 s
Wine            0.993        1.000        0.993        1.000
Cancer          0.951        0.949        0.958        0.953
Digits          0.978        0.971        0.964        0.965
Ruidosa         0.880        0.872        0.862        0.879

Posto médio (1 = melhor):
  Floresta padrão  2.12
  Boosting padrão  2.62
  Busca aleatória  3.12
  FLAML (10 s)     2.12

Teste de Friedman: estatística = 1.74, valor-p = 0.629  (k = 4 métodos, N = 4 bases)
```

No nosso caso, o posto médio da floresta e o do FLAML é o mesmo (2,12), o do boosting é 2,62 e o da busca aleatória é 3,12; o valor-p do Friedman (0,629) está longe de rejeitar a equivalência. Leia o resultado com humildade. Com apenas **quatro bases**, o teste de Friedman tem **pouquíssimo poder**: mesmo que um método seja melhor, é difícil rejeitar a hipótese de equivalência com tão poucas "rodadas". Demšar (2006) já alertava para a necessidade de um número razoável de bases. Pesquisas sérias usam dezenas (Gijsbers *et al.*, 2024). O que fizemos aqui é um **exercício de método**, não uma prova de superioridade.

> **Atenção: três divisões não são três bases**
>
> Usamos três divisões de cada base para estabilizar a nota **de cada base**. Elas **não** contam como três bases independentes: são recortes dos mesmos dados. Por isso o Friedman foi calculado com a média por base (N = 4), e não com 12 pontos.

10.7 O que as comparações da literatura nos ensinam

Sem inventar números, vale resumir o que a literatura de comparação indica, com as devidas cautelas:

1. **Não existe vencedor universal.** Os estudos comparativos mostram que o desempenho relativo depende da base, da tarefa e do orçamento (Gijsbers *et al.*, 2024; Zöller; Huber, 2021).
2. **O orçamento pesa.** Algumas ferramentas se destacam com orçamentos curtos; outras, com mais tempo.
3. **Ensembles ajudam.** Os sistemas que combinam muitos modelos (como AutoGluon e H2O) se beneficiam da diversidade (Erickson *et al.*, 2020; LeDell; Poirier, 2020), com o custo de modelos maiores e previsões mais lentas.
4. **Árvores de decisão em *boosting* dominam tabelas.** Em dados tabulares típicos, modelos baseados em árvores seguem muito competitivos frente às redes neurais (Grinsztajn; Oyallon; Varoquaux, 2022).
5. **Cuidado com a leitura de resultados de propaganda.** Compare sempre em bases e orçamentos equivalentes aos **seus**.

Há também novidades que mudam o cenário. Um exemplo é o TabPFN, um modelo fundacional (pré-treinado) para dados tabulares pequenos, que faz previsões sem busca de hiperparâmetros por conjunto e é descrito em publicação de alto impacto (Hollmann *et al.*, 2025). Ele mostra que o futuro do AutoML pode incluir **modelos pré-treinados** que substituem parte da busca.

10.8 Escolhendo uma ferramenta

Não há receita única, mas um roteiro de perguntas ajuda.

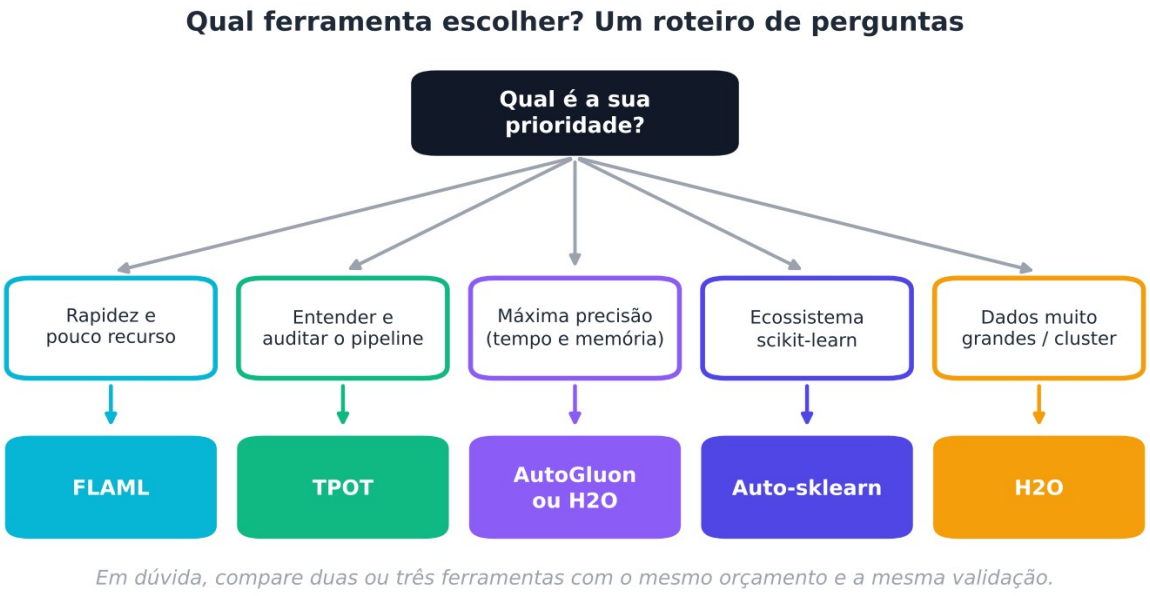


Figura 10.7 — Um roteiro de perguntas para escolher entre ferramentas de AutoML.

Resumindo as ideias do roteiro:

- **Pouco tempo ou poucos recursos?** Prefira uma ferramenta leve, como o FLAML (Wang *et al.*, 2021).
- **Quer entender e auditar o *pipeline*?** O TPOT exporta o código do melhor *pipeline* (Olson; Moore, 2016).
- **Quer a máxima precisão e tem tempo e memória?** Ferramentas com ensembles empilhados, como AutoGluon e H2O, são boas candidatas (Erickson *et al.*, 2020; LeDell; Poirier, 2020).
- **Trabalha no ecossistema scikit-learn e quer meta-aprendizado?** Auto-sklearn (Feurer *et al.*, 2015; 2022).

- **Tem dados muito grandes ou um *cluster*?** O H2O foi pensado para plataformas distribuídas (LeDell; Poirier, 2020).

10.9 Boas práticas

1. **Comece por uma linha de base** (modelo padrão) para saber quanto o AutoML realmente ganha.
2. **Fixe e registre** semente, orçamento, métrica e versões das bibliotecas.
3. **Dê o mesmo orçamento** a todos os métodos que for comparar.
4. **Use um teste final intocado** e olhe-o uma única vez por método.
5. **Repita com várias divisões** e mostre a variação, não só a média.
6. **Inspecione o resultado:** qual algoritmo venceu? A configuração faz sentido? Há ensemble? Qual o tamanho do modelo final?
7. **Pense na produção:** um ensemble com dezenas de modelos pode ser lento e caro para servir (Capítulo 14).
8. **Desconfie de ganhos pequenos:** diferenças dentro da variação entre divisões não são evidência.

Resumo do capítulo

MEMORIZE **Cartões de memorização**

- **Ferramentas:** Auto-sklearn (meta-aprendizado + bayesiana + ensemble), TPOT (evolução de *pipelines*), H2O e AutoGluon (muitos modelos + ensembles empilhados), FLAML (busca de baixo custo).
- **Contrato comum:** dados + orçamento entram; modelo (ou ensemble) e *leaderboard* saem.
- **FLAML:** `AutoML().fit(X, y, task=..., time_budget=..., metric=...)`; inspecione `best_estimator`, `best_config` e `config_history`.
- **Orçamento de tempo:** mais tempo ajuda no início, depois os retornos diminuem; e o tempo de relógio prejudica a reprodutibilidade exata.
- **Comparação justa:** mesmas divisões, mesmo orçamento, teste intocado, várias divisões, várias bases e linhas de base.
- **Postos e Friedman:** compare métodos em várias bases pelo posto médio; poucas bases dão pouco poder.
- **Sem vencedor universal:** escolha pela tarefa, pelo tempo e pelos recursos.

Exercícios de múltipla escolha

Escolha **uma** alternativa em cada questão. O gabarito está no Apêndice A.

Questão 1. Qual é o "contrato" comum à maioria das ferramentas de AutoML para dados tabulares?

a) Entram dados e um orçamento; saem um modelo (ou ensemble) pronto e, em geral, uma tabela de modelos.
b) Entram imagens; saem textos.
c) Entram modelos prontos; saem novos dados sintéticos.
d) Entram apenas hiperparâmetros; saem os dados limpos.

Questão 2. Qual é a ideia central do TPOT?

a) Treinar redes neurais profundas sem validação.
b) Substituir o cientista de dados por regras fixas.
c) Evoluir pipelines inteiros por programação genética.
d) Usar apenas regressão linear.

Questão 3. O que o Auto-sklearn combina, segundo o capítulo?

a) Apenas seleção de atributos.
b) Apenas busca em grade.
c) Apenas redes convolucionais.
d) Meta-aprendizado, otimização bayesiana e construção de ensembles.

Questão 4. Qual característica o artigo do FLAML destaca?

a) Dispensar qualquer métrica.
b) Uma busca de baixo custo, leve e rápida.
c) Exigir sempre um cluster de servidores.
d) Funcionar apenas com imagens.

Questão 5. Na analogia do garimpeiro prudente, o que corresponde às "peneiradas rápidas"?

a) Remover o conjunto de teste.
b) Embaralhar os rótulos.
c) Treinar o maior modelo possível de início.
d) Testar modelos pequenos e baratos antes dos caros.

Questão 6. Na chamada do FLAML, para que serve o parâmetro `time_budget`?

a) Definir o número de árvores da floresta.
b) Definir a fração do teste.
c) Definir o tempo máximo de busca, em segundos.
d) Definir a quantidade de classes.

Questão 7. O que contém o atributo `best_estimator` de um objeto FLAML ajustado?

a) O conjunto de teste.
b) A matriz de dados.
c) O número de linhas do treino.
d) O nome do algoritmo vencedor.

Questão 8. O que guarda o atributo `config_history`?

a) A lista de bibliotecas instaladas.
b) O histórico das configurações que melhoraram o melhor resultado durante a busca.
c) Os rótulos previstos.
d) O gabarito do teste.

Questão 9. Por que fixar a semente (`seed`) não garante resultados idênticos em uma busca com orçamento de tempo?

a) Porque o FLAML ignora a semente sempre.
b) Porque o tempo de relógio muda a quantidade de tentativas concluídas.
c) Porque a semente só funciona em imagens.
d) Porque o teste muda sozinho.

Questão 10. Para obter resultados mais reprodutíveis, o que se recomenda além de fixar a semente?

a) Limitar o número de tentativas (`max_iter`) e registrar as versões das bibliotecas.
b) Trocar o conjunto de teste a cada execução.
c) Desligar a validação.
d) Usar um orçamento aleatório.

Questão 11. O que faz o parâmetro `estimator_list` no FLAML?

a) Escolhe a semente.
b) Define o número de divisões do teste.
c) Remove atributos.
d) Restringe os algoritmos que a busca pode usar.

Questão 12. O que o `best_loss_per_estimator` permite observar?

a) O melhor erro de validação obtido para cada algoritmo testado.
b) A acurácia no teste de cada algoritmo.
c) O tempo de relógio total.
d) Os rótulos de treino.

Questão 13. Por que o gráfico de orçamento (Digits, duas divisões) não deve ser lido como uma lei?

a) Porque duas divisões dão pouca informação sobre a variação.
b) Porque Digits não tem rótulos.
c) Porque o tempo é medido em dias.
d) Porque o FLAML não aceita imagens.

Questão 14. Qual destes NÃO é um cuidado para uma comparação justa?

a) Usar as mesmas divisões dos dados.
b) Usar um teste final intocado.
c) Dar o mesmo orçamento a todos os métodos.
d) Dar mais tempo ao método que você quer que vença.

Questão 15. Por que incluir uma linha de base simples (modelo padrão) na comparação?

a) Para esconder a variação.
b) Para reduzir o tamanho do conjunto de dados.
c) Para evitar o uso de validação.
d) Para saber se o AutoML realmente ganha algo sobre o modelo sem ajuste.

Questão 16. Por que a comparação do capítulo usa várias divisões dos dados?

a) Para treinar com o teste.
b) Para observar a variação do resultado entre divisões.
c) Para aumentar artificialmente a acurácia.
d) Para dispensar o teste final.

Questão 17. Na base Wine, 25% de 178 exemplos formam cerca de 45 exemplos de teste. O que isso implica?

a) Não é possível avaliar nada.
b) A acurácia é sempre exata.
c) Um único exemplo vale mais de 2 pontos percentuais, então diferenças pequenas podem ser ruído.
d) O teste é grande demais.

Questão 18. O que são "postos" (ranks) na comparação entre métodos?

a) A quantidade de classes.
b) O número de linhas das bases.
c) A posição de cada método em cada base, do melhor (1) ao pior.
d) O tempo gasto por cada método.

Questão 19. Qual hipótese o teste de Friedman testa?

a) Que todos os métodos têm acurácia 100%.
b) Que o teste é independente do treino.
c) Que as bases são iguais.
d) Que todos os métodos comparados são equivalentes.

Questão 20. Que teste posterior é mencionado depois do Friedman?

a) O de Nemenyi.
b) O de Pareto.
c) O de Rice.
d) O de Fibonacci.

Questão 21. Por que o teste de Friedman com apenas quatro bases tem pouco poder?

a) Porque os postos não existem para quatro bases.
b) Porque poucas bases dificultam rejeitar a hipótese de equivalência, mesmo havendo diferença real.
c) Porque quatro é um número proibido.
d) Porque ele só funciona com imagens.

Questão 22. Por que as três divisões de cada base não contam como três bases independentes?

a) Porque usam outro algoritmo.
b) Porque cada divisão tem rótulos diferentes.
c) Porque são recortes dos mesmos dados.
d) Porque têm tamanhos diferentes.

Questão 23. Qual é a lição do artigo de comparação AMLB (Gijsbers et al., 2024)?

a) Não registre o orçamento.
b) Use apenas uma base de dados.
c) Compare ferramentas em muitas tarefas, com divisões e orçamentos controlados.
d) Compare sem teste final.

Questão 24. Qual é a conclusão geral dos estudos comparativos citados no capítulo?

a) Não existe um vencedor universal: o desempenho depende da base, da tarefa e do orçamento.
b) Uma ferramenta vence sempre.
c) Só redes neurais funcionam em tabelas.
d) O AutoML nunca ajuda.

Questão 25. O que Grinsztajn, Oyallon e Varoquaux (2022) investigam?

a) Por que modelos baseados em árvores seguem competitivos frente ao aprendizado profundo em dados tabulares típicos.
b) Como colorir gráficos.
c) Como compactar imagens.
d) Como traduzir textos.

Questão 26. O que é o TabPFN, citado no capítulo?

a) Uma técnica de poda.
b) Um algoritmo de busca em grade.
c) Um modelo fundacional pré-treinado para dados tabulares pequenos, que dispensa busca de hiperparâmetros por conjunto.
d) Uma métrica de erro.

Questão 27. Qual ferramenta é mais indicada, segundo o roteiro do capítulo, para quem quer auditar o código do pipeline encontrado?

a) O TPOT, que pode exportar o código do melhor pipeline.
b) O TabPFN.
c) Apenas a busca em grade.
d) Nenhuma ferramenta permite isso.

Questão 28. Para pouco tempo ou poucos recursos, o capítulo sugere:

a) Evitar a validação.
b) Treinar uma rede gigante.
c) Um ensemble empilhado com dezenas de modelos.
d) Uma ferramenta leve, como o FLAML.

Questão 29. Qual é um custo de ensembles empilhados como os do AutoGluon e do H2O, segundo o capítulo?

a) Redução da diversidade.
b) Impossibilidade de usar validação.
c) Perda do conjunto de teste.
d) Modelos maiores e previsões mais lentas.

Questão 30. Por que os trechos de Auto-sklearn, TPOT, H2O e AutoGluon do capítulo não trazem resultados?

a) Porque o código não foi executado no ambiente de preparação do livro.
b) Porque os resultados são secretos.
c) Porque não existem leaderboards.
d) Porque essas ferramentas não funcionam.

Exercícios discursivos

Responda com suas palavras, em poucas linhas. Compare depois com a resposta-modelo do Apêndice A.

1. Descreva o "contrato" comum das ferramentas de AutoML para dados tabulares.
2. Explique a analogia do garimpeiro prudente e relacione-a ao FLAML.
3. Compare as ideias centrais de Auto-sklearn, TPOT, H2O AutoML, AutoGluon e FLAML.
4. Cite quatro informações que o objeto do FLAML guarda após o ajuste e explique para que cada uma serve.
5. Explique por que um orçamento de tempo prejudica a reprodutibilidade exata e como mitigar.
6. O que significa dizer que o FLAML resolve o problema CASH?
7. Descreva a curva "orçamento versus acurácia" e explique por que duas divisões não bastam para conclusões finas.
8. Liste os seis cuidados para uma comparação justa entre ferramentas.
9. Por que a linha de base é indispensável em uma avaliação de AutoML?
10. Explique o que é um posto médio e por que ele é usado para comparar métodos em várias bases.
11. O que o teste de Friedman testa e o que fazer se a hipótese for rejeitada?
12. Por que quatro bases são insuficientes para uma conclusão forte com o Friedman?
13. Explique por que três divisões de uma mesma base não equivalem a três bases.
14. Qual é o risco de interpretar diferenças pequenas na base Wine?
15. Resuma o que os estudos comparativos de AutoML indicam sobre a existência de uma ferramenta "melhor".
16. Por que modelos de árvores com boosting são tão usados em dados tabulares, segundo a literatura citada?
17. O que é o TabPFN e o que ele sugere sobre o futuro do AutoML?
18. Como escolher entre FLAML, TPOT, AutoGluon/H2O e Auto-sklearn segundo o roteiro do capítulo?
19. Por que os trechos de código de Auto-sklearn, TPOT, H2O e AutoGluon vêm com um aviso de "não executado"?
20. Liste cinco boas práticas ao usar uma ferramenta de AutoML.

Exercícios práticos em Python (scikit-learn)

Resolva os exercícios no seu computador (ou no Google Colab). Eles usam conjuntos de dados da UCI que já vêm no scikit-learn ou dados simulados. Tente resolver **antes** de consultar a solução, no Apêndice D. As saídas apresentadas lá foram obtidas com as versões de bibliotecas listadas na Apresentação.

Exercício 1 — Primeiro contato com o FLAML

NÍVEL: BÁSICO

No conjunto **Wine** (UCI, via scikit-learn), separe 25% para teste (estratificado, `random_state=0`) e ajuste um `AutoML` do FLAML com `estimator_list=["lgbm", "rf"]`, `time_budget=5`, métrica acurácia e validação cruzada de 3 partes. Mostre o algoritmo vencedor, o erro de validação e a acurácia no teste.

Pergunta: o algoritmo vencedor era o que você esperava? Por que a acurácia do teste pode diferir de 1 − erro de validação?

Exercício 2 — Tempo ou tentativas?

NÍVEL: INTERMEDIÁRIO

Ajuste o FLAML duas vezes no **Wine**, com os mesmos dados e a mesma semente: (a) com `time_budget=3` e (b) com `max_iter=12` (e `time_budget=60` apenas como limite de segurança). Em cada caso, conte quantas melhorias foram registradas em `config_history` e compare as duas execuções.

Pergunta: qual forma de limitar a busca é mais reprodutível? Por quê?

Exercício 3 — O AutoML bate a linha de base?

NÍVEL: INTERMEDIÁRIO

Gere uma base ruidosa com `make_classification(1500, 30, n_informative=8, n_redundant=4, flip_y=0.10, random_state=0)`. Em cada uma de 5 divisões (`random_state` de 0 a 4, 25% de teste), compare a acurácia no teste de: regressão logística (com `StandardScaler`), floresta aleatória padrão (200 árvores) e FLAML (`time_budget=5`). Mostre média e desvio-padrão de cada método.

Pergunta: o FLAML superou as linhas de base? A diferença é maior que a variação entre as divisões?

Exercício 4 — Postos e Friedman à mão

NÍVEL: AVANÇADO

Use cinco bases: **Iris**, **Wine**, **Breast Cancer**, **Digits** (todas do scikit-learn) e a base ruidosa do exercício anterior. Para cada base, calcule a acurácia média em validação cruzada de 5 partes de três métodos: regressão logística (com `StandardScaler`), floresta aleatória (100 árvores) e *k*-vizinhos (*k* = 5, com `StandardScaler`). Converta em postos (1 = melhor), calcule o posto médio de cada método e aplique `scipy.stats.friedmanchisquare`.

Pergunta: o teste rejeita a equivalência dos métodos ao nível de 5%? Com 5 bases, o que esperar do poder do teste?

> SAIBA MAIS **Para ir além: bases públicas da UCI e do Kaggle**
>
> - **UCI:** Wine Quality, Spambase, Adult e Bank Marketing (muitos exemplos, bom para testar orçamentos).
> - **Kaggle:** Titanic, House Prices e Spaceship Titanic (para comparar ferramentas em problemas conhecidos).
> - **Desafio:** instale o Auto-sklearn, o TPOT ou o AutoGluon em um ambiente próprio e inclua-os no duelo, mantendo o mesmo orçamento.
>
> Estes conjuntos não foram usados nem testados nos códigos do livro. Confira, nos sites, a descrição, a licença e as colunas antes de usá-los.

Projeto integrador do capítulo

Duelo de AutoML em miniatura

Fonte dos dados. Quatro bases do repositório UCI disponíveis no scikit-learn (*Iris*, *Wine*, *Breast Cancer Wisconsin* e *Digits*) e, se quiser ampliar, bases públicas do Kaggle ou da UCI baixadas por você (por exemplo, *Wine Quality* e *Spambase*).

Contexto. A equipe de dados de uma clínica quer saber se vale a pena adotar uma ferramenta de AutoML ou se um modelo padrão basta. Você fará um *benchmark* em miniatura, com todos os cuidados de uma comparação justa.

Atenção ética. Os dados são de uso didático. Nenhum dos modelos deve ser usado para decisões clínicas.

Tarefas.

1. Defina os métodos: (i) floresta aleatória padrão; (ii) `HistGradientBoostingClassifier` padrão; (iii) FLAML com **no máximo 20 tentativas** (`max_iter=20`, para ser reprodutível).
2. Para cada base, faça 3 divisões estratificadas (25% de teste, `random_state` 0, 1, 2) e registre a acurácia no teste de cada método.
3. Monte a tabela de médias por base e calcule os postos médios.
4. Aplique o teste de Friedman (N = número de bases) e comente o poder do teste.
5. Para uma base à escolha, mostre qual algoritmo o FLAML escolheu em cada divisão e quantas configurações avaliou.
6. Escreva uma conclusão (8 a 10 linhas) sobre se o AutoML se justificou, respeitando o tamanho dos testes e a variação entre divisões.

Entregáveis. Um notebook (ou script) comentado, a tabela de acurácias e postos, o resultado do Friedman e a conclusão.

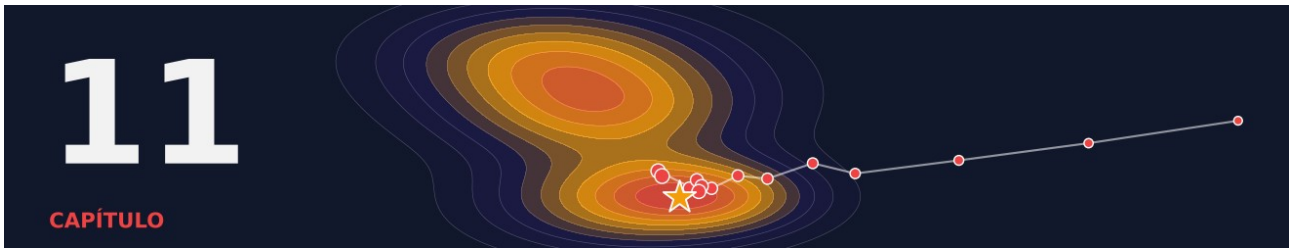
A solução de referência está no Apêndice E.

PARTE IV

Parte IV — Redes Neurais e Novos Domínios

Nesta parte:

- Cap. 11 — Busca de arquitetura neural (NAS)
- Cap. 12 — AutoML para séries temporais, texto e imagens



Capítulo 11 — Busca de arquitetura neural (NAS)

OBJETIVOS DE APRENDIZAGEM

Ao final deste capítulo você será capaz de:

- Explicar o que é uma arquitetura de rede neural e por que escolhê-la é um problema de busca.
- Descrever os três pilares do NAS: espaço de busca, estratégia de busca e estimativa de desempenho.
- Contar o tamanho de um espaço de arquiteturas e explorá-lo de forma exaustiva em um exemplo pequeno.
- Comparar busca aleatória e evolução regularizada em um mini-NAS reproduzível.
- Reconhecer as estratégias de aprendizado por reforço, evolução, gradiente e compartilhamento de pesos.
- Discutir custo, estimativas baratas de desempenho e busca com mais de um objetivo.

11.1 Quem desenha a rede?

Nos capítulos anteriores, ajustamos **hiperparâmetros** de modelos clássicos. Agora vamos olhar para as **redes neurais**, onde existe uma decisão ainda mais profunda: **qual é o desenho da rede?**

Uma **rede neural** é um modelo formado por **camadas** de pequenas unidades de cálculo (os **neurônios**). Cada camada recebe números da anterior, combina-os e passa o resultado adiante. A **arquitetura** é o desenho dessa rede: quantas camadas, quantos neurônios em cada uma, como elas se conectam e qual **função de ativação** (a "regra de disparo" dos neurônios) é usada.

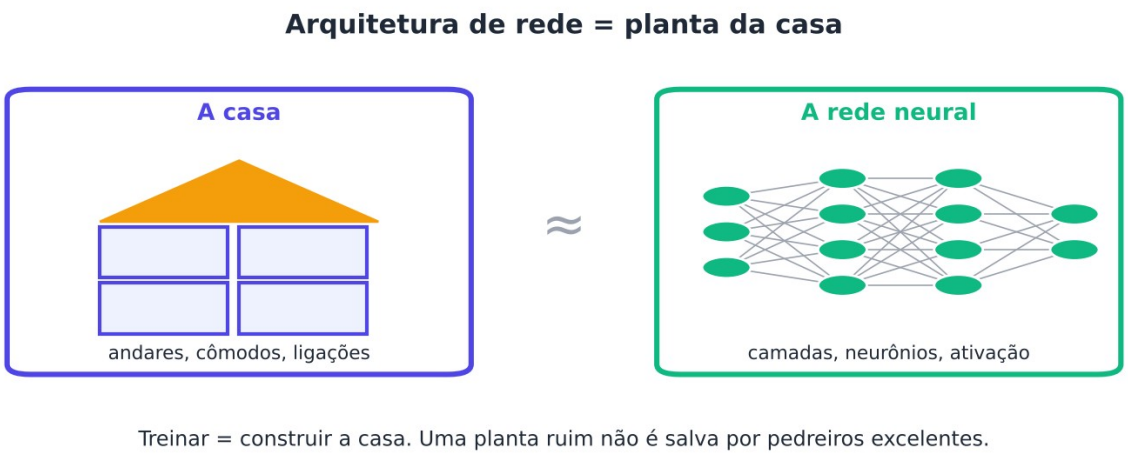


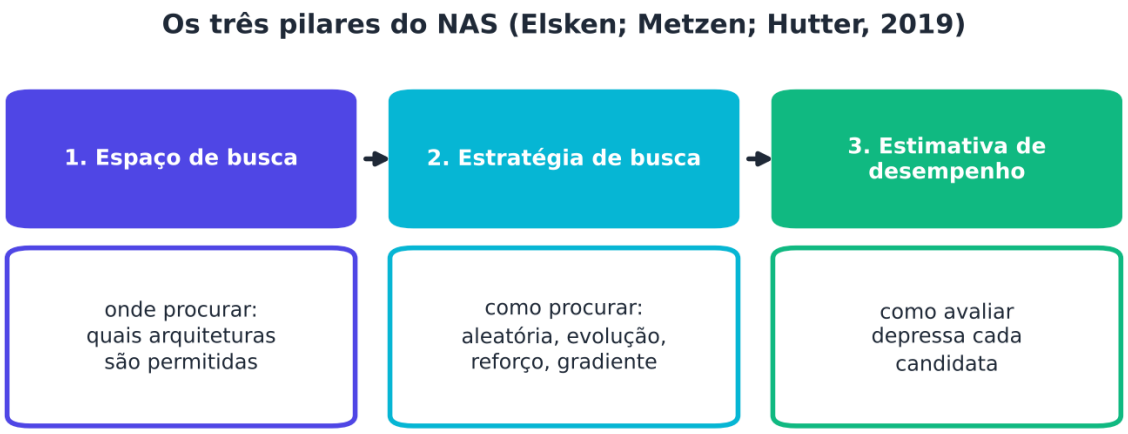
Figura 11.1 — Projetar uma rede é como projetar uma casa: muitas decisões de desenho, e cada uma muda o resultado.

> NO SEU DIA A DIA **Analogia: a planta da casa**
>
> Antes de construir uma casa, o arquiteto escolhe o número de andares, o tamanho dos cômodos e como eles se ligam. Depois, os pedreiros constroem. Em uma rede neural, o **treino** é a construção; a **arquitetura** é a planta. Uma planta ruim não é salva por pedreiros excelentes.

Durante anos, as arquiteturas foram desenhadas **à mão** por especialistas: pense nas redes residuais, que introduziram atalhos entre camadas e viabilizaram redes muito profundas (He *et al.*, 2016), ou nos Transformers, baseados em atenção (Vaswani *et al.*, 2017). A **busca de arquitetura neural** (*Neural Architecture Search*, NAS) propõe que **o computador** procure o desenho, dentro de limites que nós definimos (Elsken; Metzen; Hutter, 2019).

11.2 Os três pilares do NAS

Elsken, Metzen e Hutter (2019) organizam a literatura de NAS em **três componentes**, que usaremos como mapa do capítulo.



Mesma estrutura do AutoML clássico: espaço, estratégia e avaliação barata.

Figura 11.2 — Os três pilares do NAS: onde procurar, como procurar e como avaliar.

1. **Espaço de busca:** o conjunto de arquiteturas possíveis. É a "lista de plantas permitidas".
2. **Estratégia de busca:** a forma de explorar esse espaço (aleatória, evolutiva, por reforço, por gradiente).
3. **Estimativa de desempenho:** como saber, **depressa**, se uma arquitetura é boa. Treinar cada candidata até o fim é caríssimo.

Repare que isso é a mesma estrutura do AutoML clássico: um **espaço** (Capítulo 4), uma **estratégia** (Capítulos 4 a 7) e uma forma **barata de avaliar** (Capítulo 6). O NAS é o AutoML aplicado ao **desenho** da rede.

> **Dica: o NAS é um caso particular de HPO**
>
> Se você enxerga "número de camadas" e "neurônios por camada" como hiperparâmetros, uma parte do NAS já é busca de hiperparâmetros com um espaço **condicional** (a camada 3 só existe se a rede tiver 3 camadas). O que o NAS acrescenta é a escala e a **riqueza estrutural** do espaço (como as camadas se conectam).

11.3 O espaço de busca

O espaço de busca é a decisão mais importante: **ele define o teto** do que a busca pode achar. Se a melhor rede não está no espaço, nenhuma estratégia a encontrará. Há três tipos principais.

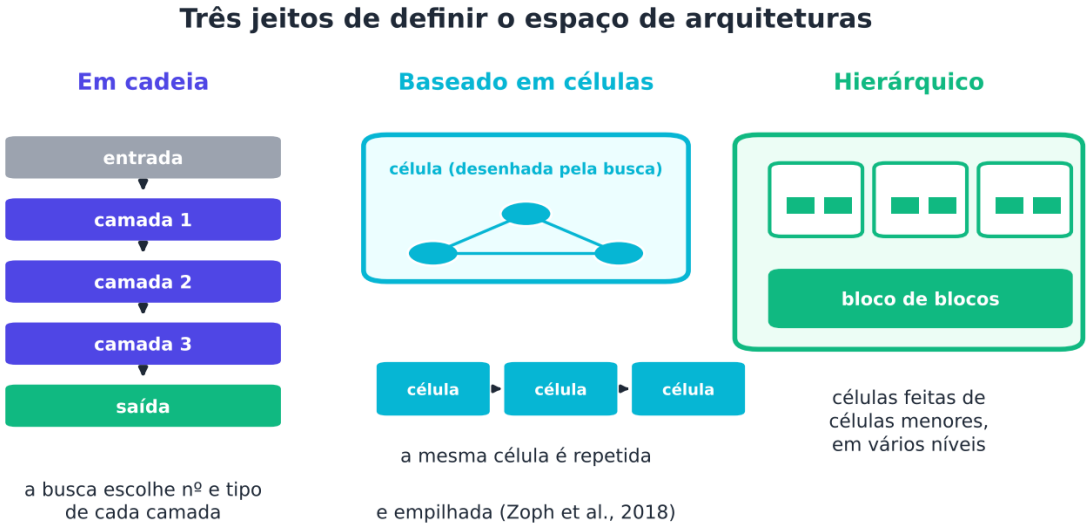


Figura 11.3 — Três jeitos de definir o espaço: cadeia de camadas, blocos repetidos (células) e espaços hierárquicos.

- **Espaço em cadeia:** a rede é uma sequência de camadas. A busca escolhe o número de camadas e o tipo e o tamanho de cada uma. É o mais simples.
- **Espaço baseado em células:** em vez de desenhar a rede inteira, a busca desenha **um bloco pequeno (a célula)**, que depois é **repetido e empilhado** para formar a rede. Zoph *et al.* (2018) mostraram que uma célula encontrada em um conjunto pequeno de imagens podia ser transferida para conjuntos maiores. Isso reduz muito o espaço a explorar.
- **Espaço hierárquico:** células formadas por células menores, em vários níveis.

Um ponto importante: o espaço **cresce explosivamente**. Se cada camada tem 5 opções de largura e a rede pode ter até 3 camadas, quantas redes existem? Com 2 funções de ativação, contamos:

$$N = 2 \times (5^1 + 5^2 + 5^3) = 2 \times 155 = 310$$

Só 310! É pequeno de propósito, e isso nos permite uma proeza didática: **avaliar todas as arquiteturas** e descobrir a verdade. Em NAS real, os espaços têm números astronômicos de arquiteturas e essa conferência é impossível.

11.4 O mapa do tesouro: avaliando todo o espaço

Vamos usar o conjunto **Digits** (UCI, via scikit-learn): 1.797 imagens 8×8 de dígitos escritos à mão, com 64 atributos e 10 classes. Reservamos 25% para o teste final e separamos 30% do restante para **validação** (onde a busca avalia as redes).

O espaço será o das 310 redes: de 1 a 3 camadas ocultas, cada uma com 8, 16, 32, 64 ou 128 neurônios, e ativação `relu` ou `tanh`. Treinamos cada rede com o `MLPClassifier` do scikit-learn (um **perceptron multicamadas**) por **40 iterações** (épocas), com a semente fixa. Também guardamos a nota com apenas 10 iterações (estimativa barata) e o **número de parâmetros** (pesos e vieses), que mede o tamanho da rede.

PYTHON

```python
import warnings; warnings.filterwarnings("ignore")
import itertools, json
import numpy as np
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPClassifier
from sklearn.preprocessing import StandardScaler

X, y = load_digits(return_X_y=True)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.25, stratify=y, random_state=0)
Xa, Xv, ya, yv = train_test_split(Xtr, ytr, test_size=0.3, stratify=ytr, random_state=0)
sc = StandardScaler().fit(Xa); Xa, Xv, Xt = sc.transform(Xa), sc.transform(Xv), sc.transform(Xte)

LARGURAS = [8, 16, 32, 64, 128]; ATIVACOES = ["relu", "tanh"]
ARQS = [(h, a) for L in (1, 2, 3) for h in itertools.product(LARGURAS, repeat=L) for a in ATIVACOES]
print("Tamanho do espaço:", len(ARQS), "arquiteturas")

def n_parametros(arq):
    d = [64] + list(arq[0]) + [10]
    return sum(i * o + o for i, o in zip(d[:-1], d[1:]))

def treina(arq, iteracoes):
    return MLPClassifier(arq[0], activation=arq[1], max_iter=iteracoes, random_state=0).fit(Xa, ya)

tabela = []
for arq in ARQS:
    m10, m40 = treina(arq, 10), treina(arq, 40)
    tabela.append({"arq": [list(arq[0]), arq[1]], "params": n_parametros(arq),
                   "v10": m10.score(Xv, yv), "v40": m40.score(Xv, yv), "teste": m40.score(Xt, yte)})
json.dump(tabela, open("/home/user/claude/automl/build/f11_cache.json", "w"))

v = np.array([r["v40"] for r in tabela]); i = int(v.argmax())
print("Validação: pior = %.3f | mediana = %.3f | melhor = %.3f" % (v.min(), np.median(v), v.max()))
print("Melhor arquitetura:", tuple(tabela[i]["arq"][0]), tabela[i]["arq"][1], "| parâmetros:",
tabela[i]["params"])
print("Seu acerto no TESTE: %.3f" % tabela[i]["teste"])
```

SAÍDA

```
Tamanho do espaço: 310 arquiteturas
Validação: pior = 0.546 | mediana = 0.928 | melhor = 0.970
Melhor arquitetura: (128, 128) relu | parâmetros: 26122
Seu acerto no TESTE: 0.964
```

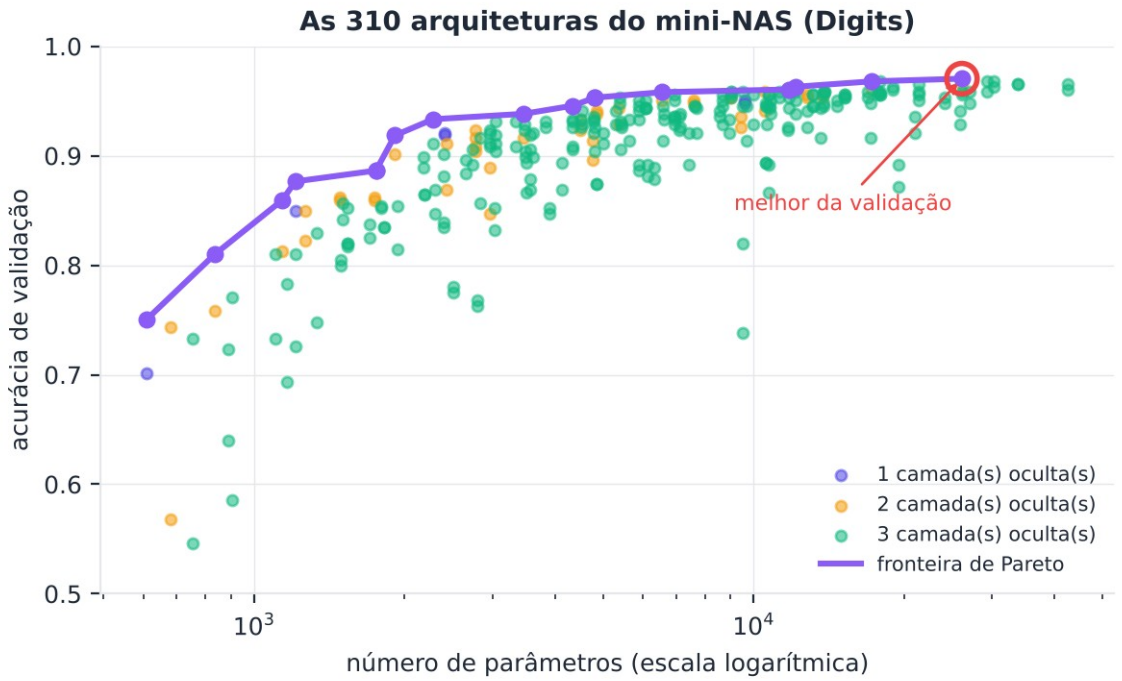


Figura 11.4 — O "mapa do tesouro": as 310 arquiteturas, com a acurácia de validação em função do tamanho da rede. A linha roxa liga as redes eficientes (fronteira de Pareto).

O mapa conta uma história útil. Há uma enorme variação (as redes mais fracas ficam muito abaixo da mediana), mas **muitas** arquiteturas diferentes chegam perto do topo. Redes pequenas demais, como a de 8 neurônios, sofrem. Já a partir de certo tamanho, **crescer não compra quase nada**: a nuvem se achata. É por isso que o tamanho da rede, e não só a acurácia, importa na escolha, como veremos na Seção 11.8.

11.5 Estratégias de busca

Em um espaço real, não podemos avaliar tudo. Precisamos de uma **estratégia**. As quatro famílias mais importantes são:

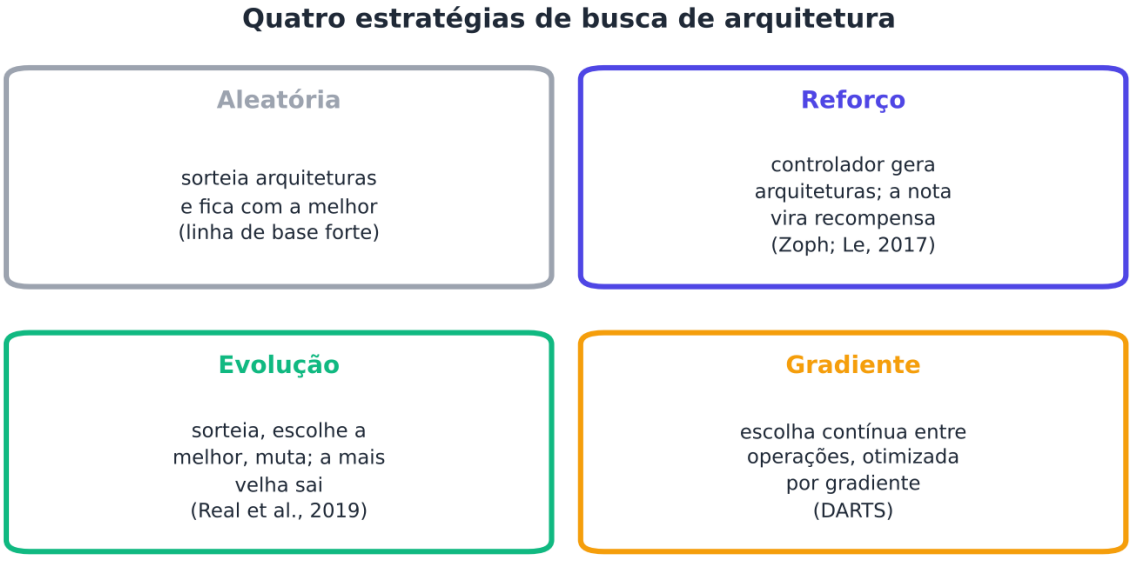


Figura 11.5 — Quatro estratégias de busca de arquitetura: aleatória, por reforço, evolutiva e por gradiente.

1. **Busca aleatória:** sorteia arquiteturas. Parece ingênua, mas é uma linha de base surpreendentemente forte (como vimos no Capítulo 4, com Bergstra e Bengio, 2012).
2. **Aprendizado por reforço:** um "controlador" (outra rede) **gera** descrições de arquiteturas, que são treinadas; a nota delas serve de **recompensa** para o controlador melhorar suas propostas (Zoph; Le, 2017).
3. **Evolução:** mantém uma população de arquiteturas, **sorteia** algumas, escolhe a melhor e **cria uma filha** com uma pequena mutação. A versão "regularizada" de Real *et al.* (2019) descarta sempre a **mais velha**, e não a pior, o que favorece arquiteturas que seguem boas ao longo do tempo.
4. **Métodos de gradiente (DARTS):** transformam a escolha entre operações em algo **contínuo**, que pode ser otimizado por gradiente descendente, junto com os pesos da rede (Liu; Simonyan; Yang, 2019).

Além dessas, o **compartilhamento de pesos** (ENAS) acelera tudo: as arquiteturas candidatas são tratadas como **partes de uma grande rede**, e os pesos são **reaproveitados** em vez de treinados do zero a cada candidata (Pham *et al.*, 2018).

> NO SEU DIA A DIA **Analogia: a seleção de uma equipe**
>
> Na **busca aleatória**, você escala jogadores por sorteio. Na **evolução**, mantém um elenco e, a cada rodada, pega um bom jogador e testa uma pequena variação dele; o jogador mais antigo se aposenta, sempre. No **reforço**, um técnico (o controlador) aprende, pelos resultados, a montar melhores escalações.

11.6 Mini-NAS: aleatória ou evolução?

Como a nossa tabela com as 310 arquiteturas já está pronta, podemos testar as estratégias **sem treinar nada de novo**: avaliar uma arquitetura passa a ser **consultar a tabela**. Isso permite repetir cada estratégia **300 vezes** e medir com segurança, algo impossível em NAS de verdade.

O **orçamento** será de **30 arquiteturas** (cerca de 10% do espaço). Compararemos:

- **Aleatória:** 30 arquiteturas sorteadas, sem repetição.
- **Evolução regularizada:** população de 8; em cada passo, sorteia 3 da população, escolhe a melhor e gera uma filha por **mutação** (trocar a largura de uma camada, inserir uma camada, remover uma camada ou trocar a ativação); a mais velha sai.

Medimos quatro coisas: a nota de validação da melhor encontrada (`val.`, mediana), o **posto** dela entre as 310 (`posto`, mediana; 1 = o melhor do espaço), a proporção de repetições em que a escolha ficou entre os 5% melhores (`top 5%`) e a acurácia média no **teste** dessa escolha (`teste`).

PYTHON

```python
import json
import numpy as np

T = json.load(open("/home/user/claude/automl/build/f11_cache.json"))
chave = lambda a: (tuple(a[0]), a[1])
V = {chave(r["arq"]): r["v40"] for r in T}; TESTE = {chave(r["arq"]): r["teste"] for r in T}
ARQS = list(V); LARGURAS = [8, 16, 32, 64, 128]; ATIVACOES = ["relu", "tanh"]
posto = lambda a: 1 + sum(v > V[a] for v in V.values())

def mutacao(a, r):
    h, ativ = list(a[0]), a[1]; op = r.integers(4)
    if op == 0: h[r.integers(len(h))] = int(r.choice(LARGURAS))        # muda uma largura
    elif op == 1 and len(h) < 3: h.insert(r.integers(len(h) + 1), int(r.choice(LARGURAS)))   # insere
```

```
camada
    elif op == 2 and len(h) > 1: h.pop(r.integers(len(h)))            # remove camada
    else: ativ = [x for x in ATIVACOES if x != ativ][0]               # troca a ativação
    return (tuple(h), ativ)

def aleatoria(B, r):
    return [ARQS[i] for i in r.choice(len(ARQS), B, replace=False)]

def evolucao(B, r, P=8, S=3):
    hist, pop, vistas = [], [], set()
    while len(hist) < P:
        a = ARQS[r.integers(len(ARQS))]
        if a not in vistas: vistas.add(a); hist.append(a); pop.append(a)
    while len(hist) < B:
        pai = max([pop[i] for i in r.choice(len(pop), S, replace=False)], key=V.get)
        for _ in range(50):
            filha = mutacao(pai, r)
            if filha not in vistas: break
        else: filha = ARQS[r.integers(len(ARQS))]
        if filha in vistas: continue
        vistas.add(filha); hist.append(filha); pop.append(filha); pop.pop(0)    # a mais velha sai
    return hist

B, R = 30, 300
curvas = {}; resumo = {}
for nome, fn in (("aleatória", aleatoria), ("evolução", evolucao)):
    linhas, cs = [], []
    for s in range(R):
        h = fn(B, np.random.default_rng(s)); b = max(h, key=V.get)
        linhas.append((V[b], posto(b), TESTE[b]))
        cs.append(np.maximum.accumulate([V[a] for a in h]))
    L = np.array(linhas); curvas[nome] = np.array(cs)
    resumo[nome] = {"val": float(np.median(L[:, 0])), "posto": float(np.median(L[:, 1])), "top5":
float(np.mean(L[:, 1] <= 16)), "teste": float(L[:, 2].mean())}
    print(f"{nome:<10s} val. = {np.median(L[:, 0]):.3f} | posto = {np.median(L[:, 1]):.1f} | "
          f"top 5% = {100 * np.mean(L[:, 1] <= 16):.0f}% | teste = {L[:, 2].mean():.3f}")
print("Melhor do espaço (validação): %.3f | teste médio das 310 redes: %.3f" % (max(V.values()),
np.mean(list(TESTE.values()))))
json.dump({"resumo": resumo, "curvas": {k: v.tolist() for k, v in curvas.items()}},
open("/home/user/claude/automl/build/f11_busca.json", "w"))
```

SAÍDA

```
aleatória  val. = 0.965 | posto = 6.0 | top 5% = 84% | teste = 0.962
evolução   val. = 0.969 | posto = 1.5 | top 5% = 90% | teste = 0.963
Melhor do espaço (validação): 0.970 | teste médio das 310 redes: 0.907
```

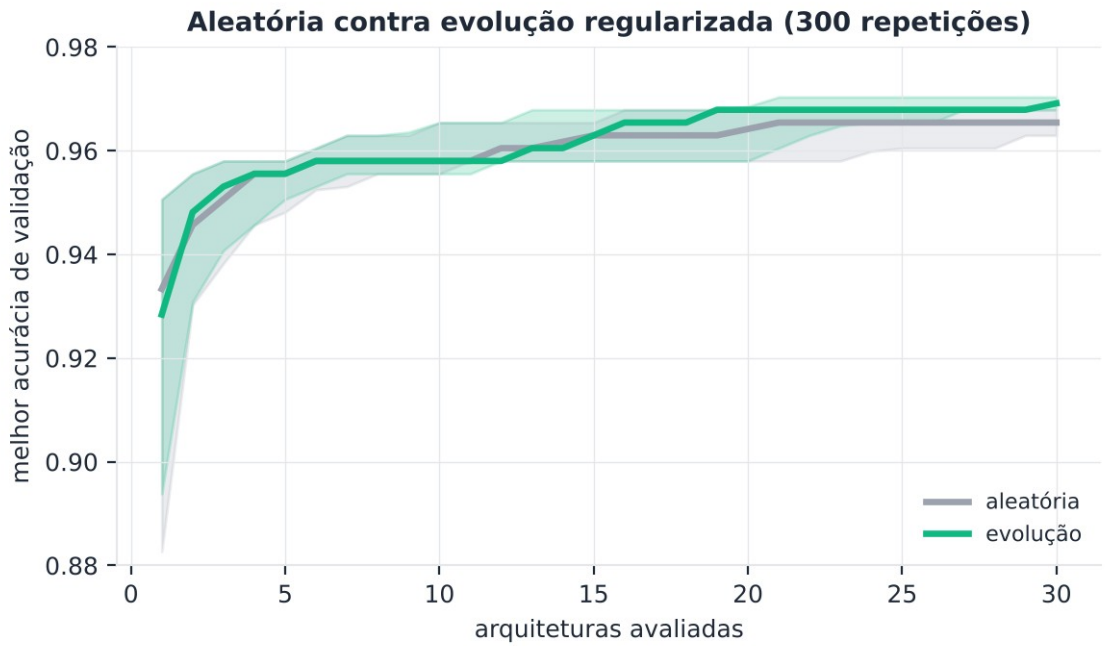


Figura 11.6 — Melhor nota de validação encontrada em função do número de arquiteturas avaliadas (mediana de 300 repetições, com faixa entre os percentis 25 e 75).

Observe com cuidado. A evolução encontra, na mediana, uma rede melhor **na validação** e muito mais bem posicionada no ranking do espaço. Mas veja a coluna do **teste**: as duas escolhas chegam a notas praticamente iguais. Duas lições: (1) no espaço pequeno e "generoso" que escolhemos, muitas redes são boas, de modo que a busca aleatória já faz um bom trabalho (como no Capítulo 4); (2) a diferença que a busca enxerga na validação é, em parte, o **otimismo da busca** que conhecemos desde o Capítulo 2: quanto mais afinadamente escolhemos pela validação, menos esse ganho se confirma no teste.

> **Atenção: "ganhar na validação" não é "ganhar no teste"**
>
> A evolução venceu a busca aleatória no critério que a própria busca otimiza (a validação). O teste, que a busca nunca viu, mostra uma diferença **muito menor**. Sempre relate o teste final, medido uma única vez.

11.7 Estimar sem treinar tudo: fidelidade, pesos compartilhados e preditores

O custo do NAS está quase todo em **treinar** cada candidata. Daí a pergunta central do terceiro pilar: como **estimar** a qualidade de uma arquitetura gastando pouco?



Figura 11.7 — Quatro maneiras de estimar o desempenho de uma arquitetura sem treiná-la por completo.

- **Treino curto (multifidelidade):** treina por poucas épocas, em menos dados ou em imagens menores. É a ideia do Hyperband e do Successive Halving (Capítulo 6).
- **Compartilhamento de pesos:** reaproveita pesos entre candidatas (Pham *et al.*, 2018).
- **Extrapolação da curva de aprendizado:** olha o começo da curva e prevê o final.
- **Preditores de desempenho:** um modelo (meta-aprendizado, Capítulo 9) prevê a nota a partir da descrição da arquitetura.

Todas essas estimativas **introduzem erro**. O que importa é que a **ordem** das arquiteturas se preserve: se a estimativa barata diz que A é melhor que B, isso deve valer também para o treino completo. Medimos isso com a **correlação de Spearman** (correlação entre **postos**; vale 1 se a ordem é idêntica e 0 se não há relação). Vamos comparar, para as 310 redes, a nota com 10 iterações (barata) e com 40 (mais cara), e ver o que ocorre se usarmos a estimativa barata para **descartar** candidatas, estilo *Successive Halving*.

```python
import json
import numpy as np
from scipy.stats import spearmanr

T = json.load(open("/home/user/claude/automl/build/f11_cache.json"))
v10 = np.array([r["v10"] for r in T]); v40 = np.array([r["v40"] for r in T])
rho, _ = spearmanr(v10, v40)
print(f"Correlação de Spearman entre a nota com 10 e com 40 iterações: {rho:.3f}")

melhores40 = set(np.argsort(-v40)[:15])                    # as 15 melhores (5%) com treino "completo"
print("\nSe avaliamos todas com 10 iterações e treinamos mais só as k melhores:")
print(" k mantidas | % do custo | melhor rede do espaço sobrevive? | nota final da melhor mantida")
for k in (10, 30, 60, 120):
    mantidas = np.argsort(-v10)[:k]
    custo = (310 * 10 + k * 40) / (310 * 40)              # em iterações de treino, relativo a treinar tudo com 40
    ok = int(np.argmax(v40)) in set(mantidas)
    print(f"   {k:>4d}     |   {100 * custo:4.0f}%    |   {'sim' if ok else 'não':>3s}
| {v40[mantidas].max():.3f}")
print("\nMelhor do espaço (40 iterações): %.3f" % v40.max())
json.dump({"v10": v10.tolist(), "v40": v40.tolist(), "rho": float(rho)},
open("/home/user/claude/automl/build/f11_fid.json", "w"))
```

SAÍDA

```
Correlação de Spearman entre a nota com 10 e com 40 iterações: 0.872

Se avaliamos todas com 10 iterações e treinamos mais só as k melhores:
 k mantidas | % do custo | melhor rede do espaço sobrevive? | nota final da melhor mantida
     10     |     28%    |   sim                            | 0.970
     30     |     35%    |   sim                            | 0.970
     60     |     44%    |   sim                            | 0.970
    120     |     64%    |   sim                            | 0.970

Melhor do espaço (40 iterações): 0.970
```

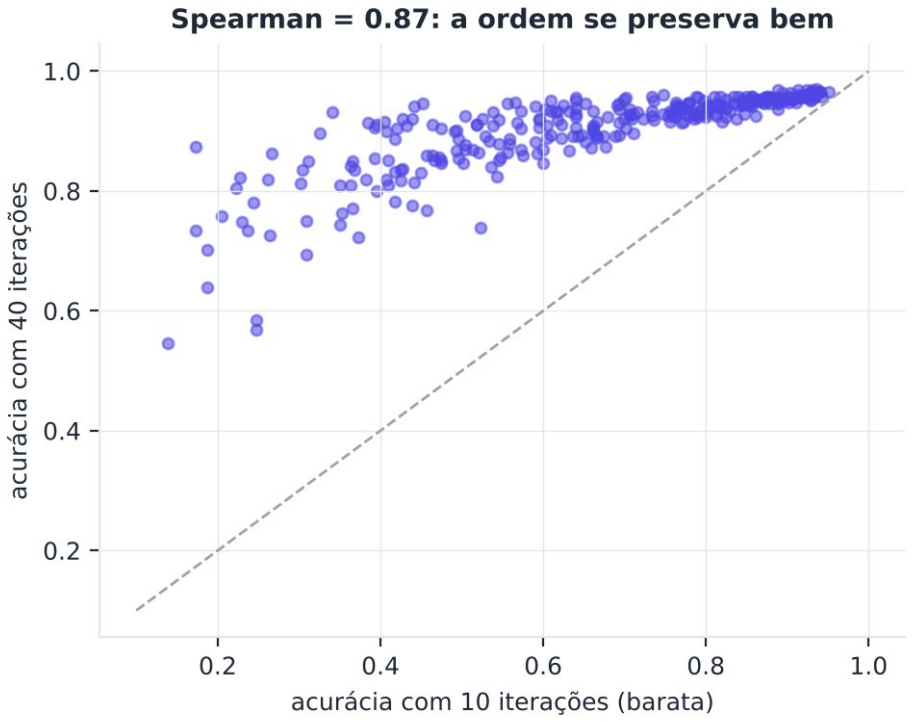


Figura 11.8 — Cada ponto é uma arquitetura: a nota com 10 iterações (barata) contra a nota com 40 (mais cara).

A estimativa barata **preserva bem a ordem** neste problema (Spearman de 0,87). Por isso o truque de "filtrar com a avaliação barata" funcionou em **todos** os cortes testados: a melhor rede do espaço sobreviveu mesmo mantendo só as 10 primeiras, com 28% do custo. Esse é um caso **favorável**; não espere garantia igual em qualquer problema. Mas cuidado: isso depende do problema. Há casos em que redes **lentas para começar** (que só brilham no fim do treino) são descartadas injustamente. Quanto menor a correlação entre as fidelidades, mais perigoso é o corte.

11.8 Redes boas e pequenas: busca com mais de um objetivo

Na prática, não queremos só a rede **mais acurada**; queremos uma rede acurada **e** rápida, pequena e barata em memória e energia. É um problema de **otimização multiobjetivo**, como vimos com a fronteira de Pareto no Capítulo 7 (Deb *et al.*, 2002).

Volte à Figura 11.4: a linha roxa liga as arquiteturas **não dominadas**, aquelas para as quais não existe outra com **mais acurácia e menos parâmetros** ao mesmo tempo. Ela mostra o *trade-off* completo. Se o melhor modelo tem

26 mil parâmetros, mas existe uma rede com uma fração disso e acurácia quase igual, a segunda pode ser a escolha certa para um celular.

PYTHON

```python
import json
import numpy as np

T = json.load(open("/home/user/claude/automl/build/f11_cache.json"))
P = np.array([r["params"] for r in T]); A = np.array([r["v40"] for r in T])
# uma rede está na fronteira se ninguém tem (acurácia >= e parâmetros <=) com ao menos uma desigualdade estrita
fronteira = [i for i in range(len(T)) if not any((A[j] >= A[i] and P[j] <= P[i]) and (A[j] > A[i] or P[j] < P[i]) for j in range(len(T)))]
fronteira.sort(key=lambda i: P[i])
print("Fronteira de Pareto (menos parâmetros → mais acurácia):")
for i in fronteira:
    print(f"  {str(tuple(T[i]['arq'][0])):<16s} {T[i]['arq'][1]:<5s} parâmetros = {P[i]:>6d} validação = {A[i]:.3f}")
melhor = int(A.argmax())
bom = [i for i in fronteira if A[i] >= A[melhor] - 0.01]
j = min(bom, key=lambda i: P[i])
print(f"\nMais acurada: {P[melhor]} parâmetros ({A[melhor]:.3f}). Menor dentro de 1 ponto da melhor: {P[j]} parâmetros ({A[j]:.3f}).")
print(f"Redução de tamanho: {100 * (1 - P[j] / P[melhor]):.0f}%")
json.dump({"fronteira": fronteira}, open("/home/user/claude/automl/build/f11_pareto.json", "w"))
```

SAÍDA

```
Fronteira de Pareto (menos parâmetros → mais acurácia):
  (8,)             tanh  parâmetros =    610   validação = 0.751
  (8, 16)          tanh  parâmetros =    834   validação = 0.810
  (8, 32)          tanh  parâmetros =   1138   validação = 0.859
  (16,)            tanh  parâmetros =   1210   validação = 0.877
  (16, 16, 16)     relu  parâmetros =   1754   validação = 0.886
  (16, 16, 16)     tanh  parâmetros =   1754   validação = 0.886
  (16, 32)         tanh  parâmetros =   1914   validação = 0.919
  (16, 32, 16)     tanh  parâmetros =   2282   validação = 0.933
  (32, 32)         tanh  parâmetros =   3466   validação = 0.938
  (32, 16, 64)     tanh  parâmetros =   4346   validação = 0.946
  (64,)            relu  parâmetros =   4810   validação = 0.953
  (64,)            tanh  parâmetros =   4810   validação = 0.953
  (64, 32)         relu  parâmetros =   6570   validação = 0.958
  (128, 8, 128)    relu  parâmetros =  11794   validação = 0.960
  (128, 16, 64)    tanh  parâmetros =  12122   validação = 0.963
  (128, 64)        relu  parâmetros =  17226   validação = 0.968
  (128, 128)       relu  parâmetros =  26122   validação = 0.970

Mais acurada: 26122 parâmetros (0.970). Menor dentro de 1 ponto da melhor: 11794 parâmetros (0.960).
Redução de tamanho: 55%
```

No nosso mapa, a rede de (128, 8, 128) tem 11.794 parâmetros e 0,960 de validação: **55% menor** que a mais acurada (26.122 parâmetros, 0,970), por apenas um ponto de acurácia. Esse é o espírito de várias famílias modernas de redes: usar busca para encontrar arquiteturas que equilibram precisão e custo, e **escalar** a rede de forma coordenada (profundidade, largura e resolução) para obter versões maiores ou menores (Tan; Le, 2019).

11.9 O custo do NAS e a "IA verde"

Treinar muitas redes consome **energia e dinheiro**. Strubell, Ganesh e McCallum (2019) mostraram que o treinamento de modelos de linguagem pode ter custo de energia e de carbono considerável, e que uma busca de arquitetura (e de hiperparâmetros) multiplica esse custo. Schwartz *et al.* (2020) defendem a **IA verde** (*Green AI*): reportar o custo computacional junto com a precisão e valorizar a eficiência, não só o recorde.

Para um estudante ou uma pequena equipe, isso leva a práticas simples:

1. **Comece pequeno:** um espaço de busca enxuto e bem pensado vale mais que um enorme.
2. **Use estimativas baratas** (treino curto, poda) e **reaproveite** resultados (cache, *warm start*).
3. **Compare com linhas de base fortes:** busca aleatória e uma arquitetura conhecida.
4. **Reporte o custo:** número de arquiteturas avaliadas, tempo e hardware.

> **Atenção: o NAS nem sempre compensa**
>
> Em dados tabulares típicos, modelos de árvores seguem muito competitivos frente a redes neurais (Grinsztajn; Oyallon; Varoquaux, 2022). Procurar arquitetura só faz sentido quando uma rede neural é realmente a escolha certa, como em imagens, áudio e texto (Capítulo 12).

11.10 Ferramentas: Auto-Keras

Para quem quer usar NAS sem implementá-lo, o **Auto-Keras** oferece uma interface simples sobre o Keras, com busca guiada de arquiteturas (Jin; Song; Hu, 2019).

> **Atenção: código não executado**
>
> O código abaixo segue a documentação, mas **não foi executado** na preparação deste livro (a biblioteca não pôde ser instalada no ambiente). Nenhum resultado é reportado.

```python
import autokeras as ak

clf = ak.ImageClassifier(max_trials=10, overwrite=True, seed=0)   # 10 arquiteturas candidatas
clf.fit(x_treino, y_treino, epochs=5, validation_split=0.2)
print(clf.evaluate(x_teste, y_teste))
modelo = clf.export_model()          # devolve um modelo Keras para salvar e reutilizar
modelo.summary()
```

11.11 Boas práticas

1. **Defina um espaço pequeno e sensato** antes de pensar em estratégias sofisticadas.
2. **Compare com busca aleatória:** se a estratégia complexa não a supera com folga, não se justifica.
3. **Relate o teste final** de uma única escolha e a variação entre sementes.
4. **Verifique a correlação** entre a estimativa barata e a avaliação completa antes de cortar candidatas.
5. **Considere o tamanho e o custo** da rede (fronteira de Pareto), não só a acurácia.
6. **Registre e relate o custo** da busca (Green AI).
7. **Prefira arquiteturas conhecidas** quando existirem boas opções prontas: o NAS é uma ferramenta, não uma obrigação.

Resumo do capítulo

MEMORIZE **Cartões de memorização**

- **Arquitetura:** o "desenho" da rede (camadas, neurônios, conexões, ativação).
- **NAS:** busca automática da arquitetura (Elsken *et al.*, 2019).
- **Três pilares:** espaço de busca, estratégia de busca, estimativa de desempenho.
- **Espaços:** em cadeia, em células e hierárquicos; o espaço define o teto da busca.
- **Estratégias:** aleatória, reforço (Zoph; Le), evolução regularizada (Real *et al.*), gradiente (DARTS) e pesos compartilhados (ENAS).
- **No nosso mini-NAS (310 redes):** evolução venceu a aleatória na validação, mas o teste ficou praticamente igual.
- **Estimativa barata:** preserva a ordem se a correlação de Spearman for alta; do contrário, o corte é perigoso.
- **Multiobjetivo:** acurácia e custo; fronteira de Pareto.
- **Green AI:** reporte o custo, comece pequeno e compare com linhas de base.

Exercícios de múltipla escolha

Escolha **uma** alternativa em cada questão. O gabarito está no Apêndice A.

Questão 1. O que é a arquitetura de uma rede neural?

a) O desenho da rede: número de camadas, neurônios, conexões e ativação.
b) O valor da taxa de aprendizado apenas.
c) O conjunto de dados de treino.
d) A métrica usada no teste.

Questão 2. O que significa NAS?

a) Normalized Accuracy Score, uma métrica de acurácia.
b) New Algorithm Selection, um tipo de pré-processamento.
c) Neural Architecture Search, a busca automática da arquitetura de uma rede.
d) Network Accuracy Sampling, uma técnica de amostragem.

Questão 3. Quais são os três pilares do NAS segundo Elsken, Metzen e Hutter (2019)?

a) Dados, rótulos e métrica.
b) Treino, validação e teste.
c) Camada, neurônio e ativação.
d) Espaço de busca, estratégia de busca e estimativa de desempenho.

Questão 4. O que define o espaço de busca?

a) O tamanho do conjunto de teste.
b) O conjunto de arquiteturas que a busca pode encontrar.
c) A velocidade do computador.
d) O número de classes do problema.

Questão 5. O que é uma função de ativação?

a) O método de validação cruzada.
b) O tempo máximo de treino.
c) A regra que decide a saída de cada neurônio a partir do que ele recebe.
d) O número de camadas ocultas.

Questão 6. O que é um espaço de busca em cadeia?

a) Uma rede formada por uma sequência de camadas, em que a busca escolhe número e tipo delas.
b) Um espaço de hiperparâmetros contínuos apenas.
c) Um espaço com apenas uma arquitetura.
d) Uma rede formada por células repetidas.

Questão 7. O que é um espaço baseado em células?

a) Aquele em que cada neurônio é escolhido isoladamente.
b) Aquele em que a busca desenha um bloco pequeno, repetido e empilhado para formar a rede.
c) Aquele sem camadas.
d) Aquele que usa só modelos lineares.

Questão 8. Por que o espaço baseado em células reduz o custo da busca?

a) Porque usa menos dados de teste.
b) Porque se procura só o desenho de um bloco, que depois é repetido.
c) Porque dispensa treino.
d) Porque elimina a validação.

Questão 9. No espaço do capítulo (1 a 3 camadas, 5 larguras, 2 ativações), quantas arquiteturas existem?

a) 30.
b) 1.000.
c) 310.
d) 155.

Questão 10. Por que o capítulo escolheu um espaço de apenas 310 arquiteturas?

a) Porque 310 é o máximo permitido pelo scikit-learn.
b) Porque as redes maiores não treinam.
c) Para poder avaliar todas e conhecer a verdade, o que é impossível em NAS real.
d) Porque o teste só aceita 310 exemplos.

Questão 11. Qual conjunto de dados foi usado no mini-NAS do capítulo?

a) Digits (UCI, via scikit-learn).
b) ImageNet.
c) Boston Housing.
d) Titanic.

Questão 12. Qual modelo do scikit-learn foi usado para treinar as arquiteturas?

a) MLPClassifier, um perceptron multicamadas.
b) LinearRegression.
c) DecisionTreeClassifier.
d) KNeighborsClassifier.

Questão 13. Como é medido o tamanho de uma rede no capítulo?

a) Pelo número de parâmetros (pesos e vieses).
b) Pelo número de linhas do conjunto de dados.
c) Pelo número de classes.
d) Pelo tempo de execução do Python.

Questão 14. Qual é a ideia da busca por aprendizado por reforço (Zoph; Le, 2017)?

a) Sorteia arquiteturas sem aprender.
b) Remove a validação.
c) Um controlador gera arquiteturas e usa a nota delas como recompensa para melhorar.
d) Calcula o gradiente da arquitetura.

Questão 15. O que caracteriza a evolução regularizada de Real et al. (2019)?

a) Descartar sempre a arquitetura mais velha da população, e não a pior.
b) Nunca usar mutação.
c) Descartar sempre a pior.
d) Usar apenas busca em grade.

Questão 16. Como a evolução cria uma nova arquitetura?

a) Remove a camada de saída.
b) Sorteia algumas da população, escolhe a melhor e aplica uma pequena mutação.
c) Soma os pesos de duas redes.
d) Sorteia uma nova arquitetura do zero.

Questão 17. O que o DARTS faz de diferente?

a) Torna a escolha entre operações contínua e a otimiza por gradiente descendente.
b) Sorteia operações.
c) Usa apenas evolução.
d) Dispensa a rede.

Questão 18. O que é o compartilhamento de pesos (ENAS)?

a) Compartilhar o conjunto de teste.
b) Usar a mesma semente em tudo.
c) Copiar pesos de outro conjunto de dados sem alterar.
d) Reaproveitar pesos entre arquiteturas candidatas, em vez de treinar cada uma do zero.

Questão 19. Por que a busca aleatória é uma boa linha de base em NAS?

a) Porque dispensa avaliação.
b) Porque é sempre a melhor.
c) Porque costuma ser surpreendentemente competitiva em espaços generosos.
d) Porque é a única possível.

Questão 20. Qual dos itens NÃO é uma forma de estimar o desempenho sem treino completo?

a) Usar o conjunto de teste para escolher a arquitetura.
b) Compartilhamento de pesos.
c) Treino curto.
d) Extrapolação da curva de aprendizado.

Questão 21. O que a correlação de Spearman mede?

a) O número de parâmetros.
b) Quão bem a ordem (postos) de duas listas coincide.
c) O tempo de treino.
d) A diferença de médias.

Questão 22. Qual foi a correlação de Spearman entre 10 e 40 iterações no mini-NAS?

a) 1,00.
b) Cerca de 0,87.
c) 0,00.
d) Negativa.

Questão 23. Por que um corte baseado em estimativa barata pode ser perigoso?

a) Porque redes que só brilham no fim do treino podem ser descartadas injustamente.
b) Porque elimina a semente.
c) Porque aumenta o conjunto de teste.
d) Porque torna o modelo linear.

Questão 24. No mini-NAS, qual estratégia obteve melhor nota de validação mediana com 30 avaliações?

a) A evolução regularizada (0,969 contra 0,965 da aleatória).
b) Nenhuma: empataram exatamente.
c) A aleatória, com folga.
d) Nenhuma foi avaliada.

Questão 25. O que o teste mostrou no mini-NAS sobre as duas estratégias?

a) As escolhas chegaram a acurácias de teste praticamente iguais.
b) A evolução venceu por 10 pontos.
c) A aleatória falhou.
d) O teste não foi medido.

Questão 26. Por que 'ganhar na validação' não significa 'ganhar no teste'?

a) Porque a validação é ruim.
b) Porque o teste não é confiável.
c) Porque a busca otimiza a validação e o ganho pode ser parcialmente otimismo da escolha.
d) Porque o teste é sempre pior.

Questão 27. O que é a fronteira de Pareto na busca de arquitetura?

a) A pior rede.
b) A lista das redes com mais parâmetros.
c) A média das redes.
d) O conjunto de redes não dominadas em acurácia e custo.

Questão 28. Segundo o capítulo, o que mostrou a fronteira de Pareto do mini-NAS?

a) Que redes pequenas não funcionam.
b) Uma rede 55% menor que a mais acurada, perdendo cerca de um ponto de acurácia.
c) Que o número de camadas não importa.
d) Que a maior rede é sempre a melhor.

Questão 29. O que defende a 'IA verde' (Schwartz et al., 2020)?

a) Usar sempre as maiores redes.
b) Esconder o custo.
c) Evitar redes neurais em qualquer caso.
d) Relatar o custo computacional junto com a precisão e valorizar a eficiência.

Questão 30. Em que situação o NAS tende a fazer mais sentido, segundo o capítulo?

a) Sempre que há dados tabulares.
b) Nunca.
c) Apenas com poucos exemplos.
d) Quando uma rede neural é a escolha certa, como em imagens, áudio e texto.

Exercícios discursivos

Responda com suas palavras, em poucas linhas. Compare depois com a resposta-modelo do Apêndice A.

1. Explique a analogia entre arquitetura de rede e planta de uma casa.
2. Descreva os três pilares do NAS e relacione-os com o AutoML clássico.
3. Por que o espaço de busca 'define o teto' do NAS?
4. Compare espaços em cadeia, baseados em células e hierárquicos.
5. Mostre como calcular o tamanho do espaço do capítulo.
6. Por que o capítulo usou um espaço pequeno e o avaliou por inteiro?
7. Explique as quatro famílias de estratégias de busca citadas.
8. Explique a evolução regularizada e por que descartar a mais velha ajuda.
9. Descreva quatro tipos de mutação usados no mini-NAS.
10. O que é compartilhamento de pesos e qual problema ele resolve?
11. Compare os resultados de busca aleatória e evolução no mini-NAS.
12. Por que a vantagem da evolução na validação é parcialmente ilusória?
13. Explique o que mede a correlação de Spearman e por que ela importa em NAS.
14. Descreva o experimento de filtragem com avaliação barata e seu resultado.
15. Quando uma avaliação barata pode enganar a busca?
16. Cite quatro maneiras de estimar o desempenho de uma arquitetura sem treino completo.
17. Explique a busca multiobjetivo e a fronteira de Pareto em NAS.
18. Interprete o resultado da fronteira do mini-NAS.
19. O que é IA verde e como ela se aplica ao NAS?
20. Quando o NAS não compensa? Dê dois motivos.

Exercícios práticos em Python (scikit-learn)

Resolva os exercícios no seu computador (ou no Google Colab). Eles usam conjuntos de dados da UCI que já vêm no scikit-learn ou dados simulados. Tente resolver **antes** de consultar a solução, no Apêndice D. As saídas apresentadas lá foram obtidas com as versões de bibliotecas listadas na Apresentação.

Exercício 1 — Contando arquiteturas

NÍVEL: BÁSICO

Considere um espaço em cadeia com 1 a 4 camadas ocultas, largura de cada camada em {16, 32, 64} e ativação em {`relu`, `tanh`, `logistic`} (uma ativação única para a rede toda). (a) Calcule o tamanho do espaço pela fórmula. (b) Confirme por enumeração com `itertools.product`. (c) Se cada arquitetura levasse 2 minutos para treinar, quantas horas levaria para avaliar todas?

Pergunta: o que acontece com o tamanho do espaço se acrescentarmos mais um valor de largura (por exemplo, 128)?

Exercício 2 — Maior é sempre melhor?

NÍVEL: INTERMEDIÁRIO

No conjunto **Digits**, sorteie 40 arquiteturas (1 a 3 camadas, larguras em {8, 16, 32, 64, 128}, ativação `relu` ou `tanh`, `random_state=0` para o sorteio), treine cada uma com `MLPClassifier` (`max_iter=40`) e meça a acurácia de validação. Calcule a correlação de Spearman entre o **número de parâmetros** e a acurácia, e mostre as três melhores redes.

Pergunta: mais parâmetros significam mais acurácia? Há redes pequenas entre as melhores?

Exercício 3 — Aleatória ou hill climbing?

NÍVEL: INTERMEDIÁRIO

No conjunto **Breast Cancer** (UCI), defina um espaço com 1 a 3 camadas ocultas (larguras em {4, 8, 16, 32, 64}) e ativação `relu` ou `tanh`. Compare, com orçamento de **15 arquiteturas** e **5 sementes**: (a) busca aleatória; (b) *hill climbing*: parte de uma arquitetura aleatória e, em cada passo, avalia uma **mutação** da melhor até agora (muda uma largura, adiciona ou remove uma camada, ou troca a ativação), ficando com a mutação se ela melhorar. Treine com `MLPClassifier(max_iter=100)` e avalie na validação; mostre a melhor nota média e o desvio de cada método.

Pergunta: qual método foi melhor? A diferença é maior que o desvio entre sementes?

Exercício 4 — Quanto se pode confiar numa avaliação barata?

NÍVEL: AVANÇADO

No conjunto **Digits**, sorteie 30 arquiteturas (como no Exercício 2) e meça a acurácia de validação com **5**, **15** e **60** iterações (épocas). (a) Calcule a correlação de Spearman de 5 e de 15 iterações com 60. (b) Simule um *Successive Halving* de duas rodadas: avalie todas com 5 iterações, mantenha as 10 melhores, reavalie-as com 60 e escolha a melhor. A escolha coincide com a melhor das 30 com 60 iterações? Qual a fração do custo (em iterações) em relação a treinar todas com 60?

Pergunta: o que os resultados dizem sobre o perigo de cortar candidatas cedo demais?

SAIBA MAIS **Para ir além: bases públicas da UCI e do Kaggle**

- **UCI:** Wine Quality, Spambase e Optical Recognition of Handwritten Digits (a base original do Digits).
- **Kaggle:** Fashion-MNIST e CIFAR-10 (imagens), para experimentar NAS com redes convolucionais e o Auto-Keras.
- **Desafio:** troque o hill climbing pela evolução regularizada do capítulo (população de 8, sorteio de 3) e compare, com várias sementes, em uma base maior.

Estes conjuntos não foram usados nem testados nos códigos do livro. Confira, nos sites, a descrição, a licença e as colunas antes de usá-los.

Projeto integrador do capítulo

Uma rede enxuta por busca de arquitetura

Fonte dos dados. *Breast Cancer Wisconsin (Diagnostic)* (UCI, via scikit-learn), com duas alterações **simuladas por você** para tornar o problema mais difícil: 70 colunas de ruído e 10% de rótulos trocados ao acaso.

Contexto. Uma equipe quer uma rede neural pequena o bastante para rodar em um dispositivo simples. A meta não é só acertar: é acertar com **poucos parâmetros**.

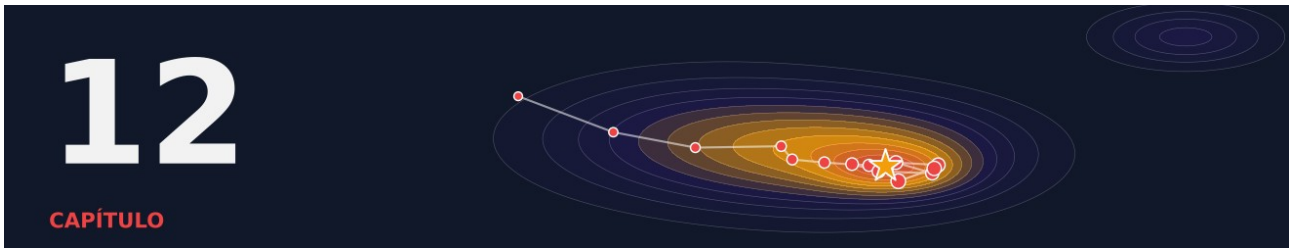
Atenção ética. Os dados são de uso didático e foram deliberadamente alterados. O modelo **não é diagnóstico médico**.

Tarefas.

1. Monte o conjunto, separe 25% para **teste final** (estratificado) e, do restante, 30% para **validação**. Padronize com `StandardScaler` ajustado só no treino.
2. Defina um espaço em cadeia: 1 a 3 camadas ocultas, larguras em {4, 8, 16, 32, 64} e ativação `relu` ou `tanh`. Calcule o tamanho do espaço.
3. Rode duas estratégias com orçamento de **40 arquiteturas**, em 3 sementes: busca aleatória e *hill climbing* (como no Exercício 3).
4. Registre, para todas as arquiteturas avaliadas, a acurácia de validação e o número de parâmetros.
5. Construa a **fronteira de Pareto** (acurácia × parâmetros) com tudo o que foi avaliado.
6. Escolha a **menor rede dentro de 1 ponto percentual** da melhor acurácia de validação e meça-a **uma única vez** no teste. Compare com a rede de maior acurácia de validação.
7. Escreva um parágrafo (8 a 10 linhas) discutindo: qual estratégia foi melhor, quanto se economizou em parâmetros e o que o tamanho do teste permite concluir.

Entregáveis. Um notebook (ou script) comentado, a tabela de resultados, a fronteira de Pareto (gráfico ou lista) e o parágrafo.

A solução de referência está no Apêndice E.



Capítulo 12 — AutoML para séries temporais, texto e imagens

OBJETIVOS DE APRENDIZAGEM

Ao final deste capítulo você será capaz de:

- Explicar o que muda no AutoML quando os dados têm ordem no tempo, são textos ou são imagens.
- Validar modelos de séries temporais sem "olhar o futuro" (validação temporal).
- Construir um mini-AutoML para previsão com atrasos (lags), diferenciação e busca de configuração.
- Buscar automaticamente a representação de um texto (n-gramas e classificador).
- Buscar uma política de aumento de dados (data augmentation) para imagens.
- Reconhecer o papel de modelos pré-treinados e de redes profundas nesses domínios.

12.1 Além das tabelas

Até aqui, quase tudo foi feito com **tabelas**: cada linha é um exemplo independente e cada coluna é um atributo. Mas muitos problemas reais não têm essa cara.

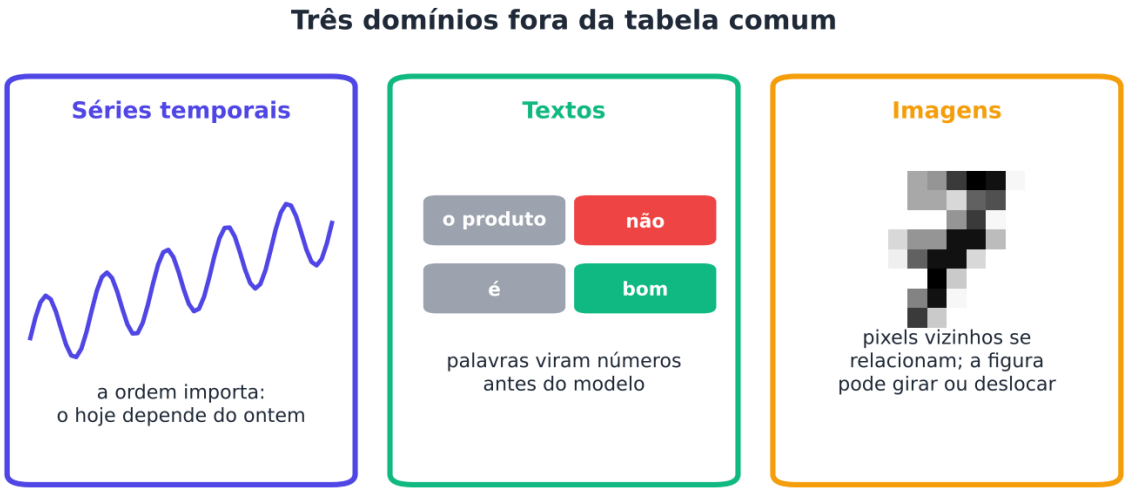


Figura 12.1 — Três domínios fora da tabela comum: séries temporais, textos e imagens.

- **Séries temporais:** valores medidos ao longo do tempo (vendas, temperatura, consumo de energia). A **ordem importa**: o hoje depende do ontem.
- **Textos:** frases e documentos. É preciso **transformar palavras em números** antes de usar qualquer modelo.

- **Imagens:** grades de pixels. Pontos vizinhos são relacionados, e a mesma figura pode aparecer deslocada ou girada.

A boa notícia é que **os princípios do AutoML não mudam**: definir um espaço de busca, avaliar com honestidade e controlar o orçamento. O que muda são as **armadilhas** e os **componentes** do espaço.

> NO SEU DIA A DIA **Analogia: três tipos de ingredientes**
>
> Na cozinha, o mesmo cozinheiro (o AutoML) lida com ingredientes diferentes: o **peixe** exige um preparo (série temporal: respeitar a ordem), o **grão** outro (texto: transformar em números) e a **fruta** outro (imagem: cuidado com a forma). A técnica de provar e ajustar é a mesma; o **preparo** muda.

12.2 Séries temporais: o tempo não volta

12.2.1 O que muda

Em uma série temporal, três cuidados são essenciais.

1. **Validação temporal.** Em dados de tabela, embaralhamos os exemplos e dividimos em partes. Em séries, isso **vaza o futuro**: o modelo treina com valores posteriores aos que vai prever. A avaliação correta usa sempre **passado para treinar e futuro para testar**, em janelas que avançam no tempo (Bergmeir; Benítez, 2012).
2. **Linhas de base fortes.** Prever "o valor de amanhã é igual ao de hoje" (**previsão ingênua**) é difícil de superar em muitas séries. Outra linha de base é a **sazonal ingênua**: "o valor deste mês é igual ao do mesmo mês do ano passado".
3. **Tendência e sazonalidade.** Séries podem **crescer** ao longo do tempo (**tendência**) e **repetir padrões** (como o verão, **sazonalidade**). Alguns modelos, como árvores, **não extrapolam**: não conseguem prever valores fora do que viram no treino.

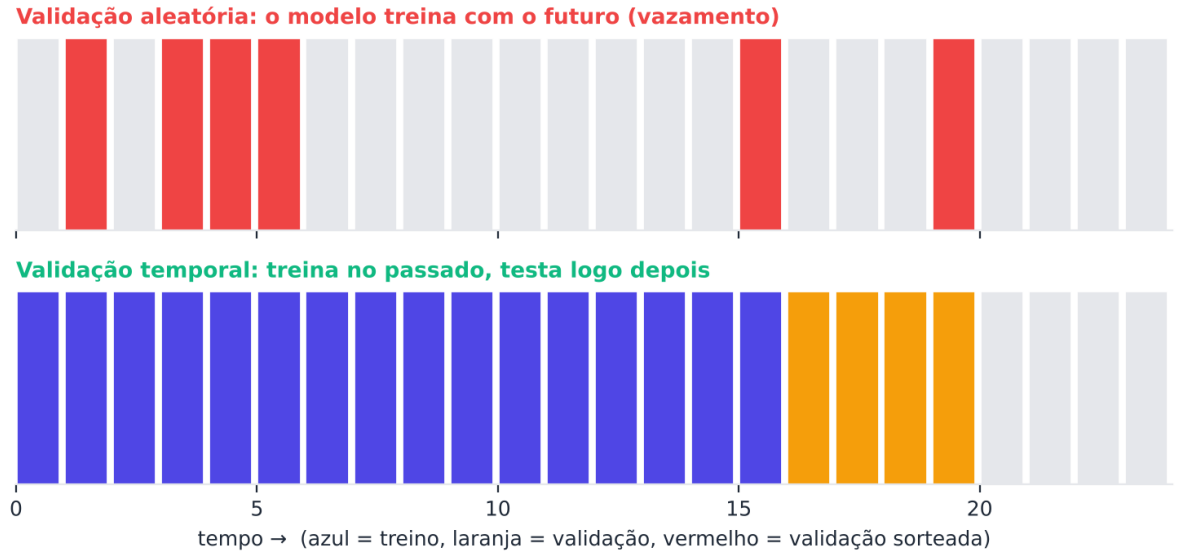


Figura 12.2 — Validação aleatória (vaza o futuro) e validação temporal (janela que avança).

12.2.2 Prever com modelos de tabela: atrasos (lags)

Podemos usar os modelos que já conhecemos transformando a série em **tabela**: para prever o valor de hoje, usamos como atributos os **valores dos últimos L meses** (os **atrasos**, ou *lags*) e o **mês do ano**.

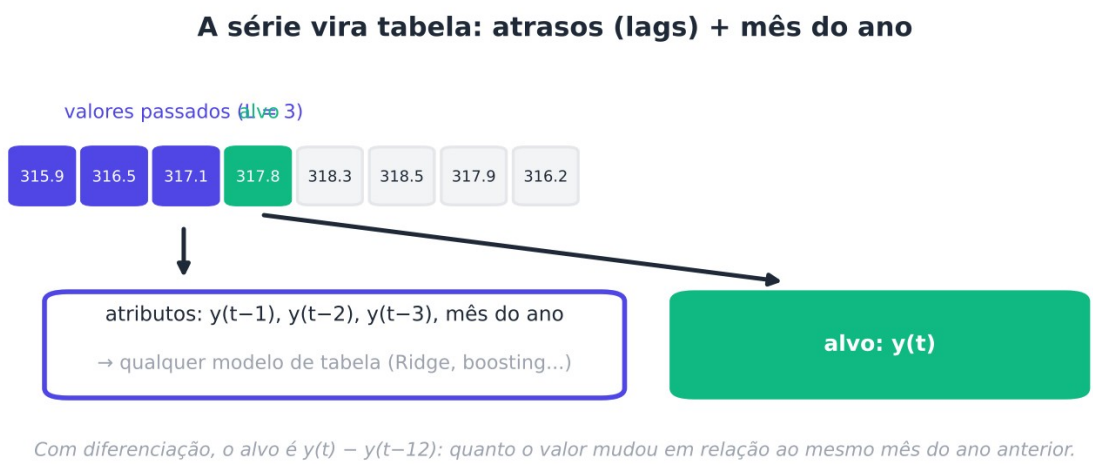


Figura 12.3 — A série vira tabela: os últimos L valores e o mês do ano são os atributos; o valor seguinte é o alvo.

Vamos usar a série de **CO_2 atmosférico medido em Mauna Loa** (dados incluídos na biblioteca `statsmodels`), de 1958 a 2001, resumida em médias **mensais** (526 meses). O CO_2 é medido em **partes por milhão (ppm)**. Ela tem **tendência de crescimento** e **sazonalidade anual** bem marcadas. Reservamos os **últimos 36 meses** para o teste.

Primeiro, uma demonstração da armadilha da validação. Medimos o erro absoluto médio (**MAE**) de dois modelos com validação aleatória e com validação temporal, usando apenas a parte de treino. O primeiro é uma regressão **Ridge** (regressão linear com uma penalidade que estabiliza os coeficientes), com a série **diferenciada** (veremos adiante). O segundo é uma **floresta de boosting** (HistGradientBoosting) sobre os valores brutos.

```python
import warnings; warnings.filterwarnings("ignore")
import numpy as np
import statsmodels.api as sm
from sklearn.linear_model import Ridge
from sklearn.ensemble import HistGradientBoostingRegressor
from sklearn.model_selection import TimeSeriesSplit, KFold

serie = sm.datasets.co2.load_pandas().data["co2"].interpolate().resample("MS").mean()
y = serie.values; n = len(y); H = 36
print("Meses na série:", n, "| período:", serie.index[0].strftime("%m/%Y"), "a", serie.index[-
1].strftime("%m/%Y"))

def monta(y, L, dif):
    X, alvo, base = [], [], []
    for t in range(max(L, 12), len(y)):
        b = y[t - 12] if dif else 0.0                    # com dif=True, prevemos a diferença para 12
meses antes
        X.append(np.r_[y[t - L:t][::-1] - b, (np.arange(12) == t % 12)]); alvo.append(y[t] - b);
base.append(b)
    return np.array(X), np.array(alvo), np.array(base)

MODELOS = {"ridge": lambda: Ridge(1.0), "hgb": lambda: HistGradientBoostingRegressor(max_iter=100,
random_state=0)}

def mae_cv(cfg, cv, ate):
    mod, L, dif = cfg; X, a, b = monta(y[:ate], L, dif); e = []
    for tr, va in cv.split(X):
        m = MODELOS[mod]().fit(X[tr], a[tr]); e.append(np.abs(m.predict(X[va]) - a[va]).mean())
```

```
    return np.mean(e)

for cfg in (("ridge", 12, True), ("hgb", 12, False)):
    a = mae_cv(cfg, KFold(5, shuffle=True, random_state=0), n - H)
    t = mae_cv(cfg, TimeSeriesSplit(5, test_size=12), n - H)
    print(f"{cfg[0]:<5s}: MAE com validação aleatória = {a:.3f} | com validação temporal = {t:.3f}")
```

SAÍDA

```
Meses na série: 526 | período: 03/1958 a 12/2001
ridge: MAE com validação aleatória = 0.248 | com validação temporal = 0.295
hgb  : MAE com validação aleatória = 0.562 | com validação temporal = 1.707
```

A validação aleatória é **otimista** nos dois casos, e **muito** no segundo: o modelo de árvores parece bom porque, no embaralhamento, ele sempre tem exemplos "vizinhos" no tempo (passado e futuro). Quando a avaliação respeita o tempo, ele precisa **extrapolar a tendência**, e não consegue. Isso é uma versão do **vazamento de dados** que já conhecemos (Capítulo 3; Kapoor; Narayanan, 2023).

12.2.3 Um mini-AutoML para previsão

Vamos montar um espaço de busca simples, no estilo do problema CASH:

Dimensão	Opções
Modelo	Ridge, HistGradientBoosting
Atrasos (L)	6, 12 ou 24 meses
Diferenciação	sim (prevê a diferença para 12 meses antes) ou não (prevê o valor bruto)

São 2 × 3 × 2 = **12 configurações**. A **diferenciação** (*differencing*) é um truque clássico para remover tendência e sazonalidade: em vez de prever o valor, prevê-se **quanto ele muda** em relação ao mesmo mês do ano anterior. A diferenciação é um dos recursos clássicos dos modelos ARIMA (Hyndman; Athanasopoulos, 2021); Hyndman e Khandakar (2008) mostram como automatizar essa escolha (e outras) em um método automático de previsão.

A seleção usa validação temporal com 4 janelas de 12 meses sobre a parte de treino. Depois, a escolhida é avaliada no teste de forma **realista**: a cada mês dos últimos 36, o modelo é **reajustado** com todo o passado disponível e prevê **um mês à frente** (validação em avanço, ou *walk-forward*). Comparamos com as linhas de base ingênua e sazonal ingênua.

PYTHON

```
import warnings; warnings.filterwarnings("ignore")
import itertools, json
import numpy as np
import statsmodels.api as sm
from sklearn.linear_model import Ridge
from sklearn.ensemble import HistGradientBoostingRegressor
from sklearn.model_selection import TimeSeriesSplit

serie = sm.datasets.co2.load_pandas().data["co2"].interpolate().resample("MS").mean()
y = serie.values; n = len(y); H = 36

def monta(y, L, dif):
    X, alvo, base = [], [], []
    for t in range(max(L, 12), len(y)):
        b = y[t - 12] if dif else 0.0
```

```
        X.append(np.r_[y[t - L:t][::-1] - b, (np.arange(12) == t % 12)]); alvo.append(y[t] - b);
base.append(b)
    return np.array(X), np.array(alvo), np.array(base)

MODELOS = {"ridge": lambda: Ridge(1.0), "hgb": lambda: HistGradientBoostingRegressor(max_iter=100,
random_state=0)}

def mae_temporal(cfg):
    mod, L, dif = cfg; X, a, b = monta(y[:n - H], L, dif); e = []
    for tr, va in TimeSeriesSplit(4, test_size=12).split(X):
        e.append(np.abs(MODELOS[mod]().fit(X[tr], a[tr]).predict(X[va]) - a[va]).mean())
    return np.mean(e)

def walk_forward(cfg):
    mod, L, dif = cfg; prev = []
    for t in range(n - H, n):
        X, a, _ = monta(y[:t], L, dif); m = MODELOS[mod]().fit(X, a)
        b = y[t - 12] if dif else 0.0
        x = np.r_[y[t - L:t][::-1] - b, (np.arange(12) == t % 12)]
        prev.append(m.predict(x[None])[0] + b)
    return np.array(prev)

configs = list(itertools.product(MODELOS, (6, 12, 24), (False, True)))
rank = sorted((mae_temporal(c), c) for c in configs)
print("Validação temporal (MAE), melhores e piores configurações:")
for e, c in rank[:3] + rank[-3:]: print(f"  {e:6.3f}  modelo={c[0]:<5s} lags={c[1]:>2d}
diferenciação={'sim' if c[2] else 'não'}")

escolhida = rank[0][1]; real = y[n - H:]
prev = walk_forward(escolhida)
ing = y[n - H - 1:n - 1]; saz = y[n - H - 12:n - 12]
print("\nTESTE (últimos 36 meses, 1 mês à frente), MAE:")
print(f"  mini-AutoML ({escolhida[0]}, lags={escolhida[1]}, dif={'sim' if escolhida[2] else 'não'}) :
{np.abs(prev - real).mean():.3f}")
print(f"  ingênua (valor do mês anterior)              : {np.abs(ing - real).mean():.3f}")
print(f"  sazonal ingênua (mesmo mês do ano anterior)  : {np.abs(saz - real).mean():.3f}")
pior = rank[-1][1]; prev_p = walk_forward(pior)
print(f"  pior configuração da busca ({pior[0]}, lags={pior[1]}, dif={'sim' if pior[2] else 'não'}) :
{np.abs(prev_p - real).mean():.3f}")
json.dump({"hist": y[n - H - 36:n - H].tolist(), "real": real.tolist(), "auto": prev.tolist(), "ing":
ing.tolist(), "pior": prev_p.tolist()},
          open("/home/user/claude/automl/build/f12_ts.json", "w"))
```

SAÍDA

```
Validação temporal (MAE), melhores e piores configurações:
   0.257  modelo=ridge lags=24 diferenciação=não
   0.266  modelo=ridge lags= 6 diferenciação=não
   0.278  modelo=ridge lags=24 diferenciação=sim
   1.777  modelo=hgb   lags=24 diferenciação=não
   1.844  modelo=hgb   lags=12 diferenciação=não
   2.109  modelo=hgb   lags= 6 diferenciação=não

TESTE (últimos 36 meses, 1 mês à frente), MAE:
  mini-AutoML (ridge, lags=24, dif=não) : 0.227
  ingênua (valor do mês anterior)              : 1.077
  sazonal ingênua (mesmo mês do ano anterior)  : 1.413
  pior configuração da busca (hgb, lags=6, dif=não) : 1.399
```

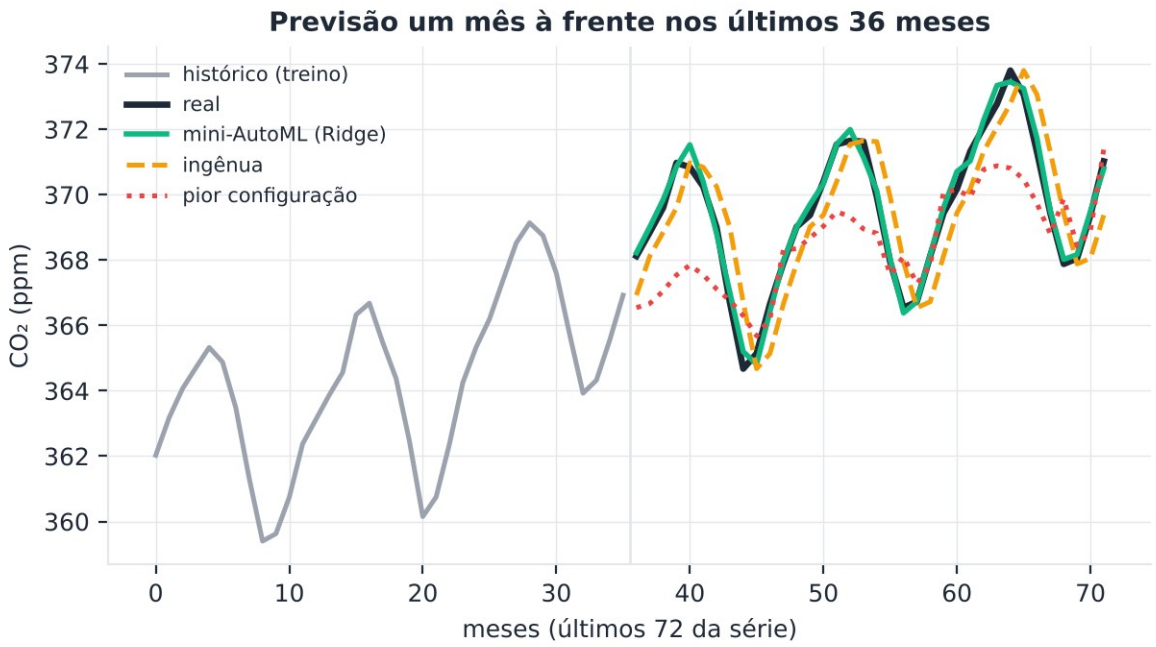


Figura 12.4 — Previsão um mês à frente nos últimos 36 meses: mini-AutoML, linha de base ingênua e a pior configuração da busca.

Duas lições. Primeiro, **a busca bem-feita importa**: a melhor configuração (Ridge com 24 atrasos) errou 0,227 ppm em média no teste, contra 1,399 da pior (floresta de boosting com 6 atrasos e valores brutos, que não extrapola a tendência): cerca de seis vezes mais. Note também que a busca escolheu **Ridge sem diferenciação**: como o Ridge é linear, ele consegue extrapolar a tendência por conta própria, e a diferenciação ajudou o boosting, mas não era indispensável ao modelo linear. Segundo, **a linha de base é indispensável**: vale conferir se o modelo realmente supera a previsão ingênua. Aqui o ganho foi claro (0,227 contra 1,077 da ingênua e 1,413 da sazonal ingênua) porque a série tem sazonalidade e tendência muito regulares; em séries mais ruidosas, a vantagem costuma diminuir.

> **Atenção: previsões de um passo e de vários passos**
>
> Prever **um mês à frente**, reajustando todo mês, é mais fácil do que prever os **12 próximos meses de uma vez**, sem ver os valores intermediários. Os erros se acumulam com o horizonte. Sempre diga qual horizonte você avaliou.

12.2.4 O que a literatura diz

A competição M4, com 100 mil séries e 61 métodos de previsão, é a grande referência de comparação na área, e seus resultados favoreceram abordagens que **combinam** métodos (Makridakis; Spiliotis; Assimakopoulos, 2020), o mesmo espírito dos *ensembles* do Capítulo 9. Em produção, há ainda o problema da **deriva de conceito** (*concept drift*): o padrão da série muda com o tempo, e o modelo envelhece (Gama *et al.*, 2014), tema do Capítulo 14. Várias ferramentas de AutoML já oferecem módulos para séries temporais; consulte a documentação da versão que você usar.

12.3 Texto: transformar palavras em números

12.3.1 O pipeline de texto

Um modelo de aprendizado de máquina clássico não lê palavras: lê **números**. Para classificar textos (por exemplo, "esta avaliação é positiva ou negativa?"), usamos um **pipeline** (Capítulo 3) com duas partes.

1. **Vetorização:** transformar cada texto em um vetor de números. A técnica clássica é o **TF-IDF** (frequência do termo × inverso da frequência nos documentos): cada palavra recebe um peso que é **alto** se aparece muito no documento e **baixo** se aparece em quase todos os documentos (como "o" ou "de").
2. **Classificador:** um modelo (regressão logística, SVM linear, Naive Bayes) que aprende com os vetores.

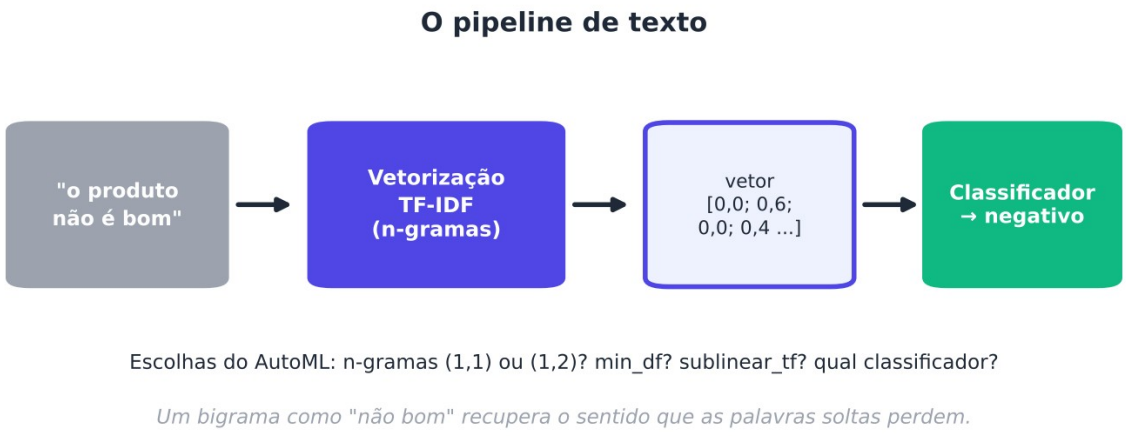


Figura 12.5 — O pipeline de texto: o texto vira um vetor TF-IDF e, depois, um classificador decide.

O espaço de busca do AutoML inclui escolhas da vetorização: usar só **palavras isoladas** (*unigramas*) ou também **pares e trios de palavras** (*n-gramas*)? Ignorar palavras muito raras (`min_df`)? Suavizar a contagem por logaritmo (`sublinear_tf`)?

> NO SEU DIA A DIA **Analogia: ler frases ou ler palavras soltas**
>
> Imagine ler apenas **as palavras soltas** de uma avaliação: "não", "bom". Você não sabe se é "não é bom" ou "é bom, não é?". Ler **pares de palavras** ("não bom") recupera o sentido. Os n-gramas fazem isso.

12.3.2 Um experimento com texto simulado

Não há acesso a bases de texto públicas no ambiente em que o livro foi preparado, e por isso vamos usar **avaliações simuladas** de produtos, geradas por um programa que você pode inspecionar. Cada frase tem a forma "o produto foi **(não)** bom", com adjetivos positivos e negativos, e **40% das frases têm negação** ("não ruim" é positivo). É um problema artificial, **de propósito**: ele exagera um fenômeno real (negação) para o efeito ficar visível. Os números **não** representam o desempenho em textos reais.

Vamos varrer o espaço (n-gramas × classificador) por validação cruzada no treino e medir o teste final com a melhor escolha.

PYTHON
```python
import warnings; warnings.filterwarnings("ignore")
import itertools
import numpy as np
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.pipeline import make_pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.svm import LinearSVC
from sklearn.naive_bayes import MultinomialNB
from sklearn.model_selection import cross_val_score, train_test_split

r = np.random.default_rng(0)
```

```
POS = ["ótimo", "bom", "excelente", "rápido", "perfeito", "útil"]; NEG = ["ruim", "péssimo", "lento",
"fraco", "horrível", "inútil"]
SUJ = ["o produto", "a entrega", "o atendimento", "o aplicativo", "a embalagem"]; VERB = ["é", "foi",
"achei", "parece"]
FIL = ["no geral", "sinceramente", "para mim", "nesta semana", "de verdade", "como esperado"]
def gera(n):
    docs, ys = [], []
    for _ in range(n):
        pos = r.random() < 0.5; adj = r.choice(POS if pos else NEG); neg = r.random() < 0.4
        if neg: pos = not pos                                    # a negação inverte o sentido
        f = f"{r.choice(SUJ)} {r.choice(VERB)} {'não ' if neg else ''}{adj}"
        if r.random() < 0.6: f = r.choice(FIL) + " " + f
        docs.append(f); ys.append(int(pos))
    return docs, np.array(ys)

D, Y = gera(1200)
Dtr, Dte, ytr, yte = train_test_split(D, Y, test_size=0.25, stratify=Y, random_state=0)
print("Exemplos de treino:")
for d, l in list(zip(Dtr, ytr))[:3]: print(f"  [{'positivo' if l else 'negativo'}] {d}")

CLASS = {"LogReg": lambda: LogisticRegression(max_iter=2000), "SVM linear": lambda: LinearSVC(), "Naive
Bayes": lambda: MultinomialNB()}
res = []
for ng, cl, sub in itertools.product([(1, 1), (1, 2), (1, 3)], CLASS, [False, True]):
    p = make_pipeline(TfidfVectorizer(ngram_range=ng, sublinear_tf=sub), CLASS[cl]())
    res.append((cross_val_score(p, Dtr, ytr, cv=5).mean(), ng, cl, sub))
res.sort(reverse=True)
print("\nValidação cruzada (acurácia), melhores e piores de 18 configurações:")
for a, ng, cl, sub in res[:3] + res[-3:]: print(f"  {a:.3f}  n-gramas={ng}  {cl:<11s}
sublinear={sub}")
a, ng, cl, sub = res[0]
p = make_pipeline(TfidfVectorizer(ngram_range=ng, sublinear_tf=sub), CLASS[cl]()).fit(Dtr, ytr)
p1 = make_pipeline(TfidfVectorizer(), LogisticRegression(max_iter=2000)).fit(Dtr, ytr)
print(f"\nTESTE com a melhor configuração : {p.score(Dte, yte):.3f}")
print(f"TESTE com o pipeline padrão     : {p1.score(Dte, yte):.3f}  (palavras isoladas, regressão
logística)")
```

SAÍDA

```
Exemplos de treino:
  [positivo] no geral a embalagem foi não fraco
  [negativo] a embalagem parece inútil
  [positivo] sinceramente a embalagem achei não lento

Validação cruzada (acurácia), melhores e piores de 18 configurações:
  1.000  n-gramas=(1, 2)  SVM linear   sublinear=True
  1.000  n-gramas=(1, 2)  SVM linear   sublinear=False
  0.999  n-gramas=(1, 3)  SVM linear   sublinear=True
  0.551  n-gramas=(1, 1)  LogReg       sublinear=False
  0.544  n-gramas=(1, 1)  Naive Bayes  sublinear=True
  0.544  n-gramas=(1, 1)  Naive Bayes  sublinear=False

TESTE com a melhor configuração : 1.000
TESTE com o pipeline padrão     : 0.607  (palavras isoladas, regressão logística)
```

O resultado é um retrato claro de **por que a representação importa**. Com palavras isoladas, o modelo fica **perto do acaso** (cerca de 55% na validação cruzada e 61% no teste): ele vê "não" e "bom", mas não sabe a relação

entre as duas. Com **bigramas**, a acurácia se aproxima de 100%. Nenhum classificador sofisticado compensaria uma representação que perdeu a informação essencial.

12.3.3 Modelos pré-treinados e a era dos Transformers

Em textos reais, o estado da arte mudou com os **modelos de linguagem pré-treinados**, como o BERT (Devlin *et al.*, 2019), baseado na arquitetura Transformer de atenção (Vaswani *et al.*, 2017). A ideia é aproveitar um modelo treinado em enormes volumes de texto e fazer um **ajuste fino** (*fine-tuning*) para a sua tarefa. Para o AutoML, isso muda o espaço de busca: em vez de escolher n-gramas, escolhem-se **qual modelo pré-treinado usar** e **como ajustá-lo** (taxa de aprendizado, número de épocas). A lógica de busca, validação e orçamento é a mesma.

12.4 Imagens: formas, deslocamentos e dados escassos

12.4.1 O que muda com imagens

Uma imagem é uma **grade de pixels**. Em imagens, a posição importa e a mesma figura pode aparecer **deslocada, girada ou com ruído**. Redes convolucionais e arquiteturas modernas, como as residuais (He *et al.*, 2016), são o padrão para imagens grandes; escolher arquitetura e escala é o território da busca de arquitetura (Capítulo 11) e de famílias eficientes como a EfficientNet (Tan; Le, 2019).

Mas existe uma decisão de AutoML **mais barata e muito eficaz**: a **ampliação de dados** (*data augmentation*). Ela consiste em criar **novas imagens de treino** aplicando pequenas distorções (deslocar, girar, adicionar ruído) às imagens existentes. O modelo aprende que um "7" deslocado de um pixel **continua sendo um 7**.



Figura 12.6 — Ampliação de dados: um mesmo dígito, com deslocamento, rotação e ruído, vira vários exemplos de treino.

O AutoAugment (Cubuk *et al.*, 2019) mostrou que **as próprias políticas de ampliação** podem ser **aprendidas por busca**, em vez de escolhidas à mão: o espaço é "quais distorções aplicar e com que intensidade".

12.4.2 Um mini-AutoAugment

Vamos fazer uma versão **minúscula** da ideia no conjunto **Digits** (UCI, via scikit-learn). Para deixar o efeito visível, criamos duas condições deliberadas: o treino tem **só 300 imagens** (escassez de dados) e o teste inclui imagens **deslocadas até 1 pixel e giradas até 10 graus** (uma mudança de condição realista, como uma câmera um pouco torta). O espaço de busca é uma **política** de três números: deslocamento máximo (0 a 2 pixels), rotação máxima (0 a 25 graus) e ruído (0 a 2). Sorteamos **15 políticas**, geramos 4 cópias distorcidas de cada imagem de treino, treinamos uma SVM e escolhemos a política pela **validação** (também distorcida, com outra semente). O teste final é medido uma única vez.

```python
import warnings; warnings.filterwarnings("ignore")
import numpy as np
from scipy.ndimage import shift, rotate
```

```
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC

X, y = load_digits(return_X_y=True)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.4, stratify=y, random_state=0)
Xtr, ytr = Xtr[:300], ytr[:300]                                   # poucos dados de treino
Xa, Xv, ya, yv = train_test_split(Xtr, ytr, test_size=0.3, stratify=ytr, random_state=0)

def distorce(Xs, r, dmax, rot, ruido):
    saida = []
    for x in Xs:
        im = shift(x.reshape(8, 8), r.uniform(-dmax, dmax, 2), order=1, mode="constant")
        if rot > 0: im = rotate(im, r.uniform(-rot, rot), reshape=False, order=1, mode="constant")
        saida.append((im + r.normal(0, ruido, im.shape)).ravel())
    return np.array(saida)

Xte_d = distorce(Xte, np.random.default_rng(123), 1.0, 10, 0.0)            # teste deslocado/girado
Xv_d = distorce(Xv, np.random.default_rng(5), 1.0, 10, 0.0)                # validação com a mesma
natureza

def treina(pol, k=4, seed=0):
    r = np.random.default_rng(seed); dmax, rot, ru = pol; Xs, ys = [Xa], [ya]
    if max(pol) > 0:
        for _ in range(k): Xs.append(distorce(Xa, r, dmax, rot, ru)); ys.append(ya)
    return SVC(C=10, gamma="scale").fit(np.vstack(Xs), np.concatenate(ys))

m0 = treina((0, 0, 0))
print(f"Sem ampliação      : validação distorcida = {m0.score(Xv_d, yv):.3f} | teste limpo =
{m0.score(Xte, yte):.3f} | teste distorcido = {m0.score(Xte_d, yte):.3f}")

r = np.random.default_rng(0); melhor = None; todas = []
for _ in range(15):
    pol = (round(float(r.uniform(0, 2)), 2), round(float(r.uniform(0, 25)), 1),
round(float(r.uniform(0, 2)), 2))
    m = treina(pol); v = m.score(Xv_d, yv); todas.append(v)
    if melhor is None or v > melhor[0]: melhor = (v, pol, m)
v, pol, m = melhor
print(f"Políticas testadas : validação de {min(todas):.3f} a {max(todas):.3f}")
print(f"Melhor política    : deslocamento={pol[0]}, rotação={pol[1]}°, ruído={pol[2]}")
print(f"Com a melhor política: validação distorcida = {v:.3f} | teste limpo = {m.score(Xte, yte):.3f} |
teste distorcido = {m.score(Xte_d, yte):.3f}")
```

SAÍDA

```
Sem ampliação      : validação distorcida = 0.544 | teste limpo = 0.962 | teste distorcido = 0.529
Políticas testadas : validação de 0.633 a 0.856
Melhor política    : deslocamento=1.44, rotação=13.1°, ruído=0.62
Com a melhor política: validação distorcida = 0.856 | teste limpo = 0.939 | teste distorcido = 0.873
```

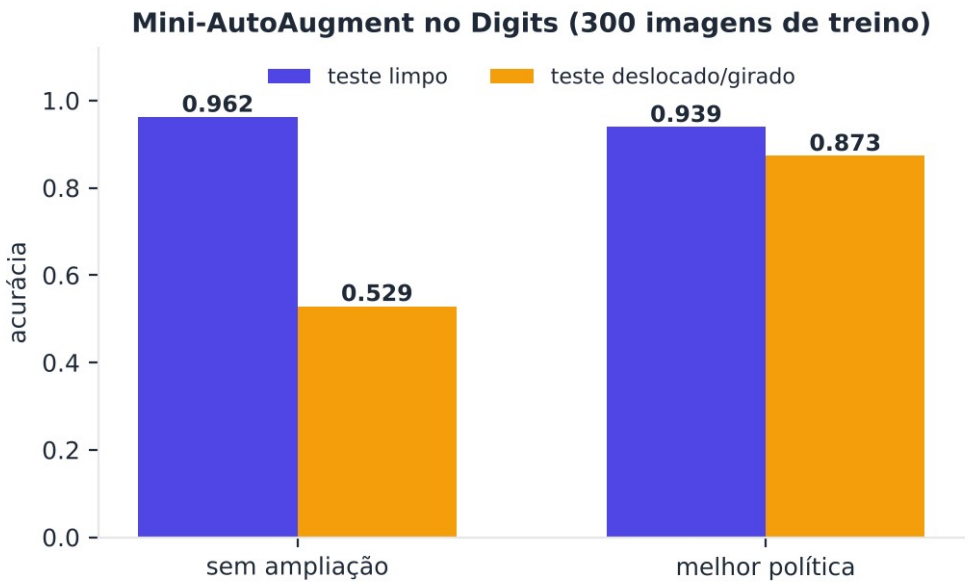


Figura 12.7 — Efeito da melhor política de ampliação: grande ganho sob distorção, pequena perda no teste limpo.

O resultado conta uma história honesta e instrutiva. Sob **distorção**, a política escolhida pela busca **recupera boa parte do desempenho** perdido. No teste **limpo**, há uma **pequena perda**: a ampliação não é de graça, porque o modelo passa a dividir sua capacidade entre as imagens originais e as distorcidas. O que a busca "sabe" depende do que a **validação** contém: como pusemos distorções na validação, ela premiou políticas robustas a distorção. Se a validação fosse limpa, a política escolhida seria outra. **A validação precisa se parecer com o uso real.**

12.4.3 Transferência e modelos pré-treinados

Com poucas imagens, treinar uma rede profunda do zero costuma falhar. A prática moderna é usar uma rede **pré-treinada** em um enorme conjunto de imagens e **reajustá-la** para o seu problema (**aprendizado por transferência**). Ferramentas como o Auto-Keras (Jin; Song; Hu, 2019), mencionado no Capítulo 11, automatizam parte dessa escolha. Em AutoML de imagens, as perguntas do espaço de busca passam a ser: **qual rede-base**, **quantas camadas reajustar**, **qual política de ampliação** e **qual taxa de aprendizado**.

12.5 Quadro-resumo



Figura 12.8 — O que muda em cada domínio e o que permanece igual.

Domínio	Armadilha principal	Componente do espaço de busca	Linha de base
Séries temporais	Vazar o futuro na validação	Atrasos, diferenciação, modelo	Ingênua e sazonal ingênua
Texto	Perder a informação na vetorização	N-gramas, pesos, classificador (ou modelo pré-treinado)	TF-IDF + regressão logística
Imagens	Poucos dados; mudanças de condição	Política de ampliação, rede-base, taxa de aprendizado	Modelo simples sem ampliação

12.6 Boas práticas

1. **Respeite a natureza dos dados:** em séries, validação temporal sempre; em texto e imagens, cuidado com a representação.
2. **Compare com linhas de base:** ingênua (séries), TF-IDF com regressão logística (texto) e modelo sem ampliação (imagens).
3. **Faça a validação se parecer com o uso real:** inclua as variações que o modelo encontrará (deslocamentos, sazonalidade nova).
4. **Diga o horizonte** (séries) e **o tipo de teste** (limpo ou distorcido).
5. **Busque a representação antes do modelo:** n-gramas, diferenciação e ampliação costumam valer mais que trocar de classificador.
6. **Prefira modelos pré-treinados** quando houver poucos dados em texto ou imagens.
7. **Não confie em dados simulados para concluir sobre o mundo real:** usamos texto simulado para ensinar um fenômeno, não para medir desempenho.
8. **Monitore a deriva** depois da implantação (Capítulo 14).

Resumo do capítulo

MEMORIZE **Cartões de memorização**

- **Séries temporais:** validação temporal (passado treina, futuro testa); nunca embaralhar.
- **Linhas de base:** ingênua e sazonal ingênua.
- **Atrasos (lags) e diferenciação** transformam a série em tabela e removem tendência; árvores não extrapolam tendência.
- **Validação aleatória** em série é otimista, e muito, para modelos que não extrapolam.
- **Texto:** TF-IDF + classificador; n-gramas capturam relações entre palavras (como a negação).
- **Representação** pesa mais que o classificador: palavras isoladas ficaram perto do acaso em nosso exemplo simulado.
- **Imagens:** ampliação de dados (deslocar, girar, ruído) e busca de políticas (AutoAugment).
- **A validação deve se parecer com o uso real:** ela define o que a busca valoriza.
- **Modelos pré-treinados** (BERT, redes de imagem) mudam o espaço de busca, não a lógica do AutoML.

Exercícios de múltipla escolha

Escolha **uma** alternativa em cada questão. O gabarito está no Apêndice A.

Questão 1. Qual é a principal armadilha do AutoML em séries temporais?

a) Usar poucos modelos.
b) Validar com embaralhamento, vazando o futuro para o treino.
c) Usar a semente 0.
d) Usar muitos atributos.

Questão 2. O que é a previsão ingênua?

a) Prever sempre zero.
b) Prever que o próximo valor é igual ao valor atual (mês anterior).
c) Prever o valor máximo.
d) Prever a média do futuro.

Questão 3. O que é a previsão sazonal ingênua?

a) Prever a média de todos os meses.
b) Prever um valor aleatório.
c) Prever o valor do mês igual ao do mesmo mês do ano anterior.
d) Prever o valor do mês anterior.

Questão 4. O que é sazonalidade?

a) O número de atributos.
b) Um erro de medição.
c) O crescimento contínuo da série.
d) Um padrão que se repete em períodos regulares, como o verão.

Questão 5. O que é tendência em uma série temporal?

a) Um tipo de modelo.
b) A média das previsões.
c) Um crescimento ou queda de longo prazo.
d) Um padrão que se repete todo ano.

Questão 6. O que são atrasos (lags)?

a) Dados de teste.
b) Erros de medição.
c) Valores passados da série usados como atributos.
d) Parâmetros do modelo.

Questão 7. O que é diferenciação em séries temporais?

a) Prever a mudança em relação a um valor anterior, em vez do valor bruto.
b) Remover outliers.
c) Normalizar a série.
d) Dividir a série em treino e teste.

Questão 8. Por que árvores de decisão e boosting têm dificuldade com tendências?

a) Porque exigem muitas camadas.
b) Porque ignoram atributos.
c) Porque são muito lentas.
d) Porque não extrapolam: não preveem valores fora do que viram no treino.

Questão 9. Qual validação é correta para séries temporais?

a) KFold com embaralhamento.
b) Treinar com o teste.
c) Escolher o modelo pelo teste.
d) Validação temporal, com janelas que avançam no tempo.

Questão 10. O que mostrou o experimento de validação com boosting em valores brutos?

a) As duas deram o mesmo valor.
b) O boosting extrapola bem a tendência.
c) A temporal foi melhor.
d) A validação aleatória foi muito mais otimista que a temporal (0,56 contra 1,71).

Questão 11. Qual configuração venceu no mini-AutoML de CO_2?

a) Boosting com 6 atrasos.
b) Ridge com 24 atrasos, sem diferenciação.
c) Floresta com 12 atrasos.
d) Naive.

Questão 12. Qual foi o erro de teste da previsão ingênua na série de CO_2 (últimos 36 meses)?

a) Cerca de 0,23 ppm.
b) Cerca de 5 ppm.
c) Cerca de 1,08 ppm.
d) Zero.

Questão 13. O que é validação em avanço (walk-forward)?

a) Reajustar o modelo a cada novo ponto e prever o seguinte.
b) Treinar uma única vez.
c) Dividir os dados aleatoriamente.
d) Usar só o futuro.

Questão 14. O que a competição M4 comparou?

a) Algoritmos de classificação de texto.
b) Métodos de visão computacional.
c) Mais de 60 métodos de previsão em 100 mil séries.
d) Apenas redes neurais.

Questão 15. O que é a deriva de conceito (concept drift)?

a) A mudança do padrão dos dados ao longo do tempo, que envelhece o modelo.
b) Uma técnica de ensemble.
c) Um erro de programação.
d) Um tipo de atributo.

Questão 16. Por que um texto precisa ser vetorizado?

a) Para eliminar o teste.
b) Porque os modelos clássicos operam sobre números, não sobre palavras.
c) Para reduzir o treino.
d) Porque palavras são proibidas.

Questão 17. O que é o TF-IDF?

a) Um tipo de validação.
b) Um classificador.
c) Um peso que é alto para palavras frequentes no documento e raras nos demais.
d) Um modelo de rede neural.

Questão 18. O que são n-gramas?

a) Sequências de n palavras consecutivas usadas como atributos.
b) Números aleatórios.
c) Classes do problema.
d) Letras isoladas.

Questão 19. Por que bigramas ajudaram tanto no texto simulado do capítulo?

a) Porque tornam o modelo mais lento.
b) Porque a negação ('não bom') inverte o sentido e só aparece na combinação de palavras.
c) Porque removem palavras raras.
d) Porque aumentam o conjunto de teste.

Questão 20. Qual foi o desempenho das palavras isoladas no teste do texto simulado?

a) Cerca de 95%.
b) Cerca de 61%.
c) 100%.
d) 10%.

Questão 21. Por que o texto simulado NÃO serve para medir desempenho real?

a) Porque foi gerado por um programa e exagera um fenômeno para fins didáticos.
b) Porque é muito longo.
c) Porque usa muitas classes.
d) Porque é proibido.

Questão 22. O que é o BERT (Devlin et al., 2019)?

a) Uma base de dados de imagens.
b) Uma métrica.
c) Um modelo de linguagem pré-treinado, ajustado depois para tarefas específicas.
d) Um algoritmo de busca em grade.

Questão 23. O que é ajuste fino (fine-tuning)?

a) Reajustar um modelo pré-treinado para a sua tarefa.
b) Remover atributos.
c) Trocar a métrica.
d) Aumentar o teste.

Questão 24. O que é ampliação de dados (data augmentation) em imagens?

a) Aumentar o tamanho do teste.
b) Criar novas imagens de treino com pequenas distorções das existentes.
c) Remover imagens ruins.
d) Trocar o classificador.

Questão 25. O que o AutoAugment (Cubuk et al., 2019) propõe?

a) Aprender por busca as políticas de ampliação de dados.
b) Substituir redes por árvores.
c) Usar apenas busca em grade.
d) Treinar sem dados.

Questão 26. Qual foi o efeito da melhor política no teste distorcido do mini-AutoAugment?

a) Subiu a 1,0.
b) A acurácia subiu de 0,529 para 0,873.
c) Não mudou.
d) Caiu a 0,3.

Questão 27. Qual foi o custo da ampliação no teste limpo?

a) Ganho de 10 pontos.
b) Uma pequena perda (0,962 para 0,939).
c) Perda de 50%.
d) Nenhuma perda.

Questão 28. Por que o 'ruído' sozinho não ajudou no teste distorcido (Exercício 4)?

a) Porque ruído é proibido.
b) Porque o teste distorcido não continha ruído, apenas deslocamento e rotação.
c) Porque a SVM não aceita ruído.
d) Porque o treino era grande.

Questão 29. Qual é a lição sobre a validação no mini-AutoAugment?

a) Ela não importa.
b) Ela deve ser sempre limpa.
c) Ela pode ser qualquer coisa.
d) Ela deve se parecer com o uso real, pois define o que a busca valoriza.

Questão 30. Qual é a prática moderna quando há poucas imagens?

a) Descartar o problema.
b) Treinar uma rede grande do zero.
c) Usar só regressão linear.
d) Usar uma rede pré-treinada e reajustá-la (aprendizado por transferência).

Exercícios discursivos

Responda com suas palavras, em poucas linhas. Compare depois com a resposta-modelo do Apêndice A.

1. Por que embaralhar uma série temporal na validação é um erro?
2. Explique a validação temporal e a validação em avanço.
3. Cite duas linhas de base para séries temporais e explique por que são necessárias.
4. Como transformar uma série em uma tabela para usar modelos do scikit-learn?
5. O que é diferenciação e por que ajudou o boosting?
6. Por que o boosting sobre valores brutos teve erro tão alto na série de CO_2?
7. Descreva o espaço de busca do mini-AutoML de séries temporais do capítulo.
8. Compare o resultado do mini-AutoML com as linhas de base.
9. Por que a previsão de 12 meses de uma vez é mais difícil que a de um mês à frente?
10. O que é o concept drift e como ele afeta modelos de séries?
11. Explique o pipeline de texto com TF-IDF.
12. Que escolhas de AutoML existem na vetorização de texto?
13. Explique por que os bigramas recuperaram o sentido no texto simulado.
14. Por que o experimento de texto do capítulo não prova nada sobre texto real?
15. Como modelos pré-treinados como o BERT mudam o espaço de busca em texto?
16. O que é ampliação de dados e por que ela ajuda com poucos exemplos?
17. Descreva o mini-AutoAugment do capítulo.
18. Interprete os resultados do mini-AutoAugment.
19. Por que a validação deve se parecer com o uso real?
20. Quando o aprendizado por transferência é preferível?

Exercícios práticos em Python (scikit-learn)

Resolva os exercícios no seu computador (ou no Google Colab). Eles usam conjuntos de dados da UCI que já vêm no scikit-learn ou dados simulados. Tente resolver **antes** de consultar a solução, no Apêndice D. As saídas apresentadas lá foram obtidas com as versões de bibliotecas listadas na Apresentação.

Exercício 1 — Validação aleatória ou temporal?

NÍVEL: BÁSICO

Com a série mensal de CO_2 de Mauna Loa (`statsmodels.api.datasets.co2`, médias mensais), monte atributos com os 12 últimos valores e o mês do ano, tendo como alvo a **diferença para o mesmo mês do ano anterior**. Reserve os últimos 36 meses e, com os dados restantes, estime o MAE de uma regressão `Ridge` com `KFold(5, shuffle=True)` e com `TimeSeriesSplit(5, test_size=12)`.

Pergunta: qual validação foi mais otimista? Por que o embaralhamento é perigoso em séries?

Exercício 2 — Três linhas de base

NÍVEL: BÁSICO

Na mesma série, nos últimos 36 meses, calcule o MAE de previsões um mês à frente de três linhas de base: (a) **ingênua** (valor do mês anterior); (b) **sazonal ingênua** (mesmo mês do ano anterior); (c) **média móvel de 12 meses** (média dos 12 últimos valores).

Pergunta: qual foi a melhor? Por que a sazonal ingênua pode perder em uma série com tendência?

Exercício 3 — Quando n-gramas importam?

NÍVEL: INTERMEDIÁRIO

Gere textos simulados de avaliação (como no capítulo) em duas versões: (a) **sem negação** e (b) com **40% de frases com negação** ("não bom"). Em cada versão, compare, por acurácia no teste, um TF-IDF com palavras isoladas e outro com unigramas e bigramas, ambos com regressão logística.

Pergunta: em que situação os bigramas ajudam? O que isso sugere sobre a escolha da representação?

Exercício 4 — Qual distorção ajuda mais?

NÍVEL: AVANÇADO

No Digits (treino de 300 imagens, 40% de teste, como no capítulo), compare, com 4 cópias distorcidas por imagem, quatro políticas: (a) só deslocamento (até 1,5 pixel); (b) só rotação (até 15 graus); (c) só ruído (desvio 0,8); (d) as três juntas. Meça a acurácia no teste limpo e no teste deslocado até 1 pixel e girado até 10 graus, com uma SVM (`C=10`).

Pergunta: qual política ajudou mais sob distorção? Alguma prejudicou o teste limpo?

> SAIBA MAIS **Para ir além: bases públicas da UCI e do Kaggle**
>
> - **UCI:** Appliances Energy Prediction, Beijing PM2.5 e Individual Household Electric Power Consumption (séries reais com muitos pontos).
> - **Kaggle:** Store Item Demand Forecasting, M5 Forecasting e IMDB Reviews (texto), além de Fashion-MNIST (imagens).
> - **Desafio:** troque as médias mensais por previsões de **12 meses à frente de uma só vez** e compare como o erro cresce com o horizonte.
>
> Estes conjuntos não foram usados nem testados nos códigos do livro. Confira, nos sites, a descrição, a licença e as colunas antes de usá-los.

Projeto integrador do capítulo

Previsão de manchas solares com mini-AutoML

Fonte dos dados. Série anual de **manchas solares** (1700 a 2008), incluída na biblioteca `statsmodels` (`sm.datasets.sunspots`; originalmente de séries públicas de observação solar). Se preferir, use uma série do Kaggle ou da UCI de sua escolha (por exemplo, consumo de energia) e adapte as tarefas.

Contexto. Um observatório quer prever o número de manchas do **próximo ano**. A série não tem sazonalidade anual, mas mostra um ciclo de cerca de 11 anos, e muitos modelos vão tentar aprendê-lo.

Atenção ética. O exercício é didático. Nenhuma previsão aqui deve apoiar decisões operacionais reais.

Tarefas.

1. Carregue a série (coluna `SUNACTIVITY`) e reserve os **últimos 40 anos** para o teste (validação em avanço, 1 ano à frente).
2. Monte atributos com os L valores anteriores (L em {4, 11, 22}), com ou sem **diferenciação** (alvo = `y[t] − y[t−1]`, somando de volta `y[t−1]` ao prever).
3. Defina um espaço com os modelos `Ridge`, `RandomForestRegressor` (100 árvores) e `HistGradientBoostingRegressor`; são 3 × 3 × 2 = 18 configurações.
4. Selecione a configuração por `TimeSeriesSplit(4, test_size=20)` usando **apenas** os dados de treino (MAE).
5. Avalie a escolhida no teste por *walk-forward* (reajuste a cada ano) e compare com as linhas de base **ingênua** e **média dos últimos 11 anos**.
6. Mostre também a **pior** configuração da validação e seu erro no teste.
7. Escreva um parágrafo (8 a 10 linhas): o mini-AutoML superou as linhas de base? Que limite o ruído natural da série impõe?

Entregáveis. Um notebook (ou script) comentado, a tabela de validação das 18 configurações, o MAE de teste dos métodos e o parágrafo.

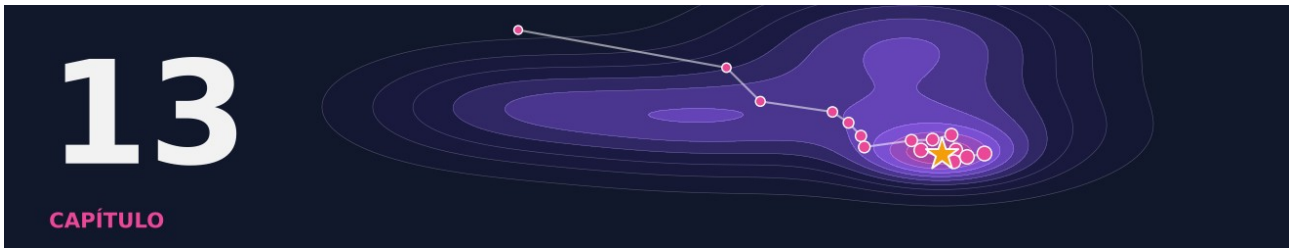
A solução de referência está no Apêndice E.

PARTE V

Parte V — Responsabilidade e Prática

Nesta parte:

- Cap. 13 — Explicabilidade, ética e limitações do AutoML
- Cap. 14 — MLOps, implantação e projeto integrador



Capítulo 13 — Explicabilidade, ética e limitações do AutoML

OBJETIVOS DE APRENDIZAGEM

Ao final deste capítulo você será capaz de:

- Distinguir modelos interpretáveis de explicações pós-hoc, e explicações globais de locais.
- Calcular e interpretar importância por permutação e valores SHAP em um modelo escolhido automaticamente.
- Medir se um modelo trata grupos de forma diferente, com taxa de seleção e igualdade de oportunidade.
- Explicar por que remover o atributo sensível não elimina o viés (variáveis proxy).
- Estimar o custo computacional de uma busca e discutir seu impacto.
- Reconhecer os limites do AutoML e aplicar uma lista de verificação ética.

13.1 Três perguntas antes de confiar

O AutoML entrega um modelo pronto em poucos minutos. Isso é uma vantagem enorme, mas traz um risco: **é fácil usar um modelo que ninguém entende**. Antes de colocar qualquer modelo para decidir algo que afete pessoas, três perguntas devem ser respondidas.

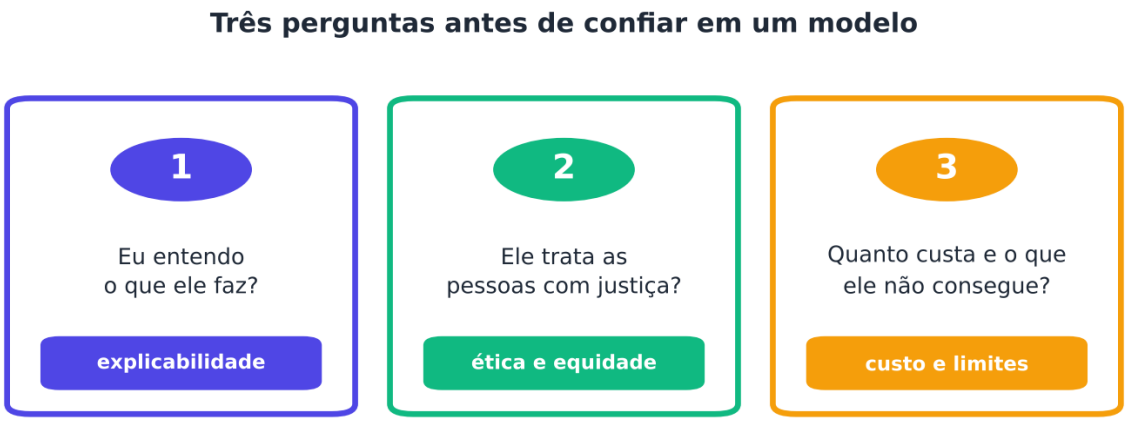


Figura 13.1 — As três perguntas antes de confiar em um modelo: eu o entendo? ele é justo? quanto ele custa?

1. **Eu entendo o que ele faz?** (explicabilidade)
2. **Ele trata as pessoas de forma justa?** (ética e equidade)
3. **Quanto ele custa, e o que ele não consegue fazer?** (custo e limites)

> NO SEU DIA A DIA **Analogia: o piloto automático**
>
> O piloto automático de um avião é uma maravilha, mas **o comandante precisa saber o que ele faz**, quando ele falha e quando assumir o controle. O AutoML é um piloto automático para modelos: não elimina a responsabilidade humana, só a desloca.

13.2 Explicabilidade

13.2.1 Interpretável ou explicado depois?

Há duas maneiras de lidar com a "caixa-preta".

- **Modelos interpretáveis por construção:** um modelo cuja lógica pode ser lida diretamente (regressão logística com poucos coeficientes, árvore pequena, regras).
- **Explicações pós-hoc:** ferramentas que, **depois** do treino, tentam explicar um modelo complexo (importância de atributos, SHAP, LIME).

Rudin (2019) defende que, em decisões de **alto risco** (saúde, crédito, justiça), deve-se **preferir modelos interpretáveis** em vez de explicar caixas-pretas, porque explicações pós-hoc **podem ser imprecisas** e não revelam o que o modelo realmente computa. E há um achado prático importante: com frequência, o modelo interpretável **não perde** em desempenho. Vamos testar isso no conjunto **Breast Cancer Wisconsin** (UCI), comparando uma regressão logística com um boosting, por validação cruzada de 10 partes.

PYTHON

```python
import warnings; warnings.filterwarnings("ignore")
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.ensemble import HistGradientBoostingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_val_score, StratifiedKFold
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

X, y = load_breast_cancer(return_X_y=True)
cv = StratifiedKFold(10, shuffle=True, random_state=0)
lr = make_pipeline(StandardScaler(), LogisticRegression(C=0.5, max_iter=3000))
hgb = HistGradientBoostingClassifier(random_state=0)
for nome, m in (("Regressão logística (interpretável)", lr), ("Boosting (caixa-preta)", hgb)):
    s = cross_val_score(m, X, y, cv=cv)
    print(f"{nome:<38s} acurácia = {s.mean():.3f} (desvio {s.std():.3f})")
```

SAÍDA

```
Regressão logística (interpretável)    acurácia = 0.981 (desvio 0.020)
Boosting (caixa-preta)                 acurácia = 0.968 (desvio 0.022)
```

As duas notas ficam **muito próximas** (0,981 contra 0,968, com desvios de cerca de 0,02): a diferença está dentro da variação entre partes e, aqui, até favorece o modelo simples. Em um problema como esse, a regressão logística entrega o desempenho do boosting com a vantagem de ser **legível**. O AutoML, ao buscar só pela métrica, pode escolher a caixa-preta mesmo quando um modelo simples basta; por isso vale **incluir modelos interpretáveis no espaço de busca** e preferi-los em caso de empate técnico.

> **Dica: a regra do empate técnico**
>
> Se um modelo simples e um complexo diferem **menos que a variação entre divisões**, escolha o simples. Você ganha explicação, velocidade e menos risco.

13.2.2 Explicação global e local

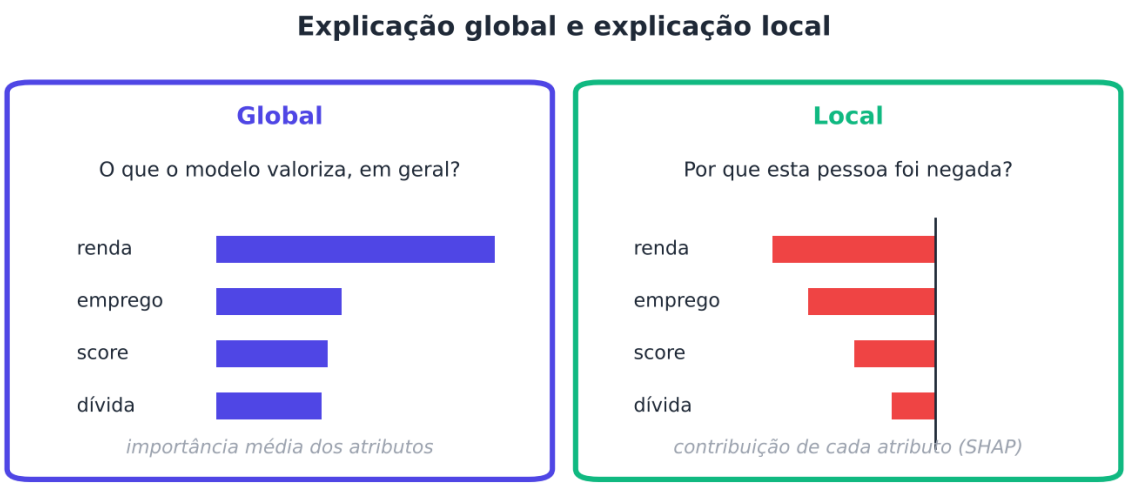


Figura 13.2 — Explicações globais (o modelo como um todo) e locais (uma decisão específica).

- **Explicação global:** responde "**o que o modelo valoriza em geral?**" (por exemplo, a importância de cada atributo).
- **Explicação local:** responde "**por que o modelo decidiu isso para esta pessoa?**".

Para praticar, vamos criar um problema de **concessão de crédito simulado**. Usaremos dados **sintéticos**, para que possamos conhecer a **verdade que, no mundo real, nunca conhecemos**. Há 6.000 pessoas, em dois grupos: o grupo 0 (70%) e o grupo 1 (30%, "minoritário"). Cada pessoa tem uma **capacidade real de pagamento** (uma variável escondida), e os atributos observados são: **renda**, **tempo de emprego**, **razão de dívida**, **score histórico** e um **CEP-score** (que, como acontece na vida real, é um pouco **correlacionado com o grupo**). A renda do grupo 1 é, em média, um pouco menor, por razões estruturais.

O ponto-chave: os **rótulos de treino** são as **decisões históricas de crédito**, que foram **enviesadas contra o grupo 1** (a aprovação exigia mais dessas pessoas). Como simulamos tudo, também sabemos **quem de fato pagaria**.

```python
import warnings; warnings.filterwarnings("ignore")
import json
import numpy as np
from sklearn.model_selection import train_test_split

r = np.random.default_rng(0); n = 6000
g = (r.random(n) < 0.3).astype(int)                          # 1 = grupo minoritário
z = r.normal(0, 1, n)                                        # capacidade real de pagamento (escondida)
renda = np.exp(0.5 * z + 0.35 * r.normal(size=n) - 0.25 * g) * 3.0
emprego = np.clip(5 + 2 * z + r.normal(0, 2, n), 0, None)
divida = np.clip(0.4 - 0.1 * z + r.normal(0, 0.1, n), 0, 1)
cep = 0.8 * g + 0.6 * r.normal(size=n)                       # variável proxy do grupo
score = z + r.normal(0, 1, n)
y_real = (z > 0).astype(int)                                 # quem de fato pagaria (só existe na
```

```
simulação)
y_hist = ((z - 0.6 * g + r.normal(0, 0.5, n)) > 0).astype(int)   # decisões históricas, enviesadas
contra o grupo 1
NOMES = ["renda", "tempo_emprego", "razao_divida", "cep_score", "score_historico"]
X = np.c_[renda, emprego, divida, cep, score]
Xtr, Xte, ytr, yte, gtr, gte, yrtr, yrte = train_test_split(X, y_hist, g, y_real, test_size=0.3,
random_state=0, stratify=y_hist)

print("Taxa de aprovação NOS DADOS HISTÓRICOS  : grupo 0 = %.2f | grupo 1 = %.2f" % (ytr[gtr ==
0].mean(), ytr[gtr == 1].mean()))
print("Taxa de bons pagadores (verdade, só aqui): grupo 0 = %.2f | grupo 1 = %.2f" % (yrtr[gtr ==
0].mean(), yrtr[gtr == 1].mean()))
```

SAÍDA

```
Taxa de aprovação NOS DADOS HISTÓRICOS  : grupo 0 = 0.51 | grupo 1 = 0.29
Taxa de bons pagadores (verdade, só aqui): grupo 0 = 0.51 | grupo 1 = 0.49
```

Veja a injustiça embutida nos dados: os dois grupos têm **praticamente a mesma proporção de bons pagadores**, mas o grupo 1 foi **aprovado bem menos** no histórico. Qualquer AutoML que otimize a acurácia contra esses rótulos vai **aprender a reproduzir** essa diferença, e a fará parecer "objetiva".

Vamos treinar um modelo de boosting (LightGBM) nos rótulos históricos e explicá-lo com três ferramentas.

- **Importância por permutação:** embaralha-se uma coluna do teste e mede-se **quanto a acurácia cai**. Se cai muito, a coluna importa. É simples e vale para qualquer modelo.
- **Valores SHAP** (Lundberg; Lee, 2017): atribuem a cada atributo a sua **contribuição** para a previsão de **cada exemplo**, com base nos **valores de Shapley** da teoria dos jogos. Somando os valores absolutos médios, obtém-se uma importância global; olhando um exemplo, uma explicação local.
- **LIME** (Ribeiro; Singh; Guestrin, 2016): explica **uma decisão** ajustando um modelo simples na vizinhança dela. Não vamos rodá-lo aqui, mas é a terceira ferramenta mais citada.

NO SEU DIA A DIA **Analogia: dividir a conta do restaurante**

Imagine que um grupo de amigos pede vários pratos e a conta chega. Como dividir **de forma justa** o que cada um deve, sabendo que alguns pratos foram compartilhados? Os valores de Shapley respondem calculando a **contribuição média de cada pessoa** em todas as ordens possíveis de chegada. O SHAP faz o mesmo com os atributos: divide a "conta" (a diferença entre a previsão e a previsão média) entre eles.

PYTHON

```
import lightgbm as lgb
import shap
from sklearn.inspection import permutation_importance

modelo = lgb.LGBMClassifier(n_estimators=150, learning_rate=0.05, num_leaves=15, random_state=0,
verbose=-1).fit(Xtr, ytr)
print("Acurácia contra os rótulos históricos: %.3f" % modelo.score(Xte, yte))

pi = permutation_importance(modelo, Xte, yte, n_repeats=10, random_state=0, scoring="accuracy")
sv = shap.TreeExplainer(modelo).shap_values(Xte); sv = sv[1] if isinstance(sv, list) else sv
shap_global = np.abs(sv).mean(0)
print("\nImportância dos atributos:")
print(f"  {'atributo':<16s} {'permutação (queda de acurácia)':>32s} {'SHAP médio |valor|':>20s}")
for i in np.argsort(-shap_global): print(f"  {NOMES[i]:<16s} {pi.importances_mean[i]:>32.3f}
```

```
{shap_global[i]:>20.3f}")

i = 0; p = modelo.predict_proba(Xte[i:i + 1])[0, 1]
print(f"\nExplicação local da pessoa #{i} (probabilidade de aprovação = {p:.3f}, grupo = {gte[i]}):")
for j in np.argsort(-np.abs(sv[i])): print(f"  {NOMES[j]:<16s} valor = {Xte[i, j]:7.2f}   contribuição
SHAP = {sv[i, j]:+.2f}")
json.dump({"nomes": NOMES, "perm": pi.importances_mean.tolist(), "shap": shap_global.tolist(), "local":
sv[i].tolist(), "valores": Xte[i].tolist(), "p": float(p)},
          open("/home/user/claude/automl/build/f13_expl.json", "w"))
```

SAÍDA

```
Acurácia contra os rótulos históricos: 0.818

Importância dos atributos:
  atributo           permutação (queda de acurácia)   SHAP médio |valor|
  renda                                       0.121                1.295
  tempo_emprego                               0.029                0.560
  score_historico                             0.035                0.509
  razao_divida                                0.028                0.503
  cep_score                                   0.009                0.168

Explicação local da pessoa #0 (probabilidade de aprovação = 0.007, grupo = 0):
  renda            valor =    1.29   contribuição SHAP = -1.73
  tempo_emprego    valor =    0.00   contribuição SHAP = -1.35
  score_historico  valor =   -1.68   contribuição SHAP = -0.89
  razao_divida     valor =    0.51   contribuição SHAP = -0.46
  cep_score        valor =   -0.52   contribuição SHAP = +0.08
```

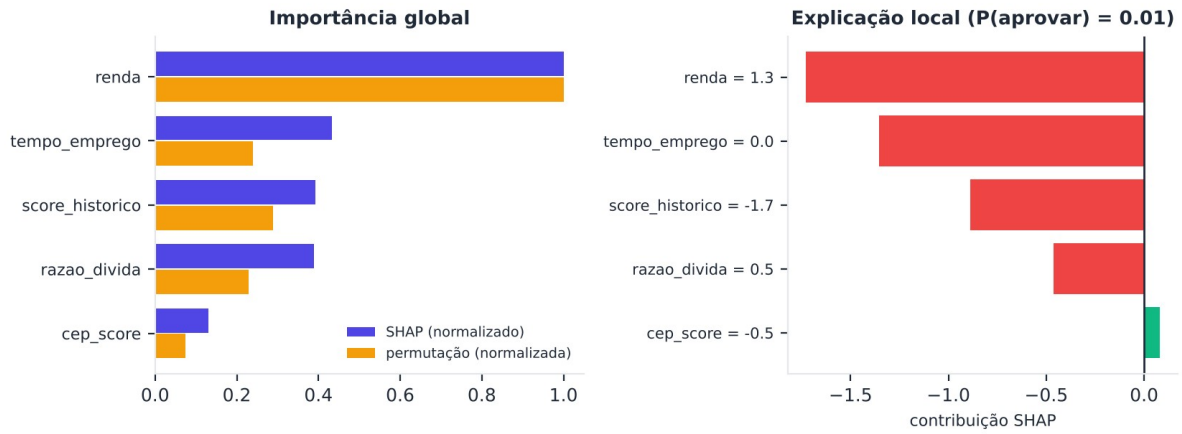


Figura 13.3 — Importância dos atributos por permutação e por SHAP (esquerda) e explicação local de uma pessoa (direita).

Duas ferramentas diferentes, mesma história: a **renda** domina. Isso é plausível, mas esconde um fato: a renda do grupo 1 é menor por razões estruturais, e, portanto, **usar a renda é, em parte, usar o grupo**. A explicação mostra *o que* o modelo usa; ela **não** diz se o uso é **justo**. Para isso precisamos de outras medidas.

Atenção: explicar não é justificar

Uma explicação mostra **como** o modelo chegou à decisão, não se a decisão é **correta ou justa**. Um modelo enviesado pode ser perfeitamente explicável.

13.3 Justiça e vieses

13.3.1 De onde vem o viés?

O viés raramente nasce do algoritmo; ele costuma **entrar pelos dados** (Mehrabi *et al.*, 2021). Algumas fontes:

- **Viés histórico:** os dados refletem decisões humanas injustas do passado (nosso exemplo).
- **Viés de representação:** um grupo aparece pouco nos dados e o modelo aprende mal sobre ele.
- **Viés de medição:** a variável medida é um mau retrato do que importa (por exemplo, "gastos médicos" como medida de "gravidade").
- **Viés de feedback:** as decisões do modelo mudam os dados futuros e reforçam a distorção.

O AutoML **não corrige** nenhum deles; ao contrário, **otimiza** a métrica que você der, inclusive reproduzindo o viés, só que mais eficientemente.

13.3.2 Como medir

Duas medidas simples, ambas calculadas **por grupo** (Hardt; Price; Srebro, 2016, para a segunda):

- **Taxa de seleção:** a proporção de pessoas **aprovadas** em cada grupo. Se for muito diferente, há **disparidade**.
- **Igualdade de oportunidade:** entre as pessoas que **de fato mereceriam** a aprovação (os verdadeiros positivos), a **taxa de aprovação** deve ser **igual** entre os grupos. É a **taxa de verdadeiros positivos (TPR)** por grupo.

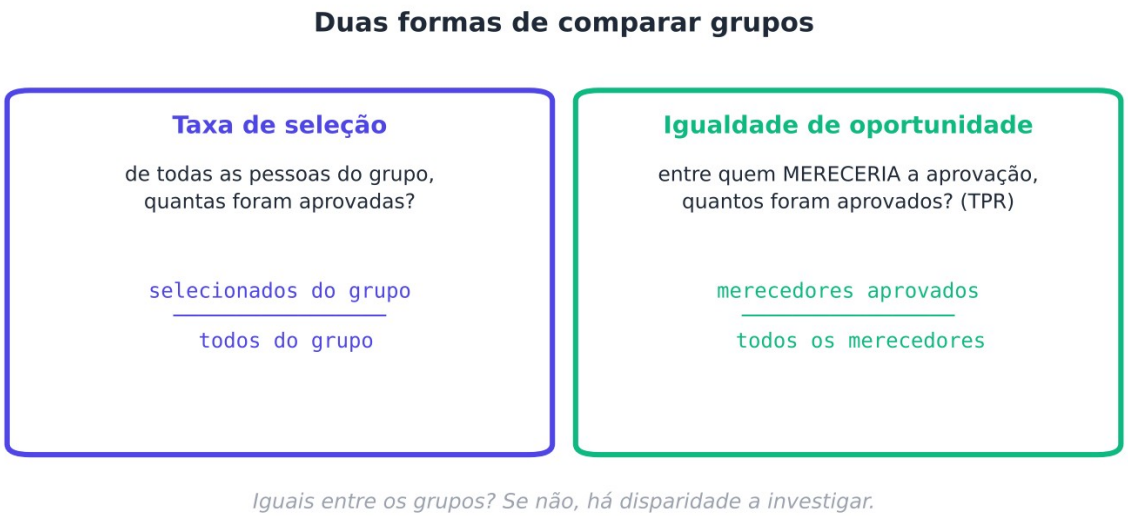


Figura 13.4 — Taxa de seleção e igualdade de oportunidade: duas formas de comparar grupos.

Em dados reais, a "verdade" (quem mereceria) quase nunca é conhecida; nós a conhecemos **porque simulamos**. Isso é um privilégio didático: mostra o que acontece de fato, e nos lembra de que, no mundo real, medimos a justiça com **rótulos que podem ser eles mesmos enviesados**.

13.3.3 "Basta remover o atributo sensível?"

A tentação é simples: **retirar** a coluna do grupo e do CEP-score e considerar o problema resolvido. Vamos testar. Treinamos o mesmo boosting em duas versões: **com** o CEP-score (o proxy) e **sem** ele. Em nenhuma delas o modelo recebe o atributo "grupo".

```python
from sklearn.ensemble import HistGradientBoostingClassifier
res = {}
print(f"{'versão':<10s} {'acurácia':>9s} | {'seleção g0':>10s} {'seleção g1':>10s} | {'TPR real g0':>11s} {'TPR real g1':>11s}")
for nome, cols in (("com CEP", [0, 1, 2, 3, 4]), ("sem CEP", [0, 1, 2, 4])):
    m = HistGradientBoostingClassifier(random_state=0).fit(Xtr[:, cols], ytr); p = m.predict(Xte[:, cols])
    sel = [p[gte == k].mean() for k in (0, 1)]; tpr = [p[(gte == k) & (yrte == 1)].mean() for k in (0, 1)]
    res[nome] = {"sel": sel, "tpr": tpr}
    print(f"{nome:<10s} {(p == yte).mean():>9.3f} | {sel[0]:>10.3f} {sel[1]:>10.3f} | {tpr[0]:>11.3f} {tpr[1]:>11.3f}")
```

```
SAÍDA
versão      acurácia | seleção g0 seleção g1 | TPR real g0 TPR real g1
com CEP        0.797 |      0.478      0.378 |       0.814       0.670
sem CEP        0.804 |      0.480      0.394 |       0.831       0.692
```

Retirar o CEP-score **não resolveu**: sem ele, o grupo 1 continua sendo aprovado bem menos (0,39 contra 0,48), e, entre os que de fato pagariam (**TPR real**), o grupo 1 é aprovado em proporção claramente menor (0,69 contra 0,83). O motivo é que outras variáveis (como a **renda**) também carregam informação sobre o grupo, e porque o modelo foi **treinado em rótulos enviesados**. Isso é o **efeito proxy**: o viés circula por variáveis correlacionadas.



Figura 13.5 — O viés não some ao apagar uma coluna: ele circula por variáveis correlacionadas e pelos próprios rótulos.

13.3.4 Mitigação: ajustar o limiar por grupo

Há várias famílias de mitigação: antes do treino (rebalancear dados), durante o treino (restrições) e **depois** (ajustar a decisão). Vamos usar a mais simples: **limiares diferentes por grupo**. O modelo gera uma **probabilidade**; normalmente aprovamos quando ela passa de 0,5. Aqui, para o grupo 1, escolhemos um limiar menor, de modo que a **taxa de seleção** dos dois grupos fique **igual**. A ideia de ajustar a decisão pós-treino para atender critérios de igualdade vem de Hardt, Price e Srebro (2016).

```python
modelo = lgb.LGBMClassifier(n_estimators=150, learning_rate=0.05, num_leaves=15, random_state=0, verbose=-1).fit(Xtr, ytr)
p = modelo.predict_proba(Xte)[:, 1]
```

```
def relatorio(nome, pred):
    sel = [pred[gte == k].mean() for k in (0, 1)]; tpr = [pred[(gte == k) & (yrte == 1)].mean() for k
in (0, 1)]
    print(f"{nome:<19s} acc hist = {(pred == yte).mean():.3f} | acc verdade = {(pred ==
yrte).mean():.3f} | "
          f"seleção {sel[0]:.2f}/{sel[1]:.2f} | TPR {tpr[0]:.2f}/{tpr[1]:.2f}")
    return {"sel": sel, "tpr": tpr}

base = relatorio("limiar único (0,5)", (p >= 0.5).astype(int))
sel0 = (p[gte == 0] >= 0.5).mean()
t1 = np.quantile(p[gte == 1], 1 - sel0)                    # limiar do grupo 1 que iguala a taxa de
seleção
ajust = relatorio("limiares por grupo", np.where(gte == 0, p >= 0.5, p >= t1).astype(int))
print(f"\nLimiar do grupo 0 = 0,50 | limiar do grupo 1 = {t1:.2f}")
json.dump({"base": base, "ajust": ajust, "sem_cep": res["sem CEP"], "com_cep": res["com CEP"]},
open("/home/user/claude/automl/build/f13_just.json", "w"))
```

SAÍDA

```
limiar único (0,5)  acc hist = 0.818 | acc verdade = 0.847 | seleção 0.49/0.37 | TPR 0.84/0.67
limiares por grupo  acc hist = 0.801 | acc verdade = 0.858 | seleção 0.49/0.49 | TPR 0.84/0.81

Limiar do grupo 0 = 0,50 | limiar do grupo 1 = 0.34
```

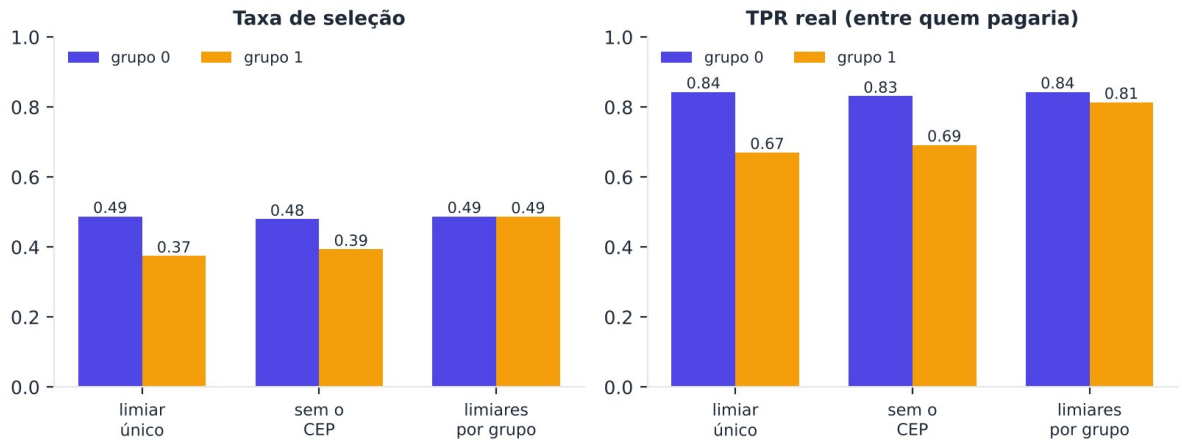


Figura 13.6 — Taxa de seleção e TPR real por grupo: com limiar único e com limiares ajustados por grupo.

Observe o que aconteceu, com honestidade. Cada linha mostra a acurácia contra o histórico (`acc hist`), a acurácia contra a verdade (`acc verdade`), a taxa de seleção dos grupos 0/1 (`seleção`) e o TPR real dos grupos 0/1 (`TPR`). Ao ajustar os limiares:

- a **acurácia contra o histórico** (o que o AutoML otimizava) **cai** um pouco, de 0,818 para 0,801, porque o histórico estava enviesado;
- a **acurácia contra a verdade** (que só conhecemos na simulação) **sobe**, de 0,847 para 0,858;
- a **taxa de seleção** dos grupos se iguala (0,49 nos dois) e o **TPR real** do grupo 1 sobe de 0,67 para 0,81, aproximando-se do grupo 0 (0,84).

Esse é um resultado **de simulação**: construímos a "verdade" para que a diferença fosse visível. Na vida real, a correção **não** é mágica. Há **trade-offs**: as definições de justiça podem entrar em conflito entre si (Mehrabi *et al.*, 2021), nem sempre é possível atender a todas ao mesmo tempo, e **escolher o critério é uma decisão humana e social**, não técnica.

> **Atenção: o que não se deve concluir**
>
> Não conclua que "o ajuste de limiares resolve o viés". Concluiu-se apenas que, **neste cenário simulado**, ele reduziu a disparidade e não prejudicou a acurácia contra a verdade. Em dados reais, a verdade é desconhecida, e o uso de atributos sensíveis para decisões pode ser **restrito por lei**. Consulte sempre especialistas jurídicos e das áreas afetadas.

13.4 O custo da busca

Cada modelo avaliado consome tempo de computação e **energia**. Strubell, Ganesh e McCallum (2019) mostraram que treinar e ajustar modelos de linguagem tem custo energético e de carbono considerável, e que a **busca** multiplica esse custo. Schwartz *et al.* (2020) propõem a **IA verde**: informar o custo computacional junto com a acurácia e valorizar a eficiência.

Para estimar o custo de um AutoML, o ponto de partida é simples: **contar quantos modelos serão treinados**.

PYTHON

```
estrategias = {
    "Grade 5x5x5 com validação cruzada de 5 partes": 5 * 5 * 5 * 5,
    "Aleatória: 60 sorteios, validação cruzada de 5": 60 * 5,
    "Bayesiana (TPE): 60 tentativas, validação cruzada de 5": 60 * 5,
    "Aleatória: 60 sorteios, holdout (1 divisão)": 60 * 1,
    "Halving/Hyperband: ~30 treinos completos equivalentes": 30,
    "Modelo padrão, sem busca": 1,
}
minutos = 2
print(f"{'estratégia':<58s} {'treinos':>8s} {'horas*':>8s}")
for nome, k in estrategias.items(): print(f"{nome:<58s} {k:>8d} {k * minutos / 60:>8.1f}")
print(f"\n* supondo {minutos} minutos por treino; multiplique pelo consumo do seu hardware para estimar a energia.")
```

SAÍDA

```
estratégia                                                treinos   horas*
Grade 5x5x5 com validação cruzada de 5 partes                 625     20.8
Aleatória: 60 sorteios, validação cruzada de 5                300     10.0
Bayesiana (TPE): 60 tentativas, validação cruzada de 5        300     10.0
Aleatória: 60 sorteios, holdout (1 divisão)                    60      2.0
Halving/Hyperband: ~30 treinos completos equivalentes          30      1.0
Modelo padrão, sem busca                                        1      0.0

* supondo 2 minutos por treino; multiplique pelo consumo do seu hardware para estimar a energia.
```

Três ideias práticas para reduzir o custo, que você já conhece: **busca aleatória em vez de grade** (Capítulo 4), **parada precoce e Hyperband** (Capítulo 6) e **warm start** (Capítulo 9). E uma ideia nova: **pergunte se a busca vale a pena**. Se um modelo padrão chega perto, gastar 100 vezes mais computação por um ganho dentro do ruído é um mau negócio.

13.5 Os limites do AutoML

O AutoML é poderoso, mas **não é mágico**. Seus limites merecem ficar à vista.

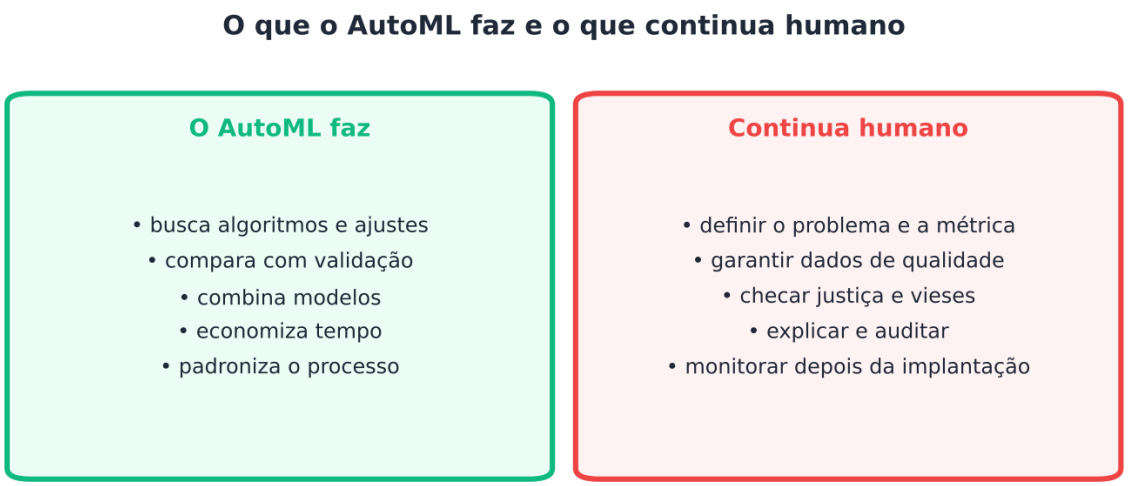


Figura 13.7 — O que o AutoML faz bem e o que continua sendo responsabilidade humana.

1. **Não define o problema.** Escolher o que prever, a métrica e o que é "bom" é humano.
2. **Não corrige dados ruins.** "Lixo entra, lixo sai": rótulos enviesados, vazamento e amostras não representativas passam direto (Kapoor; Narayanan, 2023).
3. **Não existe almoço grátis.** Nenhum algoritmo é o melhor em todos os problemas (Wolpert, 1996); por isso o AutoML precisa **procurar**, e por isso seus ganhos dependem do problema.
4. **Otimiza a validação, não o mundo.** Quanto mais se busca, maior o **otimismo da seleção** (Cawley; Talbot, 2010; Varma; Simon, 2006), tema dos Capítulos 2 e 4.
5. **Não prevê mudanças do mundo.** Se os dados mudam depois da implantação (deriva), o modelo envelhece (Capítulo 14).
6. **Tem custo.** Computação, energia e tempo de quem revisa os resultados.
7. **Pode esconder decisões.** Um pipeline escolhido automaticamente é mais difícil de auditar, a menos que você inspecione o resultado (Capítulo 10).

13.6 Lista de verificação ética para um projeto de AutoML

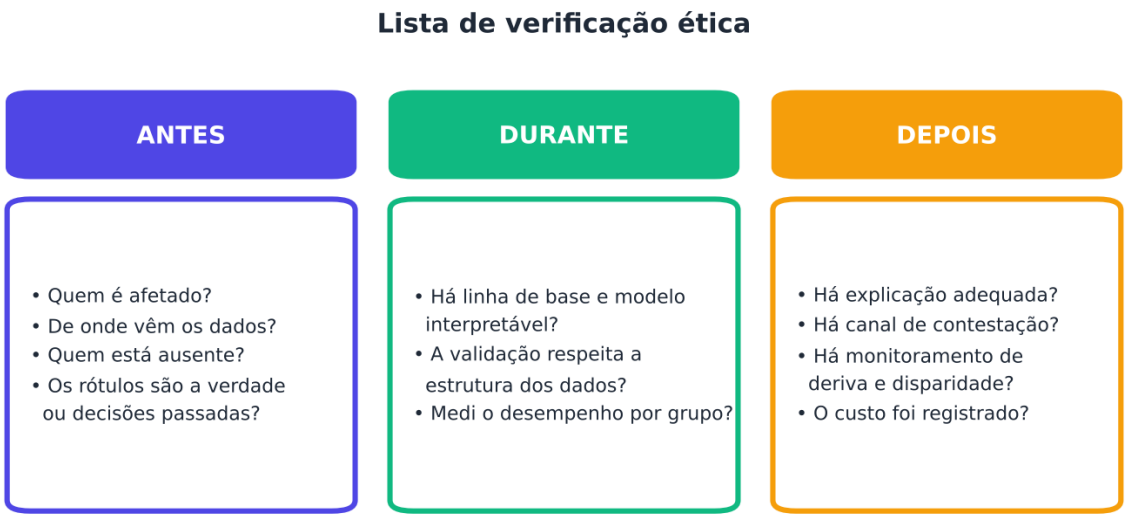


Figura 13.8 — Uma lista de verificação para antes, durante e depois do AutoML.

Antes:

- O problema está bem definido, e a decisão que o modelo apoia afeta pessoas? Quais?
- De onde vêm os dados? Quem está **sub-representado** ou **ausente**?
- Os rótulos refletem a **verdade** ou **decisões humanas passadas**?

Durante:

- Há linha de base simples e modelo interpretável no espaço de busca?
- A validação respeita a estrutura dos dados (tempo, grupos, repetição de indivíduos)?
- Medi o desempenho **por grupo** e não só a média geral?

Depois:

- Existe **explicação** adequada ao público (técnica para quem desenvolve, simples para quem é afetado)?
- Há **canal de contestação** para quem discordar de uma decisão?
- Há **monitoramento** de desempenho, de deriva e de disparidades?
- O custo computacional foi **registrado e justificado**?

13.7 Boas práticas

1. **Prefira modelos interpretáveis** quando o desempenho for equivalente, sobretudo em decisões de alto risco (Rudin, 2019).
2. **Explique em dois níveis:** global (o que o modelo valoriza) e local (por que decidiu isso).
3. **Meça por grupo:** taxa de seleção e igualdade de oportunidade, e desconfie de médias gerais.
4. **Não conte com a remoção de atributos sensíveis:** procure proxies e examine os rótulos.
5. **Questione os rótulos:** modelos aprendem as decisões do passado, inclusive as injustas.
6. **Documente o custo** e evite buscas que não se justificam.
7. **Mantenha pessoas no circuito** nas decisões que afetam vidas.

8. **Seja transparente sobre limites:** relate o que o modelo **não** sabe fazer.

Resumo do capítulo

MEMORIZE **Cartões de memorização**

- **Três perguntas:** eu entendo? é justo? quanto custa e o que não faz?
- **Interpretável × pós-hoc:** em alto risco, prefira o interpretável; no empate técnico, o simples (Rudin, 2019).
- **Global × local:** o que o modelo valoriza × por que decidiu isto.
- **Permutação, SHAP, LIME:** ferramentas de explicação pós-hoc; explicar não é justificar.
- **Viés entra pelos dados:** histórico, representação, medição e feedback.
- **Métricas por grupo:** taxa de seleção e igualdade de oportunidade (TPR).
- **Proxies:** remover o atributo sensível não remove o viés.
- **Mitigação** (limiares por grupo) reduz disparidade, mas tem trade-offs e é decisão humana.
- **Custo:** conte os treinos; prefira busca aleatória, Hyperband e warm start.
- **Limites:** o AutoML não define o problema, não corrige dados ruins e otimiza a validação.

Exercícios de múltipla escolha

Escolha **uma** alternativa em cada questão. O gabarito está no Apêndice A.

Questão 1. Qual é a diferença entre um modelo interpretável e uma explicação pós-hoc?

a) O pós-hoc é sempre mais preciso.
b) Não há diferença.
c) O interpretável só existe em imagens.
d) O interpretável tem lógica legível por construção; a pós-hoc tenta explicar, depois do treino, um modelo complexo.

Questão 2. O que Rudin (2019) defende para decisões de alto risco?

a) Eliminar a validação.
b) Usar sempre redes profundas.
c) Preferir modelos interpretáveis em vez de explicar caixas-pretas.
d) Explicar tudo com SHAP e confiar.

Questão 3. No conjunto Breast Cancer, como se comparam a regressão logística e o boosting no capítulo?

a) Têm notas muito próximas (0,981 e 0,968), dentro da variação entre partes.
b) A logística falha.
c) Não é possível comparar.
d) O boosting é muito melhor.

Questão 4. Qual é a regra do empate técnico?

a) Se um modelo simples e um complexo diferem menos que a variação entre divisões, escolha o simples.
b) Sempre escolha o complexo.
c) Escolha ao acaso.
d) Sempre escolha o mais lento.

Questão 5. O que é uma explicação global?

a) Aquela que mostra o que o modelo valoriza em geral.
b) Aquela que mede o custo.
c) Aquela que escolhe a semente.
d) Aquela que explica uma única decisão.

Questão 6. O que é uma explicação local?

a) A importância média dos atributos.
b) O tempo de treino.
c) A divisão do teste.
d) Aquela que explica por que o modelo decidiu algo para uma pessoa específica.

Questão 7. Como funciona a importância por permutação?

a) Embaralha-se uma coluna e mede-se quanto o desempenho cai.
b) Aumenta-se o treino.
c) Trocam-se os rótulos.
d) Remove-se o modelo.

Questão 8. Em que se baseiam os valores SHAP?

a) Nos valores de Shapley, da teoria dos jogos.
b) Na busca em grade.
c) Na validação cruzada.
d) No teste de Friedman.

Questão 9. Qual é a analogia do capítulo para o SHAP?

a) Sortear o pagador.
b) Escolher o prato mais barato.
c) Contar as mesas.
d) Dividir de forma justa a conta do restaurante entre os amigos.

Questão 10. O que faz o LIME (Ribeiro et al., 2016)?

a) Escolhe hiperparâmetros.
b) Remove atributos.
c) Treina uma rede maior.
d) Explica uma decisão ajustando um modelo simples na vizinhança dela.

Questão 11. Por que 'explicar não é justificar'?

a) Porque a explicação mostra como o modelo decidiu, não se a decisão é correta ou justa.
b) Porque só vale para árvores.
c) Porque o SHAP é proibido.
d) Porque explicações são sempre falsas.

Questão 12. De onde costuma vir o viés de um modelo?

a) Dos dados, como decisões históricas injustas e representação desigual.
b) Apenas do algoritmo.
c) Da métrica de acurácia.
d) Do computador.

Questão 13. O que é o viés histórico?

a) A variável medida é um mau retrato do que importa.
b) As decisões do modelo mudam os dados futuros.
c) Um grupo aparece pouco nos dados.
d) Os dados refletem decisões humanas injustas do passado.

Questão 14. O que é o viés de representação?

a) Decisões injustas do passado.
b) Erro de programação.
c) Erro de medição.
d) Um grupo aparece pouco nos dados e o modelo aprende mal sobre ele.

Questão 15. O que é o viés de feedback?

a) O erro de arredondamento.
b) As decisões do modelo alteram os dados futuros e reforçam a distorção.
c) Um tipo de ensemble.
d) Um tipo de métrica.

Questão 16. O que mede a taxa de seleção?

a) O tempo de treino.
b) O número de atributos.
c) A proporção de pessoas aprovadas em cada grupo.
d) A acurácia média.

Questão 17. O que é a igualdade de oportunidade (Hardt et al., 2016)?

a) Todos os grupos têm a mesma taxa de seleção.
b) O modelo não usa o grupo.
c) Todos têm o mesmo score.
d) Entre quem mereceria a aprovação, a taxa de aprovação (TPR) deve ser igual entre os grupos.

Questão 18. O que é uma variável proxy?

a) Um tipo de modelo.
b) Uma variável correlacionada com um atributo sensível que o carrega indiretamente.
c) Uma variável removida.
d) O rótulo.

Questão 19. O que o experimento de remover o CEP-score mostrou?

a) O grupo 1 continuou sendo aprovado menos; o viés circulou por outras variáveis e pelos rótulos.
b) A acurácia caiu a zero.
c) O viés desapareceu.
d) O grupo 0 foi prejudicado.

Questão 20. Por que o crédito simulado permite conhecer a 'verdade'?

a) Porque o modelo a revela.
b) Porque o teste a contém.
c) Porque os dados são reais.
d) Porque a capacidade real de pagamento foi gerada por nós e é conhecida.

Questão 21. O que são limiares por grupo?

a) Tipos de ativação.
b) Hiperparâmetros da árvore.
c) Faixas de preço.
d) Pontos de corte diferentes por grupo para decidir a partir da probabilidade.

Questão 22. O que aconteceu com a acurácia contra a verdade ao usar limiares por grupo no capítulo?

a) Subiu de 0,847 para 0,858.
b) Caiu a zero.
c) Não foi medida.
d) Caiu para 0,5.

Questão 23. Por que o resultado do ajuste de limiares não deve ser generalizado?

a) Porque o limiar é aleatório.
b) Porque só vale para o grupo 0.
c) Porque vale para o cenário simulado, em que a verdade é conhecida; no mundo real há trade-offs e rótulos enviesados.
d) Porque a técnica é inválida.

Questão 24. No Exercício 4, o que ocorreu ao igualar o TPR contra o histórico enviesado?

a) O TPR real do grupo 1 não melhorou (0,67 para 0,63).
b) A disparidade real desapareceu.
c) A acurácia dobrou.
d) O TPR do grupo 0 caiu a zero.

Questão 25. Segundo Strubell et al. (2019), qual é o efeito da busca de hiperparâmetros/arquitetura?

a) Multiplica o custo energético e de carbono do treino.
b) Não tem efeito.
c) Elimina o treino.
d) Reduz o custo.

Questão 26. O que propõem Schwartz et al. (2020) com a 'IA verde'?

a) Reportar o custo computacional junto com a precisão e valorizar a eficiência.
b) Esconder o custo.
c) Usar sempre as maiores redes.
d) Abolir o aprendizado de máquina.

Questão 27. Qual é o primeiro passo para estimar o custo de um AutoML?

a) Contar quantos modelos serão treinados.
b) Medir o tamanho da tela.
c) Mudar a métrica.
d) Escolher a semente.

Questão 28. Qual das alternativas é um limite do AutoML?

a) Não define o problema nem corrige dados ruins.
b) Sempre piora o modelo.
c) Não usa validação.
d) Só funciona com imagens.

Questão 29. O que diz o teorema 'sem almoço grátis' (Wolpert, 1996), conforme o capítulo?

a) Todo algoritmo é igual.
b) O AutoML é sempre melhor.
c) Dados não importam.
d) Nenhum algoritmo é o melhor em todos os problemas.

Questão 30. O que significa uma AUC de 0,857 ao prever o grupo a partir dos atributos (sem o grupo)?

a) Que os atributos permitem reconstruir o grupo com boa precisão, revelando proxies.
b) Que não há viés nos dados.
c) Que o modelo é perfeitamente justo.
d) Que o grupo é aleatório.

Exercícios discursivos

Responda com suas palavras, em poucas linhas. Compare depois com a resposta-modelo do Apêndice A.

1. Quais são as três perguntas antes de confiar em um modelo de AutoML?
2. Compare modelos interpretáveis e explicações pós-hoc, com a posição de Rudin.
3. O que a comparação no Breast Cancer mostrou e que regra o capítulo recomenda?
4. Diferencie explicação global e local, com um exemplo de cada.
5. Explique a importância por permutação e uma limitação.
6. Explique a ideia do SHAP com a analogia do capítulo.
7. Por que explicar não é justificar?
8. Cite quatro fontes de viés e dê um exemplo de cada.
9. Explique taxa de seleção e igualdade de oportunidade.
10. Por que o crédito simulado do capítulo tem rótulos enviesados e como isso aparece nos números?
11. Por que apagar o atributo sensível não elimina o viés?
12. Descreva o resultado do teste 'com CEP' e 'sem CEP'.
13. Explique a mitigação por limiares por grupo e seus resultados no capítulo.
14. Por que o resultado da mitigação não prova que ela resolve o viés em dados reais?
15. Por que igualar o TPR contra os rótulos históricos pode não corrigir a disparidade real?
16. Como detectar proxies?
17. Como estimar o custo de uma busca e como reduzi-lo?
18. Cite cinco limites do AutoML.
19. Que perguntas a lista de verificação ética propõe antes, durante e depois?
20. Por que o AutoML 'desloca' a responsabilidade humana, mas não a elimina?

Exercícios práticos em Python (scikit-learn)

Resolva os exercícios no seu computador (ou no Google Colab). Eles usam conjuntos de dados da UCI que já vêm no scikit-learn ou dados simulados. Tente resolver **antes** de consultar a solução, no Apêndice D. As saídas apresentadas lá foram obtidas com as versões de bibliotecas listadas na Apresentação.

Exercício 1 — Quem importa para o modelo?

NÍVEL: BÁSICO

No conjunto **Breast Cancer Wisconsin** (UCI), ajuste uma regressão logística (com `StandardScaler`) e um `HistGradientBoostingClassifier`, usando 70% para treino e 30% para teste (`random_state=0`, estratificado). Calcule a **importância por permutação** (10 repetições) no teste e mostre os três atributos mais importantes de cada modelo.

Pergunta: os dois modelos concordam sobre quais atributos importam? Isso é esperado?

Exercício 2 — Medindo a disparidade por grupo

NÍVEL: BÁSICO

Gere o crédito simulado do capítulo (função abaixo, que já devolve os atributos, o grupo, o rótulo histórico e a verdade). Treine uma regressão logística (com `StandardScaler`) nos rótulos históricos **sem** o atributo grupo, com 70% de treino. No teste, mostre por grupo: a taxa de seleção e a taxa de verdadeiros positivos (TPR) **contra a verdade**.

Pergunta: há disparidade? O modelo teve acesso ao grupo?

Exercício 3 — Detectando proxies

NÍVEL: INTERMEDIÁRIO

Com os mesmos dados, **sem** usar o rótulo de crédito, treine um classificador para **prever o grupo** a partir dos cinco atributos e calcule a AUC no teste. Repita removendo cada atributo, um de cada vez.

Pergunta: os atributos permitem prever o grupo? Qual atributo mais contribui? O que isso diz sobre "remover o atributo sensível"?

Exercício 4 — Igualdade de oportunidade com os rótulos disponíveis

NÍVEL: AVANÇADO

No crédito simulado, treine um `HistGradientBoostingClassifier` nos rótulos históricos (sem o grupo). Escolha, **usando apenas o teste e os rótulos históricos**, um limiar para o grupo 1 que iguale o **TPR contra o histórico** ao do grupo 0 (limiar 0,5 para o grupo 0). Depois, como só a simulação permite, calcule a acurácia contra a verdade e o TPR real de cada grupo.

Pergunta: igualar o TPR contra o histórico enviesado resolve a disparidade real? Compare com o limiar único.

> SAIBA MAIS **Para ir além: bases públicas da UCI e do Kaggle**
>
> - **UCI:** Adult (renda, com atributos sensíveis) e Statlog German Credit (crédito).
> - **Kaggle:** Give Me Some Credit e COMPAS (justiça criminal, muito discutido na literatura de equidade).
> - **Desafio:** repita a auditoria em uma base real, comparando duas definições de justiça e discutindo por que elas podem entrar em conflito.
>
> Estes conjuntos não foram usados nem testados nos códigos do livro. Confira, nos sites, a descrição, a licença e as colunas antes de usá-los.

Projeto integrador do capítulo

Auditoria ética de um modelo de AutoML

Fonte dos dados. Crédito **simulado** pelo código do capítulo, com parâmetros diferentes dos do texto: grupo minoritário de 20% (`p_min=0.2`), viés histórico de 0,4 (`vies=0.4`) e semente 7. (Se quiser trabalhar com dados reais, escolha uma base pública do Kaggle ou da UCI que contenha um atributo sensível, como *Adult* ou *German Credit*, e respeite a licença e as restrições legais de uso.)

Contexto. Uma instituição usou AutoML para criar um modelo de aprovação de crédito e pediu uma **auditoria** antes da implantação.

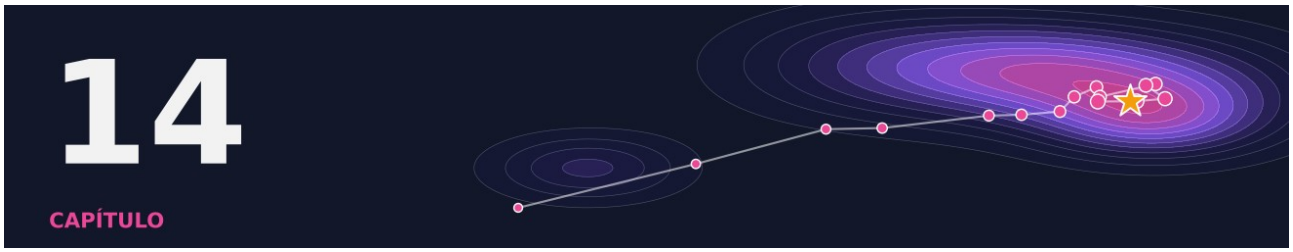
Atenção ética. Os dados são simulados. Em dados reais, decisões que usam atributos sensíveis podem ser restritas por lei, e a auditoria deve envolver especialistas jurídicos e representantes das pessoas afetadas.

Tarefas.

1. Gere os dados, separe 30% para teste (estratificado pelo rótulo histórico) e treine, **sem o atributo grupo**, o FLAML (`max_iter=20`, `time_budget=120`, `metric="accuracy"`) e uma regressão logística com `StandardScaler` (modelo interpretável).
2. Compare as duas acurácias contra o histórico e decida, pela regra do empate técnico, qual seria preferível.
3. Para o modelo do FLAML, calcule a **importância por permutação** e comente.
4. Calcule, por grupo, a **taxa de seleção** e o **TPR real** dos dois modelos.
5. Teste se o grupo pode ser previsto pelos atributos (AUC) e discuta o papel de proxies.
6. Aplique **limiares por grupo** para igualar a taxa de seleção no modelo do FLAML e reporte acurácia contra o histórico, contra a verdade e TPR real.
7. Registre o **custo** (número de treinos e tempo de busca).
8. Escreva um relatório curto (10 a 12 linhas) com recomendações: o modelo pode ser implantado? Que salvaguardas (monitoramento, contestação, revisão humana) você exigiria?

Entregáveis. Um notebook (ou script) comentado, as tabelas por grupo e o relatório.

A solução de referência está no Apêndice E.



Capítulo 14 — MLOps, implantação e projeto integrador

OBJETIVOS DE APRENDIZAGEM

Ao final deste capítulo você será capaz de:

- Explicar por que um modelo no notebook ainda não é um produto, e o que é MLOps.
- Salvar, carregar e versionar um pipeline, registrando versões e semente para reprodutibilidade.
- Medir o custo de previsão (latência) de modelos simples e de ensembles.
- Detectar deriva de dados com testes estatísticos e com o índice de estabilidade populacional (PSI).
- Definir uma política de retreino com o desafio entre campeão e desafiante.
- Executar um projeto integrador de AutoML, da definição do problema ao relatório final.

14.1 Do notebook ao mundo real

Você chegou à última etapa. Treinou modelos, buscou hiperparâmetros, combinou ensembles, explicou e auditou. Mas um detalhe ficou de fora: **o modelo só vale a pena quando alguém o usa**. Colocá-lo em uso, com segurança e de forma sustentável, é um trabalho à parte.

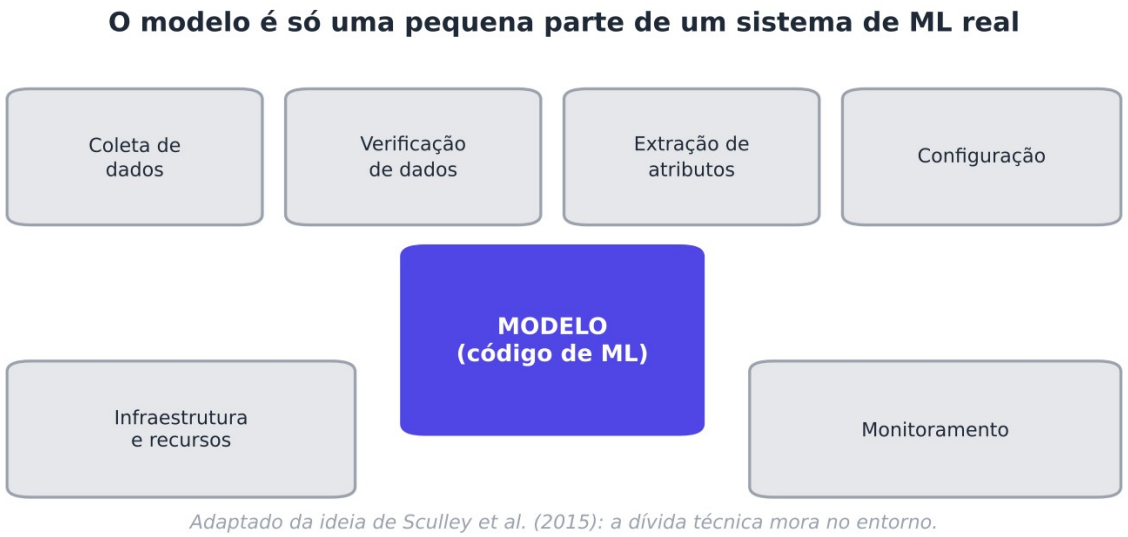


Figura 14.1 — O modelo (caixa pequena) é só uma parte de um sistema de aprendizado de máquina real (adaptado da ideia de Sculley et al., 2015).

Sculley *et al.* (2015) chamaram a atenção para o fato de que, em sistemas reais, o **código do modelo** é uma parte **pequena** do sistema: ao redor dele há coleta e verificação de dados, extração de atributos, configuração, infraestrutura, monitoramento e muito mais. Todo esse entorno gera uma **dívida técnica** escondida, que cobra juros ao longo do tempo.

> NO SEU DIA A DIA **Analogia: a cozinha do restaurante**
>
> Criar uma receita deliciosa (o modelo) é uma parte do trabalho de um restaurante. Servir **todo dia, para muita gente, com a mesma qualidade**, exige estoque, higiene, equipe, horários e controle de qualidade. O **MLOps** é o trabalho de "cozinha" do aprendizado de máquina.

MLOps (*Machine Learning Operations*) é o conjunto de práticas que combinam desenvolvimento de modelos e **operação** em produção: automação, versionamento, monitoramento e governança (Kreuzberger; Kühl; Hirschl, 2023). Neste capítulo, veremos o essencial, com experimentos reais.

14.2 O ciclo de vida de um modelo

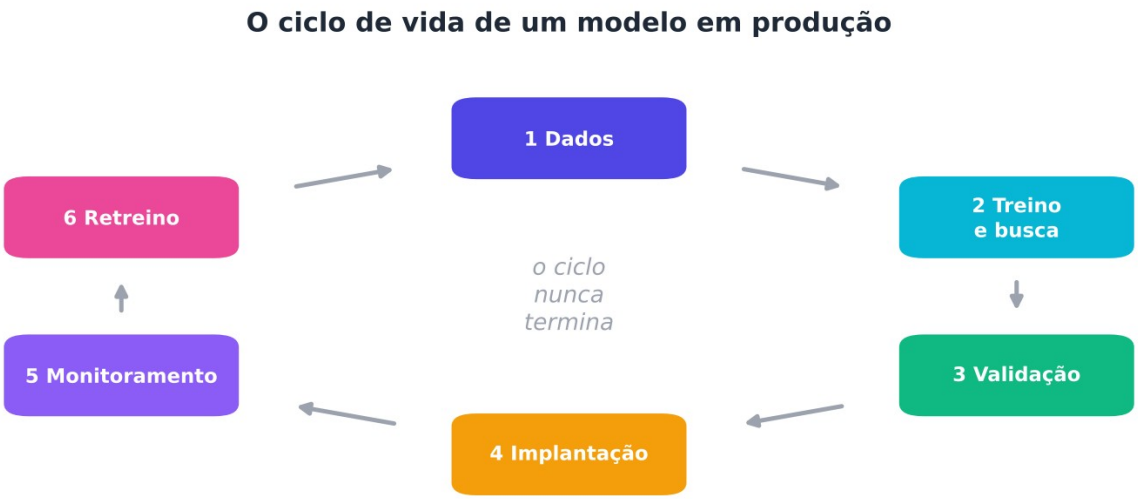


Figura 14.2 — O ciclo de vida de um modelo em produção: dados, treino, validação, implantação, monitoramento e retreino.

1. **Dados:** coleta, verificação de qualidade e versionamento.
2. **Treino e busca:** o AutoML dos capítulos anteriores.
3. **Validação:** teste final, justiça, explicações e custo.
4. **Implantação:** colocar o modelo para atender pedidos.
5. **Monitoramento:** acompanhar desempenho e dados em uso.
6. **Retreino:** atualizar quando o mundo muda, e voltar ao passo 1.

O AutoML ocupa o passo 2, mas o ciclo é **contínuo**: depois de implantado, o modelo envelhece.

14.3 Salvar e reproduzir

O primeiro requisito de produção é **salvar o pipeline inteiro**, não só o modelo: o pré-processamento (como o `StandardScaler`) tem de ir junto, senão o modelo recebe dados em outra escala. Com o scikit-learn, usamos a biblioteca `joblib`.

Também devemos registrar **como o modelo foi feito**: versões das bibliotecas, semente, parâmetros e uma **impressão digital** dos dados (um *hash*, que muda se qualquer valor mudar). Isso permite reproduzir e auditar o resultado.

PYTHON

```python
import warnings; warnings.filterwarnings("ignore")
import hashlib, json, os, sys, tempfile
import joblib, numpy as np, sklearn
from sklearn.datasets import load_breast_cancer
from sklearn.ensemble import HistGradientBoostingClassifier
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

X, y = load_breast_cancer(return_X_y=True)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.25, stratify=y, random_state=0)
pipe = make_pipeline(StandardScaler(), HistGradientBoostingClassifier(random_state=0)).fit(Xtr, ytr)

pasta = tempfile.mkdtemp()
joblib.dump(pipe, os.path.join(pasta, "modelo.joblib"))
ficha = {
    "python": sys.version.split()[0], "scikit-learn": sklearn.__version__, "numpy": np.__version__,
    "semente": 0, "divisao_teste": 0.25, "hash_treino": hashlib.sha256(Xtr.tobytes()).hexdigest()[:16],
    "acuracia_teste": round(float(pipe.score(Xte, yte)), 4),
}
json.dump(ficha, open(os.path.join(pasta, "ficha.json"), "w"), indent=2)

recarregado = joblib.load(os.path.join(pasta, "modelo.joblib"))
identicas = np.array_equal(pipe.predict_proba(Xte), recarregado.predict_proba(Xte))
print("Ficha do modelo:", json.dumps(ficha, ensure_ascii=False))
print("Previsões idênticas após recarregar:", identicas)
```

SAÍDA

```
Ficha do modelo: {"python": "3.13.16", "scikit-learn": "1.9.1", "numpy": "2.5.3", "semente": 0,
"divisa…
Previsões idênticas após recarregar: True
```

Atenção: o arquivo do modelo é código executável

Arquivos `joblib` e `pickle` podem **executar código** ao serem carregados. Nunca carregue um arquivo de modelo de origem desconhecida, e mantenha os arquivos de produção em locais protegidos.

14.4 Servir o modelo: lote ou tempo real

Há dois modos de uso.

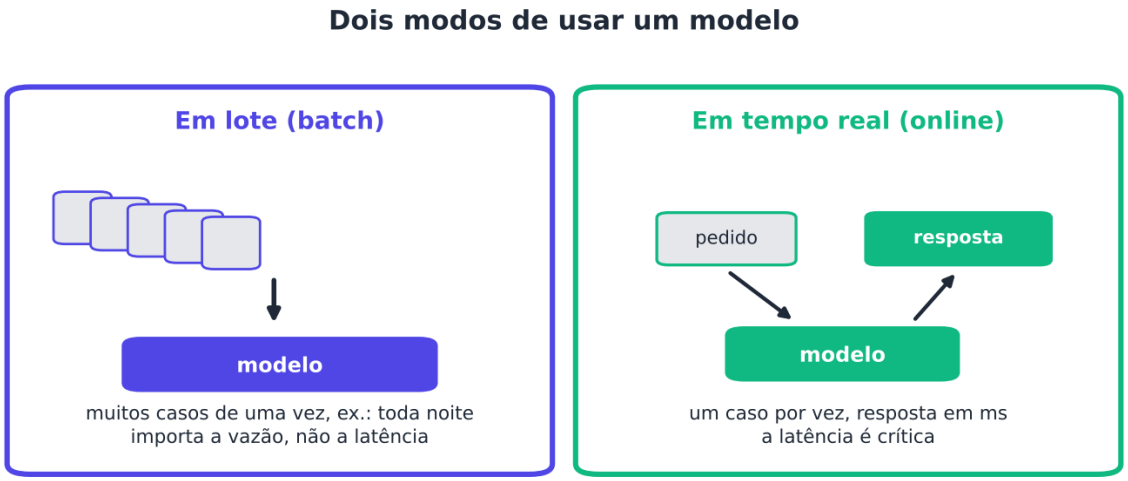


Figura 14.3 — Previsão em lote (muitos casos de uma vez, de tempos em tempos) e previsão em tempo real (um caso por vez, resposta imediata).

- **Em lote (*batch*):** o modelo processa **muitos casos de uma vez**, por exemplo toda noite. A latência (tempo de resposta) importa pouco, a vazão importa.
- **Em tempo real (*online*):** cada pedido chega **sozinho** e precisa de resposta em milissegundos, como na aprovação de uma compra. Aqui, a **latência por previsão** é crítica.

É nesse ponto que o custo dos **ensembles** (Capítulo 9) aparece. Vamos medir o tempo de **uma previsão isolada** e de **um lote de 10 mil**, em quatro modelos, no conjunto *Digits*.

```python
import warnings; warnings.filterwarnings("ignore")
import time
import numpy as np
from sklearn.datasets import load_digits
from sklearn.ensemble import RandomForestClassifier, HistGradientBoostingClassifier, VotingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC

X, y = load_digits(return_X_y=True)
lr = make_pipeline(StandardScaler(), LogisticRegression(max_iter=2000))
rf = RandomForestClassifier(300, random_state=0, n_jobs=1)
hgb = HistGradientBoostingClassifier(random_state=0)
vot = VotingClassifier([("lr", lr), ("rf", RandomForestClassifier(300, random_state=0, n_jobs=1)),
("svm", make_pipeline(StandardScaler(), SVC(probability=True, random_state=0)))], voting="soft")
modelos = {"Regressão logística": lr, "Floresta (300 árvores)": rf, "Boosting": hgb, "Ensemble de 3
modelos": vot}
lote = np.vstack([X] * 6)[:10000]
print(f"{'modelo':<26s} {'1 previsão (ms)':>16s} {'lote de 10 mil (s)':>20s}")
for nome, m in modelos.items():
    m.fit(X, y); um = X[:1]; m.predict(um)
    t = time.perf_counter()
    for _ in range(50): m.predict(um)
    um_ms = (time.perf_counter() - t) / 50 * 1000
    t = time.perf_counter(); m.predict(lote); tl = time.perf_counter() - t
    print(f"{nome:<26s} {um_ms:>16.2f} {tl:>20.2f}")
```

SAÍDA

```
modelo                      1 previsão (ms)   lote de 10 mil (s)
Regressão logística                    0.51                0.05
Floresta (300 árvores)                13.57                0.27
Boosting                               8.16                0.18
Ensemble de 3 modelos                 12.93                0.80
```

Os valores exatos dependem da máquina, e o importante é a **ordem de grandeza**. Nesta execução, a regressão logística respondeu a uma previsão isolada em cerca de 0,5 ms, contra cerca de 13 ms da floresta e do ensemble (algo como 25 vezes mais); em um lote de 10 mil casos, o ensemble levou cerca de 0,8 s contra 0,05 s da regressão logística. Em tempo real, uma diferença de milissegundos por pedido se multiplica por milhares de pedidos por segundo. Se o seu sistema precisa responder em poucos milissegundos, o ensemble campeão do AutoML pode ser **inviável**. Por isso, é bom incluir a **latência** (e o tamanho do modelo) como critério de escolha, como na fronteira de Pareto do Capítulo 11.

Dica: limite o custo desde o início

Defina o **orçamento de latência** e de memória **antes** de rodar o AutoML. Muitas ferramentas permitem restringir o tamanho do ensemble ou os algoritmos permitidos (por exemplo, com `estimator_list` no FLAML).

Para expor o modelo como serviço, usa-se uma **API** (interface que recebe pedidos e devolve respostas). O código abaixo mostra a ideia com a biblioteca FastAPI.

Atenção: código não executado

O código abaixo segue a documentação da biblioteca, mas **não foi executado** na preparação deste livro. Ele é um esboço: um serviço real exige validação de entradas, autenticação, registro de logs e testes.

PYTHON

```python
from fastapi import FastAPI
from pydantic import BaseModel
import joblib, numpy as np

app = FastAPI()
modelo = joblib.load("modelo.joblib")          # carregado uma vez, na partida do serviço

class Pedido(BaseModel):
    atributos: list[float]                     # os 30 atributos, na mesma ordem do treino

@app.post("/prever")
def prever(p: Pedido):
    x = np.array(p.atributos).reshape(1, -1)
    return {"classe": int(modelo.predict(x)[0]), "probabilidade": float(modelo.predict_proba(x)
[0].max())}
```

14.5 Monitoramento: o mundo muda

O dia em que o modelo vai para produção é o dia em que ele **começa a envelhecer**. Os dados do mundo mudam: a economia muda, os clientes mudam, os sensores se descalibram. Esse fenômeno tem dois nomes que convém distinguir (Gama *et al.*, 2014).

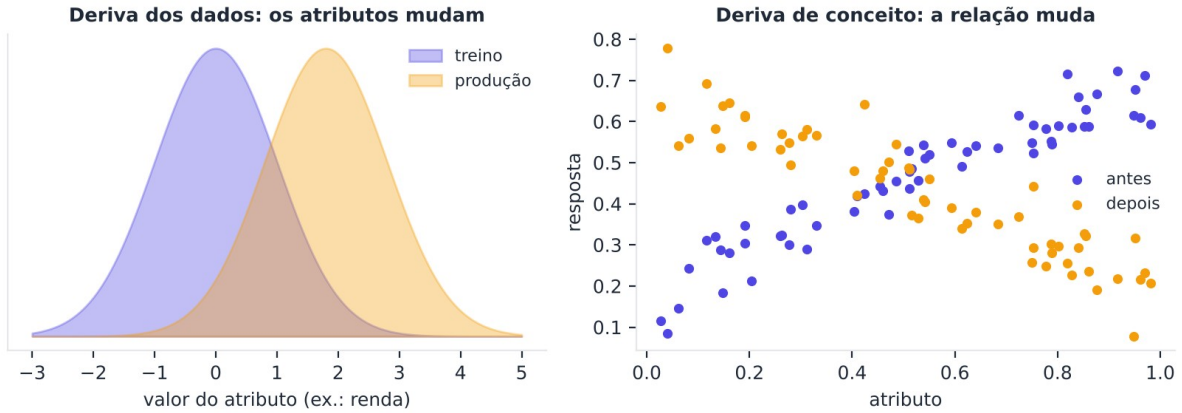


Figura 14.4 — Dois tipos de deriva: nos dados de entrada (os atributos mudam) e no conceito (a relação entre atributos e resposta muda).

- **Deriva dos dados** (*data drift* ou *covariate shift*): a **distribuição dos atributos** muda. Exemplo: após uma inflação, as rendas dos clientes sobem.
- **Deriva de conceito** (*concept drift*): a **relação** entre atributos e resposta muda. Exemplo: com a mesma renda, a chance de pagar passa a ser outra.

A deriva dos dados é **mais fácil de detectar**, pois não precisa dos rótulos (que, em produção, só chegam semanas depois). Duas ferramentas simples:

- **Teste de Kolmogorov-Smirnov (KS):** compara a distribuição de um atributo no treino e na produção. Um valor-p muito pequeno diz que as distribuições **diferem**.
- **PSI** (*Population Stability Index*, índice de estabilidade populacional): divide o atributo em faixas (pelos quantis do treino) e compara a proporção de casos em cada faixa. Quanto maior o PSI, maior a mudança. Uma convenção prática, muito usada em crédito, é considerar PSI abaixo de 0,10 como estável, entre 0,10 e 0,25 como alerta e acima de 0,25 como mudança grande; trate esses números como **regras práticas**, não como lei.

Vamos simular o que acontece em um modelo de crédito (os dados simulados do Capítulo 13). Depois da implantação, a **renda** dos clientes **sobe** gradualmente, por inflação, de 0% a +60% em seis meses, enquanto a capacidade real de pagamento não muda. Medimos mês a mês a deriva (KS e PSI) e a acurácia **contra a verdade**.

```python
import warnings; warnings.filterwarnings("ignore")
import json
import numpy as np
from scipy.stats import ks_2samp
from sklearn.ensemble import HistGradientBoostingClassifier

def gera(n, seed, infl=0.0):
    r = np.random.default_rng(seed)
    g = (r.random(n) < 0.3).astype(int); z = r.normal(0, 1, n)
    renda = np.exp(0.5 * z + 0.35 * r.normal(size=n) - 0.25 * g) * 3.0 * (1 + infl)       # a renda
sobe com a inflação
    emprego = np.clip(5 + 2 * z + r.normal(0, 2, n), 0, None); divida = np.clip(0.4 - 0.1 * z +
r.normal(0, 0.1, n), 0, 1)
    score = z + r.normal(0, 1, n)
    return np.c_[renda, emprego, divida, score], (z > 0).astype(int)

def psi(ref, novo, k=10):
    cortes = np.quantile(ref, np.linspace(0, 1, k + 1)[1:-1])
    a = np.bincount(np.searchsorted(cortes, ref), minlength=k) / len(ref)
```

```
    b = np.bincount(np.searchsorted(cortes, novo), minlength=k) / len(novo)
    a, b = np.clip(a, 1e-4, None), np.clip(b, 1e-4, None)
    return float(np.sum((b - a) * np.log(b / a)))

Xtr, ytr = gera(5000, seed=1)
modelo = HistGradientBoostingClassifier(random_state=0).fit(Xtr, ytr)
print(f"{'mês':>3s} {'inflação':>9s} {'acurácia':>9s} {'KS (renda)':>11s} {'valor-p':>9s} {'PSI
renda':>10s} {'PSI emprego':>12s}")
linhas = []
for mes in range(0, 7):
    infl = 0.10 * mes
    Xn, yn = gera(2000, seed=100 + mes, infl=infl)
    acc = modelo.score(Xn, yn); ks = ks_2samp(Xtr[:, 0], Xn[:, 0]); pr = psi(Xtr[:, 0], Xn[:, 0]); pe =
psi(Xtr[:, 1], Xn[:, 1])
    linhas.append({"mes": mes, "acc": acc, "ks": float(ks.statistic), "psi_renda": pr, "psi_emp": pe})
    print(f"{mes:>3d} {infl:>8.0%} {acc:>9.3f} {ks.statistic:>11.3f} {ks.pvalue:>9.1e} {pr:>10.3f}
{pe:>12.3f}")
json.dump(linhas, open("/home/user/claude/automl/build/f14_deriva.json", "w"))
```

SAÍDA

```
mês  inflação  acurácia  KS (renda)   valor-p  PSI renda  PSI emprego
  0       0%     0.851       0.029   1.9e-01      0.007        0.006
  1      10%     0.854       0.071   9.0e-07      0.038        0.008
  2      20%     0.856       0.119   4.3e-18      0.113        0.004
  3      30%     0.842       0.167   4.1e-35      0.165        0.009
  4      40%     0.836       0.207   6.1e-54      0.270        0.005
  5      50%     0.830       0.280   8.5e-99      0.431        0.006
  6      60%     0.808       0.313  4.4e-124      0.606        0.006
```

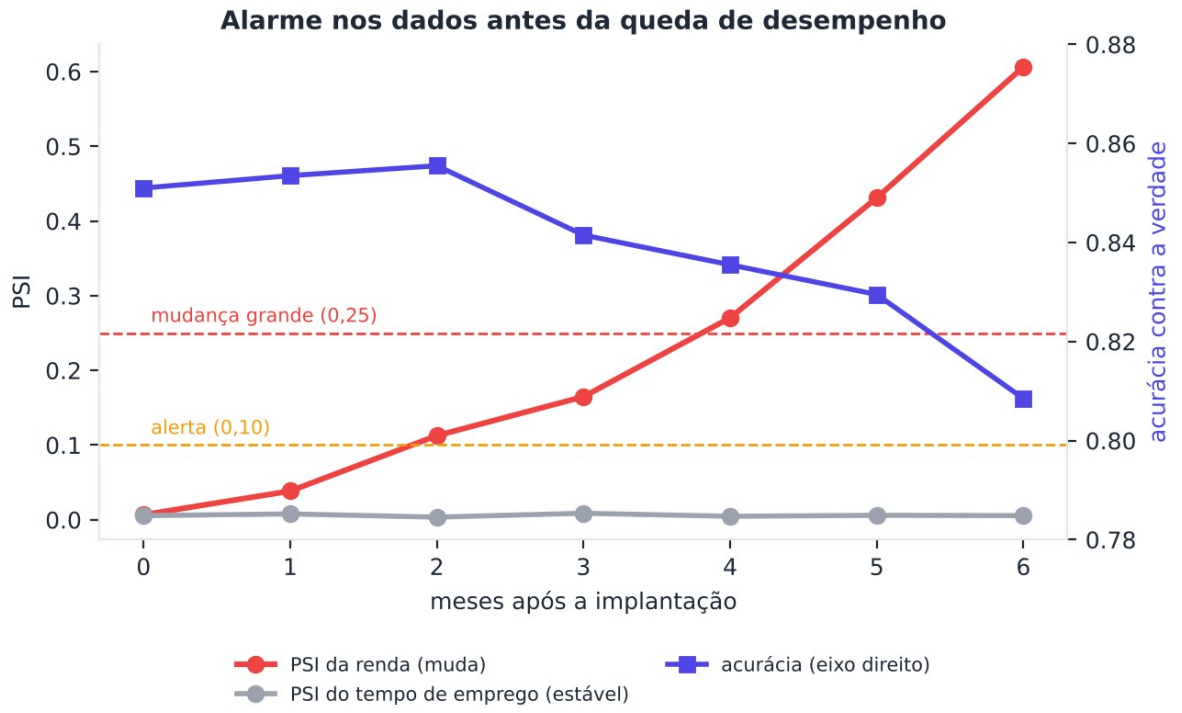


Figura 14.5 — Mês a mês: o PSI da renda dispara e a acurácia cai, enquanto o PSI de um atributo estável (tempo de emprego) fica próximo de zero.

Observe os sinais. O **PSI da renda** cresce a cada mês: passa de 0,10 no mês 2 (0,113) e de 0,25 no mês 4 (0,270). O **KS** rejeita a igualdade já no mês 1 (valor-p da ordem de 10^{-6}), quando a acurácia ainda não caiu (0,854, contra

0,851 no mês 0). A acurácia só cede de forma visível depois: 0,830 no mês 5 e 0,808 no mês 6. Essa é a utilidade do monitoramento dos **dados**: ele funciona como **alarme antecipado**, sem esperar os rótulos. O PSI de um atributo que **não mudou** (tempo de emprego) fica abaixo de 0,01, o que mostra que o alarme não dispara por qualquer motivo.

> **Atenção: alarme não é diagnóstico**
>
> Deriva dos dados **não** garante queda de desempenho (o modelo pode ser robusto à mudança), e a ausência de deriva dos dados **não** garante que o conceito não mudou. O alarme diz "olhe com atenção"; a decisão exige comparar com rótulos quando eles chegam.

14.6 Retreinar: campeão e desafiante

Quando o alarme dispara, a resposta natural é **retreinar**. Mas retreinar sempre, às cegas, também tem custo e risco. Uma prática segura é o **desafio entre campeão e desafiante**: o modelo em produção é o **campeão**; um modelo novo, treinado com dados recentes, é o **desafiante**. Os dois são avaliados nos **dados recentes com rótulos**, e o desafiante só **assume** se for **melhor com margem** (para não trocar por ruído).



Figura 14.6 — O desafio entre campeão e desafiante: o modelo novo só assume se ganhar de forma clara nos dados recentes.

No nosso exemplo, vamos supor que, no mês 6, chegaram os rótulos de 5.000 casos recentes. Usamos 2.000 para treinar o desafiante e 3.000 para o **desafio**.

```python
import numpy as np
from sklearn.ensemble import HistGradientBoostingClassifier

Xr, yr = gera(5000, seed=500, infl=0.60)                    # dados recentes, já com rótulos
X_ret, y_ret, X_des, y_des = Xr[:2000], yr[:2000], Xr[2000:], yr[2000:]
desafiante = HistGradientBoostingClassifier(random_state=0).fit(X_ret, y_ret)
n = len(y_des)
acc_c, acc_d = modelo.score(X_des, y_des), desafiante.score(X_des, y_des)
erro_padrao = np.sqrt(acc_d * (1 - acc_d) / n)
print(f"Dados do desafio: {n} casos recentes")
print(f"Campeão (treinado há 6 meses) : acurácia = {acc_c:.3f}")
print(f"Desafiante (retreinado)       : acurácia = {acc_d:.3f}  (erro padrão ≈ {erro_padrao:.3f})")
print("Diferença: %.3f -> %s" % (acc_d - acc_c, "desafiante assume" if acc_d - acc_c > 2 * erro_padrao
else "mantém o campeão"))
```

```
SAÍDA
Dados do desafio: 3000 casos recentes
Campeão (treinado há 6 meses) : acurácia = 0.826
Desafiante (retreinado)       : acurácia = 0.839  (erro padrão ≈ 0.007)
Diferença: 0.014 -> desafiante assume
```

Note o uso de uma **margem** (duas vezes o erro padrão) como critério: se a diferença fosse menor, a troca seria só ruído. Aqui, o desafiante ganhou por 1,4 ponto percentual, **no limite** da margem (0,014): a decisão de trocar é defensável, mas apertada, e uma equipe cuidadosa consideraria também o custo do retreino e repetiria o desafio com mais dados. É a mesma disciplina das comparações do Capítulo 10. Lembre ainda que a acurácia oscila de amostra para amostra (nas simulações do mês 6, ela variou de cerca de 0,81 a 0,85), o que também alerta contra decisões por diferenças pequenas.

14.7 Um pipeline completo, de ponta a ponta

Juntando tudo, o roteiro de um projeto de AutoML responsável é:



Figura 14.7 — Roteiro completo de um projeto de AutoML, do problema ao monitoramento.

1. **Defina o problema e a métrica** (Capítulo 1), incluindo o custo dos erros.
2. **Separe o teste final** antes de qualquer decisão (Capítulo 2) e verifique **vazamento** (Capítulo 3).
3. **Crie linhas de base** simples (Capítulos 2 e 10).
4. **Monte o pipeline** com pré-processamento dentro da validação (Capítulo 3).
5. **Busque** com orçamento (Capítulos 4 a 8), usando validação aninhada quando precisar estimar o ganho da busca.
6. **Combine** modelos se valer a pena (Capítulo 9) e compare ferramentas com rigor (Capítulo 10).
7. **Avalie no teste uma única vez**, por grupo e com incerteza (Capítulos 2 e 13).
8. **Explique e audite** (Capítulo 13).
9. **Implante com orçamento de latência**, registre a ficha do modelo (este capítulo).
10. **Monitore e retreine** com campeão e desafiante (este capítulo).

14.8 O projeto integrador final

O projeto integrador reúne tudo o que você aprendeu. Ele está no final deste capítulo, junto das atividades práticas, e pede que você execute o ciclo completo sobre um problema de crédito (simulado, com atributo sensível), passando por: linha de base, busca com validação, ensemble, teste final único, explicação, auditoria por grupo, ficha do modelo, monitoramento de deriva e relatório.

Um conselho de professor: **não busque a maior nota**. Busque o **projeto mais honesto**. Um relatório que diz "o AutoML empatou com a linha de base, e eis por que isso importa" vale mais que um que anuncia um ganho de 0,3 ponto sem teste de significância.

14.9 Boas práticas

1. **Salve o pipeline inteiro**, com pré-processamento, e registre a ficha do modelo (versões, semente, hash dos dados).
2. **Defina orçamento de latência e de memória** antes da busca.
3. **Prefira o modelo simples** quando ele cumprir o orçamento e empatar tecnicamente.
4. **Monitore dados e desempenho** em produção, com alarmes e limites definidos antes.
5. **Retreine com critério:** campeão e desafiante, com margem.
6. **Cuide da segurança:** não carregue arquivos de modelo desconhecidos; proteja os dados e o serviço.
7. **Mantenha pessoas no circuito** e um canal para contestar decisões (Capítulo 13).
8. **Documente tudo:** o que foi tentado, o que não funcionou e os limites do modelo.

Resumo do capítulo

MEMORIZE **Cartões de memorização**

- **O modelo é só uma parte do sistema:** a dívida técnica está no entorno (Sculley *et al.*, 2015).
- **MLOps:** práticas para operar modelos em produção (Kreuzberger *et al.*, 2023).
- **Ciclo de vida:** dados, treino, validação, implantação, monitoramento, retreino.
- **Reprodutibilidade:** salve o pipeline inteiro e registre versões, semente e hash dos dados.
- **Lote × tempo real:** a latência por previsão pesa muito no segundo caso; ensembles custam mais.
- **Deriva dos dados × deriva de conceito:** a primeira detecta-se sem rótulos (KS, PSI).
- **Alarme não é diagnóstico:** confirme com rótulos.
- **Campeão e desafiante:** o novo só assume se ganhar com margem.
- **Projeto honesto vale mais que projeto vistoso.**

Exercícios de múltipla escolha

Escolha **uma** alternativa em cada questão. O gabarito está no Apêndice A.

Questão 1. O que é MLOps?

a) Um tipo de rede neural.
b) Um algoritmo de busca de hiperparâmetros.
c) Uma métrica de acurácia.
d) O conjunto de práticas para desenvolver e operar modelos de ML em produção, com automação, versionamento e monitoramento.

Questão 2. O que dizem Sculley et al. (2015) sobre sistemas de ML reais?

a) O código do modelo é uma parte pequena; a dívida técnica está no entorno.
b) A dívida técnica não existe em ML.
c) O modelo é a única parte relevante.
d) Basta treinar o modelo.

Questão 3. Qual é a analogia do capítulo para MLOps?

a) A cozinha do restaurante: servir todos os dias com a mesma qualidade.
b) Uma viagem de avião.
c) Um campeonato.
d) Um jogo de tabuleiro.

Questão 4. Quais são as etapas do ciclo de vida do modelo?

a) Apenas treino e teste.
b) Apenas dados.
c) Dados, treino, validação, implantação, monitoramento e retreino.
d) Apenas implantação.

Questão 5. Por que salvar o pipeline inteiro e não só o modelo?

a) Porque o pré-processamento faz parte do modelo; sem ele, os dados chegariam em outra escala.
b) Porque o modelo não pode ser salvo sozinho.
c) Porque economiza memória.
d) Porque o pipeline é mais lento.

Questão 6. O que é a ficha do modelo?

a) Uma licença de uso.
b) O registro de versões, semente, parâmetros e hash dos dados usados na construção.
c) Um gráfico de desempenho.
d) A lista de clientes.

Questão 7. Para que serve o hash dos dados de treino?

a) Remove outliers.
b) Aumenta a acurácia.
c) Acelera o treino.
d) Funciona como impressão digital: muda se qualquer valor mudar.

Questão 8. Qual é o risco de carregar um arquivo joblib ou pickle desconhecido?

a) Só perda de memória.
b) Só perda de precisão.
c) Ele pode executar código ao ser carregado.
d) Nada: são arquivos de texto.

Questão 9. O que é previsão em lote?

a) Prever só o futuro.
b) Treinar com poucos dados.
c) Prever um caso por vez em milissegundos.
d) Processar muitos casos de uma vez, de tempos em tempos.

Questão 10. O que é previsão em tempo real?

a) Cada pedido chega sozinho e exige resposta imediata.
b) Processar tudo à noite.
c) Treinar o modelo a cada pedido.
d) Usar só ensembles.

Questão 11. O que mostrou o experimento de latência do capítulo?

a) O ensemble é o mais rápido.
b) Modelos simples respondem muito mais depressa que floresta e ensemble.
c) A latência independe do modelo.
d) Todos têm o mesmo custo.

Questão 12. Por que o orçamento de latência deve ser definido antes do AutoML?

a) Porque a latência não importa.
b) Porque o ensemble campeão pode ser inviável para o tempo de resposta exigido.
c) Porque simplifica o código.
d) Porque o AutoML não mede latência.

Questão 13. O que é deriva dos dados (data drift)?

a) O teste é maior.
b) A distribuição dos atributos de entrada muda.
c) A relação entre atributos e resposta muda.
d) O modelo é trocado.

Questão 14. O que é deriva de conceito (concept drift)?

a) A semente muda.
b) Apenas a média dos atributos muda.
c) A relação entre atributos e resposta muda.
d) O código muda.

Questão 15. Por que a deriva dos dados é mais fácil de detectar?

a) Porque os rótulos são gratuitos.
b) Porque é sempre pequena.
c) Porque o modelo avisa.
d) Porque não depende dos rótulos, que chegam depois.

Questão 16. O que faz o teste de Kolmogorov-Smirnov (KS)?

a) Compara a distribuição de um atributo no treino e na produção.
b) Escolhe hiperparâmetros.
c) Treina um modelo.
d) Mede a latência.

Questão 17. O que é o PSI?

a) Um limiar de classificação.
b) Um índice que compara a proporção de casos por faixa entre treino e produção.
c) Uma função de ativação.
d) Um tipo de ensemble.

Questão 18. Qual é uma convenção prática de leitura do PSI?

a) Não há convenção.
b) Sempre abaixo de 0,001.
c) Sempre acima de 1.
d) Abaixo de 0,10 estável; 0,10 a 0,25 alerta; acima de 0,25 mudança grande.

Questão 19. No experimento de inflação, quando o KS rejeitou a igualdade pela primeira vez?

a) Já no mês 1, quando a acurácia ainda não tinha caído.
b) No mês 0.
c) Nunca.
d) Só no mês 6.

Questão 20. O que o PSI do tempo de emprego (atributo estável) mostrou?

a) Disparou como o da renda.
b) Ficou acima de 1.
c) Não foi calculado.
d) Ficou abaixo de 0,01, sem alarme falso.

Questão 21. Por que 'alarme não é diagnóstico'?

a) Porque alarmes são sempre falsos.
b) Porque o PSI é inválido.
c) Porque rótulos não importam.
d) Deriva dos dados não garante queda de desempenho, e a ausência dela não garante que o conceito não mudou.

Questão 22. O que é o desafio entre campeão e desafiante?

a) Treinar dois modelos iguais.
b) Uma competição de acurácia no treino.
c) Comparar o modelo em produção com um novo modelo em dados recentes, trocando só se o novo ganhar com margem.
d) Trocar o modelo a cada semana.

Questão 23. Por que o desafiante precisa ganhar 'com margem'?

a) Para aumentar o custo.
b) Para não trocar o modelo por diferenças que são só ruído amostral.
c) Porque a regra é arbitrária.
d) Porque o campeão não pode perder.

Questão 24. No Exercício 4, em quantas das 20 repetições o desafiante assumiu?

a) Em 9 de 20.
b) Em todas.
c) Em 20 de 20.
d) Em nenhuma.

Questão 25. O que o Exercício 3 mostrou sobre o PSI com poucos dados?

a) O PSI não funciona com Wine.
b) Com amostras pequenas, o PSI é instável e pode passar do alerta sem deriva alguma.
c) O PSI é sempre zero.
d) O PSI é sempre exato.

Questão 26. Qual foi o resultado de latência do ensemble no projeto integrador?

a) Não foi medida.
b) Cerca de 0,1 ms.
c) Exatamente 20 ms.
d) Cerca de 50 ms por previsão, acima do orçamento de 20 ms.

Questão 27. O que a busca escolheu no projeto integrador e o que isso indica?

a) A floresta, por ser a mais lenta.
b) O LightGBM, por ser o mais complexo.
c) A regressão logística: o problema simulado é quase linear e o modelo simples bastou.
d) Nada.

Questão 28. Qual conselho final de professor aparece no capítulo?

a) Esconda os limites do modelo.
b) Busque o projeto mais honesto, não a maior nota.
c) Evite baselines.
d) Busque sempre o maior ensemble.

Questão 29. Por que o Exercício 1 recarrega o modelo e compara as previsões?

a) Para reduzir o tamanho do arquivo.
b) Para aumentar a acurácia.
c) Para escolher hiperparâmetros.
d) Para confirmar que o modelo salvo reproduz exatamente as previsões do original.

Questão 30. Qual é a melhor resposta quando o PSI de um atributo passa de 0,25?

a) Investigar: comparar o desempenho com rótulos recentes e avaliar um retreino.
b) Ignorar o alarme.
c) Retreinar sempre, às cegas.
d) Apagar o atributo.

Exercícios discursivos

Responda com suas palavras, em poucas linhas. Compare depois com a resposta-modelo do Apêndice A.

1. Explique, com a analogia da cozinha, por que um modelo no notebook ainda não é um produto.
2. Resuma a mensagem de Sculley et al. (2015) sobre dívida técnica em ML.
3. Descreva as seis etapas do ciclo de vida de um modelo.
4. O que deve conter a ficha de um modelo e por quê?
5. Por que não se deve carregar arquivos joblib/pickle de origem desconhecida?
6. Compare previsão em lote e em tempo real.
7. Por que ensembles podem ser inviáveis em tempo real? Use os números do capítulo.
8. Diferencie deriva dos dados e deriva de conceito, com exemplos.
9. Explique o KS e o PSI como ferramentas de monitoramento.
10. Interprete a Figura de monitoramento do capítulo.
11. Por que 'alarme não é diagnóstico'? Dê um exemplo dos dados do livro.
12. Descreva o desafio entre campeão e desafiante e a regra de margem.
13. O que o Exercício 4 mostrou sobre a decisão de trocar o modelo?
14. Por que o PSI é instável com poucos dados? Dê o exemplo do Exercício 3.
15. Liste os dez passos do roteiro completo de um projeto de AutoML.
16. Resuma o resultado do projeto integrador e a recomendação.
17. Por que o projeto integrador diz que o ensemble 'não se justifica'?
18. Quais salvaguardas você exigiria para implantar um modelo de crédito auditado?
19. O que significa 'busque o projeto mais honesto'?
20. Por que o PSI e o KS não substituem a avaliação com rótulos recentes?

Exercícios práticos em Python (scikit-learn)

Resolva os exercícios no seu computador (ou no Google Colab). Eles usam conjuntos de dados da UCI que já vêm no scikit-learn ou dados simulados. Tente resolver **antes** de consultar a solução, no Apêndice D. As saídas apresentadas lá foram obtidas com as versões de bibliotecas listadas na Apresentação.

Exercício 1 — Ficha do modelo e reprodutibilidade

NÍVEL: BÁSICO

No conjunto **Wine** (UCI), treine um pipeline (`StandardScaler` + `LogisticRegression`), salve-o com `joblib` e escreva uma ficha com versões (Python, scikit-learn), semente e um *hash* dos dados de treino. Recarregue o modelo e confirme que as previsões são idênticas. Depois, altere **um único valor** dos dados de treino e mostre que o hash muda.

Pergunta: por que o hash dos dados é útil? Por que salvar o pipeline inteiro, e não só o modelo?

Exercício 2 — Quanto custa cada previsão?

NÍVEL: INTERMEDIÁRIO

No **Digits** (UCI), ajuste uma floresta aleatória com 10, 100 e 500 árvores (`n_jobs=1`). Meça o tempo de **uma previsão isolada** (média de 50 repetições) e o de um **lote de 5.000** casos.

Pergunta: como o custo cresce com o número de árvores? Que tipo de aplicação sofreria mais?

Exercício 3 — Detectando deriva com KS e PSI

NÍVEL: INTERMEDIÁRIO

No **Wine** (UCI), use 70% dos dados como referência (treino) e 30% como "produção". Calcule o KS e o PSI de todos os atributos e mostre os três com maior PSI. Depois, **simule uma deriva**: multiplique por 1,3 o atributo `alcohol` da "produção" e recalcule KS e PSI desse atributo.

Pergunta: os dados sem deriva ficaram abaixo do limite de alerta (0,10)? O alarme detectou a deriva simulada?

Exercício 4 — Quando trocar o campeão?

NÍVEL: AVANÇADO

No crédito simulado (função abaixo), treine o campeão em dados de `infl=0`. Para 20 repetições (sementes diferentes), gere dados recentes com `infl=0,5`, treine um desafiante com 2.000 casos e compare ambos em 2.000 casos de desafio. Conte em quantas repetições o desafiante "assume" pela regra de margem (diferença > 2 erros padrão).

Pergunta: o desafiante assume sempre? Que regra você usaria para evitar trocas por ruído?

> SAIBA MAIS **Para ir além: bases públicas da UCI e do Kaggle**
>
> - **UCI:** Adult, Statlog German Credit, Bank Marketing (marketing bancário, com muitos exemplos).
> - **Kaggle:** Give Me Some Credit, Home Credit Default Risk e Titanic (para praticar o fluxo completo).
> - **Desafio:** troque a simulação por uma base real e repita o projeto, comparando FLAML, Auto-sklearn ou AutoGluon com o seu mini-AutoML, com o mesmo orçamento e a mesma validação.
>
> Estes conjuntos não foram usados nem testados nos códigos do livro. Confira, nos sites, a descrição, a licença e as colunas antes de usá-los.

Projeto integrador final

Projeto integrador: AutoML de ponta a ponta, com ética e produção

Fonte dos dados. Crédito **simulado** (função abaixo), com 20% do grupo minoritário (`p_min=0.25`), viés histórico de 0,5 (`vies=0.5`) e semente 11. Para uma versão com dados reais, use uma base pública da UCI ou do Kaggle com atributo sensível (por exemplo, *Adult*, *German Credit* ou *Give Me Some Credit*), respeitando licenças e restrições legais.

Contexto. Você é a pessoa responsável por entregar, a um banco fictício, um modelo de aprovação de crédito desenvolvido com AutoML, **pronto para auditoria e para produção**. O projeto integra todos os capítulos do livro.

Atenção ética. Os dados são simulados. Em dados reais, o uso de atributos sensíveis em decisões pode ser restrito por lei, e a auditoria exige especialistas jurídicos e representantes das pessoas afetadas.

Tarefas.

1. **Definição do problema** (Cap. 1): escreva o objetivo, a métrica (acurácia contra o histórico, mas também acompanhe a taxa de seleção e o TPR por grupo) e as restrições (latência máxima de 20 ms por previsão).
2. **Dados e validação** (Caps. 2 e 3): separe 25% para **teste final** (estratificado), do restante separe 25% para **validação** dos pesos do ensemble; não use o atributo grupo nos modelos.
3. **Linhas de base** (Caps. 2 e 10): regressão logística (com `StandardScaler`) e floresta com parâmetros padrão.
4. **Busca** (Caps. 4 a 6): com o Optuna (TPE, semente fixa), 30 tentativas, espaço CASH condicional com `LogisticRegression`, `RandomForestClassifier` e `LGBMClassifier`, avaliando por validação cruzada de 3 partes no treino.
5. **Ensemble** (Cap. 9): média das probabilidades do **melhor modelo de cada algoritmo** encontrado na busca (até 3 modelos, para garantir diversidade), treinados no treino completo e comparados, na validação, com o melhor modelo único.
6. **Teste final único** (Cap. 2): compare linha de base, melhor único e ensemble no teste, uma vez; discuta se o ganho é maior que o erro padrão.
7. **Explicação e auditoria** (Cap. 13): importância por permutação do modelo escolhido; taxa de seleção e TPR real por grupo; AUC para prever o grupo pelos atributos; limiares por grupo para igualar a seleção.
8. **Produção** (Cap. 14): salve o pipeline e a ficha (versões, semente, hash dos dados); meça a latência de uma previsão e confira o orçamento de 20 ms; simule uma deriva (`infl=0,4`) e calcule o PSI da renda e a acurácia.
9. **Relatório final** (12 a 15 linhas): recomendação (implantar ou não?), salvaguardas, limites e custo da busca.

Entregáveis. Um notebook (ou script) comentado, as tabelas pedidas e o relatório.

A solução de referência está no Apêndice E.

Apêndice A — Gabarito dos exercícios

Esta seção traz a resposta de todas as questões. Nas questões de múltipla escolha, aparecem a alternativa correta e uma breve justificativa. Nas discursivas, aparece uma **resposta-modelo**: a sua resposta pode ser redigida de outro modo, desde que contemple as mesmas ideias.

Capítulo 1 — Bem-vindo ao AutoML

Múltipla escolha

Questão	Resposta	Justificativa
1	a)	AutoML automatiza decisões como preparo, atributos, algoritmo e hiperparâmetros; a pessoa continua responsável pelo problema e pela decisão final.
2	b)	O ajudante que testa centenas de receitas e investiga as mais promissoras corresponde ao AutoML.
3	c)	Hiperparâmetros, como a profundidade máxima de uma árvore, são definidos antes do treino; os parâmetros é que são aprendidos.
4	c)	CASH vem de Combined Algorithm Selection and Hyperparameter optimization.
5	c)	As duas escolhas são interdependentes: cada algoritmo traz seus próprios botões e seu desempenho depende deles.
6	b)	Thornton e colaboradores (2013) propuseram o Auto-WEKA, que formalizou o CASH.
7	c)	Wolpert (1996) mostrou que, sem suposições sobre o problema, nenhum algoritmo supera os demais em média.
8	b)	Como o melhor algoritmo depende dos dados, a avaliação empírica é indispensável.
9	c)	O problema da seleção de algoritmos pergunta qual algoritmo resolve melhor um dado problema.
10	a)	O espaço de busca reúne todas as configurações possíveis que a estratégia de busca pode explorar.
11	d)	Todo AutoML define o que pode escolher, como escolhe o próximo candidato e como avalia cada candidato.
12	a)	O mini-AutoML sorteava 40 candidatos do espaço CASH e guardava o melhor.
13	b)	Cada candidato foi avaliado com cross_val_score e cv=5.
14	d)	Um pipeline encadeia etapas como preparo, atributos e modelo.
15	a)	Coletar e limpar dados é a etapa mais trabalhosa e, hoje, ainda muito dependente de pessoas.
16	d)	A maioria das ferramentas faz busca inteligente e monta pipelines, mas não cobre todo o ciclo.
17	d)	O AutoML otimiza o que recebe; erros, vieses e vazamentos nos dados permanecem.
18	d)	Ao testar muitas configurações, a melhor nota de validação pode ser inflada pela sorte, o que é overfitting da busca.
19	c)	Cawley e Talbot (2010) discutem o sobreajuste na seleção de modelos e o viés resultante na avaliação.
20	d)	O TPOT evolui árvores de pipelines por programação genética (Olson e Moore, 2016).
21	d)	Feurer e colaboradores (2015) combinaram meta-aprendizado e ensembles sobre o scikit-learn.
22	a)	Ambos são métodos multifidelidade que gastam pouco com candidatos pouco promissores.
23	d)	O AMLB (Gijsbers et al., 2024) é um benchmark aberto e extensível para frameworks de AutoML.
24	b)	Ao rodar várias divisões e tirar a média, reduz-se a dependência de uma divisão específica.
25	c)	As decisões de contexto, ética e confiança continuam humanas.
26	b)	O AutoML otimiza o que foi mandado otimizar; a escolha da métrica é responsabilidade humana.
27	a)	São 178 vinhos, 13 medidas químicas e 3 categorias: um problema de classificação.
28	b)	O k-vizinhos usa distâncias; colunas com números grandes dominariam o cálculo sem padronização.
29	c)	Em dados pequenos ou com forte conhecimento especializado, um bom modelo manual pode competir.
30	c)	Justamente porque os algoritmos precisam de ajustes, e em grande número, o AutoML é útil.

Discursivas (respostas-modelo)

1. AutoML é automatizar a busca pelas melhores escolhas na construção de um modelo. Analogia: ao escolher um plano de celular, em vez de comparar 100 planos à mão, um aplicativo compara tudo e mostra os três melhores; você decide.
2. Hiperparâmetro é um botão de ajuste definido antes do treino. Exemplos: profundidade máxima e mínimo de amostras por folha na árvore de decisão; número de vizinhos e tipo de peso no k-vizinhos.
3. É a seleção combinada de algoritmo e de seus hiperparâmetros. É difícil porque o espaço de busca é grande, tem estrutura condicional (cada algoritmo tem seus botões) e cada avaliação exige treinar um modelo, o que é caro.
4. Nenhum algoritmo é o melhor para todos os problemas. Como o melhor depende dos dados, é preciso testar e comparar empiricamente, e o AutoML automatiza essa comparação em larga escala.
5. Coletar e limpar; preparar; criar e selecionar atributos; escolher o algoritmo; ajustar hiperparâmetros; combinar modelos; avaliar e implantar. O AutoML clássico automatiza principalmente de atributos até a combinação de modelos, com foco em algoritmo e hiperparâmetros.
6. Porque ele otimiza o que lhe é pedido. Seguem humanas: formular o problema, definir a métrica adequada, verificar a qualidade e o viés dos dados, interpretar resultados e decidir se o modelo deve ser usado.
7. Espaço de busca (4 algoritmos com seus hiperparâmetros), estratégia de busca (sorteio aleatório) e estimativa de desempenho (validação cruzada de 5 partes).
8. O AutoML ajusta o modelo aos dados disponíveis. Se os dados têm erros ou vieses (por exemplo, histórico de crédito que excluía um grupo), o modelo aprende e reproduz o problema, só que com melhor ajuste.
9. É escolher uma configuração cuja boa nota de validação se deve em parte ao acaso. Ao testar muitas configurações, a melhor nota tende a ser otimista; o desempenho em dados novos costuma ser pior que o observado na validação.
10. Manual: usa intuição e contexto, mas testa poucas opções, é lenta e difícil de reproduzir. Automática: testa muitas opções, registra tudo e é reproduzível, mas tem custo computacional e pode superajustar à validação; não substitui o julgamento humano.
11. O Auto-WEKA formalizou e resolveu o CASH com otimização bayesiana sobre o WEKA. O Auto-sklearn levou a ideia ao ecossistema scikit-learn e acrescentou meta-aprendizado e combinação automática de modelos.
12. O TPOT representa pipelines como árvores e os evolui com programação genética: seleciona os melhores, recombina e muda partes, repetindo por gerações. Como na evolução, boas soluções se propagam e variações exploram novas possibilidades.
13. Eles avaliam os candidatos com pouco orçamento no início e descartam cedo os que parecem ruins, concentrando o orçamento nos promissores, em vez de gastar o mesmo tempo com todos.
14. Tempo de computação, consumo de energia e emissões associadas, e dificuldade de auditar milhares de tentativas. Exemplo: treinar milhares de modelos consome energia significativa, o que motiva ferramentas econômicas e relatórios de orçamento computacional.
15. É dividir os dados em partes, treinando em algumas e testando na restante, em rodízio, e tirar a média. Reduz a dependência de uma divisão específica e dá uma estimativa mais estável do desempenho.
16. Procura-se o par (algoritmo, ajustes) que minimize o erro médio em dados de validação. A média sobre k divisões evita que a escolha dependa de uma divisão favorável por sorte.
17. Fixar a semente permite reproduzir o experimento; repetir com sementes diferentes mostra a variabilidade dos resultados, evitando conclusões baseadas em uma única execução afortunada.
18. Em dados pequenos, com forte conhecimento de domínio embutido em atributos ou restrições específicas, um modelo manual bem pensado pode igualar ou superar o AutoML. A comparação deve ser feita com protocolo justo.

19. Exemplo: prever cancelamento de clientes de uma academia. O AutoML testaria algoritmos e ajustes; a pessoa decidiria a definição de cancelamento, a métrica (por exemplo, revocação), a qualidade dos dados, o uso ético da previsão e a ação resultante.
20. Porque ninguém ainda automatiza com segurança o ciclo todo, da definição do problema à decisão final. Continuam humanas a formulação do problema, a ética, a validação do contexto e a responsabilização pelas decisões.

Capítulo 2 — Aprendizado de máquina em um capítulo

Múltipla escolha

Questão	Resposta	Justificativa
1	a)	Linhas são exemplos; colunas de entrada são atributos.
2	c)	O alvo (ou rótulo) é a resposta correta que o modelo aprende a prever.
3	c)	Quando o alvo é um número contínuo, trata-se de regressão.
4	a)	O alvo é uma categoria; portanto, classificação.
5	c)	A árvore de decisão divide os dados por perguntas sucessivas sobre os atributos.
6	a)	Ele consulta os k vizinhos mais próximos e adota a classe mais frequente entre eles.
7	c)	Sem padronização, a coluna com valores maiores domina a distância entre exemplos.
8	b)	Breiman (2001) propôs combinar muitas árvores com aleatorização nos exemplos e nos atributos.
9	a)	O boosting treina modelos em sequência, focando o que os anteriores erraram.
10	c)	Parâmetros são aprendidos pelo modelo; hiperparâmetros são "botões" fixados antes do treino.
11	c)	A profundidade máxima é definida antes do treino. Os demais itens são aprendidos.
12	d)	O modelo não captura o padrão dos dados, resultando em erro alto também no treino.
13	a)	O modelo "decora" o treino e perde capacidade de generalizar.
14	b)	Modelos mais flexíveis se ajustam cada vez melhor ao treino, mesmo quando isso prejudica a generalização.
15	d)	Um bom modelo deve funcionar em dados novos, e não apenas nos exemplos vistos.
16	a)	A validação serve para decidir como o modelo deve ser configurado.
17	c)	Se o teste influencia decisões, deixa de medir o desempenho honesto.
18	c)	A estratificação preserva a distribuição das classes nas partições.
19	b)	A média sobre várias divisões reduz a variabilidade da estimativa (Kohavi, 1995).
20	b)	Cada parte serve de validação uma vez; os exemplos aparecem no treino nas demais.
21	d)	Ele acerta todos os saudáveis (95%) e erra todos os doentes.
22	a)	A revocação seria zero, pois nenhum doente é encontrado.
23	b)	Precisão = VP / (VP + FP).
24	d)	Revocação = VP / (VP + FN).
25	a)	Prioriza-se reduzir falsos negativos, isto é, aumentar a revocação.
26	d)	A linha de base dá sentido à nota do modelo.
27	b)	MAE é o erro absoluto médio.
28	c)	$R^2 = 0$ equivale a prever a média; valores negativos indicam pior que isso.
29	d)	O melhor entre muitos tende a ter nota otimista; em dados novos, voltou ao acaso.
30	c)	StratifiedKFold preserva a proporção das classes em cada parte.

Discursivas (respostas-modelo)

1. Exemplo é uma linha (um cliente). Atributo é uma coluna de entrada (idade, renda). Alvo é o que se quer prever (por exemplo, se o cliente comprou).
2. Classificação prevê categorias (spam ou não spam). Regressão prevê números contínuos (preço de um imóvel).
3. É como perguntar a opinião dos k vizinhos mais parecidos e seguir a maioria. Limitação: sensível à escala dos atributos e custoso com muitos dados, pois calcula distâncias.

4. Porque combina o voto de muitas árvores treinadas com amostras diferentes; os erros individuais tendem a se cancelar, reduzindo a variância.
5. Parâmetros são aprendidos no treino; hiperparâmetros são definidos antes. O AutoML busca bons hiperparâmetros (e algoritmos); os parâmetros, o algoritmo aprende sozinho.
6. No subajuste, ambos os erros são altos. No sobreajuste, o erro de treino é baixo e o de validação, alto. A complexidade ideal fica onde o erro de validação é mínimo.
7. Porque ele cai com a complexidade e pode chegar a zero por decoração. Só o desempenho em dados não usados no treino indica a capacidade de generalizar.
8. Treino são os exercícios com que se aprende. Validação são os simulados usados para ajustar o modo de estudar. Teste é a prova final, aberta uma vez.
9. O teste passa a influenciar decisões e deixa de ser independente; a nota fica otimista e já não mede o desempenho real em dados novos.
10. Divide-se o conjunto em k partes; treina-se em k-1 e valida-se na restante, rodando k vezes e tirando a média. Reduz a dependência de um sorteio específico.
11. Com 5% de positivos, um modelo que sempre responde negativo acerta 95%, mas encontra 0% dos positivos. A revocação revelaria a falha.
12. Precisão: positivos previstos que eram reais (filtro de spam). Revocação: positivos reais encontrados (triagem médica). F1: equilíbrio entre as duas, útil quando ambos os erros importam.
13. Precisão = 30/40 = 0,75. Revocação = 30/50 = 0,60. F1 = 2×0,75×0,60/(0,75+0,60) ≈ 0,667.
14. Dá um ponto de comparação mínimo; se o modelo não a supera com folga, não aprendeu algo útil. Em classificação, prever sempre a classe mais comum (DummyClassifier).
15. Ambos medem erro na unidade do alvo. O RMSE pune mais os erros grandes, sendo mais adequado quando erros grandes são particularmente custosos.
16. Mede a fração da variação dos dados explicada pelo modelo. R^2 negativo significa desempenho pior que prever sempre a média.
17. Entre muitos candidatos, o de melhor nota de validação tende a ter nota inflada pela sorte. Um AutoML que testa milhares de modelos reporta, portanto, uma nota otimista do vencedor.
18. Guardar um conjunto de teste intocado para avaliar apenas o modelo final, ou usar validação aninhada, e relatar essa nota, não a da busca.
19. O AutoML busca maximizar a métrica fornecida. Em dados desbalanceados, escolher acurácia levaria a modelos que ignoram a classe rara; o ideal seria F1, revocação ou AUC.
20. Um aluno decora as respostas do simulado e vai mal na prova com questões diferentes. Equivale ao modelo que acerta o treino e erra em dados novos.

Capítulo 3 — Pipelines e pré-processamento

Múltipla escolha

Questão	Resposta	Justificativa
1	d)	Imputar é substituir valores ausentes por estimativas, como média, mediana ou valor mais frequente.
2	c)	A mediana é pouco afetada por valores muito grandes ou muito pequenos.
3	c)	Média e mediana não se aplicam a categorias; usa-se a categoria mais frequente ou uma categoria "ausente".
4	d)	Em alguns problemas, o fato de um valor faltar já carrega informação (por exemplo, renda não informada).
5	d)	Sem padronização, a coluna com números maiores pesa mais na distância.
6	d)	Árvores só comparam "maior ou menor" que um limite, de modo que a escala não altera as divisões.
7	d)	O resultado tem média 0 e desvio padrão 1.
8	b)	Reescala linearmente os valores usando o mínimo e o máximo.
9	d)	O modelo entenderia que existe uma relação de ordem ou distância entre cidades.
10	a)	Cada categoria vira uma coluna que indica se ela está presente.
11	b)	A ordem das categorias deve refletir uma ordem real.
12	c)	Isso evita falhas quando aparecem categorias novas na produção.
13	c)	Ele permite, por exemplo, escalar as colunas numéricas e codificar as categóricas.
14	a)	O último passo é o modelo; os anteriores são transformações.
15	b)	Isso é o que evita vazamento quando o pipeline é usado dentro da validação cruzada.
16	d)	O modelo parece melhor do que é, porque "viu" informação que não deveria.
17	c)	A seleção de atributos olhou os rótulos das partes de validação.
18	a)	Escolher os atributos mais associados ao alvo usando todos os dados contamina a validação.
19	c)	Assim cada etapa é reaprendida só com o treino em cada divisão.
20	c)	Dois sublinhados separam o nome da etapa do nome do parâmetro.
21	a)	$3 \times 3 \times 3 \times 3 = 81$.
22	a)	A análise de componentes principais resume os dados em poucas dimensões.
23	a)	Arquivos joblib e pickle podem executar código durante o carregamento.
24	a)	Quem receber o arquivo aplica as mesmas transformações aos novos dados.
25	a)	A semente aleatória torna o experimento reproduzível.
26	b)	Eles reduzem a assimetria de variáveis como renda.
27	b)	Colunas praticamente constantes não ajudam a separar as classes.
28	b)	A interpretação semântica dos dados continua sendo humana.
29	b)	O SimpleImputer substitui valores ausentes por uma estatística da coluna, como a mediana.
30	c)	A escala foi calculada com informação da validação, que não deveria ser conhecida no treino.

Discursivas (respostas-modelo)

1. Porque chegam com problemas que impedem ou prejudicam o aprendizado: valores ausentes, atributos em escalas muito diferentes, categorias em texto e atributos inúteis. Modelos como a regressão logística recusam NaN e texto.

2. Média (dados sem extremos), mediana (dados com valores extremos), mais frequente (categóricas) e vizinhos (quando exemplos parecidos dão boas estimativas). O indicador de ausência pode ser acrescentado quando faltar é informativo.
3. Se o álcool varia de 11 a 15 e a prolina de 300 a 1600, uma diferença de 100 na prolina pesa muito mais que uma de 2 no álcool na distância euclidiana; o álcool é praticamente ignorado.
4. Padronização: média 0 e desvio 1. Min-máx: valores entre 0 e 1. Robusta: usa mediana e quartis, sofrendo menos com valores extremos.
5. Árvores comparam apenas se o valor é maior que um limite, o que não depende da escala. SVM e regressão logística usam produtos e distâncias, sensíveis à escala dos atributos.
6. Cada categoria vira uma coluna de 0 ou 1. Evita que o modelo interprete números atribuídos às categorias como se houvesse ordem ou distância entre elas.
7. Quando há ordem natural entre as categorias, como nível de escolaridade (fundamental, médio, superior) ou tamanho (P, M, G).
8. Gera um número enorme de colunas, aumentando memória e risco de sobreajuste. Na produção, usar handle_unknown="ignore" para tolerar categorias não vistas.
9. É uma sequência encadeada de etapas até o modelo. Ele garante a mesma ordem de transformações, reduz erros e, usado dentro da validação cruzada, evita vazamento.
10. Aplica tratamentos distintos a grupos de colunas. Exemplo: imputar pela mediana e padronizar as numéricas; imputar pelo mais frequente e aplicar one-hot às categóricas.
11. É quando informação de validação/teste escapa para o treino. Analogia: estudar com a prova na mão; a nota sobe sem medir aprendizado, e o modelo falha em dados reais.
12. Com dados sem sinal, selecionar os atributos com todos os dados antes da validação gerou acurácia bem acima de 50%; dentro do pipeline voltou perto do acaso. Ensina que a seleção deve ser parte do pipeline.
13. Calcular a média e o desvio da padronização com todos os dados antes de dividir; imputar valores ausentes com estatísticas calculadas incluindo o conjunto de teste; usar uma coluna que só é conhecida após o evento a prever.
14. Etapas que estimam algo dos dados devem ser reaprendidas só com o treino a cada divisão. Aplica-se a imputação, padronização, seleção de atributos, PCA e codificações que aprendem categorias.
15. Cada etapa tem opções; as combinações formam o espaço. Com GridSearchCV, nomes etapa__parametro permitem variar parâmetros e até trocar etapas inteiras, como o tipo de escala.
16. $2 \times 3 \times 2 \times 4 = 48$ pipelines, antes de contar hiperparâmetros. O número cresce de forma multiplicativa, o que torna a busca manual inviável e justifica estratégias automáticas.
17. Porque o pré-processamento treinado vai junto, garantindo transformações idênticas. O risco é que arquivos joblib/pickle podem executar código; nunca carregar arquivos de origem desconhecida.
18. Fixar sementes aleatórias, registrar as versões das bibliotecas, guardar o histórico das tentativas da busca e salvar o pipeline treinado.
19. Uma coluna de idade com valor 999 pode ser um código de "não informado". Se tratada como número, distorce médias e escalas; só uma pessoa conhece o significado e decide o tratamento.
20. O AutoML testa sistematicamente opções (imputação, escala etc.) e escolhe pela validação, economizando tempo. O cuidado é manter tudo dentro do pipeline para evitar vazamento e conferir o sentido dos dados.

Capítulo 4 — Busca em grade e aleatória

Múltipla escolha

Questão	Resposta	Justificativa
1	a)	HPO significa otimização de hiperparâmetros, núcleo do AutoML clássico.
2	d)	C é definido antes do treino. Os demais itens são aprendidos pelo modelo.
3	b)	Por exemplo, gamma só importa quando o kernel é RBF.
4	a)	Em sorteio uniforme de 0,01 a 1000, os valores pequenos quase nunca aparecem.
5	d)	Menos de 1 em 1000 sorteios uniformes cai abaixo de 1, ao passo que o sorteio logarítmico dá cerca de 40%.
6	b)	A grade percorre o produto cartesiano das listas de valores.
7	d)	5 × 5 × 5 × 5 = 625.
8	d)	O custo é k elevado a d, a chamada explosão combinatória.
9	d)	Com poucos botões importantes, a aleatória explora mais valores relevantes com o mesmo orçamento.
10	a)	Cada valor do botão irrelevante repete os mesmos pares dos botões importantes.
11	b)	RandomizedSearchCV sorteia n_iter configurações de distribuições.
12	a)	n_iter é o orçamento de tentativas.
13	b)	É a escolha adequada para hiperparâmetros como C e gamma.
14	a)	Cada ponto é uma combinação e sua cor ou altura é a nota de validação.
15	b)	Nesses casos o modelo ou memoriza pontos isolados ou fica rígido de menos.
16	b)	A grade repete poucos pares de C e gamma; a aleatória testa 27 valores distintos de cada um.
17	b)	A curva da melhor nota até agora sobe rápido no início e depois se estabiliza.
18	a)	n_jobs=-1 usa todos os núcleos disponíveis.
19	a)	A seleção entre muitas configurações infla a nota do melhor.
20	c)	O laço interno realiza a busca; o externo mede o resultado em dados que a busca nunca viu.
21	c)	A média das notas externas é a estimativa honesta.
22	d)	O objeto de busca é tratado como um estimador e avaliado externamente.
23	b)	A nota comum foi cerca de 3 pontos percentuais acima da aninhada.
24	c)	Menos dados e mais candidatos aumentam a chance de sorte na seleção.
25	b)	O custo se multiplica pelo número de partes do laço externo.
26	c)	A ideia é a mesma: a avaliação final não participa da busca.
27	a)	Mede o ganho potencial de ajustar um hiperparâmetro em comparação com um valor de referência.
28	b)	Saber de onde se parte dá sentido ao ganho obtido pela busca.
29	d)	Qualquer método sofisticado precisa superar a busca aleatória para justificar sua complexidade.
30	b)	Distribuições permitem explorar valores diferentes de cada hiperparâmetro em cada tentativa.

Discursivas (respostas-modelo)

1. São botões definidos antes do treino. Eles interagem: o melhor valor de um pode depender dos outros, de modo que ajustá-los isoladamente pode não achar a melhor combinação.
2. Contínuo (taxa de aprendizado), inteiro (número de árvores), categórico (tipo de kernel), em escala logarítmica (C do SVM) e condicional (gamma, que só existe com kernel RBF).

3. Quase todo o intervalo uniforme está acima de 1, então valores pequenos quase não são testados. Na escala logarítmica, cada ordem de grandeza tem igual chance, cobrindo todas as faixas.
4. Testa todas as combinações. São 4 × 4 × 4 = 64 combinações e 64 × 5 = 320 treinos.
5. O número de combinações cresce de forma multiplicativa (k elevado a d). Com 5 valores em 10 hiperparâmetros, são quase 10 milhões de avaliações.
6. Em muitos problemas só poucos hiperparâmetros importam, e quais importam varia. A busca aleatória testa um valor novo de cada botão em cada tentativa; a grade repete valores, desperdiçando orçamento.
7. Com poucos hiperparâmetros e faixa bem conhecida, a grade é simples e produz tabelas organizadas para exploração. Em espaços maiores, a aleatória é mais eficiente com o mesmo orçamento.
8. Com um botão quase irrelevante (tol), a grade repetiu os mesmos pares de C e gamma e obteve 0,983; a aleatória, com 27 valores distintos, superou a grade nas 8 repetições (média 0,989).
9. É o mapa da nota de validação em função dos hiperparâmetros. Mostra regiões boas e ruins, a extensão do platô de bons resultados e se há botões que quase não alteram a nota.
10. As primeiras tentativas dão a maior parte do ganho; depois, cada nova tentativa adiciona pouco. Convém fixar um orçamento de tempo ou tentativas e parar quando os ganhos forem mínimos.
11. O vencedor foi escolhido por ter a melhor nota na validação, que inclui um componente de sorte. Entre muitas configurações, a melhor tende a ter nota inflada, como entre muitos competidores em um sorteio.
12. O laço externo divide treino e teste; para cada divisão, o laço interno faz a busca só no treino externo; o melhor modelo é avaliado no teste externo; a nota final é a média dos testes externos.
13. Ambas evitam que a avaliação final participe da busca. A aninhada usa melhor poucos dados, mas custa muito mais; o teste final é mais barato e adequado quando há dados suficientes.
14. A diferença de cerca de 3 pontos percentuais é o otimismo da busca: a nota comum reflete a escolha do vencedor na validação, ao passo que a aninhada estima o desempenho do processo inteiro em dados não vistos.
15. Cresce com o menor número de dados, o maior número de configurações testadas e a maior variância da validação.
16. Calcula-se a correlação (por exemplo, de Spearman) entre cada hiperparâmetro e a nota. Cuidado: é uma análise simples; efeitos podem se combinar e o resultado depende do conjunto de dados.
17. Reduz a dimensão do espaço de busca, concentrando as tentativas nos botões que realmente influenciam o desempenho, o que dá melhores resultados com o mesmo orçamento.
18. Começar pelo default e linha de base; usar escala logarítmica; preferir aleatória à grade em espaços maiores; definir orçamento; medir com honestidade; registrar tudo; repetir com sementes. Justificativa: medir com honestidade evita otimismo.
19. Para que imputação, escala e seleção sejam reaprendidas em cada divisão da validação, evitando vazamento e permitindo buscar também escolhas do pré-processamento.
20. Porque é simples, paralelizável e surpreendentemente forte. Um método mais complexo só se justifica se superá-la com o mesmo orçamento.

Capítulo 5 — Otimização bayesiana

Múltipla escolha

Questão	Resposta	Justificativa
1	d)	Só vemos a nota para cada entrada (caixa-preta), e cada avaliação exige treinar e validar um modelo (cara).
2	a)	A BO aprende com o histórico para decidir onde avaliar a seguir.
3	b)	Ele prevê a nota de configurações não testadas, sem treinar o modelo de verdade.
4	d)	A incerteza permite que a aquisição valorize regiões pouco exploradas.
5	d)	Os demais passos são baratos em comparação com treinar o modelo real.
6	b)	Aproveitar refina o que se conhece; explorar busca descobertas maiores, com risco de perder uma avaliação.
7	d)	O próximo ponto avaliado é o de maior valor de aquisição.
8	d)	PI, EI e UCB são funções de aquisição clássicas.
9	b)	O primeiro termo premia média alta; o segundo, incerteza alta.
10	c)	Quanto maior o k, mais o algoritmo valoriza regiões incertas.
11	b)	Isso permite equilibrar explorar e aproveitar.
12	c)	O kernel expressa a suposição de suavidade da função.
13	c)	Com poucos pontos, a incerteza é grande em quase todo o espaço.
14	d)	A BO tem uma fase de aquecimento; as primeiras avaliações são pouco aproveitadas.
15	a)	A BO ficou à frente (cerca de 48% contra 32%).
16	c)	O TPE inverte a pergunta: onde as configurações boas costumam estar?
17	b)	A escolha do algoritmo condiciona quais hiperparâmetros existem.
18	a)	Florestas lidam bem com categorias, condicionais e muitas dimensões.
19	b)	Thornton e colaboradores (2013) resolveram o CASH com otimização bayesiana.
20	c)	Isso permite espaços dinâmicos e condicionais com if e laços.
21	c)	O argumento log=True pede amostragem logarítmica.
22	d)	O estudo reúne várias tentativas (trials).
23	b)	O Optuna maximiza ou minimiza o valor devolvido, conforme direction.
24	d)	O espaço condicional é descrito por código comum.
25	c)	A região boa do gamma é estreita; com pouco orçamento, o algoritmo foi descartado cedo.
26	c)	É o risco de aproveitar demais e explorar de menos.
27	c)	O problema tem 2 botões e uma região boa larga, o que favorece a busca aleatória.
28	c)	É quando aprender com o histórico compensa o custo extra.
29	c)	O custo cresce com o histórico e o desempenho cai em espaços de alta dimensão.
30	c)	A busca aleatória é a linha de base obrigatória.

Discursivas (respostas-modelo)

1. A aleatória ignora os resultados anteriores, como um garimpeiro que cava ao acaso mesmo após achar ouro. A BO usa o histórico para decidir onde olhar a seguir, concentrando as avaliações caras em regiões promissoras.
2. É uma função cuja fórmula não conhecemos e cuja avaliação é custosa (treinar e validar um modelo). Como poucas avaliações são possíveis, vale usar um método que escolha cada uma com cuidado.

3. Treinar o modelo substituto com o histórico; calcular a função de aquisição; escolher o ponto de maior aquisição; avaliar de verdade esse ponto; acrescentar o resultado ao histórico e repetir.
4. Escolher entre voltar ao restaurante favorito (aproveitar) ou testar um novo (explorar). A aquisição combina a previsão (aproveitar) e a incerteza (explorar) em uma nota de interesse para cada ponto.
5. É um modelo que prevê valores e informa a incerteza da previsão: pequena perto dos pontos medidos, grande longe deles. Isso permite a aquisição equilibrar explorar e aproveitar.
6. PI mede a chance de melhorar (ignora o tamanho do ganho); EI mede a melhoria esperada (pondera chance e tamanho); UCB soma média e k vezes a incerteza, com k controlando o apetite por explorar.
7. O primeiro termo premia pontos de média prevista alta em relação ao melhor atual; o segundo premia pontos de incerteza alta. A soma é a melhoria que se espera obter ao avaliar o ponto.
8. Comparou-se BO (GP + EI) e aleatória em 40 repetições. Com 5 avaliações a aleatória estava à frente, com 10 empataram e com 15 a BO achou a agulha com mais frequência. A BO precisa de aquecimento.
9. Separa o histórico em tentativas boas e ruins, estima as densidades l(x) e g(x) e escolhe onde l/g é maior. Funciona em espaços em árvore porque modela cada hiperparâmetro condicionalmente.
10. GP: poucos dados e poucas dimensões contínuas, com incerteza suave. TPE: espaços condicionais e categóricos. Floresta: categorias, condicionais e muitas dimensões.
11. Study é o experimento inteiro; Trial é uma tentativa com um conjunto de valores; Sampler escolhe os valores (TPE, aleatório etc.); Pruner interrompe cedo tentativas ruins.
12. Sugerir o algoritmo como variável categórica; dentro de um if para cada algoritmo, sugerir seus hiperparâmetros; construir o modelo; devolver a nota de validação; criar o estudo e chamar optimize.
13. As primeiras tentativas do SVM caíram em regiões ruins; com pouco orçamento, o TPE passou a evitá-lo. Remédios: mais orçamento, espaços mais bem desenhados, warm start com configurações conhecidas e avaliação multifidelidade.
14. O problema tem poucos botões e uma região boa larga, então o sorteio já acerta com facilidade, e o TPE gasta tentativas iniciais também aleatórias. Ensina que a BO não é garantia de melhora.
15. Dificuldade com muitas dimensões; sensibilidade a ruído e custo do modelo substituto; natureza sequencial (difícil de paralelizar); risco de overfitting da busca.
16. Cada escolha depende dos resultados anteriores. A busca aleatória paraleliza sem esforço, ao passo que a BO exige adaptações para avaliar vários pontos ao mesmo tempo.
17. Pede amostragem em escala logarítmica, dando a mesma chance a cada ordem de grandeza. Importa para hiperparâmetros como C, gamma e taxas de aprendizado.
18. Para que a diferença não seja fruto de sorte de uma execução nem de orçamentos desiguais; a busca aleatória é a linha de base que o método sofisticado deve superar.
19. Sem pontos iniciais, o modelo substituto nada sabe. As primeiras avaliações têm menor aproveitamento, então em orçamentos muito curtos a BO pode ficar atrás da aleatória.
20. A BO escolhe o ponto de melhor nota de validação entre muitos testados; como na busca aleatória, essa nota inclui sorte, e só um teste final intocado mede o desempenho real.

Capítulo 6 — Métodos multifidelidade

Múltipla escolha

Questão	Resposta	Justificativa
1	d)	A multifidelidade economiza treino ao descartar cedo candidatos fracos.
2	a)	O treino curto é uma avaliação barata e aproximada dos candidatos.
3	b)	Épocas, dados e atributos são recursos que podem ser variados; o nome do arquivo é irrelevante.
4	a)	Quem vai bem no começo tende a ir bem no fim.
5	d)	Candidatos de largada lenta podem ser eliminados cedo.
6	c)	Menos candidatos recebem mais recursos a cada rodada.
7	b)	São 9 candidatos × 3 épocas = 27 épocas.
8	d)	Quatro rodadas de 27 épocas cada: 4 × 27 = 108.
9	b)	27 × 27 = 729 épocas, mais de seis vezes o custo do Halving.
10	d)	HalvingGridSearchCV (e HalvingRandomSearchCV) são marcados como experimentais.
11	b)	As classes ainda são experimentais e exigem habilitação explícita.
12	d)	As primeiras rodadas usam poucos exemplos; as seguintes, mais.
13	a)	O custo caiu de cerca de 259 mil para cerca de 43 mil exemplos processados.
14	a)	O Hyperband varia o equilíbrio entre quantidade de candidatos e treino por candidato.
15	c)	Em vez de escolher, ele executa os dois extremos e intermediários.
16	b)	R é o orçamento máximo por configuração.
17	a)	$s_max = \log_3(81) = 4$, portanto s = 4, 3, 2, 1, 0: cinco torneios.
18	c)	É a aposta mais conservadora: não confia em cortes precoces.
19	d)	0,963 contra 0,955, com menor variação entre as sementes.
20	c)	0,969 (±0,002) contra 0,965 (±0,007).
21	d)	O Hyperband organiza os torneios e o TPE sugere os candidatos (Falkner, Klein e Hutter, 2018).
22	c)	Um modelo TPE treinado com resultados por fidelidade sugere as configurações.
23	a)	É a base da poda: o podador usa as notas parciais.
24	d)	Se retornar True, a função-objetivo levanta optuna.TrialPruned().
25	c)	É uma regra simples e robusta de poda.
26	a)	Foram podadas 20 de 40 tentativas, com 1.297 épocas contra 2.400 sem poda.
27	b)	Se todos parecem iguais em baixa fidelidade, o corte não separa os bons dos ruins.
28	d)	Compare apenas candidatos avaliados com o mesmo orçamento.
29	a)	Cortes mais suaves e orçamento inicial maior reduzem o risco.
30	a)	Menos candidatos recebem mais recursos a cada rodada, como no Successive Halving.

Discursivas (respostas-modelo)

1. Em uma peneira de futebol, treinar 270 jovens por um ano seria caro; um treino curto já revela os mais fracos. Do mesmo modo, muitas configurações ruins mostram sua fraqueza cedo, e treiná-las até o fim desperdiça recursos.

2. Fidelidade é o quanto uma avaliação se parece com a completa (barata e aproximada versus cara e precisa). Exemplos: número de épocas, fração dos dados, subconjunto de atributos, resolução das imagens, número de partes da validação cruzada.
3. Que o ranking em baixa fidelidade se parece com o de alta fidelidade. Quando falha, candidatos de largada lenta (que seriam os melhores no fim) são eliminados cedo.
4. Começa com n candidatos e orçamento pequeno r; treina todos e avalia; mantém a fração 1/η melhor; multiplica o orçamento por η; repete até restar um candidato.
5. Rodadas: 81 candidatos × 1, 27 × 3, 9 × 9, 3 × 27 e 1 × 81, cada uma com 81 épocas; total 405. Treinar todos até 81 custaria 81 × 81 = 6.561 épocas.
6. Com o SGD de passo constante, a acurácia oscila de uma época para outra. No exemplo, a melhor acurácia da última rodada (0,956) foi menor que a de rodadas anteriores (0,969).
7. Cortar para as 13 melhores manteria duas campeãs (postos 5 e 10), mas eliminaria a terceira (posto 20). Mostra que o corte precoce pode descartar boas configurações cujas curvas melhoram devagar.
8. Avalia todas as combinações com poucos exemplos e vai mantendo as melhores com mais exemplos. A nota final vem da última rodada, com mais dados mas poucos candidatos; difere da nota da grade completa, que usa todos os dados em todas.
9. O dilema é testar muitas configurações com pouco treino ou poucas com muito treino. O Hyperband executa vários torneios de Successive Halving com apostas diferentes e fica com o melhor resultado.
10. R é o máximo de recurso por configuração; η é o fator de corte. s = 4 testa muitas configurações com pouco treino (aposta ampla); s = 0 testa poucas com treino completo (aposta conservadora).
11. Com o mesmo orçamento, o Halving (0,963) superou a aleatória (0,955) com menor variação; o Hyperband (0,969, ±0,002) superou a aleatória (0,965, ±0,007) com mais estabilidade. O ganho em média foi modesto.
12. O espaço continha muitas configurações boas, então a aleatória já encontra boas. Ajuda mais com orçamento pequeno, treinos muito caros (redes profundas) e quando se quer menor dependência de sorte.
13. Usa o Hyperband para organizar os torneios e cortes, e um modelo TPE para sugerir candidatos em vez de sorteá-los. Combina bom desempenho inicial (Hyperband) com convergência rápida (otimização bayesiana).
14. Informar a nota parcial com trial.report(nota, epoca) e perguntar com trial.should_prune(); se verdadeiro, levantar optuna.TrialPruned() para interromper a tentativa.
15. Metade das tentativas foi interrompida, reduzindo as épocas de 2.400 para 1.297, com melhor acurácia quase igual (0,969 contra 0,972). A poda economizou cerca de 46% do treino sem perda relevante.
16. O ranking pode mudar (orçamento inicial razoável, η menor); fidelidade baixa demais não discrimina (testar a fidelidade mínima); notas de fidelidades diferentes não são comparáveis (comparar apenas no mesmo orçamento).
17. Para retomar o treino de onde parou quando um candidato avança de rodada, sem recomeçar do zero e sem gastar épocas repetidas.
18. Com η menor, os cortes são mais suaves (mais rodadas, menos risco de eliminar campeãs lentas, mais custo). Com η maior, os cortes são agressivos (poucas rodadas, mais economia, mais risco).
19. Porque a seleção usou notas de baixa fidelidade e ruidosas; o desempenho real do vencedor deve ser medido em dados nunca usados, com o modelo treinado completamente.
20. Porque ele roda vários torneios com graus diferentes de agressividade, de muitos candidatos com pouco treino até poucos candidatos com treino completo. Se o corte precoce for pouco confiável, os torneios conservadores compensam.

Capítulo 7 — Métodos evolutivos e programação genética

Múltipla escolha

Questão	Resposta	Justificativa
1	c)	Indivíduos diferentes, seleção dos melhores e transmissão de características aos descendentes.
2	d)	Cada indivíduo é um pipeline (escala, redução, modelo e hiperparâmetros).
3	c)	A aptidão mede quão bom é o pipeline; maior nota, maior aptidão.
4	c)	O genoma é a codificação do pipeline como conjunto de genes.
5	a)	Concentra a busca em regiões promissoras.
6	a)	Boas ideias dos pais podem se juntar no filho.
7	b)	Sem mutação, a população perde variedade e pode ficar presa em um tipo de solução.
8	a)	Evita perder o melhor resultado já encontrado.
9	c)	Os melhores têm mais chance, mas os fracos ainda podem ser escolhidos.
10	c)	Pipelines iguais reutilizam a nota já calculada, economizando tempo.
11	c)	Em um problema muito fácil, todos os métodos chegariam ao mesmo resultado.
12	b)	A população inteira melhora com a seleção, mas a mutação gera variações pouco aptas.
13	a)	Ele venceu em 4 sementes e perdeu em 1, com diferença média modesta.
14	d)	Com poucas repetições, não se pode afirmar superioridade geral.
15	b)	Árvores permitem estruturas de tamanhos e formas diferentes.
16	b)	Os nós são etapas do pipeline, e os filhos de um nó são suas entradas.
17	b)	Pedaços inteiros do tratamento dos dados mudam de dono.
18	c)	O TPOT (Tree-based Pipeline Optimization Tool) foi apresentado por Olson e colaboradores em 2016.
19	b)	A exportação como código é um ponto a favor da transparência.
20	d)	Acrescentar etapas costuma aumentar levemente a nota, o que faz o tamanho crescer.
21	a)	É a definição de dominância de Pareto.
22	c)	Ela mostra as opções de equilíbrio entre acurácia e simplicidade.
23	d)	Deb e colaboradores (2002) propuseram o NSGA-II.
24	c)	O PCA reduz de 264 colunas para 10 a 40 componentes; outros pipelines usam as 264.
25	b)	O arquivo exportado pode ser lido, editado e reutilizado.
26	c)	É importante ser transparente sobre o que foi ou não testado.
27	b)	Cada geração avalia muitos pipelines e as execuções variam com a semente.
28	d)	Os indivíduos são independentes dentro da mesma geração.
29	d)	A família bayesiana (GP, TPE, SMAC) usa modelo substituto e aquisição.
30	b)	A mutação introduz variações novas; sem ela, os indivíduos tendem a se parecer cada vez mais.

Discursivas (respostas-modelo)

1. O criador seleciona as melhores plantas a cada safra, usa suas sementes e preserva variações úteis. O algoritmo genético mantém uma população de soluções, seleciona as melhores, combina e muta, geração após geração.
2. Indivíduo: um pipeline. Genoma: a lista de escolhas que o descreve. Aptidão: a nota de validação cruzada. Geração: uma rodada de avaliação, seleção, cruzamento e mutação.

3. Cria-se uma população inicial; avalia-se a aptidão; selecionam-se os melhores; cruzam-se pares; aplicam-se mutações; forma-se a nova geração; repete-se até acabar o orçamento.
4. O cruzamento combina boas ideias de dois pais (aproveitar). A mutação introduz variação e mantém a diversidade (explorar). Juntos, equilibram aproveitamento e exploração.
5. É copiar os melhores indivíduos diretamente para a próxima geração. Garante que a melhor solução já encontrada não se perca por causa de mutações ou sorteios desfavoráveis.
6. Sorteiam-se alguns indivíduos e escolhe-se o melhor deles como pai. Os bons têm mais chance, mas indivíduos fracos podem ser escolhidos quando competem só com outros fracos, preservando variedade.
7. Com o mesmo número de pipelines avaliados e 5 sementes, o GA terminou à frente em média (cerca de 0,92 contra 0,91) e venceu em 4 de 5 sementes. A diferença é modesta, e com 5 repetições não prova superioridade geral.
8. A mutação gera filhos que podem ser ruins; a nota média reflete toda a população, e não só os melhores. O elitismo preserva o melhor, mesmo que a média oscile.
9. Árvores representam estruturas de tamanho e forma variáveis, como pipelines com números diferentes de etapas. Isso permite que o cruzamento troque subárvores inteiras e que pipelines variados convivam na população.
10. Mantém uma população de pipelines em árvore, avalia por validação cruzada, seleciona (com NSGA-II, em uma variante multiobjetivo), aplica cruzamento e mutação por gerações e exporta o melhor pipeline como código Python.
11. Acrescentar etapas costuma aumentar levemente a nota, então a seleção favorece árvores maiores. Mitiga-se com um segundo objetivo de simplicidade (otimização multiobjetivo) ou com limites de tamanho.
12. Uma solução domina outra se é melhor ou igual em todos os objetivos e estritamente melhor em pelo menos um. A fronteira é o conjunto das soluções que nenhuma outra domina: as melhores concessões entre os objetivos.
13. Escolher um celular: quanto maior a qualidade da câmera, maior o preço. Os aparelhos que não são superados em câmera e preço ao mesmo tempo formam a fronteira; os demais são piores que algum outro.
14. Tomou-se todos os pipelines avaliados, com acurácia e número de atributos, e mantiveram-se os não dominados. Ela mostra opções entre o mais preciso e o mais enxuto, permitindo escolher a concessão adequada.
15. Vantagens: flexibilidade com estruturas variáveis e objetivos múltiplos; paralelismo natural. Limitações: custo computacional elevado; aleatoriedade e muitos parâmetros próprios (população, taxas).
16. Porque precisam ser escolhidos e afetam o desempenho da busca, como os hiperparâmetros de um modelo. Isso significa que o próprio método evolutivo também exige ajustes e teste de sensibilidade.
17. Aleatória: sorteio, não aprende. Bayesiana: modelo substituto, sequencial e sensível a dimensões altas. Multifidelidade: corte precoce, o ranking pode mudar. Evolutiva: população e operadores, custo e aleatoriedade.
18. Para entender o que ele faz, verificar se é razoável para o problema, detectar vazamentos ou etapas estranhas e poder explicá-lo e mantê-lo.
19. Ao avaliar muitos pipelines, o melhor tem nota otimista por sorte. Evita-se com teste final intocado, relato de variação entre execuções e comparação com a busca aleatória.
20. Definir o orçamento (população e gerações); fixar sementes e repetir; usar elitismo e cache de avaliações; controlar a complexidade; comparar com a busca aleatória; avaliar em teste intocado e ler o pipeline final.

Capítulo 8 — Engenharia e seleção automática de atributos

Múltipla escolha

Questão	Resposta	Justificativa
1	a)	Envolve produzir atributos novos a partir dos existentes e escolher os mais úteis.
2	a)	O preparo dos ingredientes (atributos) determina o que o modelo consegue aproveitar.
3	d)	A fronteira entre círculos é curva; é preciso um atributo como $r^2 = x1^2 + x2^2$ para torná-la linear.
4	a)	No espaço (x1, r^2), uma reta horizontal separa as duas classes.
5	d)	O logaritmo comprime valores muito grandes e reduz a assimetria.
6	b)	A razão combina dois atributos em uma medida mais informativa.
7	b)	O R^2 caiu de cerca de 0,49 para cerca de 0,46: mais atributos não garantem melhora.
8	c)	O ganho depende do problema, e a avaliação honesta é indispensável.
9	b)	A seleção busca reduzir e escolher bem as colunas, e não aumentá-las.
10	b)	Por exemplo, o teste F e a informação mútua.
11	a)	Como avaliam cada atributo sozinho, não percebem combinações ou redundâncias.
12	d)	Ele treina o modelo, descarta os atributos menos importantes e repete.
13	a)	Treinam o modelo muitas vezes sobre diferentes subconjuntos.
14	a)	A seleção ocorre durante o treino do próprio modelo.
15	b)	Sem seleção, a acurácia foi menor do que a obtida com RFE e L1.
16	b)	Ambos escolheram quase só atributos reais e chegaram a cerca de 0,98.
17	d)	O limiar de corte é um hiperparâmetro; a mediana é generosa em um problema com muito ruído.
18	c)	A seleção olhando todos os rótulos antes da divisão contamina a validação.
19	d)	Ela aplica funções de agregação ao longo dos caminhos entre tabelas.
20	a)	Profundidade 1 agrega diretamente; profundidade 2 agrega os resultados das agregações.
21	d)	MEAN(compras.valor) por cliente é uma agregação típica.
22	a)	A informação relevante estava nas compras, não na idade ou na cidade.
23	c)	A relação entre agregados e cancelamento foi criada por nós.
24	d)	Agregados devem respeitar a linha do tempo da previsão.
25	b)	Eles aprendem uma estratégia de busca no grafo, com atributos interpretáveis.
26	a)	$6 \times 100 \times 99 / 2 = 29.700$.
27	d)	Com muitos candidatos, alguns parecem bons por acaso nos dados de treino.
28	d)	O resultado foi desfavorável à criação automática indiscriminada.
29	a)	Criar atributos precisa de um motivo e de avaliação honesta.
30	d)	PolynomialFeatures(2) cria x1, x2, $x1^2$, x1·x2 e $x2^2$, por exemplo.

Discursivas (respostas-modelo)

1. O chef que prepara bem os ingredientes (corta, tempera, combina) obtém um bom prato mesmo com forno comum; ingredientes mal preparados não são salvos pelo melhor forno. Do mesmo modo, bons atributos tornam o padrão visível a qualquer modelo.

2. Nos atributos originais, nenhuma reta separa os círculos. Com $r^2 = x1^2 + x2^2$, uma reta basta. Ensina que uma transformação adequada pode tornar linear um problema não linear.
3. Quadrado (relações curvas); logaritmo (variáveis assimétricas como renda); razão (gasto por visita); faixas (efeitos em degraus, como faixas de idade); data e hora (sazonalidade); agregações (dados em várias tabelas).
4. Passou-se de 10 para 65 atributos, aumentando o risco de sobreajuste em um conjunto pequeno e ruidoso. Ensina que transformar não garante melhora e deve ser avaliado.
5. Filtros: rápidos, avaliam atributos isoladamente. Wrappers: treinam o modelo em subconjuntos, precisos e caros. Embutidos: a seleção ocorre no treino (L1, importância de árvores), com bom equilíbrio.
6. Treina o modelo com todos os atributos, descarta os de menor importância, treina de novo e repete até restar o número desejado de atributos.
7. Para ser reaprendida só com o treino em cada divisão. Fora do pipeline, ela vê os rótulos da validação e infla a nota, como no experimento com 2000 atributos aleatórios.
8. Sem seleção, o ruído prejudicou o modelo; filtros ajudaram um pouco; RFE e L1 obtiveram as maiores acurácias (cerca de 0,98) escolhendo quase só atributos reais; a floresta com limiar na mediana manteve muito ruído.
9. Aplica funções de agregação ao longo dos relacionamentos entre tabelas. Para clientes e compras: média, máximo, desvio padrão e contagem dos valores de compra de cada cliente, e razões entre eles.
10. Sem as compras, o AUC ficou em cerca de 0,5; com agregados, subiu muito. Os dados foram simulados com um padrão conhecido; mostra o potencial da técnica, e não desempenho em dados reais.
11. Ocorre quando o agregado inclui informação posterior ao momento da previsão (por exemplo, média de compras futuras). Evita-se calculando os agregados apenas com dados anteriores à data de referência.
12. O número de combinações cresce depressa com o número de atributos e a ordem das operações (quase 30 mil pares×operações para 100 atributos). Com tantos candidatos, alguns parecem informativos só por sorte, causando sobreajuste da busca.
13. Os atributos gerados pioraram a acurácia. Razões: os originais já eram bons; os candidatos eram redundantes e o SelectKBest univariado os escolheu sem considerar a redundância, descartando originais úteis; poucos exemplos favorecem sinais falsos.
14. Promissora: dados em várias tabelas, em que agregações trazem informação oculta (DFS). Questionável: dados já bem representados e pequenos, com geração indiscriminada de combinações.
15. Vazamento, vazamento temporal, sobreajuste da busca, redundância, perda de interpretabilidade, custo e instabilidade. Medida contra vazamento: manter geração e seleção dentro do pipeline.
16. O modelo espera os mesmos atributos ao fazer previsões. Atributos complexos exigem código e dados disponíveis em produção, aumentando custo e risco de inconsistência entre treino e produção.
17. Colocando SelectKBest no pipeline e incluindo selectkbest__k na grade ou no espaço de busca, avaliado por validação cruzada junto aos demais hiperparâmetros.
18. O espaço de transformações é representado como um grafo e uma estratégia de busca é aprendida a partir de execuções anteriores. Os atributos resultantes continuam interpretáveis por usarem funções matemáticas padrão.
19. Partir do conhecimento do problema; medir o ganho contra uma linha de base; manter tudo dentro do pipeline; limitar a explosão gerando a partir de poucos atributos; documentar cada atributo; checar vazamento temporal.
20. Porque influenciam o desempenho e dependem do problema. Escolhê-los com validação cruzada dentro do pipeline evita ajustes arbitrários e vazamento.

Capítulo 9 — Meta-aprendizado, warm start e ensembles

Múltipla escolha

Questão	Resposta	Justificativa
1	b)	É "aprender a aprender": aproveitar resultados de problemas anteriores.
2	b)	A experiência acumulada orienta por onde começar.
3	b)	Rice (1976) descreveu o problema com quatro espaços.
4	a)	O quadro tem problemas, características, mapeamento de seleção e algoritmos, mais uma medida de desempenho.
5	c)	Por exemplo, número de exemplos, entropia das classes e assimetria média.
6	b)	Uma árvore rasa, por exemplo, indica o tipo de estrutura que os dados têm.
7	a)	Entropia máxima significa classes balanceadas; baixa, classes muito desiguais.
8	b)	Rivolli e colaboradores (2022) tratam dessa falta de padronização.
9	a)	Foram usados conjuntos de make_classification com tamanhos, ruídos e separabilidades variados.
10	a)	Evita avaliar o meta-modelo em conjuntos que ele já viu.
11	b)	No experimento, era o SVM, que foi o melhor em mais da metade dos conjuntos.
12	b)	O ganho foi humilde: 0,593 de acerto nos dois, e perda 0,0100 contra 0,0110.
13	a)	Sistemas reais usam centenas de conjuntos e meta-atributos mais ricos.
14	a)	O warm start reaproveita configurações que funcionaram bem em problemas parecidos.
15	c)	Feurer, Springenberg e Hutter (2015) propuseram inicializar a otimização bayesiana dessa forma.
16	b)	A escolha gulosa busca configurações complementares.
17	d)	O melhor gamma depende do número de atributos d.
18	a)	O portfólio chegou muito perto do melhor valor da grade em Iris, Wine, Cancer e Digits.
19	a)	Com muitas tentativas, a busca comum acaba alcançando os mesmos valores.
20	d)	Ele coloca configurações na fila das primeiras tentativas.
21	b)	Os erros de uns compensam os de outros.
22	a)	A média de muitos palpites independentes costuma ficar perto do valor real.
23	d)	Wolpert (1992) propôs treinar um modelo de nível superior sobre as previsões dos modelos de base.
24	b)	Previsões feitas no próprio treino são otimistas e enganam o meta-modelo.
25	b)	O stacking foi um pouco além da votação, no problema simulado e difícil.
26	c)	Quando um modelo já é quase perfeito, há pouco espaço para a combinação melhorar.
27	c)	A seleção gulosa para frente pode escolher o mesmo modelo várias vezes, o que funciona como peso.
28	b)	As contagens finais viram os pesos.
29	c)	A simples média capturou boa parte do ganho; a seleção acrescentou um pouco.
30	c)	A busca de arquiteturas é tema do Capítulo 11 e não faz parte do Auto-sklearn original.

Discursivas (respostas-modelo)

1. Um médico experiente reconhece um caso parecido com outros e vai direto aos exames mais prováveis, enquanto o iniciante pede todos. O meta-aprendizado usa a experiência de conjuntos anteriores para decidir por onde começar em um conjunto novo.

2. Espaço dos problemas (conjuntos de dados), espaço das características (meta-atributos), mapeamento de seleção (meta-modelo que escolhe o algoritmo) e espaço dos algoritmos. A medida de desempenho avalia o quanto o algoritmo escolhido resolve bem o problema.
3. Simples/estatísticos (número de exemplos); teoria da informação (entropia das classes); complexidade do problema (sobreposição entre classes); landmarkers (nota de uma árvore rasa).
4. É a nota de um modelo simples e rápido no problema. Funciona como sonda do tipo de estrutura dos dados (por exemplo, se classes são separáveis por regras simples), e é barato de calcular.
5. Simularam-se 150 conjuntos, calcularam-se meta-atributos e rotularam-se com o melhor entre quatro algoritmos. Treinou-se uma floresta aleatória como meta-modelo, avaliada deixando um conjunto de fora, e comparada com a regra de recomendar sempre o algoritmo mais frequente.
6. Acertou o melhor algoritmo na mesma proporção que a regra boba (cerca de 59%), com perda média só um pouco menor. Razões: poucos conjuntos, simulados; poucos meta-atributos; algoritmos de desempenhos parecidos, com o SVM competitivo quase sempre.
7. Porque as diferenças entre algoritmos padrão costumam ser pequenas e dependem de detalhes sutis dos dados; é preciso muitos conjuntos e meta-atributos ricos para aprender regras confiáveis.
8. Começa a busca com configurações que funcionaram bem em problemas parecidos. É robusto porque, mesmo que a recomendação não seja ótima, ela só define o ponto de partida; a busca continua e pode corrigir.
9. Avaliaram-se 25 configurações de SVM em 40 conjuntos simulados de origem; escolheram-se 3 de forma gulosa para maximizar o melhor resultado médio do portfólio; testaram-se essas 3 em conjuntos reais, comparando com 3 e 6 configurações sorteadas.
10. Com 3 configurações do portfólio, chegou-se perto do melhor valor possível em todos os conjuntos reais; o sorteio de 3 ficou atrás. O benefício é maior no começo da busca, quando cada tentativa vale mais.
11. O melhor gamma depende do número de atributos d. Usando gamma_rel/d, as configurações se tornam comparáveis e transferíveis entre conjuntos de dimensões diferentes.
12. Modelos diferentes erram em lugares diferentes; a combinação compensa erros individuais (como a média dos chutes no concurso do boi gordo), reduzindo a variância e melhorando a previsão, desde que haja diversidade.
13. Bagging/florestas: muitos modelos do mesmo tipo em amostras diferentes. Boosting: modelos em sequência corrigindo erros. Votação: média ou maioria de modelos de tipos diferentes. Stacking: um meta-modelo aprende a combinar as previsões.
14. Começa com ensemble vazio; a cada passo acrescenta o modelo da biblioteca que mais melhora o ensemble na validação, podendo repetir modelos; após um número de passos, as contagens viram os pesos.
15. O ensemble superou o melhor modelo escolhido pela validação e quase todos os modelos individuais. A média simples dos 40 modelos já chegou perto, mostrando que a diversidade responde por boa parte do ganho; a seleção acrescentou um pouco.
16. Os pesos são escolhidos na validação; ao testar muitas combinações, o ensemble pode se ajustar ao acaso da validação. Evita-se com teste final intocado e relatando a nota nele.
17. Meta-aprendizado inicia a busca com configurações promissoras; a otimização bayesiana avalia candidatos em um espaço amplo (classificadores e pré-processadores); a construção de ensembles combina os modelos avaliados, em vez de ficar com um só.
18. Buscou-se um funcionamento sem configuração manual, com meta-aprendizado sem meta-atributos e uma estratégia de bandidos para alocar orçamento, com ganhos reportados em um benchmark aberto de 39 conjuntos.
19. Custo (mais memória e tempo de previsão), perda de interpretabilidade e avaliação honesta (pesos na validação, medida no teste). Exemplo: um ensemble de 30 modelos custa cerca de 30 vezes mais para prever, o que pesa em produção.
20. O Auto-sklearn combina busca bayesiana (Cap. 5), orçamento por bandidos (Cap. 6), pipelines (Cap. 3), validação honesta (Caps. 2 e 4), meta-aprendizado e ensembles.

Capítulo 10 — Ferramentas de AutoML na prática

Múltipla escolha

Questão	Resposta	Justificativa
1	a)	Dados e orçamento entram; modelo ou ensemble e *leaderboard* saem.
2	c)	O TPOT usa programação genética para evoluir pipelines (Olson; Moore, 2016).
3	d)	Essas são as três ideias da arquitetura do Auto-sklearn.
4	b)	O FLAML começa por configurações baratas e aumenta o esforço onde vale a pena.
5	d)	O FLAML investe primeiro em configurações baratas e só aprofunda onde há sinal.
6	c)	É o orçamento de tempo da busca.
7	d)	Por exemplo, `lgbm`, `rf` ou `extra_tree`.
8	b)	Registra tentativa, algoritmo, configuração e tempo de cada melhoria.
9	b)	Em 10 segundos, máquinas ou execuções diferentes concluem números diferentes de tentativas.
10	a)	Isso reduz a dependência do tempo de relógio.
11	d)	Restringir pode acelerar a busca, mas pode excluir o melhor algoritmo.
12	a)	É o resultado do problema CASH, por algoritmo.
13	a)	A diferença entre orçamentos pode caber dentro da variação entre divisões.
14	d)	Orçamentos desiguais invalidam a comparação.
15	d)	Se o AutoML não supera a linha de base, ele não se justifica.
16	b)	Uma só divisão pode favorecer ou prejudicar um método por acaso.
17	c)	Com 45 exemplos, 1 exemplo equivale a cerca de 2,2 pontos percentuais.
18	c)	O posto médio resume a posição ao longo das bases.
19	d)	Se for rejeitada, comparam-se os pares com um teste posterior.
20	a)	Demšar (2006) descreve o uso do teste de Nemenyi.
21	b)	Com poucos blocos, o teste detecta apenas diferenças enormes.
22	c)	Por isso o Friedman usou a média por base (N = 4).
23	c)	O benchmark controla tarefas, divisões e orçamentos.
24	a)	Cada ferramenta tem pontos fortes em contextos diferentes.
25	a)	É um estudo de referência sobre dados tabulares.
26	c)	Descrito por Hollmann et al. (2025).
27	a)	O TPOT exporta o pipeline como código Python.
28	d)	O FLAML foi desenhado para custo baixo.
29	d)	Muitos modelos custam mais memória e tempo na produção.
30	a)	O livro avisa que esses trechos seguem a documentação, mas não foram executados.

Discursivas (respostas-modelo)

1. Você entrega os dados de treino e um orçamento (em geral, tempo). A ferramenta escolhe sozinha o tratamento dos dados, os algoritmos e os hiperparâmetros, avalia os candidatos por validação e devolve o melhor modelo (ou um ensemble), em geral com uma tabela dos modelos avaliados.
2. O garimpeiro faz peneiradas rápidas em vários pontos e só cava fundo onde viu brilho. O FLAML testa primeiro configurações baratas (modelos pequenos) e só investe mais onde há ganho.

3. Auto-sklearn: meta-aprendizado, otimização bayesiana e ensemble. TPOT: programação genética de pipelines. H2O e AutoGluon: muitos modelos combinados em ensembles empilhados. FLAML: busca de baixo custo, leve e rápida.
4. best_estimator (algoritmo vencedor), best_config (hiperparâmetros), best_loss (erro de validação) e config_history (histórico das configurações que melhoraram o melhor resultado, útil para inspeção e diagnóstico).
5. O número de tentativas concluídas em N segundos depende da máquina e da carga. Para mitigar, limita-se o número de tentativas (max_iter), fixa-se a semente e registram-se as versões das bibliotecas.
6. Ele escolhe o algoritmo e seus hiperparâmetros ao mesmo tempo, avaliando vários algoritmos e guardando o melhor resultado de cada um.
7. No início, mais tempo melhora o modelo; depois os ganhos diminuem (retornos decrescentes). Com duas divisões, a diferença entre orçamentos pode caber na variação entre divisões, então só se vê tendência, não uma lei.
8. Mesmos dados e divisões; mesmo orçamento; teste final intocado; várias divisões; várias bases; linhas de base simples.
9. Porque mostra o ganho real do AutoML sobre um modelo padrão e sem ajuste. Se o ganho for nulo ou menor que a variação, a automação não se justifica naquele problema.
10. Em cada base, ordenam-se os métodos (1 = melhor) e tira-se a média dos postos. Isso evita que uma base com notas muito altas ou muito baixas domine o resumo e compara a posição relativa dos métodos.
11. Testa que todos os métodos são equivalentes em todas as bases. Se for rejeitada, comparam-se os pares com um teste posterior, como o de Nemenyi (Demšar, 2006).
12. Com poucas bases o teste tem baixo poder: só detecta diferenças muito grandes. A pesquisa séria usa dezenas de bases; um experimento com quatro serve como exercício de método.
13. São recortes dos mesmos dados, portanto não independentes entre si. Elas estabilizam a nota daquela base, mas não ampliam o número de "rodadas" independentes do teste.
14. O teste tem cerca de 45 exemplos, e cada exemplo vale mais de 2 pontos percentuais. Diferenças de um ou dois exemplos estão dentro do ruído.
15. Não há vencedor universal: o desempenho relativo depende da base, da tarefa e do orçamento (Gijsbers et al., 2024; Zöller; Huber, 2021). Por isso se deve comparar em condições equivalentes às do seu problema.
16. Estudos como o de Grinsztajn, Oyallon e Varoquaux (2022) mostram que modelos baseados em árvores seguem muito competitivos frente a redes neurais em dados tabulares típicos.
17. É um modelo fundacional (pré-treinado) para dados tabulares pequenos, que prevê sem busca de hiperparâmetros por conjunto (Hollmann et al., 2025). Sugere que modelos pré-treinados podem substituir parte da busca.
18. FLAML para pouco tempo ou recursos; TPOT para auditar o pipeline; AutoGluon ou H2O para máxima precisão com tempo e memória; Auto-sklearn no ecossistema scikit-learn com meta-aprendizado; H2O para dados muito grandes ou cluster.
19. Porque essas bibliotecas não puderam ser instaladas no ambiente em que o livro foi preparado. O código segue a documentação oficial, mas as versões mudam e nenhum resultado é reportado.
20. Comece por uma linha de base; registre semente, orçamento, métrica e versões; dê o mesmo orçamento aos métodos comparados; use um teste final intocado; repita com várias divisões e inspecione o modelo final.

Capítulo 11 — Busca de arquitetura neural (NAS)

Múltipla escolha

Questão	Resposta	Justificativa
1	a)	Em analogia, é a planta da casa; o treino é a construção.
2	c)	NAS é a busca de arquitetura neural (Elsken et al., 2019).
3	d)	É a organização usada para estruturar a literatura.
4	b)	O espaço define o teto: o que não está nele não pode ser achado.
5	c)	Exemplos: relu e tanh.
6	a)	É o espaço mais simples.
7	b)	Zoph et al. (2018) usaram células transferidas entre conjuntos de imagens.
8	b)	O espaço a explorar fica muito menor que o da rede inteira.
9	c)	2 × (5 + 25 + 125) = 310.
10	c)	Isso permitiu repetir cada estratégia 300 vezes consultando uma tabela.
11	a)	1.797 imagens 8×8 de dígitos, 10 classes.
12	a)	O MLPClassifier permite escolher camadas e ativação.
13	a)	Parâmetros são os pesos e vieses aprendidos.
14	c)	A recompensa é o desempenho da arquitetura gerada.
15	a)	Isso favorece arquiteturas que seguem boas ao longo do tempo.
16	b)	Mutação: mudar largura, inserir/remover camada ou trocar ativação.
17	a)	Liu, Simonyan e Yang (2019) propuseram a busca diferenciável.
18	d)	Pham et al. (2018) tratam candidatas como partes de uma grande rede.
19	c)	Como no Capítulo 4, ela é difícil de superar com folga.
20	a)	Escolher pelo teste causa vazamento; as demais reduzem custo.
21	b)	Vale 1 se a ordem é idêntica.
22	b)	Isso indica que a estimativa barata preservou bem a ordem.
23	a)	Quanto menor a correlação entre fidelidades, maior o risco.
24	a)	A diferença é pequena, mas a evolução ficou mais bem posicionada.
25	a)	A vantagem na validação foi em parte otimismo da busca.
26	c)	É o efeito do vencedor visto no Capítulo 2.
27	d)	Nenhuma outra rede é melhor nos dois objetivos.
28	b)	11.794 parâmetros contra 26.122, 0,960 contra 0,970.
29	d)	Green AI propõe eficiência como critério de avaliação.
30	d)	Em dados tabulares típicos, árvores seguem muito competitivas.

Discursivas (respostas-modelo)

1. A planta define andares, cômodos e ligações, e os pedreiros apenas constroem; na rede, a arquitetura define camadas, neurônios e conexões, e o treino a 'constrói'. Uma planta ruim não é salva pelo treino.
2. Espaço de busca (o que pode ser encontrado), estratégia de busca (como explorar) e estimativa de desempenho (como avaliar depressa). Equivalem a espaço de hiperparâmetros, método de otimização e avaliação barata (multifidelidade).

3. Porque a busca só pode encontrar arquiteturas que pertencem ao espaço. Se a melhor rede não está nele, nenhuma estratégia a encontrará.
4. Em cadeia: sequência de camadas escolhidas pela busca. Células: a busca desenha um bloco pequeno, repetido e empilhado, reduzindo o espaço. Hierárquico: células feitas de células menores, em vários níveis.
5. Para L camadas há 5^L combinações de largura; somando L = 1 a 3 dá 5 + 25 + 125 = 155; com 2 ativações, 310.
6. Para conhecer a verdade (a melhor rede) e repetir as estratégias 300 vezes consultando uma tabela, algo impossível em espaços reais, o que permitiu medir com segurança.
7. Aleatória: sorteia. Reforço: um controlador gera arquiteturas e recebe a nota como recompensa. Evolução: sorteia da população, escolhe a melhor, muta e descarta a mais velha. Gradiente (DARTS): escolha contínua otimizada por gradiente.
8. Em cada passo sorteia-se uma amostra, escolhe-se a melhor, gera-se uma filha por mutação e remove-se a mais velha. Isso favorece arquiteturas que continuam boas e evita que uma sorte isolada domine a população.
9. Mudar a largura de uma camada, inserir uma camada, remover uma camada e trocar a função de ativação.
10. Reaproveitar pesos entre candidatas, tratadas como partes de uma grande rede. Resolve o alto custo de treinar cada arquitetura do zero.
11. Na validação, a evolução teve mediana um pouco maior (0,969 contra 0,965) e posto mediano 1,5 contra 6; no teste, as escolhas ficaram praticamente iguais.
12. A busca escolhe o que vai melhor na validação, então parte do ganho é otimismo da seleção. O teste, que a busca nunca viu, mostra diferença muito menor.
13. Mede a concordância entre as ordens de duas listas. Importa porque uma estimativa barata só é útil se preservar a ordem das arquiteturas em relação ao treino completo.
14. Avaliaram-se as 310 redes com 10 iterações e mantiveram-se as k melhores para treinar até 40. A melhor rede sobreviveu em todos os cortes testados (k = 10 a 120), com custo de 28% a 64%.
15. Quando redes lentas para começar, que só brilham no fim do treino, são descartadas cedo. Quanto menor a correlação entre fidelidades, maior o risco.
16. Treino curto (multifidelidade), compartilhamento de pesos, extrapolação da curva de aprendizado e preditores de desempenho.
17. Busca-se equilibrar acurácia e custo (parâmetros, tempo, energia). A fronteira reúne as redes não dominadas, aquelas sem outra melhor nos dois critérios ao mesmo tempo.
18. A rede de 11.794 parâmetros (0,960) é 55% menor que a mais acurada (26.122; 0,970), por cerca de um ponto de acurácia, o que pode ser preferível em dispositivos simples.
19. É a proposta de reportar o custo computacional junto com a precisão e valorizar a eficiência (Schwartz et al., 2020). Em NAS: começar pequeno, usar estimativas baratas, reaproveitar resultados e relatar o custo da busca.
20. Em dados tabulares típicos, em que árvores seguem muito competitivas (Grinsztajn et al., 2022), e quando há arquiteturas conhecidas e boas, pois a busca é cara e pode não superar linhas de base.

Capítulo 12 — AutoML para séries temporais, texto e imagens

Múltipla escolha

Questão	Resposta	Justificativa
1	b)	Em séries, o passado treina e o futuro testa.
2	b)	É uma linha de base difícil de superar em muitas séries.
3	c)	Aproveita a sazonalidade, mas ignora a tendência.
4	d)	Exemplo: o ciclo anual de CO_2.
5	c)	A série de CO_2 cresce ao longo das décadas.
6	c)	Ex.: y(t−1), y(t−2), y(t−3).
7	a)	Ajuda a remover tendência e sazonalidade.
8	d)	Isso explica o erro enorme do boosting sobre valores brutos.
9	d)	TimeSeriesSplit respeita a ordem.
10	d)	A aleatória permite interpolar entre vizinhos no tempo.
11	b)	Erro de teste de 0,227.
12	c)	O mini-AutoML teve 0,227.
13	a)	Simula o uso real.
14	c)	Makridakis et al. (2020).
15	a)	Gama et al. (2014).
16	b)	TF-IDF é uma técnica clássica.
17	c)	Palavras como 'o' e 'de' recebem peso baixo.
18	a)	Bigramas: 'não bom'.
19	b)	Palavras isoladas ficaram perto do acaso.
20	b)	Perto do acaso em um problema com 40% de negação.
21	a)	Os números ilustram um efeito, não o desempenho em textos reais.
22	c)	Baseado em Transformers.
23	a)	Reaproveita o conhecimento do pré-treino.
24	b)	Ex.: deslocar, girar, adicionar ruído.
25	a)	O espaço é quais distorções aplicar e com que intensidade.
26	b)	Grande ganho sob distorção.
27	b)	O modelo divide capacidade entre imagens originais e distorcidas.
28	b)	A ampliação deve imitar a variação do uso real.
29	d)	Como a validação tinha distorções, premiou políticas robustas a elas.
30	d)	Reaproveita representações aprendidas em grandes conjuntos.

Discursivas (respostas-modelo)

1. Porque o modelo treina com valores posteriores aos que vai prever (vazamento do futuro), o que não ocorre na prática e torna a estimativa de erro otimista.

2. Na temporal, treina-se no passado e valida-se em janelas posteriores que avançam no tempo. Em avanço (walk-forward), reajusta-se o modelo a cada novo ponto e prevê-se o seguinte, simulando o uso real.
3. Ingênua (valor anterior) e sazonal ingênua (mesmo mês do ano anterior). São difíceis de superar em muitas séries e mostram se o modelo realmente agrega valor.
4. Usam-se como atributos os L valores anteriores (atrasos) e informações como o mês do ano; o alvo é o valor seguinte (ou a diferença em relação a um valor anterior).
5. Prever a mudança em relação ao mesmo mês do ano anterior. Remove tendência e sazonalidade e evita que a árvore precise extrapolar valores fora do que viu no treino.
6. Porque árvores não extrapolam: ao prever meses com valores acima de todo o treino, repetem valores que já viram, subestimando a tendência de crescimento.
7. Modelo (Ridge ou HistGradientBoosting), atrasos (6, 12 ou 24) e diferenciação (sim ou não): 12 configurações, escolhidas por validação temporal.
8. O mini-AutoML teve MAE 0,227, contra 1,077 da ingênua e 1,413 da sazonal ingênua; a pior configuração teve 1,399.
9. Porque os erros se acumulam com o horizonte e os valores intermediários não são observados; por isso é preciso informar o horizonte avaliado.
10. É a mudança do padrão dos dados ao longo do tempo. O modelo envelhece e perde precisão, exigindo monitoramento e reajuste.
11. O texto é vetorizado (cada palavra ou n-grama recebe um peso que é alto se for frequente no documento e raro nos outros) e um classificador, como a regressão logística, decide a classe.
12. Faixa de n-gramas, frequência mínima (min_df), TF sublinear, e também o classificador usado.
13. A negação só se revela na combinação 'não bom'. Palavras isoladas veem 'não' e 'bom' separadamente e não sabem a relação; o bigrama a captura.
14. Porque os dados foram gerados por um programa que exagera a negação. Ele ilustra um fenômeno, mas os números não representam o desempenho em textos reais.
15. Em vez de escolher n-gramas, escolhe-se qual modelo pré-treinado usar e como ajustá-lo (taxa de aprendizado, épocas), mantendo a lógica de busca, validação e orçamento.
16. É criar novas imagens por pequenas distorções (deslocar, girar, ruído). Ensina ao modelo invariâncias (um 7 deslocado continua 7) e reduz o sobreajuste.
17. Sorteiam-se 15 políticas (deslocamento, rotação, ruído), geram-se 4 cópias distorcidas por imagem, treina-se uma SVM e escolhe-se a política pela validação distorcida; o teste é medido uma única vez.
18. No teste distorcido, a acurácia subiu de 0,529 para 0,873; no limpo, caiu de 0,962 para 0,939. A ampliação traz robustez com pequeno custo.
19. Porque ela define o que a busca valoriza. Com validação distorcida, escolhem-se políticas robustas a distorção; com validação limpa, outra política seria escolhida.
20. Quando há poucos dados em imagens ou texto: reaproveita-se uma rede pré-treinada em grandes conjuntos e reajusta-se para a tarefa.

Capítulo 13 — Explicabilidade, ética e limitações do AutoML

Múltipla escolha

Questão	Resposta	Justificativa
1	d)	Rudin (2019) defende preferir interpretáveis em alto risco.
2	c)	Explicações pós-hoc podem ser imprecisas.
3	a)	Por isso vale preferir o modelo simples.
4	a)	Ganha-se explicação, velocidade e menos risco.
5	a)	Ex.: importância média dos atributos.
6	d)	Ex.: contribuições SHAP de um exemplo.
7	a)	Se cai muito, a coluna importa.
8	a)	Lundberg e Lee (2017).
9	d)	Divide-se a 'conta' entre os atributos.
10	d)	É uma explicação local.
11	a)	Um modelo enviesado pode ser perfeitamente explicável.
12	a)	Mehrabi et al. (2021).
13	d)	É o caso do nosso crédito simulado.
14	d)	Falta de exemplos de um grupo.
15	b)	Cria um ciclo que se retroalimenta.
16	c)	Se muito diferente entre grupos, há disparidade.
17	d)	Compara o TPR por grupo.
18	b)	Ex.: CEP-score e renda.
19	a)	Seleção 0,48/0,39 e TPR real 0,83/0,69.
20	d)	No mundo real, essa verdade é desconhecida.
21	d)	No capítulo, igualam a taxa de seleção.
22	a)	A acurácia contra o histórico caiu de 0,818 para 0,801.
23	c)	A escolha do critério é uma decisão humana e social.
24	a)	O padrão de comparação estava contaminado.
25	a)	O custo da busca pode ser muito maior que o de um único treino.
26	a)	Green AI.
27	a)	Treinos × tempo por treino × consumo.
28	a)	O AutoML otimiza a métrica que você der, sobre os dados que você der.
29	d)	Por isso o AutoML precisa procurar.
30	a)	AUC de 0,5 seria o acaso; 0,857 indica forte relação entre atributos e grupo.

Discursivas (respostas-modelo)

1. Eu entendo o que ele faz (explicabilidade)? Ele trata as pessoas com justiça (ética)? Quanto custa e o que ele não consegue (custo e limites)?

2. Interpretáveis têm lógica legível; pós-hoc explicam depois. Rudin (2019) defende preferir os interpretáveis em alto risco, pois explicações pós-hoc podem ser imprecisas e não revelam o que o modelo realmente calcula.
3. Regressão logística e boosting tiveram notas muito próximas (0,981 e 0,968), dentro da variação. Regra do empate técnico: escolha o modelo simples.
4. Global: o que o modelo valoriza em geral (importância média de atributos). Local: por que decidiu isso para esta pessoa (contribuições SHAP de um exemplo).
5. Embaralha-se uma coluna e mede-se a queda de desempenho. Com atributos correlacionados, a importância se reparte de forma instável e é propriedade do modelo, não do mundo.
6. Como dividir de forma justa a conta do restaurante pela contribuição média de cada pessoa em todas as ordens, o SHAP divide a diferença entre a previsão e a média entre os atributos (valores de Shapley).
7. A explicação mostra como o modelo decidiu, não se a decisão é justa. Um modelo enviesado pode ser perfeitamente explicável.
8. Histórico (decisões passadas injustas), representação (grupo pouco presente), medição (variável que retrata mal o que importa) e feedback (decisões do modelo alteram dados futuros).
9. Taxa de seleção: proporção de aprovados no grupo. Igualdade de oportunidade: entre quem mereceria a aprovação, a taxa de aprovação (TPR) deve ser igual entre os grupos.
10. As decisões históricas exigiam mais do grupo 1: aprovação de 0,29 contra 0,51, embora a proporção de bons pagadores fosse parecida (0,49 contra 0,51).
11. Porque variáveis proxy (renda, CEP) o reconstroem e os rótulos históricos já carregam o viés; o modelo aprende o mesmo padrão.
12. Em ambos, o grupo 1 foi aprovado menos (seleção 0,38 e 0,39 contra 0,48) e teve TPR real menor (0,67 e 0,69 contra 0,81 e 0,83).
13. Escolhe-se um limiar menor para o grupo 1 para igualar a taxa de seleção. A seleção igualou-se (0,49), o TPR real do grupo 1 subiu de 0,67 para 0,81 e a acurácia contra a verdade subiu de 0,847 para 0,858.
14. Em dados reais a verdade é desconhecida, os rótulos podem ser enviesados e as definições de justiça podem entrar em conflito. A escolha do critério é decisão humana, social e, às vezes, legal.
15. Porque o histórico é o enviesado: igualar contra ele considera 'correto' o que ele aprovou. No exercício, o TPR real do grupo 1 até caiu (0,67 para 0,63).
16. Treinar um classificador para prever o atributo sensível a partir dos demais atributos: AUC alta indica que o grupo pode ser reconstruído; remover atributos um a um mostra os que mais contribuem.
17. Contando quantos modelos serão treinados (× tempo por treino × consumo). Reduz-se com busca aleatória em vez de grade, parada precoce/Hyperband, warm start e perguntando se a busca vale a pena.
18. Não define o problema; não corrige dados ruins; não há almoço grátis; otimiza a validação (otimismo da seleção); não prevê mudanças do mundo; tem custo; pode esconder decisões.
19. Antes: quem é afetado, origem dos dados, ausentes, rótulos. Durante: linha de base e modelo interpretável, validação adequada, desempenho por grupo. Depois: explicação, contestação, monitoramento, custo.
20. Porque automatiza a busca de modelos, mas não decide o problema, a métrica, a qualidade dos dados, a justiça, a explicação nem o monitoramento, que continuam humanos.

Capítulo 14 — MLOps, implantação e projeto integrador

Múltipla escolha

Questão	Resposta	Justificativa
1	d)	Kreuzberger, Kühl e Hirschl (2023).
2	a)	Há coleta de dados, configuração, infraestrutura e monitoramento ao redor.
3	a)	Criar a receita é só uma parte do trabalho.
4	c)	O ciclo é contínuo.
5	a)	O StandardScaler, por exemplo, deve ir junto.
6	b)	Permite reproduzir e auditar.
7	d)	No exercício, mudar 0,001 em um valor mudou o hash.
8	c)	Nunca carregue modelos de origem desconhecida.
9	d)	A latência importa pouco; a vazão importa.
10	a)	A latência por previsão é crítica.
11	b)	Cerca de 0,5 ms contra cerca de 13 ms por previsão isolada.
12	b)	Muitas ferramentas permitem restringir algoritmos e tamanho do ensemble.
13	b)	Ex.: rendas sobem por inflação.
14	c)	Ex.: com a mesma renda, a chance de pagar passa a ser outra.
15	d)	Em produção, rótulos chegam semanas depois.
16	a)	Valor-p pequeno indica distribuições diferentes.
17	b)	Population Stability Index.
18	d)	São regras práticas, não leis.
19	a)	Isso mostra o valor do alarme antecipado.
20	d)	O alarme não dispara por qualquer motivo.
21	d)	No projeto integrador, o PSI foi 0,304 e a acurácia não caiu.
22	c)	Evita trocas por ruído.
23	b)	Ex.: diferença maior que 2 erros padrão.
24	a)	Mesmo com ganho médio positivo, nem sempre passa da margem.
25	b)	A 'produção' tinha só 54 casos.
26	d)	O melhor único teve cerca de 0,5 ms.
27	c)	O AutoML não garante que o complexo vença.
28	b)	Um relatório honesto vale mais que um ganho vistoso sem teste.
29	d)	Reprodutibilidade: previsões idênticas após salvar e carregar.
30	a)	O alarme pede investigação, não ação automática.

Discursivas (respostas-modelo)

1. Criar a receita (o modelo) é só uma parte; servir todo dia, com qualidade, exige estoque, higiene, equipe e controle (dados, infraestrutura, monitoramento): o trabalho de MLOps.
2. O código do modelo é pequeno diante do sistema (coleta e verificação de dados, atributos, configuração, infraestrutura, monitoramento). Esse entorno gera dívida técnica escondida que cobra juros ao longo do tempo.

3. Dados, treino e busca, validação, implantação, monitoramento e retreino, voltando aos dados; o ciclo é contínuo.
4. Versões das bibliotecas, semente, parâmetros e hash dos dados, para reproduzir, auditar e saber com que dados e ambiente o modelo foi feito.
5. Porque eles podem executar código arbitrário ao serem carregados, o que é um risco de segurança.
6. Lote: muitos casos de uma vez, de tempos em tempos; importa a vazão. Tempo real: um caso por vez, resposta em milissegundos; a latência é crítica.
7. Cada previsão exige rodar vários modelos: cerca de 13 ms contra 0,5 ms da regressão logística por previsão isolada, e 0,8 s contra 0,05 s em lote de 10 mil. Em milhares de pedidos por segundo, a diferença pesa.
8. Dados: a distribuição dos atributos muda (rendas sobem com a inflação). Conceito: a relação entre atributos e resposta muda (com a mesma renda, a chance de pagar é outra).
9. KS: teste que compara a distribuição de um atributo entre treino e produção. PSI: compara a proporção de casos por faixa (quantis do treino); regras práticas: abaixo de 0,10 estável, 0,10 a 0,25 alerta, acima de 0,25 mudança grande.
10. O PSI da renda cresce mês a mês (cruza 0,10 no mês 2 e 0,25 no mês 4) e o KS rejeita já no mês 1, antes de a acurácia cair (0,851 para 0,808 no mês 6); o PSI do tempo de emprego fica abaixo de 0,01.
11. Deriva dos dados não implica queda de desempenho: no projeto integrador, com PSI de 0,304, a acurácia contra a verdade não caiu (0,853 para 0,864). O alarme pede investigação com rótulos.
12. O modelo novo, treinado com dados recentes, é comparado ao em produção em dados recentes com rótulos; só assume se a diferença superar uma margem (por exemplo, 2 erros padrão), evitando trocas por ruído.
13. O ganho médio do desafiante foi positivo (+0,015), mas ele só assumiu em 9 de 20 repetições; é preciso margem, mais dados e consideração do custo.
14. Com 10 faixas e só 54 casos, a proporção por faixa oscila muito. No Wine, mesmo sem deriva, o PSI de total_phenols foi 0,954; as regras práticas valem para amostras grandes.
15. Problema e métrica; teste final e vazamento; linhas de base; pipeline; busca; ensemble e ferramentas; teste único por grupo; explicar e auditar; implantar com ficha; monitorar e retreinar.
16. A busca escolheu a regressão logística; os métodos ficaram empatados no teste (0,804 a 0,808); o ensemble custou cerca de 50 ms contra o orçamento de 20 ms. Recomenda-se o modelo simples, com monitoramento por grupo, limiares sujeitos a revisão, contestação e revisão humana.
17. Não ganhou do modelo simples no teste (diferenças dentro do ruído), estourou o orçamento de latência e é mais difícil de explicar.
18. Monitoramento de desempenho, deriva e disparidades por grupo; canal de contestação; revisão humana; revisão jurídica do uso de limiares ou atributos sensíveis; registro da ficha e do custo.
19. Valorizar relatórios que reconhecem empates, limites, custo e incerteza, em vez de anunciar ganhos pequenos sem teste de significância.
20. Eles detectam mudanças nos dados de entrada, mas não dizem se o desempenho caiu nem se a relação entre atributos e resposta mudou; só os rótulos recentes permitem medir o desempenho real.

Apêndice B — Glossário

Termos técnicos usados no livro, explicados de forma simples.

Acurácia — proporção de previsões corretas em um problema de classificação.

Algoritmo genético — método de busca inspirado na evolução, com população, seleção, cruzamento e mutação.

AMLB — benchmark aberto para comparar ferramentas de AutoML em muitas tarefas, com divisões e orçamentos controlados.

Amostragem de hiperparâmetros — sorteio de valores de hiperparâmetros a partir de distribuições, como na busca aleatória.

Ampliação de dados (data augmentation) — criação de novos exemplos de treino por pequenas distorções dos existentes, como deslocar ou girar uma imagem.

Arquitetura (de uma rede neural) — desenho da rede, com número de camadas, de neurônios, conexões e funções de ativação.

Árvore de decisão — modelo que divide os dados por perguntas sucessivas sobre os atributos até chegar a uma previsão.

Atraso (lag) — valor passado de uma série usado como atributo para prever o valor seguinte.

Atributo (feature) — cada coluna usada como entrada de um modelo.

AUC — área sob a curva ROC; mede a capacidade de separar duas classes, de 0,5 (acaso) a 1 (perfeito).

Auto-Keras — biblioteca que oferece busca de arquiteturas neurais sobre o Keras.

Auto-sklearn — sistema de AutoML sobre o scikit-learn que combina meta-aprendizado, otimização bayesiana e ensembles.

AutoGluon — sistema de AutoML que empilha muitos modelos em camadas, com ênfase em robustez.

AutoML — aprendizado de máquina automatizado; automatiza escolhas como pré-processamento, algoritmo, hiperparâmetros e ensembles.

Avaliação barata — estimativa rápida da qualidade de um candidato, por exemplo com poucas épocas ou menos dados.

Baseline (linha de base) — modelo simples usado como referência mínima de comparação.

Benchmark — conjunto padronizado de tarefas e regras para comparar métodos.

Bias (viés) de dados — distorção nos dados que leva o modelo a tratar grupos de forma desigual.

BOHB — método que combina o Hyperband com otimização bayesiana.

Boosting — ensemble em que cada modelo novo tenta corrigir os erros dos anteriores.

Busca aleatória — sorteia combinações de hiperparâmetros e avalia cada uma.

Busca bayesiana (otimização bayesiana) — usa os resultados anteriores para decidir qual configuração testar a seguir.

Busca em grade — testa todas as combinações de uma grade definida de valores.

Camada — conjunto de neurônios de uma rede que processam, em paralelo, a saída da camada anterior.

Campeão e desafiante — prática de comparar o modelo em produção com um novo modelo em dados recentes, trocando só se o novo ganhar com margem.

CASH — problema da seleção combinada de algoritmo e hiperparâmetros.

Célula (em NAS) — bloco pequeno de uma rede, desenhado pela busca e repetido para formar a rede toda.

Classificação — tarefa de prever uma categoria.

Classificador — modelo que prevê categorias.

Cobertura do espaço — quanto do espaço de busca as tentativas realmente exploram.

Concept drift (deriva de conceito) — mudança na relação entre atributos e resposta ao longo do tempo.

Configuração — conjunto de valores de hiperparâmetros (e do algoritmo) de um candidato.

Conjunto de teste — dados reservados para a avaliação final, usados uma única vez.

Conjunto de treino — dados usados para ajustar os parâmetros do modelo.

Conjunto de validação — dados usados para comparar candidatos e escolher entre eles.

Correlação de Spearman — correlação entre os postos de duas listas; vale 1 se a ordem coincide.

Covariate shift (deriva dos dados) — mudança na distribuição dos atributos de entrada.

Cross-validation (validação cruzada) — divisão dos dados em partes que se revezam como validação.

Custo computacional — tempo, memória e energia consumidos para treinar e usar modelos.

DARTS — método de busca de arquiteturas que torna a escolha de operações contínua e a otimiza por gradiente.

Data leakage (vazamento de dados) — uso, no treino, de informação que não estaria disponível na hora real da previsão.

Deriva — mudança dos dados ou do conceito que faz o modelo envelhecer.

Desvio-padrão — medida de dispersão dos valores em torno da média.

Diferenciação — transformar a série para prever a mudança em relação a um valor anterior, removendo tendência e sazonalidade.

Dimensionalidade — número de atributos.

Divisão estratificada — divisão que mantém a proporção das classes em cada parte.

Early stopping (parada precoce) — interromper o treino quando a validação deixa de melhorar.

Efeito do vencedor (winner's curse) — otimismo da nota do melhor candidato escolhido entre muitos.

Embedding — representação de itens (como palavras) por vetores de números.

Ensemble — combinação de vários modelos em uma única previsão.

Entropia — medida de incerteza ou de desequilíbrio entre classes.

Época — uma passagem completa do treino por todos os dados.

Erro padrão — estimativa da incerteza de uma média ou de uma proporção.

Espaço condicional — espaço em que alguns hiperparâmetros só existem se outra escolha foi feita.

Espaço de busca — conjunto de todas as configurações ou arquiteturas que a busca pode considerar.

Estimador (no scikit-learn) — objeto que aprende a partir de dados, com os métodos fit e predict.

Estratégia de busca — forma de explorar o espaço de busca (aleatória, evolutiva, bayesiana, por gradiente).

Explicabilidade — capacidade de explicar por que um modelo toma suas decisões.

Explicação global — descrição do que o modelo valoriza em geral.

Explicação local — explicação de uma decisão específica.

Extrapolar — prever valores fora da faixa vista no treino.

Fidelidade (em multifidelidade) — nível de custo de uma avaliação, como número de épocas ou fração dos dados.

FLAML — biblioteca leve de AutoML com busca de baixo custo.

Fronteira de Pareto — conjunto de soluções para as quais nenhuma outra é melhor em todos os objetivos ao mesmo tempo.

Função de aquisição — regra da otimização bayesiana que decide qual ponto avaliar a seguir.

Função de ativação — regra que define a saída de um neurônio, como relu e tanh.

Gradiente — direção de maior variação de uma função; usado para ajustar parâmetros por otimização.

Hiperparâmetro — ajuste do algoritmo definido antes do treino, como a profundidade de uma árvore.

Holdout — divisão simples em treino e validação (ou teste).

Hyperband — método de multifidelidade que roda várias execuções de Successive Halving com orçamentos diferentes.

Igualdade de oportunidade — critério em que a taxa de verdadeiros positivos é igual entre os grupos.

Imputação — preenchimento de valores ausentes.

Interpretabilidade — grau em que a lógica do modelo pode ser lida diretamente por uma pessoa.

Intervalo de confiança — faixa de valores plausíveis para uma quantidade estimada.

Kolmogorov-Smirnov (KS) — teste que compara duas distribuições.

Landmarker — nota de um modelo simples e rápido usada como meta-atributo.

Latência — tempo entre o pedido e a resposta de uma previsão.

Leaderboard — tabela que ordena os modelos avaliados por desempenho.

LightGBM — biblioteca eficiente de boosting de árvores.

LIME — método que explica uma decisão ajustando um modelo simples na vizinhança dela.

Meta-aprendizado — aprender a aprender; usar a experiência de problemas anteriores para decidir melhor em um novo.

Meta-atributo — característica que descreve um conjunto de dados inteiro.

Meta-modelo — modelo que prevê qual algoritmo ou configuração funciona melhor em um conjunto, a partir de meta-atributos.

Métrica — número que resume o desempenho de um modelo, como acurácia ou erro.

MLOps — práticas para desenvolver, implantar e operar modelos em produção.

Mutação — pequena alteração aleatória de uma solução em um algoritmo evolutivo.

NAS (Neural Architecture Search) — busca automática da arquitetura de uma rede neural.

Neurônio — unidade de cálculo de uma rede neural.

Normalização e padronização — reescala dos atributos, por exemplo para média 0 e desvio 1.

NSGA-II — algoritmo genético multiobjetivo que ordena as soluções por dominância de Pareto.

Orçamento — limite de tempo, de tentativas ou de recursos de uma busca.

Otimismo da busca — tendência de a nota do melhor candidato superestimar seu desempenho real.

Overfitting (sobreajuste) — quando o modelo memoriza o treino e generaliza mal.

Parâmetro — valor que o modelo aprende durante o treino, como os pesos de uma rede.

Paridade demográfica — critério em que a taxa de seleção é igual entre os grupos.

Perceptron multicamadas (MLP) — rede neural de camadas totalmente conectadas.

Permutação (importância por) — mede a queda de desempenho ao embaralhar uma coluna.

Pipeline — sequência de etapas (pré-processamento e modelo) tratada como um único objeto.

Poda (pruning) — interrupção antecipada de tentativas pouco promissoras.

Posto (rank) — posição de um método quando comparado aos demais.

Previsão ingênua — prever que o próximo valor é igual ao último valor observado.

Previsão sazonal ingênua — prever que o valor é igual ao do mesmo período do ano anterior.

Proxy (variável) — variável correlacionada com um atributo sensível que o carrega indiretamente.

PSI (Population Stability Index) — índice que compara a distribuição de um atributo entre dois períodos.

Quantil — valor que divide os dados em partes de mesmo tamanho.

Rede neural — modelo formado por camadas de neurônios conectados.

Regressão logística — modelo linear para classificação que estima probabilidades.

Regressão — tarefa de prever um número.

Regularização — penalidade que evita modelos complexos demais.

Retreino — ajuste do modelo com dados mais recentes.

Ridge — regressão linear com penalidade que estabiliza os coeficientes.

Sazonalidade — padrão que se repete em períodos regulares.

SHAP — método que atribui a cada atributo sua contribuição para uma previsão, com base nos valores de Shapley.

SMAC — método de otimização bayesiana baseado em florestas aleatórias.

Stacking — ensemble em que um modelo final aprende a combinar as previsões de outros.

Successive Halving — descarta, a cada rodada, a metade menos promissora dos candidatos e dá mais recursos aos restantes.

SVM (máquina de vetores de suporte) — modelo que busca a fronteira de maior margem entre classes.

Taxa de seleção — proporção de pessoas aprovadas em um grupo.

Taxa de verdadeiros positivos (TPR) — entre os casos positivos reais, a proporção que o modelo acertou.

Tendência — crescimento ou queda de longo prazo de uma série.

TF-IDF — peso que valoriza palavras frequentes no documento e raras na coleção.

TPE — estimador de Parzen estruturado em árvore, usado pelo Optuna e pelo Hyperopt.

TPOT — ferramenta que evolui pipelines por programação genética.

Transfer learning (aprendizado por transferência) — reaproveitar um modelo pré-treinado e reajustá-lo à sua tarefa.

Transformer — arquitetura de rede baseada em atenção, usada em modelos de linguagem.

Underfitting (subajuste) — quando o modelo é simples demais para captar o padrão.

Validação aninhada — validação em duas camadas, uma para escolher e outra para estimar o desempenho de quem escolhe.

Validação temporal — validação que treina no passado e testa no futuro, sem embaralhar.

Variância — sensibilidade do modelo ou da estimativa às variações dos dados.

Vetorização — transformação de textos em vetores numéricos.

Warm start — partida a quente; começar a busca por configurações que funcionaram em problemas parecidos.

XGBoost — biblioteca de boosting de árvores.

Apêndice C — Guia rápido

C.1 Qual método de busca usar?

Use esta tabela como ponto de partida. Ela resume o que aprendemos; o ideal é sempre **comparar com busca aleatória**.

Situação	Opção sugerida	Capítulo
Poucos hiperparâmetros (até 2) e treino barato	Busca em grade	4
Vários hiperparâmetros, orçamento limitado	Busca aleatória (linha de base forte)	4
Treino caro, poucas tentativas possíveis	Otimização bayesiana (TPE, SMAC, Optuna)	5
Treino com muitas épocas ou muitos dados	Successive Halving, Hyperband ou BOHB (multifidelidade)	6
Espaço grande, discreto, com estrutura de pipeline	Evolução (algoritmos genéticos, TPOT)	7
Muitos atributos, poucos exemplos	Seleção de atributos dentro do pipeline	8
Já se resolveram problemas parecidos	Meta-aprendizado e warm start	9
Máxima precisão e há tempo e memória	Ensembles (média, stacking, seleção de Caruana)	9
Quer um modelo pronto rapidamente	Ferramenta de AutoML (FLAML, Auto-sklearn, AutoGluon, H2O)	10
Rede neural com arquitetura a definir	NAS, com espaço pequeno e linha de base aleatória	11
Dados em série temporal	Validação temporal, atrasos, diferenciação	12

C.2 Receitas de código

Os trechos abaixo reúnem os padrões usados no livro. Eles vêm dos capítulos e das soluções, mas **as interfaces das bibliotecas mudam**: confira a documentação da sua versão.

Pipeline sem vazamento

```python
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_val_score

pipe = make_pipeline(StandardScaler(), LogisticRegression(max_iter=2000))
print(cross_val_score(pipe, X, y, cv=5).mean())      # o scaler é ajustado só no treino de cada parte
```

Busca aleatória

```python
from scipy.stats import loguniform
from sklearn.model_selection import RandomizedSearchCV
from sklearn.svm import SVC

busca = RandomizedSearchCV(SVC(), {"C": loguniform(1e-2, 1e3), "gamma": loguniform(1e-4, 1e0)},
                           n_iter=30, cv=5, random_state=0).fit(Xtr, ytr)
```

```
print(busca.best_params_, busca.best_score_)
```

Validação aninhada (estimar o ganho da busca)

```python
from sklearn.model_selection import cross_val_score, GridSearchCV
interna = GridSearchCV(SVC(), {"C": [0.1, 1, 10], "gamma": [0.001, 0.01, 0.1]}, cv=3)
print(cross_val_score(interna, X, y, cv=5).mean())    # a busca roda dentro de cada parte externa
```

Successive Halving no scikit-learn

```python
from sklearn.experimental import enable_halving_search_cv  # noqa
from sklearn.model_selection import HalvingRandomSearchCV
h = HalvingRandomSearchCV(modelo, distribuicoes, factor=3, random_state=0).fit(Xtr, ytr)
```

Otimização bayesiana (TPE) com Optuna

```python
import optuna
def objetivo(trial):
    C = trial.suggest_float("C", 1e-3, 1e3, log=True)
    return cross_val_score(SVC(C=C, gamma="scale"), Xtr, ytr, cv=3).mean()
estudo = optuna.create_study(direction="maximize", sampler=optuna.samplers.TPESampler(seed=0))
estudo.optimize(objetivo, n_trials=30)
estudo.enqueue_trial({"C": 1.0})                    # warm start: configuração conhecida na fila
(antes do optimize)
```

FLAML

```python
from flaml import AutoML
a = AutoML()
a.fit(Xtr, ytr, task="classification", time_budget=60, metric="accuracy", eval_method="cv", n_splits=3,
seed=0)
print(a.best_estimator, a.best_config, a.best_loss)
```

Ensemble por votação suave

```python
from sklearn.ensemble import VotingClassifier
vot = VotingClassifier([("lr", lr), ("rf", rf), ("svm", svm)], voting="soft").fit(Xtr, ytr)
```

Séries temporais: validação sem olhar o futuro

```python
from sklearn.model_selection import TimeSeriesSplit
for tr, va in TimeSeriesSplit(5, test_size=12).split(X):
    ...   # treina em X[tr] (passado) e valida em X[va] (futuro)
```

Explicação e auditoria

```python
from sklearn.inspection import permutation_importance
pi = permutation_importance(modelo, Xte, yte, n_repeats=10, random_state=0)
# por grupo: taxa de seleção = pred[grupo == k].mean(); TPR = pred[(grupo == k) & (y_verdade == 1)].mean()
```

Salvar e recarregar o pipeline

```python
import joblib
joblib.dump(pipe, "modelo.joblib")
pipe = joblib.load("modelo.joblib")                # nunca carregue arquivos de origem desconhecida
```

C.3 Fórmulas e métricas em palavras

Conceito	Em palavras
Acurácia	acertos ÷ total de casos
Taxa de verdadeiros positivos (TPR)	entre os casos positivos reais, a proporção que o modelo acertou
MAE	média dos erros em valor absoluto
Erro padrão de uma proporção	raiz de p(1 − p) ÷ n; indica a incerteza da acurácia
Fronteira de Pareto	soluções sem outra melhor em todos os objetivos ao mesmo tempo
Correlação de Spearman	concordância entre as ordens de duas listas (1 = mesma ordem)
Posto médio	média das posições de um método em várias bases (1 = melhor)
PSI	soma, por faixa, de (proporção nova − proporção antiga) × log(proporção nova ÷ antiga)
Taxa de seleção	proporção de aprovados em um grupo

C.4 Lista de verificação de um projeto

Antes de buscar

- O problema e a métrica refletem o custo real dos erros?
- O **teste final** foi separado antes de qualquer decisão?
- Há risco de **vazamento** (pré-processamento fora da validação, dados repetidos, futuro no passado)?
- Existe uma **linha de base** simples?

Durante a busca

- O espaço é razoável, e o orçamento, definido?
- A validação respeita a estrutura dos dados (tempo, grupos)?
- Use **validação aninhada** (ou um teste intocado) para medir o ganho da busca.
- Registre semente, versões e orçamento.

Depois da busca

- Compare no teste uma única vez, com **incerteza** (erro padrão, várias divisões).
- O ensemble ou o AutoML ganharam da linha de base **além do ruído**?
- Explique o modelo (global e local) e meça o desempenho **por grupo**.
- Meça a **latência** e o custo; compare com o orçamento.

Em produção

- Salve o pipeline inteiro e a ficha do modelo.
- Monitore dados (KS, PSI) e desempenho (com rótulos recentes).
- Retreine com **campeão e desafiante**, com margem.
- Mantenha pessoas no circuito e canal de contestação.

C.5 As dez armadilhas mais comuns

1. **Escolher pelo teste** (o teste vira validação).
2. **Vazamento** por pré-processar antes de dividir.
3. **Efeito do vencedor:** nota do melhor candidato é otimista.
4. **Comparar sem linha de base** ou com orçamentos desiguais.
5. **Tirar conclusões de uma única divisão** ou de poucos exemplos de teste.
6. **Embaralhar uma série temporal** na validação.
7. **Confiar em explicações como justificativas.**
8. **Achar que remover o atributo sensível elimina o viés** (proxies e rótulos enviesados).
9. **Ignorar o custo** (tempo, energia, latência).
10. **Esquecer o monitoramento** depois da implantação.

C.6 Mapa dos capítulos

Parte	Capítulos	Ideia central
I — Fundamentos	1 a 3	O problema CASH, a avaliação honesta e os pipelines sem vazamento
II — Otimização de Hiperparâmetros	4 a 7	Grade e aleatória; bayesiana; multifidelidade; evolução
III — Automação do Pipeline	8 a 10	Atributos; meta-aprendizado e ensembles; ferramentas
IV — Redes Neurais e Novos Domínios	11 e 12	NAS; séries temporais, texto e imagens
V — Responsabilidade e Prática	13 e 14	Explicabilidade, ética e limites; MLOps e projeto integrador

Apêndice D — Soluções dos exercícios práticos

Cada solução traz o código, a **saída obtida** na elaboração do livro e um comentário sobre o que observar. Há várias maneiras corretas de resolver um exercício: o importante é que o seu resultado seja coerente e que você consiga explicá-lo. Pequenas diferenças numéricas podem ocorrer com outras versões das bibliotecas. Os gráficos aparecem quando você executa o código.

Capítulo 1 — Bem-vindo ao AutoML

Exercício 1 — Por que precisamos de uma linha de base

PYTHON

```python
from sklearn.datasets import load_breast_cancer
from sklearn.dummy import DummyClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import cross_val_score

X, y = load_breast_cancer(return_X_y=True)
bobo = DummyClassifier(strategy="most_frequent")
logis = make_pipeline(StandardScaler(), LogisticRegression(max_iter=2000))

for nome, m in [("Linha de base (bobo)", bobo), ("Regressão logística", logis)]:
    notas = cross_val_score(m, X, y, cv=5)
    print(f"{nome:22s} acurácia = {notas.mean():.3f}")
print("Proporção da classe mais comum:", round(y.mean(), 3))
```

SAÍDA

```
Linha de base (bobo)   acurácia = 0.627
Regressão logística    acurácia = 0.981
Proporção da classe mais comum: 0.627
```

EXEMPLO **O que observar**

O modelo "bobo" acerta cerca de 63% só por responder sempre a classe majoritária. Qualquer modelo útil precisa superar essa linha de base com folga. Todo projeto de AutoML deve começar por uma linha de base.

Exercício 2 — Ampliando o espaço de busca

PYTHON

```python
import numpy as np
from sklearn.datasets import load_wine
from sklearn.model_selection import cross_val_score
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.svm import SVC

X, y = load_wine(return_X_y=True)
rng = np.random.default_rng(0)

def sortear():
    alg = rng.choice(["arvore", "knn", "logistica", "floresta", "svm", "gboost"])
    if alg == "arvore":
        hp = {"max_depth": int(rng.integers(1, 15)), "min_samples_leaf": int(rng.integers(1, 20))}
```

```
        m = DecisionTreeClassifier(random_state=0, **hp)
    elif alg == "knn":
        hp = {"n_neighbors": int(rng.integers(1, 30))}
        m = make_pipeline(StandardScaler(), KNeighborsClassifier(**hp))
    elif alg == "logistica":
        hp = {"C": float(10 ** rng.uniform(-3, 3))}
        m = make_pipeline(StandardScaler(), LogisticRegression(max_iter=3000, **hp))
    elif alg == "floresta":
        hp = {"n_estimators": int(rng.integers(10, 150)), "max_depth": int(rng.integers(2, 15))}
        m = RandomForestClassifier(random_state=0, **hp)
    elif alg == "svm":
        hp = {"C": float(10 ** rng.uniform(-2, 3)), "gamma": float(10 ** rng.uniform(-4, 0))}
        m = make_pipeline(StandardScaler(), SVC(**hp))
    else:
        hp = {"n_estimators": int(rng.integers(20, 150)), "learning_rate": float(10 ** rng.uniform(-2,
0))}
        m = GradientBoostingClassifier(random_state=0, **hp)
    return str(alg), hp, m

resultados = []
for _ in range(60):
    alg, hp, m = sortear()
    resultados.append((float(cross_val_score(m, X, y, cv=5).mean()), alg, hp))

resultados.sort(key=lambda t: -t[0])
for nota, alg, hp in resultados[:3]:
    print(f"{nota:.3f}  {alg:10s}", {k: round(v, 3) if isinstance(v, float) else v for k, v in
hp.items()})
print("Algoritmos entre os 10 melhores:", sorted({a for _, a, _ in resultados[:10]}))
```

SAÍDA

```
0.989  logistica  {'C': 0.758}
0.983  logistica  {'C': 1.419}
0.983  logistica  {'C': 0.822}
Algoritmos entre os 10 melhores: ['floresta', 'logistica', 'svm']
```

EXEMPLO **O que observar**

O topo é ocupado por regressões logísticas com valores de C parecidos, mas floresta aleatória e SVM também aparecem entre os 10 melhores, com diferenças na terceira casa decimal. Em um conjunto pequeno como este, essas diferenças estão dentro do ruído da validação cruzada. Não faz sentido "coroar" o primeiro lugar com excesso de confiança.

Exercício 3 — O efeito da semente

PYTHON

```
import numpy as np
from sklearn.datasets import load_wine
from sklearn.model_selection import cross_val_score
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
```

```python
from sklearn.svm import SVC

X, y = load_wine(return_X_y=True)

def melhor_da_busca(semente, n=30):
    rng = np.random.default_rng(semente)
    melhor = 0.0
    for _ in range(n):
        alg = rng.choice(["knn", "logistica", "floresta", "svm"])
        if alg == "knn":
            m = make_pipeline(StandardScaler(), KNeighborsClassifier(int(rng.integers(1, 30))))
        elif alg == "logistica":
            m = make_pipeline(StandardScaler(), LogisticRegression(C=10 ** rng.uniform(-3, 3),
max_iter=3000))
        elif alg == "floresta":
            m = RandomForestClassifier(int(rng.integers(10, 150)), max_depth=int(rng.integers(2, 15)),
random_state=0)
        else:
            m = make_pipeline(StandardScaler(), SVC(C=10 ** rng.uniform(-2, 3), gamma=10 **
rng.uniform(-4, 0)))
        melhor = max(melhor, cross_val_score(m, X, y, cv=5).mean())
    return float(melhor)

notas = [melhor_da_busca(s) for s in range(5)]
print("Melhores notas por semente:", [round(n, 3) for n in notas])
print(f"Média = {np.mean(notas):.3f}  Desvio padrão = {np.std(notas):.3f}")
```

SAÍDA

```
Melhores notas por semente: [0.983, 0.983, 0.989, 0.989, 0.994]
Média = 0.988  Desvio padrão = 0.004
```

EXEMPLO O que observar

A melhor nota varia de uma semente para outra. Relatar só uma execução esconde essa variação. O correto é informar média e dispersão de várias execuções.

Exercício 4 — Contando o tamanho do espaço de busca

PYTHON

```python
from math import prod

espaco = {"max_depth": 10, "min_samples_leaf": 8, "criterion": 2}
n = prod(espaco.values())
print("Combinações:", n, "| tempo total:", n * 3, "s =", round(n * 3 / 60, 1), "min")

espaco["max_features"] = 10
n = prod(espaco.values())
print("Com 4 hiperparâmetros:", n, "| tempo total:", n * 3, "s =", round(n * 3 / 3600, 2), "h")
```

SAÍDA

```
Combinações: 160 | tempo total: 480 s = 8.0 min
Com 4 hiperparâmetros: 1600 | tempo total: 4800 s = 1.33 h
```

EXEMPLO **O que observar**

O número de combinações cresce de forma multiplicativa. Acrescentar um único hiperparâmetro com 10 valores multiplicou o custo por 10. Em espaços reais, com dezenas de botões, testar tudo é impossível. Daí a necessidade de estratégias inteligentes de busca.

Capítulo 2 — Aprendizado de máquina em um capítulo

Exercício 1 — Curvas de aprendizado

PYTHON

```python
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import learning_curve
from sklearn.tree import DecisionTreeClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

X, y = load_breast_cancer(return_X_y=True)
tamanhos = [0.1, 0.3, 0.6, 1.0]
modelos = {"Árvore sem limite": DecisionTreeClassifier(random_state=0),
           "Regressão logística": make_pipeline(StandardScaler(), LogisticRegression(max_iter=2000))}
for nome, m in modelos.items():
    n, tr, va = learning_curve(m, X, y, train_sizes=tamanhos, cv=5, scoring="accuracy")
    print(nome)
    for ni, a, b in zip(n, tr.mean(1), va.mean(1)):
        print(f"  treino com {ni:3d} exemplos: treino={a:.3f}  validação={b:.3f}  distância={a-b:.3f}")
```

SAÍDA

```
Árvore sem limite
  treino com  45 exemplos: treino=1.000  validação=0.817  distância=0.183
  treino com 136 exemplos: treino=1.000  validação=0.882  distância=0.118
  treino com 273 exemplos: treino=1.000  validação=0.919  distância=0.081
  treino com 455 exemplos: treino=1.000  validação=0.917  distância=0.083
Regressão logística
  treino com  45 exemplos: treino=1.000  validação=0.793  distância=0.207
  treino com 136 exemplos: treino=0.987  validação=0.944  distância=0.043
  treino com 273 exemplos: treino=0.984  validação=0.967  distância=0.017
  treino com 455 exemplos: treino=0.989  validação=0.981  distância=0.009
```

EXEMPLO **O que observar**

Com todos os dados, a árvore sem limite acerta 100% no treino, mas obtém 0,917 na validação: distância de 0,083, sinal de sobreajuste. A regressão logística chega a 0,981 na validação, com distância de apenas 0,009. Curvas de aprendizado mostram também se mais dados ajudariam: se a validação ainda sobe com mais exemplos, coletar dados pode valer mais que ajustar modelos.

Exercício 2 — Métricas em dados desbalanceados

PYTHON

```python
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.dummy import DummyClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
```

```python
from sklearn.metrics import accuracy_score, precision_score, recall_score, roc_auc_score

X, y = load_breast_cancer(return_X_y=True)
rng = np.random.default_rng(0)
mal = rng.choice(np.where(y == 0)[0], 15, replace=False)    # 15 malignos (classe rara)
ben = np.where(y == 1)[0]                                    # todos os benignos
idx = np.concatenate([mal, ben]); X, y = X[idx], y[idx]
y = 1 - y                                                    # agora 1 = maligno (positivo, raro)
print("Proporção de casos raros:", round(y.mean(), 3))

Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, stratify=y, random_state=0)
modelos = {
    "Sempre benigno": DummyClassifier(strategy="most_frequent"),
    "Logística comum": make_pipeline(StandardScaler(), LogisticRegression(max_iter=2000)),
    "Logística balanceada": make_pipeline(StandardScaler(), LogisticRegression(max_iter=2000,
class_weight="balanced")),
}
for nome, m in modelos.items():
    m.fit(Xtr, ytr); p = m.predict(Xte); pr = m.predict_proba(Xte)[:, 1]
    print(f"{nome:21s} acur={accuracy_score(yte,p):.3f}
prec={precision_score(yte,p,zero_division=0):.3f} "
          f"revoc={recall_score(yte,p):.3f} AUC={roc_auc_score(yte,pr):.3f}")
```

SAÍDA

```
Proporção de casos raros: 0.04
Sempre benigno        acur=0.955 prec=0.000 revoc=0.000 AUC=0.500
Logística comum       acur=0.973 prec=1.000 revoc=0.400 AUC=0.996
Logística balanceada  acur=0.973 prec=0.750 revoc=0.600 AUC=0.991
```

EXEMPLO **O que observar**

A acurácia engana: o modelo "bobo" tem 0,955 com revocação zero. As duas regressões logísticas têm a mesma acurácia (0,973), mas encontram parcelas diferentes dos casos raros: revocação de 0,40 (comum) contra 0,60 (balanceada), esta última à custa de menor precisão (0,75 contra 1,00). Na triagem, a revocação da classe rara importa mais. Com apenas 5 casos raros no teste, as métricas variam muito: tire conclusões com cautela.

Exercício 3 — Uma divisão ou várias?

PYTHON

```python
import numpy as np
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.neighbors import KNeighborsClassifier
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

X, y = load_wine(return_X_y=True)
modelo = make_pipeline(StandardScaler(), KNeighborsClassifier())

notas = []
for semente in range(30):
    Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, random_state=semente)
    notas.append(modelo.fit(Xtr, ytr).score(Xte, yte))
print(f"30 divisões: menor={min(notas):.3f}  maior={max(notas):.3f}  média={np.mean(notas):.3f}")
```

```
cv = cross_val_score(modelo, X, y, cv=5)
print(f"Validação cruzada (5 partes): média={cv.mean():.3f}  desvio={cv.std():.3f}")
```

SAÍDA

```
30 divisões: menor=0.907  maior=1.000  média=0.967
Validação cruzada (5 partes): média=0.949  desvio=0.038
```

EXEMPLO **O que observar**

Em 30 divisões, a acurácia foi de 0,907 a 1,000 (média 0,967). Quem relata apenas a melhor divisão está sendo otimista, e quem relata a pior, pessimista. A validação cruzada resume a variação em média e desvio padrão, e é mais honesta.

Exercício 4 — Reproduzindo o efeito do vencedor

PYTHON

```python
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

X, y = load_breast_cancer(return_X_y=True)
rng = np.random.default_rng(0)
y = rng.permutation(y)                                  # destrói a relação entre X e y
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.4, stratify=y, random_state=0)

melhor, melhor_m = 0.0, None
for _ in range(100):
    if rng.random() < 0.5:
        m = make_pipeline(StandardScaler(), KNeighborsClassifier(int(rng.integers(1, 40))))
    else:
        m = DecisionTreeClassifier(max_depth=int(rng.integers(1, 12)), random_state=0)
    nota = cross_val_score(m, Xtr, ytr, cv=5).mean()
    if nota > melhor:
        melhor, melhor_m = nota, m
melhor_m.fit(Xtr, ytr)
print(f"Melhor nota de validação (100 candidatos): {melhor:.3f}")
print(f"Nota no teste separado                   : {melhor_m.score(Xte, yte):.3f}")
print(f"Proporção da classe majoritária (linha de base): {max(y.mean(), 1-y.mean()):.3f}")
```

SAÍDA

```
Melhor nota de validação (100 candidatos): 0.642
Nota no teste separado                   : 0.614
Proporção da classe majoritária (linha de base): 0.627
```

EXEMPLO O que observar

Sem sinal nos dados, o teste (0,614) fica próximo da linha de base (0,627). A validação do vencedor (0,642) fica acima do teste. Aqui o efeito é modesto, porque o conjunto de treino é relativamente grande (341 exemplos); no exemplo do capítulo, com apenas 120 exemplos, ele foi bem maior. Quanto menor a amostra e maior o número de candidatos, maior o otimismo. O teste final intocado é a defesa.

Capítulo 3 — Pipelines e pré-processamento

Exercício 1 — Qual imputação escolher?

PYTHON

```python
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.impute import SimpleImputer, KNNImputer
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_val_score, StratifiedKFold

X, y = load_breast_cancer(return_X_y=True)
rng = np.random.default_rng(0)
X = X.copy(); X[rng.random(X.shape) < 0.15] = np.nan
print("Fração de ausentes:", round(np.isnan(X).mean(), 3))

imputadores = {
    "média": SimpleImputer(strategy="mean"),
    "mediana": SimpleImputer(strategy="median"),
    "k-vizinhos": KNNImputer(n_neighbors=5),
    "mediana + indicador": SimpleImputer(strategy="median", add_indicator=True),
}
cv = StratifiedKFold(5, shuffle=True, random_state=0)
for nome, imp in imputadores.items():
    pipe = make_pipeline(imp, StandardScaler(), LogisticRegression(max_iter=3000))
    n = cross_val_score(pipe, X, y, cv=cv)
    print(f"{nome:20s} acurácia = {n.mean():.3f} (±{n.std():.3f})")
```

SAÍDA

```
Fração de ausentes: 0.15
média                acurácia = 0.967 (±0.020)
mediana              acurácia = 0.970 (±0.013)
k-vizinhos           acurácia = 0.965 (±0.018)
mediana + indicador  acurácia = 0.949 (±0.020)
```

EXEMPLO O que observar

Média (0,967), mediana (0,970) e k-vizinhos (0,965) ficam praticamente empatados: as diferenças são menores que o desvio padrão (cerca de 0,02). Por isso, não vale "coroar" um método por centésimos. A versão com indicador de ausência ficou um pouco abaixo (0,949): como os ausentes foram sorteados ao acaso, o fato de faltar não traz informação, e as colunas extras de indicadores só acrescentam ruído.

Exercício 2 — Ordinal ou one-hot?

PYTHON

```python
import numpy as np
from sklearn.preprocessing import OrdinalEncoder, OneHotEncoder
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline
from sklearn.model_selection import cross_val_score
```

```
rng = np.random.default_rng(0)
cidades = np.array(list("ABCDE"))
taxa = {"A": 0.10, "B": 0.80, "C": 0.20, "D": 0.90, "E": 0.15}
X = rng.choice(cidades, size=(2000, 1))
y = np.array([rng.random() < taxa[c] for c in X[:, 0]]).astype(int)

ordinal = make_pipeline(OrdinalEncoder(), LogisticRegression())
onehot = make_pipeline(OneHotEncoder(), LogisticRegression())
print("Acurácia com ordinal:", round(cross_val_score(ordinal, X, y, cv=5).mean(), 3))
print("Acurácia com one-hot:", round(cross_val_score(onehot, X, y, cv=5).mean(), 3))
```

SAÍDA

```
Acurácia com ordinal: 0.571
Acurácia com one-hot: 0.839
```

EXEMPLO O que observar

A taxa de sucesso não cresce nem decresce em linha reta com as letras (alterna entre baixa e alta). Um modelo linear sobre códigos 0 a 4 só consegue uma relação monótona, e por isso vai mal. O one-hot dá a cada cidade seu próprio peso e captura o padrão.

Exercício 3 — Vazamento: a nota honesta

PYTHON

```python
import numpy as np
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.feature_selection import SelectKBest, f_classif
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

X0, y = load_wine(return_X_y=True)
res = []
for semente in range(5):
    rng = np.random.default_rng(semente)
    X = np.hstack([X0, rng.normal(size=(len(y), 1000))])
    Xtr, Xte, ytr, yte = train_test_split(X, y, train_size=60, stratify=y, random_state=semente)

    # (a) seleção FORA do pipeline (vazamento)
    sel = SelectKBest(f_classif, k=15).fit(Xtr, ytr)
    modelo = make_pipeline(StandardScaler(), LogisticRegression(max_iter=3000))
    val_a = cross_val_score(modelo, sel.transform(Xtr), ytr, cv=5).mean()
    teste_a = modelo.fit(sel.transform(Xtr), ytr).score(sel.transform(Xte), yte)

    # (b) seleção DENTRO do pipeline
    pipe = make_pipeline(SelectKBest(f_classif, k=15), StandardScaler(),
LogisticRegression(max_iter=3000))
    val_b = cross_val_score(pipe, Xtr, ytr, cv=5).mean()
    teste_b = pipe.fit(Xtr, ytr).score(Xte, yte)
    res.append((val_a, teste_a, val_b, teste_b))

va, ta, vb, tb = np.mean(res, axis=0)
```

```
print(f"(a) fora do pipeline  : validação = {va:.3f} | teste final = {ta:.3f} | otimismo = {va - ta:
+.3f}")
print(f"(b) dentro do pipeline: validação = {vb:.3f} | teste final = {tb:.3f} | otimismo = {vb - tb:
+.3f}")
```

SAÍDA

```
(a) fora do pipeline  : validação = 0.983 | teste final = 0.951 | otimismo = +0.032
(b) dentro do pipeline: validação = 0.917 | teste final = 0.951 | otimismo = -0.034
```

EXEMPLO O que observar

Em (a), a validação fica acima do teste final: a seleção "viu" os rótulos das partes de validação, e a nota é otimista. Em (b), a estimativa fica abaixo do teste, o que é esperado e honesto: com apenas 60 exemplos, cada rodada da validação interna treina com menos dados do que o modelo final. O exercício mostra duas coisas: vazamento infla a nota, e a estimativa correta pode ser levemente pessimista. Com mais dados ou atributos mais informativos, o efeito do vazamento diminui, mas o princípio permanece.

Exercício 4 — Buscando dentro do pipeline

PYTHON

```
import pandas as pd
from sklearn.datasets import load_wine
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler, MinMaxScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import GridSearchCV

X, y = load_wine(return_X_y=True)
pipe = Pipeline([("escala", StandardScaler()), ("knn", KNeighborsClassifier())])
grade = {"escala": [StandardScaler(), MinMaxScaler(), "passthrough"], "knn__n_neighbors": [1, 3, 5, 9,
15]}
busca = GridSearchCV(pipe, grade, cv=5).fit(X, y)

res = pd.DataFrame(busca.cv_results_)[["param_escala", "param_knn__n_neighbors", "mean_test_score"]]
res["param_escala"] = res["param_escala"].astype(str).str.replace("()", "", regex=False)
print(res.sort_values("mean_test_score", ascending=False).head(3).to_string(index=False))
print("Pior combinação:", round(res["mean_test_score"].min(), 3), "| melhor:",
round(res["mean_test_score"].max(), 3))
```

SAÍDA

```
  param_escala  param_knn__n_neighbors  mean_test_score
StandardScaler                       9         0.966349
  MinMaxScaler                       9         0.960794
StandardScaler                      15         0.955238
Pior combinação: 0.691 | melhor: 0.966
```

EXEMPLO O que observar

A escala importou muito: a pior combinação (sem escala) chega a 0,691, enquanto a melhor (padronização, 9 vizinhos) chega a 0,966. A diferença entre `StandardScaler` (0,966) e `MinMaxScaler` (0,961) é pequena. Esta busca é um mini-AutoML sobre o pré-processamento.

Capítulo 4 — Busca em grade e aleatória

Exercício 1 — Sua primeira busca em grade

PYTHON

```python
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_wine
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import GridSearchCV, StratifiedKFold

X, y = load_wine(return_X_y=True)
cv = StratifiedKFold(5, shuffle=True, random_state=0)
pipe = make_pipeline(StandardScaler(), KNeighborsClassifier())
grade = {"kneighborsclassifier__n_neighbors": list(range(1, 26, 2)),
         "kneighborsclassifier__weights": ["uniform", "distance"]}
busca = GridSearchCV(pipe, grade, cv=cv).fit(X, y)

n = len(busca.cv_results_["params"])
print("Combinações:", n, "| treinos:", n * 5)
print("Melhor:", busca.best_params_, "| acurácia =", round(busca.best_score_, 3))

res = pd.DataFrame(busca.cv_results_)
res["k"] = res["param_kneighborsclassifier__n_neighbors"].astype(int)
res["peso"] = res["param_kneighborsclassifier__weights"].astype(str)
for peso, g in res.groupby("peso"):
    plt.plot(g["k"], g["mean_test_score"], "o-", label=peso)
    print(f"peso {peso:8s}: melhor = {g['mean_test_score'].max():.3f} | pior =
{g['mean_test_score'].min():.3f}")
plt.xlabel("n_neighbors"); plt.ylabel("acurácia (validação cruzada)"); plt.legend(); plt.show()
```

SAÍDA

```
Combinações: 26 | treinos: 130
Melhor: {'kneighborsclassifier__n_neighbors': 21, 'kneighborsclassifier__weights': 'uniform'} |
acurácia = 0.983
peso distance: melhor = 0.983 | pior = 0.955
peso uniform : melhor = 0.983 | pior = 0.955
```

EXEMPLO O que observar

Os dois tipos de peso chegam à mesma nota máxima (0,983) e à mesma mínima (0,955). A melhor combinação (21 vizinhos, peso uniforme) não é muito diferente de várias vizinhas. Nesse tipo de paisagem, em que há uma faixa de valores com notas parecidas, o rigor excessivo em "achar o melhor valor" é desperdício.

Exercício 2 — Grade ou aleatória? Com a floresta, empatou

PYTHON

```python
import numpy as np
from scipy.stats import randint, uniform
```

```python
from sklearn.datasets import load_breast_cancer
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import GridSearchCV, RandomizedSearchCV, StratifiedKFold

X, y = load_breast_cancer(return_X_y=True)
cv = StratifiedKFold(5, shuffle=True, random_state=0)

grade = {"n_estimators": [20, 60], "max_depth": [3, None], "max_features": ["sqrt", 0.5]}
g = GridSearchCV(RandomForestClassifier(random_state=0), grade, cv=cv).fit(X, y)
print(f"Grade ({len(g.cv_results_['params'])} combinações): melhor = {g.best_score_:.4f}")

espaco = {"n_estimators": randint(10, 80), "max_depth": randint(2, 12), "max_features": uniform(0.05,
0.9)}
notas = [RandomizedSearchCV(RandomForestClassifier(random_state=0), espaco, n_iter=8, cv=cv,
random_state=s).fit(X, y).best_score_
         for s in range(4)]
print("Aleatória (8 tentativas) x 4 sementes:", np.round(notas, 4), "| média =", round(np.mean(notas),
4))
```

SAÍDA

```
Grade (8 combinações): melhor = 0.9649
Aleatória (8 tentativas) x 4 sementes: [0.9614 0.9666 0.9614 0.9631] | média = 0.9631
```

EXEMPLO **O que observar**

Aqui a grade (0,965) e a aleatória (média 0,963) praticamente empataram. Isso é coerente com a teoria: a vantagem da aleatória aparece quando poucos hiperparâmetros importam *e* a grade é pobre em valores distintos. Com 3 botões e uma grade pequena, não há vantagem clara. A regra "aleatória é melhor" é uma tendência, e não uma lei.

Exercício 3 — Escala logarítmica faz diferença

PYTHON

```python
import numpy as np
from scipy.stats import uniform, loguniform
from sklearn.datasets import load_wine
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import RandomizedSearchCV, StratifiedKFold

X, y = load_wine(return_X_y=True)
cv = StratifiedKFold(5, shuffle=True, random_state=0)
pipe = make_pipeline(StandardScaler(), LogisticRegression(max_iter=5000))

def buscar(dist, semente):
    r = RandomizedSearchCV(pipe, {"logisticregression__C": dist}, n_iter=10, cv=cv,
random_state=semente).fit(X, y)
    return r.best_score_

for nome, dist in [("uniforme", uniform(0.001, 1000)), ("logarítmico", loguniform(1e-3, 1e3))]:
    notas = [buscar(dist, s) for s in range(10)]
    print(f"{nome:12s} média = {np.mean(notas):.4f} | pior = {np.min(notas):.4f}")
```

SAÍDA

```
uniforme     média = 0.9837 | pior = 0.9832
logarítmico  média = 0.9904 | pior = 0.9832
```

EXEMPLO O que observar

O espaço logarítmico tem média maior (0,990 contra 0,984). No espaço uniforme, quase todos os sorteios caem em valores de C acima de 1; o espaço logarítmico também testa valores pequenos e, em média, encontrou notas melhores. O pior resultado de cada espaço é igual (0,983), então o ganho está em acertar com maior frequência, e não em evitar o pior caso.

Exercício 4 — Medindo o otimismo da busca

PYTHON

```python
import numpy as np
from scipy.stats import loguniform
from sklearn.datasets import load_wine
from sklearn.svm import SVC
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import RandomizedSearchCV, cross_val_score, StratifiedKFold,
train_test_split

X0, y0 = load_wine(return_X_y=True)
espaco = {"svc__C": loguniform(1e-2, 1e3), "svc__gamma": loguniform(1e-5, 1)}
dif = []
for s in range(6):
    X, _, y, _ = train_test_split(X0, y0, train_size=60, stratify=y0, random_state=s)
    pipe = make_pipeline(StandardScaler(), SVC())
    interno = StratifiedKFold(5, shuffle=True, random_state=s)
    externo = StratifiedKFold(5, shuffle=True, random_state=50 + s)
    a = RandomizedSearchCV(pipe, espaco, n_iter=40, cv=interno, random_state=s).fit(X, y).best_score_
    b = cross_val_score(RandomizedSearchCV(pipe, espaco, n_iter=40, cv=interno, random_state=s), X, y,
cv=externo).mean()
    dif.append(a - b)
    print(f"semente {s}: busca = {a:.3f} | aninhada = {b:.3f} | otimismo = {a - b:+.3f}")
print(f"Otimismo médio: {np.mean(dif):+.3f}")
```

SAÍDA

```
semente 0: busca = 0.983 | aninhada = 0.967 | otimismo = +0.017
semente 1: busca = 1.000 | aninhada = 0.983 | otimismo = +0.017
semente 2: busca = 0.967 | aninhada = 0.917 | otimismo = +0.050
semente 3: busca = 0.983 | aninhada = 0.967 | otimismo = +0.017
semente 4: busca = 0.983 | aninhada = 0.933 | otimismo = +0.050
semente 5: busca = 0.967 | aninhada = 0.983 | otimismo = -0.017
Otimismo médio: +0.022
```

EXEMPLO O que observar

O otimismo médio é positivo (cerca de +0,02), mas varia bastante de semente para semente e pode até ser negativo em uma delas: com poucos dados, as estimativas são ruidosas. A conclusão confiável vem da média de várias repetições, e não de um único experimento.

Capítulo 5 — Otimização bayesiana

Exercício 1 — Sua própria otimização bayesiana

PYTHON

```python
import numpy as np
from scipy.stats import norm
from sklearn.gaussian_process import GaussianProcessRegressor
from sklearn.gaussian_process.kernels import Matern, WhiteKernel, ConstantKernel as Ck

g = lambda x: np.sin(3 * x) * np.exp(-0.25 * x) + 0.5 * np.sin(9 * x)
malha = np.linspace(0, 6, 601)[:, None]
maximo = g(malha[:, 0]).max()
print("Máximo verdadeiro (malha fina):", round(maximo, 3))

def bo(semente, n_total=12, n_inicial=3):
    rng = np.random.default_rng(semente)
    X = list(rng.uniform(0, 6, n_inicial)); Y = [g(x) for x in X]
    for _ in range(n_total - n_inicial):
        k = Ck(1.0) * Matern(length_scale=0.5, nu=2.5) + WhiteKernel(1e-6, (1e-10, 1e-3))
        gp = GaussianProcessRegressor(k, normalize_y=True, n_restarts_optimizer=2,
random_state=semente).fit(np.array(X)[:, None], Y)
        mu, sd = gp.predict(malha, return_std=True); sd = np.maximum(sd, 1e-9)
        z = (mu - max(Y) - 0.01) / sd; ei = (mu - max(Y) - 0.01) * norm.cdf(z) + sd * norm.pdf(z)
        x = float(malha[np.argmax(ei), 0]); X.append(x); Y.append(g(x))
    return np.maximum.accumulate(Y)

def aleatoria(semente, n_total=12):
    return np.maximum.accumulate(g(np.random.default_rng(500 + semente).uniform(0, 6, n_total)))

B = np.array([bo(s) for s in range(20)]); A = np.array([aleatoria(s) for s in range(20)])
print("avaliações | melhor médio (BO x aleatória) | % a menos de 0,05 do máximo")
for k in (4, 8, 12):
    print(f"{k:10d} | {B[:, k-1].mean():.3f} x {A[:, k-1].mean():.3f}            | "
          f"{100*(B[:, k-1] > maximo-0.05).mean():3.0f}% x {100*(A[:, k-1] > maximo-0.05).mean():3.0f}
%")
```

SAÍDA

```
Máximo verdadeiro (malha fina): 1.04
avaliações | melhor médio (BO x aleatória) | % a menos de 0,05 do máximo
         4 | 0.487 x 0.640            |   5% x   5%
         8 | 0.720 x 0.747            |  30% x  10%
        12 | 0.903 x 0.801            |  55% x  10%
```

EXEMPLO **O que observar**

Na execução de referência, a aleatória estava à frente com 4 avaliações (0,640 contra 0,487) e com 8 (0,747 contra 0,720), mas, com 12, a BO passou à frente (0,903 contra 0,801) e ficou a menos de 0,05 do máximo em 55% das repetições, contra 10% da aleatória. É o mesmo padrão do capítulo: a BO precisa de aquecimento e rende mais quando o orçamento permite que o modelo substituto aprenda.

Exercício 2 — Optuna em uma floresta aleatória

PYTHON

```python
import numpy as np
import optuna
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import cross_val_score, StratifiedKFold

optuna.logging.set_verbosity(optuna.logging.WARNING)
X, y = load_breast_cancer(return_X_y=True)
cv = StratifiedKFold(5, shuffle=True, random_state=0)

def objetivo(trial):
    modelo = RandomForestClassifier(
        n_estimators=trial.suggest_int("n_estimators", 20, 150),
        max_depth=trial.suggest_int("max_depth", 2, 15),
        min_samples_leaf=trial.suggest_int("min_samples_leaf", 1, 10),
        max_features=trial.suggest_float("max_features", 0.1, 1.0),
        random_state=0, n_jobs=-1)
    return cross_val_score(modelo, X, y, cv=cv).mean()

estudo = optuna.create_study(direction="maximize", sampler=optuna.samplers.TPESampler(seed=0))
estudo.optimize(objetivo, n_trials=25)
print("Melhor nota:", round(estudo.best_value, 4))
print("Melhores valores:", {k: round(v, 3) if isinstance(v, float) else v for k, v in
estudo.best_params.items()})

notas = [t.value for t in estudo.trials]
plt.plot(range(1, 26), np.maximum.accumulate(notas), "o-"); plt.xlabel("tentativa"); plt.ylabel("melhor
nota até agora"); plt.show()

imp = optuna.importance.get_param_importances(estudo)
print("Importância dos hiperparâmetros:")
for nome, v in imp.items():
    print(f"  {nome:17s} {v:.2f}")
print("Variação das notas: de", round(min(notas), 3), "a", round(max(notas), 3))
```

SAÍDA

```
Melhor nota: 0.9702
Melhores valores: {'n_estimators': 77, 'max_depth': 15, 'min_samples_leaf': 1, 'max_features': 0.335}
Importância dos hiperparâmetros:
  min_samples_leaf  0.60
  max_features      0.20
  max_depth         0.19
  n_estimators      0.01
Variação das notas: de 0.939 a 0.97
```

EXEMPLO **O que observar**

As notas variam pouco (de 0,939 a 0,970), pois a floresta é robusta a hiperparâmetros neste conjunto fácil. O Optuna atribuiu a maior importância ao tamanho mínimo da folha (0,60), seguido de `max_features` e `max_depth` (cerca de 0,20) e quase nenhuma ao número de árvores (0,01). Como a análise se baseia em diferenças pequenas e em apenas 25 tentativas, leia esses valores com cautela: em problemas mais difíceis, a variação das notas é maior e a análise é mais informativa.

Exercício 3 — Reproduzindo e corrigindo a falha do CASH

PYTHON

```python
import optuna
from sklearn.datasets import load_digits
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import cross_val_score, StratifiedKFold

optuna.logging.set_verbosity(optuna.logging.WARNING)
X, y = load_digits(return_X_y=True); X = X / 16.0
cv = StratifiedKFold(3, shuffle=True, random_state=0)

def rodar(gamma_min, gamma_max, semente=0, n=30):
    def objetivo(trial):
        alg = trial.suggest_categorical("algoritmo", ["svm", "floresta", "logistica"])
        if alg == "svm":
            m = SVC(C=trial.suggest_float("C", 1e-3, 1e4, log=True), gamma=trial.suggest_float("gamma",
gamma_min, gamma_max, log=True))
        elif alg == "floresta":
            m = RandomForestClassifier(trial.suggest_int("arvores", 20, 150),
max_depth=trial.suggest_int("prof", 3, 20), random_state=0, n_jobs=-1)
        else:
            m = make_pipeline(StandardScaler(), LogisticRegression(C=trial.suggest_float("lrC", 1e-4,
1e3, log=True), max_iter=2000))
        return cross_val_score(m, X, y, cv=cv).mean()
    est = optuna.create_study(direction="maximize", sampler=optuna.samplers.TPESampler(seed=semente))
    est.optimize(objetivo, n_trials=n)
    resumo = {}
    for t in est.trials:
        r = resumo.setdefault(t.params["algoritmo"], [0, 0.0]); r[0] += 1; r[1] = max(r[1], t.value)
    return est.best_params["algoritmo"], est.best_value, resumo

for rotulo, (a, b) in [("gamma 1e-8 a 10 ", (1e-8, 10)), ("gamma 1e-5 a 1  ", (1e-5, 1))]:
    alg, nota, resumo = rodar(a, b)
    print(f"{rotulo}: vencedor = {alg} ({nota:.3f})")
    for nome, (n, m) in resumo.items():
        print(f"    {nome:10s} tentativas = {n:2d} | melhor = {m:.3f}")
```

SAÍDA

```
gamma 1e-8 a 10 : vencedor = floresta (0.977)
    floresta   tentativas = 20 | melhor = 0.977
    logistica  tentativas =  5 | melhor = 0.971
    svm        tentativas =  5 | melhor = 0.299
gamma 1e-5 a 1  : vencedor = svm (0.981)
    floresta   tentativas = 10 | melhor = 0.975
    logistica  tentativas =  4 | melhor = 0.971
    svm        tentativas = 16 | melhor = 0.981
```

EXEMPLO O que observar

Com o espaço amplo, o SVM teve apenas 5 tentativas e sua melhor nota foi 0,299, enquanto a floresta venceu com 0,977. Ao restringir o *gamma* a uma faixa razoável (10^{-5} a 1), o SVM ganhou 16 tentativas, chegou a 0,981 e venceu. A lição: o conhecimento humano sobre faixas plausíveis de hiperparâmetros melhora muito a eficiência de uma busca automática. Espaços absurdamente amplos desperdiçam o orçamento.

Exercício 4 — Quanto explorar? O coeficiente do UCB

PYTHON

```python
import numpy as np
from sklearn.gaussian_process import GaussianProcessRegressor
from sklearn.gaussian_process.kernels import Matern, WhiteKernel, ConstantKernel as Ck

f = lambda x: 0.6 * np.exp(-((x - 3) / 1.2) ** 2) + 1.0 * np.exp(-((x - 8.2) / 0.35) ** 2)
candidatos = np.linspace(0, 10, 501)[:, None]

def bo_ucb(semente, kappa, n_total=15, n_inicial=3):
    rng = np.random.default_rng(semente)
    X = list(rng.uniform(0, 10, n_inicial)); Y = [f(x) for x in X]
    for _ in range(n_total - n_inicial):
        k = Ck(1.0) * Matern(length_scale=1.0, nu=2.5) + WhiteKernel(1e-6, (1e-10, 1e-3))
        gp = GaussianProcessRegressor(k, normalize_y=True, n_restarts_optimizer=2,
random_state=semente).fit(np.array(X)[:, None], Y)
        mu, sd = gp.predict(candidatos, return_std=True)
        x = float(candidatos[np.argmax(mu + kappa * sd), 0]); X.append(x); Y.append(f(x))
    return max(Y)

print("kappa | melhor médio | % que achou a agulha")
for kappa in (0.5, 2, 6):
    r = np.array([bo_ucb(s, kappa) for s in range(30)])
    print(f"{kappa:5} | {r.mean():12.3f} | {100*(r > 0.95).mean():5.0f}%")
```

SAÍDA

```
kappa | melhor médio | % que achou a agulha
  0.5 |        0.728 |    30%
    2 |        0.744 |    33%
    6 |        0.786 |    33%
```

EXEMPLO O que observar

Na execução de referência, as médias foram 0,728 (κ=0,5), 0,744 (κ=2) e 0,786 (κ=6), e a fração que achou a agulha ficou entre 30% e 33% nos três casos. Com 30 repetições, essas diferenças são pequenas e não permitem declarar um vencedor com segurança. O que se vê é a tendência esperada: κ maior explora mais, o que ajuda a encontrar a agulha estreita, mas pode atrapalhar em outros problemas. O ponto de equilíbrio depende do problema e do orçamento.

Capítulo 6 — Métodos multifidelidade

Exercício 1 — Fazendo a conta do Successive Halving

PYTHON

```python
def tabela_sh(n, r, eta):
    linhas, total = [], 0
    while True:
        custo = n * r
        linhas.append((n, r, custo)); total += custo
        if n <= 1: break
        n, r = max(1, n // eta), r * eta
    return linhas, total

for n, r, eta in [(64, 1, 4), (81, 1, 3)]:
    linhas, total = tabela_sh(n, r, eta)
    r_max = linhas[-1][1]
    print(f"n={n}, r={r}, η={eta}")
    for cand, rec, custo in linhas:
        print(f"   {cand:3d} candidatos x {rec:3d} épocas = {custo:4d}")
    print(f"   custo total = {total} | treinar todos até {r_max} épocas = {n * r_max} | economia = {100
* (1 - total / (n * r_max)):.0f}%")
```

SAÍDA

```
n=64, r=1, η=4
    64 candidatos x   1 épocas =   64
    16 candidatos x   4 épocas =   64
     4 candidatos x  16 épocas =   64
     1 candidatos x  64 épocas =   64
   custo total = 256 | treinar todos até 64 épocas = 4096 | economia = 94%
n=81, r=1, η=3
    81 candidatos x   1 épocas =   81
    27 candidatos x   3 épocas =   81
     9 candidatos x   9 épocas =   81
     3 candidatos x  27 épocas =   81
     1 candidatos x  81 épocas =   81
   custo total = 405 | treinar todos até 81 épocas = 6561 | economia = 94%
```

EXEMPLO **O que observar**

Para (64, 1, 4) o custo é 64+64+64+64 = 256 contra 4.096 (economia de 94%). Para (81, 1, 3) o custo é 405 contra 6.561 (economia de 94%). A economia vem de cada rodada custar quase o mesmo e do número de rodadas crescer só de forma logarítmica.

Exercício 2 — Halving aleatório na floresta

PYTHON

```python
import numpy as np
from scipy.stats import randint, uniform
from sklearn.experimental import enable_halving_search_cv
from sklearn.model_selection import RandomizedSearchCV, HalvingRandomSearchCV
from sklearn.datasets import load_digits
```

```
from sklearn.ensemble import RandomForestClassifier

X, y = load_digits(return_X_y=True)
espaco = {"max_depth": randint(3, 25), "max_features": uniform(0.05, 0.55), "min_samples_leaf":
randint(1, 8)}

aleat = RandomizedSearchCV(RandomForestClassifier(n_estimators=150, random_state=0, n_jobs=-1), espaco,
n_iter=30, cv=3, random_state=0).fit(X, y)
halv = HalvingRandomSearchCV(RandomForestClassifier(random_state=0, n_jobs=-1), espaco,
n_candidates=30, resource="n_estimators",
                             max_resources=150, min_resources=10, factor=3, cv=3,
random_state=0).fit(X, y)

custo_aleat = 30 * 150 * 3
custo_halv = sum(c * r * 3 for c, r in zip(halv.n_candidates_, halv.n_resources_))
print("Halving: candidatos por rodada", halv.n_candidates_, "| árvores por rodada", halv.n_resources_)
print(f"Aleatória : melhor = {aleat.best_score_:.3f} | custo = {custo_aleat:,} árvores treinadas")
print(f"Halving   : melhor = {halv.best_score_:.3f} | custo = {custo_halv:,} árvores treinadas")
print(f"Economia de custo: {100 * (1 - custo_halv / custo_aleat):.0f}%")
```

SAÍDA

```
Halving: candidatos por rodada [30, 10, 4] | árvores por rodada [10, 30, 90]
Aleatória : melhor = 0.942 | custo = 13,500 árvores treinadas
Halving   : melhor = 0.939 | custo = 2,880 árvores treinadas
Economia de custo: 79%
```

EXEMPLO **O que observar**

O halving gasta uma fração do custo, pois a maioria dos candidatos é avaliada com poucas árvores e descartada cedo. A nota final fica próxima da aleatória. Lembre que a nota do halving vem da última rodada (poucos candidatos, mais recursos) e que a escolha final entre candidatos muito parecidos pode variar com a semente.

Exercício 3 — Comparando podadores

PYTHON

```
import numpy as np
import optuna
from sklearn.datasets import load_digits
from sklearn.linear_model import SGDClassifier
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

optuna.logging.set_verbosity(optuna.logging.WARNING)
X, y = load_digits(return_X_y=True)
Xtr, Xva, ytr, yva = train_test_split(X, y, test_size=0.3, stratify=y, random_state=0)
esc = StandardScaler().fit(Xtr); Xtr, Xva = esc.transform(Xtr), esc.transform(Xva)
EPOCAS = 60

def objetivo(trial):
    m = SGDClassifier(loss=trial.suggest_categorical("loss", ["hinge", "log_loss", "modified_huber"]),
                      alpha=trial.suggest_float("alpha", 1e-8, 1e1, log=True),
eta0=trial.suggest_float("eta0", 1e-6, 1e1, log=True),
                      learning_rate="constant", random_state=0)
    for epoca in range(EPOCAS):
```

```
        m.partial_fit(Xtr, ytr, classes=np.arange(10))
        nota = m.score(Xva, yva)
        trial.report(nota, epoca)
        if trial.should_prune():
            raise optuna.TrialPruned()
    return nota

podadores = {"sem poda": optuna.pruners.NopPruner(),
             "MedianPruner": optuna.pruners.MedianPruner(n_startup_trials=5, n_warmup_steps=3),
             "HyperbandPruner": optuna.pruners.HyperbandPruner(min_resource=3, max_resource=EPOCAS,
reduction_factor=3)}
for nome, p in podadores.items():
    est = optuna.create_study(direction="maximize", sampler=optuna.samplers.TPESampler(seed=0),
pruner=p)
    est.optimize(objetivo, n_trials=40)
    podadas = sum(t.state == optuna.trial.TrialState.PRUNED for t in est.trials)
    epocas = sum(len(t.intermediate_values) for t in est.trials)
    print(f"{nome:16s} podadas = {podadas:2d}/40 | épocas = {epocas:4d} | melhor acurácia =
{est.best_value:.3f}")
```

SAÍDA

```
sem poda         podadas =  0/40 | épocas = 2400 | melhor acurácia = 0.972
MedianPruner     podadas = 20/40 | épocas = 1297 | melhor acurácia = 0.969
HyperbandPruner  podadas = 25/40 | épocas = 1198 | melhor acurácia = 0.972
```

EXEMPLO **O que observar**

Sem poda, foram 2.400 épocas. O `MedianPruner` podou 20 tentativas e usou 1.297 épocas; o `HyperbandPruner` podou 31 e usou 964 épocas, o menos de todos. A melhor acurácia foi 0,972 sem poda e 0,969 nos dois podadores: uma perda de três milésimos, que com uma única semente está dentro do ruído. Só conclusões repetidas em várias sementes são confiáveis.

Exercício 4 — O fator η importa

PYTHON

```
import math
import numpy as np
from sklearn.datasets import load_digits
from sklearn.linear_model import SGDClassifier
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

def carregar_dados(semente=0):
    X, y = load_digits(return_X_y=True)
    Xtr, Xva, ytr, yva = train_test_split(X, y, test_size=0.3, stratify=y, random_state=semente)
    e = StandardScaler().fit(Xtr); return e.transform(Xtr), e.transform(Xva), ytr, yva

class Candidato:
    def __init__(self, rng, dados):
        self.hp = {"alpha": 10 ** rng.uniform(-8, 1), "eta0": 10 ** rng.uniform(-6, 1), "loss":
str(rng.choice(["hinge", "log_loss", "modified_huber"]))}
        self.modelo = SGDClassifier(loss=self.hp["loss"], alpha=self.hp["alpha"],
learning_rate="constant", eta0=self.hp["eta0"], random_state=int(rng.integers(1e6)))
        self.dados, self.epocas, self.nota = dados, 0, 0.0
```

```
    def treinar_ate(self, epocas):
        Xtr, Xva, ytr, yva = self.dados
        while self.epocas < epocas:
            self.modelo.partial_fit(Xtr, ytr, classes=np.arange(10)); self.epocas += 1; self.nota =
self.modelo.score(Xva, yva)
        return self.nota

def sucessivo(candidatos, r_min, eta, r_max):
    r, gasto = r_min, 0
    while True:
        for c in candidatos:
            antes = c.epocas; c.treinar_ate(int(min(r, r_max))); gasto += c.epocas - antes     # nunca
passa de r_max
        if len(candidatos) <= 1 or r >= r_max: break
        candidatos = sorted(candidatos, key=lambda c: -c.nota)[:max(1, len(candidatos) // eta)]; r *=
eta
    return max(candidatos, key=lambda c: c.nota), gasto

dados = carregar_dados(0)
for eta in (2, 3, 5):
    gastos, notas = [], []
    for s in range(8):
        cands = [Candidato(np.random.default_rng(2000 + s * 1000 + i), dados) for i in range(81)]
        v, g = sucessivo(cands, 1, eta, 81); gastos.append(g); notas.append(v.nota)
    print(f"η = {eta}: gasto médio = {np.mean(gastos):6.0f} épocas | acurácia média =
{np.mean(notas):.3f} (±{np.std(notas):.3f})")
```

SAÍDA

```
η = 2: gasto médio =    305 épocas | acurácia média = 0.964 (±0.003)
η = 3: gasto médio =    297 épocas | acurácia média = 0.965 (±0.004)
η = 5: gasto médio =    261 épocas | acurácia média = 0.961 (±0.004)
```

EXEMPLO O que observar

O gasto foi parecido nos três casos (305, 297 e 261 épocas), porque, ao retomar o treino de onde parou, cada rodada só paga as épocas novas. O η = 5, com cortes mais agressivos, gastou menos (261) e teve a menor acurácia média (0,961, contra 0,964 para η = 2 e 0,965 para η = 3). Mas essas diferenças de acurácia são pequenas, do tamanho do desvio padrão entre sementes (cerca de 0,004), e não permitem concluir com segurança. O que fica é a ideia: η maior economiza um pouco mais, ao risco de eliminar bons candidatos mais cedo.

Capítulo 7 — Métodos evolutivos e programação genética

Exercício 1 — Seleção por torneio: quem é escolhido?

PYTHON

```python
import numpy as np

rng = np.random.default_rng(0)
aptidoes = np.arange(1, 21)                    # 1 = pior, 20 = melhor
for k in (2, 3, 5):
    escolhidos = np.array([max(rng.choice(20, k, replace=False), key=lambda i: aptidoes[i]) for _ in
range(20000)])
    freq = np.bincount(escolhidos, minlength=20) / 20000
    print(f"k={k}: melhor escolhido em {100*freq[19]:4.1f}% | pior escolhido em {100*freq[0]:4.1f}% |
aptidão média dos escolhidos = {aptidoes[escolhidos].mean():.2f}")
print("Se a escolha fosse totalmente aleatória: 5,0% para cada indivíduo e aptidão média 10,50")
```

SAÍDA

```
k=2: melhor escolhido em 10.2% | pior escolhido em  0.0% | aptidão média dos escolhidos = 14.01
k=3: melhor escolhido em 14.8% | pior escolhido em  0.0% | aptidão média dos escolhidos = 15.74
k=5: melhor escolhido em 24.8% | pior escolhido em  0.0% | aptidão média dos escolhidos = 17.49
Se a escolha fosse totalmente aleatória: 5,0% para cada indivíduo e aptidão média 10,50
```

EXEMPLO **O que observar**

Com k = 2, o melhor é escolhido em 10,2% das vezes (o dobro do acaso, que seria 5%); com k = 3, em 14,8%; com k = 5, em 24,8%. O pior indivíduo nunca é escolhido em nenhum dos casos (em qualquer torneio ele perde para os outros competidores). A aptidão média dos escolhidos sobe de 10,5 (acaso) para 14,0, 15,7 e 17,5. Torneios maiores aumentam a pressão de seleção: a busca fica mais gananciosa (aproveita mais, explora menos) e perde diversidade mais rápido. Torneios pequenos mantêm mais diversidade.

Exercício 2 — Algoritmo genético para os hiperparâmetros de um SVM

PYTHON

```python
import numpy as np
from sklearn.datasets import load_digits
from sklearn.svm import SVC
from sklearn.model_selection import cross_val_score, StratifiedKFold

X, y = load_digits(return_X_y=True); X = X / 16.0
cv = StratifiedKFold(3, shuffle=True, random_state=0)
limites = np.array([[-2, 3], [-5, 0]])

def nota(ind):
    return cross_val_score(SVC(C=10 ** ind[0], gamma=10 ** ind[1]), X, y, cv=cv).mean()

def ga(semente, pop=8, geracoes=5):
    rng = np.random.default_rng(semente)
    P = rng.uniform(limites[:, 0], limites[:, 1], (pop, 2)); aval = 0; melhor = 0
    for g in range(geracoes + 1):
        f = np.array([nota(i) for i in P]); aval += pop; melhor = max(melhor, f.max())
        if g == geracoes: break
```

```
        nova = [P[f.argmax()].copy()]                                   # elitismo de 1
        while len(nova) < pop:
            a, b = (P[max(rng.choice(pop, 3, replace=False), key=lambda i: f[i])] for _ in range(2))
            w = rng.uniform(0, 1); filho = w * a + (1 - w) * b          # cruzamento por média
ponderada
            filho = filho + rng.normal(0, 0.3, 2)                       # mutação gaussiana
            nova.append(np.clip(filho, limites[:, 0], limites[:, 1]))
        P = np.array(nova)
    return melhor, aval

def aleatoria(semente, n):
    rng = np.random.default_rng(100 + semente)
    return max(nota(rng.uniform(limites[:, 0], limites[:, 1])) for _ in range(n))

res = []
for s in range(5):
    m, n = ga(s); res.append((m, aleatoria(s, n)))
r = np.array(res)
print("semente | genético | aleatória")
for s, (a, b) in enumerate(r): print(f"{s:7d} | {a:8.3f} | {b:9.3f}")
print(f"média   | {r[:,0].mean():8.3f} | {r[:,1].mean():9.3f}   (avaliações por método: {n})")
```

SAÍDA

```
semente | genético | aleatória
      0 |    0.991 |     0.991
      1 |    0.991 |     0.991
      2 |    0.991 |     0.991
      3 |    0.991 |     0.991
      4 |    0.991 |     0.991
média   |    0.991 |     0.991   (avaliações por método: 48)
```

EXEMPLO **O que observar**

Em todas as 5 sementes, os dois métodos terminaram com a mesma acurácia (0,991): empate completo. Em um problema de apenas dois botões e com uma região boa larga, a busca aleatória já acerta com facilidade, e o GA não tem vantagem a mostrar com 48 avaliações. O GA tende a ajudar mais em espaços maiores, estruturados e com regiões boas estreitas.

Exercício 3 — Fronteira de Pareto: acurácia × esparsidade

PYTHON

```
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import cross_val_score, StratifiedKFold

X, y = load_breast_cancer(return_X_y=True)
cv = StratifiedKFold(5, shuffle=True, random_state=0)
pontos = []
for C in np.logspace(-3, 2, 15):
    modelo = make_pipeline(StandardScaler(), LogisticRegression(penalty="l1", solver="liblinear", C=C))
    acc = cross_val_score(modelo, X, y, cv=cv).mean()
```

```
    nz = int((modelo.fit(X, y)[-1].coef_ != 0).sum())
    pontos.append((nz, acc, C))

domina = lambda a, b: a[0] <= b[0] and a[1] >= b[1] and (a[0] < b[0] or a[1] > b[1])
fronteira = sorted({(nz, round(acc, 4), round(C, 4)) for nz, acc, C in pontos if not any(domina(q, (nz,
acc)) for q in pontos)})
print("coeficientes não nulos | acurácia | C")
for nz, acc, C in fronteira:
    print(f"{nz:22d} | {acc:8.3f} | {C}")
print("Acurácia máxima:", round(max(p[1] for p in pontos), 3))
```

SAÍDA

```
coeficientes não nulos | acurácia | C
                     0 |    0.373 | 0.001
                     0 |    0.373 | 0.0023
                     4 |    0.953 | 0.0268
                     7 |    0.967 | 0.0611
                     9 |    0.974 | 0.1389
                    12 |    0.979 | 0.3162
                    16 |    0.979 | 0.7197
Acurácia máxima: 0.979
```

EXEMPLO **O que observar**

Com 0 coeficientes, o modelo é constante (acurácia de 0,373, que é a fração de uma das classes). Com apenas 4 coeficientes não nulos, a acurácia já é 0,953; com 9, 0,975; com 12, 0,979; e o máximo (0,981) exige 16. Ou seja, 9 atributos entregam cerca de 0,6 ponto percentual a menos que o máximo, com pouco mais da metade dos atributos. Decidir quantos atributos "valem a pena" é uma decisão humana que a fronteira torna explícita.

Exercício 4 — Mutação e elitismo em um GA simples

PYTHON

```
import numpy as np

f = lambda x: 0.6 * np.exp(-((x - 3) / 1.2) ** 2) + 1.0 * np.exp(-((x - 8.2) / 0.35) ** 2)

def ga(semente, sigma, elitismo, pop=10, geracoes=8):
    rng = np.random.default_rng(semente)
    P = rng.uniform(0, 10, pop); melhor = 0.0
    for g in range(geracoes + 1):
        aptidao = f(P); melhor = max(melhor, aptidao.max())
        if g == geracoes: break
        nova = [P[aptidao.argmax()]] if elitismo else []
        while len(nova) < pop:
            a, b = (P[max(rng.choice(pop, 3, replace=False), key=lambda i: aptidao[i])] for _ in
range(2))
            w = rng.uniform(); filho = w * a + (1 - w) * b + rng.normal(0, sigma) if sigma > 0 else w *
a + (1 - w) * b
            nova.append(float(np.clip(filho, 0, 10)))
        P = np.array(nova)
    return melhor

print("mutação σ | elitismo | melhor médio | % que achou a agulha")
for sigma, elit in [(0, False), (0, True), (0.5, False), (0.5, True)]:
```

```
    r = np.array([ga(s, sigma, elit) for s in range(40)])
    print(f"{sigma:9} | {str(elit):8s} | {r.mean():12.3f} | {100 * (r > 0.95).mean():5.0f}%")
```

SAÍDA

```
mutação σ | elitismo | melhor médio | % que achou a agulha
        0 | False    |        0.777 |    35%
        0 | True     |        0.785 |    40%
      0.5 | False    |        0.802 |    35%
      0.5 | True     |        0.839 |    60%
```

EXEMPLO O que observar

Na execução de referência, a média do melhor valor foi 0,777 (sem mutação, sem elitismo), 0,785 (sem mutação, com elitismo), 0,802 (com mutação, sem elitismo) e 0,839 (com mutação e elitismo), e a fração que achou a agulha foi de 35%, 40%, 35% e 60%. A combinação de mutação com elitismo foi a mais eficaz. Sem mutação, o cruzamento por média só gera pontos entre os pais, de modo que a população se contrai e perde a capacidade de explorar; o elitismo protege o melhor já encontrado, mas sozinho não cria novidade. Com 40 sementes, as diferenças menores (entre as três primeiras configurações) são pequenas e não conclusivas.

Capítulo 8 — Engenharia e seleção automática de atributos

Exercício 1 — Quando o produto de dois atributos é a resposta

PYTHON

```python
import numpy as np
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import cross_val_score, KFold

rng = np.random.default_rng(0)
x1, x2 = rng.normal(size=1000), rng.normal(size=1000)
y = x1 * x2 + rng.normal(0, 0.3, 1000)

cv = KFold(5, shuffle=True, random_state=0)
a = cross_val_score(LinearRegression(), np.c_[x1, x2], y, cv=cv, scoring="r2").mean()
b = cross_val_score(LinearRegression(), np.c_[x1, x2, x1 * x2], y, cv=cv, scoring="r2").mean()
print(f"(a) só x1 e x2      : R² = {a:.3f}")
print(f"(b) com x1·x2       : R² = {b:.3f}")
```

SAÍDA

```
(a) só x1 e x2      : R² = -0.032
(b) com x1·x2       : R² = 0.915
```

EXEMPLO **O que observar**

Em (a), o modelo linear não consegue representar uma interação multiplicativa e obtém um R^2 próximo de zero. Em (b), o atributo certo torna o problema linear e o R^2 chega perto de 0,9. Atenção: os dados foram simulados com essa relação; em dados reais, não sabemos de antemão qual transformação funcionará, e é por isso que a avaliação cuidadosa é essencial.

Exercício 2 — Quantos atributos manter?

PYTHON

```python
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.feature_selection import SelectKBest, f_classif
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import cross_val_score, StratifiedKFold

X0, y = load_breast_cancer(return_X_y=True)
rng = np.random.default_rng(0)
X = np.hstack([X0, rng.normal(size=(len(y), 200))])                # 30 reais + 200 de ruído
cv = StratifiedKFold(5, shuffle=True, random_state=0)

print("k    | acurácia")
for k in (2, 5, 10, 20, 30, 50, 100, 230):
    pipe = make_pipeline(StandardScaler(), SelectKBest(f_classif, k=k),
LogisticRegression(max_iter=3000))
```

```
    print(f"{k:4d} | {cross_val_score(pipe, X, y, cv=cv).mean():.3f}")
sem = make_pipeline(StandardScaler(), LogisticRegression(max_iter=3000))
print("sem seleção (230 atributos):", round(cross_val_score(sem, X, y, cv=cv).mean(), 3))
```

SAÍDA

```
k    | acurácia
   2 | 0.935
   5 | 0.947
  10 | 0.960
  20 | 0.984
  30 | 0.970
  50 | 0.946
 100 | 0.935
 230 | 0.921
sem seleção (230 atributos): 0.921
```

EXEMPLO O que observar

A acurácia sobe de 0,935 (k = 2) até 0,984 (k = 20) e depois cai: 0,970 (k = 30), 0,946 (k = 50), 0,935 (k = 100) e 0,921 sem seleção (230 atributos). Com k pequeno, o modelo perde informação útil; com k grande, o ruído volta a entrar. O melhor valor fica em uma faixa intermediária, abaixo do número de atributos realmente informativos (30): é a curva em "U invertido", típica, que mostra por que k deve ser tratado como hiperparâmetro.

Exercício 3 — RFECV: deixar a validação cruzada escolher

PYTHON

```python
import numpy as np
from sklearn.datasets import load_wine
from sklearn.feature_selection import RFECV
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import cross_val_score, StratifiedKFold

Xw, y = load_wine(return_X_y=True)
rng = np.random.default_rng(0)
X = np.hstack([Xw, rng.normal(size=(len(y), 100))])                # 13 reais + 100 de ruído

def novo_pipeline():
    sel = RFECV(LogisticRegression(max_iter=3000), step=5, cv=StratifiedKFold(5, shuffle=True,
random_state=1))
    return make_pipeline(StandardScaler(), sel, LogisticRegression(max_iter=3000))

pipe = novo_pipeline().fit(X, y)
suporte = pipe[1].support_
print(f"RFECV escolheu {suporte.sum()} atributos, dos quais {suporte[:13].sum()} estão entre os 13
reais")
externa = cross_val_score(novo_pipeline(), X, y, cv=StratifiedKFold(5, shuffle=True,
random_state=0)).mean()
print(f"Acurácia da validação externa do pipeline completo: {externa:.3f}")
```

SAÍDA

```
RFECV escolheu 8 atributos, dos quais 8 estão entre os 13 reais
```

```
Acurácia da validação externa do pipeline completo: 0.989
```

EXEMPLO O que observar

O RFECV escolheu 8 atributos, todos entre os 13 reais: nenhum ruído foi mantido. A validação externa do pipeline completo foi de 0,989. Essa validação externa é essencial: a nota interna do RFECV foi usada para escolher o número de atributos e é, portanto, otimista, enquanto a validação externa avalia o processo inteiro com dados que a seleção não viu.

Exercício 4 — Mais agregados, mais ganho?

PYTHON

```python
import numpy as np
import pandas as pd
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.model_selection import cross_val_score, StratifiedKFold

rng = np.random.default_rng(0)
n = 800
clientes = pd.DataFrame({"id": range(n), "idade": rng.integers(18, 75, n), "cidade": rng.choice(["A",
"B", "C"], n)})
n_compras = rng.poisson(8, n) + 1
media = rng.uniform(30, 120, n)
variab = rng.uniform(0.1, 1.2, n)
linhas = []
for i in range(n):
    valores = media[i] * np.exp(rng.normal(0, variab[i], n_compras[i]))
    linhas += [(i, v) for v in valores]
compras = pd.DataFrame(linhas, columns=["id_cliente", "valor"])

g = compras.groupby("id_cliente")["valor"]
agg = g.agg(media_valor="mean", max_valor="max", std_valor="std", n_compras="count")
agg["min_valor"] = g.min(); agg["mediana_valor"] = g.median()
agg["acima_100"] = compras.assign(a=compras["valor"] > 100).groupby("id_cliente")["a"].sum()
agg = agg.fillna(0).reset_index().rename(columns={"id_cliente": "id"})
dados = clientes.merge(agg, on="id")
dados["cv_valor"] = dados["std_valor"] / dados["media_valor"]

risco = 2.0 * dados["cv_valor"] - 0.02 * dados["n_compras"]
y = (rng.random(n) < 1 / (1 + np.exp(-3 * (risco - risco.median())))).astype(int)

base = pd.get_dummies(dados[["idade", "cidade"]]).astype(float)
col_a = ["media_valor", "max_valor", "std_valor", "n_compras", "cv_valor"]
col_b = col_a + ["min_valor", "mediana_valor", "acima_100"]
cv = StratifiedKFold(5, shuffle=True, random_state=0)
for nome, cols in [("(a) agregados do capítulo", col_a), ("(b) mais mínimo, mediana e acima de 100",
col_b)]:
    auc = cross_val_score(GradientBoostingClassifier(random_state=0), pd.concat([base, dados[cols]],
axis=1), y, cv=cv, scoring="roc_auc").mean()
    print(f"{nome:42s} AUC = {auc:.3f}")
```

SAÍDA

```
(a) agregados do capítulo                  AUC = 0.857
(b) mais mínimo, mediana e acima de 100    AUC = 0.853
```

EXEMPLO **O que observar**

O AUC foi 0,857 em (a) e 0,853 em (b): os atributos extras não ajudaram (a pequena queda está dentro do ruído da validação cruzada). O padrão simulado depende da variabilidade relativa e do número de compras, que já estavam em (a); mínimo, mediana e contagem acima de 100 são redundantes ou pouco informativos. Acrescentar atributos só compensa quando eles trazem informação nova.

Capítulo 9 — Meta-aprendizado, warm start e ensembles

Exercício 1 — O retrato de um conjunto de dados

PYTHON

```python
import numpy as np
from scipy.stats import entropy
from sklearn.datasets import load_wine

def meta(X, y):
    n, d = X.shape; c = len(np.unique(y)); p = np.bincount(y) / n
    return {"n": n, "d": d, "n/d": round(n / d, 2), "classes": c, "entropia": round(float(entropy(p) /
np.log(c)), 3)}

X, y = load_wine(return_X_y=True)
rng = np.random.default_rng(0)
print("(a) original             :", meta(X, y))
print("(b) com 100 colunas ruído:", meta(np.hstack([X, rng.normal(size=(len(y), 100))]), y))
idx = rng.choice(len(y), 60, replace=False)
print("(c) amostra de 60        :", meta(X[idx], y[idx]))
```

SAÍDA

```
(a) original             : {'n': 178, 'd': 13, 'n/d': 13.69, 'classes': 3, 'entropia': 0.989}
(b) com 100 colunas ruído: {'n': 178, 'd': 113, 'n/d': 1.58, 'classes': 3, 'entropia': 0.989}
(c) amostra de 60        : {'n': 60, 'd': 13, 'n/d': 4.62, 'classes': 3, 'entropia': 0.993}
```

EXEMPLO **O que observar**

Em (b), só mudam d e n/d: o ruído altera a dimensão, mas não a distribuição das classes. Em (c), mudam n, n/d e possivelmente a entropia (a amostra pode desbalancear as classes). O número de classes só muda se uma classe desaparecer. Cada meta-atributo captura um aspecto: tamanho, dimensionalidade, balanceamento. Nenhum deles diz nada sobre quão "difícil" é separar as classes: para isso, servem as medidas de complexidade e os landmarkers.

Exercício 2 — Votação dura ou suave?

PYTHON

```python
from sklearn.datasets import make_classification
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier, VotingClassifier
from sklearn.model_selection import cross_val_score, StratifiedKFold

X, y = make_classification(n_samples=2000, n_features=20, n_informative=8, n_redundant=4, n_classes=3,
                           n_clusters_per_class=3, class_sep=0.9, flip_y=0.08, random_state=1)
base = [("svm", SVC(C=10, gamma=0.02, probability=True, random_state=0)), ("knn",
KNeighborsClassifier(5, weights="distance")),
        ("lr", LogisticRegression(C=1, max_iter=2000)), ("rf", RandomForestClassifier(100,
random_state=0, n_jobs=-1))]
cv = StratifiedKFold(5, shuffle=True, random_state=0)
for tipo in ("hard", "soft"):
```

```
    print(f"votação {tipo:4s}: acurácia = {cross_val_score(VotingClassifier(base, voting=tipo), X, y,
cv=cv).mean():.3f}")
```

SAÍDA

```
votação hard: acurácia = 0.747
votação soft: acurácia = 0.759
```

EXEMPLO **O que observar**

Compare os dois números. A votação suave usa mais informação (a confiança de cada modelo) do que a dura (apenas o voto), o que costuma lhe dar vantagem, sobretudo com poucas classes e modelos bem calibrados. Mas a diferença pode ser pequena, e há situações em que a dura se sai tão bem quanto a suave.

Exercício 3 — Diversidade importa

PYTHON

```
import itertools
import numpy as np
from sklearn.datasets import make_classification
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier, VotingClassifier
from sklearn.model_selection import train_test_split

X, y = make_classification(n_samples=2000, n_features=20, n_informative=8, n_redundant=4, n_classes=3,
                           n_clusters_per_class=3, class_sep=0.9, flip_y=0.08, random_state=1)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.25, stratify=y, random_state=0)

def discordancia(modelos):
    prev = [m.fit(Xtr, ytr).predict(Xte) for m in modelos]
    return np.mean([np.mean(a != b) for a, b in itertools.combinations(prev, 2)])

floresta = lambda s: RandomForestClassifier(100, random_state=s, n_jobs=-1)
tres_florestas = [("rf%d" % s, floresta(s)) for s in (1, 2, 3)]
quatro_tipos = [("svm", SVC(C=10, gamma=0.02, probability=True, random_state=0)), ("knn",
KNeighborsClassifier(5, weights="distance")),
                ("lr", LogisticRegression(C=1, max_iter=2000)), ("rf", floresta(0))]

print(f"(a) uma floresta               : teste = {floresta(0).fit(Xtr, ytr).score(Xte, yte):.3f}")
print(f"(b) três florestas (sementes)  : teste = {VotingClassifier(tres_florestas,
voting='soft').fit(Xtr, ytr).score(Xte, yte):.3f} | discordância média = {discordancia([m for _, m in
tres_florestas]):.3f}")
print(f"(c) quatro tipos de modelos    : teste = {VotingClassifier(quatro_tipos,
voting='soft').fit(Xtr, ytr).score(Xte, yte):.3f} | discordância média = {discordancia([m for _, m in
quatro_tipos]):.3f}")
```

SAÍDA

```
(a) uma floresta               : teste = 0.704
(b) três florestas (sementes)  : teste = 0.702 | discordância média = 0.115
(c) quatro tipos de modelos    : teste = 0.742 | discordância média = 0.298
```

EXEMPLO O que observar

Três florestas com sementes diferentes discordam pouco entre si, então combiná-las traz ganho mínimo sobre uma só (cada floresta já é um ensemble). Já modelos de tipos diferentes discordam muito mais, e sua combinação tende a render mais. A diversidade é a matéria-prima dos ensembles: sem ela, combinar é quase o mesmo que repetir o mesmo modelo.

Exercício 4 — Warm start com o Optuna

PYTHON

```python
import numpy as np
import optuna
from sklearn.datasets import load_wine
from sklearn.svm import SVC
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import cross_val_score, StratifiedKFold

optuna.logging.set_verbosity(optuna.logging.WARNING)
X, y = load_wine(return_X_y=True); d = X.shape[1]
cv = StratifiedKFold(3, shuffle=True, random_state=0)
PORTFOLIO = [(1, 1), (10, 1), (10, 0.1)]                                    # (C, gamma relativo)

def objetivo(trial):
    C = trial.suggest_float("C", 1e-2, 1e3, log=True)
    g_rel = trial.suggest_float("gamma_rel", 1e-3, 1e1, log=True)
    return cross_val_score(make_pipeline(StandardScaler(), SVC(C=C, gamma=g_rel / d)), X, y,
cv=cv).mean()

def rodar(semente, quente):
    est = optuna.create_study(direction="maximize", sampler=optuna.samplers.TPESampler(seed=semente))
    if quente:
        for C, g in PORTFOLIO: est.enqueue_trial({"C": C, "gamma_rel": g})
    est.optimize(objetivo, n_trials=10)
    return np.maximum.accumulate([t.value for t in est.trials])

frio = np.array([rodar(s, False) for s in range(5)]); quente = np.array([rodar(s, True) for s in
range(5)])
print("tentativas | partida a frio | com warm start")
for k in (1, 3, 10):
    print(f"{k:10d} | {frio[:, k-1].mean():14.3f} | {quente[:, k-1].mean():.3f}")
```

SAÍDA

```
tentativas | partida a frio | com warm start
         1 |          0.865 | 0.983
         3 |          0.986 | 0.989
        10 |          0.989 | 0.989
```

EXEMPLO O que observar

O warm start ajuda sobretudo nas primeiras tentativas: com 1 e 3 tentativas, a versão com portfólio já parte de bons valores, enquanto a partida a frio ainda sorteia. Com 10 tentativas, a diferença diminui ou desaparece, pois a busca comum também chega a boas regiões. Em um conjunto fácil como o Wine, o teto de acurácia é alto, e as duas versões acabam próximas.

Capítulo 10 — Ferramentas de AutoML na prática

Exercício 1 — Primeiro contato com o FLAML

PYTHON

```python
import warnings; warnings.filterwarnings("ignore")
from flaml import AutoML
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split

X, y = load_wine(return_X_y=True)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.25, stratify=y, random_state=0)
a = AutoML()
a.fit(Xtr, ytr, task="classification", estimator_list=["lgbm", "rf"], time_budget=5, metric="accuracy",
      eval_method="cv", n_splits=3, seed=0, verbose=0)
print("Vencedor            :", a.best_estimator)
print("Erro de validação   : %.4f (acurácia %.4f)" % (a.best_loss, 1 - a.best_loss))
print("Acurácia no teste   : %.4f" % (a.predict(Xte) == yte).mean())
```

SAÍDA

```
Vencedor            : rf
Erro de validação   : 0.0375 (acurácia 0.9625)
Acurácia no teste   : 1.0000
```

EXEMPLO **O que observar**

O FLAML escolheu a floresta aleatória (`rf`), com erro de validação de 0,0375 (acurácia de validação 0,9625), e acertou 100% no teste. Essa diferença não é contradição: a validação cruzada usa partes menores do treino (133 exemplos, 3 partes), enquanto o teste tem apenas 45 exemplos e, por sorte, ficou fácil. Um teste de 45 exemplos tem muita variação: uma acurácia de 1,000 aqui **não** significa que o modelo seja perfeito. O algoritmo vencedor pode mudar com a semente e o tempo.

Exercício 2 — Tempo ou tentativas?

PYTHON

```python
import warnings; warnings.filterwarnings("ignore")
from flaml import AutoML
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split

X, y = load_wine(return_X_y=True)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.25, stratify=y, random_state=0)

def roda(**kw):
    a = AutoML()
    a.fit(Xtr, ytr, task="classification", metric="accuracy", eval_method="cv", n_splits=3, seed=0,
verbose=0,
          estimator_list=["lgbm", "rf", "extra_tree"], **kw)
    return len(a.config_history), a.best_estimator, round(a.best_loss, 4)

print("(a) por tempo (3 s)   :", roda(time_budget=3), "|", roda(time_budget=3))
print("(b) por tentativas (12):", roda(time_budget=60, max_iter=12), "|", roda(time_budget=60,
```

```
max_iter=12))
```

SAÍDA

```
(a) por tempo (3 s)   : (10, 'extra_tree', 0.0303) | (10, 'extra_tree', 0.0303)
(b) por tentativas (12): (5, 'lgbm', 0.0601) | (5, 'lgbm', 0.0601)
```

EXEMPLO **O que observar**

Nesta máquina, as duas execuções de cada caso deram o mesmo resultado: (a) por tempo, 10 melhorias registradas e `extra_tree` com erro 0,0303; (b) por tentativas, 5 melhorias e `lgbm` com erro 0,0601. A coincidência em (a) **não é garantida**: em outra máquina ou sob outra carga, 3 segundos podem comportar mais ou menos tentativas. Limitar por `max_iter` depende menos do relógio e, por isso, é a forma mais reprodutível. Note também que `config_history` registra as configurações que **melhoraram** o melhor resultado até então, e não todas as avaliadas; e que os resultados de (a) e (b) diferem simplesmente porque as buscas foram interrompidas em pontos diferentes (erro de 0,0303 contra 0,0601).

Exercício 3 — O AutoML bate a linha de base?

PYTHON

```python
import warnings; warnings.filterwarnings("ignore")
import numpy as np
from flaml import AutoML
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier

X, y = make_classification(1500, 30, n_informative=8, n_redundant=4, flip_y=0.10, random_state=0)
R = {"Reg. logística": [], "Floresta padrão": [], "FLAML (5 s)": []}
for s in range(5):
    Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.25, stratify=y, random_state=s)
    m1 = make_pipeline(StandardScaler(), LogisticRegression(max_iter=2000)).fit(Xtr, ytr)
    m2 = RandomForestClassifier(200, random_state=s, n_jobs=-1).fit(Xtr, ytr)
    a = AutoML(); a.fit(Xtr, ytr, task="classification", time_budget=5, metric="accuracy",
eval_method="cv", n_splits=3, seed=s, verbose=0)
    for nome, m in zip(R, (m1, m2, a)): R[nome].append((m.predict(Xte) == yte).mean())
for nome, v in R.items(): print(f"{nome:<16s} média = {np.mean(v):.3f}  desvio = {np.std(v):.3f}")
```

SAÍDA

```
Reg. logística   média = 0.846  desvio = 0.012
Floresta padrão  média = 0.875  desvio = 0.014
FLAML (5 s)      média = 0.873  desvio = 0.013
```

EXEMPLO **O que observar**

Médias nas 5 divisões: regressão logística 0,846, floresta padrão 0,875 e FLAML 0,873, com desvios de cerca de 0,012 a 0,014. O FLAML supera a regressão logística por uma margem (cerca de 0,027) maior que o dobro do desvio, mas **empata** com a floresta padrão (diferença de 0,002, muito menor que a variação entre divisões). Em outras palavras: neste problema, 5 segundos de AutoML não melhoraram uma floresta sem ajuste. Isso é um resultado legítimo e frequente; a linha de base serve justamente para revelá-lo.

Exercício 4 — Postos e Friedman à mão

PYTHON

```python
import warnings; warnings.filterwarnings("ignore")
import numpy as np
from scipy.stats import rankdata, friedmanchisquare
from sklearn.datasets import load_iris, load_wine, load_breast_cancer, load_digits, make_classification
from sklearn.model_selection import cross_val_score, StratifiedKFold
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.neighbors import KNeighborsClassifier

bases = {"Iris": load_iris(return_X_y=True), "Wine": load_wine(return_X_y=True),
         "Cancer": load_breast_cancer(return_X_y=True), "Digits": load_digits(return_X_y=True),
         "Ruidosa": make_classification(1500, 30, n_informative=8, n_redundant=4, flip_y=0.10,
random_state=0)}
modelos = {"Reg. logística": make_pipeline(StandardScaler(), LogisticRegression(max_iter=3000)),
           "Floresta": RandomForestClassifier(100, random_state=0, n_jobs=-1),
           "k-vizinhos": make_pipeline(StandardScaler(), KNeighborsClassifier(5))}
cv = StratifiedKFold(5, shuffle=True, random_state=0)
M = np.array([[cross_val_score(m, X, y, cv=cv).mean() for m in modelos.values()] for X, y in
bases.values()])
for b, lin in zip(bases, M): print(f"{b:<8s}", "  ".join(f"{v:.3f}" for v in lin))
P = np.array([rankdata(-l) for l in M])
print("Postos médios:", dict(zip(modelos, P.mean(0).round(2))))
est, p = friedmanchisquare(*M.T)
print(f"Friedman: estatística = {est:.2f}, valor-p = {p:.3f}")
```

SAÍDA

```
Iris     0.960  0.940  0.953
Wine     0.983  0.972  0.961
Cancer   0.979  0.965  0.965
Digits   0.969  0.973  0.977
Ruidosa  0.863  0.889  0.832
Postos médios: {'Reg. logística': np.float64(1.6), 'Floresta': np.float64(2.1), 'k-vizinhos':
np.float64(2.3)}
Friedman: estatística = 1.37, valor-p = 0.504
```

EXEMPLO **O que observar**

Os postos médios foram 1,6 (regressão logística), 2,1 (floresta) e 2,3 (k-vizinhos), e o teste de Friedman deu estatística 1,37 e valor-p 0,504: **não rejeita** a equivalência dos métodos. Isso é o esperado com apenas 5 bases, em que o poder do teste é baixo; a regressão logística liderou em três bases (Iris, Wine e Cancer), mas a floresta ganhou na base ruidosa e o k-vizinhos no Digits. Nenhum método dominou.

Capítulo 11 — Busca de arquitetura neural (NAS)

Exercício 1 — Contando arquiteturas

PYTHON

```python
import itertools

larguras, ativacoes, max_camadas = [16, 32, 64], ["relu", "tanh", "logistic"], 4
formula = len(ativacoes) * sum(len(larguras) ** L for L in range(1, max_camadas + 1))
enumerado = sum(1 for L in range(1, max_camadas + 1) for _ in itertools.product(larguras, repeat=L) for
_ in ativacoes)
print("Pela fórmula :", formula)
print("Enumerando   :", enumerado)
print("Tempo total  : %.1f horas (2 min por arquitetura)" % (formula * 2 / 60))

larguras2 = larguras + [128]
novo = len(ativacoes) * sum(len(larguras2) ** L for L in range(1, max_camadas + 1))
print("Com largura 128 também:", novo, f"(x{novo / formula:.1f})")
```

SAÍDA

```
Pela fórmula : 360
Enumerando   : 360
Tempo total  : 12.0 horas (2 min por arquitetura)
Com largura 128 também: 1020 (x2.8)
```

EXEMPLO **O que observar**

O espaço tem 3 × (3 + 9 + 27 + 81) = 360 arquiteturas (fórmula e enumeração coincidem). Avaliar todas levaria 12 horas, mesmo com um espaço tão pequeno. Acrescentar a largura 128 leva o espaço a 1.020 arquiteturas (2,8 vezes maior): o tamanho cresce **exponencialmente** com o número de camadas e depressa com cada opção nova. Por isso a estratégia de busca e a estimativa barata importam tanto.

Exercício 2 — Maior é sempre melhor?

PYTHON

```python
import warnings; warnings.filterwarnings("ignore")
import numpy as np
from scipy.stats import spearmanr
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPClassifier
from sklearn.preprocessing import StandardScaler

X, y = load_digits(return_X_y=True)
Xa, Xv, ya, yv = train_test_split(X, y, test_size=0.25, stratify=y, random_state=0)
sc = StandardScaler().fit(Xa); Xa, Xv = sc.transform(Xa), sc.transform(Xv)
r = np.random.default_rng(0)
res = []
for _ in range(40):
    L = int(r.integers(1, 4)); h = tuple(int(r.choice([8, 16, 32, 64, 128])) for _ in range(L)); ativ =
str(r.choice(["relu", "tanh"]))
    d = [64, *h, 10]; params = sum(i * o + o for i, o in zip(d[:-1], d[1:]))
```

```
    acc = MLPClassifier(h, activation=ativ, max_iter=40, random_state=0).fit(Xa, ya).score(Xv, yv)
    res.append((acc, params, h, ativ))
rho, _ = spearmanr([p for _, p, _, _ in res], [a for a, _, _, _ in res])
print("Spearman (parâmetros x acurácia): %.2f" % rho)
for acc, params, h, ativ in sorted(res, reverse=True)[:3]: print(f"  {acc:.3f}  {params:>6d} parâmetros
{h} {ativ}")
print("Menor rede entre as 10 melhores:", min(sorted(res, reverse=True)[:10], key=lambda t: t[1])[1],
"parâmetros")
```

SAÍDA

```
Spearman (parâmetros x acurácia): 0.73
  0.978   17226 parâmetros  (128, 64) tanh
  0.969   13770 parâmetros  (64, 128) relu
  0.969   13770 parâmetros  (64, 128) relu
Menor rede entre as 10 melhores: 4810 parâmetros
```

EXEMPLO **O que observar**

A correlação de Spearman entre parâmetros e acurácia foi de 0,73: **em geral**, redes maiores acertam mais, mas a relação está longe de perfeita. As três melhores redes têm de 13 mil a 17 mil parâmetros e, entre as dez melhores, a menor tem apenas 4.810 parâmetros: há redes **pequenas** quase tão boas quanto as grandes. Crescer ajuda no começo, depois os ganhos se achatam.

Exercício 3 — Aleatória ou hill climbing?

PYTHON

```
import warnings; warnings.filterwarnings("ignore")
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPClassifier
from sklearn.preprocessing import StandardScaler

X, y = load_breast_cancer(return_X_y=True)
Xa, Xv, ya, yv = train_test_split(X, y, test_size=0.3, stratify=y, random_state=0)
sc = StandardScaler().fit(Xa); Xa, Xv = sc.transform(Xa), sc.transform(Xv)
LARG, ATIV = [4, 8, 16, 32, 64], ["relu", "tanh"]
cache = {}
def nota(a):
    if a not in cache: cache[a] = MLPClassifier(a[0], activation=a[1], max_iter=100,
random_state=0).fit(Xa, ya).score(Xv, yv)
    return cache[a]
def sorteia(r):
    return (tuple(int(r.choice(LARG)) for _ in range(int(r.integers(1, 4)))), str(r.choice(ATIV)))
def muta(a, r):
    h, at = list(a[0]), a[1]; op = int(r.integers(4))
    if op == 0: h[int(r.integers(len(h)))] = int(r.choice(LARG))
    elif op == 1 and len(h) < 3: h.insert(int(r.integers(len(h) + 1)), int(r.choice(LARG)))
    elif op == 2 and len(h) > 1: h.pop(int(r.integers(len(h))))
    else: at = "tanh" if at == "relu" else "relu"
    return (tuple(h), at)
B = 15; R = {"aleatória": [], "hill climbing": []}
for s in range(5):
    r = np.random.default_rng(s)
```

```
    R["aleatória"].append(max(nota(sorteia(r)) for _ in range(B)))
    r = np.random.default_rng(100 + s); atual = sorteia(r); melhor = nota(atual)
    for _ in range(B - 1):
        f = muta(atual, r)
        if nota(f) > melhor: atual, melhor = f, nota(f)
    R["hill climbing"].append(melhor)
for k, v in R.items(): print(f"{k:<14s} média = {np.mean(v):.4f}  desvio = {np.std(v):.4f}  valores =
{np.round(v, 3).tolist()}")
```

SAÍDA

```
aleatória      média = 0.9661  desvio = 0.0023  valores = [0.965, 0.965, 0.971, 0.965, 0.965]
hill climbing  média = 0.9637  desvio = 0.0023  valores = [0.965, 0.965, 0.965, 0.965, 0.959]
```

EXEMPLO O que observar

Médias em 5 sementes: aleatória 0,9661 e hill climbing 0,9637, ambas com desvio de 0,0023. A diferença (0,0024) é do tamanho do desvio e o hill climbing não foi melhor: com 15 avaliações em um espaço generoso, a busca aleatória já alcança bons valores (0,965 em quase todas as sementes). Em orçamentos tão curtos, estratégias guiadas têm pouco tempo para se pagar. Não há evidência de que um método supere o outro neste experimento.

Exercício 4 — Quanto se pode confiar numa avaliação barata?

PYTHON

```
import warnings; warnings.filterwarnings("ignore")
import numpy as np
from scipy.stats import spearmanr
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPClassifier
from sklearn.preprocessing import StandardScaler

X, y = load_digits(return_X_y=True)
Xa, Xv, ya, yv = train_test_split(X, y, test_size=0.25, stratify=y, random_state=0)
sc = StandardScaler().fit(Xa); Xa, Xv = sc.transform(Xa), sc.transform(Xv)
r = np.random.default_rng(1)
arqs = [(tuple(int(r.choice([8, 16, 32, 64, 128])) for _ in range(int(r.integers(1, 4)))),
str(r.choice(["relu", "tanh"]))) for _ in range(30)]
def nota(a, it): return MLPClassifier(a[0], activation=a[1], max_iter=it, random_state=0).fit(Xa,
ya).score(Xv, yv)
N = {it: np.array([nota(a, it) for a in arqs]) for it in (5, 15, 60)}
print("Spearman  5 x 60 iterações: %.2f" % spearmanr(N[5], N[60])[0])
print("Spearman 15 x 60 iterações: %.2f" % spearmanr(N[15], N[60])[0])
mantidas = np.argsort(-N[5])[:10]
escolha = mantidas[np.argmax(N[60][mantidas])]
print("Melhor das 30 (60 iterações): índice", int(np.argmax(N[60])), "nota %.3f" % N[60].max())
print("Escolha do halving          : índice", int(escolha), "nota %.3f" % N[60][escolha])
print("Custo relativo: %.0f%%" % (100 * (30 * 5 + 10 * 60) / (30 * 60)))
```

SAÍDA

```
Spearman  5 x 60 iterações: 0.86
Spearman 15 x 60 iterações: 0.90
Melhor das 30 (60 iterações): índice 16 nota 0.980
Escolha do halving          : índice 16 nota 0.980
```

```
Custo relativo: 42%
```

EXEMPLO O que observar

As correlações com 60 iterações foram de 0,86 (5 iterações) e 0,90 (15 iterações): quanto mais cara a estimativa, mais fiel a ordem. O halving manteve as 10 melhores com 5 iterações e escolheu exatamente a melhor rede das 30 (nota 0,980), com 42% do custo. O resultado é favorável porque a correlação é alta; com correlação menor, a melhor rede poderia ser cortada na primeira rodada. Vale lembrar que este é um único sorteio de 30 arquiteturas: seria preciso repetir com outras sementes para medir a frequência de acertos.

Capítulo 12 — AutoML para séries temporais, texto e imagens

Exercício 1 — Validação aleatória ou temporal?

PYTHON

```python
import warnings; warnings.filterwarnings("ignore")
import numpy as np
import statsmodels.api as sm
from sklearn.linear_model import Ridge
from sklearn.model_selection import KFold, TimeSeriesSplit

y = sm.datasets.co2.load_pandas().data["co2"].interpolate().resample("MS").mean().values
y = y[:-36]
X, a = [], []
for t in range(12, len(y)):
    X.append(np.r_[y[t - 12:t][::-1] - y[t - 12], np.arange(12) == t % 12]); a.append(y[t] - y[t - 12])
X, a = np.array(X), np.array(a)
for nome, cv in (("aleatória", KFold(5, shuffle=True, random_state=0)), ("temporal ",
TimeSeriesSplit(5, test_size=12))):
    e = [np.abs(Ridge(1.0).fit(X[tr], a[tr]).predict(X[va]) - a[va]).mean() for tr, va in cv.split(X)]
    print(f"Validação {nome}: MAE = {np.mean(e):.3f}")
```

SAÍDA

```
Validação aleatória: MAE = 0.248
Validação temporal : MAE = 0.295
```

EXEMPLO **O que observar**

O MAE foi de 0,248 com a validação aleatória e de 0,295 com a temporal: a aleatória é **otimista**, embora, neste caso (um modelo linear em série diferenciada), a diferença seja pequena. Com um modelo que não extrapola, como o boosting sobre valores brutos, a diferença é enorme (0,56 contra 1,71 no capítulo). O embaralhamento é perigoso porque deixa o modelo treinar com meses posteriores aos que ele prevê, algo impossível na prática.

Exercício 2 — Três linhas de base

PYTHON

```python
import numpy as np
import statsmodels.api as sm

y = sm.datasets.co2.load_pandas().data["co2"].interpolate().resample("MS").mean().values
n, H = len(y), 36
real = y[n - H:]
ing = y[n - H - 1:n - 1]
saz = y[n - H - 12:n - 12]
mm = np.array([y[t - 12:t].mean() for t in range(n - H, n)])
for nome, p in (("ingênua", ing), ("sazonal ingênua", saz), ("média móvel de 12", mm)):
    print(f"{nome:<18s} MAE = {np.abs(p - real).mean():.3f}")
print("Crescimento médio anual da série nos últimos 36 meses: %.2f ppm/ano" % ((y[-1] - y[-37]) / 3))
```

SAÍDA

```
ingênua            MAE = 1.077
sazonal ingênua    MAE = 1.413
média móvel de 12  MAE = 1.964
Crescimento médio anual da série nos últimos 36 meses: 1.36 ppm/ano
```

EXEMPLO O que observar

A melhor foi a ingênua (MAE 1,077), seguida da sazonal ingênua (1,413) e da média móvel de 12 meses (1,964). A série cresce cerca de 1,36 ppm por ano; a sazonal ingênua repete o valor de um ano antes e, por isso, erra, em média, pelo crescimento anual (cerca de 1,4), e a média móvel erra ainda mais, por ficar atrasada em relação à tendência. Qualquer método que ignore a tendência é penalizado.

Exercício 3 — Quando n-gramas importam?

PYTHON

```python
import warnings; warnings.filterwarnings("ignore")
import numpy as np
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline

POS = ["ótimo", "bom", "excelente", "rápido", "perfeito", "útil"]; NEG = ["ruim", "péssimo", "lento",
"fraco", "horrível", "inútil"]
SUJ = ["o produto", "a entrega", "o atendimento", "o aplicativo", "a embalagem"]

def gera(n, p_neg, seed=0):
    r = np.random.default_rng(seed); docs, ys = [], []
    for _ in range(n):
        pos = r.random() < 0.5; adj = r.choice(POS if pos else NEG); neg = r.random() < p_neg
        if neg: pos = not pos
        docs.append(f"{r.choice(SUJ)} é {'não ' if neg else ''}{adj}"); ys.append(int(pos))
    return docs, np.array(ys)

for p_neg in (0.0, 0.4):
    D, Y = gera(1200, p_neg); Dtr, Dte, ytr, yte = train_test_split(D, Y, test_size=0.25, stratify=Y,
random_state=0)
    for ng in ((1, 1), (1, 2)):
        m = make_pipeline(TfidfVectorizer(ngram_range=ng), LogisticRegression(max_iter=2000)).fit(Dtr,
ytr)
        print(f"negação = {int(p_neg * 100):>2d}% | n-gramas {ng}: teste = {m.score(Dte, yte):.3f}")
```

SAÍDA

```
negação =  0% | n-gramas (1, 1): teste = 1.000
negação =  0% | n-gramas (1, 2): teste = 1.000
negação = 40% | n-gramas (1, 1): teste = 0.613
negação = 40% | n-gramas (1, 2): teste = 1.000
```

EXEMPLO O que observar

Sem negação, as palavras isoladas e os bigramas empatam em 1,000: não há nada que os bigramas capturem a mais. Com 40% de negação, as palavras isoladas caem para 0,613, enquanto os bigramas mantêm 1,000. Os bigramas só ajudam quando a informação está na **ordem ou na combinação** de palavras. Isso mostra que a representação certa depende do problema, e por isso ela entra no espaço de busca.

Exercício 4 — Qual distorção ajuda mais?

PYTHON

```python
import warnings; warnings.filterwarnings("ignore")
import numpy as np
from scipy.ndimage import shift, rotate
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC

X, y = load_digits(return_X_y=True)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.4, stratify=y, random_state=0)
Xtr, ytr = Xtr[:300], ytr[:300]

def distorce(Xs, r, dmax, rot, ruido):
    out = []
    for x in Xs:
        im = shift(x.reshape(8, 8), r.uniform(-dmax, dmax, 2), order=1, mode="constant")
        if rot > 0: im = rotate(im, r.uniform(-rot, rot), reshape=False, order=1, mode="constant")
        out.append((im + r.normal(0, ruido, im.shape)).ravel())
    return np.array(out)

Xte_d = distorce(Xte, np.random.default_rng(123), 1.0, 10, 0.0)
politicas = {"sem ampliação": (0, 0, 0), "só deslocamento": (1.5, 0, 0), "só rotação": (0, 15, 0), "só
ruído": (0, 0, 0.8), "as três": (1.5, 15, 0.8)}
for nome, (d, rot, ru) in politicas.items():
    r = np.random.default_rng(0); Xs, ys = [Xtr], [ytr]
    if max(d, rot, ru) > 0:
        for _ in range(4): Xs.append(distorce(Xtr, r, d, rot, ru)); ys.append(ytr)
    m = SVC(C=10, gamma="scale").fit(np.vstack(Xs), np.concatenate(ys))
    print(f"{nome:<16s} teste limpo = {m.score(Xte, yte):.3f} | teste distorcido = {m.score(Xte_d,
yte):.3f}")
```

SAÍDA

```
sem ampliação    teste limpo = 0.955 | teste distorcido = 0.533
só deslocamento  teste limpo = 0.947 | teste distorcido = 0.809
só rotação       teste limpo = 0.955 | teste distorcido = 0.687
só ruído         teste limpo = 0.953 | teste distorcido = 0.527
as três          teste limpo = 0.951 | teste distorcido = 0.876
```

EXEMPLO O que observar

Sob distorção, a política das três juntas foi a melhor (0,876), seguida de só deslocamento (0,809) e só rotação (0,687); só ruído (0,527) praticamente não ajudou, igual a não ampliar (0,533), pois o teste distorcido não contém ruído. No teste limpo, todas ficaram entre 0,947 e 0,955, com perda de no máximo 1 ponto. A lição: a ampliação deve **imitar a variação do uso real** (deslocamento e rotação, aqui); ruído é inútil quando o teste não o tem.

Capítulo 13 — Explicabilidade, ética e limitações do AutoML

Exercício 1 — Quem importa para o modelo?

PYTHON

```python
import warnings; warnings.filterwarnings("ignore")
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.ensemble import HistGradientBoostingClassifier
from sklearn.inspection import permutation_importance
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

d = load_breast_cancer(); Xtr, Xte, ytr, yte = train_test_split(d.data, d.target, test_size=0.3,
stratify=d.target, random_state=0)
modelos = {"Reg. logística": make_pipeline(StandardScaler(), LogisticRegression(C=0.5, max_iter=3000)),
"Boosting": HistGradientBoostingClassifier(random_state=0)}
for nome, m in modelos.items():
    m.fit(Xtr, ytr); pi = permutation_importance(m, Xte, yte, n_repeats=10, random_state=0)
    top = np.argsort(-pi.importances_mean)[:3]
    print(f"{nome:<15s} acurácia = {m.score(Xte, yte):.3f} | top 3:", ", ".join(f"{d.feature_names[i]}
({pi.importances_mean[i]:.3f})" for i in top))
```

SAÍDA

```
Reg. logística  acurácia = 0.965 | top 3: compactness error (0.016), worst texture (0.015), perimeter
error (0.014)
Boosting        acurácia = 0.942 | top 3: worst perimeter (0.022), worst texture (0.011), area error
(0.008)
```

EXEMPLO **O que observar**

Os dois modelos **não concordam**: a regressão logística destaca `compactness error`, `worst texture` e `perimeter error`, enquanto o boosting destaca `worst perimeter`, `worst texture` e `area error`. Isso é esperado: este conjunto tem muitos atributos **fortemente correlacionados** (perímetro, área e raio medem quase o mesmo), de modo que a importância por permutação se reparte entre eles de forma instável, e modelos diferentes usam atributos diferentes para chegar a notas parecidas (0,965 e 0,942, num teste de 171 exemplos). Importâncias são **propriedades do modelo**, não verdades sobre o mundo.

Exercício 2 — Medindo a disparidade por grupo

PYTHON

```python
import warnings; warnings.filterwarnings("ignore")
import numpy as np
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
```

```
def gera(n=6000, vies=0.6, p_min=0.3, seed=0):
    r = np.random.default_rng(seed)
    g = (r.random(n) < p_min).astype(int); z = r.normal(0, 1, n)
    renda = np.exp(0.5 * z + 0.35 * r.normal(size=n) - 0.25 * g) * 3.0
    emprego = np.clip(5 + 2 * z + r.normal(0, 2, n), 0, None)
    divida = np.clip(0.4 - 0.1 * z + r.normal(0, 0.1, n), 0, 1)
    cep = 0.8 * g + 0.6 * r.normal(size=n); score = z + r.normal(0, 1, n)
    y_real = (z > 0).astype(int); y_hist = ((z - vies * g + r.normal(0, 0.5, n)) > 0).astype(int)
    return np.c_[renda, emprego, divida, cep, score], g, y_hist, y_real

X, g, yh, yr = gera()
Xtr, Xte, gtr, gte, ytr, yte, yrtr, yrte = train_test_split(X, g, yh, yr, test_size=0.3,
random_state=0, stratify=yh)
m = make_pipeline(StandardScaler(), LogisticRegression(max_iter=2000)).fit(Xtr, ytr); p =
m.predict(Xte)
for k in (0, 1):
    print(f"grupo {k}: seleção = {p[gte == k].mean():.3f} | TPR real = {p[(gte == k) & (yrte ==
1)].mean():.3f} | n = {(gte == k).sum()}")
print("Acurácia contra o histórico: %.3f | contra a verdade: %.3f" % ((p == yte).mean(), (p ==
yrte).mean()))
```

SAÍDA

```
grupo 0: seleção = 0.462 | TPR real = 0.831 | n = 1274
grupo 1: seleção = 0.354 | TPR real = 0.649 | n = 526
Acurácia contra o histórico: 0.817 | contra a verdade: 0.855
```

EXEMPLO **O que observar**

Há disparidade clara. O grupo 0 teve seleção de 0,462 e TPR real de 0,831; o grupo 1, seleção de 0,354 e TPR real de 0,649: entre quem **de fato pagaria**, o grupo 1 é aprovado bem menos. O modelo **não** recebeu o grupo, mas aprendeu com rótulos enviesados e com atributos correlacionados (renda). A acurácia contra a verdade (0,855) é até maior que contra o histórico (0,817), pois o histórico está contaminado pelo viés.

Exercício 3 — Detectando proxies

PYTHON

```
import warnings; warnings.filterwarnings("ignore")
import numpy as np
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import roc_auc_score
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

def gera(n=6000, vies=0.6, p_min=0.3, seed=0):
    r = np.random.default_rng(seed)
    g = (r.random(n) < p_min).astype(int); z = r.normal(0, 1, n)
    renda = np.exp(0.5 * z + 0.35 * r.normal(size=n) - 0.25 * g) * 3.0
    emprego = np.clip(5 + 2 * z + r.normal(0, 2, n), 0, None)
    divida = np.clip(0.4 - 0.1 * z + r.normal(0, 0.1, n), 0, 1)
    cep = 0.8 * g + 0.6 * r.normal(size=n); score = z + r.normal(0, 1, n)
    y_real = (z > 0).astype(int); y_hist = ((z - vies * g + r.normal(0, 0.5, n)) > 0).astype(int)
    return np.c_[renda, emprego, divida, cep, score], g, y_hist, y_real
```

```
NOMES = ["renda", "tempo_emprego", "razao_divida", "cep_score", "score_historico"]
X, g, yh, yr = gera()
Xtr, Xte, gtr, gte = train_test_split(X, g, test_size=0.3, random_state=0, stratify=g)
def auc(cols):
    m = make_pipeline(StandardScaler(), LogisticRegression(max_iter=2000)).fit(Xtr[:, cols], gtr)
    return roc_auc_score(gte, m.predict_proba(Xte[:, cols])[:, 1])
print("AUC para prever o grupo com todos os atributos: %.3f" % auc(list(range(5))))
for i, nome in enumerate(NOMES): print(f"  sem {nome:<16s} AUC = {auc([j for j in range(5) if j != i]):.3f}")
```

SAÍDA

```
AUC para prever o grupo com todos os atributos: 0.857
  sem renda            AUC = 0.827
  sem tempo_emprego    AUC = 0.854
  sem razao_divida     AUC = 0.856
  sem cep_score        AUC = 0.694
  sem score_historico  AUC = 0.855
```

EXEMPLO **O que observar**

Mesmo sem o grupo, os atributos permitem prevê-lo com AUC de 0,857 (0,5 seria o acaso). O atributo que mais contribui é o `cep_score` (a AUC cai a 0,694 sem ele), seguido da `renda` (0,827). Os demais quase não pesam. Isso mostra que **remover o atributo sensível não impede o modelo de "conhecê-lo"**: variáveis proxy o reconstroem. Note ainda que, mesmo sem o `cep_score`, o grupo continua previsível pela renda; portanto, retirar só o proxy mais óbvio não basta.

Exercício 4 — Igualdade de oportunidade com os rótulos disponíveis

PYTHON

```
import warnings; warnings.filterwarnings("ignore")
import numpy as np
from sklearn.ensemble import HistGradientBoostingClassifier
from sklearn.model_selection import train_test_split

def gera(n=6000, vies=0.6, p_min=0.3, seed=0):
    r = np.random.default_rng(seed)
    g = (r.random(n) < p_min).astype(int); z = r.normal(0, 1, n)
    renda = np.exp(0.5 * z + 0.35 * r.normal(size=n) - 0.25 * g) * 3.0
    emprego = np.clip(5 + 2 * z + r.normal(0, 2, n), 0, None)
    divida = np.clip(0.4 - 0.1 * z + r.normal(0, 0.1, n), 0, 1)
    cep = 0.8 * g + 0.6 * r.normal(size=n); score = z + r.normal(0, 1, n)
    y_real = (z > 0).astype(int); y_hist = ((z - vies * g + r.normal(0, 0.5, n)) > 0).astype(int)
    return np.c_[renda, emprego, divida, cep, score], g, y_hist, y_real

X, g, yh, yr = gera()
Xtr, Xte, gtr, gte, ytr, yte, yrtr, yrte = train_test_split(X, g, yh, yr, test_size=0.3,
random_state=0, stratify=yh)
p = HistGradientBoostingClassifier(random_state=0).fit(Xtr, ytr).predict_proba(Xte)[:, 1]
alvo = (p[(gte == 0) & (yte == 1)] >= 0.5).mean()
cand = np.linspace(0.05, 0.9, 171)
t1 = min(cand, key=lambda t: abs((p[(gte == 1) & (yte == 1)] >= t).mean() - alvo))
for nome, pred in (("limiar único 0,5", (p >= 0.5).astype(int)), (f"limiar do grupo 1 = {t1:.2f}",
np.where(gte == 0, p >= 0.5, p >= t1).astype(int))):
    tprh = [pred[(gte == k) & (yte == 1)].mean() for k in (0, 1)]; tprr = [pred[(gte == k) & (yrte ==
```

```
1)].mean() for k in (0, 1)]
    print(f"{nome:<24s} TPR histórico = {tprh[0]:.2f}/{tprh[1]:.2f} | TPR real =
{tprr[0]:.2f}/{tprr[1]:.2f} | acc verdade = {(pred == yrte).mean():.3f}")
```

SAÍDA

```
limiar único 0,5         TPR histórico = 0.77/0.80 | TPR real = 0.81/0.67 | acc verdade = 0.831
limiar do grupo 1 = 0.54 TPR histórico = 0.77/0.77 | TPR real = 0.81/0.63 | acc verdade = 0.826
```

EXEMPLO **O que observar**

O limiar do grupo 1 (0,54) igualou o TPR contra o **histórico** (0,77 nos dois grupos), mas o TPR **real** do grupo 1 caiu de 0,67 para 0,63, e a acurácia contra a verdade caiu de 0,831 para 0,826. Ou seja, igualar a oportunidade usando os **rótulos enviesados** como referência não corrige a disparidade real, e pode até piorá-la: o critério considera "correto" o que o histórico aprovou. Se o rótulo é parte do problema, não pode ser o padrão de comparação.

Capítulo 14 — MLOps, implantação e projeto integrador

Exercício 1 — Ficha do modelo e reprodutibilidade

PYTHON

```python
import hashlib, json, os, sys, tempfile
import joblib, numpy as np, sklearn
from sklearn.datasets import load_wine
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

X, y = load_wine(return_X_y=True)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.25, stratify=y, random_state=0)
pipe = make_pipeline(StandardScaler(), LogisticRegression(max_iter=2000)).fit(Xtr, ytr)
h = lambda A: hashlib.sha256(A.tobytes()).hexdigest()[:16]
pasta = tempfile.mkdtemp(); joblib.dump(pipe, os.path.join(pasta, "m.joblib"))
ficha = {"python": sys.version.split()[0], "scikit-learn": sklearn.__version__, "semente": 0,
"hash_treino": h(Xtr)}
print("Ficha:", json.dumps(ficha))
print("Previsões idênticas após recarregar:", np.array_equal(pipe.predict(Xte),
joblib.load(os.path.join(pasta, "m.joblib")).predict(Xte)))
Xalt = Xtr.copy(); Xalt[0, 0] += 0.001
print("Hash depois de alterar um valor em 0,001:", h(Xalt), "| igual ao original?", h(Xalt) == h(Xtr))
```

SAÍDA

```
Ficha: {"python": "3.13.16", "scikit-learn": "1.9.1", "semente": 0, "hash_treino": "fd7f3e15e3067f85"}
Previsões idênticas após recarregar: True
Hash depois de alterar um valor em 0,001: f6366c8ec490c57a | igual ao original? False
```

EXEMPLO **O que observar**

O pipeline recarregado deu previsões idênticas às do original. O hash dos dados mudou por completo ao alterarmos **um único valor** em 0,001: ele é uma "impressão digital" que revela qualquer mudança nos dados de treino, o que permite saber se dois modelos foram treinados com os mesmos dados. Salva-se o pipeline inteiro porque o pré-processamento (como o `StandardScaler`) faz parte do modelo: sem ele, os dados de produção chegariam em outra escala. Os valores do hash e das versões dependem do ambiente.

Exercício 2 — Quanto custa cada previsão?

PYTHON

```python
import time
import numpy as np
from sklearn.datasets import load_digits
from sklearn.ensemble import RandomForestClassifier

X, y = load_digits(return_X_y=True); lote = np.vstack([X] * 3)[:5000]
print(f"{'árvores':>8s} {'1 previsão (ms)':>16s} {'lote 5 mil (s)':>15s}")
for k in (10, 100, 500):
    m = RandomForestClassifier(k, random_state=0, n_jobs=1).fit(X, y); m.predict(X[:1])
    t = time.perf_counter()
```

```
    for _ in range(50): m.predict(X[:1])
    ms = (time.perf_counter() - t) / 50 * 1000
    t = time.perf_counter(); m.predict(lote); print(f"{k:>8d} {ms:>16.2f} {time.perf_counter() -
t:>15.3f}")
```

SAÍDA

```
 árvores  1 previsão (ms)  lote 5 mil (s)
      10            0.64          0.005
     100            4.87          0.054
     500           22.05          0.261
```

EXEMPLO O que observar

O custo de uma previsão isolada cresceu de 0,71 ms (10 árvores) para 5,46 ms (100) e 19,38 ms (500), isto é, aproximadamente **proporcional ao número de árvores**; o lote de 5.000 foi de 0,007 s para 0,193 s. As aplicações em tempo real (milhares de pedidos por segundo, com respostas em poucos milissegundos) sofreriam mais; em lote, o custo é diluído. Os valores exatos dependem da máquina.

Exercício 3 — Detectando deriva com KS e PSI

PYTHON

```python
import numpy as np
from scipy.stats import ks_2samp
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split

def psi(ref, novo, k=10):
    cortes = np.quantile(ref, np.linspace(0, 1, k + 1)[1:-1])
    a = np.bincount(np.searchsorted(cortes, ref), minlength=k) / len(ref)
    b = np.bincount(np.searchsorted(cortes, novo), minlength=k) / len(novo)
    a, b = np.clip(a, 1e-4, None), np.clip(b, 1e-4, None)
    return float(np.sum((b - a) * np.log(b / a)))

d = load_wine(); Xr, Xp = train_test_split(d.data, test_size=0.3, random_state=0)
psis = [psi(Xr[:, j], Xp[:, j]) for j in range(Xr.shape[1])]
print("Sem deriva - três maiores PSI:", ", ".join(f"{d.feature_names[j]} ({psis[j]:.3f})" for j in
np.argsort(psis)[::-1][:3]))
j = d.feature_names.index("alcohol"); Xd = Xp.copy(); Xd[:, j] *= 1.3
print(f"alcohol sem deriva : KS = {ks_2samp(Xr[:, j], Xp[:, j]).statistic:.3f} | PSI = {psi(Xr[:, j],
Xp[:, j]):.3f}")
print(f"alcohol com +30%   : KS = {ks_2samp(Xr[:, j], Xd[:, j]).statistic:.3f} | PSI = {psi(Xr[:, j],
Xd[:, j]):.3f}")
```

SAÍDA

```
Sem deriva - três maiores PSI: total_phenols (0.954), nonflavanoid_phenols (0.591), color_intensity
(0.241)
alcohol sem deriva : KS = 0.098 | PSI = 0.115
alcohol com +30%   : KS = 1.000 | PSI = 8.192
```

EXEMPLO **O que observar**

Atenção: mesmo sem deriva alguma, o PSI de alguns atributos passou do limite de alerta (por exemplo, `total_phenols` 0,954 e `nonflavanoid_phenols` 0,591), porque a "produção" tem apenas 54 casos e o PSI, calculado em 10 faixas, é muito instável com poucos dados. O KS do `alcohol` sem deriva foi 0,098 (PSI 0,115, no alerta); com +30%, o KS chegou a 1,000 e o PSI a 8,192, uma deriva óbvia. Lição: o PSI exige **amostras razoáveis** (centenas ou milhares de casos) e as regras práticas de 0,10 e 0,25 não valem para amostras pequenas; use também testes estatísticos e inspeção visual.

Exercício 4 — Quando trocar o campeão?

PYTHON

```python
import warnings; warnings.filterwarnings("ignore")
import numpy as np
from sklearn.ensemble import HistGradientBoostingClassifier

def gera(n=6000, vies=0.6, p_min=0.3, seed=0, infl=0.0):
    r = np.random.default_rng(seed)
    g = (r.random(n) < p_min).astype(int); z = r.normal(0, 1, n)
    renda = np.exp(0.5 * z + 0.35 * r.normal(size=n) - 0.25 * g) * 3.0 * (1 + infl)
    emprego = np.clip(5 + 2 * z + r.normal(0, 2, n), 0, None)
    divida = np.clip(0.4 - 0.1 * z + r.normal(0, 0.1, n), 0, 1)
    cep = 0.8 * g + 0.6 * r.normal(size=n); score = z + r.normal(0, 1, n)
    y_real = (z > 0).astype(int); y_hist = ((z - vies * g + r.normal(0, 0.5, n)) > 0).astype(int)
    return np.c_[renda, emprego, divida, cep, score], g, y_hist, y_real

X0, g0, y0, _ = gera(5000, seed=1); campeao = HistGradientBoostingClassifier(random_state=0).fit(X0[:,
[0, 1, 2, 4]], y0)
assume, difs = 0, []
for s in range(20):
    X, g, yh, yr = gera(4000, seed=100 + s, infl=0.5); X = X[:, [0, 1, 2, 4]]
    des = HistGradientBoostingClassifier(random_state=0).fit(X[:2000], yr[:2000])
    ac, ad = campeao.score(X[2000:], yr[2000:]), des.score(X[2000:], yr[2000:])
    ep = np.sqrt(ad * (1 - ad) / 2000); difs.append(ad - ac); assume += (ad - ac) > 2 * ep
print(f"Diferença média (desafiante - campeão) = {np.mean(difs):+.3f} | mínima = {min(difs):+.3f} |
máxima = {max(difs):+.3f}")
print(f"O desafiante assumiu em {assume} de 20 repetições")
```

SAÍDA

```
Diferença média (desafiante - campeão) = +0.015 | mínima = -0.005 | máxima = +0.030
O desafiante assumiu em 9 de 20 repetições
```

EXEMPLO **O que observar**

A diferença média do desafiante sobre o campeão foi de +0,015 (de −0,005 a +0,030), e ele assumiu em apenas 9 das 20 repetições pela regra de margem. Ou seja, **o desafiante não assume sempre**, mesmo havendo um ganho médio positivo, pois em boa parte das amostras a diferença não passa de 2 erros padrão. Para evitar trocas por ruído, use margem estatística, avalie em mais dados (ou em várias janelas de tempo) e considere o custo do retreino.

Apêndice E — Soluções de referência dos projetos integradores

Estas são **soluções de referência**: caminhos possíveis, e não os únicos. Compare com o seu trabalho, observe as decisões e discuta as diferenças. A avaliação de um projeto deve considerar também a qualidade da argumentação, e não apenas o código. Os critérios de avaliação de cada projeto aparecem antes do código.

Capítulo 1 — Bem-vindo ao AutoML

Orçamento de busca e honestidade na avaliação

Critérios de avaliação

- O teste final foi separado antes da busca e não foi usado para escolher nada.
- A busca ocorreu apenas no conjunto de treino, com validação cruzada.
- Cada orçamento foi repetido com várias sementes e os resultados foram resumidos por média.
- O gráfico compara validação e teste, com eixos e legenda claros.
- O texto discute o efeito de buscar mais, a diferença entre validação e teste e o risco de superajuste da busca.

```python
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC

X, y = load_breast_cancer(return_X_y=True)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.25, stratify=y, random_state=0)

def sortear(rng):
    alg = rng.choice(["knn", "logistica", "floresta", "svm"])
    if alg == "knn":
        return make_pipeline(StandardScaler(), KNeighborsClassifier(int(rng.integers(1, 30))))
    if alg == "logistica":
        return make_pipeline(StandardScaler(), LogisticRegression(C=10 ** rng.uniform(-3, 3),
max_iter=3000))
    if alg == "floresta":
        return RandomForestClassifier(int(rng.integers(10, 150)), max_depth=int(rng.integers(2, 15)),
random_state=0)
    return make_pipeline(StandardScaler(), SVC(C=10 ** rng.uniform(-2, 3), gamma=10 ** rng.uniform(-4,
0)))

def buscar(orcamento, semente):
    rng = np.random.default_rng(semente)
    melhor, melhor_m = -1.0, None
    for _ in range(orcamento):
        m = sortear(rng)
        nota = cross_val_score(m, Xtr, ytr, cv=5).mean()
        if nota > melhor:
            melhor, melhor_m = nota, m
    melhor_m.fit(Xtr, ytr)
    return float(melhor), float(melhor_m.score(Xte, yte))

orcamentos = [5, 10, 20, 40, 80]
val, teste = [], []
```

```
for o in orcamentos:
    r = [buscar(o, s) for s in range(5)]
    val.append(np.mean([a for a, _ in r])); teste.append(np.mean([b for _, b in r]))
    print(f"orçamento {o:3d}: validação = {val[-1]:.3f} | teste final = {teste[-1]:.3f}")

plt.plot(orcamentos, val, "o-", label="validação cruzada (usada na busca)")
plt.plot(orcamentos, teste, "s-", label="teste final (nunca visto)")
plt.xscale("log"); plt.xlabel("nº de tentativas"); plt.ylabel("acurácia"); plt.legend()
plt.title("Mais busca ajuda? Validação vs. teste"); plt.show()
```

SAÍDA

```
orçamento   5: validação = 0.971 | teste final = 0.948
orçamento  10: validação = 0.978 | teste final = 0.951
orçamento  20: validação = 0.985 | teste final = 0.955
orçamento  40: validação = 0.987 | teste final = 0.957
orçamento  80: validação = 0.988 | teste final = 0.957
```

EXEMPLO Discussão dos resultados

Na execução de referência, a nota de validação subiu de 0,971 (5 tentativas) para 0,988 (80), enquanto a do teste final foi de 0,948 para 0,957. Dois fatos se destacam. Primeiro, os ganhos são pequenos e diminuem: de 40 para 80 tentativas, quase nada muda. Segundo, a validação fica cerca de 3 pontos percentuais acima do teste final: ela é otimista, porque foi usada para escolher o vencedor. Por isso o teste final deve ficar intocado até o fim.

Capítulo 2 — Aprendizado de máquina em um capítulo

Protocolo completo de avaliação em Digits

Critérios de avaliação

- O teste final foi separado antes de qualquer comparação e usado uma única vez.
- A linha de base foi calculada e comparada com os modelos.
- Os algoritmos foram comparados por validação cruzada estratificada, com média e desvio padrão.
- Foram reportadas acurácia, F1 macro e matriz de confusão do modelo escolhido.
- O texto compara validação e teste e discute as confusões mais comuns entre dígitos.

```python
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split, cross_val_score, StratifiedKFold
from sklearn.dummy import DummyClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, f1_score, confusion_matrix, ConfusionMatrixDisplay

X, y = load_digits(return_X_y=True)
print("Formato:", X.shape, "| exemplos por classe:", np.bincount(y))
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.25, stratify=y, random_state=0)

cv = StratifiedKFold(5, shuffle=True, random_state=0)
base = cross_val_score(DummyClassifier(strategy="most_frequent"), Xtr, ytr, cv=cv).mean()
print(f"Linha de base: {base:.3f}")

candidatos = {
    "Árvore": DecisionTreeClassifier(random_state=0),
    "k-vizinhos": make_pipeline(StandardScaler(), KNeighborsClassifier()),
    "Reg. logística": make_pipeline(StandardScaler(), LogisticRegression(max_iter=3000)),
    "Floresta": RandomForestClassifier(200, random_state=0),
}
res = {}
for nome, m in candidatos.items():
    n = cross_val_score(m, Xtr, ytr, cv=cv)
    res[nome] = n.mean()
    print(f"{nome:15s} validação = {n.mean():.3f} (±{n.std():.3f})")

vencedor = max(res, key=res.get)
modelo = candidatos[vencedor].fit(Xtr, ytr)
p = modelo.predict(Xte)
print(f"\nVencedor: {vencedor}")
print(f"Teste final: acurácia = {accuracy_score(yte, p):.3f} | F1 macro = {f1_score(yte, p,
average='macro'):.3f}")

ConfusionMatrixDisplay(confusion_matrix(yte, p)).plot(cmap="Blues", colorbar=False)
```

```
plt.title(f"Matriz de confusão no teste final ({vencedor})"); plt.show()
cm = confusion_matrix(yte, p); np.fill_diagonal(cm, 0)
i, j = np.unravel_index(cm.argmax(), cm.shape)
print(f"Confusão mais frequente: dígito {i} previsto como {j} ({cm[i, j]} vezes)")
```

SAÍDA

```
Formato: (1797, 64) | exemplos por classe: [178 182 177 183 181 182 181 179 174 180]
Linha de base: 0.100
Árvore          validação = 0.829 (±0.033)
k-vizinhos      validação = 0.973 (±0.009)
Reg. logística  validação = 0.971 (±0.010)
Floresta        validação = 0.977 (±0.010)

Vencedor: Floresta
Teste final: acurácia = 0.987 | F1 macro = 0.987
Confusão mais frequente: dígito 0 previsto como 4 (1 vezes)
```

EXEMPLO **Discussão dos resultados**

Na execução de referência, k-vizinhos (0,973), regressão logística (0,971) e floresta (0,977) ficam acima de 97% em validação, muito acima da linha de base (0,100) e da árvore (0,829). A floresta venceu e obteve 0,987 no teste final, coerente com a validação. Houve muito poucos erros no teste (cerca de 1%), e o par mais confundido (0 previsto como 4) ocorreu uma única vez, então não convém tirar conclusões fortes sobre quais dígitos se confundem. Este protocolo (teste intocado, linha de base, validação cruzada) será reaproveitado nos próximos capítulos.

Capítulo 3 — Pipelines e pré-processamento

Pipeline robusto para dados sujos

Critérios de avaliação

- Os dados foram "sujos" de forma explícita e documentada (ausentes e categorias).
- Todo o pré-processamento está dentro do pipeline, sem vazamento.
- O teste final foi separado antes e usado uma única vez.
- A busca em grade incluiu uma escolha de pré-processamento (imputação), e não apenas do modelo.
- O pipeline foi salvo, recarregado e funcionou com uma categoria nunca vista.
- O texto discute o que ainda exige atenção humana (significado dos dados, métrica, ética).

```python
import numpy as np
import pandas as pd
import joblib
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split, cross_val_score, GridSearchCV, StratifiedKFold
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.ensemble import RandomForestClassifier

dados = load_breast_cancer(as_frame=True)
df, y = dados.data.copy(), dados.target
rng = np.random.default_rng(0)

# 1) Sujar os dados de propósito
df["tamanho"] = pd.qcut(df["mean radius"], 3, labels=["pequeno", "médio", "grande"]).astype(str)
df["textura"] = pd.qcut(df["mean texture"], 3, labels=["lisa", "média", "áspera"]).astype(str)
df = df.drop(columns=["mean radius", "mean texture"])
num = [c for c in df.columns if c not in ("tamanho", "textura")]
cat = ["tamanho", "textura"]
for c in num:
    df.loc[rng.random(len(df)) < 0.10, c] = np.nan
print("Formato:", df.shape, "| ausentes (numéricas):", round(df[num].isna().mean().mean(), 3))

# 2) Teste final
Xtr, Xte, ytr, yte = train_test_split(df, y, test_size=0.25, stratify=y, random_state=0)

# 3) Pré-processamento por tipo de coluna
def preparo(estrategia="median"):
    return ColumnTransformer([
        ("num", Pipeline([("imp", SimpleImputer(strategy=estrategia)), ("esc", StandardScaler())]),
num),
        ("cat", Pipeline([("imp", SimpleImputer(strategy="most_frequent")),
                          ("oh", OneHotEncoder(handle_unknown="ignore"))]), cat),
    ])

# 4) Comparar modelos
```

```
cv = StratifiedKFold(5, shuffle=True, random_state=0)
modelos = {"logística": LogisticRegression(max_iter=3000), "k-vizinhos": KNeighborsClassifier(),
           "floresta": RandomForestClassifier(200, random_state=0)}
res = {}
for nome, m in modelos.items():
    res[nome] = cross_val_score(Pipeline([("p", preparo()), ("m", m)]), Xtr, ytr, cv=cv).mean()
    print(f"{nome:11s} validação = {res[nome]:.3f}")
melhor = max(res, key=res.get); print("Escolhido:", melhor)

# 5) Busca DENTRO do pipeline (imputação + hiperparâmetro do modelo)
pipe = Pipeline([("p", preparo()), ("m", modelos[melhor])])
grade = {"p__num__imp__strategy": ["mean", "median"]}
if melhor == "logística": grade["m__C"] = [0.01, 0.1, 1, 10]
elif melhor == "k-vizinhos": grade["m__n_neighbors"] = [3, 5, 9, 15]
else: grade["m__max_depth"] = [4, 8, None]
busca = GridSearchCV(pipe, grade, cv=cv).fit(Xtr, ytr)
print("Melhores ajustes:", busca.best_params_, "| validação =", round(busca.best_score_, 3))

# 6) Teste final (uma vez), salvar, recarregar e prever com categoria nova
print("Teste final:", round(busca.score(Xte, yte), 3))
joblib.dump(busca.best_estimator_, "/tmp/pipeline_cap3.joblib")
rec = joblib.load("/tmp/pipeline_cap3.joblib")
novas = Xte.iloc[:3].copy(); novas.iloc[2, novas.columns.get_loc("tamanho")] = "gigante"
print("Previsões em linhas novas:", rec.predict(novas).tolist(), "(a 3ª tem categoria nunca vista)")
```

SAÍDA

```
Formato: (569, 30) | ausentes (numéricas): 0.103
logística   validação = 0.967
k-vizinhos  validação = 0.962
floresta    validação = 0.958
Escolhido: logística
Melhores ajustes: {'m__C': 1, 'p__num__imp__strategy': 'mean'} | validação = 0.974
Teste final: 0.93
Previsões em linhas novas: [1, 0, 0] (a 3ª tem categoria nunca vista)
```

EXEMPLO **Discussão dos resultados**

Na execução de referência, os três modelos ficam próximos em validação (0,958 a 0,967); a regressão logística foi escolhida e a busca dentro do pipeline chegou a 0,974 de validação. No teste final, a acurácia foi 0,930, abaixo da validação: é uma diferença esperada, em parte porque a validação foi usada para escolher e em parte pelo tamanho pequeno do teste (143 exemplos). O pipeline funciona com categoria nova graças a `handle_unknown="ignore"`. O ganho principal do projeto é de **robustez e organização**: um único objeto recebe tabelas sujas e devolve previsões, sem vazamento.

Capítulo 4 — Busca em grade e aleatória

Relatório de otimização de hiperparâmetros em Digits

Critérios de avaliação

- O teste final foi separado antes e usado apenas uma vez, no vencedor.
- Os três espaços de busca são plausíveis e usam escala logarítmica quando adequado.
- Todos os algoritmos tiveram o mesmo orçamento (12 tentativas) e a mesma validação.
- O gráfico de convergência compara os algoritmos e é discutido no texto.
- A análise de importância é apresentada com cautela (correlação simples, poucas tentativas).
- O texto discute o otimismo da nota de validação e o que mudaria com mais orçamento.

```python
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.stats import loguniform, randint, uniform, spearmanr
from sklearn.datasets import load_digits
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier, HistGradientBoostingClassifier
from sklearn.model_selection import train_test_split, RandomizedSearchCV, StratifiedKFold

X, y = load_digits(return_X_y=True); X = X / 16.0
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.25, stratify=y, random_state=0)
cv = StratifiedKFold(3, shuffle=True, random_state=0)

algoritmos = {
    "SVM": (SVC(), {"C": loguniform(1e-2, 1e3), "gamma": loguniform(1e-5, 1)}),
    "Floresta": (RandomForestClassifier(random_state=0, n_jobs=-1),
                 {"n_estimators": randint(20, 150), "max_depth": randint(3, 20), "max_features":
uniform(0.05, 0.9)}),
    "Boosting": (HistGradientBoostingClassifier(max_iter=60, random_state=0),
                 {"learning_rate": loguniform(0.02, 0.5), "max_leaf_nodes": randint(4, 40),
"l2_regularization": loguniform(1e-4, 10)}),
}
buscas = {}
for nome, (modelo, espaco) in algoritmos.items():
    buscas[nome] = RandomizedSearchCV(modelo, espaco, n_iter=12, cv=cv, random_state=0,
refit=True).fit(Xtr, ytr)
    print(f"{nome:9s} melhor validação = {buscas[nome].best_score_:.3f}")

for nome, b in buscas.items():
    notas = pd.Series(b.cv_results_["mean_test_score"])
    plt.plot(range(1, 13), notas.cummax(), "o-", label=nome)
plt.xlabel("tentativas"); plt.ylabel("melhor nota até agora"); plt.legend(); plt.title("Convergência da
busca"); plt.show()

vencedor = max(buscas, key=lambda n: buscas[n].best_score_)
res = pd.DataFrame(buscas[vencedor].cv_results_)
print("\nVencedor:", vencedor)
for h in algoritmos[vencedor][1]:
    print(f"  {h:18s} rho = {spearmanr(res['param_' + h].astype(float), res['mean_test_score'])[0]:
+.2f}")
```

```
print(f"Teste final do vencedor: {buscas[vencedor].score(Xte, yte):.3f}")
```

SAÍDA

```
SVM       melhor validação = 0.989
Floresta  melhor validação = 0.967
Boosting  melhor validação = 0.967

Vencedor: SVM
  C                 rho = +0.53
  gamma             rho = +0.55
Teste final do vencedor: 0.993
```

EXEMPLO Discussão dos resultados

Na execução de referência, o SVM venceu com folga na validação (0,989), enquanto a floresta e o boosting empataram em 0,967. Com apenas 12 tentativas e poucas iterações de boosting, não dá para dizer que o boosting é pior em geral: ele pode simplesmente precisar de mais orçamento. A correlação de Spearman (C: +0,53; gamma: +0,55) é uma indicação grosseira com 12 pontos, e não uma conclusão. O teste final (0,993) ficou ligeiramente acima da validação: o otimismo da busca é uma tendência média, e não se manifesta em toda execução.

Capítulo 5 — Otimização bayesiana

AutoML minimalista com Optuna em regressão (Diabetes)

Critérios de avaliação

- O teste final foi separado antes e usado uma única vez.
- Foram calculadas a linha de base (média) e a Ridge padrão antes da busca.
- O espaço CASH é condicional, com hiperparâmetros em escala logarítmica quando adequado.
- TPE e aleatória foram comparados com o mesmo orçamento e várias sementes.
- O gráfico de convergência foi construído e discutido.
- O texto discute limitações (conjunto pequeno, variação entre sementes, caráter didático, ausência de uso clínico).

```python
import numpy as np
import optuna
import matplotlib.pyplot as plt
from sklearn.datasets import load_diabetes
from sklearn.dummy import DummyRegressor
from sklearn.linear_model import Ridge
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
from sklearn.svm import SVR
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split, cross_val_score, KFold
from sklearn.metrics import r2_score, mean_squared_error

optuna.logging.set_verbosity(optuna.logging.WARNING)
X, y = load_diabetes(return_X_y=True)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.25, random_state=0)
cv = KFold(5, shuffle=True, random_state=0)

print("Linha de base (média) R²:", round(cross_val_score(DummyRegressor(), Xtr, ytr, cv=cv,
scoring="r2").mean(), 3))
print("Ridge padrão          R²:", round(cross_val_score(Ridge(), Xtr, ytr, cv=cv,
scoring="r2").mean(), 3))

def construir(trial):
    alg = trial.suggest_categorical("algoritmo", ["ridge", "floresta", "boosting", "svr"])
    if alg == "ridge":
        return make_pipeline(StandardScaler(), Ridge(alpha=trial.suggest_float("alpha", 1e-3, 1e3,
log=True)))
    if alg == "floresta":
        return RandomForestRegressor(trial.suggest_int("rf_arvores", 20, 150),
max_depth=trial.suggest_int("rf_prof", 2, 10),
                                     min_samples_leaf=trial.suggest_int("rf_folha", 1, 15),
random_state=0, n_jobs=-1)
    if alg == "boosting":
        return GradientBoostingRegressor(n_estimators=trial.suggest_int("gb_arvores", 20, 200),
learning_rate=trial.suggest_float("gb_lr", 0.01, 0.3, log=True),
                                         max_depth=trial.suggest_int("gb_prof", 1, 4), random_state=0)
    return make_pipeline(StandardScaler(), SVR(C=trial.suggest_float("svr_C", 1e-1, 1e3, log=True),
epsilon=trial.suggest_float("svr_eps", 1e-2, 1.0, log=True),
```

```
                                        gamma=trial.suggest_float("svr_gamma", 1e-4, 1e-1,
log=True)))

def objetivo(trial):
    return cross_val_score(construir(trial), Xtr, ytr, cv=cv, scoring="r2").mean()

N = 40; curvas = {"TPE": [], "Aleatória": []}; melhores = []
for semente in range(3):
    for nome, sampler in [("TPE", optuna.samplers.TPESampler(seed=semente)), ("Aleatória",
optuna.samplers.RandomSampler(seed=semente))]:
        est = optuna.create_study(direction="maximize", sampler=sampler); est.optimize(objetivo,
n_trials=N)
        curvas[nome].append(np.maximum.accumulate([t.value for t in est.trials]));
melhores.append((est.best_value, est))
for nome, c in curvas.items():
    c = np.array(c); plt.plot(range(1, N + 1), c.mean(0), label=nome)
    print(f"{nome:10s} melhor R² médio após {N} tentativas: {c[:, -1].mean():.3f} (variação entre
sementes: {c[:, -1].min():.3f} a {c[:, -1].max():.3f})")
plt.xlabel("tentativas"); plt.ylabel("melhor R² até agora"); plt.legend(); plt.title("TPE x aleatória
(média de 3 sementes)"); plt.show()

valor, est = max(melhores, key=lambda t: t[0])
modelo = construir(optuna.trial.FixedTrial(est.best_params)).fit(Xtr, ytr)
p = modelo.predict(Xte)
print("\nMelhor algoritmo:", est.best_params["algoritmo"], "| R² de validação:", round(valor, 3))
print(f"Teste final: R² = {r2_score(yte, p):.3f} | RMSE = {np.sqrt(mean_squared_error(yte, p)):.1f}")
print(f"Linha de base (média) no teste: RMSE = {np.sqrt(mean_squared_error(yte,
DummyRegressor().fit(Xtr, ytr).predict(Xte))):.1f}")
pr = Ridge().fit(Xtr, ytr).predict(Xte)
print(f"Ridge padrão no teste             : R² = {r2_score(yte, pr):.3f} | RMSE =
{np.sqrt(mean_squared_error(yte, pr)):.1f}")
```

SAÍDA

```
Linha de base (média) R²: -0.014
Ridge padrão          R²: 0.425
TPE        melhor R² médio após 40 tentativas: 0.524 (variação entre sementes: 0.524 a 0.524)
Aleatória  melhor R² médio após 40 tentativas: 0.550 (variação entre sementes: 0.547 a 0.552)

Melhor algoritmo: svr | R² de validação: 0.552
Teste final: R² = 0.315 | RMSE = 58.3
Linha de base (média) no teste: RMSE = 70.5
Ridge padrão no teste             : R² = 0.357 | RMSE = 56.5
```

EXEMPLO Discussão dos resultados

Na execução de referência, a linha de base tem R^2 de −0,014 e a Ridge padrão, 0,425 na validação. A busca chegou a 0,552 (aleatória: 0,550; TPE: 0,524, com as três sementes do TPE convergindo para o mesmo valor): neste problema, a aleatória não ficou atrás. O resultado mais instrutivo vem do teste final: o melhor modelo (um SVR) obteve R^2 = 0,315 e RMSE = 58,3, **pior** que a Ridge padrão no mesmo teste (R^2 = 0,357; RMSE = 56,5), embora ambos superem a linha de base (RMSE = 70,5). Ou seja, a nota de validação de 0,552 foi bastante otimista: com 331 exemplos de treino, a busca de 40 tentativas ajustou-se ao acaso da validação. Além disso, o teste tem só 111 exemplos, e sua nota também é ruidosa. A lição: em conjuntos pequenos, um AutoML não garante superar um modelo simples bem escolhido, e só o teste final revela isso.

Capítulo 6 — Métodos multifidelidade

Halving em um modelo de boosting: custo e qualidade

Critérios de avaliação

- O teste final foi separado antes e cada vencedor foi avaliado nele uma única vez.
- Os dois métodos usaram o mesmo espaço de busca, o mesmo número de candidatos e a mesma validação.
- O custo foi calculado de forma explícita e comparável (candidatos × iterações × partes).
- O texto distingue economia de custo de qualidade final e discute o ruído com uma única execução.
- O texto discute quando a multifidelidade compensa (treinos caros, orçamento limitado).

PYTHON

```python
import numpy as np
from scipy.stats import loguniform, randint
from sklearn.experimental import enable_halving_search_cv
from sklearn.model_selection import train_test_split, RandomizedSearchCV, HalvingRandomSearchCV
from sklearn.datasets import load_digits
from sklearn.ensemble import HistGradientBoostingClassifier

X, y = load_digits(return_X_y=True)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.25, stratify=y, random_state=0)
espaco = {"learning_rate": loguniform(0.02, 0.5), "max_leaf_nodes": randint(4, 40),
          "l2_regularization": loguniform(1e-4, 10), "min_samples_leaf": randint(5, 40)}

aleat = RandomizedSearchCV(HistGradientBoostingClassifier(max_iter=100, random_state=0), espaco,
n_iter=24, cv=3, random_state=0, n_jobs=-1).fit(Xtr, ytr)
halv = HalvingRandomSearchCV(HistGradientBoostingClassifier(random_state=0), espaco, n_candidates=24,
resource="max_iter",
                             min_resources=10, max_resources=100, factor=3, cv=3, random_state=0,
n_jobs=-1).fit(Xtr, ytr)

custo_a = 24 * 100 * 3
custo_h = sum(c * r * 3 for c, r in zip(halv.n_candidates_, halv.n_resources_))
print("Halving: candidatos por rodada", halv.n_candidates_, "| iterações por rodada",
halv.n_resources_)
print(f"Custo aproximado (iterações de boosting): aleatória = {custo_a:,} | halving = {custo_h:,} |
economia = {100 * (1 - custo_h / custo_a):.0f}%")
print(f"Validação: aleatória = {aleat.best_score_:.3f} | halving = {halv.best_score_:.3f}")
print(f"Teste final: aleatória = {aleat.score(Xte, yte):.3f} | halving = {halv.score(Xte, yte):.3f}")
```

SAÍDA

```
Halving: candidatos por rodada [24, 8, 3] | iterações por rodada [10, 30, 90]
Custo aproximado (iterações de boosting): aleatória = 7,200 | halving = 2,250 | economia = 69%
Validação: aleatória = 0.970 | halving = 0.966
Teste final: aleatória = 0.973 | halving = 0.973
```

EXEMPLO Discussão dos resultados

Na execução de referência, o halving usou 2.250 iterações de boosting no total, contra 7.200 da busca aleatória (economia de 69%). A validação foi 0,970 (aleatória) contra 0,966 (halving), mas no teste final as duas empataram em 0,973. Ou seja, neste caso a economia foi "de graça". Com uma única execução e um teste de 450 exemplos, diferenças de uma ou duas décimas de ponto percentual não são conclusivas.

Capítulo 7 — Métodos evolutivos e programação genética

Um mini-TPOT com fronteira de Pareto em Breast Cancer

Critérios de avaliação

- O teste final foi separado antes e cada pipeline escolhido foi avaliado nele uma única vez.
- O GA e a busca aleatória usaram o mesmo número de avaliações, com várias sementes.
- A fronteira de Pareto foi calculada corretamente (acurácia × número de atributos).
- A escolha do pipeline enxuto seguiu uma regra explícita (perda máxima de 1 ponto percentual).
- O texto discute a incerteza (poucos exemplos, poucas sementes) e a decisão entre acerto e simplicidade.

```python
import copy
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.feature_selection import SelectKBest, f_classif
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split, cross_val_score, StratifiedKFold

X0, y0 = load_breast_cancer(return_X_y=True)
rng_d = np.random.default_rng(0)
idx = rng_d.choice(len(y0), 250, replace=False)
X = np.hstack([X0[idx], rng_d.normal(size=(250, 100))]); y = y0[idx]
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, stratify=y, random_state=0)
cv = StratifiedKFold(3, shuffle=True, random_state=0)
KS = [5, 10, 20, 40, 0]                                  # 0 = todos os atributos

def sortear_modelo(rng, m):
    if m == "logistica": return {"C": float(10 ** rng.uniform(-3, 2))}
    if m == "knn":       return {"kv": int(rng.integers(1, 25))}
    return {"arvores": int(rng.integers(20, 120)), "prof": int(rng.integers(2, 15))}

def sortear(rng):
    g = {"escala": str(rng.choice(["nenhuma", "padrao"])), "k": int(rng.choice(KS)), "modelo":
str(rng.choice(["logistica", "knn", "floresta"]))}
    g.update(sortear_modelo(rng, g["modelo"])); return g

def construir(g):
    passos = [StandardScaler()] if g["escala"] == "padrao" else []
    if g["k"] > 0: passos.append(SelectKBest(f_classif, k=g["k"]))
    m = g["modelo"]
    if m == "logistica": passos.append(LogisticRegression(C=g["C"], max_iter=500))
    elif m == "knn":     passos.append(KNeighborsClassifier(g["kv"]))
    else:                passos.append(RandomForestClassifier(g["arvores"], max_depth=g["prof"],
random_state=0, n_jobs=-1))
    return make_pipeline(*passos)

def n_atributos(g): return g["k"] if g["k"] > 0 else X.shape[1]
```

```
def aptidao(g, cache):
    chave = str(sorted(g.items()))
    if chave not in cache: cache[chave] = float(cross_val_score(construir(g), Xtr, ytr, cv=cv).mean())
    return cache[chave]

def cruzar(a, b, rng):
    f = {k: (a[k] if rng.random() < 0.5 else b[k]) for k in ("escala", "k")}
    pai = a if rng.random() < 0.5 else b
    f.update({k: v for k, v in pai.items() if k not in ("escala", "k")}); return f

def mutar(g, rng, p=0.3):
    g = copy.deepcopy(g)
    if rng.random() < p: g["escala"] = str(rng.choice(["nenhuma", "padrao"]))
    if rng.random() < p: g["k"] = int(rng.choice(KS))
    if rng.random() < p:
        g = {k: g[k] for k in ("escala", "k")}; g["modelo"] = str(rng.choice(["logistica", "knn",
"floresta"])); g.update(sortear_modelo(rng, g["modelo"]))
    return g

def evoluir(semente, pop=12, geracoes=5, elite=2):
    rng = np.random.default_rng(semente); cache = {}
    P = [sortear(rng) for _ in range(pop)]; todos = []
    for ger in range(geracoes + 1):
        f = np.array([aptidao(g, cache) for g in P]); todos += list(zip(P, f))
        if ger == geracoes: break
        ordem = np.argsort(-f); nova = [copy.deepcopy(P[i]) for i in ordem[:elite]]
        torneio = lambda: P[max(rng.choice(pop, 3, replace=False), key=lambda i: f[i])]
        while len(nova) < pop: nova.append(mutar(cruzar(torneio(), torneio(), rng), rng))
        P = nova
    return cache, todos

def aleatoria(semente, n):
    rng = np.random.default_rng(500 + semente); cache = {}
    return max(aptidao(sortear(rng), cache) for _ in range(n))

resultados = []
for s in range(3):
    cache, todos = evoluir(s)
    resultados.append((max(cache.values()), aleatoria(s, len(cache)), len(cache)))
    if s == 0: todos0 = todos
for s, (a, b, n) in enumerate(resultados): print(f"semente {s}: {n} avaliações | genético = {a:.3f} |
aleatória = {b:.3f}")

unicos = {str(sorted(g.items())): (g, float(f)) for g, f in todos0}
pontos = [(n_atributos(g), f, g) for g, f in unicos.values()]
domina = lambda a, b: a[0] <= b[0] and a[1] >= b[1] and (a[0] < b[0] or a[1] > b[1])
fronteira = sorted([p for p in pontos if not any(domina(q, p) for q in pontos)], key=lambda p: p[0])
fronteira = list({(n, round(f, 4)): (n, f, g) for n, f, g in fronteira}.values())          # remove
duplicatas (mesmos objetivos)
print("\nFronteira de Pareto (semente 0): atributos | acurácia de validação | modelo")
for n, f, g in fronteira: print(f"  {n:9d} | {f:.3f} | {g['modelo']}, escala={g['escala']}")

melhor = max(fronteira, key=lambda p: p[1]); limite = melhor[1] - 0.01
enxuto = min([p for p in fronteira if p[1] >= limite], key=lambda p: p[0])
for nome, p in [("mais acurado", melhor), ("enxuto", enxuto)]:
    teste = construir(p[2]).fit(Xtr, ytr).score(Xte, yte)
    print(f"Pipeline {nome:12s}: {p[0]:3d} atributos | validação = {p[1]:.3f} | TESTE FINAL =
{teste:.3f}")
```

SAÍDA

```
semente 0: 39 avaliações | genético = 0.977 | aleatória = 0.971
semente 1: 39 avaliações | genético = 0.977 | aleatória = 0.977
semente 2: 39 avaliações | genético = 0.983 | aleatória = 0.977

Fronteira de Pareto (semente 0): atributos | acurácia de validação | modelo
          5 | 0.937 | floresta, escala=nenhuma
         10 | 0.960 | floresta, escala=nenhuma
         20 | 0.971 | floresta, escala=padrao
         40 | 0.977 | floresta, escala=nenhuma
Pipeline mais acurado:  40 atributos | validação = 0.977 | TESTE FINAL = 0.920
Pipeline enxuto      :  20 atributos | validação = 0.971 | TESTE FINAL = 0.933
```

EXEMPLO **Discussão dos resultados**

Na execução de referência, o GA terminou à frente da aleatória em duas das três sementes e empatou em uma (0,977 contra 0,971; 0,977 contra 0,977; 0,983 contra 0,977), com apenas 39 avaliações por busca. A fronteira da semente 0 vai de 5 atributos (0,937) a 40 atributos (0,977). O pipeline "enxuto" (20 atributos, validação 0,971) obteve 0,933 no teste final e o "mais acurado" (40 atributos, validação 0,977) obteve 0,920: o menor foi igual ou melhor, ou seja, a escolha do pipeline simples saiu "de graça" (a diferença de 1 ponto percentual equivale a um exemplo em 75). Atenção: a acurácia do teste (0,92 a 0,93) ficou bem abaixo da validação (0,97), o que mostra o otimismo da busca e a incerteza de um teste com apenas 75 exemplos.

Capítulo 8 — Engenharia e seleção automática de atributos

Engenharia de atributos honesta em Diabetes

Critérios de avaliação

- Toda a engenharia (quadráticos, seleção, Lasso) está dentro do pipeline, sem vazamento.
- Os pipelines foram comparados por validação cruzada repetida, com média e desvio padrão.
- O teste final foi separado antes e usado apenas para o vencedor e para a linha de base (pipeline A).
- O texto compara a diferença entre pipelines com a variação da validação cruzada.
- A recomendação final é coerente com os resultados e reconhece as limitações (conjunto pequeno, caráter didático).

```python
import numpy as np
from sklearn.datasets import load_diabetes
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler, PolynomialFeatures
from sklearn.linear_model import Ridge, Lasso
from sklearn.feature_selection import SelectKBest, f_regression
from sklearn.ensemble import GradientBoostingRegressor
from sklearn.model_selection import train_test_split, cross_val_score, RepeatedKFold
from sklearn.metrics import r2_score, mean_squared_error

X, y = load_diabetes(return_X_y=True)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.25, random_state=0)
poly = lambda: PolynomialFeatures(2, include_bias=False)

pipelines = {
    "A: Ridge": make_pipeline(StandardScaler(), Ridge(alpha=1.0)),
    "B: quadráticos + Ridge": make_pipeline(poly(), StandardScaler(), Ridge(alpha=10.0)),
    "C: quadráticos + SelectKBest(15) + Ridge": make_pipeline(poly(), StandardScaler(),
SelectKBest(f_regression, k=15), Ridge(alpha=1.0)),
    "D: quadráticos + Lasso": make_pipeline(poly(), StandardScaler(), Lasso(alpha=0.5,
max_iter=20000)),
    "E: gradient boosting": GradientBoostingRegressor(n_estimators=100, learning_rate=0.05,
max_depth=2, random_state=0),
}
cv = RepeatedKFold(n_splits=5, n_repeats=3, random_state=0)
res = {}
for nome, p in pipelines.items():
    r = cross_val_score(p, Xtr, ytr, cv=cv, scoring="r2")
    res[nome] = r.mean()
    print(f"{nome:42s} R² = {r.mean():.3f} (±{r.std():.3f})")

melhor = max(res, key=res.get)
print("\nMelhor na validação:", melhor)
for nome in dict.fromkeys([melhor, "A: Ridge"]):
    p = pipelines[nome].fit(Xtr, ytr).predict(Xte)
    print(f"Teste final | {nome:42s} R² = {r2_score(yte, p):.3f} | RMSE =
{np.sqrt(mean_squared_error(yte, p)):.1f}")
```

SAÍDA

```
A: Ridge                                  R² = 0.524 (±0.063)
B: quadráticos + Ridge                    R² = 0.466 (±0.076)
C: quadráticos + SelectKBest(15) + Ridge  R² = 0.499 (±0.063)
D: quadráticos + Lasso                    R² = 0.488 (±0.070)
E: gradient boosting                      R² = 0.480 (±0.061)

Melhor na validação: A: Ridge
Teste final | A: Ridge                                  R² = 0.358 | RMSE = 56.5
```

EXEMPLO **Discussão dos resultados**

O pipeline mais simples (A, Ridge) foi o melhor na validação (R^2 = 0,524), à frente de todos os que acrescentaram sofisticação: quadráticos + Ridge (0,466), com SelectKBest (0,499), com Lasso (0,488) e gradient boosting (0,480). As diferenças entre A e os demais (de 0,025 a 0,058) são da ordem do desvio padrão da validação cruzada (cerca de 0,06), o que mostra que a vantagem não é grande, mas todos os caminhos sofisticados ficaram atrás. No teste final, o pipeline A obteve R^2 = 0,358 e RMSE = 56,5, abaixo da validação (0,524): o teste tem só 111 exemplos e a validação foi usada para escolher. A recomendação honesta é adotar o modelo mais simples.

Capítulo 9 — Meta-aprendizado, warm start e ensembles

Um mini Auto-sklearn: warm start, busca TPE e ensemble

Critérios de avaliação

- O teste final foi separado antes; a validação foi usada apenas para os pesos do ensemble; a busca usou validação cruzada no treino.
- O espaço CASH é condicional e usa escala logarítmica quando adequado.
- O warm start foi implementado com enqueue_trial.
- As três opções (melhor único, média simples, Caruana) foram comparadas no mesmo teste final, usado uma única vez.
- O texto discute o tamanho do teste, o ruído de rótulos e o fato de a média simples poder bastar.

```python
import numpy as np
import optuna
from sklearn.datasets import load_breast_cancer
from sklearn.svm import SVC
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split, cross_val_score, StratifiedKFold
from sklearn.metrics import accuracy_score, log_loss

optuna.logging.set_verbosity(optuna.logging.WARNING)
X0, y0 = load_breast_cancer(return_X_y=True)
rng = np.random.default_rng(0)
X = np.hstack([X0, rng.normal(size=(len(y0), 100))])
y = np.where(rng.random(len(y0)) < 0.10, 1 - y0, y0)                    # 10% de rótulos trocados
d = X.shape[1]
Xa, Xte, ya, yte = train_test_split(X, y, test_size=0.25, stratify=y, random_state=0)
Xtr, Xva, ytr, yva = train_test_split(Xa, ya, test_size=1/3, stratify=ya, random_state=0)
cv = StratifiedKFold(3, shuffle=True, random_state=0)

guardados = {}                                                          # probabilidades de cada tentativa
def objetivo(trial):
    alg = trial.suggest_categorical("alg", ["lr", "svm", "rf"])
    if alg == "lr":
        m = make_pipeline(StandardScaler(), LogisticRegression(C=trial.suggest_float("lr_C", 1e-4, 1e2, log=True), max_iter=3000))
    elif alg == "svm":
        m = make_pipeline(StandardScaler(), SVC(C=trial.suggest_float("svm_C", 1e-2, 1e3, log=True),
                          gamma=trial.suggest_float("svm_gamma", 1e-5, 1e-1, log=True), probability=True, random_state=0))
    else:
        m = RandomForestClassifier(trial.suggest_int("rf_n", 20, 150), max_depth=trial.suggest_int("rf_prof", 2, 14), random_state=0, n_jobs=-1)
    nota = cross_val_score(m, Xtr, ytr, cv=cv).mean()
    m.fit(Xtr, ytr); guardados[trial.number] = (m.predict_proba(Xva), m.predict_proba(Xte))
    return nota
```

```
est = optuna.create_study(direction="maximize", sampler=optuna.samplers.TPESampler(seed=0))
est.enqueue_trial({"alg": "svm", "svm_C": 1.0, "svm_gamma": 1.0 / d})       # warm start
est.enqueue_trial({"alg": "rf", "rf_n": 100, "rf_prof": 8})
est.optimize(objetivo, n_trials=30)

ids = sorted(guardados); PV = [guardados[i][0] for i in ids]; PT = [guardados[i][1] for i in ids]
melhor = max(est.trials, key=lambda t: t.value).number
print(f"Melhor modelo único (validação cruzada = {est.best_value:.3f}): {est.best_params['alg']}")

escolhidos, atual = [], None                                                  # seleção de Caruana na
validação
for _ in range(20):
    perdas = []
    for j in range(len(PV)):
        cand = PV[j] if not escolhidos else (atual * len(escolhidos) + PV[j]) / (len(escolhidos) + 1)
        perdas.append(log_loss(yva, np.clip(cand, 1e-12, 1), labels=[0, 1]))
    j = int(np.argmin(perdas)); atual = PV[j] if not escolhidos else (atual * len(escolhidos) +
PV[j]) / (len(escolhidos) + 1); escolhidos.append(j)
pesos = np.bincount(escolhidos, minlength=len(PV))

teste_unico = accuracy_score(yte, PT[ids.index(melhor)].argmax(1))
teste_media = accuracy_score(yte, (sum(PT) / len(PT)).argmax(1))
teste_caruana = accuracy_score(yte, (sum(p * w for p, w in zip(PT, pesos)) / pesos.sum()).argmax(1))
print(f"\nTESTE FINAL ({len(yte)} exemplos):")
print(f"  melhor modelo único        : {teste_unico:.3f}")
print(f"  média simples de 30 modelos: {teste_media:.3f}")
print(f"  ensemble de Caruana        : {teste_caruana:.3f} ({(pesos > 0).sum()} modelos distintos)")
```

SAÍDA

```
Melhor modelo único (validação cruzada = 0.863): rf

TESTE FINAL (143 exemplos):
  melhor modelo único        : 0.811
  média simples de 30 modelos: 0.818
  ensemble de Caruana        : 0.818 (3 modelos distintos)
```

EXEMPLO Discussão dos resultados

O resultado depende do ruído: com 10% de rótulos trocados, o teto de acurácia é de cerca de 90%, e as diferenças entre as três opções tendem a ser pequenas, dentro do ruído de um teste de 143 exemplos. Veja na saída se o ensemble superou o melhor único e se a média simples bastou. Mesmo que o ensemble ganhe por um ou dois exemplos, isso não é conclusivo; o valor do exercício está em montar o fluxo completo (warm start, busca, ensemble) de forma honesta.

Capítulo 10 — Ferramentas de AutoML na prática

Duelo de AutoML em miniatura

Critérios de avaliação

- Os três métodos foram comparados nas mesmas divisões, com o mesmo teste final intocado.
- O FLAML foi limitado por número de tentativas (max_iter), e o texto explica por quê.
- A tabela traz médias por base e postos médios corretos (1 = melhor).
- O Friedman usou N = número de bases (e não de divisões), e o texto discute o baixo poder.
- A conclusão considera o tamanho dos testes e a variação entre divisões, sem declarar vencedor universal.

PYTHON

```
import warnings; warnings.filterwarnings("ignore")
import numpy as np
from flaml import AutoML
from scipy.stats import rankdata, friedmanchisquare
from sklearn.datasets import load_iris, load_wine, load_breast_cancer, load_digits
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier, HistGradientBoostingClassifier

bases = {"Iris": load_iris(return_X_y=True), "Wine": load_wine(return_X_y=True),
         "Cancer": load_breast_cancer(return_X_y=True), "Digits": load_digits(return_X_y=True)}
nomes = ["Floresta", "Boosting", "FLAML(20)"]
M = []; escolhas = {}
for b, (X, y) in bases.items():
    acc = {n: [] for n in nomes}; esc = []
    for s in (0, 1, 2):
        Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.25, stratify=y, random_state=s)
        m1 = RandomForestClassifier(200, random_state=s, n_jobs=-1).fit(Xtr, ytr)
        m2 = HistGradientBoostingClassifier(random_state=s).fit(Xtr, ytr)
        a = AutoML(); a.fit(Xtr, ytr, task="classification", time_budget=120, max_iter=20,
metric="accuracy",
                            eval_method="cv", n_splits=3, seed=s, verbose=0)
        for n, m in zip(nomes, (m1, m2, a)): acc[n].append((m.predict(Xte) == yte).mean())
        esc.append((a.best_estimator, len(a.config_history)))
    M.append([np.mean(acc[n]) for n in nomes]); escolhas[b] = esc
M = np.array(M)
print(f"{'':<8s}" + "  ".join(f"{n:>10s}" for n in nomes))
for b, lin in zip(bases, M): print(f"{b:<8s}" + "  ".join(f"{v:10.3f}" for v in lin))
P = np.array([rankdata(-l) for l in M])
print("Postos médios:", dict(zip(nomes, P.mean(0).round(2))))
est, p = friedmanchisquare(*M.T)
print(f"Friedman: estatística = {est:.2f}, valor-p = {p:.3f} (N = {len(M)})")
print("Escolhas do FLAML (algoritmo, nº de configurações):", {b: e for b, e in escolhas.items()})
```

SAÍDA

```
          Floresta    Boosting   FLAML(20)
Iris         0.974       0.974       0.982
Wine         0.993       1.000       0.970
Cancer       0.951       0.949       0.939
Digits       0.978       0.971       0.947
Postos médios: {'Floresta': np.float64(1.62), 'Boosting': np.float64(1.88), 'FLAML(20)':
```

```
np.float64(2.5)}
Friedman: estatística = 1.73, valor-p = 0.420 (N = 4)
Escolhas do FLAML (algoritmo, nº de configurações): {'Iris': [('xgboost', 2), ('lgbm', 1), ('lgbm',
4)], 'Wine': [('x…
```

EXEMPLO Discussão dos resultados

Com `max_iter=20`, o FLAML ficou com o pior posto médio (2,50), contra 1,62 da floresta e 1,88 do boosting; o valor-p do Friedman (0,420, com N = 4) não rejeita a equivalência. Note que o FLAML registrou poucas melhorias (de 1 a 8 configurações em `config_history`) e escolheu quase sempre `lgbm`. Com apenas 20 tentativas e poucos exemplos, a busca não teve tempo de superar bons padrões. Repare que o limite por tentativas, ao contrário do orçamento de tempo, torna a execução reprodutível, mas não necessariamente mais forte. Ao concluir, evite declarar vencedor: com N = 4 bases e testes pequenos (45 a 450 exemplos), as diferenças estão dentro do ruído.

Capítulo 11 — Busca de arquitetura neural (NAS)

Uma rede enxuta por busca de arquitetura

Critérios de avaliação

- O espaço e o seu tamanho foram definidos e calculados corretamente (310 arquiteturas).
- As duas estratégias usaram o mesmo orçamento e as mesmas sementes de avaliação.
- A fronteira de Pareto foi construída com todas as arquiteturas avaliadas.
- A escolha da rede pequena usou apenas a validação; o teste foi medido uma única vez para cada uma das duas redes.
- O texto discute o tamanho do teste, o ruído de rótulos e evita declarar vencedor sem evidência.

```python
import warnings; warnings.filterwarnings("ignore")
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPClassifier
from sklearn.preprocessing import StandardScaler

X, y = load_breast_cancer(return_X_y=True)
rng = np.random.default_rng(0)
X = np.hstack([X, rng.normal(size=(len(y), 70))])
y = np.where(rng.random(len(y)) < 0.10, 1 - y, y)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.25, stratify=y, random_state=0)
Xa, Xv, ya, yv = train_test_split(Xtr, ytr, test_size=0.3, stratify=ytr, random_state=0)
sc = StandardScaler().fit(Xa); Xa, Xv, Xt = sc.transform(Xa), sc.transform(Xv), sc.transform(Xte)
D = X.shape[1]
LARG, ATIV = [4, 8, 16, 32, 64], ["relu", "tanh"]
print("Tamanho do espaço:", len(ATIV) * sum(len(LARG) ** L for L in range(1, 4)))
cache = {}
def avalia(a):
    if a not in cache:
        m = MLPClassifier(a[0], activation=a[1], max_iter=100, random_state=0).fit(Xa, ya)
        d = [D, *a[0], 2]; cache[a] = (m.score(Xv, yv), sum(i * o + o for i, o in zip(d[:-1], d[1:])),
m.score(Xt, yte))
    return cache[a]
def sorteia(r): return (tuple(int(r.choice(LARG)) for _ in range(int(r.integers(1, 4)))),
str(r.choice(ATIV)))
def muta(a, r):
    h, at = list(a[0]), a[1]; op = int(r.integers(4))
    if op == 0: h[int(r.integers(len(h)))] = int(r.choice(LARG))
    elif op == 1 and len(h) < 3: h.insert(int(r.integers(len(h) + 1)), int(r.choice(LARG)))
    elif op == 2 and len(h) > 1: h.pop(int(r.integers(len(h))))
    else: at = "tanh" if at == "relu" else "relu"
    return (tuple(h), at)
B = 40; vistas = set(); melhores = {"aleatória": [], "hill climbing": []}
for s in range(3):
    r = np.random.default_rng(s); h = [sorteia(r) for _ in range(B)]
    melhores["aleatória"].append(max(avalia(a)[0] for a in h)); vistas.update(h)
    r = np.random.default_rng(50 + s); atual = sorteia(r); melhor = avalia(atual)[0]; vistas.add(atual)
    for _ in range(B - 1):
        f = muta(atual, r); vistas.add(f)
```

```
        if avalia(f)[0] > melhor: atual, melhor = f, avalia(f)[0]
    melhores["hill climbing"].append(melhor)
for k, v in melhores.items(): print(f"{k:<14s} melhor validação (3 sementes): {np.round(v, 3).tolist()}
média = {np.mean(v):.3f}")
T = [(a,) + avalia(a) for a in vistas]            # (arq, validação, parâmetros, teste)
front = [t for t in T if not any((u[1] >= t[1] and u[2] <= t[2]) and (u[1] > t[1] or u[2] < t[2]) for u
in T)]
front.sort(key=lambda t: t[2])
print("Arquiteturas distintas avaliadas:", len(T), "| na fronteira:", len(front))
for a, v, p, _ in front[-6:]: print(f"  {str(a[0]):<14s} {a[1]:<5s} parâmetros = {p:>5d}  validação =
{v:.3f}")
melhor = max(T, key=lambda t: t[1]); pequena = min([t for t in T if t[1] >= melhor[1] - 0.01],
key=lambda t: t[2])
print(f"\nMais acurada na validação : {melhor[0][0]} {melhor[0][1]} | {melhor[2]} parâmetros | val =
{melhor[1]:.3f} | TESTE = {melhor[3]:.3f}")
print(f"Menor dentro de 1 ponto    : {pequena[0][0]} {pequena[0][1]} | {pequena[2]} parâmetros | val =
{pequena[1]:.3f} | TESTE = {pequena[3]:.3f}")
print(f"Redução de parâmetros: {100 * (1 - pequena[2] / melhor[2]):.0f}%  (teste = {len(yte)}
exemplos)")
```

SAÍDA

```
Tamanho do espaço: 310
aleatória      melhor validação (3 sementes): [0.82, 0.797, 0.812]  média = 0.810
hill climbing  melhor validação (3 sementes): [0.82, 0.828, 0.82]  média = 0.823
Arquiteturas distintas avaliadas: 117 | na fronteira: 5
  (4,)           tanh  parâmetros =   414  validação = 0.789
  (4, 32)        tanh  parâmetros =   630  validação = 0.797
  (8, 4)         relu  parâmetros =   854  validação = 0.812
  (32, 4)        tanh  parâmetros =  3374  validação = 0.820
  (64, 8, 4)     relu  parâmetros =  7030  validação = 0.828

Mais acurada na validação : (64, 8, 4) relu | 7030 parâmetros | val = 0.828 | TESTE = 0.706
Menor dentro de 1 ponto    : (32, 4) tanh | 3374 parâmetros | val = 0.820 | TESTE = 0.706
Redução de parâmetros: 52%  (teste = 143 exemplos)
```

EXEMPLO **Discussão dos resultados**

O espaço tem 310 arquiteturas; foram avaliadas 117 distintas. O *hill climbing* teve média de validação ligeiramente maior (0,823 contra 0,810 da aleatória), com apenas 3 sementes: é uma diferença pequena, que não permite declarar vencedor. A fronteira de Pareto tem 5 redes. A mais acurada na validação (64, 8, 4; 7.030 parâmetros; validação 0,828) e a menor dentro de 1 ponto (32, 4; 3.374 parâmetros; 0,820) tiveram a **mesma** acurácia no teste (0,706), com 52% menos parâmetros. O teste (0,706) ficou muito abaixo da validação (0,82): há ruído de rótulos (10%), 70 colunas de ruído, validação pequena (cerca de 300 exemplos) e o efeito de a busca escolher pelo melhor valor da validação. Com apenas 143 exemplos de teste, a incerteza é grande. A lição prática: a rede 52% menor custou, aqui, praticamente nada em desempenho.

Capítulo 12 — AutoML para séries temporais, texto e imagens

Previsão de manchas solares com mini-AutoML

Critérios de avaliação

- A validação usou apenas dados de treino e respeitou a ordem do tempo (TimeSeriesSplit).
- O teste foi feito em avanço (reajuste a cada ano), com horizonte de 1 ano.
- As linhas de base (ingênua e média de 11 anos) foram calculadas nas mesmas condições.
- A pior configuração foi apresentada e comparada.
- O texto discute o ruído natural da série e evita declarar superioridade sem evidência.

```python
import warnings; warnings.filterwarnings("ignore")
import itertools
import numpy as np
import statsmodels.api as sm
from sklearn.linear_model import Ridge
from sklearn.ensemble import RandomForestRegressor, HistGradientBoostingRegressor
from sklearn.model_selection import TimeSeriesSplit

y = sm.datasets.sunspots.load_pandas().data["SUNACTIVITY"].values.astype(float)
n, H = len(y), 40
MOD = {"ridge": lambda: Ridge(1.0), "floresta": lambda: RandomForestRegressor(100, random_state=0,
n_jobs=-1),
       "hgb": lambda: HistGradientBoostingRegressor(max_iter=100, random_state=0)}

def monta(y, L, dif):
    X, a = [], []
    for t in range(L, len(y)):
        b = y[t - 1] if dif else 0.0
        X.append(y[t - L:t][::-1] - b); a.append(y[t] - b)
    return np.array(X), np.array(a)

def mae_cv(cfg):
    mod, L, dif = cfg; X, a = monta(y[:n - H], L, dif)
    return np.mean([np.abs(MOD[mod]().fit(X[tr], a[tr]).predict(X[va]) - a[va]).mean() for tr, va in
TimeSeriesSplit(4, test_size=20).split(X)])

def walk(cfg):
    mod, L, dif = cfg; p = []
    for t in range(n - H, n):
        X, a = monta(y[:t], L, dif); m = MOD[mod]().fit(X, a); b = y[t - 1] if dif else 0.0
        p.append(m.predict((y[t - L:t][::-1] - b)[None])[0] + b)
    return np.array(p)

configs = list(itertools.product(MOD, (4, 11, 22), (False, True)))
rank = sorted((mae_cv(c), c) for c in configs)
print("Validação temporal (MAE): 4 melhores e a pior")
for e, c in rank[:4] + rank[-1:]: print(f"  {e:6.2f}  {c[0]:<9s} lags={c[1]:>2d} dif={'sim' if c[2]
else 'não'}")
real = y[n - H:]
```

```
esc = rank[0][1]; pior = rank[-1][1]
print("\nTESTE (últimos 40 anos, 1 ano à frente), MAE:")
print(f"  mini-AutoML {esc[0]}, lags={esc[1]}, dif={'sim' if esc[2] else 'não'} : {np.abs(walk(esc) -
real).mean():.2f}")
print(f"  pior configuração {pior[0]}, lags={pior[1]}, dif={'sim' if pior[2] else 'não'}:
{np.abs(walk(pior) - real).mean():.2f}")
print(f"  ingênua                                  : {np.abs(y[n - H - 1:n - 1] - real).mean():.2f}")
print(f"  média dos últimos 11 anos                : {np.abs(np.array([y[t - 11:t].mean() for t in
range(n - H, n)]) - real).mean():.2f}")
```

SAÍDA

```
Validação temporal (MAE): 4 melhores e a pior
   10.60  floresta  lags=22 dif=sim
   10.79  floresta  lags=11 dif=sim
   11.46  hgb       lags=22 dif=sim
   11.56  ridge     lags=11 dif=não
   15.19  ridge     lags= 4 dif=sim

TESTE (últimos 40 anos, 1 ano à frente), MAE:
  mini-AutoML floresta, lags=22, dif=sim : 12.59
  pior configuração ridge, lags=4, dif=sim: 17.94
  ingênua                                  : 22.74
  média dos últimos 11 anos                : 40.42
```

EXEMPLO **Discussão dos resultados**

A configuração escolhida pela validação foi a floresta aleatória com 22 atrasos e diferenciação (MAE de validação 10,60); no teste, ela errou 12,59 manchas, contra 22,74 da ingênua e 40,42 da média dos últimos 11 anos. A pior configuração (Ridge com 4 atrasos e diferenciação) errou 17,94 no teste, ainda melhor que a ingênua. O mini-AutoML superou as linhas de base com folga neste experimento, mas cuidado: o MAE de cerca de 12 é grande em relação à amplitude da série, refletindo o **ruído natural** do fenômeno solar, que nenhum modelo elimina. Também não se deve extrapolar de uma série e de uma janela de 40 anos para outras situações.

Capítulo 13 — Explicabilidade, ética e limitações do AutoML

Auditoria ética de um modelo de AutoML

Critérios de avaliação

- Os modelos foram treinados sem o atributo grupo, e o teste foi separado antes de qualquer decisão.
- A comparação entre FLAML e regressão logística usou a regra do empate técnico.
- As métricas foram calculadas por grupo (seleção e TPR real) para os dois modelos.
- O papel dos proxies foi investigado (AUC para prever o grupo) e discutido.
- A mitigação foi aplicada e discutida com trade-offs, sem declarar o problema resolvido.
- O relatório cita o custo, as salvaguardas e as limitações da simulação.

```python
import warnings; warnings.filterwarnings("ignore")
import time
import numpy as np
from flaml import AutoML
from sklearn.inspection import permutation_importance
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import roc_auc_score
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

def gera(n=6000, vies=0.6, p_min=0.3, seed=0):
    r = np.random.default_rng(seed)
    g = (r.random(n) < p_min).astype(int); z = r.normal(0, 1, n)
    renda = np.exp(0.5 * z + 0.35 * r.normal(size=n) - 0.25 * g) * 3.0
    emprego = np.clip(5 + 2 * z + r.normal(0, 2, n), 0, None)
    divida = np.clip(0.4 - 0.1 * z + r.normal(0, 0.1, n), 0, 1)
    cep = 0.8 * g + 0.6 * r.normal(size=n); score = z + r.normal(0, 1, n)
    y_real = (z > 0).astype(int); y_hist = ((z - vies * g + r.normal(0, 0.5, n)) > 0).astype(int)
    return np.c_[renda, emprego, divida, cep, score], g, y_hist, y_real

NOMES = ["renda", "tempo_emprego", "razao_divida", "cep_score", "score_historico"]
X, g, yh, yr = gera(vies=0.4, p_min=0.2, seed=7)
Xtr, Xte, gtr, gte, ytr, yte, yrtr, yrte = train_test_split(X, g, yh, yr, test_size=0.3,
random_state=0, stratify=yh)
t0 = time.time(); a = AutoML()
a.fit(Xtr, ytr, task="classification", time_budget=120, max_iter=20, metric="accuracy",
eval_method="cv", n_splits=3, seed=0, verbose=0)
tempo = time.time() - t0
lr = make_pipeline(StandardScaler(), LogisticRegression(max_iter=2000)).fit(Xtr, ytr)
print(f"FLAML ({a.best_estimator}): acc = {(a.predict(Xte) == yte).mean():.3f} | Reg. logística: acc =
{lr.score(Xte, yte):.3f}")
pi = permutation_importance(a.model.estimator, Xte, yte, n_repeats=10, random_state=0)
print("Permutação (FLAML):", {n: round(float(v), 3) for n, v in zip(NOMES, pi.importances_mean)})

def rel(nome, pred):
    s = [pred[gte == k].mean() for k in (0, 1)]; t = [pred[(gte == k) & (yrte == 1)].mean() for k in
(0, 1)]
```

```
    print(f"{nome:<22s} acc hist = {(pred == yte).mean():.3f} | acc verdade = {(pred ==
yrte).mean():.3f} | seleção {s[0]:.2f}/{s[1]:.2f} | TPR real {t[0]:.2f}/{t[1]:.2f}")
rel("FLAML", a.predict(Xte)); rel("Reg. logística", lr.predict(Xte))
pg = make_pipeline(StandardScaler(), LogisticRegression(max_iter=2000)).fit(Xtr, gtr)
print("AUC para prever o grupo pelos atributos: %.3f" % roc_auc_score(gte, pg.predict_proba(Xte)[:,
1]))
p = a.predict_proba(Xte)[:, 1]; sel0 = (p[gte == 0] >= 0.5).mean(); t1 = np.quantile(p[gte == 1], 1 -
sel0)
rel("FLAML + limiares grupo", np.where(gte == 0, p >= 0.5, p >= t1).astype(int))
print(f"Limiar do grupo 1 = {t1:.2f} | melhorias em config_history = {len(a.config_history)} | tempo de
busca = {tempo:.0f} s")
```

SAÍDA

```
FLAML (lgbm): acc = 0.800 | Reg. logística: acc = 0.796
Permutação (FLAML): {'renda': 0.128, 'tempo_emprego': 0.011, 'razao_divida': 0.032, 'cep_score': 0.0,
'score_historic…
FLAML                 acc hist = 0.800 | acc verdade = 0.848 | seleção 0.48/0.37 | TPR real 0.83/0.71
Reg. logística        acc hist = 0.796 | acc verdade = 0.856 | seleção 0.46/0.35 | TPR real 0.82/0.69
AUC para prever o grupo pelos atributos: 0.833
FLAML + limiares grupo acc hist = 0.791 | acc verdade = 0.856 | seleção 0.48/0.49 | TPR real 0.83/0.87
Limiar do grupo 1 = 0.39 | melhorias em config_history = 4 | tempo de busca = 1 s
```

EXEMPLO **Discussão dos resultados**

FLAML escolheu `lgbm` e obteve 0,800 de acurácia, contra 0,796 da regressão logística: **empate técnico**, o que, pela regra do capítulo, favorece o modelo interpretável. A importância por permutação é dominada pela `renda` (0,128); o `cep_score` quase não pesa (0,0), mas isso não o inocenta, pois a renda carrega informação do grupo: o grupo pode ser previsto pelos atributos com AUC de 0,833. As disparidades persistem nos dois modelos (seleção de 0,48/0,37 no FLAML; TPR real de 0,83/0,71). Com limiares por grupo, a seleção se iguala (0,48/0,49), o TPR real do grupo 1 sobe para 0,87 e a acurácia contra a verdade se mantém (0,856), com pequena queda contra o histórico (0,791). O custo da busca foi pequeno (4 melhorias registradas e cerca de 1 segundo), pois foi limitada a 20 tentativas. Recomendação: não implantar sem revisão humana, monitoramento por grupo e canal de contestação; os resultados valem para a **simulação**, não para dados reais.

Capítulo 14 — MLOps, implantação e projeto integrador

Projeto integrador: AutoML de ponta a ponta, com ética e produção

Critérios de avaliação

- O teste final foi separado antes de qualquer decisão e usado uma única vez, com erro padrão discutido.
- O atributo grupo não foi usado nos modelos; a validação do ensemble ficou separada do teste.
- A busca (Optuna/TPE) e o ensemble foram comparados com linhas de base simples e com o melhor modelo único.
- A auditoria por grupo inclui seleção, TPR real, proxies (AUC) e mitigação, discutindo trade-offs.
- A ficha do modelo, a latência frente ao orçamento e a simulação de deriva foram apresentadas.
- O relatório é honesto: cita limites, custo da busca, a natureza simulada dos dados e salvaguardas.

```python
import warnings; warnings.filterwarnings("ignore")
import hashlib, json, os, sys, tempfile, time
import joblib, numpy as np, optuna, sklearn
import lightgbm as lgb
from sklearn.ensemble import RandomForestClassifier
from sklearn.inspection import permutation_importance
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import roc_auc_score
from sklearn.model_selection import cross_val_score, train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
optuna.logging.set_verbosity(optuna.logging.WARNING)

def gera(n=6000, vies=0.6, p_min=0.3, seed=0, infl=0.0):
    r = np.random.default_rng(seed)
    g = (r.random(n) < p_min).astype(int); z = r.normal(0, 1, n)
    renda = np.exp(0.5 * z + 0.35 * r.normal(size=n) - 0.25 * g) * 3.0 * (1 + infl)
    emprego = np.clip(5 + 2 * z + r.normal(0, 2, n), 0, None)
    divida = np.clip(0.4 - 0.1 * z + r.normal(0, 0.1, n), 0, 1)
    cep = 0.8 * g + 0.6 * r.normal(size=n); score = z + r.normal(0, 1, n)
    y_real = (z > 0).astype(int); y_hist = ((z - vies * g + r.normal(0, 0.5, n)) > 0).astype(int)
    return np.c_[renda, emprego, divida, cep, score], g, y_hist, y_real
def psi(ref, novo, k=10):
    cortes = np.quantile(ref, np.linspace(0, 1, k + 1)[1:-1])
    a = np.bincount(np.searchsorted(cortes, ref), minlength=k) / len(ref)
    b = np.bincount(np.searchsorted(cortes, novo), minlength=k) / len(novo)
    a, b = np.clip(a, 1e-4, None), np.clip(b, 1e-4, None)
    return float(np.sum((b - a) * np.log(b / a)))

NOMES = ["renda", "tempo_emprego", "razao_divida", "score_historico"]
X, g, yh, yr = gera(6000, vies=0.5, p_min=0.25, seed=11); X = X[:, [0, 1, 2, 4]]            # sem cep e
sem grupo
Xa, Xte, ya, yte, ga, gte, yra, yrte = train_test_split(X, yh, g, yr, test_size=0.25, random_state=0,
stratify=yh)
Xtr, Xv, ytr, yv = train_test_split(Xa, ya, test_size=0.25, random_state=0, stratify=ya)

def constroi(p):
    if p["alg"] == "lr": return make_pipeline(StandardScaler(), LogisticRegression(C=p["C"],
max_iter=2000))
```

```
    if p["alg"] == "rf": return RandomForestClassifier(p["n"], max_depth=p["prof"], random_state=0,
n_jobs=-1)
    return lgb.LGBMClassifier(n_estimators=p["n"], learning_rate=p["lr"], num_leaves=p["folhas"],
random_state=0, verbose=-1)

def objetivo(t):
    alg = t.suggest_categorical("alg", ["lr", "rf", "lgbm"])
    if alg == "lr": p = {"alg": alg, "C": t.suggest_float("C", 1e-3, 1e2, log=True)}
    elif alg == "rf": p = {"alg": alg, "n": t.suggest_int("n_rf", 50, 200), "prof":
t.suggest_int("prof", 3, 14)}
    else: p = {"alg": alg, "n": t.suggest_int("n_lgb", 50, 300), "lr": t.suggest_float("lr", 0.01, 0.3,
log=True), "folhas": t.suggest_int("folhas", 4, 32)}
    return cross_val_score(constroi(p), Xtr, ytr, cv=3).mean()

def params(t):
    d = t.params; a = d["alg"]
    return {"alg": a, **({"C": d["C"]} if a == "lr" else {"n": d["n_rf"], "prof": d["prof"]} if a ==
"rf" else {"n": d["n_lgb"], "lr": d["lr"], "folhas": d["folhas"]})}

estudo = optuna.create_study(direction="maximize", sampler=optuna.samplers.TPESampler(seed=0));
estudo.optimize(objetivo, n_trials=30)
ordem = sorted([t for t in estudo.trials if t.value is not None], key=lambda t: -t.value)
escolhidos, vistos = [], set()
for t in ordem:                                          # o melhor de cada algoritmo, para ter
diversidade
    if t.params["alg"] not in vistos: vistos.add(t.params["alg"]); escolhidos.append(t)
modelos = [constroi(params(t)).fit(Xtr, ytr) for t in escolhidos]
print("Melhor da busca:", params(ordem[0]), "| CV = %.3f" % ordem[0].value)
print("Algoritmos no ensemble:", [params(t)["alg"] for t in escolhidos])
P = lambda ms, A: np.mean([m.predict_proba(A)[:, 1] for m in ms], axis=0)
print("Validação: melhor único = %.3f | ensemble = %.3f" % (modelos[0].score(Xv, yv), ((P(modelos, Xv)
>= 0.5) == yv).mean()))

base_lr = make_pipeline(StandardScaler(), LogisticRegression(max_iter=2000)).fit(Xtr, ytr); base_rf =
RandomForestClassifier(100, random_state=0, n_jobs=-1).fit(Xtr, ytr)
pred_ens = (P(modelos, Xte) >= 0.5).astype(int)
cands = {"reg. logística (base)": base_lr.predict(Xte), "floresta padrão (base)": base_rf.predict(Xte),
"melhor único": modelos[0].predict(Xte), "ensemble por algoritmo": pred_ens}
print("\nTESTE FINAL (n = %d, erro padrão ~ %.3f):" % (len(yte), np.sqrt(0.8 * 0.2 / len(yte))))
for nome, p in cands.items(): print(f"  {nome:<24s} acurácia = {(p == yte).mean():.3f}")

def rel(nome, pred):
    s = [pred[gte == k].mean() for k in (0, 1)]; t = [pred[(gte == k) & (yrte == 1)].mean() for k in
(0, 1)]
    print(f"  {nome:<22s} acc hist = {(pred == yte).mean():.3f} | acc verdade = {(pred ==
yrte).mean():.3f} | seleção {s[0]:.2f}/{s[1]:.2f} | TPR real {t[0]:.2f}/{t[1]:.2f}")
print("\nAUDITORIA POR GRUPO:"); rel("ensemble", pred_ens)
pi = permutation_importance(modelos[0], Xte, yte, n_repeats=10, random_state=0)
print("  Permutação do melhor único:", {n: round(float(v), 3) for n, v in zip(NOMES,
pi.importances_mean)})
Xg_tr, Xg_te, g_tr, g_te = train_test_split(X, g, test_size=0.3, random_state=0, stratify=g)
pg = make_pipeline(StandardScaler(), LogisticRegression(max_iter=2000)).fit(Xg_tr, g_tr)
print("  AUC para prever o grupo pelos atributos: %.3f" % roc_auc_score(g_te, pg.predict_proba(Xg_te)
[:, 1]))
p = P(modelos, Xte); sel0 = (p[gte == 0] >= 0.5).mean(); t1 = np.quantile(p[gte == 1], 1 - sel0)
rel("ensemble + limiares", np.where(gte == 0, p >= 0.5, p >= t1).astype(int))

pasta = tempfile.mkdtemp(); joblib.dump(modelos[0], os.path.join(pasta, "modelo.joblib"))
```

```
ficha = {"python": sys.version.split()[0], "scikit-learn": sklearn.__version__, "semente": 0,
"hash_treino": hashlib.sha256(Xtr.tobytes()).hexdigest()[:16]}
um = Xte[:1]; modelos[0].predict(um); t0 = time.perf_counter()
for _ in range(50): modelos[0].predict(um)
lat = (time.perf_counter() - t0) / 50 * 1000; um_ens = time.perf_counter()
for _ in range(50): P(modelos, um)
lat_e = (time.perf_counter() - um_ens) / 50 * 1000
print("\nPRODUÇÃO: ficha =", json.dumps(ficha)); print(f"  Latência por previsão: melhor único =
{lat:.1f} ms | ensemble = {lat_e:.1f} ms (orçamento: 20 ms)")
Xd, g_d, yh_d, yr_d = gera(3000, vies=0.5, p_min=0.25, seed=99, infl=0.4); Xd = Xd[:, [0, 1, 2, 4]]
print(f"  Deriva (inflação de 40%): PSI da renda = {psi(Xtr[:, 0], Xd[:, 0]):.3f} | acurácia contra a
verdade: base = {(modelos[0].predict(Xte) == yrte).mean():.3f} -> {(modelos[0].predict(Xd) ==
yr_d).mean():.3f}")
```

SAÍDA

```
Melhor da busca: {'alg': 'lr', 'C': 0.06514548596465879} | CV = 0.800
Algoritmos no ensemble: ['lr', 'rf', 'lgbm']
Validação: melhor único = 0.804 | ensemble = 0.798

TESTE FINAL (n = 1500, erro padrão ~ 0.010):
  reg. logística (base)    acurácia = 0.808
  floresta padrão (base)   acurácia = 0.785
  melhor único             acurácia = 0.806
  ensemble por algoritmo   acurácia = 0.804

AUDITORIA POR GRUPO:
  ensemble               acc hist = 0.804 | acc verdade = 0.851 | seleção 0.47/0.38 | TPR real
0.83/0.72
  Permutação do melhor único: {'renda': 0.088, 'tempo_emprego': 0.023, 'razao_divida': 0.03,
'score_historico': 0.028}
  AUC para prever o grupo pelos atributos: 0.650
  ensemble + limiares    acc hist = 0.797 | acc verdade = 0.855 | seleção 0.47/0.47 | TPR real
0.83/0.82

PRODUÇÃO: ficha = {"python": "3.13.16", "scikit-learn": "1.9.1", "semente": 0, "hash_treino":
"af40eb641ad0027c"}
  Latência por previsão: melhor único = 0.5 ms | ensemble = 50.5 ms (orçamento: 20 ms)
  Deriva (inflação de 40%): PSI da renda = 0.304 | acurácia contra a verdade: base = 0.853 -> 0.864
```

EXEMPLO Discussão dos resultados

A busca (TPE, 30 tentativas) escolheu a **regressão logística** (C ≈ 0,065, CV 0,800): o problema simulado é praticamente linear, e o modelo simples bastou. No teste final (1.500 casos, erro padrão cerca de 0,010), regressão logística (0,808), melhor único (0,806), ensemble por algoritmo (0,804) e floresta padrão (0,785) ficam todos próximos: **as diferenças entre os três primeiros estão dentro do ruído**, e só a floresta padrão ficou um pouco abaixo. O ensemble não se justifica aqui, sobretudo porque ele custou **50,5 ms por previsão, acima do orçamento de 20 ms**, contra 0,5 ms do melhor único (o valor depende da máquina, mas a ordem de grandeza é o ponto). Na auditoria, o ensemble seleciona 0,47 do grupo 0 e 0,38 do grupo 1 e tem TPR real de 0,83 contra 0,72; os atributos permitem prever o grupo com AUC de 0,650 (sem o CEP-score o proxy é mais fraco, mas a renda ainda carrega informação). Com limiares por grupo, a seleção se iguala (0,47) e o TPR real do grupo 1 sobe para 0,82, com a acurácia contra a verdade se mantendo (0,855 contra 0,851) e leve queda contra o histórico (0,797). Na deriva de 40% de inflação, o PSI da renda foi 0,304 (acima de 0,25, "mudança grande"), mas a acurácia contra a verdade **não caiu** (0,853 para 0,864): alarme não é diagnóstico, e a deriva é um sinal para investigar. Recomendação: implantar o **melhor único (ou a regressão logística)**, que é simples, rápido e interpretável, com monitoramento por grupo, limiares por grupo sujeitos a revisão jurídica, canal de contestação e revisão humana. Limites: dados simulados, uma única semente e um problema quase linear.

Referências

Lista de artigos de periódicos e conferências de referência, além de livros reconhecidos, citados ao longo do texto.

AKIBA, T.; SANO, S.; YANASE, T.; OHTA, T.; KOYAMA, M. Optuna: a next-generation hyperparameter optimization framework. In: ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD), 2019.

BERGMEIR, C.; BENÍTEZ, J. M. On the use of cross-validation for time series predictor evaluation. Information Sciences, 2012.

BERGSTRA, J.; BARDENET, R.; BENGIO, Y.; KÉGL, B. Algorithms for hyper-parameter optimization. In: Advances in Neural Information Processing Systems (NIPS), v. 24, 2011. p. 2546-2554.

BERGSTRA, J.; BENGIO, Y. Random search for hyper-parameter optimization. Journal of Machine Learning Research, v. 13, p. 281-305, 2012.

BISCHL, B. et al. Hyperparameter optimization: foundations, algorithms, best practices, and open challenges. WIREs Data Mining and Knowledge Discovery, v. 13, n. 2, 2023.

BRAZDIL, P.; VAN RIJN, J. N.; SOARES, C.; VANSCHOREN, J. Metalearning: applications to automated machine learning and data mining. 2. ed. Cham: Springer, 2022.

BREIMAN, L. Random forests. Machine Learning, v. 45, n. 1, p. 5-32, 2001.

CARUANA, R.; NICULESCU-MIZIL, A.; CREW, G.; KSIKES, A. Ensemble selection from libraries of models. In: International Conference on Machine Learning (ICML), 2004.

CAWLEY, G. C.; TALBOT, N. L. C. On over-fitting in model selection and subsequent selection bias in performance evaluation. Journal of Machine Learning Research, v. 11, p. 2079-2107, 2010.

CHEN, T.; GUESTRIN, C. XGBoost: a scalable tree boosting system. In: ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD), 2016. p. 785-794.

CUBUK, E. D.; ZOPH, B.; MANE, D.; VASUDEVAN, V.; LE, Q. V. AutoAugment: learning augmentation strategies from data. In: IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2019. p. 113-123.

DEB, K.; PRATAP, A.; AGARWAL, S.; MEYARIVAN, T. A fast and elitist multiobjective genetic algorithm: NSGA-II. IEEE Transactions on Evolutionary Computation, v. 6, n. 2, p. 182-197, 2002.

DEMŠAR, J. Statistical comparisons of classifiers over multiple data sets. Journal of Machine Learning Research, v. 7, p. 1-30, 2006.

DEVLIN, J.; CHANG, M.-W.; LEE, K.; TOUTANOVA, K. BERT: pre-training of deep bidirectional transformers for language understanding. In: Conference of the North American Chapter of the Association for Computational Linguistics (NAACL-HLT), 2019. p. 4171-4186.

DUA, D.; GRAFF, C. UCI Machine Learning Repository. Irvine: University of California, School of Information and Computer Science, 2019.

ELSKEN, T.; METZEN, J. H.; HUTTER, F. Neural architecture search: a survey. Journal of Machine Learning Research, v. 20, n. 55, p. 1-21, 2019.

ERICKSON, N. et al. AutoGluon-Tabular: robust and accurate AutoML for structured data. In: ICML Workshop on Automated Machine Learning (AutoML), 2020.

FALKNER, S.; KLEIN, A.; HUTTER, F. BOHB: robust and efficient hyperparameter optimization at scale. In: International Conference on Machine Learning (ICML), 2018. PMLR, v. 80, p. 1437-1446.

FEURER, M.; EGGENSPERGER, K.; FALKNER, S.; LINDAUER, M.; HUTTER, F. Auto-Sklearn 2.0: hands-free AutoML via meta-learning. Journal of Machine Learning Research, v. 23, n. 261, p. 1-61, 2022.

FEURER, M.; KLEIN, A.; EGGENSPERGER, K.; SPRINGENBERG, J. T.; BLUM, M.; HUTTER, F. Efficient and robust automated machine learning. In: Advances in Neural Information Processing Systems (NIPS), v. 28, 2015. p. 2962-2970.

FEURER, M.; SPRINGENBERG, J. T.; HUTTER, F. Initializing Bayesian hyperparameter optimization via meta-learning. In: AAAI Conference on Artificial Intelligence, 2015. p. 1128-1135.

FISHER, R. A. The use of multiple measurements in taxonomic problems. Annals of Eugenics, v. 7, n. 2, p. 179-188, 1936.

FRIEDMAN, J. H. Greedy function approximation: a gradient boosting machine. The Annals of Statistics, v. 29, n. 5, p. 1189-1232, 2001.

GAMA, J.; ŽLIOBAITĖ, I.; BIFET, A.; PECHENIZKIY, M.; BOUCHACHIA, A. A survey on concept drift adaptation. ACM Computing Surveys, v. 46, n. 4, art. 44, 2014.

GIJSBERS, P. et al. AMLB: an AutoML benchmark. Journal of Machine Learning Research, v. 25, n. 101, p. 1-65, 2024.

GOODFELLOW, I.; BENGIO, Y.; COURVILLE, A. Deep learning. Cambridge: MIT Press, 2016.

GRINSZTAJN, L.; OYALLON, E.; VAROQUAUX, G. Why do tree-based models still outperform deep learning on typical tabular data? In: Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track, 2022.

GUYON, I.; ELISSEEFF, A. An introduction to variable and feature selection. Journal of Machine Learning Research, v. 3, p. 1157-1182, 2003.

HARDT, M.; PRICE, E.; SREBRO, N. Equality of opportunity in supervised learning. In: Advances in Neural Information Processing Systems (NIPS), v. 29, 2016. p. 3315-3323.

HASTIE, T.; TIBSHIRANI, R.; FRIEDMAN, J. The elements of statistical learning: data mining, inference, and prediction. 2. ed. New York: Springer, 2009.

HE, K.; ZHANG, X.; REN, S.; SUN, J. Deep residual learning for image recognition. In: IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016. p. 770-778.

HE, X.; ZHAO, K.; CHU, X. AutoML: a survey of the state-of-the-art. Knowledge-Based Systems, v. 212, art. 106622, 2021.

HOLLMANN, N. et al. Accurate predictions on small data with a tabular foundation model. Nature, v. 637, p. 319-326, 2025.

HUTTER, F.; HOOS, H. H.; LEYTON-BROWN, K. Sequential model-based optimization for general algorithm configuration. In: Learning and Intelligent Optimization (LION 5), 2011. Lecture Notes in Computer Science, v. 6683.

HUTTER, F.; KOTTHOFF, L.; VANSCHOREN, J. (ed.). Automated machine learning: methods, systems, challenges. Cham: Springer, 2019.

HYNDMAN, R. J.; ATHANASOPOULOS, G. Forecasting: principles and practice. 3. ed. Melbourne: OTexts, 2021.

HYNDMAN, R. J.; KHANDAKAR, Y. Automatic time series forecasting: the forecast package for R. Journal of Statistical Software, v. 27, n. 3, p. 1-22, 2008.

JAMIESON, K.; TALWALKAR, A. Non-stochastic best arm identification and hyperparameter optimization. In: International Conference on Artificial Intelligence and Statistics (AISTATS), 2016. PMLR, v. 51, p. 240-248.

JIN, H.; SONG, Q.; HU, X. Auto-Keras: an efficient neural architecture search system. In: ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD), 2019. p. 1946-1956.

JONES, D. R.; SCHONLAU, M.; WELCH, W. J. Efficient global optimization of expensive black-box functions. Journal of Global Optimization, v. 13, p. 455-492, 1998.

KANTER, J. M.; VEERAMACHANENI, K. Deep feature synthesis: towards automating data science endeavors. In: IEEE International Conference on Data Science and Advanced Analytics (DSAA), 2015. p. 1-10.

KAPOOR, S.; NARAYANAN, A. Leakage and the reproducibility crisis in machine-learning-based science. Patterns, v. 4, n. 9, art. 100804, 2023.

KE, G. et al. LightGBM: a highly efficient gradient boosting decision tree. In: Advances in Neural Information Processing Systems (NIPS), v. 30, 2017.

KHURANA, U.; SAMULOWITZ, H.; TURAGA, D. Feature engineering for predictive modeling using reinforcement learning. In: AAAI Conference on Artificial Intelligence, v. 32, 2018. p. 3407-3414.

KOHAVI, R. A study of cross-validation and bootstrap for accuracy estimation and model selection. In: International Joint Conference on Artificial Intelligence (IJCAI), 1995. p. 1137-1145.

KREUZBERGER, D.; KÜHL, N.; HIRSCHL, S. Machine learning operations (MLOps): overview, definition, and architecture. IEEE Access, v. 11, p. 31866-31879, 2023.

LEDELL, E.; POIRIER, S. H2O AutoML: scalable automatic machine learning. In: ICML Workshop on Automated Machine Learning (AutoML), 2020.

LI, L.; JAMIESON, K.; DESALVO, G.; ROSTAMIZADEH, A.; TALWALKAR, A. Hyperband: a novel bandit-based approach to hyperparameter optimization. Journal of Machine Learning Research, v. 18, n. 185, p. 1-52, 2018.

LINDAUER, M. et al. SMAC3: a versatile Bayesian optimization package for hyperparameter optimization. Journal of Machine Learning Research, v. 23, n. 54, p. 1-9, 2022.

LIU, H.; SIMONYAN, K.; YANG, Y. DARTS: differentiable architecture search. In: International Conference on Learning Representations (ICLR), 2019.

LORENA, A. C.; GARCIA, L. P. F.; LEHMANN, J.; SOUTO, M. C. P.; HO, T. K. How complex is your classification problem? A survey on measuring classification complexity. ACM Computing Surveys, v. 52, n. 5, art. 107.

LUNDBERG, S. M.; LEE, S.-I. A unified approach to interpreting model predictions. In: Advances in Neural Information Processing Systems (NIPS), v. 30, 2017.

MAKRIDAKIS, S.; SPILIOTIS, E.; ASSIMAKOPOULOS, V. The M4 Competition: 100,000 time series and 61 forecasting methods. International Journal of Forecasting, v. 36, n. 1, p. 54-74, 2020.

MEHRABI, N.; MORSTATTER, F.; SAXENA, N.; LERMAN, K.; GALSTYAN, A. A survey on bias and fairness in machine learning. ACM Computing Surveys, v. 54, n. 6, art. 115, 2021.

OLSON, R. S.; BARTLEY, N.; URBANOWICZ, R. J.; MOORE, J. H. Evaluation of a tree-based pipeline optimization tool for automating data science. In: Genetic and Evolutionary Computation Conference (GECCO), 2016. p. 485-492.

OLSON, R. S.; MOORE, J. H. TPOT: a tree-based pipeline optimization tool for automating machine learning. In: ICML Workshop on Automatic Machine Learning, 2016. PMLR, v. 64, p. 66-74.

PEDREGOSA, F. et al. Scikit-learn: machine learning in Python. Journal of Machine Learning Research, v. 12, p. 2825-2830, 2011.

PHAM, H.; GUAN, M. Y.; ZOPH, B.; LE, Q. V.; DEAN, J. Efficient neural architecture search via parameters sharing. In: International Conference on Machine Learning (ICML), 2018. PMLR, v. 80, p. 4095-4104.

PROBST, P.; BOULESTEIX, A.-L.; BISCHL, B. Tunability: importance of hyperparameters of machine learning algorithms. Journal of Machine Learning Research, v. 20, n. 53, p. 1-32, 2019.

RASMUSSEN, C. E.; WILLIAMS, C. K. I. Gaussian processes for machine learning. Cambridge: MIT Press, 2006.

REAL, E.; AGGARWAL, A.; HUANG, Y.; LE, Q. V. Regularized evolution for image classifier architecture search. In: AAAI Conference on Artificial Intelligence, v. 33, 2019. p. 4780-4789.

REN, P. et al. A comprehensive survey of neural architecture search: challenges and solutions. ACM Computing Surveys, v. 54, n. 4, art. 76, 2021.

RIBEIRO, M. T.; SINGH, S.; GUESTRIN, C. "Why should I trust you?": explaining the predictions of any classifier. In: ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD), 2016. p. 1135-1144.

RICE, J. R. The algorithm selection problem. Advances in Computers, v. 15, p. 65-118, 1976.

RIVOLLI, A.; GARCIA, L. P. F.; SOARES, C.; VANSCHOREN, J.; CARVALHO, A. C. P. L. F. Meta-features for meta-learning. Knowledge-Based Systems, v. 240, art. 108101, 2022.

RUDIN, C. Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead. Nature Machine Intelligence, v. 1, p. 206-215, 2019.

SCHWARTZ, R.; DODGE, J.; SMITH, N. A.; ETZIONI, O. Green AI. Communications of the ACM, v. 63, n. 12, p. 54-63, 2020.

SCULLEY, D. et al. Hidden technical debt in machine learning systems. In: Advances in Neural Information Processing Systems (NIPS), v. 28, 2015. p. 2503-2511.

SHAHRIARI, B.; SWERSKY, K.; WANG, Z.; ADAMS, R. P.; DE FREITAS, N. Taking the human out of the loop: a review of Bayesian optimization. Proceedings of the IEEE, v. 104, n. 1, p. 148-175, 2016.

SNOEK, J.; LAROCHELLE, H.; ADAMS, R. P. Practical Bayesian optimization of machine learning algorithms. In: Advances in Neural Information Processing Systems (NIPS), v. 25, 2012. p. 2951-2959.

STRUBELL, E.; GANESH, A.; MCCALLUM, A. Energy and policy considerations for deep learning in NLP. In: Annual Meeting of the Association for Computational Linguistics (ACL), 2019. p. 3645-3650.

TAN, M.; LE, Q. V. EfficientNet: rethinking model scaling for convolutional neural networks. In: International Conference on Machine Learning (ICML), 2019. PMLR, v. 97, p. 6105-6114.

THORNTON, C.; HUTTER, F.; HOOS, H. H.; LEYTON-BROWN, K. Auto-WEKA: combined selection and hyperparameter optimization of classification algorithms. 2013.

VANSCHOREN, J.; VAN RIJN, J. N.; BISCHL, B.; TORGO, L. OpenML: networked science in machine learning. SIGKDD Explorations, v. 15, n. 2, p. 49-60.

VARMA, S.; SIMON, R. Bias in error estimation when using cross-validation for model selection. BMC Bioinformatics, v. 7, art. 91, 2006.

VASWANI, A. et al. Attention is all you need. In: Advances in Neural Information Processing Systems (NIPS), v. 30, 2017. p. 5998-6008.

WANG, C.; WU, Q.; WEIMER, M.; ZHU, E. FLAML: a fast and lightweight AutoML library. In: Conference on Machine Learning and Systems (MLSys), 2021.

WOLPERT, D. H. Stacked generalization. Neural Networks, v. 5, n. 2, p. 241-259, 1992.

WOLPERT, D. H. The lack of a priori distinctions between learning algorithms. Neural Computation, v. 8, n. 7, p. 1341-1390, 1996.

ZÖLLER, M.-A.; HUBER, M. F. Benchmark and survey of automated machine learning frameworks. Journal of Artificial Intelligence Research, v. 70, p. 409-472, 2021.

ZOPH, B.; LE, Q. V. Neural architecture search with reinforcement learning. In: International Conference on Learning Representations (ICLR), 2017.

ZOPH, B.; VASUDEVAN, V.; SHLENS, J.; LE, Q. V. Learning transferable architectures for scalable image recognition. In: IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2018.